互连网络的原理与实践

Principles and Practices of Interconnection Networks

William James Dally　Brian Towles

原著 Morgan Kaufmann ／ Elsevier，2004

目录

名家推荐 **xiii**

版权与出版信息 **xv**

致谢 **xvi**

前言 **xvii**

本书的组织结构 xvii

互连网络教学 xx

作者简介 **xxi**

第一章 互连网络导论 **1**

1.1 关于互连网络的三个问题 1

1.2 互连网络的用途 3

1.2.1 处理器-存储器互连 4

1.2.2 I/O 互连 7

1.2.3 分组交换结构 9

1.3 网络基础 11

1.3.1 拓扑结构 11

1.3.2 路由 14

1.3.3 流控 15

1.3.4 路由器架构 16

1.3.5 互连网络的性能 16

1.4 历史 18

1.5 本书的组织结构 19

第二章 一个简单的互连网络 **20**

2.1 网络规格与约束 20

2.2 拓扑 20

2.3 路由 23

2.4 流控 26
2.5 路由器设计 26
2.6 性能分析 31
2.7 练习题 35

第三章 拓扑基础 **37**
3.1 术语 38
3.1.1 通道与节点 38
3.1.2 直接网络与间接网络 39
3.1.3 割与二分 39
3.1.4 路径 40
3.1.5 对称性 41
3.2 流量模式 41
3.3 性能 42
3.3.1 吞吐量与最大通道负载 42
3.3.2 延迟 45
3.3.3 路径多样性 47
3.4 封装成本 50
3.5 案例研究：SGI Origin 2000 54
3.6 文献注释 60
3.7 练习 60

第四章 蝶形网络 **65**
4.1 蝶形网络的结构 65
4.2 同构蝶形网络 66
4.3 性能与封装成本 67
4.4 路径多样性与额外级数 70
4.5 案例研究：BBN Butterfly 73
4.6 文献注释 73
4.7 练习 75

第五章 环面网络 **77**
5.1 环面网络的结构 77
5.2 性能 81

5.2.1 吞吐量 81
5.2.2 延迟 84
5.2.3 路径多样性 85
5.3 构建网格和环面网络 86
5.4 快速立方体 89
5.5 案例研究：MIT J-Machine 91
5.6 文献注释 94
5.7 练习 94

第六章 非阻塞网络 99

6.1 无阻塞网络与无干扰网络 99
6.2 交叉开关网络 100
6.3 Clos 网络 104
6.3.1 Clos 网络的结构与特性 104
6.3.2 严格非阻塞 Clos 网络上的单播路由 106
6.3.3 可重排 Clos 网络上的单播路由 108
6.3.4 使用矩阵分解路由 Clos 网络 113
6.3.5 Clos 网络上的多播路由 114
6.3.6 多于三级的 Clos 网络 118
6.4 Beneš 网络 119
6.5 排序网络 120
6.6 案例研究：Velio VC2002（Zeus）梳理交换机 122
6.7 文献注释 127
6.8 练习题 127

第七章 集中与切片 130

7.1 集中器与分配器 130
7.1.1 集中器 130
7.1.2 分配器 132
7.2 切片与分割 133
7.2.1 位切片 133
7.2.2 维度切片 134
7.2.3 通道切片 135
7.3 多级网络的切片 136

7.4 案例研究：Tiny Tera 中的位切片 140
7.5 文献注释 141
7.6 练习 141

第八章 路由基础 **143**
8.1 一个路由示例 144
8.2 路由算法的分类 145
8.3 路由关系 146
8.4 确定性路由 147
8.4.1 蝶形网络中的目的地标签路由 148
8.4.2 立方网络中的维序路由 150
8.5 案例研究：Cray T3D 中的维序路由 152
8.6 文献注释 154
8.7 练习 155

第九章 无关路由 **157**
9.1 Valiant 的随机路由算法 157
9.1.1 环面拓扑上的 Valiant 算法 157
9.1.2 间接网络上的 Valiant 算法 159
9.2 最小 oblivious 路由 159
9.2.1 折叠 Clos（胖树）上的最小 oblivious 路由 159
9.2.2 环面上的最小 oblivious 路由 161
9.3 负载均衡的无关路由 162
9.4 无关路由的分析 163
9.5 案例研究：Avici 太比特交换路由器（TSR）中的无关路由 166
9.6 文献注释 168
9.7 练习 168

第十章 自适应路由 **171**
10.1 自适应路由基础 171
10.2 最小自适应路由 173
10.3 全自适应路由 175
10.4 负载均衡自适应路由 175
10.5 基于搜索的路由 177

10.6 案例研究：Thinking Machines CM-5 中的自适应路由 177
10.7 文献注释 182
10.8 练习 182

第十一章 路由机制 **184**
11.1 基于表的路由 184
11.1.1 源路由 184
11.1.2 节点表路由 188
11.2 算法路由 190
11.3 案例研究：IBM Vulcan 网络中的 Oblivious 源路由 192
11.4 文献注释 197
11.5 练习 197

第十二章 流控制基础 **200**
12.1 资源与分配单元 200
12.2 无缓冲流量控制 203
12.3 电路交换 205
12.4 文献注释 207
12.5 练习题 207

第十三章 缓冲式流控制 **209**
13.1 包缓冲区流量控制 210
13.2 Flit 缓冲流控 212
13.2.1 虫洞流控 212
13.2.2 虚拟信道流控 217
13.3 缓冲区管理与背压 225
13.3.1 基于信用的流控 225
13.3.2 开/关流量控制 227
13.3.3 确认/否认流量控制 230
13.4 微片预留流控 230
13.4.1 微片预留路由器 233
13.4.2 输出调度 233
13.4.3 输入调度 235
13.5 文献注释 236

13.6 练习题 236

第十四章 死锁与活锁 238

14.1 死锁 239
14.1.1 代理与资源 239
14.1.2 等待与持有关系 239
14.1.3 资源依赖 241
14.1.4 一些示例 241
14.1.5 高层（协议）死锁 243
14.2 死锁避免 244
14.2.1 资源类 244
14.2.2 受限物理路由 248
14.2.3 混合死锁避免 251
14.3 自适应路由 253
14.3.1 路由子函数与扩展依赖 254
14.3.2 Duato 的无死锁自适应算法协议 258
14.4 死锁恢复 258
14.4.1 回退恢复 258
14.4.2 渐进恢复 259
14.5 活锁 260
14.6 案例研究：Cray T3E 中的死锁避免 260
14.7 文献注释 261
14.8 练习 262

第十五章 服务质量 266

15.1 服务类别与服务合约 266
15.2 突发性与网络延迟 267
15.2.1 （σ，ρ）调节流 267
15.2.2 计算延迟 268
15.3 保证服务的实现 271
15.3.1 聚合资源分配 271
15.3.2 资源预留 272
15.4 尽力而为服务的实现 273
15.4.1 延迟公平性 275

15.4.2 吞吐量公平性 276
15.5 资源分离 278
15.5.1 树饱和 278
15.5.2 无干扰网络 278
15.6 案例研究：ATM 服务类别 279
15.7 案例研究：Avici TSR 中的虚拟网络 280
15.8 文献注释 282
15.9 练习题 282

第十六章 路由器体系结构 284
16.1 基本路由器架构 284
16.1.1 框图 284
16.1.2 路由器流水线 286
16.2 停转 288
16.3 用信用关闭环路 291
16.4 重新分配通道 291
16.5 推测与前瞻 294
16.6 Flit 与信用编码 295
16.7 案例研究：Alpha 21364 路由器 298
16.8 文献注释 301
16.9 练习 301

第十七章 路由器数据通路组件 303
17.1 输入缓冲区组织 303
17.1.1 缓冲区划分 303
17.1.2 输入缓冲数据结构 306
17.1.3 输入缓冲区分配 313
17.2 交换机 314
17.2.1 总线交换机 314
17.2.2 交叉开关 316
17.2.3 网络交换机 321
17.3 输出组织 321
17.4 案例研究：IBM Colony 路由器的数据通路 322
17.5 文献注释 324

17.6 练习 324

第十八章 仲裁 328

18.1 仲裁时序 328
18.2 公平性 330
18.3 固定优先级仲裁器 330
18.4 可变优先级迭代仲裁器 332
18.4.1 oblivious 仲裁器 332
18.4.2 轮询仲裁器 334
18.4.3 授权保持电路 334
18.4.4 加权轮询仲裁器 337
18.5 矩阵仲裁器 337
18.6 排队仲裁器 339
18.7 练习 341

第十九章 分配 342

19.1 表示方法 342
19.2 精确算法 345
19.3 可分离分配器 346
19.3.1 并行迭代匹配 351
19.3.2 iSLIP 351
19.3.3 孤独输出分配器 352
19.4 波前分配器 353
19.5 增量分配与批量分配 355
19.6 多级分配 357
19.7 分配器的性能 359
19.8 案例研究：Tiny Tera 分配器 363
19.9 文献注释 364
19.10 习题 364

第二十章 网络接口 367

20.1 处理器-网络接口 367
20.1.1 双寄存器接口 368
20.1.2 寄存器映射接口 370

20.1.3 基于描述符的接口 370
20.1.4 消息接收 370
20.2 共享内存接口 371
20.2.1 处理器-网络接口 371
20.2.2 缓存一致性 373
20.2.3 内存-网络接口 374
20.3 线缆-交换结构接口 376
20.4 案例研究：MIT M-Machine 网络接口 378
20.5 文献注释 381
20.6 习题 382

第二十一章 错误控制 384
21.1 认识故障：失效模式与故障模型 384
21.2 错误控制过程：检测、隔离与恢复 386
21.3 链路级错误控制 386
21.3.1 链路监测 386
21.3.2 链路级重传 388
21.3.3 通道重配置、降级与关闭 389
21.4 路由器错误控制 391
21.5 网络级错误控制 392
21.6 端到端错误控制 393
21.7 文献注释 393
21.8 习题 393

第二十二章 总线 396
22.1 总线基础 396
22.2 总线仲裁 399
22.3 高性能总线协议 403
22.3.1 总线流水线化 403
22.3.2 拆分事务总线 404
22.3.3 突发消息 407
22.4 从总线到网络 408
22.5 案例研究：PCI 总线 410
22.6 文献注释 411

22.7 习题 412

第二十三章 性能分析 414

23.1 互连网络性能指标 415
23.1.1 吞吐量 415
23.1.2 延迟 416
23.1.3 容错性 417
23.1.4 常见测量陷阱 417
23.2 分析 420
23.2.1 排队论 421
23.2.2 概率分析 427
23.3 验证 429
23.4 案例研究：BBN Monarch 网络的效率与损耗 430
23.5 文献注释 433
23.6 习题 433

第二十四章 仿真 435

24.1 仿真详细程度 435
24.2 网络工作负载 436
24.2.1 应用驱动的工作负载 436
24.2.2 合成工作负载 437
24.3 仿真测量 439
24.3.1 模拟器预热 440
24.3.2 稳态抽样 440
24.3.3 置信区间 442
24.4 模拟器设计 443
24.4.1 仿真方法 443
24.4.2 源端队列建模 445
24.4.3 随机数生成 446
24.4.4 故障排查 449
24.5 文献注释 449
24.6 习题 449

第二十五章 仿真实例 455

25.1 路由 455
25.1.1 延迟 455
25.1.2 吞吐量分布 459
25.2 流控性能 459
25.2.1 虚拟通道 459
25.2.2 网络规模 461
25.2.3 注入过程 462
25.2.4 优先级划分 463
25.2.5 稳定性 465
25.3 容错性 466

附录 A 符号约定 **468**

附录 B 术语释义 **470**

附录 C 网络模拟器 **473**

参考文献 **474**

原书索引 **499**

名家推荐

“本书的学术水准在这一领域无出其右。对于互连网络而言，本书就像 Hennessy 和 Patterson 的著作之于计算机体系结构：它是一部权威的一站式参考资料，以清晰而系统的方法阐释最重要的概念。对材料的论述无论广度还是深度都十分出色……这是一部必读的杰作！”

——Timothy Mark Pinkston，南加州大学

“这是关于现代互连网络最全面、最连贯的著作。作为该领域的领军人物，Dally 和 Towles 充分发挥了他们作为研究人员和工程师的丰富经验，既阐述这类网络背后的理论，也介绍如何实际构建网络。对于任何学习、分析或设计互连网络的人而言，本书都是必不可少的。”

——Stephen W. Keckler，得克萨斯大学奥斯汀分校

“本书既可作为出色的教材和宝贵的研究参考书，也能为系统设计者提供极为实用的补充资料。它不仅记录并清晰呈现关键研究成果，而且对实践问题的深刻探讨独具特色。通过说明实际设计约束如何影响互连网络设计的方方面面，本书巧妙地将过去几十年的理论研究成果与真实系统设计联系起来。这种视角正是工程教育迫切需要的。”

——Li-Shiuan Peh，普林斯顿大学

“*Principles and Practices of Interconnection Networks* 兼具三大优点：全面、文笔出色且具有权威性。随着互连对计算机系统性能和成本的影响日益增大，人们对本书的需求也越来越迫切。它将成为学生和教师的极佳工具，也必将帮助工程师构建更好的网络。”

——Steve Scott，Cray 公司

“Dally 和 Towles 凭借两人合计三十年的经验，写出了一部阐明计算机互连网络理论与实践的著作。一方面，他们推导基本原理并列举可供选择的设计方案；另一方面，他们呈现了大量案例研究，也敢于对当前的设计取舍和未来趋势提出基于丰富经验的见解。对于关注或参与互连网络设计的人而言，本书是‘必买’之作。”

——Mark Hill，威斯康星大学麦迪逊分校

“本书将立即成为互连网络领域的经典参考文献。Dally 教授的开创性研究引入了严格的评估技术，以及应对高性能计算机系统通信挑战的创造性方案，从而戏剧性地、深远而持久地改变了这一领域。这部结构严谨的教科书将使学生和经验丰富的从业者同样受益。其内容呈现方式和习题源自多年课堂教学的经验。总之，这是一本不可或缺的资料。”

——Craig Stunkel，IBM

版权与出版信息

原书书名： Principles and Practices of Interconnection Networks
作者： William James Dally、Brian Towles
出版机构： Morgan Kaufmann Publishers（Elsevier 旗下品牌）
ISBN： 0-12-200751-4
版权： © 2004 Elsevier, Inc.，保留所有权利。
原版出版地址： 500 Sansome Street, Suite 400, San Francisco, CA 94111, USA。

原版封面图像：Frank Stella，*Takht-i-Sulayan-I*（1967）；封面设计：Hannus Design Associates；文字设计：Rebecca Evans & Associates；文字排版：Integra Software Services Pvt., Ltd.。原版出版总监 Diane D. Cerra，高级编辑 Denise E. M. Penrose，出版服务经理 Simon Crump，项目经理 Marcy Barnes-Henrie，编辑协调员 Alyson Day，编辑助理 Summer Block，文字编辑 Catherine Albano，校对 Deborah Prato，索引编制 Sharon Hilgenberg。

原版图 3.10 版权所有 © 2003 Silicon Graphics, Inc.，经授权使用。图 3.13 的转载授权来自 ACM，原始文献为 James Laudon 与 Daniel Lenoski，*The SGI Origin: A ccNUMA Highly Scalable Server*, ISCA 1997, 第 241–251 页。图 10.7 来自 Thinking Machines Corp.；图 11.5 由 Ray Mains 提供，照片来源：`http://www.mauigateway.com/~raymains/`。

商标及产品名称归各自权利人所有。未经原出版者书面许可，不得擅自复制或传播原版内容。授权事宜可联系 Elsevier 的版权部门，网址：`https://www.elsevier.com`，或原版所载邮件地址 `permissions@elsevier.com.uk`。

美国国会图书馆分类号： TK5105.5.D327 2003；004.6’5–dc22。**编目控制号：** 2003058915。

致谢

我们深深感谢众多为本书创作做出贡献的人。南加州大学的 Timothy Pinkston 和普林斯顿大学的 Li-Shiuan Peh 是第一批勇敢尝试使用本书草稿授课的人（除了作者之外）。他们的评论极大地提高了成书的质量。Mitchell Gusat、Mark Hill、Li-Shiuan Peh、Timothy Pinkston 和 Craig Stunkel 仔细审阅了本书草稿，并提供了宝贵的意见，促成了许多改进。

许多人（主要是原始网络的设计者）为案例研究提供了信息并验证了其准确性。Randy Rettberg 提供了关于 BBN Butterfly 和 Monarch 的信息。Charles Leiserson 和 Bradley Kuszmaul 补充了 Thinking Machines CM-5 网络的细节。Craig Stunkel 和 Bulent Abali 提供了关于 IBM SP1 和 SP2 的信息。关于 Alpha 21364 的信息由 Shubu Mukherjee 提供。Steve Scott 提供了关于 Cray T3E 的信息。Greg Thorson 提供了 T3E 的图片。

本材料的许多开发工作受到了在斯坦福大学和麻省理工学院与我们合作进行互连网络研究项目的学生和员工的影响，包括 Andrew Chien、Scott Wills、Peter Nuth、Larry Dennison、Mike Noakes、Andrew Chang、Hiromichi Aoki、Rich Lethin、Whay Lee、Li-Shiuan Peh、Jin Namkoong、Arjun Singh 和 Amit Gupta。

本材料是在多年教授互连网络课程（麻省理工学院的 6.845 和斯坦福大学的 EE482B）的过程中发展起来的。这些课程的学生帮助我们磨练了对材料的理解和表述。EE482B 过去的助教 Li-Shiuan Peh 和 Kelly Shaw 尤其值得感谢。

多年来，我们从与同事的讨论中学到了很多，包括 Jose Duato（瓦伦西亚）、Timothy Pinkston（南加州大学）、Sudha Yalamanchili（佐治亚理工）、Anant Agarwal（麻省理工学院）、Tom Knight（麻省理工学院）、Gill Pratt（麻省理工学院）、Steve Ward（麻省理工学院）、Chuck Seitz（Myricom）和 Shubu Mukherjee（英特尔）。我们对互连网络的实践理解得益于与 Justin Rattner（英特尔）、Dave Dunning（英特尔）、Steve Oberlin（Cray）、Greg Thorson（Cray）、Steve Scott（Cray）、Burton Smith（Cray）、Phil Carvey（BBN 和 Avici）、Larry Dennison（Avici）、Allen King（Avici）、Derek Chiou（Avici）、Gopalkrishna Ramamurthy（Velio）和 Ephrem Wu（Velio）的工业合作。

Denise Penrose、Summer Block 和 Alyson Day 在整个项目过程中给予了我们帮助。

我们还要感谢 Catherine Albano 和 Deborah Prato 的精心编辑，以及我们的生产经理 Marcy Barnes-Henrie，她引导本书走过了从手稿到成品有时颇为艰难的历程。

最后，我们的家人：Sharon、Jenny、Katie 和 Liza Dally 以及 Herman 和 Dana Towles 给予了巨大的支持并做出了重大牺牲，使我们能够有时间专注于写作。

前言

各类数字电子系统正迅速变得受限于通信。在这些系统中，限制成本、性能、尺寸和功耗的因素是数据的移动，而不是算术或控制逻辑。与此同时，长期以来作为系统互连主要形式的总线，已无法跟上不断增长的性能需求。

互连网络为这一通信危机提供了一种有吸引力的解决方案，并正在数字系统中变得无处不在。精心设计的互连网络能高效利用稀缺的通信资源——以最低的成本和能耗在客户端之间提供高带宽、低延迟的通信。

互连网络历史上仅用于高端超级计算机和电信交换机，如今已出现在各种规模、各种类型的数字系统中。它们被用于从大型超级计算机到小型嵌入式片上系统（SoC）的系统，以及包括处理器间通信、处理器-存储器互连、输入/输出和存储交换机、路由器交换结构以及替代专用布线在内的应用中。

事实上，随着系统复杂性和集成度的不断提高，许多设计者发现路由数据包比布线更高效。使用互连网络而非专用布线可以共享稀缺带宽，从而以高占空比高效利用带宽。相比之下，专用布线大部分时间处于空闲状态。使用网络还强制对通信资源进行规范、结构化的使用，使系统更易于设计、调试和优化。

互连网络的基本原理相对简单，设计一个高效满足给定应用所有要求的互连网络很容易。然而，如果不理解基本原理，也很容易设计出即使能工作也性能低下的互连网络。经验丰富的工程师设计过这样的网络：出现死锁，由于拓扑选择或路由算法不当而存在性能瓶颈，以及由于流控不佳而只能实现峰值性能的一小部分。如果设计者理解几条简单的原理，这些错误本可以轻易避免。

本书借鉴了作者二十多年来设计互连网络的经验。我们设计了数十个网络，这些网络如今构成了高性能计算机（包括消息传递和共享内存）、互联网路由器、电信电路交换机和 I/O 互连的骨干。这些系统围绕各种拓扑设计，包括交叉开关、环面网络、Clos 网络和蝶形网络。我们开发了虫洞路由和虚拟通道流控。在设计这些系统和开发这些方法的过程中，我们学到了许多关于什么有效、什么无效的经验教训。在本书中，我们以基于拓扑、路由、流控和路由器架构的一套简单的互连网络设计原理的形式，与读者分享这些经验带来的益处。

本书的组织结构

本书以两章导论开始，然后分为五个部分，分别涉及拓扑、路由、流控、路由器架构和性能。图 1 展示了本书的图形化大纲，显示了各节和各章之间的依赖关系。我们从第 1 章开始，描述什么是互连网络、它们如何被使用、不同应用的性能要求，以及

如何通过拓扑、路由和流控的设计选择来满足这些要求。为了使这些概念具体化并激发本书其余部分的兴趣，第 2 章详细描述了一个简单的互连网络：从拓扑一直到每个路由器的 Verilog 代码。这个例子的细节揭开了路由和流控这些抽象主题的神秘面纱，而这个简单网络的性能问题则激发了本书其余部分描述的更复杂的方法和设计途径。

设计互连网络的第一步是在给定一组封装约束的情况下，选择满足应用吞吐量、延迟和成本要求的拓扑。第 3 章至第 7 章探讨拓扑设计空间。我们从第 3 章开始，建立拓扑度量。拓扑的二分带宽和直径分别限制了其可实现的吞吐量和延迟，而其路径多样性决定了对抗性流量下的性能和容错能力。拓扑受到可用封装技术和成本要求的约束，模块引脚限制和系统线二分共同决定了可实现的通道宽度。在第 4 章至第 6 章中，我们讨论几种常见拓扑的性能度量和封装约束：蝶形网络、环面网络和非阻塞网络。我们对拓扑的讨论在第 7 章结束，涵盖集中和拓扑切片，这些方法用于处理突发流量并将拓扑映射到封装模块。

一旦选择了拓扑，路由算法就决定了有多少二分带宽可以转化为系统吞吐量，以及延迟能多接近直径极限。第 8 章至第 11 章描述路由问题及一系列解决方案。[1]好的路由算法在拓扑的通道之间负载均衡流量，以处理对抗性流量模式，同时利用良性流量模式的局部性。我们在第 8 章通过考虑环网络上的路由来介绍这个问题，并表明朴素的贪婪算法在对抗性流量下性能不佳。我们接着描述第 9 章的无关路由算法与第 10 章的自适应路由算法。本书关于路由的部分以第 11 章对路由实现机制的讨论结束。

流控机制通过沿途分配信道带宽和缓冲区容量，将数据包从源端沿路径排序到目的端。良好的流控机制避免资源空闲或数据包因资源限制而阻塞，从而能够分别实现大部分潜在吞吐量并最小化延迟。糟糕的流控机制可能因资源空闲而浪费吞吐量，因不必要地阻塞数据包而增加延迟，甚至可能导致死锁或活锁。这些主题将在第 12 章至第 15 章中探讨。

路由算法和流控方法中蕴含的策略在路由器中实现。第 16 章至第 22 章描述了路由器和网络接口的微架构。在这些章节中，我们介绍了路由器的构建模块并展示了它们如何组成。然后我们展示了路由器如何流水线化以每个周期处理一个 flit 或数据包。第 18 章和第 19 章特别关注仲裁和分配问题，因为这些功能对路由器性能至关重要。

为了将所有这些主题结合起来，本书以第 23 章至第 25 章对网络性能的讨论作为结尾。在第 23 章中，我们首先定义基本的性能指标，并指出一些可能导致误导性测量的常见陷阱。我们接着介绍了排队论和概率分析在预测互连网络性能中的应用。在第 24 章中，我们描述了如何使用模拟来预测网络性能，涵盖工作负载、测量方法和模拟器设计。最后，第 25 章给出了一些性能结果示例。

[1]英文原书此处写作“Chapters 4 through 11”，但根据本书目录与本段前后内容，讨论路由的实际范围是第 8～11 章；译文依章节内容校订，并保留此说明。

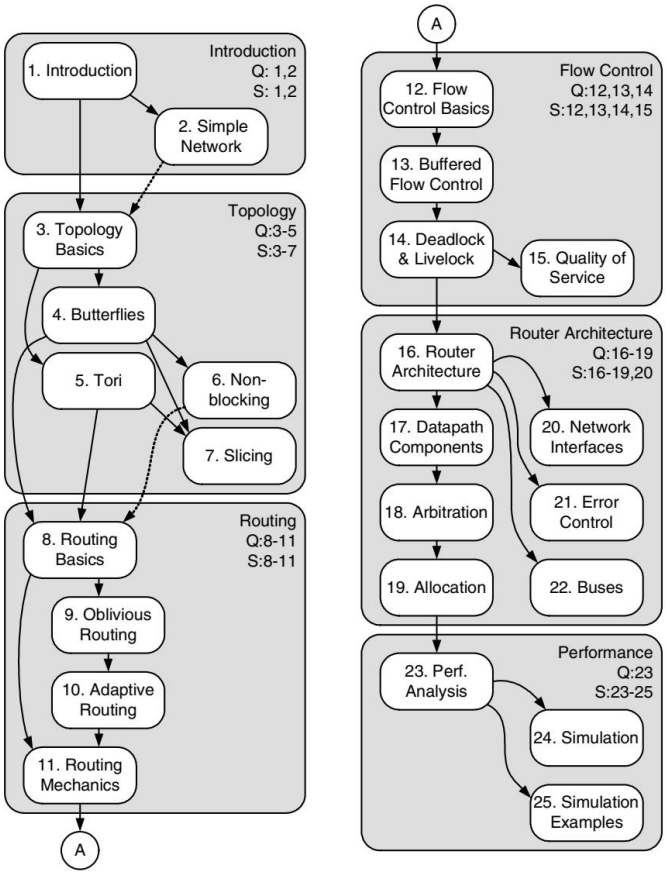


图 1　本书大纲，展示了各章之间的依赖关系。主要部分以阴影区域表示。任何关于该主题的课程都应涵盖的章节位于阴影区域的左侧。可选章节位于右侧。依赖关系由箭头指示。实线箭头表示要理解箭头头部的章节，必须理解箭头尾部的章节。虚线箭头表示理解箭头尾部的章节对理解箭头头部的章节有帮助，但不是必需的。每个阴影区域中的符号推荐了在季度课程（Q）和学期课程（S）中应涵盖的章节。

互连网络教学

作者们利用本书中的材料在麻省理工学院（6.845）和斯坦福大学（EE482b）教授互连网络研究生课程超过 10 年。多年来，这些课程的课堂笔记不断发展和完善。成果就是本书。

关于互连网络的季度或学期课程可以按照本书的大纲进行，如图 1 所示。个别讲师可以增加或删除可选章节（显示在阴影区域的右侧），以根据自身需求调整课程。

使用本书的季度课程的一种安排如表 1 所示。每讲大致对应本书的一章。学期课程可以从相同的基本大纲开始，并增加可选章节中的额外材料。

在使用本书教授研究生互连网络课程时，我们通常会布置一个研究或设计项目（除了布置每章选定的习题）。典型项目包括在给定一组约束的情况下设计一个互连网络（或网络的一个组件），并比较不同设计的性能。设计项目将课程材料整合在一起供学生使用。他们看到互连网络设计不同方面的相互作用，并能够直接应用所学原理。

表 1　互连网络十周课程的教学进度示例。每周通常介绍相应的一章；第 3 周的第 6 章只讲授至第 6.3.1 节。

周次	主题	章节
1	引言	1, 2
2	网络拓扑	3, 4
3	网络拓扑	5, (6)
4	路由	8, 9
5	路由	10, 11
6	流控制	12, 13, 14
7	路由器体系结构	16, 17
8	仲裁与分配	18, 19
9	性能分析	23
10	复习	

采用本书开设的季度制课程（斯坦福 EE482B）的教学材料可在 http://cva.stanford.edu/ee482b 在线获取。该页面还包括最近几次开设该课程的示例项目和学生论文。

作者简介

Bill Dally 在弗吉尼亚理工学院获得电气工程学士学位，在斯坦福大学获得电气工程硕士学位，在加州理工学院获得计算机科学博士学位。Bill 和他的团队开发了系统架构、网络架构、信号传输、路由和同步技术，这些技术如今在大多数大型并行计算机中都能找到。在贝尔电话实验室工作期间，Bill 为 BELLMAC32 微处理器的设计做出了贡献，并设计了 MARS 硬件加速器。在加州理工学院，他设计了 MOSSIM 模拟引擎和 Torus 路由芯片，开创了虫洞路由和虚拟通道流控。在麻省理工学院担任电气工程和计算机科学教授期间，他的团队构建了 J-Machine 和 M-Machine 实验性并行计算机系统，开创了机制与编程模型分离的先河，并展示了极低开销的同步和通信机制。Bill 目前是斯坦福大学电气工程和计算机科学教授。他在斯坦福的团队开发了 Imagine 处理器，引入了流处理和分区寄存器组织的概念。Bill 曾与 Cray Research 和 Intel 合作，将这些创新中的许多融入商用并行计算机。他还与 Avici Systems 合作，将该技术融入互联网路由器，并联合创立了 Velio Communications 以商业化高速信号传输技术。他是 IEEE 会士、ACM 会士，并获得了众多荣誉，包括 ACM Maurice Wilkes 奖。他目前领导高速信号传输、计算机架构和网络架构方面的项目。他在这些领域发表了 150 多篇论文，并且是教科书《Digital Systems Engineering》（剑桥大学出版社，1998 年）的作者。

Brian Towles 于 1999 年在佐治亚理工学院获得计算机工程学士学位，2002 年在斯坦福大学获得电气工程硕士学位。他目前正在斯坦福大学攻读电气工程博士学位。他的研究兴趣包括互连网络、网络算法和并行计算机体系结构。

第一章　互连网络导论

数字系统在现代社会中无处不在。数字计算机被用于从模拟物理系统到管理大型数据库再到准备文档的各种任务。数字通信系统中继电话呼叫、视频信号和互联网数据。音频和视频娱乐正越来越多地以数字形式交付和处理。最后，从汽车到家用电器，几乎所有产品都受到数字控制。

数字系统由三个基本构建模块组成：逻辑、存储和通信。逻辑变换和组合数据——例如，通过执行算术运算或做出决策。存储保存数据以供后续检索，在时间上移动数据。通信将数据从一个位置移动到另一个位置。本书讨论数字系统的通信组件。具体来说，它探讨用于在数字系统子系统之间传输数据的互连网络。

当今大多数数字系统的性能受限于其通信或互连，而不是其逻辑或存储。在高端系统中，大部分功耗用于驱动导线，大部分时钟周期花费在导线延迟上，而不是门延迟上。随着技术的进步，存储器和处理器变得小巧、快速且廉价。然而，光速保持不变。决定系统组件之间互连的引脚密度和布线密度正以比组件本身更慢的速度扩展。此外，组件之间的通信频率远远落后于现代处理器的时钟频率。这些因素共同使互连成为未来数字系统成功的关键因素。

随着设计者努力更有效地利用稀缺的互连带宽，互连网络正成为现代数字系统系统级通信问题的近乎通用的解决方案。互连网络最初是为多计算机的苛刻通信需求而开发的，现在正开始取代总线成为标准的系统级互连。它们也在专用系统中取代专用布线，因为设计者发现路由数据包比路由导线既更快又更经济。

1.1　关于互连网络的三个问题

在进一步讨论之前，我们将回答一些关于互连网络的基本问题：什么是互连网络？在哪里可以找到它们？为什么它们很重要？

什么是互连网络？ 如图 1.1 所示，互连网络是一种在终端之间传输数据的可编程系统。该图显示了六个终端 T1 到 T6 连接到一个网络。当终端 T3 希望与终端 T5 通信某些数据时，它向网络发送包含数据的消息，网络将消息传递给 T5。网络是可编程的，因为它在不同的时间点建立不同的连接。图中的网络可以在一个周期内将消息从 T3 传递到 T5，然后在下一个周期使用相同的资源将消息从 T3 传递到 T1。网络是一个系统，因为它由许多组件组成：缓冲区、通道、交换机和控制器，它们协同工作以传递数据。

符合这一宽泛定义的网络出现在许多规模上。片上网络可以在单个处理器内的存储器阵列、寄存器和算术单元之间传递数据。板级和系统级网络将处理器连接到存储

器，或将输入端口连接到输出端口。最后，局域网和广域网将企业内或全球范围内的不同系统连接在一起。在本书中，我们将注意力限制在较小的规模：从芯片级到系统级。已经有许多优秀的教材讨论更大规模的网络。然而，系统级及以下的问题，其中通道短且数据速率非常高，与大规模的问题根本不同，需要不同的解决方案。

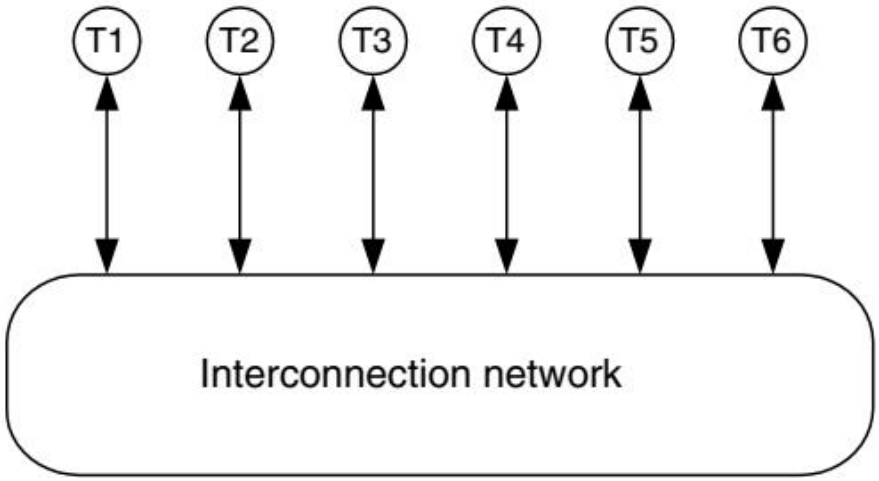


图 1.1 互连网络的功能视图。终端（标记为 T1 到 T6）使用通道连接到网络。通道两端的箭头表示它是双向的，支持数据进出互连网络。

在哪里可以找到互连网络？ 它们几乎用于所有大到需要连接两个组件的数字系统中。互连网络最常见的应用是在计算机系统和通信交换机中。在计算机系统中，它们将处理器连接到存储器，将输入/输出（I/O）设备连接到 I/O 控制器。它们在通信交换机和网络路由器中将输入端口连接到输出端口。它们还在控制系统中将传感器和执行器连接到处理器。任何在系统两个组件之间传输比特的地方，都可能找到互连网络。

直到 20 世纪 80 年代末，大多数这些应用都由一种非常简单的互连网络提供服务：多点总线。如果本书当时写成，它可能是一本关于总线设计的书。我们将第 22 章专门讨论总线，因为它们在许多应用中仍然重要。然而，今天所有高性能互连都是通过点对点互连网络而不是总线进行的，并且越来越多历史上基于总线的系统每年都在转向网络。这一趋势是由于非均匀的性能扩展。对互连性能的需求随着处理器性能（以每年 50% 的速度）和网络带宽而增加。另一方面，导线并没有变得更快。光速和 24 号铜线的衰减不会随着更好的半导体技术而改善。结果，总线无法跟上带宽需求，而点对点互连网络，既比总线运行更快又提供并发性，正在迅速取代总线。

为什么互连网络很重要？ 因为它们是许多系统性能的限制因素。处理器与存储器之间的互连网络在很大程度上决定了计算机系统的存储器延迟和存储器带宽这两个关键性能因素。[1] 通信交换机中互连网络（在此语境下有时称为“交换结构”）的性能在很大程度上决定了交换机的容量（数据速率和端口数量）。由于互连需求的增长速度超过了底层线路能力的提升速度，互连已成为大多数系统中的关键瓶颈。

互连网络是专用布线的一种有吸引力的替代方案，因为它们允许多个低占空比信

[1]如果考虑到现代存储器芯片访问时间中的大部分其实是通信延迟，这一点就尤其明显。

号共享稀缺的布线资源。在图 1.1 中，假设每个终端每 100 个周期需要与其他每个终端通信一个字。我们可以为每对终端之间提供一条专用的字宽通道，总共需要 30 条单向通道。然而，每条通道在 99% 的时间里都处于空闲状态。相反，如果我们将 6 个终端连接成一个环，则只需要 6 条通道。（T1 连接到 T2，T2 连接到 T3，依此类推，最后 T6 连接到 T1。）采用环形网络后，通道数量减少了五倍，通道占空比从 1% 提高到 12.5%。

1.2 互连网络的用途

为了理解互连网络设计所面临的要求，考察它们在数字系统中的使用方式是有益的。在本节中，我们考察互连网络的三种常见用途，并了解这些应用如何驱动网络需求。具体来说，对于每种应用，我们将考察它如何决定以下网络参数：

1. 终端数量；
2. 每个终端的峰值带宽；
3. 每个终端的平均带宽；
4. 所需延迟；
5. 消息大小或消息大小的分布；
6. 预期的流量模式；
7. 所需的服务质量；
8. 互连网络所需的可靠性和可用性。

我们已经看到，网络中的终端（或端口）数量对应于必须连接到网络的组件数量。除了知道终端数量之外，设计者还需要知道终端将如何与网络交互。

每个终端都需要从网络获得一定量的带宽，通常以比特每秒（bit/s）表示。除非另有说明，我们假设终端带宽是对称的——即终端的输入和输出带宽相等。峰值带宽是终端在短时间内向网络请求的最大数据速率，而平均带宽是终端所需的平均数据速率。如下文关于处理器-存储器互连设计的一节所示，在试图最小化互连网络的实现成本时，了解峰值带宽和平均带宽都变得很重要。

除了网络必须接受和传递消息的速率之外，传递单个消息所需的时间，即消息延迟，也是网络的一项指定指标。虽然理想的网络同时支持高带宽和低延迟，但这两个参数之间往往存在权衡。例如，支持高带宽的网络倾向于让网络资源保持忙碌，这通常会导致对资源的争用。当两个或多个消息想要使用网络中的同一共享资源时，就会发生争用。除一个消息外，所有这些消息都将不得不等待该资源变为空闲，从而增加了消息的延迟。相反，如果通过降低带宽需求来减少资源利用率，延迟也会降低。

消息大小，即消息以比特为单位的长度，是另一个重要的设计考虑因素。如果消

息很小，网络中的开销对性能的影响可能比开销可以在较大消息的长度上分摊的情况更大。在许多系统中，存在几种可能的消息大小。

来自每个终端的消息如何分布到所有可能的目的终端，定义了网络的流量模式。例如，每个终端可能以相等的概率向所有其他终端发送消息。这就是随机流量模式。相反，如果终端倾向于只向附近的其他终端发送消息，底层网络可以利用这种空间局部性来降低成本。然而，在其他网络中，规范对任意流量模式都成立是很重要的。

一些网络还需要服务质量（QoS）。粗略地说，QoS 涉及在某些服务策略下对资源的公平分配。例如，当多个消息争用网络中的同一资源时，这种争用可以通过多种方式解决。可以根据消息等待所涉资源的时间长短，按先来先服务的原则处理消息。另一种方法给予在网络中停留时间最长的消息优先权。在这些分配策略和其他分配策略之间进行选择，取决于网络所需的服务。

最后，互连网络所需的可靠性和可用性会影响设计决策。可靠性是衡量网络正确执行消息传递任务频率的指标。在大多数情况下，消息需要 100% 无丢失地传递。实现 100% 可靠的网络可以通过添加专用硬件来检测和纠正错误、使用更高层的软件协议，或混合使用这些方法来实现。也可能允许一小部分消息被网络丢弃，正如我们将在下文关于分组交换结构的一节中看到的那样。网络的可用性是网络可用且正确运行的时间比例。在互联网路由器中，通常规定 99.999% 的可用性——每年总停机时间少于五分钟。提供这种可用性水平的挑战在于，用于实现网络的组件往往每分钟会故障数次。因此，网络必须设计成能够检测并快速从这些故障中恢复，同时继续运行。

1.2.1 处理器-存储器互连

图 1.2 展示了使用互连网络将处理器连接到存储器的两种方法。图 1.2（a）展示了一种舞厅式架构[2]，其中 P 个处理器通过互连网络连接到 M 个存储体。大多数现代机器使用图 1.2（b）所示的集成节点配置，其中处理器和存储器组合在一个集成节点中。在这种安排下，每个处理器可以通过通信交换机 C 访问其本地存储器，而无需使用网络。

两种配置对网络的要求列于表 1.1 中。处理器端口的数量可能达到数千个，例如最大配置的 Cray T3E 中有 2,176 个处理器端口，或者小到单个处理器的 1 个。如今，64 到 128 个处理器的配置在高端服务器中很常见，并且这个数字随着时间的推移而增加。对于组合节点配置，每个处理器端口也是存储器端口。另一方面，在舞厅配置中，存储器端口的数量通常远大于处理器端口的数量。例如，一个高端向量处理器有 32 个处理器端口，向 4,096 个存储体发出请求。这种大比例最大化了存储器带宽，并减少了两个处理器同时需要访问同一存储体时发生体冲突的概率。

现代微处理器每秒执行约 10^9 条指令，每条指令可能需要从存储器获取两个 64

[2]之所以称为“舞厅式架构”，是因为处理器排在网络一侧、存储体排在另一侧的布局，类似于旧式舞厅中男女分别在舞池两侧列队。

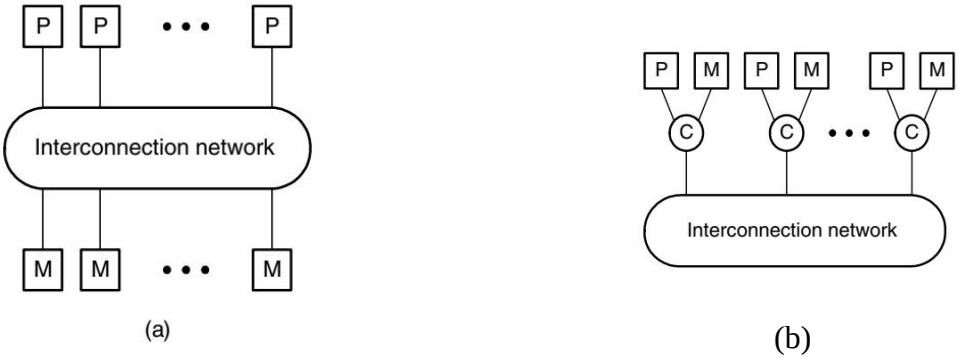


图 1.2 使用互连网络连接处理器与存储器。(a) 舞厅式架构：处理器（P）与存储体（M）分别连接到网络。(b) 集成节点架构：处理器和存储器共用网络端口，并可在本地访问一个存储体。

表 1.1 处理器—存储器互连网络的参数

参数	取值
处理器端口数	1–2,048
存储器端口数	0–4,096
峰值带宽	8 Gbytes/s
平均带宽	400 Mbytes/s
消息延迟	100 ns
消息大小	64 或 576 位
流量模式	任意
服务质量	无
可靠性	不允许消息丢失
可用性	0.999–0.99999

位字（一个用于指令本身，一个用于数据）。如果这些引用之一在缓存中未命中，通常从存储器中获取一个 8 字的块。如果我们真的需要每个周期从存储器获取 2 个字，这将需要 16 GB/s 的带宽。幸运的是，只有约三分之一的指令引用存储器中的数据，并且缓存能有效减少实际必须引用存储体的次数。在典型的缓存未命中率下，平均带宽要低一个数量级以上——约为 400 MB/s。[3]然而，为了避免因串行化而增加存储器延迟，大多数处理器仍然需要能够以每条指令一个字的峰值速率从存储器系统获取数据。如果我们过度限制这个峰值带宽，突然爆发的存储器请求会迅速堵塞处理器的网络端口。通过较低带宽的网络端口挤压这种高带宽突发请求的过程，类似于堵塞的水槽缓慢排水，被称为串行化，并会增加消息延迟。为了避免请求突发期间的串行化，我们需要 8 GB/s 的峰值带宽。

处理器性能对存储器延迟非常敏感，因此也对互连网络的延迟敏感，存储器请求和回复通过该网络传输。在表 1.1 中，我们列出了 100 ns 的延迟要求，因为这是典型存储器系统在没有网络时的基本延迟。如果我们的网络增加额外的 100 ns 延迟，我们就将有效存储器延迟加倍了。当加载和存储指令在处理器缓存中未命中时（并且在集成节点配置中不是寻址到本地存储器），它们被转换为读请求和写请求数据包，并通过网络转发到相应的存储体。每个读请求数据包包含要读取的存储器地址，每个写请求数据包包含存储器地址以及要写入的一个字或缓存行。在相应的存储体收到请求数据包后，它执行请求的操作并发送相应的读回复或写回复数据包。[4]注意，我们已经开始在网络中区分消息和数据包。消息是网络客户端（在本例中为处理器和存储器）到网络的传输单元。在网络接口处，单个消息可以创建一个或多个数据包。这种区分允许简化底层网络，因为大消息可以分解为几个较小的数据包，或者不等长的消息可以拆分为固定长度的数据包。由于此处理器-存储器互连中创建的消息相对较小，我们假设消息和数据包之间一一对应。

图 1.3 处理器—存储器互连所需的两种数据包格式。短数据包（64 位）用于读请求、写回复；长数据包（576 位）用于读回复、写请求，其中包含 512 位缓存行数据。

读请求和写回复数据包不包含任何数据，但存储一个地址。这个地址加上网络使用的一些头部和数据包类型信息可以轻松地容纳在 64 位内。读回复和写请求数据包包含相同的 64 位头部和地址信息，加上 512 位缓存行的内容，从而形成 576 位的数

[3]不过，平均带宽需求对应用程序十分敏感。有些应用程序的局部性很差，导致缓存未命中率很高，需要从存储器获取 2 Gbytes/s 或更高的带宽。

[4]如果计算机通过互连网络运行缓存一致性协议，还需要若干额外类型的数据包；但基本约束相同。

据包。这两种数据包格式如图 1.3 所示。

正如处理器-存储器互连的典型情况，我们不需要任何特定的 QoS。这是因为网络本质上是自节流的。也就是说，如果网络变得拥塞，存储器请求将需要更长时间才能完成。由于处理器只能有有限数量的未完成请求，它们将开始空闲，等待回复。因为处理器在空闲时不会创建新请求，所以网络的拥塞减少了。这种稳定行为被称为自节流。大多数 QoS 保证仅在网络拥塞时影响网络，但自节流倾向于避免拥塞，从而使 QoS 在处理器-存储器互连中不太有用。

此应用需要一个本身可靠且无丢包的网络。内存请求包和应答包不能丢失。丢失的请求包会导致内存操作永远挂起。轻则导致用户程序因超时而崩溃，重则可能拖垮整个系统。可靠性可以叠加在不可靠网络之上——例如，让每个网络接口保留每个已发送数据包的副本，直到收到确认，并在丢包时重传。（见第 21 章。）然而，这种方法通常会给处理器-内存互连带来不可接受的延迟。根据应用的不同，处理器-内存互连需要从三个九（99.9%）到五个九（99.999%）的可用性。

1.2.2 I/O 互连

互连网络也用于计算机系统中，将 I/O 设备（如磁盘驱动器、显示器和网络接口）连接到处理器和/或内存。图 1.4 展示了一个典型 I/O 网络的示例，用于将一组磁盘驱动器（位于图的底部）连接到一组主机适配器。该网络的基本运行方式与处理器—存储器互连相同，但数据传输粒度与时序要求不同。这些差异，尤其是对延迟容忍度的提高，推动网络设计朝着截然不同的方向发展。

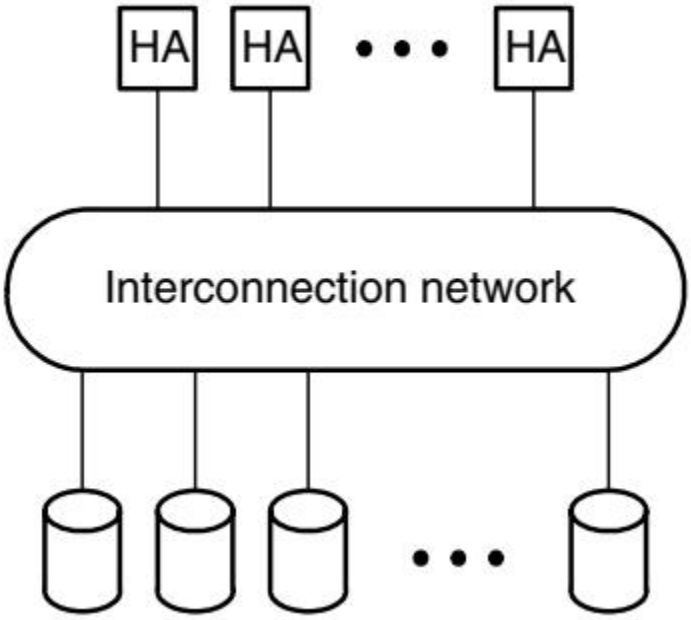


图 1.4 一个典型的 I/O 网络将多个主机适配器连接到更多数量的 I/O 设备——在本例中为磁盘驱动器。

磁盘操作通过传输 4 KB 或更大的扇区来完成。由于磁盘的旋转延迟加上重新定

位磁头所需的时间，一个扇区访问的延迟可能长达数毫秒。磁盘读取通过从主机适配器发送一个控制包来完成，该包指定要读取的磁盘地址（设备和扇区）以及作为读取目标的内存块。当磁盘收到请求后，它会调度磁头移动以读取所请求的扇区。一旦磁盘读取了所请求的扇区，它就会向相应的主机适配器发送一个响应包，其中包含该扇区并指定目标内存块。

表 1.2 列出了高性能 I/O 互连网络的参数。该网络最多连接 64 个主机适配器，并且每个主机适配器可能连接许多物理设备，例如硬盘驱动器。在此示例中，每个主机适配器最多可连接 64 个 I/O 设备，总共 4,096 个设备。更典型的系统可能会将几个主机适配器连接到大约一百个设备。

磁盘端口具有很高的峰值带宽与平均带宽之比。当磁盘传输连续扇区时，其读取数据速率可高达 200 MB/s。这个数字决定了表中显示的峰值带宽。更常见的情况是，磁盘必须在扇区之间执行磁头移动，平均耗时 5 ms（或更长），导致平均数据速率为每 5 ms 一个 4 KB 扇区，即低于 1 MB/s。由于每个主机端口处理来自 64 个磁盘端口的聚合流量，因此它们的峰值带宽与平均带宽之比较低。

设备端口处峰值带宽与平均带宽之间的巨大差异要求采用具有集中功能的网络拓扑。虽然设计一个能同时支持所有设备峰值带宽的网络当然足够，但由此产生的网络将非常昂贵。或者，我们可以设计网络仅支持平均带宽，但正如在处理器-内存互连示例中所讨论的，这会引入串行化延迟。由于峰值带宽与平均带宽之比很高，这种串行化延迟将相当大。一种更有效的方法是将许多设备的请求集中到一个“聚合”端口上。这个聚合端口的平均带宽与共享它的设备数量成正比。然而，由于单个设备很少向网络请求其峰值带宽，因此许多设备中不太可能有两个以上的设备同时向聚合端口要求其峰值带宽。通过集中，我们有效地降低了峰值带宽与平均带宽需求之间的比率，从而允许更廉价的实现，而不会产生过多的串行化延迟。

表 1.2 输入／输出互连网络的参数

参数	取值
设备端口数	1–4,096
主机端口数	1–64
峰值带宽	200 Mbytes/s
平均带宽	1 Mbytes/s（设备端）；64 Mbytes/s（主机端）
消息延迟	10 μs
消息大小	32 字节或 4 Kbytes
流量模式	任意
可靠性	不允许消息丢失[a]
可用性	0.999～0.99999

注 a：允许少量消息丢失，因为 I/O 操作失败后的错误恢复要比存储器访问失败后的错误恢复更为平稳。

与处理器-内存网络一样，消息有效载荷大小呈双峰分布，但两种模式之间的差距更大。网络承载短（32 字节）消息以请求读操作、确认写操作和执行磁盘控制。另

一方面，读应答和写请求消息需要较长的消息（原书此处写为 8 KB）。[5]

由于磁盘操作的固有延迟很大（毫秒级），并且作为单位传输的数据量很大（4 KB），因此网络对延迟不太敏感。将延迟增加到 $10\mu s$ 会导致性能下降可忽略不计。这种宽松的延迟规范使得构建高效的 I/O 网络比构建延迟至关重要的同等处理器-内存网络要简单得多。

用于集群并行计算机中快速消息传递的处理器间通信网络，在带宽和粒度方面实际上与 I/O 网络非常相似，因此将不单独讨论。这些网络通常被称为系统区域网络（SAN），它们与 I/O 网络的主要区别在于对消息延迟更敏感，通常要求网络延迟小于几微秒。

在磁盘存储用于为企业保存关键数据的应用中，需要极高的可用性。如果存储网络宕机，业务就会中断。存储系统的可用性达到 0.99999（五个九）——每年停机时间不超过五分钟——并不罕见。

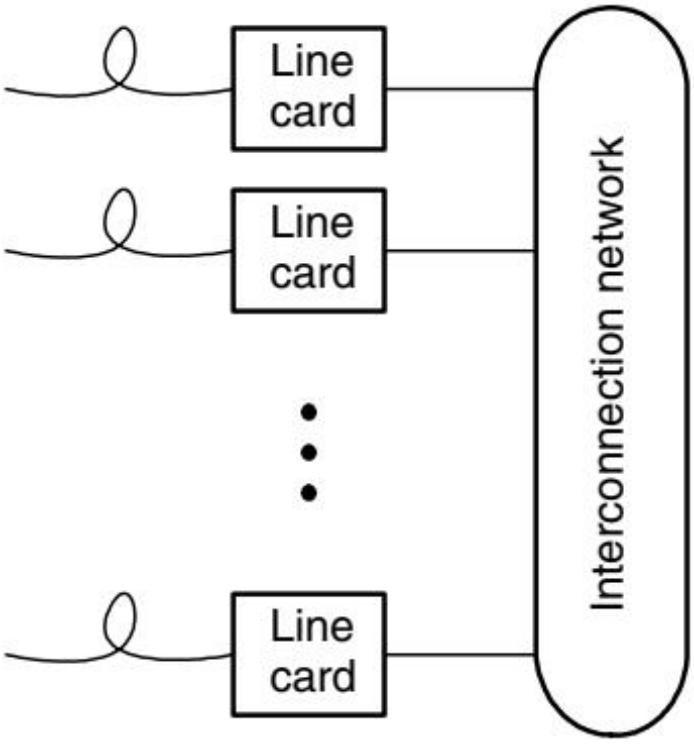


图 1.5 一些网络路由器使用互连网络作为交换结构，在通过网络通道发送和接收数据包的线路卡之间传递数据包。

1.2.3 分组交换结构

互连网络一直在取代总线和交叉开关，成为通信网络交换机和路由器的交换结构。在这种应用中，互连网络充当更大规模网络（局域网或广域网）中路由器的组成元件。图 1.5 展示了这种应用的一个示例。一组线路卡端接大规模网络通道（通常是

[5]校订注：原著表 1.2 将消息大小列为“32 bytes or 4 Kbytes”，正文却使用“8-Kbyte messages”。两处不一致为原著已有差异，本译稿分别保留，以免在没有其他设计条件佐证时擅自统一数值。

带宽为 2.5 Gbits/s 或 10 Gbits/s 的光纤)。[6]线路卡处理每个数据包或信元，以确定其目的地、验证其是否符合服务协议、重写数据包的某些字段，并更新统计计数器。然后，线路卡将每个数据包转发到交换结构。接着，交换结构负责将每个数据包从其源线路卡转发到其目的线路卡。在目的端，数据包被排队并调度在输出网络通道上传输。

表 1.3 显示了用作交换结构的典型互连网络的特征。交换结构要求与处理器-存储器和 I/O 网络要求之间最大的区别在于其高平均带宽和对服务质量的需求。

交换结构的大数据包尺寸及其对延迟不敏感的特性简化了网络设计，因为延迟和消息开销不必高度优化。确切的数据包大小取决于路由器使用的协议。对于互联网协议（IP），数据包范围从 40 字节到 64 Kbytes，[7]大多数数据包的长度为 40、100 或 1,500 字节。与我们其他两个示例一样，数据包分为短控制消息和大数据传输。

表 1.3 分组交换结构的参数

参数	**取值**
端口数	4–512
峰值带宽	10 Gbits/s
平均带宽	7 Gbits/s
消息延迟	10 μs
数据包有效载荷大小	40–64 Kbytes
流量模式	任意
可靠性	丢包率小于 10^{-15}
服务质量	需要保证
可用性	0.999～0.99999

网络交换结构不像处理器-存储器或 I/O 互连那样具有自节流特性。每个线路卡继续发送稳定的数据包流，无论交换结构中的拥塞情况如何，同时交换结构必须为某些类别的数据包提供保证带宽。为了满足这种服务保证，交换结构必须是非干扰的。也就是说，发往线路卡 a 的过多流量（可能是由于瞬时过载）不应干扰或“窃取”发往不同线路卡 b 的流量的带宽，即使发往 a 的消息和发往 b 的消息在整个交换结构中共享资源。这种非干扰需求对网络交换结构的底层实现提出了独特的要求。

交换结构的一个有趣方面可能简化其设计：在某些应用中，可以接受丢弃极小比例的数据包——比如每 10^{15} 个数据包中丢弃一个。在由于其他原因已经执行数据包丢弃的情况下，这是允许的，原因包括输入光纤上的位错误（通常误码率在 10^{-12} 到 10^{-15} 范围内）到线路卡队列溢出。在这些情况下，通常由更高层协议处理丢弃的数据包，因此路由器可以通过丢弃相关数据包来处理极不可能的情况（例如内部位错误），只要这些丢弃的速率远低于因其他原因导致的数据包丢弃速率。这与处理器-存储器互连形成对比，在后者中单个丢失的数据包可能使机器锁死。

[6]以本书出版时期为准，典型的高端 IP 路由器可连接 8 至 40 条 10 Gbits/s 通道，至少有一家厂商的产品可扩展到 512 条。预计随着路由器总带宽大约每十八个月翻一番，这些数字还将增长。

[7]以太网协议把最大数据包长度限制在不超过 1,500 字节。

1.3 网络基础

为了满足特定应用（如上述应用）的性能规格，网络设计者必须在技术约束内实现网络的拓扑、路由和流控。正如我们在前几节中所说，互连网络效率的关键在于通信资源是共享的。互连网络不是在每个终端对之间创建专用通道，而是由一组通过共享通道连接的共享路由器节点实现。这些节点的连接模式定义了网络的拓扑。然后，消息通过从源终端到目的终端经过共享通道和节点的多跳传递在终端之间传送。良好的拓扑利用网络封装技术的特性（例如芯片封装上的引脚数或机柜之间可连接的电缆数）来最大化网络的带宽。

一旦选择了拓扑，消息在网络中到达目的地可能有多条路径（节点和通道的序列）。路由决定消息实际采用这些可能路径中的哪一条。良好的路径选择可以最小化路径长度（通常以访问的节点或通道数衡量），同时平衡网络共享资源上的需求。路径长度显然影响消息通过网络时的延迟，而资源上的需求或负载是衡量该资源被使用频率的指标。如果一个资源过度利用而另一个资源闲置，称为负载不平衡，则网络传递的消息总带宽会降低。

流控决定哪些消息随时间获得对特定网络资源的访问权。随着资源利用率的增加，流控的这种影响变得更加关键，良好的流控以最小延迟转发数据包，并在高负载下避免资源闲置。

1.3.1 拓扑结构

互连网络由一组共享的路由器节点和通道组成，网络的拓扑结构指的是这些节点和通道的排列方式。互连网络的拓扑结构类似于路线图。通道（如同道路）将数据包（如同汽车）从一个路由器节点（交叉路口）传送到另一个路由器节点。例如，图 1.6 所示的网络由 16 个节点组成，每个节点连接到 8 条通道，每个邻居各有一条通道进出。这个特定的网络具有环面拓扑结构。在图中，节点用圆圈表示，每对通道（每个方向一条）用连接两个节点的线表示。这种拓扑结构也是一种直接网络，其中每个终端与拓扑结构的 16 个节点之一相关联。

良好的拓扑结构能够利用可用封装技术的特性，以最低成本满足应用的带宽和延迟要求。为了最大化带宽，拓扑结构应充分利用底层封装技术提供的对分带宽，即系统中点处的带宽。

例如，图 1.7 展示了图 1.6 中的网络可能采用的封装方式。四个节点一组放置在垂直印刷电路板上。然后使用背板电路板连接其中四块电路板，就像 PCI 卡插入 PC 主板一样。对于这个系统，对分带宽是可以通过该背板传输的最大带宽。假设背板足够宽，可以容纳 256 条信号线，每条信号线以 1 Gbit/s 的数据速率运行，则总对分带宽为 256 Gbits/s。

回顾图 1.6，恰好有 16 条单向通道穿过我们拓扑结构的中点——请记住，图中的线代表两条通道，每个方向一条。为了充分利用 256 条信号线的对分带宽，穿过对分

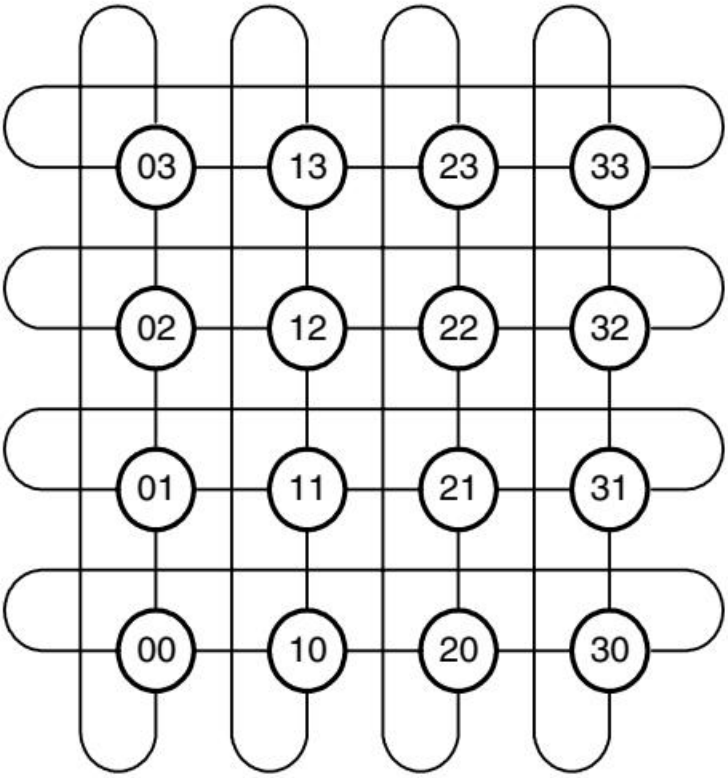


图 1.6 网络拓扑结构是节点（用编号 00 至 33 的圆圈表示）和连接节点的通道的排列。图中每条线表示一对通道（每个方向一条）。在这个 4×4 二维环面（或 4 元 2 立方体）拓扑结构中，每个节点连接到 8 条通道：与其 4 个邻居各有一条通道进出。

带宽的每条通道宽度应为 256/16 = 16 条信号线。然而，我们还必须考虑到每个节点将封装在单个 IC 芯片上。在这个例子中，每个芯片只有足够的引脚支持 128 条信号线。由于我们的拓扑结构要求每个节点总共 8 条通道，每个芯片的引脚限制将通道宽度限制为 128/8 = 16 条信号线。幸运的是，引脚限制给出的通道宽度恰好与充分利用对分带宽所需的信号线数量相匹配。

相比之下，考虑图 1.8 所示的 16 节点环形网络。每个节点连接 4 条通道，因此引脚限制将通道宽度限制为 128/4 = 32 条信号线。四条通道穿过对分带宽，因此我们希望将这些通道设计为 256/4 = 64 条信号线宽以充分利用对分带宽，但引脚将通道宽度限制为仅一半。因此，在相同的技术约束下，环形拓扑结构提供的带宽仅为环面拓扑结构的一半。在带宽方面，环面拓扑结构显然是更优选择，在系统中点处为每个节点提供完整的 32 Gbits/s 带宽。

然而，高带宽并不是衡量拓扑结构性能的唯一指标。假设我们有另一个应用，在相同的技术约束下只需要 16 Gbits/s 的带宽，但也需要尽可能低的延迟。此外，假设该应用使用相当长的 4,096 位数据包。为了实现低延迟，拓扑结构必须在节点间小平均距离的期望与低串行化延迟之间取得平衡。

节点之间的距离称为跳数，以消息到达目的地平均需要经过的通道和节点数量来衡量。减少这个距离需要增加节点度（进入和离开每个节点的通道数量）。然而，由

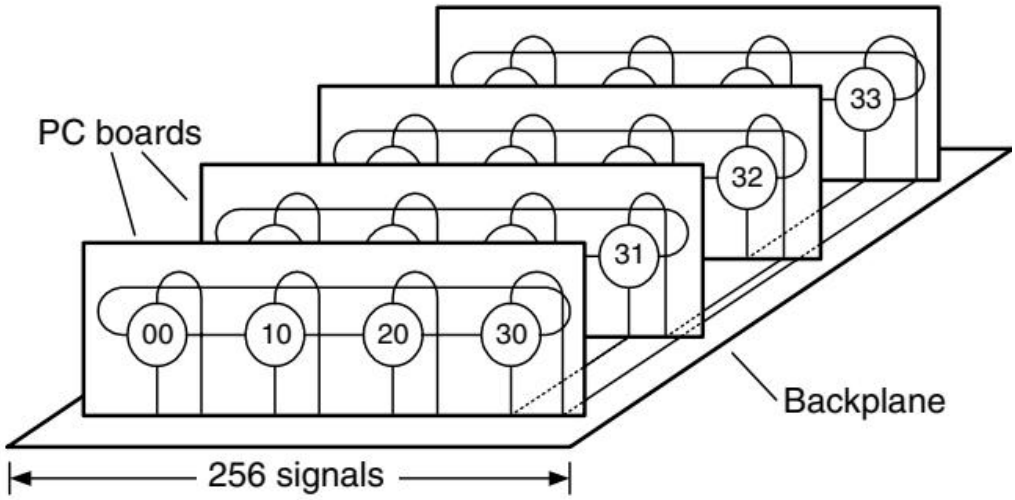


图 1.7 16 节点环面拓扑结构的封装。四个节点一组封装在单个印刷电路板上，其中四块电路板连接到单个背板。第三列的背板通道显示在背板的右边缘。背板宽度上的信号线数量（256）定义了该特定封装的对分带宽。

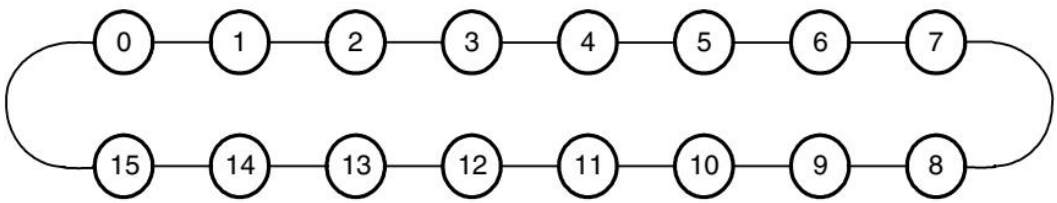


图 1.8 在我们示例的约束下，16 节点环形网络比图 1.6 的 16 节点二维环面具有更低的延迟。这种延迟是以较低的吞吐量为代价实现的。

于每个节点受到固定的引脚限制，增加通道数量会导致通道宽度变窄。通过窄通道发送大数据包会引起串行化延迟。为了了解这种权衡如何影响拓扑结构选择，我们重新审视我们的两种 16 节点拓扑结构，但现在我们关注消息延迟。

首先，为了量化由跳数引起的延迟，需要假设一种流量模式。为简单起见，我们使用随机流量，其中每个节点以相等概率向其他每个节点发送数据。随机流量下的平均跳数就是节点间的平均距离。对于我们的环面拓扑结构，平均距离为 2，对于环形，平均距离为 4。在典型网络中，每跳延迟可能为 20 ns，对应于环面拓扑结构的总跳延迟为 40 ns，环形为 80 ns。

然而，环形的宽通道使其串行化延迟低得多。通过 32 条信号线的通道发送 4,096 位数据包需要 4,096/32 = 128 个通道周期。我们的信号速率为 1 GHz，对应周期为 1 ns，因此环形的串行化延迟为 128 ns。如果我们的网络设计高效，这种串行化时间只需支付一次，这使得通过环形的每个数据包平均延迟为 80 + 128 = 208 ns。对环面拓扑结构进行类似计算，得到串行化延迟为 256 ns，总延迟为 296 ns。尽管环形的平均跳数更大，但物理封装的约束使其对于这些长数据包具有更低的延迟。

正如我们在这里所看到的，没有一种拓扑对所有应用都是最优的。不同的拓扑适用于不同的约束和要求。关于拓扑的详细讨论见第 3 章至第 7 章。

1.3.2 路由

网络采用的路由方法决定了数据包从源终端节点到目的终端节点所经过的路径。路由或路径是一组有序的通道 $P = \{c_1, c_2, \ldots, c_k\}$，其中通道 c_i 的输出节点等于通道 c_{i+1} 的输入节点，源是通道 c_1 的输入，目的是通道 c_k 的输出。在某些网络中，每个源到每个目的只有一条路由，而在其他网络中，例如图 1.6 中的环面网络，存在多条可能的路径。当存在多条路径时，好的路由算法无论提供的流量模式如何，都能在通道间均匀地平衡负载。继续我们的路线图类比：拓扑提供了路线图、道路和交叉口，路由方法则驾驶汽车，在每个交叉口决定转向哪条路。就像在道路上引导车辆一样，重要的是分散流量——平衡不同道路上的负载，而不是让一条道路拥堵而平行的道路空闲。

图 1.9 显示了图 1.6 网络中从节点 01 到节点 22 的两条不同路由。在图 1.9（a）中，数据包采用维序路由，首先在 x 维路由到达节点 21，然后在 y 维路由到达目的节点 22。这条路由是最小路由，因为它是从 01 到 22 的最短路径之一。（共有六条。）图 1.9（b）绘出了从节点 01 到节点 22 的另一条路径。这是一条非最小路径，需要 5 跳，而最短路径为 3 跳。[8]

虽然维序路由简单且最小，但对于某些流量模式，它可能产生显著的负载不平衡。例如，考虑在图 1.9（a）中从节点 11 到节点 20 添加另一条维序路由。这条路由也使

[8]校订注：英文原著第 16 页的对应正文写作“from 00 to 22”，但本段开头、图 1.9 的图题及图中的实际箭头均以节点 01 为起点。本译稿依据图示与图题将正文统一为节点 01，并保留此处说明，供读者核对原版。

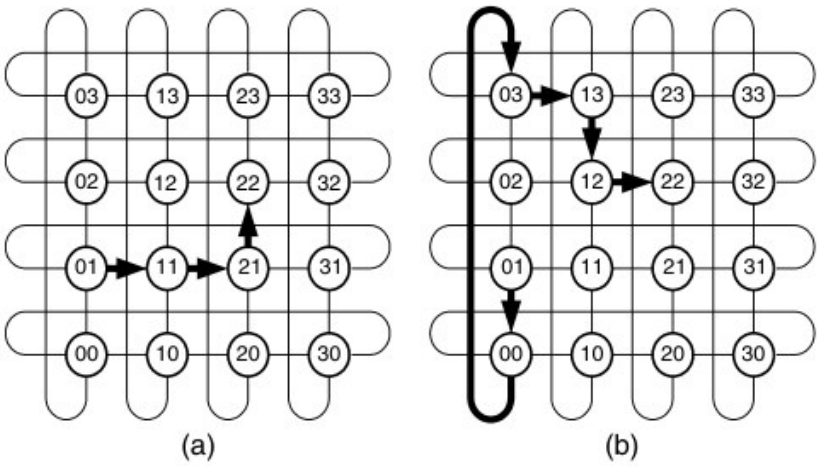


图 1.9 图 1.6 所示二维环面中从节点 01 到节点 22 的两种路由：(a) 维序最小路由，先沿 x 维移动，再沿 y 维移动；(b) 需要更多跳数的非最小路由。

用了从节点 11 到节点 21 的通道，使其负载加倍。通道的负载是终端节点试图通过它发送的平均带宽量。将负载归一化为终端可以向网络注入数据的最大速率，该通道的负载为 2。在这种情况下，更好的路由算法可以将归一化通道负载降低到 1。因为维序路由在这条单一通道上放置了两倍的必要负载，所以在这种流量模式下网络的最终带宽将只有其最大值的一半。更一般地说，所有在每个源-目的对之间选择单一固定路径的路由算法（称为确定性路由算法）尤其容易因负载不平衡而导致低带宽。这些以及其他路由算法设计问题将在第 8 章至第 10 章中更详细地描述。

1.3.3 流控

流控管理数据包在沿其路由前进时对资源的分配。大多数互连网络中的关键资源是通道和缓冲区。我们已经看到了通道在节点之间传输数据包的作用。缓冲区是节点内实现的存储，例如寄存器或存储器，允许数据包在节点暂时停留。继续我们的类比：拓扑决定了路线图，路由方法驾驶汽车，流控控制交通灯，决定汽车何时可以前进到下一段道路（通道）或何时必须驶入停车场（缓冲区）以让其他车辆通过。

为了实现拓扑和路由方法的性能潜力，流控策略必须避免可能导致通道空闲的资源冲突。例如，它不应阻塞一个可以使用空闲通道的数据包，因为它正在等待一个被阻塞在繁忙通道上的数据包所持有的缓冲区。这种情况类似于阻塞一辆想直行的汽车，因为它跟在一辆等待交通间隙左转的汽车后面。解决方案，在流控中以及在高速公路上，是添加一条（左转）车道来解耦资源依赖关系，使被阻塞的数据包或汽车能够在不等待的情况下前进。

好的流控策略是公平的并且避免死锁。不公平的流控策略可能导致数据包无限期等待，就像一辆汽车试图从繁忙的街道左转而没有信号灯一样。死锁是当一组数据包循环等待彼此释放资源，从而无限期阻塞时发生的情况——这种情况与我们的路线图

类比中的交通堵塞不无相似。

我们通常使用时间-空间图来描述流控方法，如图 1.10 所示。该图显示了（a）存储转发流控和（b）直通流控的时间-空间图。在两个图中，时间显示在水平轴上，空间显示在垂直轴上。时间以周期表示。空间通过列出用于发送数据包的通道来显示。每个数据包被分成五个固定大小的 flit。flit（flow-control digit，流控单元）是流控方法识别的最小信息单位。选择小的固定大小简化了路由器设计，而不会对长度不是 flit 大小倍数的数据包产生大的开销。单个数据包的每个 flit（由标记的方框表示）正在通过网络的四个通道发送。在图中，方框显示在特定 flit 使用通道带宽的周期内。如图 1.10 所示，流控技术的选择可以显著影响数据包通过网络时的延迟。

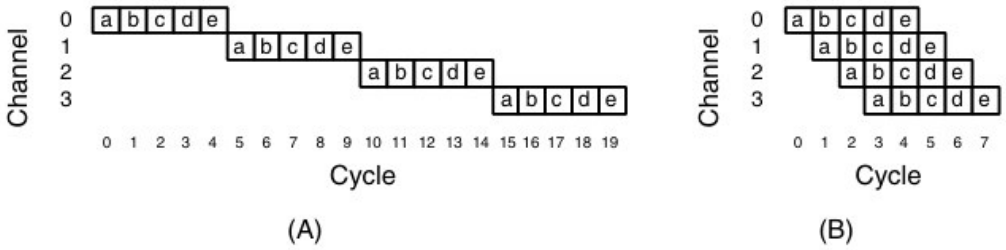


图 1.10 两种流控方式的时空图。(a) 存储转发：每一跳都要收到完整数据包后才能继续；(b) 直通传输：数据包的后续 flit 可以与前面的 flit 流水式传输。横轴为时钟周期（Cycle），纵轴为通道（Channel）。

第 12 章和第 13 章将更详细地讨论流量控制。与流量控制和路由相关的死锁、活锁、树饱和以及服务质量问题将在第 14 章和第 15 章中讨论。

1.3.4 路由器架构

图 1.11 展示了图 1.6 网络中 16 个节点之一的内部简化视图。四个输入通道各关联一个缓冲区，这些缓冲区暂存到达的 flit，直到它们能够被分配离开所需的资源。一旦确保一个 flit 在其路径上的下一个路由器（下游路由器）中有缓冲区空间，它就可以开始竞争交叉开关的访问权。交叉开关可以配置为将路由器的任何输入缓冲区连接到任何输出通道，但约束是每个输入最多连接到一个输出，每个输出最多连接到一个输入。解决所有对交叉开关和路由器其他共享资源的潜在请求的任务落在分配器上。要前进到下一个路由器，输入缓冲区中的一个 flit 必须在其路由的下一个节点上分配到缓冲区空间、路由的下一个通道上的带宽，并且必须赢得穿越交叉开关的分配。路由器架构将在第 16 章至第 21 章中详细讨论。

1.3.5 互连网络的性能

互连网络的性能主要通过延迟与提供流量的关系曲线来描述，如图 1.12 所示。该图显示了数据包的平均延迟（从数据包的第一位到达源终端到数据包的最后一位到达目的终端的时间）作为提供流量（网络中每个源终端产生的平均流量，单位为比特/秒）

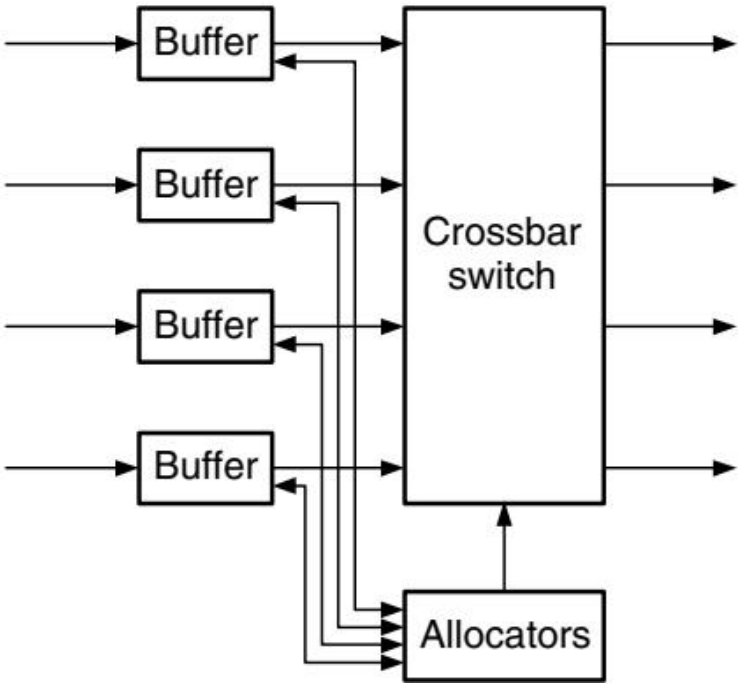


图 1.11 路由器的简化框图。通过输入通道到达的 flit 存储在关联每个输入的缓冲区中。一组分配器为待处理的 flit 分配下一个节点上的缓冲区和通道带宽。当一个 flit 被分配了所需的资源后，它通过交叉开关转发到输出通道。

的函数。要绘制特定的延迟与提供流量曲线，还必须指定流量模式（例如，随机流量）。

尽管延迟与提供流量曲线给出了互连网络最终性能的最准确视图，但它们没有简单的闭式表达式，通常通过离散事件模拟获得。为了在互连网络的早期设计阶段指导我们对权衡的理解，我们采用增量方法来研究网络性能，遵循我们对拓扑、路由和流量控制的探索。

零负载延迟给出了数据包通过网络的平均延迟的下界。零负载假设是数据包从不与其他数据包竞争网络资源。在此假设下，数据包的平均延迟是其序列化延迟加上其跳延迟。例如，考虑图 1.6 所示的环面网络，数据包长度 L = 512 比特，通道带宽 b = 16 Gbits/s，随机流量。在这种情况下，序列化延迟为 L/b = 32 ns。随机流量下可能的最低跳延迟出现在最小路由时，平均跳数为 $H_{min} = 2$。对于路由器延迟 $t_r = 10$ ns，最小跳延迟为 $H_{min}t_r = 20$ ns。这给出了数据包通过网络的平均延迟的下界为 $32 + 20 = 52$ ns，仅基于网络的拓扑、封装和流量模式。结合网络中实际使用的路由算法的平均跳数 H_{avg}，可以得到更紧的数据包延迟界，因为 $H_{avg} \geq H_{min}$。最后，网络采用的流量控制可以进一步降低性能，低于拓扑和路由给出的界。例如，如果我们的网络采用存储转发流量控制（图 1.10[a]），零负载延迟在原著中写为 $H_{\text{avg}}t_r \times L/b$，而不是 $H_{\text{avg}}t_r + L/b$。[9]实际的零负载延迟 T_0 结合了拓扑的约束以及实际性能、路由和流量控制。这些逐步更紧的延迟界在图 1.12 中显示为曲线的水平渐近线。

[9]校订注：原版第 20～21 页在此处使用乘号“×”，按所定义量纲直接相乘会得到时间的平方。这里忠实保留原书写法并指出量纲疑点；是否应理解为 $H_{\text{avg}}(t_r + L/b)$，仍需依据后续章节模型验证。

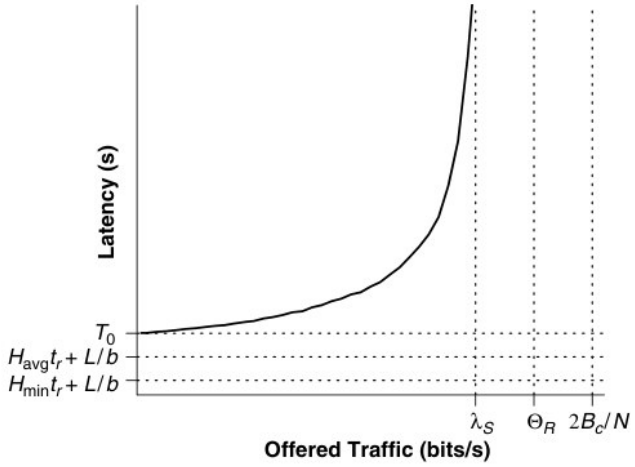


图 1.12 互连网络的延迟与提供流量关系。在低负载时，延迟趋近于零负载延迟 T_0；在饱和吞吐量 λ_s 处，延迟急剧增加。饱和吞吐量受拓扑上界 $2B_c/N$ 和路由上界 Θ_R 共同限制。

类似的方法给出了一组网络吞吐量的上界。虽然每个源向网络提供特定量的流量，但吞吐量或接受流量是流量（比特/秒）被递送到目的终端的速率。对于我们的随机流量示例，一半的流量必须穿过网络的对分。对分由 16 个通道组成，总带宽为 $B_c = 256$ Gbits/s。因此，每个节点的流量不能超过 $2B_c/N$ 或 32 Gbits/s。这个界假设流量在对分上完全平衡。因此，特定路由算法 R 不能超过这个界，并且考虑路由算法的吞吐量 Θ_R 在发生负载不平衡时实际上可能更低（$\Theta_R \leq 2B_c/N$）。最后，如果我们的流量控制由于资源依赖导致通道空闲，网络的饱和吞吐量 λ_s 可能显著小于 Θ_R 的界。这三个吞吐量界在图 1.12 中显示为垂直渐近线。

采用这种增量方法来研究性能——推导由拓扑、路由和流量控制引起的逐步更紧的界——使我们能够探索我们考虑的每个设计决策如何影响性能，而不会用不必要的细节使分析复杂化。例如，我们可以看到拓扑选择如何独立于路由和流量控制影响延迟。相比之下，试图一次性处理性能问题会难以看出任何一个设计选择的影响。

在建立了增量性能模型之后，我们将在第 23 章至第 25 章中全面考虑性能问题。在这些章节中，我们将讨论性能测量的一些微妙之处，介绍基于排队论和概率论的性能估计分析方法，讨论用于测量性能的仿真方法，并给出若干测量示例。

1.4 历史

互连网络有着跨越数十年的丰富历史。网络至少沿着三条并行的线索发展：电话交换网络、处理器间通信以及处理器-存储器互连。

电话交换网络的历史与电话本身一样悠久。早期的电话网络由机电式纵横制交换机或机电式步进制交换机组成。直到 20 世纪 80 年代，大多数本地电话交换机仍由机电继电器构成，尽管当时长途交换机已完全实现电子化和数字化。电话交换领域的关键进展包括 1953 年的非阻塞多级 Clos 网络 [37] 和 1962 年的 Beneš 网络 [17]。如今

许多大型电话交换机仍然基于 Clos 网络或类似 Clos 的网络构建。

最早的处理器间互连网络是连接在二维阵列中相邻处理器寄存器之间的连接。1962 年的 Solomon 机器 [172] 就是这类处理器阵列的一个例子。这些早期网络不执行路由。因此，处理器必须显式地为非相邻节点中继通信，导致性能不佳且编程复杂度高。到 20 世纪 80 年代中期，路由器芯片（如环面路由芯片 [56]）被开发出来，无需处理器干预即可通过中间节点转发消息。

多年来，处理器间互连网络经历了一系列拓扑结构的流行变迁——这主要是由封装和其他技术限制所推动的。早期的机器，如 Solomon[172]、Illiac[13] 和 MPP，由于其物理规整性而基于简单的二维网格或环面网络。从 20 世纪 70 年代末开始，二叉 n-立方体或超立方体网络因其低直径而流行起来。许多围绕超立方体网络设计的机器涌现出来，例如 Ametek S14、Cosmic Cube[163]、nCUBE 计算机 [134，140] 和 Intel iPSC 系列 [38，155]。在 20 世纪 80 年代中期，研究表明在实际封装限制下，低维网络优于超立方体 [2，46]，大多数机器又回归到二维或三维网格或环面网络。因此，过去十年中建造的大多数机器都回归到这些网络，包括 J-machine[138]、Cray T3D[95] 和 T3E[162]、Intel DELTA[117] 以及 Alpha 21364[131] 等。如今，路由器芯片的高引脚带宽相对于消息长度促使人们使用节点度更高的网络，如蝶形网络和 Clos 网络。我们可以预期在未来十年内会转向此类网络。

处理器-存储器互连网络出现于 20 世纪 60 年代末，当时并行处理器系统引入了对齐网络，允许任何处理器访问任何存储体而不会给其他处理器带来负担 [110]。最小的机器为此采用交叉开关，而较大的机器则采用蝶形（或等效）拓扑的网络，以舞厅式布局排列。这种主题的变体在整个 20 世纪 80 年代被用于许多共享内存并行处理器。

互连网络演进的这三条线索最近已经融合。自 20 世纪 90 年代初以来，处理器-存储器和处理器间互连网络的设计几乎没有差别。事实上，相同的路由器芯片已被用于两者。电话交换中的 Clos 和 Beneš 网络的变体也以胖树拓扑 [113] 的形式出现在多处理器网络中。

我们对历史的讨论集中在拓扑上，因为它是网络最明显的属性。当然，路由和流控方法与拓扑并行发展。早期的路由芯片采用简单的确定性路由以及电路交换或存储转发分组交换。后来的路由器采用了自适应路由，配合复杂的死锁避免方案和虚拟通道流控。

1.5 本书的组织结构

我们将在下一章开始，完整描述一个简单的互连网络，从拓扑一直到逻辑门，以便读者在深入细节之前对互连网络有一个“全景”视图。其余章节涵盖细节内容。它们被组织为五个主要部分：拓扑、路由、流控、路由器架构和性能。每个部分分为若干章，每部分的第一章涵盖基础知识，后续章节涵盖更复杂的主题。

第二章　一个简单的互连网络

在本章中，我们研究一个简单互连网络的架构和设计，以提供一个全局视图。我们将研究最简单的可能网络：采用丢弃流控的蝶形网络。尽管由此产生的网络成本高昂，但它强调了互连网络设计的许多关键方面。在后续章节中，我们将学习如何构建更高效和实用的网络。

2.1　网络规格与约束

与所有工程设计问题一样，网络设计始于一组描述我们希望构建什么的规格说明，以及一组限制潜在解决方案范围的约束条件。本章示例网络的规格总结在表 2.1 中。这些规格包括网络规模（64 端口）和每个端口所需的带宽。如表所示，峰值带宽和平均带宽相等，意味着输入以 0.25 Gbyte/s 的速率持续注入消息。预期流量为随机流量，即每个输入以等概率发送到每个输出，消息大小为 4 到 64 字节。此外，服务质量和可靠性规格允许丢弃分组。也就是说，并非每个分组都需要成功送达目的地。正如我们将看到的，丢弃分组的能力将简化我们的流控实现。当然，实际的规格集会更加详细和具体。例如，QoS 规格会指明可以丢弃的分组比例以及在何种情况下丢弃。然而，这组规格足以说明我们设计的许多要点。

我们示例网络设计的约束条件如表 2.2 所示。这些约束规定了每一级封装的容量和成本。我们的网络由芯片组成，这些芯片组装在电路板上，电路板又通过电缆连接。约束条件规定了在每个层级上可以穿过模块接口的信号数量[1]，以及每个模块的成本。对于电缆，约束条件还规定了在不降低电缆带宽的情况下可以传输的最长距离。[2]

2.2　拓扑

为简单起见，我们的示例网络采用蝶形拓扑。从单个输入端口的角度来看，蝶形网络看起来像一棵树。（见图 2.1。）树的每一层都包含交换节点，与终端节点不同，交换节点不发送或接收数据包，只转发数据包。此外，每条通道都是单向的，如箭头所示，从输入节点流向输出节点（图中从左到右）。然而，选择蝶形拓扑并不能完成拓扑设计的工作。我们还必须决定网络的加速比，确定蝶形网络的基数，并确定拓扑如何映射到封装层级上。

[1]注意，信号线的数量不一定等于引脚数量。例如，差分信号传输很常见，但每个信号需要两个引脚。

[2]对于给定类型的电缆，带宽与距离平方的乘积 Bd^2 为常数。因此，要让电缆的传输距离加倍，带宽就必须降为原来的四分之一，即 250 Mbits/s。

表 2.1 示例网络的设计规格

参数	取值
输入端口数	64
输出端口数	64
峰值带宽	0.25 Gbyte/s
平均带宽	0.25 Gbyte/s
消息延迟	100 ns
消息长度	4–64 字节
流量模式	随机
服务质量	允许丢包
可靠性	允许丢包

表 2.2 示例网络的设计约束

参数	取值
端口位宽	2 位
信号速率	1 GHz
每块芯片的信号线数	150
芯片成本	$200
芯片引脚带宽	1 Gbit/s
每块电路板的信号线数	750
电路板成本	$200
每根电缆的信号线数	80
电缆成本	$50
电缆长度	在 1 Gbit/s 速率下为 4 m

网络的加速比是网络总输入带宽与网络理想容量之比。容量定义为在给定流量模式下，假设完美的路由和流控，网络所能达到的最佳吞吐量。以加速比为 1 进行设计意味着输入的需求与网络交付流量的理想能力完全匹配。提供更高的加速比增加了设计的余量，并允许实现中的非理想因素。在某种意义上，加速比类似于土木工程师对结构安全系数的概念。例如，一座安全系数为 4 的建筑设计为能够承受比其规格大 4 倍的应力。

对于蝶形网络，将网络的每条通道的带宽设置为与单个输入端口相同，可得到加速比为 1。要理解这一点，考虑随机流量下对任何特定通道的需求——将每个输入通过该通道发送的流量比例相加，总是得到等于输入端口带宽的需求。在我们的简单网络中，这对应于将通道的带宽设计为 0.25 Gbyte/s。然而，基于我们在后续设计选择中将选择简单性而非效率的事实，我们选择加速比为 8。虽然这个加速比相当大（布鲁克林大桥的设计安全系数仅为 6!），但我们将在本书的进程中很快学会如何降低加速比需求。

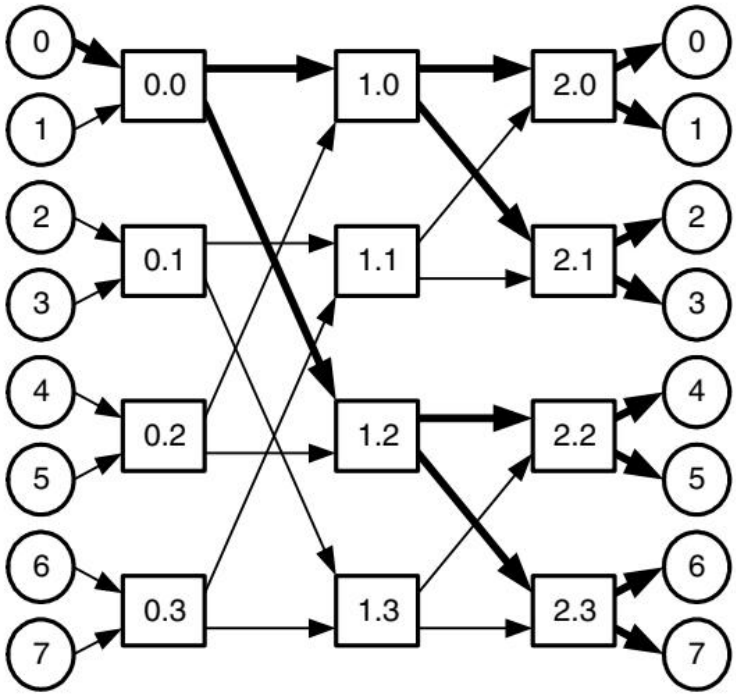


图 2.1 一个 8 节点蝶形网络。数据从左侧的输入节点（圆圈）经过三级交换节点（矩形）流向右侧的输出节点（圆圈）。交换节点标有它们的级数和地址。所有通道都是单向的，如箭头所示。

我们对加速比的选择以及封装约束决定了每个交换节点的输入和输出数量，称为蝶形网络的基数。例如，图 2.1 中的蝶形网络设计为基数为 2。每个交换节点实现在单个芯片上，因此通道总数（输入和输出）乘以通道宽度不得超过每芯片 150 个信号的限制。为了提供 8 的加速比，我们需要网络通道带宽为 $8 \times 0.25 = 2$ Gbytes/s，这需要 16 个信号，每个信号以 1 Gbit/s 运行。允许 2 个额外的开销信号，通道宽度为 18 个信号，我们只能在一个芯片上容纳 $150/18 \approx 8$ 条通道。因此，我们选择基数 4 的蝶形网络，每个交换节点有 4 个输入通道和 4 个输出通道，总共 8 条通道。

为了将每个输入端口连接到所有 64 个输出端口，我们的蝶形网络需要 $\log_4 64 = 3$ 级交换节点。因此，我们得到一个基数为 4、三级的蝶形网络，简称四元三级蝶形网络。该网络的完整拓扑如图 2.2 所示。虽然这个图乍看可能令人生畏，但它是我们在图 2.1 中介绍的较小蝶形网络的扩展。将输入 1 连接到 64 个输出中每一个的通道再次形成一棵树（以粗体显示），但现在树的度数为 4——我们网络的基数。

设计拓扑的最后一步是封装。我们已经通过每个芯片放置一个交换节点做出了一个封装决策。通过选择网络的基数以满足每芯片约束，我们知道交换节点在封装设计约束内。这些芯片必须安装在电路板上，并且为了最小化成本，我们希望尽可能多地将交换芯片安装在电路板上。然而，我们受到约束，不能超过进入或离开电路板的最大引脚数 750 个信号。3 这个约束由可以通过板边缘连接器布线的最大信号数驱动——连接器密度（信号/米）乘以连接器长度（米）。

图 2.2 中显示了交换节点在电路板之间的有效划分。8 块电路板中的每一块的边界由虚线框表示。如图所示，网络通过将第一级交换机放置在 4 块电路板上，每块板 4 个芯片来封装。接下来的 2 级封装在 4 块板上，每块板包含 8 个芯片，连接为 16 端口蝶形网络。我们通过观察每个板有 32 条通道（每条通道包含 18 个信号）进入和离开，来验证每个电路板的引脚约束得到满足。这给我们总共 $32 \times 18 = 576$ 个信号的引脚数，轻松符合我们 750 个信号的约束。细心的读者会注意到，我们可以在第一级板上放置 5 个路由器芯片（总共 40 条通道或 720 个信号），但这将效率不高，因为我们仍然需要 4 块第一级板。此外，我们不能在第二级板上放置 10 个路由器芯片，因为所需的 46 条通道（828 个信号）将超过我们板的引脚数。

最后，板与板之间的连接通过电缆实现，如图 2.3 所示。图中的每条粗灰线对应一根电缆，承载四条 18 位通道，从一块电路板连接到另一块电路板。8 块电路板通过 16 根这样的电缆连接在一起：从第一级的每块板到第二级和第三级的每块板各一根。由于 8 块电路板都安装在一个机箱内，这些电缆的长度都远在最大允许长度之内。

退一步看，我们可以看到拓扑中的交换节点如何将任何输入连接到任何输出。第一级交换机在承载其余级的 4 块电路板之间进行选择。第二级交换机在所选电路板上构成第三级的 4 个芯片中选择 1 个。最后，最后一级交换机选择所需的输出端口。当数据包通过网络进行路由时，我们利用这种分治结构。

2.3 路由

我们的简单蝶形网络采用目的地标签路由 4，其中目的地址的比特用于在网络每一级选择输出端口。在我们的 64 节点网络中，目的地址是 6 位。每一级交换机使用 2 个地址位来选择 4 个交换机输出中的 1 个，将数据包导向剩余节点的适当四分之一。例如，考虑将数据包从输入 12 路由到输出节点 $35 = 100011_2$。目的地址的最高 2 位（10）选择交换机 0.3 的第三个输出端口，将数据包带到交换机 1.11。然后，中间的 2 位（00）选择交换机 1.11 的第一个输出端口。从交换机 2.8，最低 2 位（11）选择最

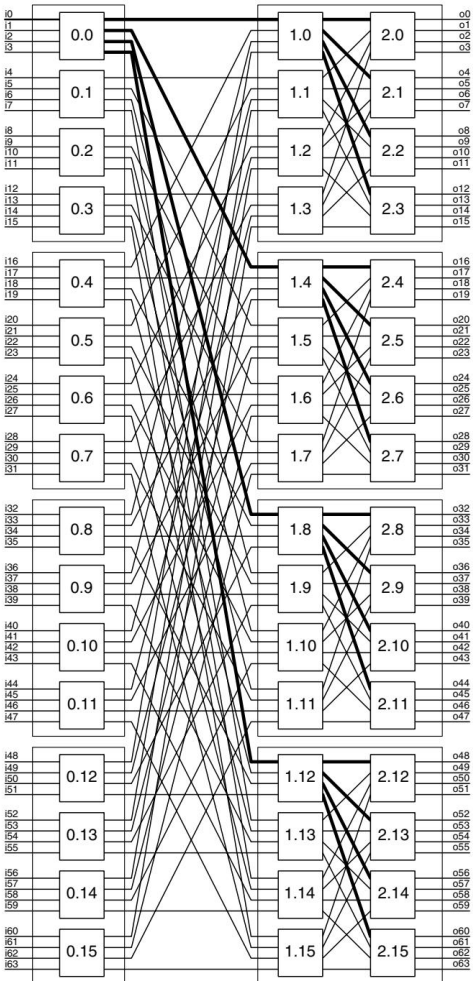


图 2.2 我们的基数为 4 的 3 级蝶形网络的拓扑和封装。通道是单向的，数据从左（输入）流向右（输出）。

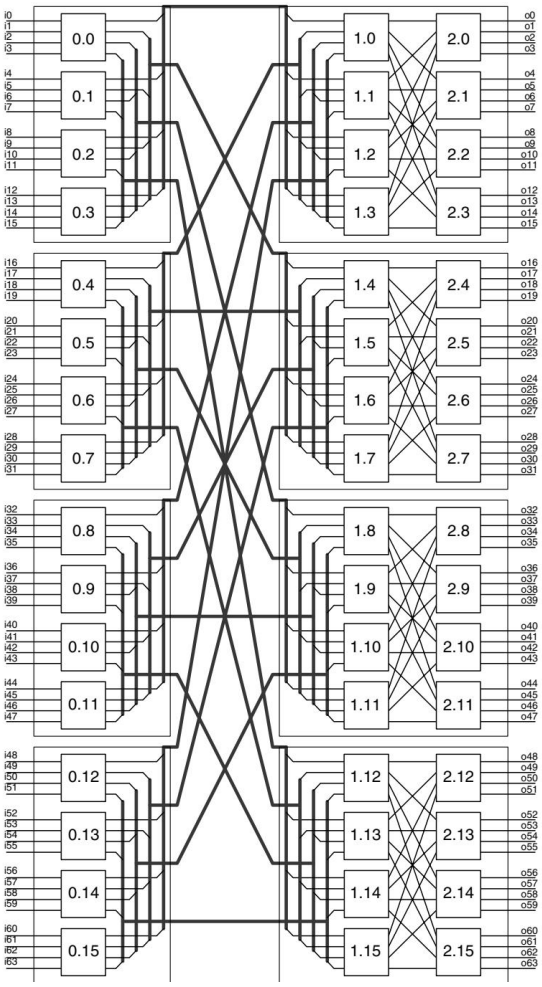


图 2.3 我们的基数为 4 的 3 级蝶形网络的板对板连接的布线。

后一个输出端口，将数据包传递到输出端口 35。

注意，这个交换机输出选择的序列完全独立于数据包的输入端口。例如，从节点 51 到输出 35 的路由遵循相同的选择序列：第一级的第三个交换机端口，第二级的第一个端口，第三级的最后一个端口。因此，我们可以通过仅随每个数据包存储目的地址来实现路由算法。

为了统一，我们所有的交换节点都操作目的地址字段的最高 2 位。然后，在数据包离开节点之前，地址字段左移两位，丢弃刚刚使用的位，并将下一个 2 位暴露在最高位位置。例如，在第一级路由到节点 35 之后，原始地址 100011_2 被移位为 001100_2。这种约定允许我们在网络的每个位置使用相同的交换节点，而无需特殊配置。它还便于将网络扩展到更大数量的节点，仅受地址字段大小的限制。

2.4 流控

我们网络的通道每个周期传输 16 位宽的物理传输单元（phit，physical digit）。然而，我们已经指定网络必须传递包含 32 到 512 位数据的整个数据包。因此，我们使用一个简单的协议，如图 2.4 所示，将 phit 组装成数据包。如图所示，每个数据包由一个头 phit 后跟零个或多个有效载荷 phit 组成。头 phit 表示新数据包的开始，并且还包含我们的路由算法使用的目的地址。有效载荷 phit 保存数据包的实际数据，分成 16 位块。给定数据包的 phit 必须是连续的，不中断。然而，在数据包之间可以在通道上传输任意数量的空 phit。为了区分头部单元和有效载荷单元，并表示数据包的结束，我们在每个通道上附加一个 2 位类型字段。该字段将每个 16 位字描述为头（H）、有效载荷（P）或空（N）字。数据包可以是任意长度，但总是由单个 H 字后跟零个或多个 P 字，再后跟零个或多个 N 字组成。使用正则表达式表示法，链路上流动的零个或多个数据包可以描述为 $(HP*N^*)^*$。

现在我们已经将 phit 组装成数据包，我们可以进入流控的主要任务：为数据包分配资源。为简单起见，我们的蝶形网络使用丢弃流控。如果数据包到达交换机时所需的输出端口正在使用，则数据包被丢弃。流控假设某种更高层的端到端错误控制协议最终会重传丢弃的数据包。丢弃数据包是最差的流控方法之一，因为它具有高丢包率，并且浪费了最终被丢弃的数据包的通道带宽。正如我们将在第 12 章中看到的，有更好的流控机制。然而，丢弃流控非常适合我们当前的目的，因为它在概念和实现上都极其简单。

2.5 路由器设计

我们蝶形网络中的每个交换节点都是一个路由器，能够在其输入上接收数据包，根据路由算法确定其目的地，然后将数据包转发到适当的输出。到目前为止我们做出的设计决策导致了一个非常简单的路由器。单个路由器的框图如图 2.5 所示。路由器

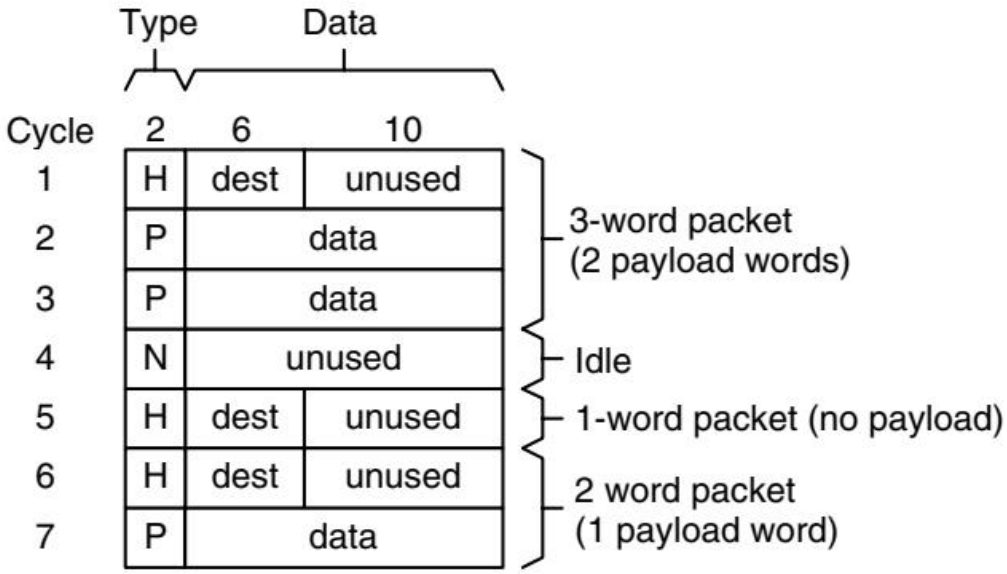


图 2.4 我们简单网络的数据包格式。时间（以周期为单位）显示在垂直方向，而通道的 18 个信号显示在水平方向。最左边的信号包含 phit 类型，而剩余的 16 个信号包含目的地址或数据，或者在空 phit 的情况下未使用。

表 2.3 示例网络的 phit 类型编码

类型	编码
H	11
P	10
N	00

的数据通路包括四个 18 位输入寄存器、四个 18 位 4：1 多路复用器、四个移位器（用于移位头 phit 的路由字段）和四个 18 位输出寄存器。数据通路包括 144 位寄存器和约 650 个门（等效 2 输入 NAND）。

每个时钟周期，phit 到达输入寄存器，并被路由到所有四个多路复用器。在每个多路复用器处，相关的分配器检查每个 phit 的类型以及每个头 phit 的下一跳字段，并相应地设置开关。来自所选输入的 phit 接下来被路由到一个移位器。在分配器的控制下，移位器将所有头 phit 左移两位，以丢弃当前路由字段并暴露下一个路由字段。有效载荷 phit 则原样通过。

路由器的控制完全驻留在与每个输出相关联的四个分配器中，这些分配器控制多路复用器和移位器。每个分配器，顾名思义，将输出端口分配给四个输入端口之一。其中一个分配器的原理图如图 2.6 所示。分配器由四个几乎相同的位切片组成，每个位切片分为三个部分：解码、仲裁和保持。在解码部分，每个输入 phit 的高四位被解码。每个解码器生成两个信号。如果输入 i 上的 phit 是头 phit，且路由字段的高两位与输出端口号匹配，则信号 $request_i$ 为真。该信号表明输入 phit 请求使用输出端口来路由以该头 phit 开始的数据包。解码器还生成信号 $payload_i$，如果输入 i 上的 phit 是有效载荷 phit，则该信号为真。分配器的保持逻辑使用该信号在数据包的持续时间内保持一个通道（只要有效载荷 phit 位于所选输入端口上）。

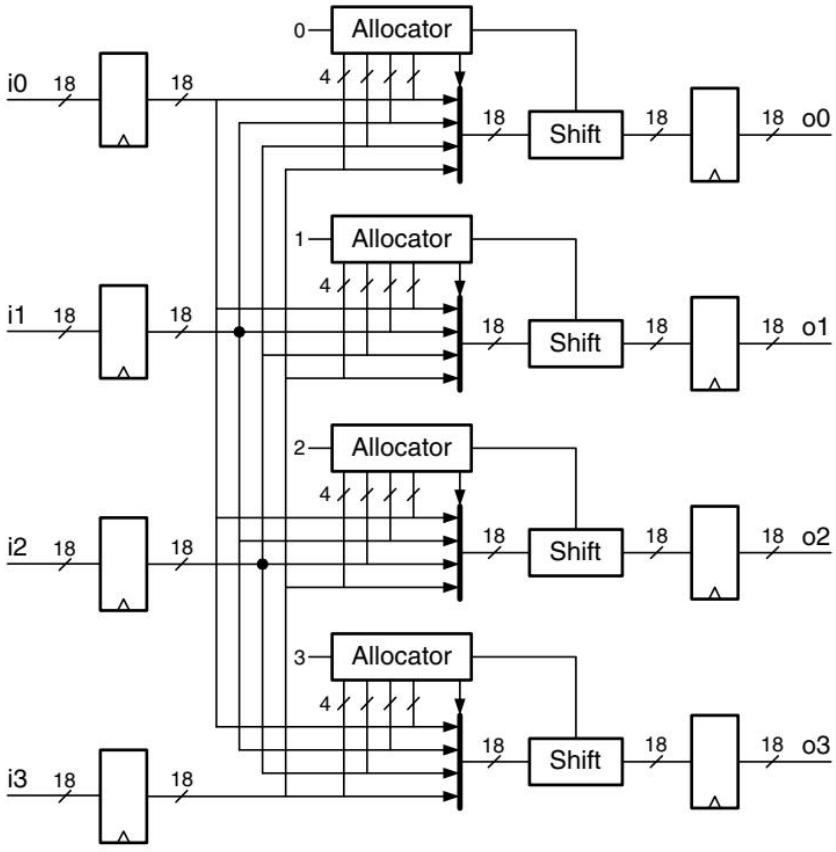


图 2.5 简单网络的路由器框图。

分配器的第二级是一个四输入固定优先级 5 仲裁器。仲裁器接受四个请求信号并

生成四个授权信号。如果输出端口可用（由信号 avail 指示），仲裁器将端口授予第一个（最上面的）发出请求的输入端口。断言授权信号会导致相应的选择信号被断言，从而选择多路复用器的相应输入传递到输出。如果任何授权信号被断言，则表明一个头正在通过多路复用器，并且移位信号被断言以使移位器移位路由字段。

分配器的最后一级在数据包的持续时间内保持输出端口到输入端口的分配。信号 $last_i$ 表示输入端口 i 在上一个周期被选中。如果该端口在本周期携带有效载荷 phit，则该有效载荷是同一数据包的一部分，并且通过断言信号 $hold_i$ 来保持通道。该信号导致 $select_i$ 被断言并且 avail 被清除，从而防止仲裁器将端口分配给新的头。

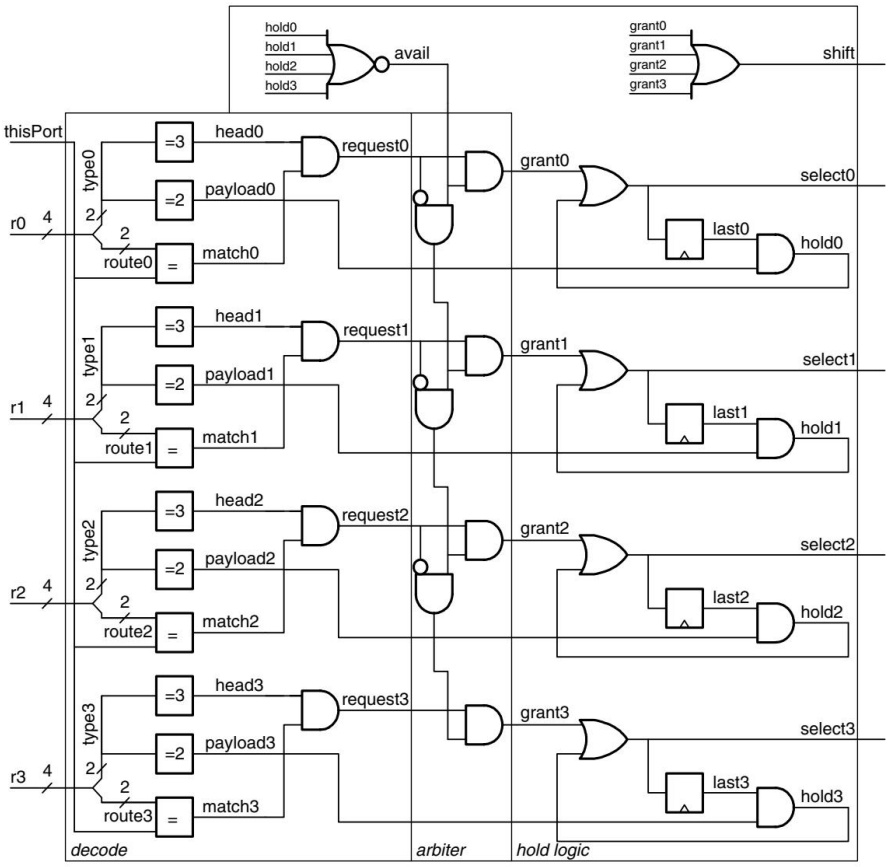


图 2.6 图 2.5 中路由器的分配器。

在本书中，我们将经常使用 Verilog 寄存器传输语言（RTL）模型来描述硬件。Verilog 模型是模块的文本描述，描述其输入、输出和内部功能。图 2.6 中分配器的 Verilog 描述在图 2.7 中给出。模块声明在注释之后开始，给出模块名 alloc，并声明其八个输入和输出。接下来的五行根据维度和宽度声明这些输入和输出。接下来，声明内部线和寄存器。真正的逻辑从十个 assign 语句开始。这些语句描述了分配器的组合逻辑，并且完全对应于图 2.6 的原理图。

```
// allocator: assigns output port to input port based on type
//    of input phit and current field of routing header
//    once assigned, holds a port for the duration of a packet
//    (as long as payload phits are on input).
```

```
//    uses fixed priority arbitration (r0 is highest).
module alloc(clk, thisPort, r0, r1, r2, r3, select, shift) ;
input clk ;    // chip clock
input [1:0] thisPort ;    // identifies this output port
input [3:0] r0,r1,r2,r3 ;    // top four bits of each input phit
output [3:0] select ;    // radial select to multiplexer
output shift ;    // directs shifter to discard upper two bits
wire [3:0] grant, select, head, payload, match, request, hold ;
wire [2:0] pass ;
reg [3:0] last ;
wire avail ;


assign head = {r3[3:2]==3,r2[3:2]==3,r1[3:2]==3,r0[3:2]==3} ;
assign payload = {r3[3:2]==2,r2[3:2]==2,r1[3:2]==2,r0[3:2]==2} ;
assign match = {r3[1:0]==thisPort,r2[1:0]==thisPort,
    r1[1:0]==thisPort,r0[1:0]==thisPort} ;
assign request = head&match ;
assign pass = {pass[1:0],avail}&~request[2:0] ;
assign grant = request&{pass,avail} ;
assign hold = last&payload ;
assign select = grant|hold ;
assign avail = ~(|hold) ;
assign shift = |grant ;

always @(posedge clk) last = select ;
endmodule
```

图 2.7　示例网络分配器的 Verilog 实现代码

最后，always 语句定义了保持状态 last [3：0] 的触发器。

除了作为一种方便的文本方式描述特定硬件之外，Verilog 还用作仿真输入语言和综合输入语言。因此，在以这种方式描述我们的硬件之后，我们可以对其进行仿真以验证其正确操作，然后综合出门级设计，以便在 ASIC 或 FPGA 上实现。供您参考，整个路由器的 Verilog 描述在图 2.8 中给出。为简洁起见，省略了多路复用器和移位器模块的描述。

```
// simple four-input four output router with dropping flow control
module simple_router(clk,i0,i1,i2,i3,o0,o1,o2,o3) ;
    input clk ;    // chip clock
    input [17:0] i0,i1,i2,i3;    // input phits
    output [17:0] o0,o1,o2,o3;    // output phits

reg [17:0] r0,r1,r2,r3;    // outputs of input registers
reg [17:0] o0,o1,o2,o3;    // output registers
wire [17:0] s0,s1,s2,s3;    // output of shifters
wire [17:0] m0,m1,m2,m3;    // output of multiplexers
wire [3:0] sel0, sel1, sel2, sel3;    // multiplexer control
wire shift0, shift1, shift2, shift3;    // shifter control

// the four allocators
```

```
alloc a0(clk, 2'b00, r0[17:14], r1[17:14], r2[17:14], r3[17:14], sel0, shift0);
alloc a1(clk, 2'b01, r0[17:14], r1[17:14], r2[17:14], r3[17:14], sel1, shift1);
alloc a2(clk, 2'b10, r0[17:14], r1[17:14], r2[17:14], r3[17:14], sel2, shift2);
alloc a3(clk, 2'b11, r0[17:14], r1[17:14], r2[17:14], r3[17:14], sel3, shift3);

// multiplexers
mux4_18 mx0(sel0, r0, r1, r2, r3, m0);
mux4_18 mx1(sel1, r0, r1, r2, r3, m1);
mux4_18 mx2(sel2, r0, r1, r2, r3, m2);
mux4_18 mx3(sel3, r0, r1, r2, r3, m3);

// shifters
shiftp sh0(shift0, m0, s0);
shiftp sh1(shift1, m1, s1);
shiftp sh2(shift2, m2, s2);
shiftp sh3(shift3, m3, s3);

// flip flops
always @(posedge clk)
begin
    r0=i0; r1=i1; r2=i2; r3=i3;
    o0=s0; o1=s1; o2=s2; o3=s3;
end

endmodule
```

图 2.8 示例网络路由器的 Verilog 实现代码（依据英文原版图 2.8 以等宽字体重排）

2.6 性能分析

我们通过三个指标来评判互连网络：成本、延迟和吞吐量。延迟和吞吐量都是性能指标：延迟是数据包穿越网络所需的时间，吞吐量是网络每秒可以从输入传输到输出的比特数。对于我们的示例网络（采用丢弃流控），这些性能指标在很大程度上受数据包被丢弃的概率影响。

我们的分析从网络的一个简单模型开始，针对丢弃的数据包被网络重新发送的情况。（见图 2.9。）首先，由于网络输入和输出之间的对称性以及随机流量模式，考虑来自网络单个输入的数据包就足够了。如图 2.9 所示，数据包以速率 λ 注入网络。λ 不是以比特每秒表示，而是归一化为 2 Gbytes/s 的通道带宽，因此 $\lambda = 1$ 对应于以通道允许的最大速率注入数据包。在数据包进入网络之前，它们与通过网络重新发送的数据包合并。这两个速率之和 p_0 是注入网络第一级的数据包总速率。在第一级中，可能发生一些数据包冲突，较小比例的数据包 p_1 将通过而不会被丢弃。速率差 $p_0 - p_1$ 表示已被丢弃的数据包。如果要重新发送丢弃的数据包，这些数据包会流回输入并重新注入。类似地，第二级和第三级的输出速率 p_2 和 p_3 将由于更多的冲突和丢弃的数据包而继续降低。

这些输出速率可以通过从网络输入端开始向输出端逐步计算来迭代得出。我们的蝶形网络中的单级由 4×4 交叉开关构成，并且根据对称性，开关每个输入端的输入包速率是相等的。因此，对于网络的第 $i+1$ 级，交叉开关每个端口的输入速率为 p_i。由于

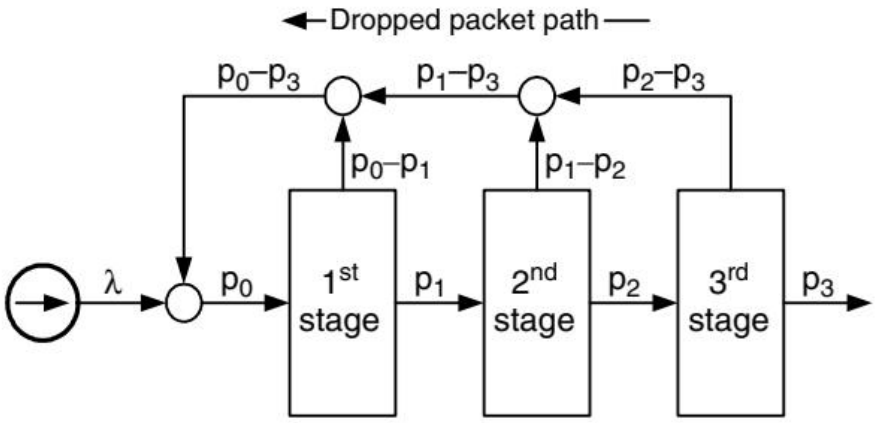


图 2.9 一个具有丢弃流控和丢弃包重注入的 3 级蝶形网络的简单分析模型。

速率已经归一化，它们也可以被解释为在任意特定周期内一个包到达输入端的概率。那么，一个包离开特定输出端 p_{i+1} 的概率等于 1 减去没有包想要该输出端的概率。由于业务模式是随机的，每个输入端将以概率 $p_i/4$ 想要一个输出端，因此没有输入端想要特定输出端的概率就是

$$\left(1-\frac{p_i}{4}\right)^4. \tag{2.1}$$

因此，第 $i+1$ 级的输出速率 p_{i+1} 为

$$p_{i+1}=1-\left(1-\frac{p_i}{4}\right)^4. \tag{2.2}$$

将方程 2.2 应用 n = 3 次，每次对应网络的一级，并暂时忽略重发的包（$p_0=\lambda$），我们计算出，当输入占空比为 $\lambda=0.125$（对应于加速比为 8）时，三个开关级输出端的占空比分别为 0.119、0.114 和 0.109。也就是说，在网络输入端提供 0.125 链路容量的业务量时，网络接受的业务量或吞吐量仅为 0.109。剩余的 0.016（占包的 12.6%）由于网络中的冲突而被丢弃。一旦丢弃的包被重新注入网络，就会出现一种有趣的动态。重发丢弃的包会增加网络的有效输入速率 p_0。这反过来又增加了丢弃包的数量，如此循环。如果这个反馈回路稳定下来，使得网络能够支持由此产生的输入速率（$p_0 \leq 1$），那么注入网络的业务量将等于输出的业务量（$p_3=\lambda$）。

图 2.10 绘制了我们示例网络的提供业务量与吞吐量之间的关系。两个轴都归一化为网络的理想容量。我们看到，在非常低的负载下，几乎所有业务量都能通过网络，吞吐量等于提供业务量。然而，随着提供业务量的增加，丢弃很快成为一个主要因素，如果包不被重发，网络的吞吐量将远低于提供业务量。最终吞吐量饱和，达到 43.2% 的渐近线。无论向网络提供多少业务量，无论包是否被重发，我们都无法实现超过信道容量 43.2% 的吞吐量。注意，我们可以以低至 2.5 的加速比运行这个网络，有效地将最大注入速率限制在 0.4。然而，最初选择 8 的加速比将在延迟方面带来好处，正如我们将看到的。此外，我们的网络实现不到其容量的一半，这是实践中几乎从不使用丢弃流控的主要原因。在第 12 章中，我们将看到如何构建流控机制，使我们能够

在不超过 90% 信道容量的情况下运行网络而不会饱和。

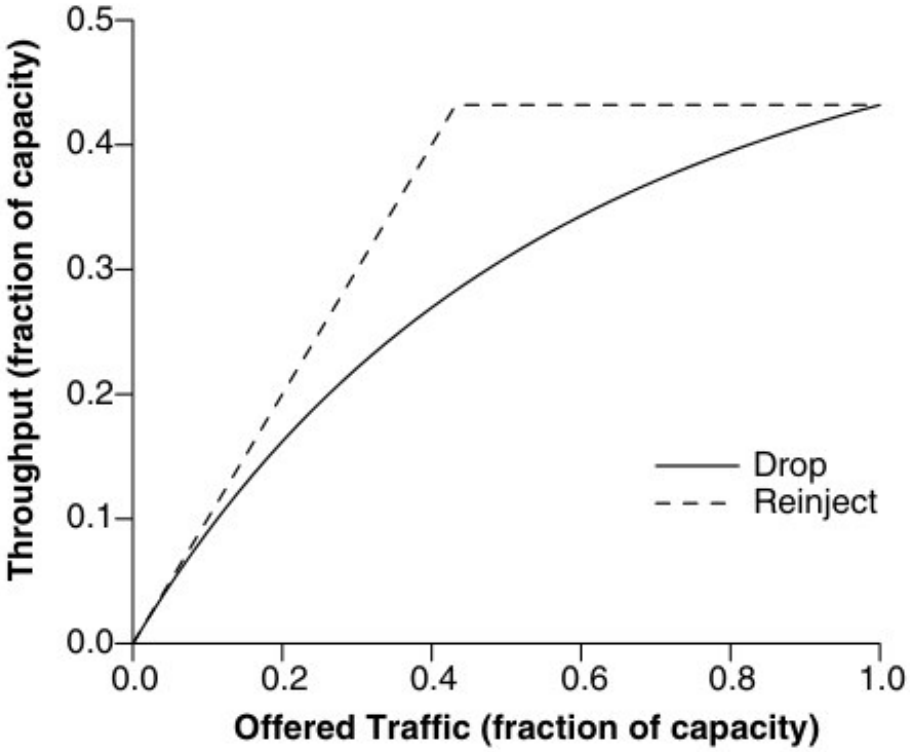


图 2.10 简单网络的吞吐量随提供负载变化：丢弃与重注入策略的对比

在整个关于吞吐量的讨论中，我们将提供业务量和吞吐量都表示为信道容量的分数。我们在全书中遵循这一惯例，因为它比用每秒比特数表示这些数字提供了更多的洞察力。例如，通过说明网络在 24% 容量时饱和，比说明它在 480 MB/s 时饱和，我们能更深入地了解流控方法的相对性能。在下面的讨论中，我们以类似的方式归一化延迟。2.6 性能分析一个包的延迟取决于它在成功穿越网络之前必须重传的次数。当网络中没有其他包时，一个包的头部在 6 个时钟周期内穿越网络。该包在三个级中的每一级通过两个触发器寄存器被时钟驱动。为清晰起见，我们将这个 6 周期延迟称为相对延迟 1.0，并将更高负载下的网络延迟相对于这个数字来表示。在本节中，我们只关注头部延迟。包的总延迟还包括一个等于包长度除以信道带宽的串行化项 L/b，它反映了包的尾部赶上头部所需的时间。

随着负载的增加，一部分包 p_D 被丢弃，必须重传。

$$P_D = \frac{p_0 - p_3}{p_0}$$

这些重传包中同样的比例会再次被丢弃，并必须第二次重传，如此反复。我们通过将这些情况下的延迟按情况发生的概率加权求和来计算包的平均延迟。假设（不现实地）源立即发现包被丢弃并在 6 个周期后重传，那么被丢弃 i 次的包的延迟为 $i + 1$。一个包恰好被丢弃 i 次的概率为 $P_D^i(1 - P_D)$。对加权延迟求和，我们计算出平均延迟为

$$T = \sum_{i=0}^{\infty}(i+1)P_D^i(1-P_D) = \frac{1}{1-P_D} = \frac{p_0}{p_3}. \tag{2.3}$$

这个延迟作为提供业务量的函数绘制在图 2.11 中。当网络无负载时，延迟从 1 开始（6 个时钟周期）。随着吞吐量的增加，一些包被丢弃，平均延迟增加，在吞吐量约为 0.39 时翻倍。最后，网络在吞吐量为 0.43 时达到饱和。此时，在任何延迟下都无法实现更大的吞吐量。

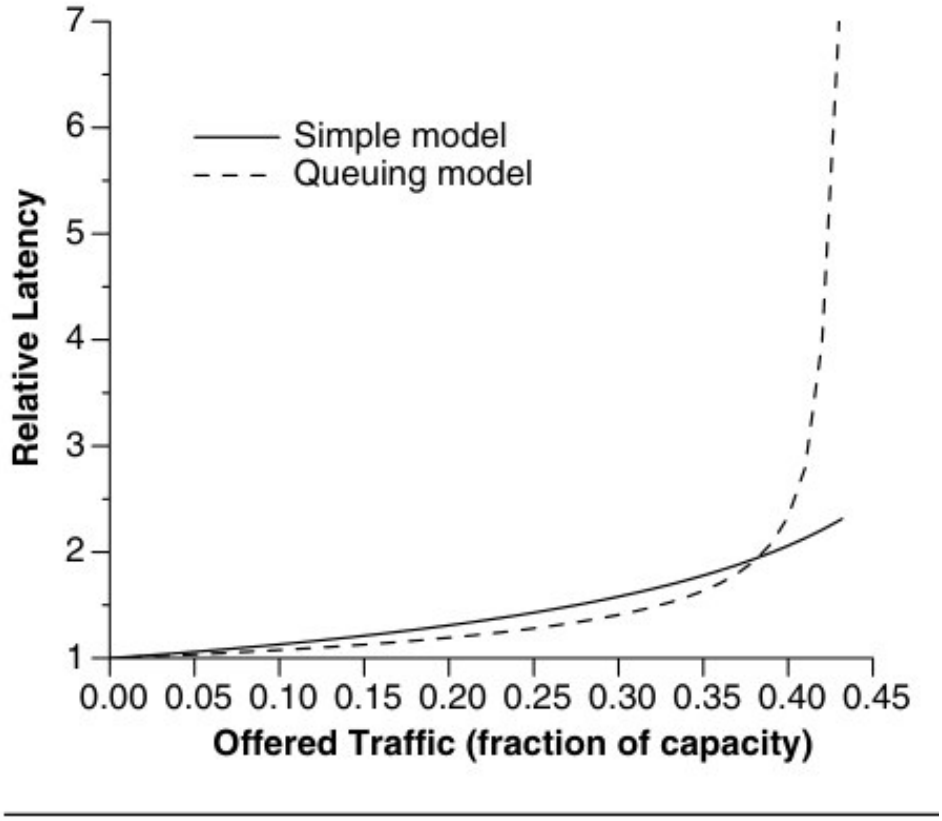


图 2.11 简单网络的相对延迟与提供负载关系：简单模型与排队模型

对于适度的负载，方程 2.3 给出了一个合理的延迟模型，但当网络接近饱和时，包可以立即重发的假设就值得怀疑了。这是因为重发的包和新注入的包同时被引入网络的机会越来越大。在实际实现中，一个包必须等待另一个包先行，从而产生额外的排队延迟。图 2.11 还显示了另一条包含排队时间模型的延迟曲线。这条曲线的形状——随着吞吐量接近饱和，延迟增长到无穷大——在互连网络中更为典型。两条曲线都与练习 2.9 中的模拟进行了比较。

需要牢记的是，公式 2.3 和图 2.11 给出的是数据包的平均延迟。对于许多应用而言，我们不仅关心平均延迟，还关心延迟的概率分布。特别是，我们可能关注最坏情况下的延迟或延迟的变化（有时称为抖动）。例如，在视频回放系统中，播放前用于缓存数据包的缓冲区大小由抖动决定，而非平均延迟。对于我们的示例网络，数据包以相对延迟 i 被接收的概率由下式给出：

$$P(T=i)=P_D^{(i-1)}\left(1-P_D\right).$$

这种指数分布的延迟导致最大延迟无限大，因而抖动也无限大。更现实地，我们可以用一定比例（例如 99%）的数据包所达到的延迟界限来表达抖动。

上述性能指标均针对均匀随机流量。对于蝶形网络，这是最佳情况。正如我们将看到的，对于某些流量模式，如位反转 6，网络的性能远差于此处描述的情况。蝶形网络对不良流量模式的敏感性很大程度上归因于从网络的每个输入到每个输出只有一条路径。我们将看到，具有路径多样性的网络在困难负载下表现要好得多。

2.7 练习题

2.1 简单网络的成本。使用表 2.1 和表 2.2 中的数据计算我们简单网络的成本。

2.2 加入功率约束。将每块板上的芯片数量限制为六块，以确保能够为这些芯片提供足够的功率，并且其热量能够适当散发。在原有约束集的基础上，提出一种满足此新约束的封装方案。这种封装的成本是多少?

2.3 公平分配器。修改图 2.7 中给出的分配器的 Verilog 代码，以实现一种更公平的仲裁类型。通过仿真验证你的新分配器，并描述你的设计。

2.4 增加路由器的度数。如果将简单路由器扩展为 5×5 而非 4×4 的交换机，则可用于实现二维网格或环面网络。描述如何扩展路由器和数据包格式以增加额外的输入和输出端口。

2.5 减少同一数据包的多次丢弃。修改 Verilog 代码，使重传的数据包具有优先权。也就是说，如果在蝶形网络中同一交换机上有两个头微片请求同一输出，分配器应始终优先将资源分配给重传的数据包。在微片头中添加一个优先级字段以包含此信息，并假设在数据包重新注入时已适当设置。

2.6 多次丢弃与平均延迟。引入一种减少数据包多次丢弃的方案（如练习题 2.5 中的方案）是否会降低平均数据包延迟？解释原因。

2.7 更大蝶形网络对丢弃的影响。如果我们向示例蝶形网络添加更多级（并相应增加节点数量），丢弃数据包的比例会增加吗? 如果改为增加交换机的度数，丢弃比例将如何变化？仅考虑丢弃概率，在添加节点时，扩展交换机的度数还是增加级数更有效?

2.8 现实的丢弃延迟。在实际网络中，丢弃的数据包在重传之前通常有显著的延迟。首先，必须留出时间让确认到达源端，然后允许超时以考虑确认中的延迟。修改公式 2.3 以反映此延迟。

2.9 仿真。编写一个简单的计算机程序来模拟我们的示例网络。使用该程序，通过实验确定网络延迟作为提供流量的函数。你的结果与公式 2.3 和图 2.11 的分析结果相比如何？还与排队模型给出的相对延迟进行比较

$$T=\frac{T_0}{2}+\frac{p_0}{p_3}\left(\frac{T_0}{2}+\frac{p_0}{2(1-p_0)}\right)$$

其中 T_0 是零负载延迟——即从未被网络丢弃的数据包的延迟。
2.10 仿真。向练习题 2.9 的模拟器添加超时机制，并将结果与练习题 2.8 中开发的模型进行比较。评论任何主要差异。

注释

- 3. 在实际系统中，电路板上的信号密度也会受到限制。
- 5. 在实践中，人们绝不会在这样的应用中使用固定优先级仲裁器，因为它会导致路由器极其不公平，在网络中使用时可能导致活锁或饥饿问题。在第 18 章和第 19 章中，我们将研究更好的仲裁和分配方法。
- 6. 在位反转流量模式中，地址为 $(b_{n-1}, b_{n-2}, \ldots, b_0)$ 的节点向地址为 $(b_0, b_1, \ldots, b_{n-1})$ 的节点发送数据包。

第三章　拓扑基础

网络拓扑是指互连网络中通道和节点的静态排列——即数据包传输的道路。选择网络拓扑是设计网络的第一步，因为路由策略和流控方法在很大程度上取决于拓扑。在选择路由并调度该路由的遍历之前，需要一张路线图。如第 2 章中的示例所示，拓扑不仅指定网络类型（例如，蝶形），还指定细节，例如交换机的基数、级数以及每个通道的宽度和比特率。

选择良好的拓扑主要是将网络需求与可用封装技术相匹配的工作。一方面，设计由端口数量、每个端口的带宽和占空比驱动；另一方面，由每芯片和每板可用的引脚、线密度、可用信号速率以及电缆的长度要求驱动。

我们根据成本和性能选择拓扑。成本由实现网络所需的芯片数量和复杂性，以及这些芯片之间在板上或通过电缆的互连密度和长度决定。性能有两个组成部分：带宽和延迟。这两个指标都由拓扑以外的因素决定——例如，流控、路由策略和流量模式。为了仅评估拓扑，我们开发了诸如对分带宽、通道负载和路径延迟等度量，以反映拓扑对性能的影响。

网络设计者常犯的一个错误是试图让网络拓扑与当前问题的数据通信模式相匹配。表面上看，这似乎是个好主意。毕竟，如果一台机器执行的是具有树状通信模式的分治算法，那么树形网络难道不是处理这种模式的最佳选择吗？答案通常是否定的。由于种种原因，专用网络通常并非良策。由于问题中的动态负载不均衡，或者问题规模与机器规模不匹配，这类网络上的负载通常平衡得很差。如果为了平衡负载而重新定位数据和线程，问题与网络之间的匹配就会丧失。针对特定问题的网络往往无法很好地映射到可用的封装技术，需要长线或高节点度。最后，这类网络缺乏灵活性。如果算法改变为使用不同的通信模式，网络无法轻易改变。使用一个良好的通用网络几乎总是比设计一个拓扑与问题匹配的网络更好。

图 3.1 展示了三个示例拓扑。图 3.1（a）显示了一个每维三个节点的二维环面，即一个 3 元 2 立方体。立方体网络中的每个节点既是终端也是交换节点。图 3.1（b）显示了一个三级基数为二的蝶形网络，即一个 2 元 3 飞。蝶形网络明确区分了终端节点和交换节点，两端是仅作为终端的节点，中间是仅作为交换的节点（矩形）。图 3.1（c）显示了一个不规则网络。

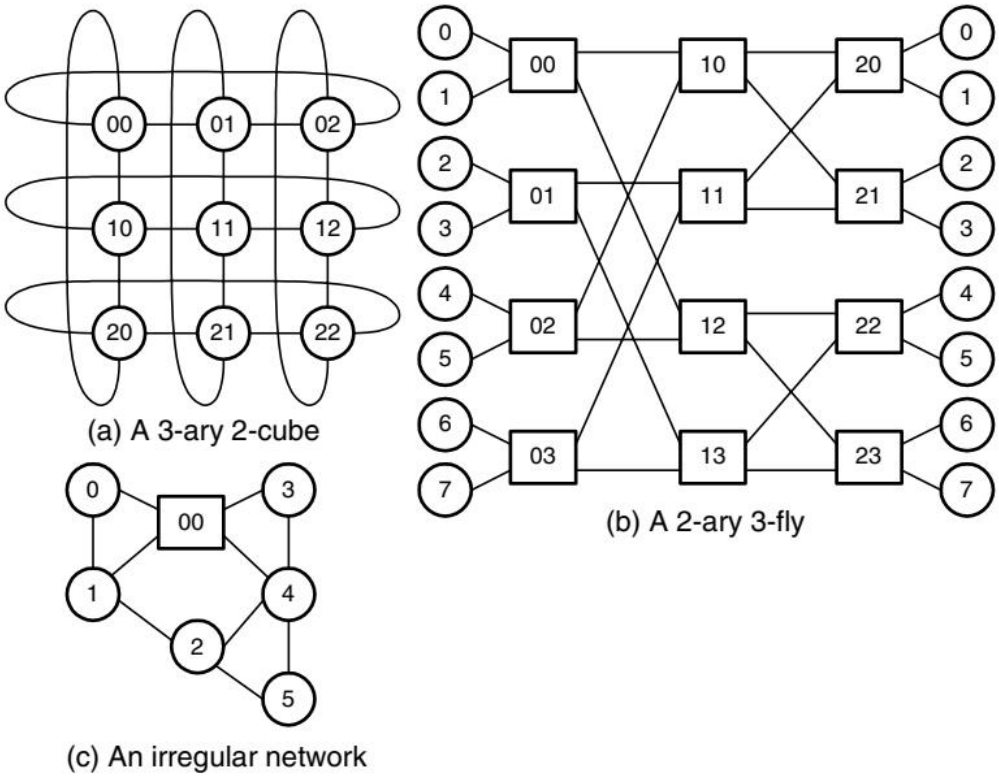


图 3.1 示例网络拓扑：（a）一个 3 元 2 立方体，（b）一个 2 元 3 飞，以及（c）一个不规则网络。

3.1 术语

3.1.1 通道与节点

互连网络的拓扑由一组节点 N^* 和连接这些节点的一组通道 C 指定。消息在一组终端节点 N 中产生和终止，其中 $N \subseteq N^*$。在所有节点都是终端的网络中，我们简单地将节点集合称为 N。每条通道 $c = (x, y) \in C$ 连接一个源节点 x 到一个目的节点 y，其中 $x, y \in N^*$。我们将通道 c 的源节点记为 s_c，目的节点记为 d_c。注意，图 3.1 中的每条边表示一对通道，每个方向一条。这种拓扑定义等价于一个有向图，毫不奇怪，用于描述拓扑的许多术语大量借鉴了图论。为方便表示，我们经常将网络中的节点数简写为 N^* 而不是 $|N^*|$，通道数也类似。

通道 $c = (x, y)$ 的特征包括：宽度 w_c 或 w_{xy}，即它包含的并行信号数量；频率 f_c 或 f_{xy}，即每个信号上传输比特的速率；以及延迟 t_c 或 t_{xy}，即一个比特从 x 传输到 y 所需的时间。对于大多数通道，延迟与通道的物理长度直接相关，$l_c = vt_c$，其中 v 是传播速度。通道 c 的带宽为 $b_c = w_c f_c$。在所有通道带宽相同的常见情况下，我们省略下标，将网络通道带宽称为 b。

每个交换节点 x 有一个通道集合 $C_x = C_{Ix} \cup C_{Ox}$。其中 $C_{Ix} = \{c \in C | d_c = x\}$ 是输入通道集合，$C_{Ox} = \{c \in C | s_c = x\}$ 是输出通道集合。x 的度为 $\delta_x = |C_x|$，即入度 $\delta_{Ix} = |C_{Ix}|$ 和出度 $\delta_{Ox} = |C_{Ox}|$ 之和。当所有 $x \in N^*$ 的度相同时，我们省略下标，用 δ 表示度。

3.1.2 直接网络与间接网络

网络节点可以是作为数据包源和汇的终端节点，可以是将数据包从输入端口转发到输出端口的交换节点，或者两者兼具。在直接网络中，如图 3.1（a）的环面，网络中的每个节点既是终端又是交换机。另一方面，在间接网络中，如图 3.1（b）的蝶形网络，一个节点要么是终端（圆形节点），要么是交换机（矩形节点）。它不能同时承担两种功能。在直接网络中，数据包直接在终端节点之间转发，而在间接网络中，数据包通过专用交换节点间接转发。有些网络，如图 3.1（c）的随机网络，既不是直接网络也不是间接网络。每个直接网络都可以通过将每个节点拆分为独立的终端节点和交换节点，重绘为间接网络，如图 3.2 所示。对于这类网络，直接网络与间接网络之间的区别在很大程度上只是学术上的。

直接网络的一个潜在优势是，终端的资源（通常包括一台计算机）可供每个交换机使用。在一些早期网络中，交换功能由运行在终端 CPU 上的软件实现，缓冲则使用终端计算机的内存 [163, 192]。然而，软件交换既非常慢又占用终端资源。因此，如今很少使用。

3.1 术语

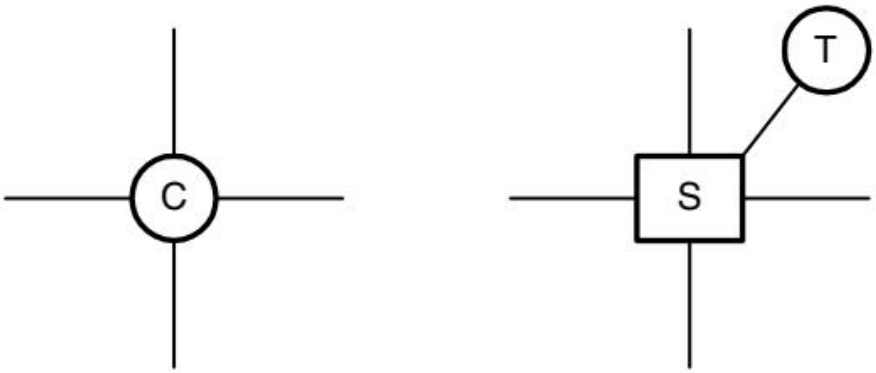


图 3.2 组合节点由一个终端节点和一个交换节点组成。

3.1.3 割与二分

网络的一个割，$C(N_1, N_2)$，是一组通道，它将所有节点的集合 N^* 划分为两个不相交的集合 N_1 和 N_2。$C(N_1, N_2)$ 中的每个元素都是一条源在 N_1 中、目的在 N_2 中的通道，或者相反。割中的通道数为 $|C(N_1, N_2)|$，割的总带宽为

$$B(N_1, N_2) = \sum_{c \in C(N_1, N_2)} b_c.$$

网络的二分是一个将整个网络几乎平分为两半的割，使得 $|N_2| \leq |N_1| \leq |N_2| + 1$，并且也将终端节点几乎平分为两半，使得 $|N_2 \cap N| \leq |N_1 \cap N| \leq |N_2 \cap N| + 1$。网络的通道二分，$B_C$，是网络所有二分中的最小通道数。

$$B_C = \min_{\text{bisections}} |C(N_1, N_2)|$$

网络的二分带宽 B_B 是网络所有二分中的最小带宽。

$$B_B = \min_{\text{bisections}} B(N_1, N_2)$$

对于通道带宽统一为 b 的网络，$B_B = bB_C$。在本章后文（第 3.4 节）建立的成本模型中，我们使用网络的二分带宽来估计实现该网络所需的全局布线量。

3.1.4 路径

网络中的路径是一个有序的通道集合 $P = \{c_1, c_2, \ldots, c_n\}$，其中对于 $i = 1 \ldots (n-1)$，有 $d_{c_i} = s_{c_{i+1}}$。路径也称为路由。路径的源点是 $s_P = s_{c_1}$。类似地，路径的目的地是 $d_P = d_{c_n}$。路径的长度或跳数是 $|P|$。如果对于特定的网络及其路由函数，在所有源-目的地对之间都至少存在一条路径，则称该网络是连通的。
从节点 x 到节点 y 的最小路径是连接这两个节点的跳数最少的路径。从节点 x 到节点 y 的所有最小路径的集合记为 R_{xy}。$H(x, y)$ 是 x 和 y 之间最小路径的跳数。网络的直径 H_{max} 是网络中所有终端节点对之间的最大最小跳数。

$$H_{\max} = \max_{x,y \in N} H(x, y)$$

对于由出度为 δ_O 的交换机组成的具有 N 个终端的全连接网络，H_{max} 的上界为

$$H_{\max} \geq \log_{\delta_O} N. \tag{3.1}$$

或者对于对称交换机，其中 $\delta_I = \delta_O = \delta/2$，

$$H_{\max} \geq \log_{\delta/2} N.$$

每个终端在一跳后最多可到达 δ_O 个其他终端，两跳后最多可到达 δ_O^2 个，H 跳后最多可到达 δ_O^H 个。如果我们设 $\delta_O^H = N$ 并求解 H，就得到公式 3.1，它给出了网络直径的下界。对于该下界是紧的网络，例如蝶形网络，没有路径多样性。所有决策都用于选择目的节点，没有剩余决策用于在替代路径之间进行选择。
网络的平均最小跳数 H_{min} 定义为所有源和目的地之间的平均跳数。

$$H_{\min} = \frac{1}{N^2} \sum_{x,y \in N} H(x, y)$$

虽然 H_{min} 表示可能的最小平均跳数，但具体实现可能会选择纳入一些非最小路径。在这种情况下，实际平均跳数 H_{avg} 是基于网络使用的路径（而不仅仅是最小路径）定义的，并且 $H_{avg} \geq H_{min}$。
路径的物理距离为

$$D(P) = \sum_{c \in P} l_c$$

路径的延迟为 $t(P) = D(P)/v$。节点对之间的距离和延迟以及网络的平均和最大距离和延迟的定义方式与跳数相同。

3.1.5 对称性

拓扑的对称性在负载均衡和路由中起着重要作用，我们将在后续章节中讨论。如果存在一个自同构将任意节点 a 映射到另一个节点 b，则网络是顶点对称的。通俗地说，在顶点对称网络中，从所有节点的视角来看拓扑是相同的。这可以简化路由，因为所有节点共享相同的网络路线图，因此可以使用相同的方向路由到相同的相对位置。
在边对称网络中，存在一个自同构将任意通道 a 映射到另一个通道 b。边对称可以改善网络通道间的负载均衡，因为没有理由偏爱某一个通道。

3.2 流量模式

在介绍拓扑的性能指标之前，考虑互连网络中消息的空间分布是有用的。我们用流量矩阵 Λ 表示这些消息分布，其中每个矩阵元素 $\lambda_{s,d}$ 给出从节点 s 发送到节点 d 的流量比例。表 3.1 列出了一些用于评估互连网络的常见静态流量模式。历史上，其中一些模式基于特定应用中出现的通信模式。例如，矩阵转置或角转操作引发转置模式，而快速傅里叶变换（FFT）或排序应用可能导致混洗置换 [175]，流体动力学模拟通常呈现邻居模式。流量的时间方面也对网络性能有重要影响，将在第 24.2 节讨论。

表 3.1 典型网络流量模式

名称	流量模式
随机	$\lambda_{sd} = 1/N$
置换	$d = \pi(s)$
位置换	$d_i = s_{f(i)} \oplus g(i)$
按位取反	$d_i = \neg s_i$
位反序	$d_i = s_{b-i-1}$
位循环移位	$d_i = s_{i+1} \mod b$
混洗	$d_i = s_{i-1} \mod b$
转置	$d_i = s_{i+b/2} \mod b$
数字置换	$d_x = f(s_{g(x)})$
龙卷风	$d_x = s_x + (\lceil k/2 \rceil - 1) \mod k$
邻居	$d_x = s_x + 1 \mod k$

随机流量，其中每个源等可能地发送到每个目的地，是网络评估中最常用的流量模式。随机流量非常良性，因为通过使流量均匀分布，即使对于通常负载均衡很差的拓扑和

路由算法，也能平衡负载。一些非常糟糕的拓扑和路由算法在仅用随机流量评估时看起来非常好。

为了对拓扑或路由算法施加压力，我们通常使用置换流量，其中每个源 s 将其所有流量发送到单个目的地 $d = \pi(s)$。置换的流量矩阵 Λ 是一个置换矩阵 Π，其中每一行和每一列包含一个非零条目，所有其他条目为零。因为置换将负载集中在各个源-目的地对上，所以它们对拓扑和路由算法的负载均衡施加压力。

位置换是置换的一个子集，其中目的地地址通过置换和选择性地取反源地址的位来计算。例如，如果四位源地址是 $\{s_3, s_2, s_1, s_0\}$，则位反转流量模式的目的地是 $\{s_0, s_1, s_2, s_3\}$，位补流量模式的目的地是 $\{\neg s_3, \neg s_2, \neg s_1, \neg s_0\}$，对于混洗，目的地是 $\{s_2, s_1, s_0, s_3\}$。

数字置换是一类类似的置换子集，其中目的地址的各位数字由源地址的各位数字计算得出。此类置换仅适用于终端地址可表示为 n 位、基数为 k 的数字的网络，例如 k 元 n 立方（环面）网络（第 5 章）和 k 元 n 飞（蝶形）网络（第 4 章）。龙卷风模式是针对环面拓扑设计的一种对抗模式，而邻居流量则衡量拓扑利用局部性的能力。

3.3 性能

我们根据成本和性能为网络选择拓扑。在本节中，我们讨论三个关键性能指标：吞吐量、延迟和路径多样性。这些指标将在 3.4 节中再次讨论，届时它们将与网络的实现成本相关联。

3.3.1 吞吐量与最大通道负载

网络的吞吐量是网络每个输入端口接受的数据速率，以比特每秒为单位。吞吐量是整个网络的属性，既取决于路由和流控（如第 2 章所示），也取决于拓扑。然而，我们可以通过测量在完美流控和路由下网络所能承载的吞吐量来确定拓扑的理想吞吐量。这是假设路由在网络中的替代路径上完美均衡负载，并且流控在瓶颈通道上不留空闲周期时所能达到的吞吐量。为简单起见，本节我们仅给出所有通道带宽均为 b 的网络的吞吐量和负载方程。这一限制将在练习 3.5 中解除。

当网络中的某个通道饱和时，即达到最大吞吐量。如果没有通道饱和，网络可以承载更多流量，因此未运行在最大吞吐量下。因此，要计算吞吐量，我们必须考虑通道负载。我们定义通道 c 上的负载 γ_c 为通道 c 所需的带宽与输入端口带宽之比。等价地，该比值是根据给定流量模式，当每个输入注入一个单位流量时必须通过通道 c 的流量。由于是比值，通道负载是无量纲量。除非另有说明，我们考虑均匀流量下的通道负载。

在特定流量模式下，承载最大流量比例的通道决定了拓扑的最大通道负载 γ_{max}，$\gamma_{\max} = \max_{c \in C} \gamma_c$。当提供的流量达到网络吞吐量时，该瓶颈通道上的负载将等于通道带宽 b。任何具有指定模式的额外流量都会使该通道过载。因此，我们将拓扑的理想吞吐量 Θ_{ideal} 定义为使瓶颈通道饱和的输入带宽

$$\Theta_{\text{ideal}} = \frac{b}{\gamma_{\max}} \tag{3.2}$$

其中 b 的单位为比特每秒，γ_{max} 无量纲。

对于任意拓扑和任意流量模式的一般情况，计算 γ_{max} 需要求解如下所述的多商品流问题。然而，对于均匀流量，我们可以用少得多的努力计算出 γ_{max} 的一些上下界。网络对分通道上的负载给出了 γ_{max} 的下界，进而给出了吞吐量的上界。对于均匀流量，我们平均知道一半的流量，即 N/2 个数据包，必须通过对分通道 B_C。当这些数据包均匀分布在对分通道上时，达到最佳吞吐量。因此，每个对分通道上的负载 γ_B 至少为

$$\gamma_{\max} \geq \gamma_{\text{B}} = \frac{N}{2B_C}. \tag{3.3}$$

结合方程 3.2 和 3.3，我们得到理想吞吐量的上界

$$\Theta_{\text{ideal}} \leq \frac{2bB_C}{N} = \frac{2B_B}{N}. \tag{3.4}$$

例如，考虑均匀流量下的 k 节点环。$B_C = 4$ 个通道穿过网络对分，每个方向两个。（图 3.3 显示了一个 8 节点环。）因此，由方程 3.4 可知 $\Theta_{\text{ideal}} \leq$ 8b/k。对于环，事实证明这个界是精确的。

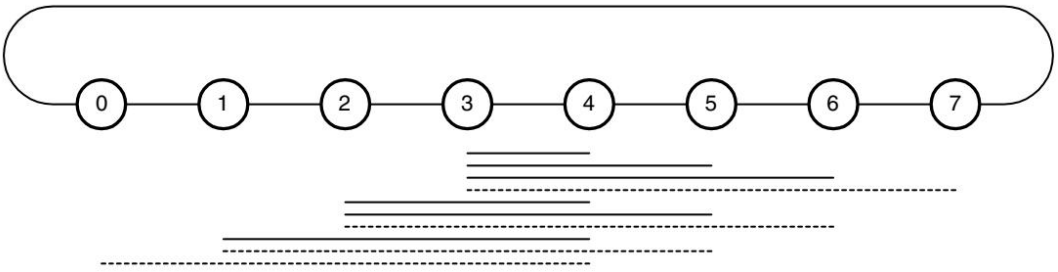


图 3.3 一个 8 节点环。

图 3.3 一个 8 节点环。

通道负载的另一个有用下界可以用类似方式计算。乘积 H_min N 给出了通道需求——对于给定流量模式，传递一轮数据包所需的通道遍历次数。如果我们假设所有通道均等负载的最佳情况，将此需求除以通道数即可界定网络中每个通道的负载：

$$\gamma_{\text{c,LB}} = \gamma_{\text{max,LB}} = \frac{H_{\min} N}{C}. \tag{3.5}$$

这些下界可以通过一个简单的最大通道负载上界来补充，即考虑一个在所有最小路径上均等均衡负载的路由函数。也就是说，如果有 $|R_{xy}|$ 条路径，则每条路径的每个通道被计入 $1/|R_{xy}|$。最大负载 $\gamma_{\text{max,UB}}$ 是所有通道中最大的 $\gamma_{\text{c,UB}}$。数学上，我们将这些负载定义为

$$\begin{aligned}\gamma_{\text{c,UB}} &= \frac{1}{N}\sum_{x\in N}\sum_{y\in N}\sum_{P\in R_{xy}}\begin{cases}1/|R_{xy}| & \text{if } c\in P\\ 0 & \text{otherwise}\end{cases}\\ \gamma_{\text{max,UB}} &= \max_{c\in C}\gamma_{\text{c,UB}}.\end{aligned} \tag{3.6}$$

对于任何拓扑，$\gamma_{\text{max,LB}} \le \gamma_{\text{max}} \le \gamma_{\text{max,UB}}$，并且在边对称拓扑（例如环面）的情况下，两个界都精确等于 γ_{max}。

为了了解如何使用通道负载来估计理想吞吐量，考虑如图 3.3 所示的八节点环网络（一个 8 元 1 立方）的情况。该拓扑是边对称的；因此，我们的简单界等于最大通道负载，并且我们还知道所有通道上的负载相同，所以计算单个通道上的负载就足够了。我们对通道（3，4）应用上界方法，即从节点 3 到节点 4 的右向通道。方程 3.6 的求和由图中的网络下方的线示出。每条线表示使用该通道的一条路径。虚线表示计为一半的路径。对于这些长度为四的路径，存在第二条沿顺时针方向遍历网络的最小路径。执行求和，我们看到有六条实线（六对节点总是使用（3，4））和四条虚线（四对节点有一半时间使用（3，4）），总和为八。将此和除以 N = 8，我们看到通道负载为 γ_{max} = 1。

我们还可以验证，下界给出了相同的最大信道负载。在这种情况下，平均数据包传输 $H_{min} = 2$ 跳。每个节点还贡献两个信道，一个向左，一个向右；因此，通过这种方法得到的信道负载也是 $\gamma_{max} = H_{min}N/C = 2 \cdot 8/16 = 1$。

为了在一般情况下计算 γ_{max}，我们必须找到数据包在网络中的最优分布，以最小化信道负载。这个任务可以表述为一个凸优化问题。虽然确定这些问题的解决方案超出了本书的范围，但我们将给出问题的表述。

对于网络中的每个目的地 d，我们用长度为 $|C|$ 的向量 x_d 表示发往 d 的数据包在网络信道上的平均分布。通过在每個节点添加流量平衡方程来维持有效的分布：传入分布之和减去传出信道上的分布之和必须等于该节点产生（正值）或吸收（负值）的平均数据包数。对于所有 $d \in N^*$，这些平衡方程对每个分布 x_d 都成立。在均匀流量下，对于分布 x_d，所有终端节点（包括目的地 d）产生 1/N 单位的流量，目的地 d 吸收 1 单位。这通过使用一个 $|N^*|$ 元素的平衡向量 f_d 来表示：

$$f_{d,i} = \begin{cases}1/N - 1 \text{ if } d = i \text{ and } d \in N,\\ 1/N \text{ if } d \neq i \text{ and } d \in N,\\ 0 \text{ otherwise}\end{cases} \tag{3.7}$$

其中 $f_{d,i}$ 是 f_d 的第 i 个元素。然后，我们使用一个 $N^* \times C$ 的节点-弧关联矩阵 A 来表示拓扑，其中

$$A_{n,c} = \begin{cases}+1 \text{ if } s_c = n,\\ -1 \text{ if } d_c = n,\\ 0 \text{ otherwise.}\end{cases} \tag{3.8}$$

那么，目标是最小化任何一条信道上的最大负载，整个优化问题可以写为

$$\text{minimize} \quad \max_{c \in C} \sum_{d \in N} x_{d,c}$$

$$\text{subjectto} \quad Ax_d = f_d \quad \text{and} \tag{3.9}$$

$$x_d \geq 0 \quad \text{for all } d \in N^*.$$

这个凸优化问题的解给出了最优的最大信道负载 γ_{max}。[1] 方程 3.6 和 3.9 估计了均匀流量下网络的吞吐量：每个节点向每个节点发送的可能性相等。通过调整平衡向量 f_d，可以将任意流量模式纳入方程 3.9。类似地，方程 3.6 可以通过用 λ_{xy}（x 发送到 y 的概率）对最终求和进行加权来适应。

$$\gamma_c(\Lambda) = \sum_{x \in N} \sum_{y \in N} \lambda_{xy} \sum_{P \in R_{xy}} \begin{cases} 1/|R_{xy}| & \text{if } c \in P \\ 0 & \text{otherwise} \end{cases}$$

最后，通过用任意流量模式的平均跳数替换来修改方程 3.5：

$$H_{\min}(\Lambda) = \sum_{x \in N} \sum_{y \in N} \lambda_{xy} H(x, y).$$

我们通常将网络在均匀流量下的理想吞吐量 $\Theta(U)$ 称为网络的容量。将网络在任意非均匀流量模式 Λ 下的吞吐量表示为容量的分数：$\Theta(\Lambda)/\Theta(U)$ 通常很有用。如果网络的所有信道具有相同的带宽，那么将容量的分数表示为 $\gamma_{\max}(U)/\gamma_{\max}(\Lambda)$ 也是等价的。

3.3.2 延迟

网络的延迟是指数据包穿越网络所需的时间，从数据包头部到达输入端口到数据包尾部离开输出端口。我们将延迟 T 分为两个部分

$$T = T_h + \frac{L}{b}.$$

头部延迟 T_h 是消息头部穿越网络所需的时间，而序列化延迟，

$$T_s = L/b \tag{3.10}$$

是尾部赶上的时间——即长度为 L 的数据包以带宽 b 穿过信道的时间。

与吞吐量一样，延迟不仅取决于拓扑，还取决于路由、流控制和路由器的设计。然而，如上所述，我们在这里关注拓扑对延迟的贡献，稍后将考虑其他影响。

在没有竞争的情况下，头部延迟是由拓扑决定的两个因素之和：路由器延迟 T_r 和飞行时间 T_w。路由器延迟是在路由器中花费的时间，而飞行时间是在导线上花费的时间。对于平均跳数为 H_{min}、通过单个路由器的延迟为 t_r 的网络，平均路由器延

迟为 $T_r = H_{min}t_r$。对于平均距离为 D_{min}、传播速度为 v 的网络，平均飞行时间为 $T_w = D_{min}/v$。
结合这些组件，得到无竞争情况下平均延迟的以下表达式

$$T_0 = H_{\min}t_r + \frac{D_{\min}}{v} + \frac{L}{b}. \tag{3.11}$$

这三项对应于总延迟的三个组成部分：交换机延迟、飞行时间和序列化延迟。我们将此延迟称为 T_0，因为它表示在零负载下没有竞争时的延迟。随着负载增加，方程中会增加第四项 T_c，它反映了等待资源（即竞争）所花费的时间。
网络的拓扑及其在物理封装上的映射在很大程度上决定了方程中的三个关键参数。平均跳数 H_{min} 完全是拓扑的属性。平均距离 D_{min} 受封装和拓扑的影响。最后，带宽 b 由节点度（拓扑的属性）和封装约束决定。
图 3.4 显示了数据包从节点 x 经由中间节点 y 沿两跳路由传播的甘特图，以说明方程 3.11 的三个项。图表的第一行显示了数据包的每个 phit 到达节点 x。在该数据包的第一个 phit 离开节点 x 之前，会产生一个跳路由延迟 t_r。然后，在 phit 到达节点 y 之前，会发生链路延迟 t_{xy}。在第一个 phit 到达节点 z 之前，会产生第二个路由延迟和链路延迟。在消息的第一个 phit 到达 z 后，经过一个序列化延迟，完整的数据包被接收。图底部的条形图总结了延迟的来源。单斜线区域对应于路由延迟，浅灰色区域对应于链路延迟，双斜线区域对应于序列化延迟。

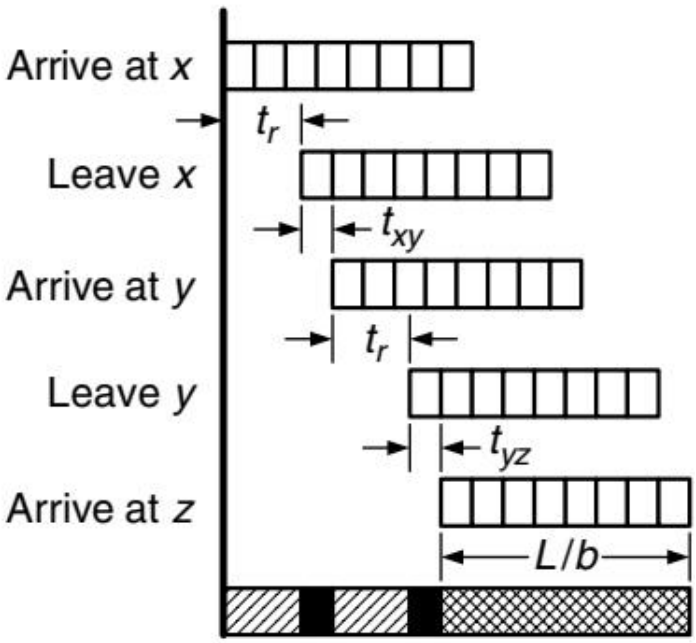


图 3.4 甘特图显示了在无竞争情况下数据包穿越两个信道的延迟。

考虑一个 64 节点网络，平均跳数 $H_{avg} = 4$，通道宽度为 16 位。每个通道 c 的工作频率为 $f_c = 1$ GHz，传输延迟为 $t_c = 5$ ns。如果单个路由器的延迟为 $t_r = 8$ ns（4 个 2 ns 时钟），则总路由延迟为 $8 \cdot 4 = 32$ ns（16 个时钟）。在我们的示例中，4 段线延迟各为 5 ns，总计增加 $T_w = 20$ ns（10 个时钟）。如果数据包长度 L = 64 字节，且所有通道具

有均匀的 2 Gbytes/s 带宽，则串行化延迟为 $64/2 = 32$ ns（16 个时钟）。因此，对于该网络，$T_0 = 32 + 20 + 32 = 84$ ns。

3.3.3 路径多样性

在大多数节点对之间存在多条最小路径的网络（对于大多数 $x, y \in N$，$|R_{xy}| > 1$）比仅存在单条节点到节点路由的网络（$|R_{xy}| = 1$）更具鲁棒性。我们将这一特性称为路径多样性，它通过均衡通道间的负载以及允许网络容忍故障通道和节点，增强了网络的鲁棒性。

到目前为止，我们主要关注的是网络在随机流量下的吞吐量，其中每个节点向任何其他节点发送消息的概率相等。对于许多网络而言，这种随机流量是一种最佳情况负载，因为通过在节点对之间均匀分配流量，它也均匀地均衡了网络通道间的负载。对于许多网络来说，更具挑战性的情况是任意置换流量，其中每个节点 x 将其所有流量发送给恰好一个其他节点 $\pi(x)$，其中 π 为某个置换。在没有路径多样性的情况下，某些置换会将相当大比例的流量集中在单个瓶颈通道上，导致吞吐量显著下降。

作为路径多样性在负载均衡中重要性的一个例子，考虑在具有单位带宽通道的 2 元 4 飞网络和具有半单位带宽通道的 4 元 2 立方体网络上发送位旋转流量。这两个网络在随机流量下均有 $\gamma_{max} = 1$。对于位旋转（BR）流量，地址为 $\{b_3, b_2, b_1, b_0\}$ 的节点仅向地址为 $\{b_2, b_1, b_0, b_3\}$ 的节点发送数据包。换言之，该流量对应于洗牌置换

$\{0, 2, 4, 6, 8, 10, 12, 14, 1, 3, 5, 7, 9, 11, 13, 15\}$

即，节点 0 发送给自身，节点 1 发送给节点 2，依此类推。图 3.5 展示了当该置换应用于 2 元 4 飞网络时发生的流量集中现象。如图中粗线所示，来自节点 0、1、8 和 9 的所有数据包必须经过通道（10，20）。类似地，来自节点 2、3、10 和 11 的所有流量必须经过通道（11，23）；4、5、12、13 集中在（16，24）；6、7、14、15 集中在（17，27）。因此，对于这种流量模式，$\gamma_{\text{max,BR}} = 4$，吞吐量因此为容量的 $\gamma_{\text{max}}/\gamma_{\text{max,BR}} = 25\%$。

图 3.6 展示了该置换流量如何映射到 4 元 2 立方体上。两条路由不经过任何通道，四条经过一个通道，四条经过两个通道（例如，每条有两条可能的最小路由，$H(5, 10) = 2$ 且 $|R_{5,10}| = 2$），四条经过三个通道（有三条替代路由），四条经过四个通道（有 24 条替代路由）。在最小路由下，单跳通道成为瓶颈。没有替代的最小路由，因此 $\gamma_{max,BR} = 1$。对于该拓扑，均匀流量给出 $\gamma_{max} = 0.5$，因此在位反转模式下的吞吐量为容量的 $\gamma_{max}/\gamma_{max,BR} = 50\%$。然而，如果我们允许四条单跳路由将其一半流量以非最小方式沿相反方向路由，则流量均匀分布。例如，从节点 1 到 2 的一半流量走路由 $\{(1, 2)\}$，而剩余一半走 $\{(1, 0), (0, 3), (3, 2)\}$。由此产生的吞吐量提高到容量的 89%，11% 的下降是由于非最小路由所经过的额外跳数。

这个例子说明了通常需要非最小路由来均衡通道间的负载。尽管非最小路由完成了更多的总工作（经过更多通道），但由于更好的负载均衡，整体性能得到改善。然而，非最小路由会增加实现复杂性。例如，使用非最小路由的网络需要特别注意避免死锁，我们将在第 14 章中看到。

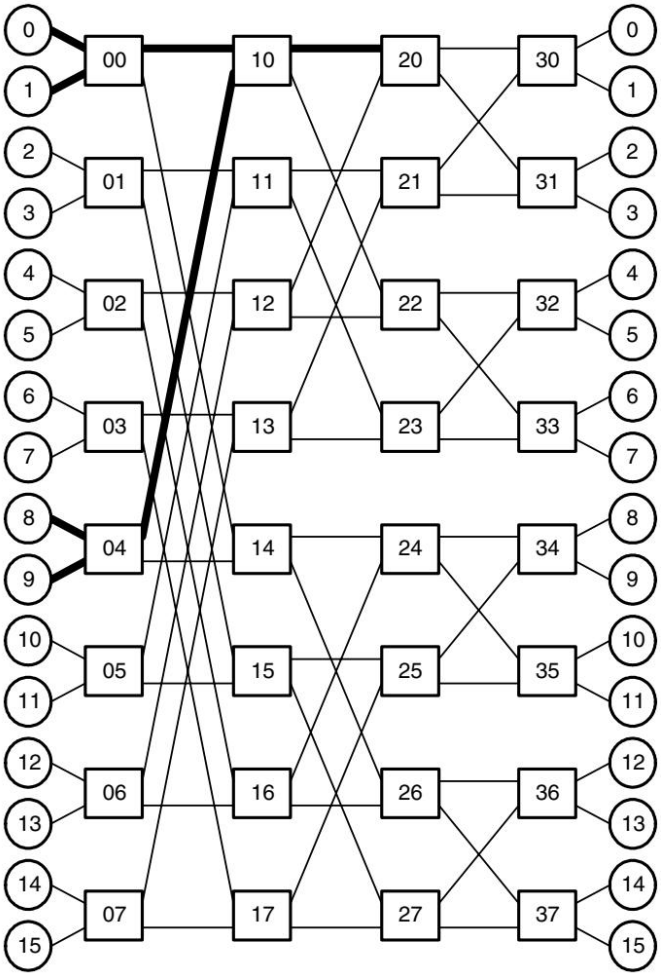


图 3.5 在 2 元 4 飞网络上路由洗牌置换导致所有流量集中在网络中心四分之一的通道上，使性能下降为原来的四分之一。

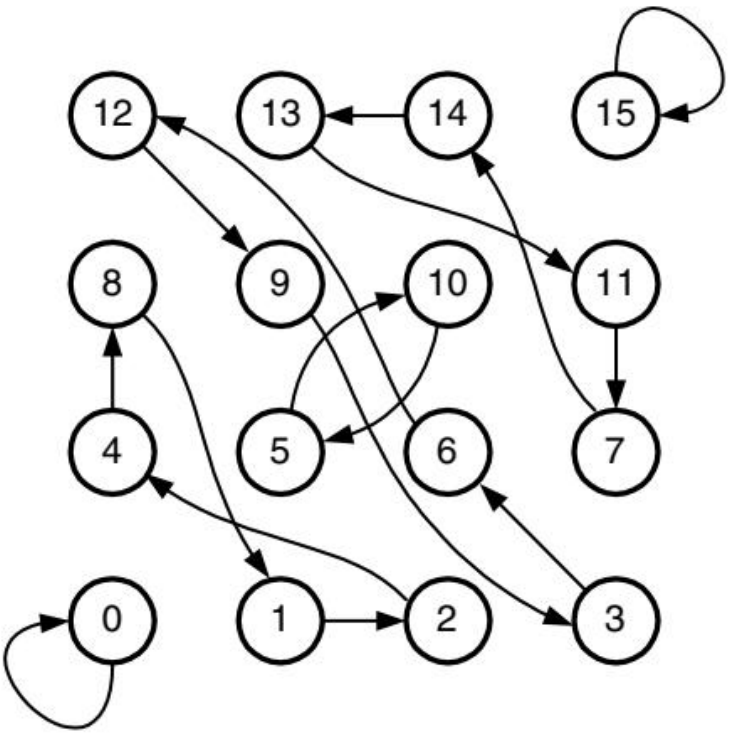


图 3.6 在 4 元 2 立方体上路由洗牌置换。在最小路由下，吞吐量下降为原来的二分之一，因为所有单跳流量必须经过带宽为 0.5 的单个通道。在非最小路由下，流量均匀分布，吞吐量为容量的 0.89。

增加路径多样性的另一个重要优势是网络处理故障（如节点或链路故障）的能力。例如，对于图 3.5 中的蝶形网络，如果从交换机 07 到 17 的链路发生故障，则从源 14 到目的 15 没有可行路径。由于互连网络的平均无故障时间随着系统组件数量的增加而减少，因此大型互连网络必须能够容忍一个或多个故障节点或链路，这一点至关重要。衡量网络处理故障能力的一个指标是路由函数在每个源-目的对之间允许的边不相交或节点不相交路径的数量。如果一组路径不共享任何公共链路，则称该组路径是边不相交的。因此，如果网络中的一条链路发生故障，它保证最多影响边不相交路径集合中的一条路径。一般来说，如果对于给定的路由函数，所有节点之间的最小边不相交路径数为 j，则只要链路故障数少于 j，网络就保证连通。

节点不相交路径是一组不共享任何公共节点的路径，源节点和目的节点除外。与边不相交路径类似，如果一个节点发生故障，它只会影响节点不相交路径集合中的一条路径，当然，除非故障节点是这些路径的源节点或目的节点。节点不相交路径集合也是边不相交的，因此，如果一个网络和路由函数在每个源-目的节点对之间至少有 j 条节点不相交路径，那么它最多可以容忍 j 次链路加节点的总故障。

一组不幸的故障可能全部影响网络中某个特定节点的邻居。对于这个不幸的节点，如果它的所有邻居节点、所有入链路、所有出链路或任何等效的组合发生故障，则无法路由到该节点或从该节点路由出去。因此，网络有可能在 $\min_x [\min\{|C_{Ix}|, |C_{Ox}|\}]$ 次故障后不再连通。

3.4 封装成本

在构建网络时，拓扑的节点被映射到物理系统中的封装模块、芯片、板和机箱上。拓扑和封装技术的特性以及节点的布局决定了通道带宽的约束。没有这些约束，就无法评估拓扑或在拓扑之间进行公平比较。在本节中，我们基于典型的两级封装层次结构开发一个简单的封装成本模型，其中通道宽度 w 受每个节点的引脚数和全局布线总量的约束。我们还讨论了频率 f（通道带宽的另一个因素）如何受封装选择的影响。

在封装模型的第一级层次中，各个路由器通过局部布线连接。与全局布线相比，局部布线成本低且资源丰富，因此必须安排拓扑以利用空间局部性——相邻节点应在物理上彼此靠近放置。这种安排允许使用局部布线代替全局布线。例如，考虑一个系统，其中 16 个节点可以容纳在单个印刷电路（PC）板上。通过将 4×4 的节点阵列封装在 PC 板上（如图 3.7 所示），四分之三的引脚仍保留在 PC 板本地，只有 32 个通道需要跨越模块边界。

一旦找到有效的节点局部布局，通道宽度的主要约束就变成每个节点上可用的引脚数。对于每个节点最大引脚数为 W_n 的节点，通道宽度 w 受以下约束：

$$w \leq \frac{W_n}{\delta}. \tag{3.12}$$

层次结构的第二级通过全局布线连接本地节点组。这种全局布线的典型实现是连接多个独立节点板的背板，如图 3.8 所示。任何全局信号从一个板出发，穿过一个电连接器，进入背板，再穿过另一个连接器到达另一个板。在这个级别上，可用的全局导线数 W_s 限制了各个通道的宽度。例如，在背板上构建的网络，其导线对分受背板导线密度的限制。考虑到通孔空间，典型的 PC 板在每个信号层上可以支持 1 根导线/mm 的导线密度，对于差分信号则为 0.5 个信号/mm。中等成本的 PC 板可能总共有 8 个布线层，x 方向 4 层，y 方向 4 层，每个方向的总导线密度为 2 个信号/mm。

为了估计特定拓扑所需的全局通道数，我们使用拓扑的最小通道对分 B_C。最小对分是网络的一个切割，它将网络几乎分成两半，同时切割尽可能少的导线。因此，对分创建的两个节点集代表了将节点划分为本地组进行封装的良好分区。虽然这个模型有局限性，因为许多网络可能需要划分为两个以上的本地组以满足封装技术的约束，但在这些情况下，它通常是一个很好的估计。

使用最小对分，可用的全局布线将通道宽度约束为

$$w \leq \frac{W_s}{B_C}. \tag{3.13}$$

虽然我们的讨论集中在两级封装层次结构上，但公式 3.13 也可以应用于更大系统中必要的额外封装级别。（见练习 5.3。）

3.4 封装成本

结合公式 3.12 和 3.13，我们得到对通道宽度的总体约束

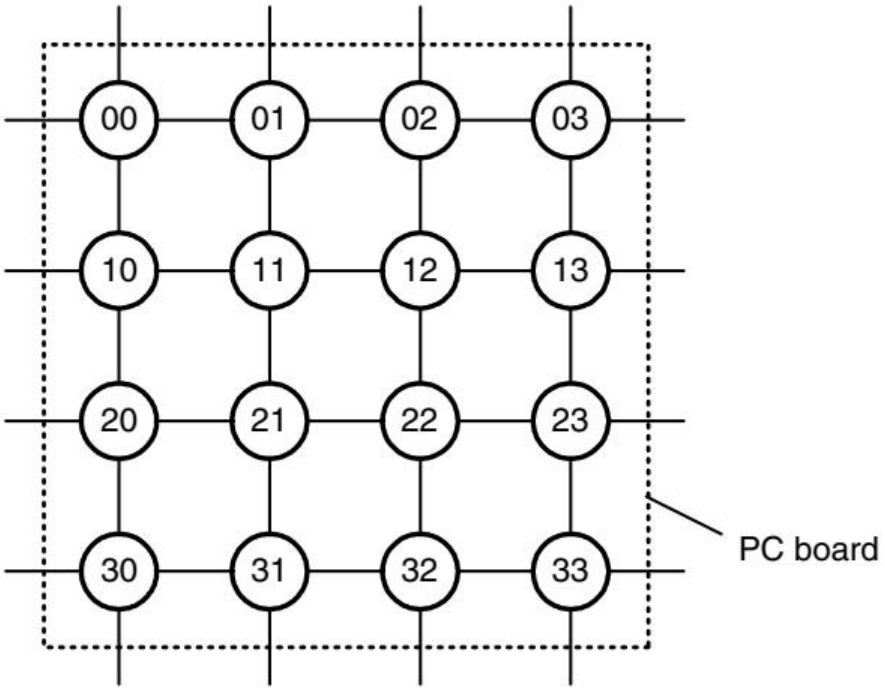


图 3.7 一个 4×4 的节点阵列封装在 PC 板上，使得所有节点引脚中的 $\frac{3}{4}$ 通过局部布线连接到板上的其他引脚。

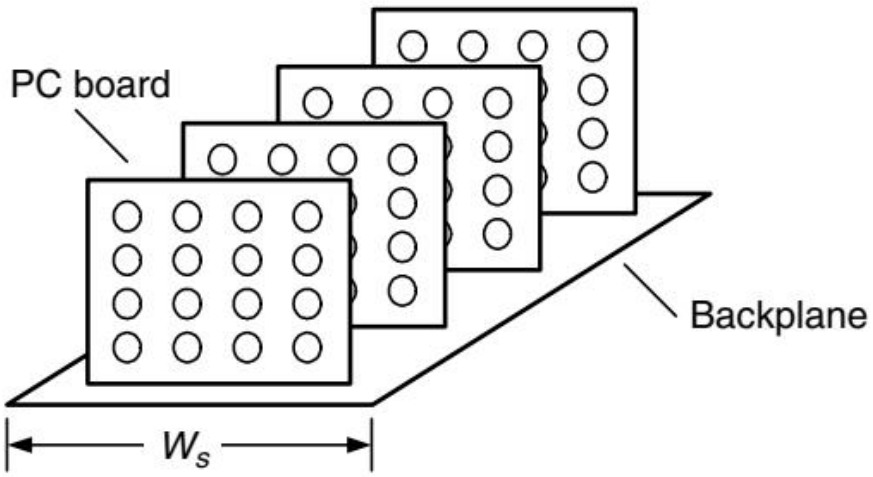


图 3.8 使用背板连接多个节点板。

$$w \le \min\left(\frac{W_n}{\delta}, \frac{W_s}{B_C}\right). \tag{3.14}$$

公式 3.14 的第一项往往主导低度网络（例如环）。这些网络受节点引脚限制。另一方面，高度网络（例如二进制 n 立方体）往往受对分限制，公式 3.14 的第二项占主导地位。

我们也可以用带宽而不是通道宽度来表达封装约束。节点的最大带宽为 $B_n = fW_n$，系统对分处的最大带宽为 $B_s = fW_s$。使用带宽，我们可以重写公式 3.14，给出每个通道的最大带宽为

$$b \le \min\left(\frac{B_n}{\delta}, \frac{B_s}{B_C}\right).$$

除了封装层次结构不同级别可用的布线宽度外，另一个重要考虑因素是这些导线的长度。网络通道的长度必须保持较短，因为超过临界长度后，信号频率会随导线长度呈二次方下降：2

$$f = \min\left(f_0, f_0\left(\frac{l_w}{l_c}\right)^{-2}\right).$$

导线的临界长度 l_c 是导线的标称信号速率 3 f_0、导线的物理特性以及系统可容忍的频率相关衰减量的函数。

表 3.2 显示了在 2 GHz 信号速率下几种常见导线的临界长度，假设可容忍的衰减不超过 1 dB。4 密度-成本约束导致大多数网络由 PC 板中的带状线、细线电缆或两者构建。这些互连（对应表的前两行）只能运行很短的距离（小于一米），之后就会达到临界导线长度，数据速率开始随长度呈二次方下降。

通过在线路中插入中继器，可以构建长通道并以高比特率运行。然而，插入中继器的成本与插入交换机大致相同，因此这种做法意义不大。更好的做法是采用一种使通道长度保持在临界长度以下的拓扑结构，在长路由中插入交换机而非中继器。线速与线长之间的关系使得使用需要长通道的拓扑结构来构建高速电网络变得不切实际。

表 3.2 2 GHz 下常见导线的临界长度（无均衡）

导线类型	**临界长度** l_c
5 mil 带状导线（stripguide）	0.10 m
30 AWG 双绞线	0.56 m
24 AWG 双绞线	1.11 m
RG59U 同轴电缆	10.00 m

通过使用光信号，也可以构建以高比特率运行的长通道。虽然光纤也会衰减和色散信号，从而限制距离，但其衰减和色散速率远低于电传输线。单模光纤可以在需要再生之前传输信号数十到数百公里。光通道的缺点是成本。2003 年，一条光通道的成本是相同带宽电通道的十倍以上。

图 3.9 展示了两种不同的 6 节点拓扑之间的比较。第一种拓扑是一个简单的 6 环，$\delta = 4$ 且 $B_C = 4$。第二种拓扑属于一大类称为凯莱图的图，其 $\delta = 6$ 且 $B_C = 10$。我们的封装技术允许每个节点 $W_n = 140$ 个引脚，背板宽度为 $W_s = 200$ 个信号。因此，为了在这些拓扑之间进行公平比较，选择通道宽度 w 使得两种拓扑都满足封装约束。首先，对于环，

$$w \leq \min\left(\frac{W_n}{\delta}, \frac{W_s}{B_C}\right) = \min\left(\frac{140}{4}, \frac{200}{4}\right) = 35. \tag{3.15}$$

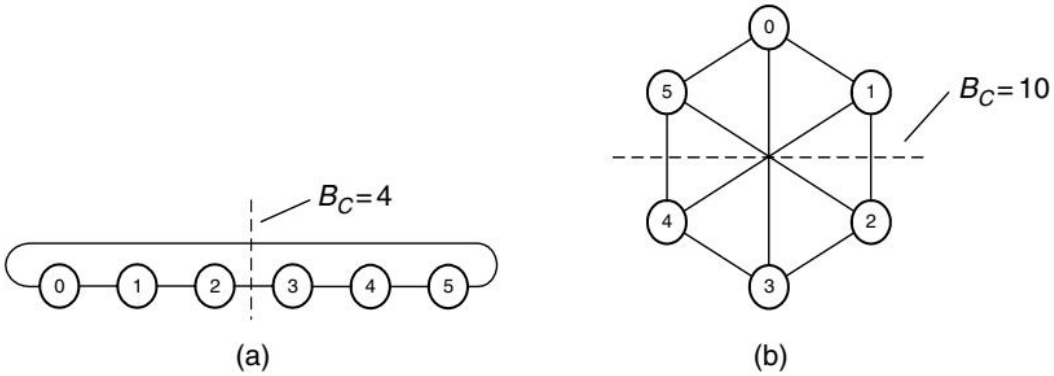


图 3.9 （a）6 环和（b）凯莱图的等分宽度和度数。每条边代表两条方向相反的单向通道。

对于凯莱图，

$$w \leq \min\left(\frac{W_n}{\delta}, \frac{W_s}{B_C}\right) = \min\left(\frac{140}{6}, \frac{200}{10}\right) = 20. \tag{3.16}$$

使用这些宽度和 $f = 1$ GHz 的信号频率，环的通道带宽为 35 Gbits/s，凯莱图为 20 Gbits/s。如果消息长度 $L = 1024$ 比特，路由器延迟 $t_r = 20$ ns，并给定最大通道负载和平均跳数，则可以比较两个网络的理想吞吐量和零负载延迟。（见表 3.3。）如图所示，凯莱图提供了更好的理想吞吐量，而在给定的封装约束下，环具有更低的零负载延迟。检查公式 3.15 和 3.16 表明，环的通道宽度受引脚带宽限制，而凯莱图能够利用全等分宽度。这导致凯莱图具有更好的等分利用率和更好的理想吞吐量。然而，凯莱图更高的度数限制了个体通道的大小，导致更高的串行化延迟和环更好的零负载延迟。人们可能会认为，仅从拓扑结构来看，这个延迟结果是反直觉的，因为凯莱图具有更小的平均跳数。然而，通过考虑特定封装对通道宽度的限制，我们看到凯莱图的实际延迟实际上高于环——在这种情况下，跳数的减少被串行化延迟的增加所压倒。

3.5 案例研究：SGI Origin 2000

为了给出一个具体的拓扑示例，我们将看看 SGI Origin 2000 [108] 的互连网络。Origin 2000 系统如图 3.10 所示，于 1997 年首次发布。Origin 2000 支持多达 512 个节点，每个节点上有 2 个 MIPS R10000 [197] 处理器。由于它是共享内存多处理器，对网络的要求是低延迟和高吞吐量。

表 3.3 环网络与凯莱图的封装性能示例

指标	环网络	凯莱图
b	35 Gbits/s	20 Gbits/s
H_{avg}	3/2	7/6
γ_{max}	3/4	7/18
Θ_{ideal}	≈ 46.7 Gbits/s	≈ 51.4 Gbits/s
T_h	30 ns	≈ 23.3 ns
T_s	≈ 29.3 ns	≈ 51.2 ns
T_0	≈ 69.3 ns	≈ 74.5 ns

图 3.10 SGI Origin 2000 服务器的机柜式和桌面式配置

通过检查这台机器的网络，我们将看到将互连网络映射到封装层次结构所涉及的许多问题。我们还将探讨从一组固定组件构建互连网络的问题，这些组件支持从几个节点到数百个节点的可扩展机器规模。

Origin 2000 网络基于 SGI SPIDER 路由芯片 [69]。SPIDER 芯片提供 6 个双向网络通道。每个通道宽 20 位，以 400 MHz 运行，通道带宽为 6.4 Gbits/s。因此，网络的每个节点有六个 6.4 Gbits/s 链路，总节点带宽为 38.4 Gbits/s。所有这些通道都可以通过背板驱动，其中三个通道具有能够驱动长达五米电缆的物理接口。

图 3.11 说明了 Origin 2000 网络拓扑如何随着节点数量的增加而变化。在所有配置中，网络将 2 个处理节点（4 个处理器）连接到每个终端路由器。5 这些终端连接使用路由器上 6 个通道中的 2 个，留下 4 个通道连接到其他路由器。最多 16 个路由器（32 个节点，64 个处理器）的系统配置为二进制 n 立方体，每个路由器最多连接到 4 个维度上的相邻路由器，如图 3.11（a）所示。如果并非所有 4 个维度都被使用（例如，在 8 路由器系统中），则未使用的通道可以跨机器连接以减少网络直径。

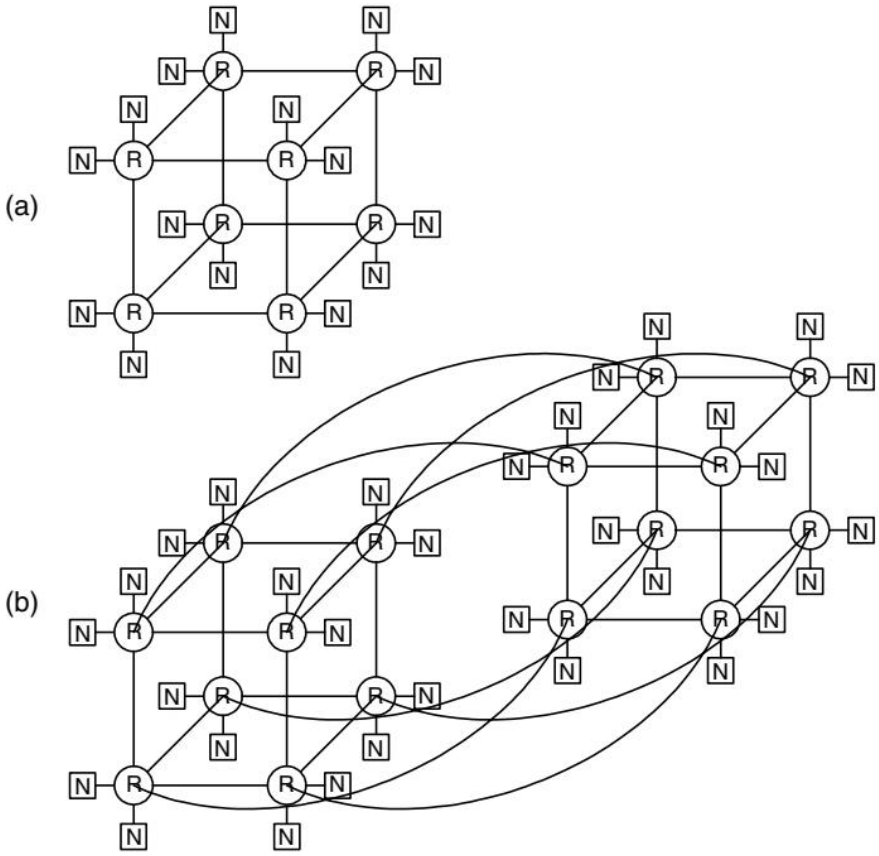


图 3.11 Origin 2000 在八路由器和十六路由器配置下的二进制超立方体网络拓扑

超过 16 个路由器的机器采用分层网络实现，如图 3.12 所示。这些较大的配置由 8 路由器（16 节点、32 处理器）的本地子网组成，这些子网配置为二元 3 立方体，每个节点上留有一个通道开放。然后使用八个仅路由器的全局子网将 8 路由器子网连接在一起。对于具有 2^n 个路由器的机器，每个全局网络是一个具有 m = n - 3 维的二元立方体。例如，最大 256 路由器（512 节点、1024 处理器）配置使用 8 个 32 节点二元 5 立方体作为全局子网。本地子网 j 中路由器 i 的开放通道连接到全局子网 i 的路由器 j。对于 32 节点机器（n = 5）的特殊情况，每个 4 端口全局子网由单个路由器芯片实现。这种结构实际上是一个 Clos 网络（见第 6.3 节），其中各个交换机由二元 n 立方

体构成。

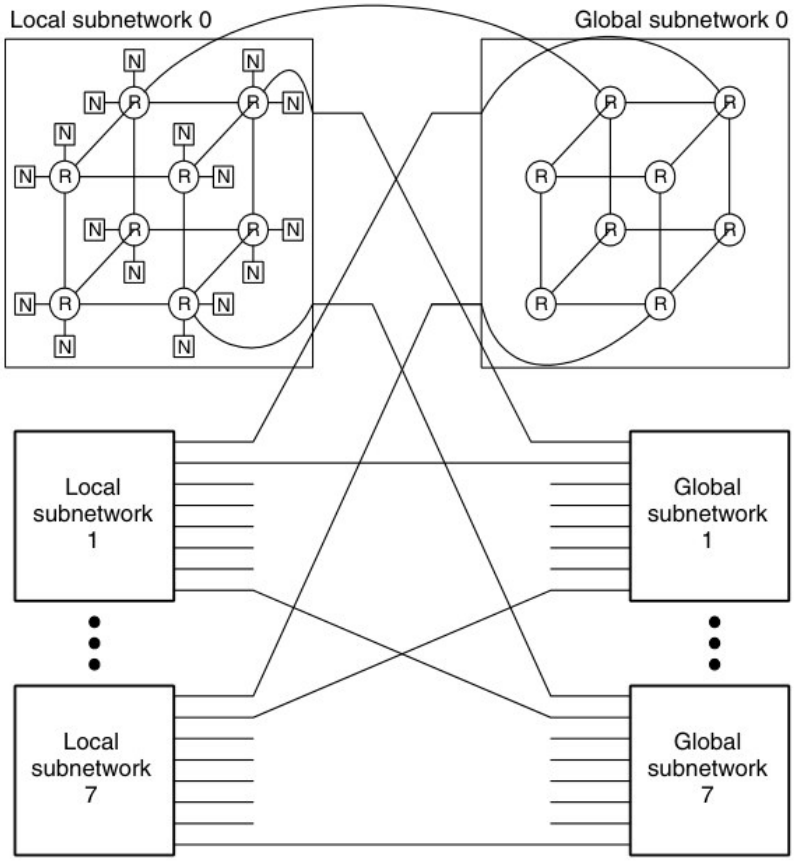


图 3.12 超过 32 个节点的 Origin 2000：由子网连接形成的层次化拓扑

Origin 2000 采用板、模块和机架的层次结构封装，如图 3.13 所示。每个节点（2 个处理器）封装在单个 16 英寸 ×11 英寸的电路板上。每个路由器芯片也封装在单独的电路板上。四个节点板（8 个处理器）和 2 个路由器板封装在一个机箱中，并通过中板连接。每个机箱的剩余空间用于 I/O 子系统。每个机柜中放置两个机箱（四个路由器）。一个系统最多可以有 64 个机柜（256 个路由器）。在大型系统中，使用额外的仅路由器机柜来实现全局子网。

超过 32 个节点的 Origin 2000 机器使用分层拓扑，其中 8 路由器（16 节点）二元 3 立方体子网通过每个节点 1 条链路连接到 8 个中心二元 n 立方体网络，$n \leq 5$。该图显示了一个 128 节点的机器，其中有 8 个本地子网连接到 8 个二元 3 立方体全局子网。在每个路由器板的 6 个通道中，2 个连接到中板上的 4 个节点板中的 2 个，1 个连接到中板上的另一个路由器，其余 3 个引出到背板连接器以连接到其他机箱。每个路由器上的这些背板连接之一用于连接到同一机柜中的另一个机箱，第二个连接到本地子网中的第二个机柜，第三个连接到全局子网或 16 路由器、4 机柜系统中的第二对机柜。表 3.4 显示了在完全由 6 端口路由器构建网络的约束下，这种分层拓扑如何满足共享内存多处理器的要求。如公式 3.11 所示，零负载延迟随平均跳数和距离增长，两者都随直径增长，而序列化延迟（公式 3.10）由路由器通道的 20 位宽度固定。为了保持低延迟，Origin 2000 使用的拓扑中直径（以及因此跳数）随机器规模对数增长。采用分

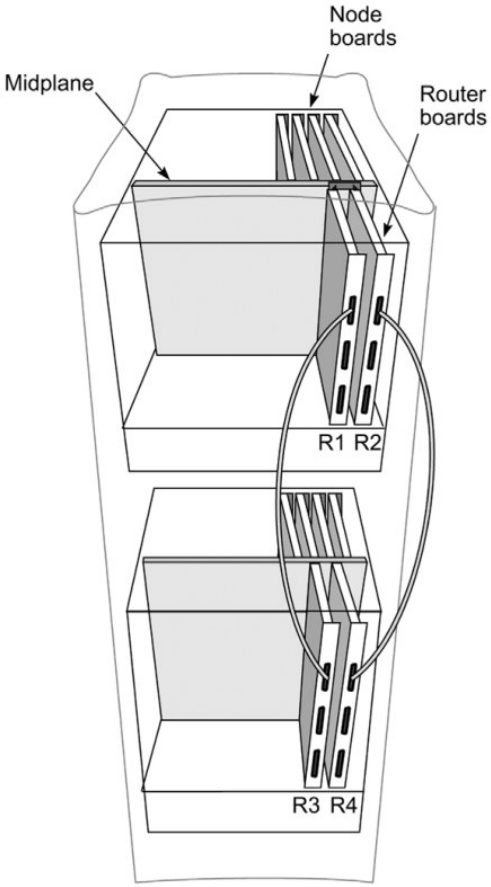
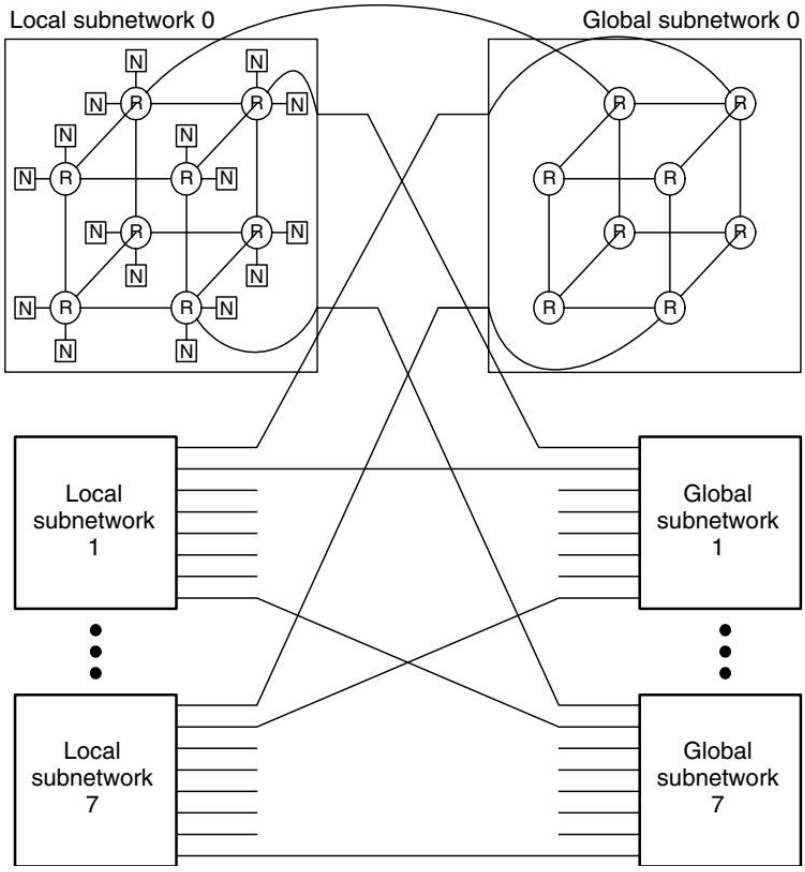
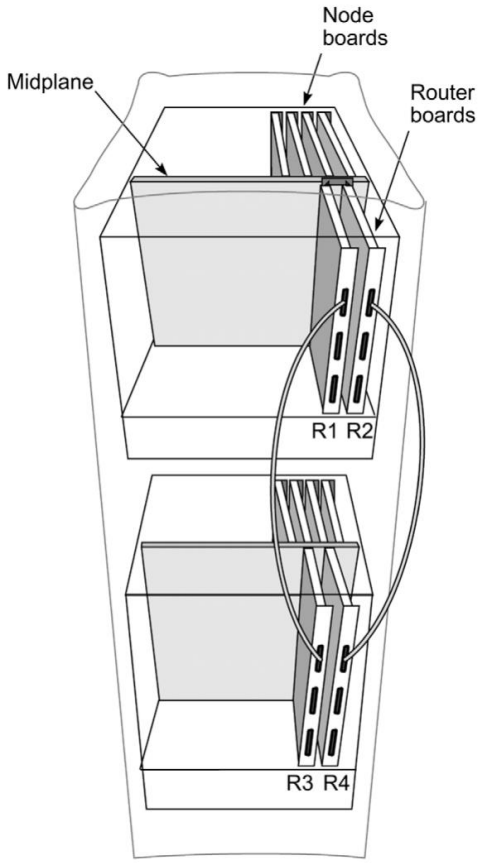


图 3.13 四路由器（16 处理器）Origin 2000 系统的封装结构



超过 32 个节点的 Origin 2000 机器使用分层拓扑，其中 8 路由器（16 节点）二元 3 立方体子网通过每个节点 1 条链路连接到 8 个中心二元 n 立方体网络，$n \leq 5$。该图显示了一个 128 节点的机器，其中有 8 个本地子网连接到 8 个二元 3 立方体全局子网。

层网络允许设计者在机器规模大于度数 4（节点使用 2 个端口）二元 n 立方体所能实现的规模时继续对数缩放。我们将延迟的精确计算留作练习 3.8。



4 路由器（16 处理器）Origin 2000 系统的封装。经 Laudon/Lenoski [ISCA ’97] © [1997] ACM，Inc。许可转载。

Origin 2000 拓扑通过随着机器扩展为每个节点提供平坦的对分带宽，满足了共享内存多处理器的带宽要求。对于每种机器配置，机器的对分切割 N 个通道，其中 N 是路由器数量（每个方向 N/2）。对于小型机器，二元 n 立方体网络有 2^n 个路由器和 2^n 个跨对分的通道，每个方向 2^{n-1} 个。对于较大的机器，每个节点有一个通道连接到全局子网，每个全局子网的对分带宽等于其输入带宽。

表 3.4 Origin 2000 网络配置与性能

Size (nodes)	**Topology**	**Chassis**	**Diameter**	B_C
4	binary 1-cube	1	3	2
8	binary 2-cube	2	4	4
16	binary 3-cube	4	5	8
32	binary 4-cube	8	6	16
64	4 3-cubes × 8 4 × 4 switches	16	7	32
128	8 3-cubes × 8 3-cubes	32	9	64
256	16 3-cubes × 8 4-cubes	64	10	128

512	32 3-cubes × 8 5-cubes	128	11	256

3.6 文献注释

尽管我们在本书接下来的章节中重点介绍两种最常见的互连网络，但还有其他几种值得注意的拓扑。立方体连接环 [153] 保持了超立方体网络的低跳数（直径），但节点度数恒定。胖树 [113] 已被证明是一种通用的网络拓扑，因为它们可以在多对数时间内模拟任何其他拓扑的行为。4 元胖树用于 Connection Machine CM-5 的网络 [114]。（见第 10.6 节。）Cayley 图 [7] 是一类包含立方体连接环的拓扑，提供简单的路由和比同等大小的超立方体网络更低的度数。

3.7 练习

3.1 环中的龙卷风流量。考虑一个 8 节点环网络，其中每个节点向环上 3 跳远的节点发送流量。即节点 i 向 $i+3 \pmod 8$ 发送流量。每个通道的带宽为 1 Gbit/s，每个输入提供 512 Mbits/s 的流量。

如果在该网络上使用最小路由，通道负载、理想吞吐量和加速比是多少？对于允许非最小路由且采用非最小路由的概率按其距离加权，使得数据包以 5/8 的概率走三跳路由、以 3/8 的概率走五跳路由的情况，重新计算这些数字。

3.2 最坏情况通道负载界限。假设对分通道是负载最重的通道，且所有通道具有相等的带宽 b，推导最坏情况流量下最大通道负载的下界。

3.3 通道负载界限的局限性。找到一个拓扑，其中公式 3.6 给出的最大通道负载上界不紧。你找到的拓扑是否需要非最小路由来最优平衡流量？解释你的答案。

3.4 对称拓扑通道负载界限的紧性。证明公式 3.5 和 3.6 中的最大通道负载界限等于任何边对称拓扑的最优负载。

3.5 非对称通道带宽下的吞吐量。当通道带宽不相等时，推导网络理想吞吐量的表达式。如有必要，修改 γ_c 和 γ_{max} 的定义，使其与新的吞吐量表达式一致。

3.6 串行化延迟对拓扑选择的影响。系统设计者需要构建一个网络，以尽可能小的数据包延迟连接 64 个处理器节点。为了最小化成本，每个路由器与其对应的处理器放置在同一芯片上（直接网络），并且每个处理器芯片有 128 个引脚专用于网络接口。每个引脚的带宽为 2 Gbits/s，平均数据包长度为 L = 512 位。路由器的跳延迟为 $t_r = 15$ ns。忽略线路延迟（$T_w = 0$）。

（a）设计者首先考虑全连接拓扑。即每个节点都有一个专用通道连接到其他所有节点。该网络的平均路由器延迟 $T_{r_{min}}$ 和串行化延迟 T_s 是多少？平均零负载消息延迟 T_0 是多少？

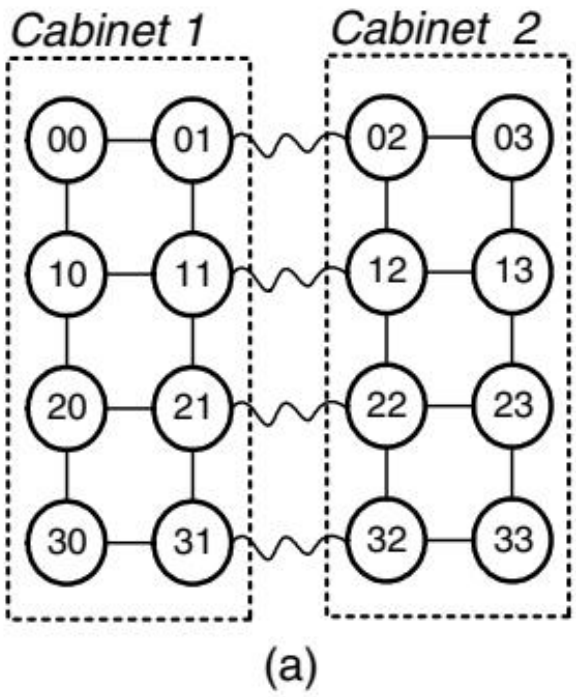
（b）重新计算环形拓扑的延迟。该环的 H_{min} 为 16。（见第 5.2.2 节。）

3.7 非随机流量下的延迟。对于第 1.3.1 节中描述的环面和环形网络，在计算延迟时假设了随机流量。这导致了环形具有更优延迟性能的结论。是否存在任何流量模式会使

环面具有更低的延迟? 如果没有，请解释原因。为简单起见，假设环面的节点 (i, j) 映射到环的节点 $(4i + j)$。（例如，环面 00 映射到环 0，环面 13 映射到环 7，依此类推。）

3.8 Origin 2000 中的延迟。Origin 2000（第 3.5 节）中的每个机架宽 19 英寸，高 72 英寸。假设一个机架内的所有电缆长度为 48 英寸，机架之间的所有电缆必须布线在活动地板下，并且电缆的传播速度为 2×10^8 m/s。此外，假设所有消息长度为 512 位（16 个 32 位字）。计算在均匀流量下，配置 16 个节点和配置 512 个节点的 Origin 2000 的平均零负载消息延迟，包括线路延迟 T_w。

3.9 部分随机布线带来的直径改进。随机拓扑，即节点通过通道随机连接，已知具有若干良好的图论性质，例如低直径。但这以封装和路由的规律性为代价，使得大多数纯随机网络不切实际。然而，混合方法仍然可以在不牺牲封装便利性的情况下实现随机化的一些好处 [191]。考虑图 3.14（a）中的网格网络，其中网络的左侧和右侧封装在单独的机柜中。



同轴电缆（图中的波浪线）连接两个机柜之间的对应节点。

（a）这个网格网络的直径是多少?

（b）如果电缆连接被随机置换，这个网络的直径会如何变化?（图 3.14[b] 显示了这样一种置换。）在所有置换中，网络的最小和最大直径是多少? 给出一个实现最小直径的置换。

3.10 胖树网络的性能。图 3.15 显示了一个 16 节点、基数为 2 的胖树拓扑。

（a）假设所有通道带宽等于终端节点的注入和弹出速率，这个网络的容量是多少?

（b）考虑一种在该网络上进行路由的随机方法：对于每个数据包，首先从源节点“向上”路由到树的顶部（即图中 8 个小的中心节点），沿途在两个可能的“向上”通道中随机选择，然后通过“向下”通道路由到目的地。对于任何流量模式，使用这种路

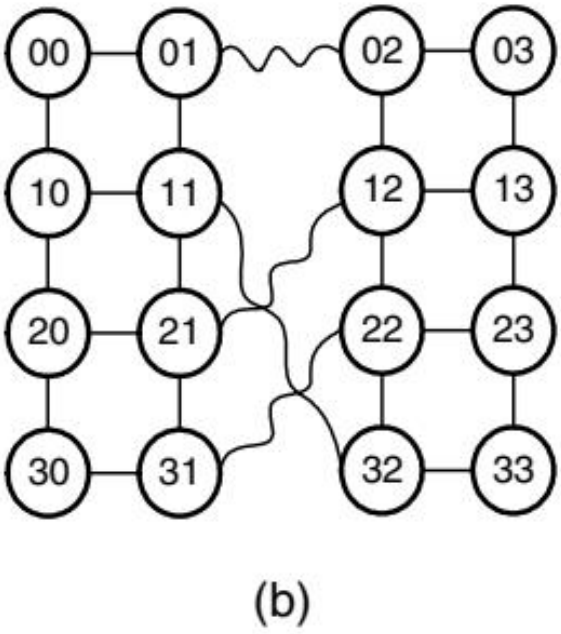


图 3.14 一个 16 节点网络跨 2 个机柜的封装（a）使用同轴电缆创建网格拓扑，以及（b）机柜之间电缆的随机排列。

由算法所能达到的最大通道负载是多少?

3.11 立方体连接环拓扑的性能和封装。图 3.16 中的拓扑称为 3^{rd} 阶立方体连接环。

（a）这个拓扑的通道对分 B_C 是多少?

（b）如果使用最小路由，最大跳数 H_{max} 是多少？平均跳数 H_{min} 是多少?

（c）现在我们要在每节点 $W_n = 128$ 个信号和背板 $W_s = 180$ 个信号的约束下封装这个拓扑。假设数据包大小 L = 200 位，信号频率为 800 MHz，并忽略线路长度。在这些约束下，最大通道宽度 w 是多少？这个网络是引脚还是对分带宽受限？路由器延迟 t_r 需要是多少才能确保 75 ns 的零负载延迟?

3.12 性能的物理限制。使用简单的思想，可以计算出网络直径和封装后节点间物理距离的可实现界限。首先，如果使用基数 k 的交换机（出度 k），N 个节点的网络的最小可能直径是多少？现在假设每个节点的体积为 V。在三维中，什么封装形状使节点间的最大距离最小？假设节点数量很大，这个距离是多少?

图 3.16 3^{rd} 阶立方体连接环。

3.13 任意网络的容量。编写一个程序，使用公式 3.9 中的优化问题来确定任意网络拓扑在均匀流量下的容量。程序的输入仅仅是拓扑对应的节点-弧关联矩阵，输出是网络的容量。为简单起见，假设所有通道具有相等的带宽，并且所有节点的弹出和注入速率也相等。最后，使用优化问题的这种替代形式可能更简单:

最小化 t

$$\text{subjectto} Ax_d = f_d \quad \text{and}$$

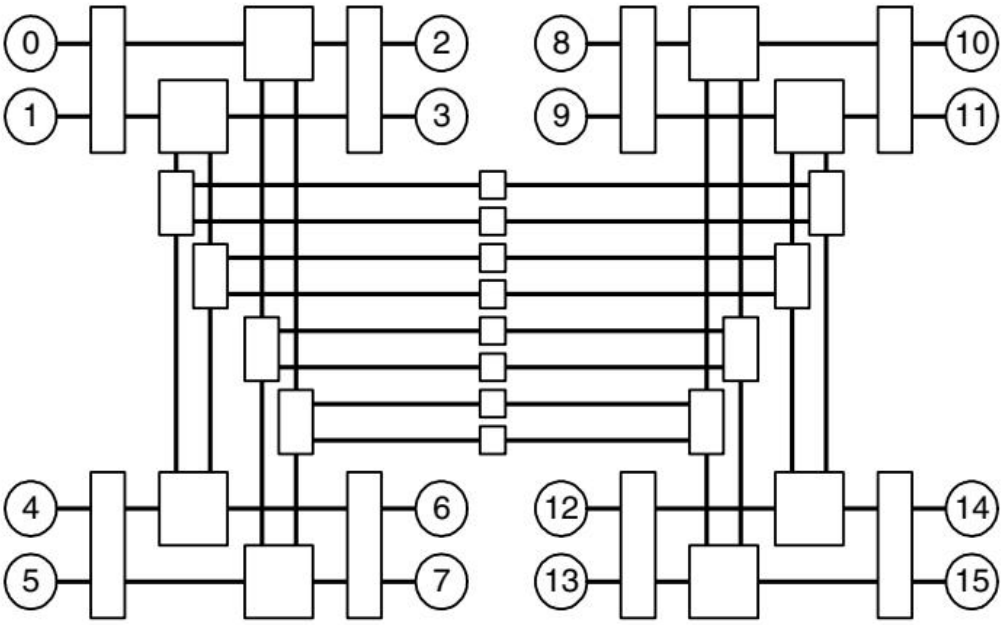


图 3.15 一个 16 节点、基数为 2 的胖树。所有矩形节点都是交换节点。

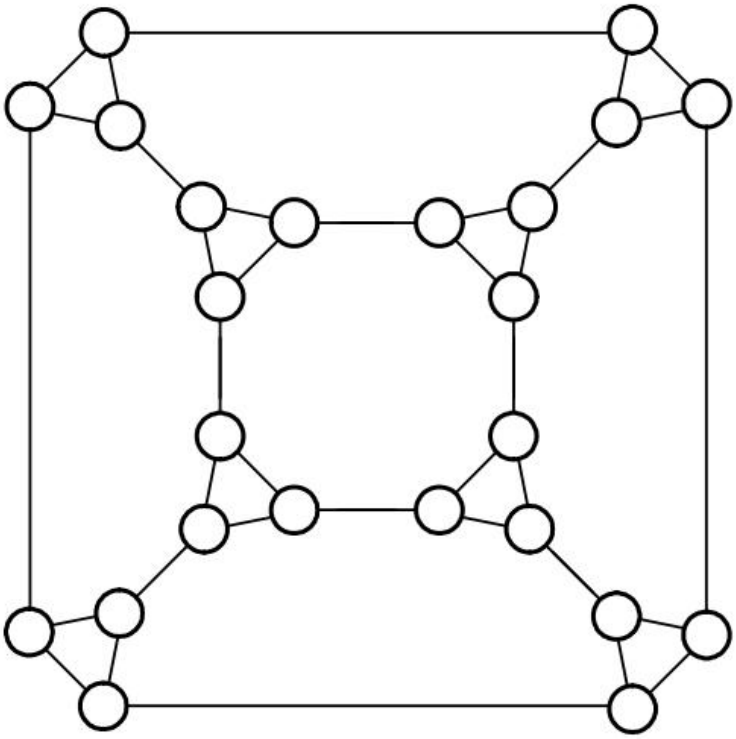


图 3.16 3^{rd} 阶立方体连接环。

$$\sum_{d \in N} x_{d,c} \leq t \quad \text{for all } c \in C$$

$$x_d \geq 0 \quad \text{for all } d \in N^*$$

通过引入一个额外变量 t，该优化问题可表示为一个线性规划，并可用专用工具或软件包（如 MATLAB）中提供的线性规划例程求解。

注释

- 1. 这种网络优化问题在优化文献中通常称为多商品流问题。例如参见 [6]。
- f0f_{0}
- f0/2f_{0} / 2
- 2. 这种带宽限制是由导线的趋肤效应电阻引起的。详细论述见 [55] 的第 3.3.4 节。
- 3. 注意，f0f_{0} 的信号速率对应于 f0/2f_{0} / 2 的最大频率。
- 4. 通过均衡信号 [54]，可以容忍高得多的频率相关衰减，从而可以驱动长得多的导线。
- 5. 这是集中化的一个例子，我们将在第 7.1.1 节中讨论。

第四章　蝶形网络

尽管多年来提出了众多拓扑结构，但实际构建的网络几乎都采用源自两大主要家族的拓扑：蝶形（k 元 n 飞）或环面（k 元 n 立方体）。本章将定义蝶形网络家族并探讨其特性。环面网络将在第 5 章讨论。

蝶形网络是典型的间接网络。对于具有度 $\delta = 2k$ 的交换机的 N 节点网络，蝶形拓扑具有最小直径 $H = \log_k N + 1$。尽管这一最优直径颇具吸引力，但蝶形网络有两个主要缺点。首先，基本蝶形网络没有路径多样性：从每个源节点到每个目的节点恰好只有一条路由。正如我们将看到的，这个问题可以通过向蝶形添加额外级来解决。这些额外级改善了网络的路径多样性，同时将网络直径保持在最优值的两倍以内。

其次，蝶形网络无法避免使用必须穿越至少机器直径一半的长线。由于在临界长度以上，导线的速度随距离呈二次方下降，这些长线使得蝶形网络在中等规模及更大规模的互连网络中吸引力降低。然而，蝶形网络的对数直径和简单路由使其及其变体成为许多应用中最受欢迎的互连网络之一。

4.1　蝶形网络的结构

我们已经见过许多 k 元 n 飞的例子。图 2.2 所示的简单网络是一个 4 元 3 飞（三级基数为四的交换机），图 3.1（b）显示了一个 2 元 3 飞，图 3.5 显示了一个 2 元 4 飞。许多其他“飞”也是可能的；例如，k 不必是 2 的幂。

一个 k 元 n 飞网络由 k^n 个源终端节点、n 级 k^{n-1} 个 $k \times k$ 交叉开关节点，以及最后的 k^n 个目的终端节点组成。我们采用将蝶形网络的源节点绘制在左侧、目的节点绘制在右侧的约定。除非另有说明，蝶形网络中的所有通道都是单向的，且从左向右流动。在大多数蝶形网络的实现中，源终端节点和目的终端节点在物理上是并置的，尽管它们通常被绘制为逻辑上分离。我们将每一对这样的源节点和目的节点视为单个终端节点，因此一个 k 元 n 飞总共有 $N = k^n$ 个终端。

我们用 n 位基数 k 的数 $\{d_{n-1}, d_{n-2}, \ldots, d_0\}$ 来标记每个终端节点以及每个交换节点发出的通道。前 n-1 位 $\{d_{n-1}, d_{n-2}, \ldots, d_1\}$ 标识交换机，最后一位 d_0 标识交换机上的终端。级间的连线对终端地址进行置换。在第 i-1 级和第 i 级之间（编号从 0 开始），连线交换数字 d_{n-i} 和 d_0。图 4.1 展示了 2 元 4 飞的节点标记和部分通道标记。为了区分不同级的节点和通道，将级号附加到其标签后，用句点分隔。例如，第 1 级中节点 $\{1, 0, 1\} = 5$ 在图中标记为 1.5。

根据这些置换，k 元 n 飞上的路由很容易理解。交换级 i 将终端地址的当前最低位数字 d_0 设置为任意值。然后，交换级 i-1 和级 i 之间的连线级将该值放置到位置 d_{n-i}。因此，第一级设置 d_{n-1}，第二级设置 d_{n-2}，依此类推，最后一级设置 d_0，此时 d_0 已

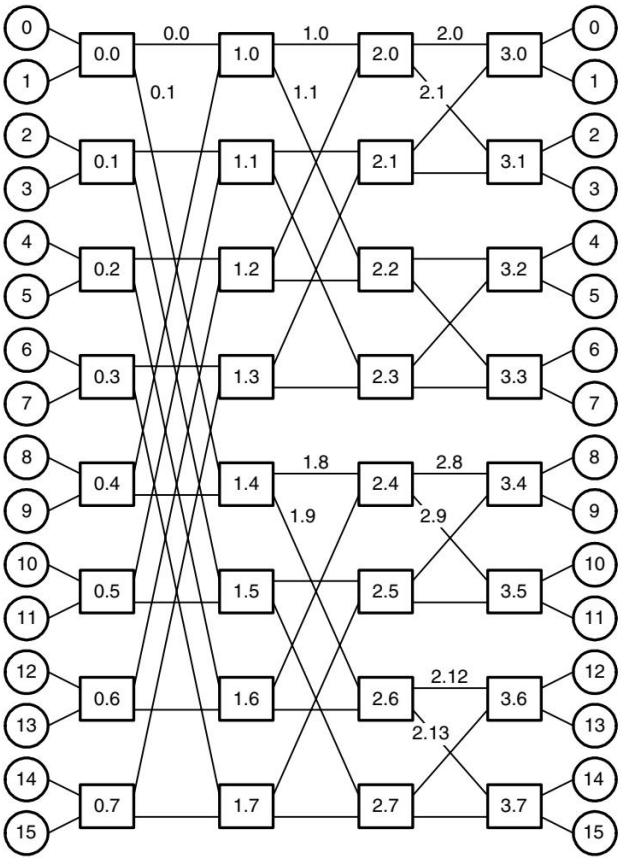


图 4.1 2 元 4 飞的标记。节点用其级号与地址拼接来标记。部分通道也用其级和地址标记。

在正确位置，因此无需进一步连线。

4.2 同构蝶形网络

多年来，人们提出了许多多级网络：洗牌交换网络、蝶形网络、数据操纵器网络和翻转网络等，仅举几例。然而，事实证明它们都是同一个网络 [195]，只是部分交换机重新编号。也就是说，它们是同构的。

网络 K 由其节点集和通道集定义：$K = (N^*, C)$。两个网络 $K_1 = (N_1, C_1)$ 和 $K_2 = (N_2, C_2)$ 同构，如果存在顶点的一个置换 π，使得边 $\{u, v\} \in C_1$ 当且仅当 $\{\pi(u), \pi(v)\} \in C_2$。作为这种同构的一个例子，图 4.2 展示了 2 元 3 飞的两种画法：作为洗牌交换网络和作为蝶形网络。洗牌交换网络在每一级之间具有相同的连线，执行洗牌置换。1 该置换将第 i 级中地址为 $\{d_2, d_1, d_0\}$ 的输出终端连接到第 i+1 级中地址为 $\{d_1, d_0, d_2\}$ 的输入终端。

图 4.2 所示的两个网络通过一个非常简单的映射同构：交换节点 11 和 12 的位置简单地互换。为使映射清晰，两个网络中的这些交换节点均采用蝶形编号。理解同构的最简单方法是考虑从输入终端 $\{a_2, a_1, a_0\}$ 到输出终端 $\{b_2, b_1, b_0\}$ 的路由。表 4.1 列出了在两个网络中访问的交换端口序列。除了第 1 级交换机的地址相反外，序列完全相同：在蝶形网络中为 $\{b_2, a_1\}$，而在洗牌交换网络中为 $\{a_1, b_2\}$。因此，两个网络中的路由

访问相同的交换机，只是对第一级交换机进行了这种简单的重新标记。

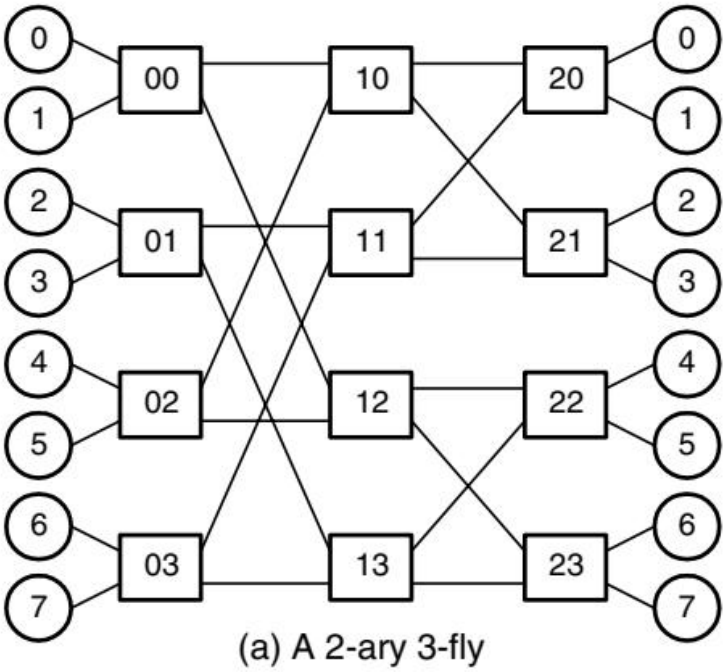


图 4.2 以两种方式绘制的 2 元 3 飞：（a）作为传统蝶形网络，（b）作为洗牌交换网络。唯一的区别在于交换节点 11 和 12 的位置。

（b）绘制为洗牌交换网络的 2 元 3 飞

表 4.1 蝶形和洗牌交换网络的路由

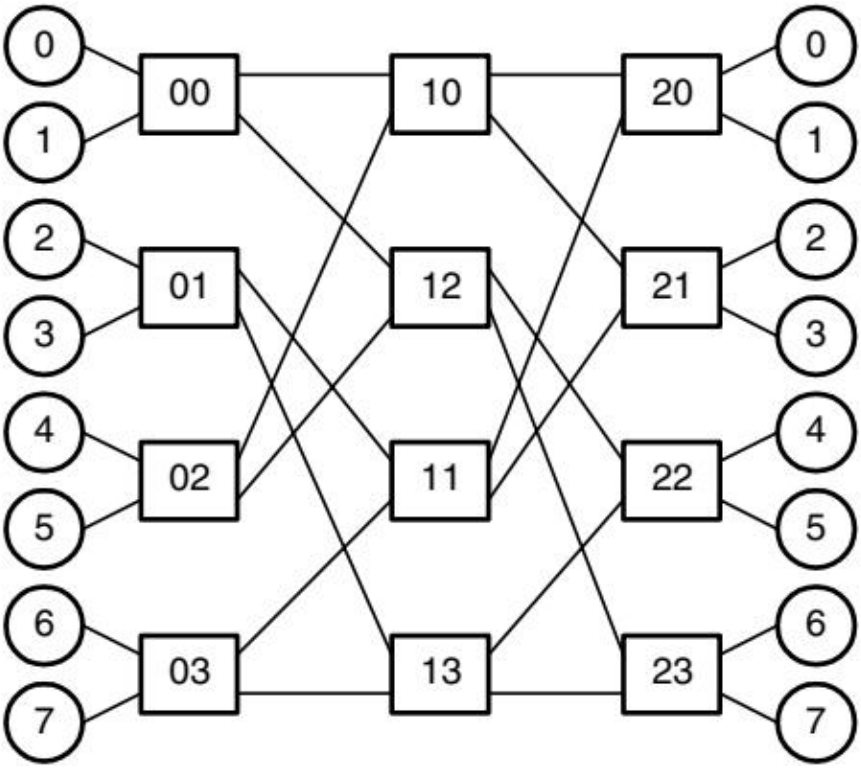
Stage	Butterfly	Shuffle Exchange
Stage 0 input	$\{a_2, a_1, a_0\}$	$\{a_2, a_1, a_0\}$
Stage 0 output	$\{a_2, a_1, b_2\}$	$\{a_2, a_1, b_2\}$
Stage 1 input	$\{b_2, a_1, a_2\}$	$\{a_1, b_2, a_2\}$
Stage 1 output	$\{b_2, a_1, b_1\}$	$\{a_1, b_2, b_1\}$
Stage 2 input	$\{b_2, b_1, a_1\}$	$\{b_2, b_1, a_1\}$
Stage 2 output	$\{b_2, b_1, b_0\}$	$\{b_2, b_1, b_0\}$

4.3 性能与封装成本

正如我们在第 3 章中所见，拓扑结构的特征在于其吞吐量、延迟和路径多样性。本节讨论蝶形网络的吞吐量和延迟，而路径多样性将在第 4.4 节中讨论。所有性能比较均使用第 3.4 节中介绍的两级封装模型。

一个 k 元 n 飞网络具有 k^n 个输入终端、k^n 个输出终端和 nk^{n-1} 个交换节点。每个交换节点的度为

$$\delta_{\text{fly}} = 2k. \tag{4.1}$$



（b）绘制为洗牌交换网络的 2 元 3 飞

通道对分带宽为

$$B_{C,\text{fly}} = \frac{N}{2} \tag{4.2}$$

其中 N 为偶数。N 为奇数的情况将在练习 4.7 中讨论。

尽管蝶形网络不是边对称网络，但每一级之间的通道是对称的，这足以使第 3.3.1 节中推导的最大通道负载界限是紧致的（即在均匀流量下所有通道负载相等）。因此，通道负载的公式仅需要平均跳数。

在蝶形网络中发送的任何数据包的跳数无论源和目的地如何都相同，并且简单地等于级数加一，2 即

$$H_{\text{min,fly}} = n + 1.$$

使用此跳数和公式 3.5，通道负载为

$$\gamma_{\text{U,fly}} = \frac{NH_{\text{min,fly}}}{C} = \frac{k^n(n+1)}{k^n(n+1)} = 1.$$

正如我们刚刚针对均匀流量所展示的，一半的流量穿过网络对分，每个对分通道的负载为 $\gamma_{fly,uniform} = 1$。对于反向流量模式，即节点 i 向节点 N - i - 1 发送数据包，或任何其他模式，其中网络一半中的每个节点向另一半中的节点发送数据包，所有流量都将穿过一个对分通道，负载增加到 $\gamma_{fly,rev} = 2$。平均与最坏情况对分流量之间的这种

差异并非蝶形网络独有。然而，正如我们将在第 4.4 节中看到的，蝶形网络中路径多样性的缺乏可能通过将大部分流量集中到单个通道上而将 γ 提高到高达 $\sqrt{N}$。

在两级封装层次结构下，蝶形网络的通道宽度通过将公式 4.1 和 4.2 代入公式 3.14 计算得出

$$w_{\text{fly}} \le \min\left(\frac{W_n}{\delta_{\text{fly}}}, \frac{W_s}{B_{C,\text{fly}}}\right) = \min\left(\frac{W_n}{2k}, \frac{2W_s}{N}\right).$$

然后，在均匀负载下，因此 $\gamma = 1$，理想吞吐量为

$$\Theta_{\text{ideal,fly}} = \frac{f w_{\text{fly}}}{\gamma} = \min\left(\frac{B_n}{2k}, \frac{2B_s}{N}\right) \tag{4.3}$$

其中 B_n 和 B_s 分别是节点带宽和对分带宽。

对于大多数蝶形网络，目标首先是获得最大吞吐量，其次是使消息延迟最小化。为实现这一目标，人们选择使网络受对分带宽限制的最大 k 值。这发生在

$$k = \left\lfloor \frac{NB_n}{4B_s} \right\rfloor. \tag{4.4}$$

4.3 性能与封装成本此 k 值给出最小直径，从而最小化延迟中的 H 部分，同时最大化通道带宽。这也最大化了理想吞吐量并最小化了串行化延迟。任何小于此值的 k 都不会提高吞吐量，只会因额外的跳数而增加延迟 3。

对于 k 元 n 飞，延迟的所有三个组成部分 $(T_s, T_r,$ 和 $T_w)$ 都受到 k 和 n 选择的影响。蝶形网络的串行化延迟为 $T_s = L/b$，由公式 4.3 中给出的带宽决定。T_r 简单地是 $t_r H_{\text{min}} = t_r(n+1)$。线路延迟 T_w 的精确值取决于 k 元 n 飞的封装方式。任何规模的 k 元 n 飞都需要许多长通道。第一级中一半的通道跨越到机器的另一半，第二级中一半的通道跨越到不同的象限，依此类推。在许多蝶形网络中，这些通道长于临界线路长度，如果要全速运行，必须在线路中插入中继器。

为了说明本节描述的蝶形网络性能指标，考虑一个 $N = 2^{12}$ 节点的 k 元 n 飞，它要路由 L = 512 位的数据包，并封装在具有线路对分 $W_s = 2^{14}$、节点引脚数 $W_n = 2^8$ 和通道频率 f = 1 GHz 的技术中。我们还假设路由器延迟为 $t_r = 10$ ns，并忽略线路延迟。根据公式 4.4，我们计算得到 $k = \frac{2^{12}\times 2^8}{4\times 2^{14}} = 16$。因此，为了获得最佳性能，这将是一个 16 元 3 飞。当 $\delta = 2k = 32$ 时，每个通道的宽度最多为 $w = \frac{W_n}{\delta} = 8$ 位。这给出了一个跳数 H = 4、理想吞吐量 $\Theta_{ideal} = b/\gamma = 8$ Gbits/s、平均延迟 T = 104 ns 的网络。该蝶形网络在所有 k 和 n 选择下的性能总结在表 4.2 中。

表 4.2 4,096 节点蝶形网络的吞吐量与延迟

n	k	ws	wn	w	Θ_{ideal} (Gbits/s)	$T_h(ns)$	$T_S(ns)$	T (ns)
1	4,096	8	0.03125	0.03125	0.03125	20	16,384	16,404
2	64	8	2	2	2	30	256	286
3	16	8	8	8	8	40	64	104
4	8	8	16	8	8	50	64	114
6	4	8	32	8	8	70	64	134

12	2	8	64	8	8	130	64	194	

4.4 路径多样性与额外级数

在 k 元 n 飞网络中不存在路径多样性：$|R_{xy}| = 1 \forall x, y \in N$。当流量模式非均匀时，由于通道间的负载不平衡，这可能导致吞吐量显著下降。

这种负载不均衡可以通过在网络中增加额外的级来缓解 [166]。为了说明增加级如何减轻这个问题，我们首先来看流量是如何集中到瓶颈通道上的。

设 k = 2，n 为偶数，且 m = n/2。使用第 4.1 节描述的地址格式，假设数据包从节点 $\{x_m, \ldots, x_2, a_m, \ldots, a_1, x_1\}$ 发送到节点 $\{b_m, \ldots, b_1, y_m, \ldots, y_0\}$，对于所有 $x \in \mathcal{Z}(2^m)$，其中 $y = \pi(x)$ 是 $\mathcal{Z}(2^m)$ 元素的任意排列。为简洁起见，我们将用单个字符表示 m 位地址，例如 $x = \{x_m, \ldots, x_1\}$。

在路由的第一级，来自节点 x 的数据包经过交换机 $\{x_m, \ldots, x_2, a\}$。在下一级路由中，同一个数据包经过交换机 $\{b_m, x_{m-1}, \ldots, x_2, a\}$。这一过程持续到第 m 级，此时数据包位于交换机 $\{b_m, \ldots, b_2, a\}$，以及第 $(m+1)$ 级，此时数据包位于交换机 $\{b, a_{m-1}, \ldots, a_1\}$。由于第 m 级交换机不是 x 的函数，我们模式中的所有 $2^m = \sqrt{N}$ 个数据包都经过这个交换机。第 $(m+1)$ 级交换机也是如此，并且由于这两个交换机之间只有一条通道，该通道上的负载为 $\sqrt{N}$。找到这个过载通道的地址留作习题 4.4。如果我们把蝶形网络画成二维的，这种流量集中就很容易可视化，如图 4.3 所示。该图表明，一个 k 元 n 飞网络可以看作 2^m 个 y 轴交换平面，每个平面每级有 2^m 个交换机，随后是同样数量的 z 轴交换平面，每级也有相同数量的交换机。这是因为前 m 级只调整最高 m 位交换机地址位，因此只沿 y 维度（垂直方向）移动数据包。接下来的 m 级只调整最低地址位，沿 z 维度（进入页面方向）移动数据包。5

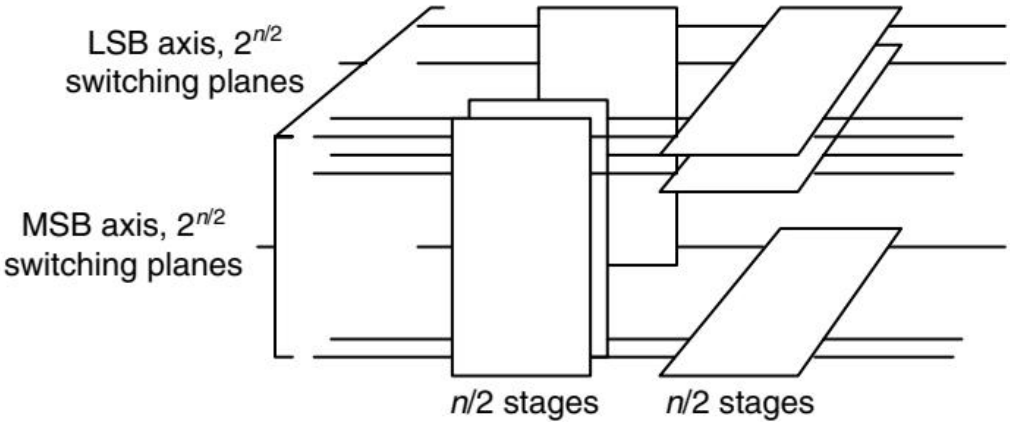


图 4.3 蝶形网络的二维视图。前 m 级解析最高 m 位交换机地址位，将数据包路由到图中 y 轴上的正确位置。接下来的 m 级解析剩余的 m 位交换机地址位，将数据包路由到 z 轴上的正确位置。

要了解蝶形网络如何变得拥塞，考虑当连接到第一个 y 轴网络的所有输入端口试图向连接到第一个 z 轴网络的所有输出端口发送数据包时会发生什么。所有 2^m 个数据包都必须经过连接这两个网络的单一通道，这正好对应于我们上面描述的排列流量。

现在假设我们在网络前端增加 m 级，并且这些级的连线与网络中最后 m 级完全相同——即它们调整交换机地址的最低 m 位。这些额外的级提供了 2^m 的路径多样性。即 $|R_{xy}| = 2^m \forall x, y \in N$。为了看到这一点，考虑将数据包从 $\{s, t\}$ 路由到 $\{u, v\}$。新的 m 级可以将数据包从 $\{s, t\}$ 路由到 $\{s, i\}$，其中 $i \in \mathcal{Z}(2^m)$ 是任意的。通过适当的路由策略，可以在 2^m 个中间地址中均匀选择 i，从 (s, t) 到 (u, v) 的流量因而均匀分布到第 $m-1$ 级的 2^m 条中间通道 (s, i) 上。然后，接下来的 $\frac{n}{2}$ 级（原始网络的前半部分）将数据包从 $\{s, i\}$ 路由到 $\{u, i\}$。最后，最后 $\frac{n}{2}$ 级将数据包从 $\{u, i\}$ 路由到 $\{u, v\}$。注意，我们的病态流量模式，从 $\{x_m, \ldots, x_2, a, x_1\}$ 到 $\{b, y\}$，在这个网络中不会集中到单一通道上，因为额外的级将这些流量均匀分布在通道 $\{x_m, \ldots, x_2, a_m, i\}$ 和随后的 $\{b, i\}$ 上。

图 4.4 展示了向 2 元 3 飞网络添加一个额外级如何赋予网络 2 的路径多样性。在未修改的网络中，位反转流量——即从 $0 \to 0$，$4 \to 1$ 等——会将所有流量集中到第二级输出的一半链路上。添加额外的输入级消除了这个问题，使位反转流量的通道负载均匀为 $\gamma = 1$。

乐观的读者此时可能会认为问题已经解决：通过向蝶形网络添加 m 级，我们可以完美地实现网络的负载均衡。不幸的是，情况并非如此。仍然存在两个问题。首先，由于网络的通道二分宽度 $B_{C,fly}$ 小于 N，网络无法处理所有节点都跨越二分宽度的流量（例如，节点 i 发送到 N - i - 1）。其次，对于更大的网络，n > 3，流量集中的问题仍然存在，但集中因子为 $2^{n/4}$ 而不是 $2^{n/2}$。

为了看到集中问题，我们将用 r = n/4 位字符串来表示交换机地址（假设 n = 2m = 4r，其中 m 和 r 为某些整数）。考虑从 $\{x_r, \ldots, x_2, a, x_1, s, t\}$ 到 $\{b, y, u, v\}$ 的路由，其中 $y = \pi(x)$ 是 $\mathcal{Z}(2^r)$ 元素的排列。经过额外的输入级后，我们位于通道 $\{x_r, \ldots, x_2, a, x_1, i, j\}$。然后我们的中间级将我们带到 $\{b, y, i, j\}$。由于地址的最低两位在中间级期间是固定的，我们重现了与原始对手相同的集中。然而，在这种情况下，总负载为 $2^r = 2^{n/4}$。直观地看，注意网络的中间级等价于 2^m 个独立的网络（由低地址位索引），每个网络有 2^m 个端口。我们只是将问题从整个网络转移到了这些多个中间网络中。

图 4.5 给出了一个带有 m 个额外级的 k 元 n 飞的二维视图。这些额外级形成了一组 2^m 个 m 级 z 轴网络，每个网络每级有 2^{m-1} 个开关。这些网络在正确路由的情况下，将流量均匀地分布到 z 维度上，防止了从 y 轴网络到 z 轴网络的交叉处发生拥塞。

然而，拥塞仍然可能完全发生在 y 轴内部。例如，我们可以将每个 y 轴网络分成两个 m/2 级网络，并将流量集中在这两个网络的交界处。

蝶形网络的负载均衡问题可以通过复制网络的所有 n 级来解决。得到的 2n 级网络相当于两个背靠背的 n 级蝶形网络，称为 Beneš 网络（发音为 Ben-ish）。它的通道二分宽度为 N，因此可以处理像反转流量这样的模式。它还通过将每对节点之间的流量分配到 N 条替代路径上，完美地实现了负载均衡。事实上，Beneš 网络是无阻塞的，我们将在第 6.4 节中讨论。

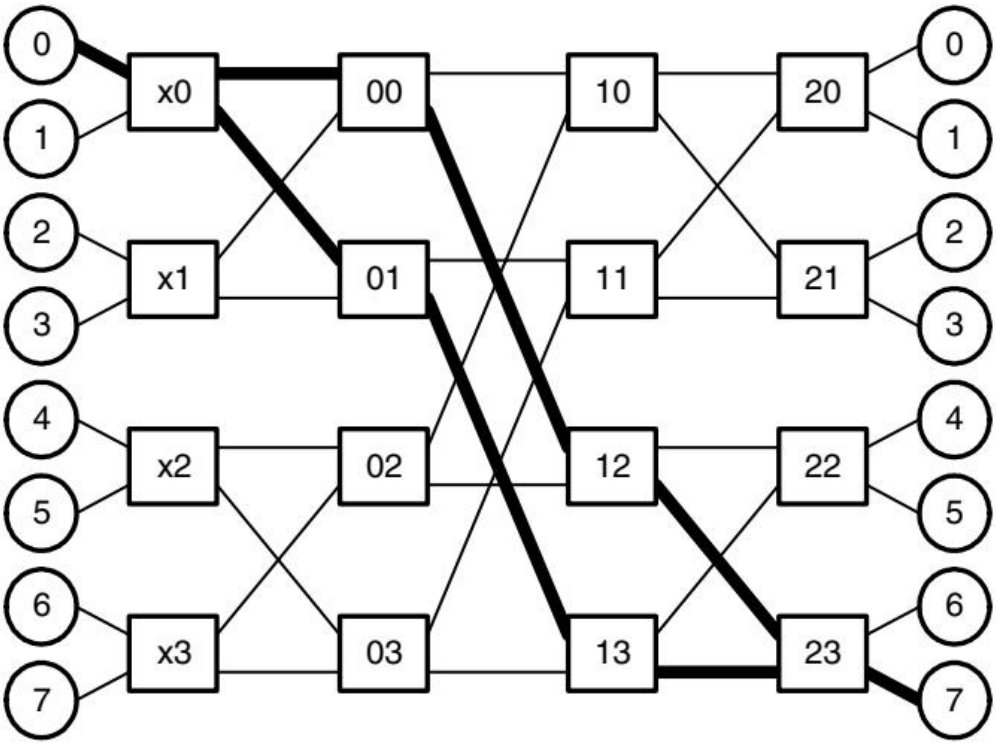


图 4.4 一个带有额外级的 2 元 3 飞在所有节点对之间的路由上提供了 2 的路径分集。节点 0 和 7 之间的两条路径以粗体显示。

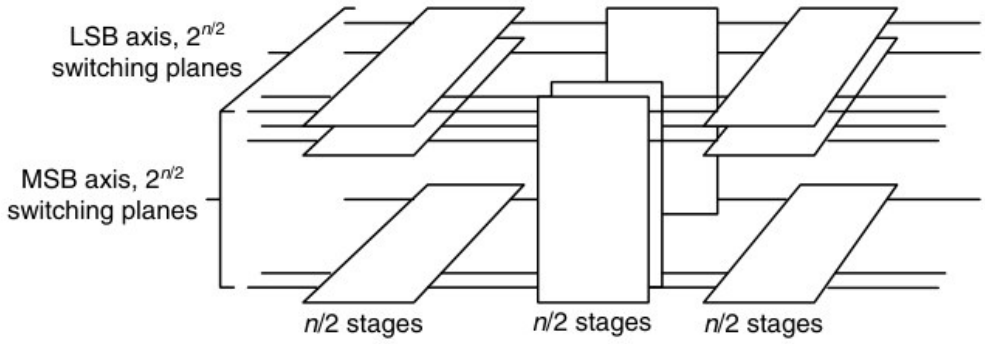


图 4.5 带额外级的蝶形网络二维示意图

4.5 案例研究：BBN Butterfly

Bolt、Beranek 和 Neumann（BBN）高级计算机系统公司通过使用具有 k 元 n 飞拓扑的互连网络，构建了一系列共享内存并行处理器。这些机器中的第一台于 1981 年推出，是 BBN Butterfly。Butterfly 最多连接 256 个处理节点，每个节点由一个 8 MHz 的 Motorola 68000 微处理器和 512 KB 内存组成。1987 年，Butterfly GP-1000 的处理器升级为更快的 Motorola 68020，内存增加到每节点 4 MB。1989 年，TC-2000 再次升级，采用 Motorola 88100 RISC 处理器，配有 88200 缓存和内存管理芯片，每节点内存高达 16 MB。Butterfly 的每个处理器都能够通过互连网络读写任何其他节点的内存。这不是一台缓存一致的机器。最初的 Butterfly 和 GP-1000 没有缓存，而 TC-2000 只允许缓存本地数据。Monarch（第 23.4 节）被设计为 TC-2000 的后续产品，但从未完成。

远程内存访问通过一个带有额外级的 4 元 n 飞网络进行，如图 4.67 所示，用于 64 节点机器。图中每个通道宽 4 位，以 8 MHz（也是 68000 处理器的时钟频率）运行，通道带宽为 b = 32 Mbits/s。该网络包含 4 级 4×4 开关，而不是没有额外级时所需的 $\log_4(64) = 3$ 级。额外级提供了路径分集，并提高了对抗性流量模式下的吞吐量，如第 4.4 节所述。使用基数 4 开关时，额外级在任何源和目的地之间提供了四条边不相交的路径。当数据包传输时，网络接口随机选择通过交换机的四条路径之一。8 一个 4 位掩码使系统能够在一条或多条路径发生故障时禁用 4 条路径的任意组合。

这些 4×4、4 位宽的开关是在一块 12 英寸高的 PC 板上使用分立 TTL 逻辑实现的，通道接口使用发射极耦合逻辑（ECL）线路驱动器和接收器。开关卡和处理器/内存卡以任意模式插入 19 英寸机架中。所有网络布线均通过卡正面的带状电缆连接实现。

4.5 案例研究：BBN Butterfly

通过交换机的延迟为三个 125 ns 时钟，其中一个用于消耗指定交换机输出端口的路由半字节，因此图 4.6 的 4 级网络具有十二个 125 ns 时钟的端到端零负载延迟，即 $1.5\mu s$。均匀流量下的吞吐量（忽略路由开销）为每节点 32 Mbits/s。然而，对抗性流量模式可能导致吞吐量低得多。（见练习 4.6。）

该网络采用了类似于我们在第 2 章简单网络中使用的丢弃流控。当多个数据包请求同一个输出端口时，一个数据包被授予该端口，其余数据包被丢弃。丢弃的数据包在短暂超时后重新传输。每次数据包重新传输时，它都会再次随机选择到达目的地的四条路径之一。这使得同一组数据包之间发生多次碰撞的可能性不大。

4.6 文献注释

多年来，蝶形拓扑以多种形式出现。早期的例子包括 Stone 的混洗交换网络 [175]、Feng 的数据操纵器 [65]、Lawrie 的 omega 网络 [110]、Batcher 的翻转网络 [15] 以及 Pease 的间接二进制立方体拓扑 [144]。Wu 和 Feng 随后证明了所有这些网络都与我们在本章研究的蝶形网络同构 [195]。Kruskal 和 Snir 对蝶形网络和类蝶形拓扑进行了进一步比较 [106]。此外，一些商业和学术系统围绕蝶形网络构建，例如 NYU Ultracomputer[75]、

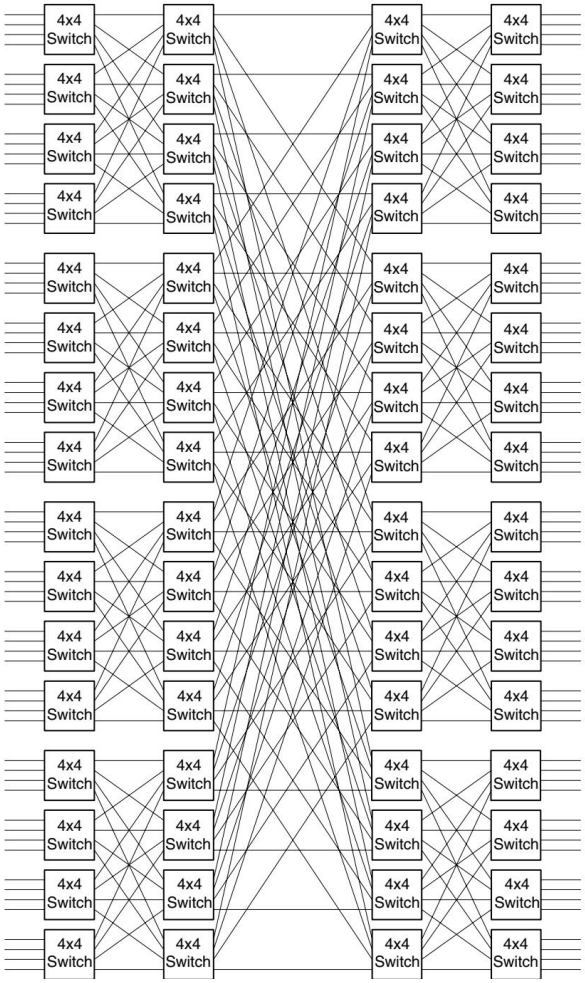


图 4.6 BBN Butterfly（1981）使用了一个带有额外级开关的 4 元 n 飞网络将处理器连接到内存。每个通道的带宽为 32 Mbits/s，交换机丢弃消息以解决争用。该图显示了一个 4 元 3+1 飞，它用 4 级 4×4 开关连接 64 个节点。

本章研究的 BBN Butterfly[85] 以及最近的 NEC Cenju-4 计算机 [133]。

4.7 练习

4.1 蝶形网络与混洗交换网络之间的同构。一个基数为 2 的混洗交换网络由 $n = \log_2 N$ 个交换级组成，每一级以完美混洗模式连接。完美混洗将第 i 级中地址为 $\{a_{n-1}, a_{n-2}, \ldots, a_0\}$ 的输出端连接到第 $i+1$ 级中地址为 $\{a_{n-2}, \ldots, a_0, a_{n-1}\}$ 的输入端。证明具有相同节点数的 2 元 n 飞与基数为 2 的混洗交换网络同构。（图 4.2 给出了一个例子。）

4.2 位反转下的吞吐量。当路由位反转流量模式时，4 元 2 飞达到的容量比例是多少?

4.3 封装蝶形拓扑。你需要使用 $W_n = 128$ 和 $W_s = 1024$ 的封装技术连接 2^{10} 个节点。选择一个蝶形拓扑，首先最大化网络吞吐量，然后最小化延迟。这个吞吐量和相应的延迟是多少？假设 L = 512 位，f = 1 GHz，$t_r = 10$ ns。同时，忽略线延迟。

4.4 对抗性流量下的过载通道。在第 4.4 节中，发现一种排列流量模式会将特定通道负载增加到 $\sqrt{N}$。使用第 4.1 节中描述的蝶形网络通道地址表示法，确定这个高度过载通道的地址。

4.5 相同的额外级。考虑图 4.4 中的网络。假设你添加第二个冗余级，其布线方式与网络的第一级完全相同。这会增加路径多样性吗？最大通道负载会降低吗?

4.6 BBN Butterfly 上的最坏情况流量。描述一种导致最坏情况吞吐量（即最大化 γ_{max}）的流量模式，在图 4.6 所示的 BBN Butterfly 网络上。一般来说，具有奇数级数的蝶形网络的最坏情况吞吐量是多少?

4.7 封装奇数基数蝶形网络。一个 3 元 2 飞网络的最小对分通道数 B_C 是多少？请记住，由于本例中 N = 9，对分所分隔的两组节点的大小将相差 1。这种对分是否代表了网络的良好封装？如果不是，请提出一种替代切割或切割集，将网络划分为大小大致相等的部分，并且确实对应于高效的封装。

4.8 混合基数蝶形网络。绘制一个 12 节点蝶形网络的拓扑，首先使用 2 个交换级，然后使用 3 个交换级。每一级应包含具有相同基数的交换机，但基数可以因级而异。同时，不要留下任何未使用的交换机端口。（例如，不要用 4 个未连接的终端来实现一个 16 节点蝶形网络。）这两种拓扑与 12 的因式分解有何关系?

4.9 受损蝶形网络。考虑一个 2 元 n 飞网络，其某一级中有一个故障交换机节点。网络的错误恢复机制将此故障交换机从网络中移除，消息不再允许通过它路由。如果没有额外的级，这个故障显然会导致网络断开——某些源-目的对之间将没有路径。对于一个具有 x 个额外级的蝶形网络，其中 $0 \le x \le n$，在单个交换机故障后网络保持连通的概率是多少？对于 x = 1，在两个交换机故障后网络仍然连通的概率是多少?

4.10 蝶形网络布局中的线长。计算在平面布局的 2 元蝶形网络中，消息经过的平均线长。假设所有节点对齐在二维网格上，节点间距为 10 厘米。例如，蝶形网络的第一级位于网格的第一列，需要 $10 \times 2^{n-1}$ 厘米的垂直空间。下一级布置在下一个网格列中，与第一级水平间隔 10 厘米，依此类推。为简单起见，从每个交换机节点的中心测量

距离，并忽略到终端节点的布线。假设 n 为偶数，比较平面布局与图 4.3 中使用的布局的线长。对于大型网络，线长缩短了多少？4.7 练习

注释

- 1. 这是洗牌时对一副牌进行的排列。牌 0 保持在位置 0，牌 1 移到位置 2，牌 2 移到位置 4，依此类推。
- 2. 这假设源和目的地是与交换节点不同的终端节点。如果源和目的地节点是交换节点，则跳数为 n–1n - 1。
- 3. 在第 7 章中，我们将介绍对蝶形网络进行位切片或通道切片的选项，这允许在保持网络对分带宽受限的同时减小直径。特别是通道切片，允许串行化延迟与直径之间进行权衡。
- 4. $\mathcal{Z}(i)$ 是整数集合 $\{0, \ldots, i-1\}$ 的简写。
- 5. 蝶形网络的这种二维布局已被用于高效地封装小型蝶形网络。采用这种方法时，前半部分级被封装在一组垂直电路卡上，而剩余的级则封装在水平电路卡上。这些卡通过如图 4.3 所示的中间板连接在一起。
- 6. 经过冗余级传输的数据包的输出端口可以随机选择，也可以通过选择任何空闲输出端口来选择。后一种策略通常称为偏转路由。
- 7. TC-2000 使用了略有不同的网络，采用 8 元 n 飞拓扑。这里我们将讨论限制在原始 Butterfly 和 GP-1000 上使用的基数 4 网络。
- 8. 这是 oblivious routing 的一个例子，我们将在第 9 章讨论。

第五章　环面网络

环面和网格网络，即 k 元 n 立方体，将 $N = k^n$ 个节点以规则 n 维网格形式排列，每个维度有 k 个节点，最近邻之间有通道。它们涵盖从环（n = 1）到二元 n 立方体（k = 2，也称为超立方体）的一系列网络。

这些网络因多种原因而具有吸引力。这种规则的物理排列与封装约束很好地匹配。在低维度下，环面具有均匀的短线，允许无需中继器的高速操作。环面中的逻辑最小路径几乎总是物理最小路径。这种物理守恒使得环面和网格网络能够利用通信节点之间的物理局部性。对于本地通信模式，例如每个节点向第一维中的邻居发送，延迟远低于随机流量，吞吐量远高于随机流量。另一方面，蝶形网络无法利用这种局部性。

环面具有良好的路径多样性，即使在排列流量下也能实现良好的负载均衡。此外，由于环面或网格网络中的所有通道都是双向的，它们可以利用双向信号传输，从而更有效地利用引脚和线路。

环面网络的一个缺点是它们比对数网络具有更大的跳数。这使得它们的延迟略高于最小界限，并增加了网络的引脚成本。然而，请注意，为了路径多样性，跳数的增加是必要的。

设计者可以通过选择网络的维度 n 来确定环面网络的属性。正如我们将在第 5.2.1 节中看到的，网络的吞吐量随维度单调增加，直到网络变得对分受限。超过该点后，吞吐量不再增加。网络延迟在两个极端都很高。在低维度下，延迟主要由高跳数 H 决定，而在高维度下，串行化延迟 T_s 占主导。最小延迟通常在相对较低的维度下实现，通常在 2 到 4 之间。为了最小化延迟和线长，我们通常选择 n 为使网络对分受限的最小维度。

5.1　环面网络的结构

一个 n 维、基数为 k 的环面，即 k 元 n 立方体，由 $N = k^n$ 个节点组成，这些节点排列在一个 n 维立方体中，每个维度上有 k 个节点。作为一个直接网络，这 N 个节点中的每一个同时充当网络的输入终端、输出终端和交换节点。每个节点被分配一个 n 位基数为 k 的地址 $\{a_{n-1}, \ldots, a_0\}$，并通过一对通道（每个方向一个）连接到所有地址中恰好有一位数字相差 $\pm 1 (\text{mod } k)$ 的节点。这要求每个节点在每个维度上有 2 条通道，总共 2nN 条通道。环面是正则的（所有节点具有相同的度数），并且也是边对称的，这有助于改善通道间的负载均衡。

我们已经看到了许多 k 元 n 立方体的例子。图 3.1（a）显示了一个 3 元 2 立方体（每个维度有三个节点的二维环面），而一个 8 元 1 立方体（一个 8 节点环）如图 3.3 所示。一般来说，任意 k 元 n 立方体可以通过迭代添加维度来构造，如图 5.1 所示。一个 k

元 1 立方体（图 5.1[a]）仅仅是一个 k 节点环。将这些 1 立方体中的 k 个连接成一个循环，就添加了第二个维度，形成一个 k 元 2 立方体（图 5.1[b]）。这个过程一次一个维度地继续，组合 k 个 k 元 $(n-1)$ 立方体来形成一个 k 元 n 立方体（图 5.1[c]）。

网格网络是一种环面网络，其中每个方向上从地址 a_{k-1} 到地址 a_0 的连接被省略。例如，图 5.2 比较了一个 4 元 2 立方体（环面）和一个 4 元 2 网格。网格网络具有相同的节点度数，但二分通道数量是相同基数和维度的环面的一半。尽管网格具有非常自然的二维布局，使通道长度保持较短，但它放弃了环面的边对称性。这可能导致许多流量模式下的负载不均衡，因为中心通道的需求可能显著高于边缘通道。正如我们将在第 5.3 节中看到的，通过使用折叠布局，环面的最大通道长度可以减少到网格的两倍。

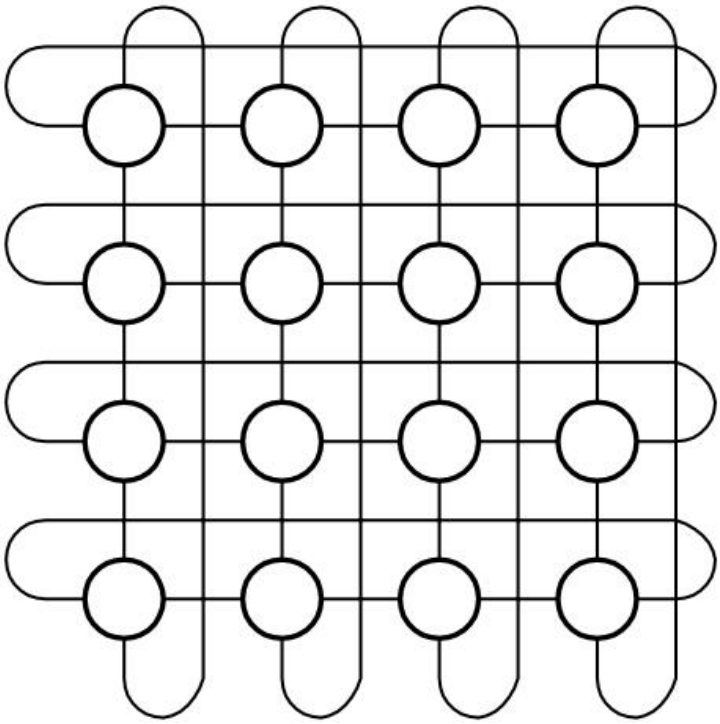


图 5.1 三个 k 元 n 立方体网络：（a）一个 4 元 1 立方体；（b）一个 4 元 2 立方体，通过将 4 个 1 立方体中的相同元素连接成环构造而成；（c）一个 k 元 n 立方体，通过将 k 个 k 元 $(n-1)$ 立方体中的相同节点连接成循环构造而成。

环面可以是单向的，在每个维度上只有从 a_i 到 a_{i+1} 方向的通道，也可以是双向的，在相连节点之间两个方向都有通道。网格也可以做成单向或双向的；然而，单向网格必须在网络的行之间交替通道方向以保持网络完全连接。除非采用同时双向信号传输，否则双向网络需要两倍的引脚数和两倍的线二分带宽。即使成本增加，双向网络通常更受青睐，因为它们具有更低的跳数 H 和更大的路径多样性。除非另有说明，我们考虑环面或网格为双向的。

环面网络的每个维度可以有不同的基数。例如，图 5.3 展示了一个 2，3，4 元 3 网格，其在 y 维度上的基数为 2，在

（a）一个 4 元 1 立方体

（c）从 k 个 k 元（n-1）立方体构造一个 k 元 n 立方体

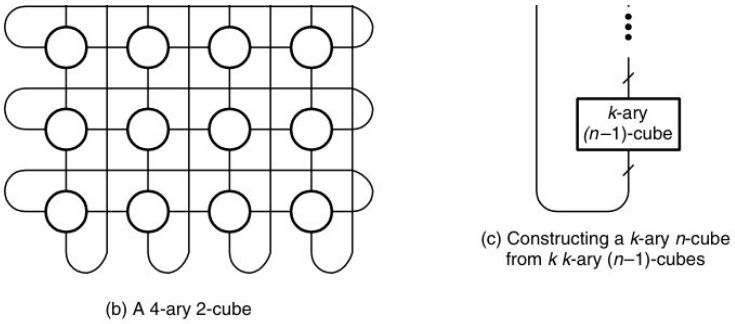


Figure 5.1 Three k-ary n-cube networks: (a) a 4-ary 1-cube; (b) a 4-ary 2-cube that is constructed from 4 1-cubes by connecting like elements in a ring; and (c) a k-ary n-cube that is constructed by connecting like nodes of k k-ary $(n-1)$-cubes in a cycle.

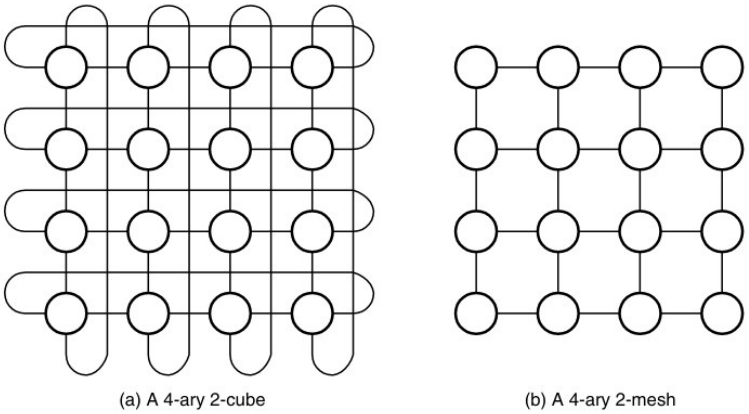
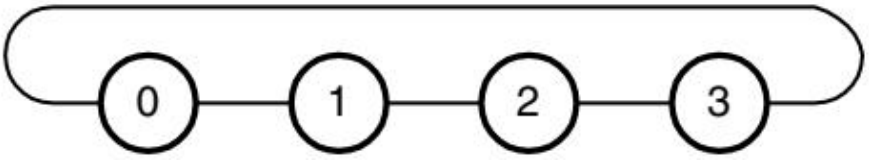
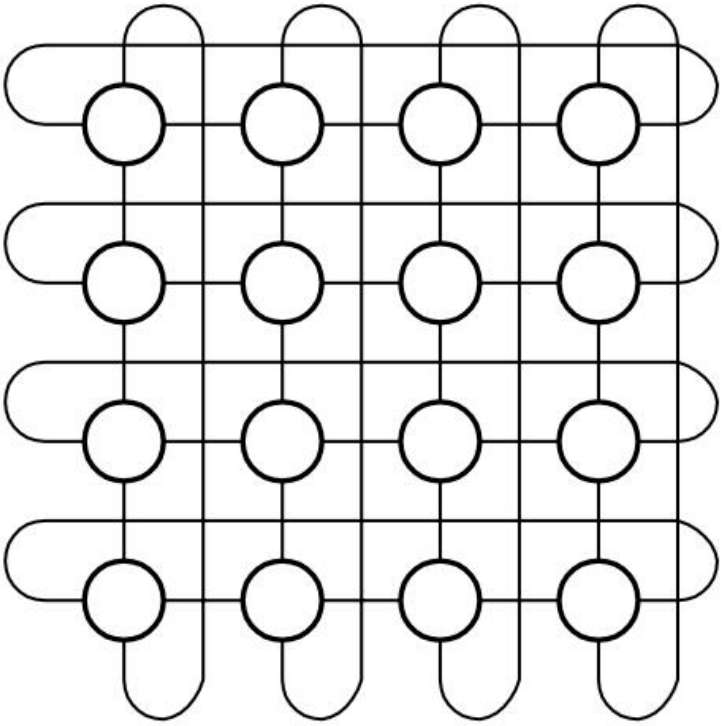
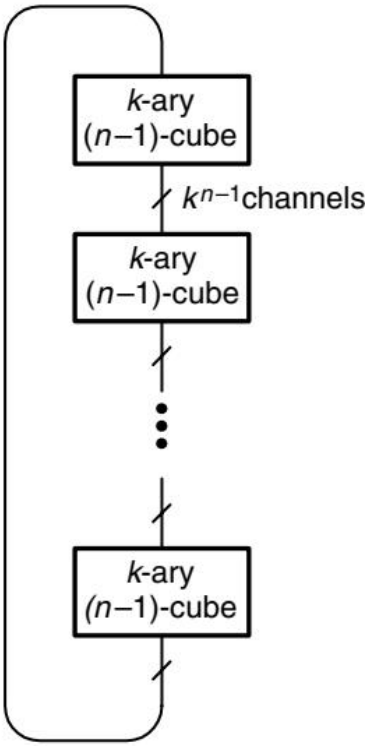


图 5.2 环面网络与网格网络的拓扑比较



（a）一个 4 元 1 立方体





（c）从 k 个 k 元（n-1）立方体构造一个 k 元 n 立方体

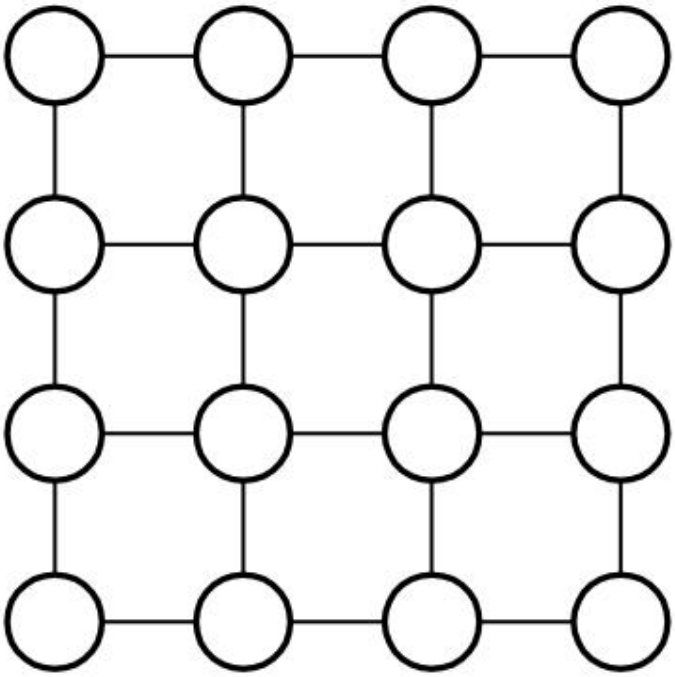


图 5.2 环面和网格网络：（a）环面网络（4 元 2 立方体）在两个维度上都包含从节点 3 到节点 0 的连接，但（b）网格网络（4 元 2 网格）省略了此连接。5.1 环面网络的结构在 z 维度上的基数为 3，在 x 维度上的基数为 4。混合基数的环面和网格通常出于封装和模块化的实际原因而构建。然而，混合基数环面不再具有边对称性，混合基数网格与单一基数网格相比具有更多的不对称性。这些不对称性引入了负载不均衡，并且对于许多流量模式，包括均匀流量，较长维度上的通道负载大于较短维度上的负载。以均匀流量为例——例如，γ_x——图 5.3 中 x 维度上的负载将是 γ_z 的两倍。

5.2 性能

环面网络的性能，与任何网络一样，由其吞吐量、延迟和路径多样性来表征。

5.2.1 吞吐量

在两级封装模型中，吞吐量受限于引脚带宽或二分带宽。我们首先考虑二分限制，并计算网络的通道二分带宽为

$$B_{C,\mathrm{T}} = 4k^{n-1} = \frac{4N}{k} \tag{5.1}$$

$$B_{C,\mathrm{M}} = 2k^{n-1} = \frac{2N}{k} \tag{5.2}$$

对于偶数 k。

产生公式 5.1 的最小二分可以通过图 5.1（c）来可视化。当 k 为偶数时，最外层维度

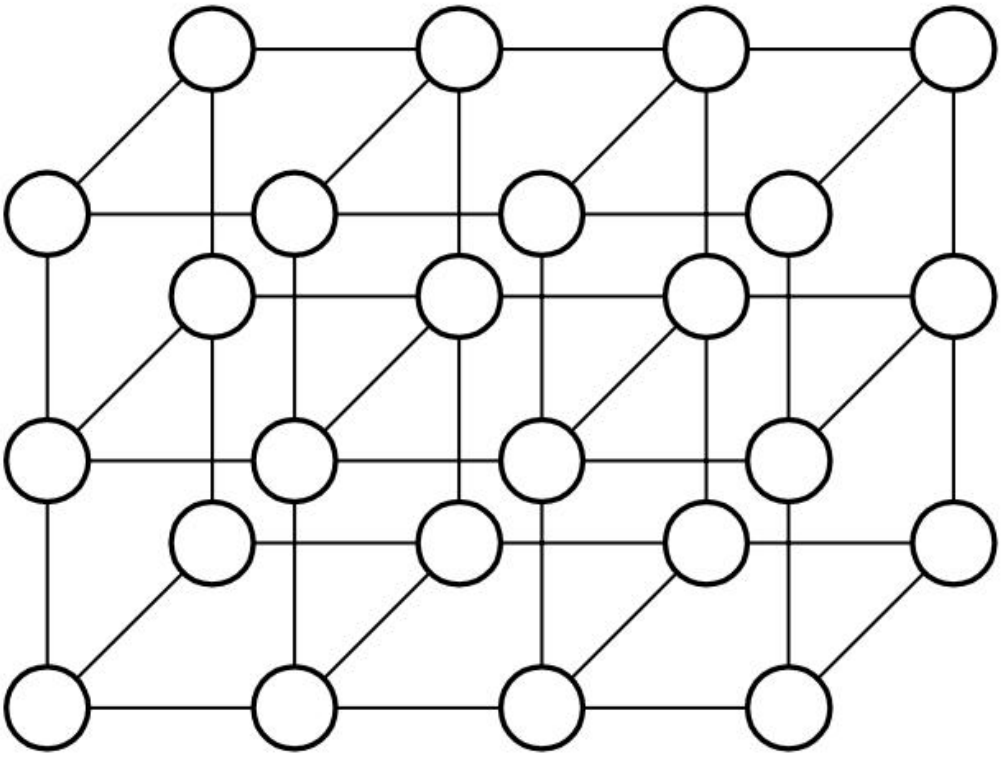


图 5.3 一个混合基数的 2，3，4 元 3 网格在其三个维度上具有不同的基数。

的循环中有偶数个 k 元 $(n-1)$ 立方体。最小二分将这个循环在其中心点分开，切割 2 组 k^{n-1} 条双向通道，即总共 $4k^{n-1}$ 条通道。对于网格，最小二分类似，只是不存在环绕通道，因此切割的通道数减半。

由于环面既是节点对称的又是边对称的，均匀流量下的通道负载可以通过将公式 5.1 代入公式 3.3，从二分通道负载确定。

$$\gamma_{\mathrm{T,U}} = \frac{N}{2B_C} = \frac{N}{2} \times \frac{k}{4N} = \frac{k}{8} \tag{5.3}$$

由于环面的边对称性，我们可以通过将环面的跳数从公式 5.10 代入公式 3.5 来找到相同的结果，并得到

$$\gamma_{\mathrm{T,U}} = \frac{NH_{\min}}{C} = \frac{N(nk/4)}{2nN} = \frac{k}{8} \tag{5.4}$$

对于偶数 k。

由于网格是不对称的，它无法达到上述用于环面的基于跳数的下界。相反，均匀流量下网格的通道负载为

$$\gamma_{\mathrm{M,U}} = \frac{k}{4} \tag{5.5}$$

对于偶数 k。直观上，这个结果来自于网格在均匀流量下可以使其二分通道饱和的事实。一半的流量，即每周期 N/2 个数据包，平均将穿过二分，在 $B_{C,mesh} = 2N/k$ 条通道上平衡负载，得到平均负载 $(N/2)/(2N/k) = k/4$。对于非二分通道，平均负载随

着远离阵列中心而进一步下降。1
对于最坏情况下的流量，所有流量都跨越二分带宽，信道负载加倍。2

$$\gamma_{\mathrm{T,W}} = \frac{N}{B_C} = \frac{k}{4}$$

$$\gamma_{\mathrm{M,W}} = \frac{N}{2B_C} = \frac{k}{2}$$

5.2 性能
为了考虑引脚带宽限制，我们首先计算每个节点的度数。每个双向立方体节点在其 n 个维度中的每个维度上有 4 个连接。3，4

$$\delta_{\mathrm{T}} = \delta_{\mathrm{M}} = 4n \tag{5.6}$$

将公式 5.1 和 5.6 代入公式 3.14，得到环面的最大信道宽度。

$$w_{\mathrm{T}} \leq \min\left(\frac{W_n}{4n}, \frac{kW_s}{4N}\right) \tag{5.7}$$

类似地，对于网格我们得到

$$w_{\mathrm{M}} \leq \min\left(\frac{W_n}{4n}, \frac{kW_s}{2N}\right).$$

根据信道宽度和信道负载，我们然后计算均匀负载下的吞吐量如下

$$\Theta_{\mathrm{ideal,T}} = \frac{f w_{\mathrm{T}}}{\gamma} = \frac{8}{k}\min\left(\frac{B_n}{4n}, \frac{kB_s}{4N}\right),$$

$$\Theta_{\mathrm{ideal,M}} = \frac{f w_{\mathrm{M}}}{\gamma} = \frac{4}{k}\min\left(\frac{B_n}{4n}, \frac{kB_s}{2N}\right).$$

一般来说，环面的最佳吞吐量发生在维度足够高以保持网络二分带宽受限，并且足够小以使所有线长低于临界线长时。暂时忽略线长，当以下条件满足时达到最大吞吐量

$$nk \leq \frac{NB_n}{B_s}. \tag{5.8}$$

为了保持线短并最小化串行化延迟（见下文），我们通常选择满足公式 5.8 的最小 n。例如，假设你必须构建一个 $N = 2^{12}$ 节点的环面，其中 $W_s = 2^{14}$，$W_n = 2^8$，f = 1 GHz。表 5.1 比较了这种大小的替代环面网络。对于表的每一行，维度 n 和基数 k 决定了其余属性。表中显示了乘积 nk，必须保持小于 $\frac{NW_n}{W_s} = 64$ 以满足公式 5.8。该值在低于此限制的位置以粗体显示。显示了三种信道宽度：w_n，由于节点引脚引起的宽度限制；w_s，由于二分带宽引起的宽度限制；以及 w，实际信道宽度（w_n 和 w_s 中的较小者）。对于节点引脚受限的情况，信道宽度以斜体显示。最后，表中显示了吞吐量 Θ，单位为 Gbit/s。

表 5.1 4,096 节点环面网络的吞吐量

n	k	nk	ws	wn	w	Θ_{ideal} (Gbits/s)
1	4,096	4,096	4,096	64	64	0.125
2	64	128	64	32	32	4
3	16	48	16	21	16	8
4	8	32	8	16	8	8
6	4	24	4	10	4	8
12	2	24	2	5	2	8

表 5.1 显示吞吐量随维度单调增加，直到网络在 n = 3 时变为二分带宽受限。超过该点后，吞吐量持平，因为设计使用了所有可用的二分带宽布线。最可能的是，我们将为此网络选择 16 元 3 立方体拓扑，因为它在最小维度下提供了最大吞吐量。我们将在下一节中看到低维度如何在延迟和线长方面带来好处。

5.2.2 延迟

环面网络的延迟强烈依赖于维度。在低维度极端，延迟由高跳数主导；在高维度极端，由于窄信道宽度 w，串行化延迟主导。最佳延迟通常出现在较低的中间维度。
环面网络的串行化延迟通过将公式 5.7 代入公式 3.10 得到。

$$T_{s,\mathrm{T}} = \frac{L}{b} = \frac{L}{f \min\left(\frac{W_n}{4n}, \frac{kW_s}{4N}\right)} = \frac{1}{f} \max\left(\frac{4nL}{W_n}, \frac{4NL}{kW_s}\right).$$

类似地，对于网格，

$$T_{s,\mathrm{M}} = \frac{1}{f} \max\left(\frac{4nL}{W_n}, \frac{2NL}{kW_s}\right). \tag{5.9}$$

当 n 大，因而 k 小时，公式 5.9 的引脚项和二分带宽项都变大。然而，二分带宽项通常占主导。当 n 小，因而 k 大时，T_s 小。然而，在这种情况下，跳数主导延迟。
环面网络中的平均最小跳数通过对所有节点对之间的最短距离取平均来确定，得到

$$H_{\mathrm{min,T}} = \begin{cases} \dfrac{nk}{4}, & k \text{ 为偶数}, \\ n\left(\dfrac{k}{4} - \dfrac{1}{4k}\right), & k \text{ 为奇数}. \end{cases} \tag{5.10}$$

表 5.2 4,096 节点环面网络的延迟

n	k	w	Θ_{ideal}	T_h (ns)	T_s (ns)	T(ns)
1	4,096	64	0.125	10,240	8	10,248
2	64	32	4	320	16	336
3	16	16	8	120	32	152

4	8	8	8	80	64	144
6	4	4	8	60	128	188
12	2	2	8	60	256	316

在均匀流量和偶数基数下，数据包平均在每个 n 维度上传播四分之一的路程，即 $\frac{k}{4}$ 跳。因此，跳数为 $\frac{nk}{4}$。对于奇数 k，平均距离包括一个小的附加因子。
类似地，对于网格，跳数为

$$H_{\text{min,M}} = \begin{cases} \dfrac{nk}{3}, & k \text{ 为偶数}, \\ n\left(\dfrac{k}{3} - \dfrac{1}{3k}\right), & k \text{ 为奇数}. \end{cases} \tag{5.11}$$

为了看到 T_s 和 H 对延迟的综合影响，考虑我们的示例 $N = 2^{12}$ 节点环面，并设 L = 512 位，$t_r = 8$ ns，$t_c = 2$ ns，每跳总共 10 ns。表 5.2 和图 5.4 比较了随着维度变化时的头部延迟 T_h、串行化延迟 T_s 和总延迟 T。表显示最小延迟 144 ns 出现在 n = 4 时。然而，n = 3 时的 152 ns 延迟足够接近，可能会选择它以获得较低维度的封装和线长优势。

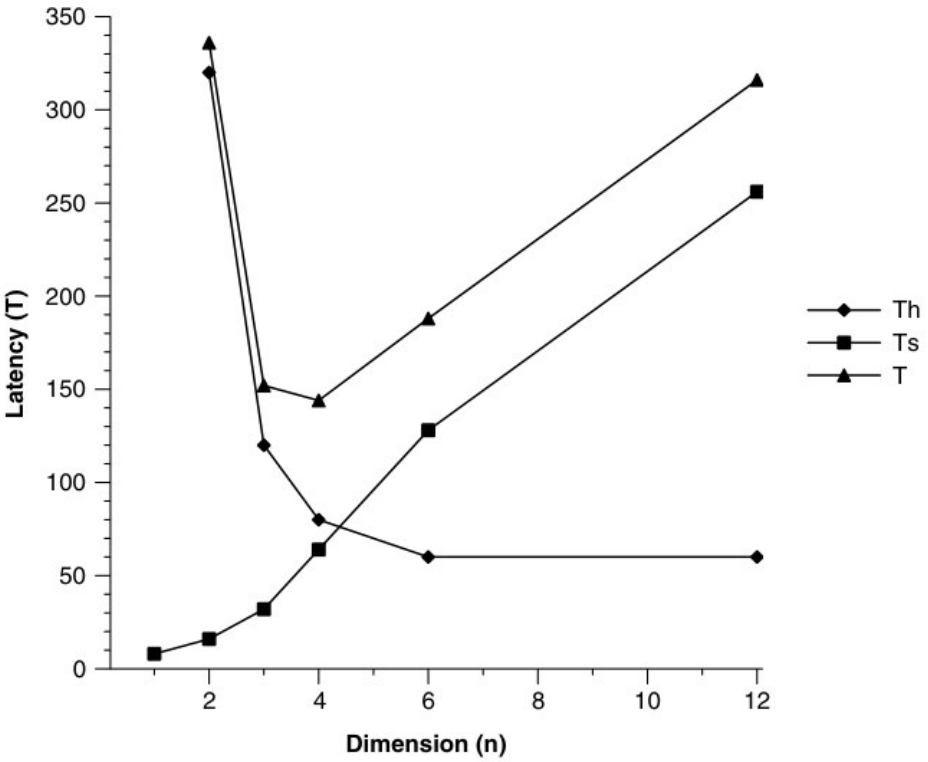


图 5.4 4096 节点环面网络的延迟随维数变化曲线

5.2.3 路径多样性

在环面网络中，每对节点之间存在许多不同的路径。通过选择性地在这些路径上分配流量，可以在网络信道之间平衡负载，即使对于非常不规则的流量模式也是如此。这

种路径多样性还使网络能够通过沿替代路径路由流量，围绕故障信道快速重新配置。在源节点 a 和目的节点 b 之间存在多少条路径？为了回答这个问题，我们将首先研究简单情况，其中我们只考虑最小路径，并且所有路径在每个维度上采取相同方向（单向路由）。可能路径的数量随维度迅速增加。对于一维网络，只有一条路径。对于二维网络，其中 a 和 b 在 x 维度上相隔 Δ_x 跳，在 y 维度上相隔 Δ_y 跳，路径数量由下式给出

$$|R_{ab}| = \binom{\Delta_x + \Delta_y}{\Delta_x}.$$

有这么多方式选择在 $\Delta_x + \Delta_y$ 总跳数中取出 Δ_x 跳的位置。类似地，在三维中，z 维度有 Δ_z 跳，公式为

$$|R_{ab}| = \binom{\Delta_x + \Delta_y + \Delta_z}{\Delta_x}\binom{\Delta_y + \Delta_z}{\Delta_y} = \frac{(\Delta_x + \Delta_y + \Delta_z)!}{\Delta_x!\Delta_y!\Delta_z!}.$$

如上所述，第一项给出了从总数中选择 Δ_x 个 x 跳的位置的方式数。第二项给出了从剩余的 y 和 z 跳中选择 Δ_y 个 y 跳的位置的方式数。这两项相乘得到唯一路径的总数。一般来说，如果有 n 个维度，编号从 0 到 n - 1，并且在第 i 个维度上从 a 到 b 有 Δ_i 跳，那么从 a 到 b 的最小单向路径总数为

$$|R_{ab}| = \prod_{i=0}^{n-1}\binom{\sum_{j=i}^{n-1}\Delta_j}{\Delta_i} = \frac{\left(\sum_{i=0}^{n-1}\Delta_i\right)!}{\prod_{i=0}^{n-1}\Delta_i!}.$$

乘积的每一项给出了在所有剩余跳中选择在第 i 维进行跳转的方式数。正如预期的那样，最小路径的数量随着维度和距离的增加而迅速增长。例如，在一个三维网络中，当 $\Delta_x = 3$、$\Delta_y = 3$ 和 $\Delta_z = 3$ 时，路径数量为 1,680。一些路由算法还会纳入非最小路径，在这些情况下，路径多样性几乎可以无限大。

5.3 构建网格和环面网络

要构建一个网络，并确定诸如通道长度 l 等属性，必须将网络的抽象节点映射到物理空间中的实际位置。根据封装技术的不同，映射可以是一维空间（例如，沿线性背板的板卡）、二维空间（例如，PC 板上的芯片）或三维空间（例如，放置在体积中任意位置的模块）。虽然我们生活的世界是三维的，但一些封装系统将网络限制在一维或二维中。

环面和网格网络特别容易映射到物理空间，且连线均匀短小。最简单的情况是网络是一个网格，其维度数与封装技术的物理维度数相同。在这种情况下，节点的 n 位地址 $\{a_1, \ldots, a_n\}$ 也是其在 n 维笛卡尔空间中的位置。数学上，节点在维度 i 上的位置等于其在维度 i 上的地址：$p_i = a_i$。所有通道连接物理上相邻的节点，因此长度均为单位长度 5。

如果环面以相同的方式封装，且 $p_i = a_i$，那么在每个维度上从节点 k - 1 到节点 0 的长环绕通道长度将为 k。这种长通道可能导致过高的延迟或需要较慢的信号速率。通过折叠环面网络可以避免这个问题，如图 5.5 和 5.6 所示。在折叠环面中，到物理空间的映射为

$$p_i = \begin{cases} 2a_i \text{if} a_i < \frac{k}{2} \\ 2k - 2a_i - 1 \text{if} a_i \geq \frac{k}{2} \end{cases} .$$

折叠环面消除了长环绕通道，但代价是其他通道的长度加倍。如图 5.5 所示，一维环面或环形网络通过将前半部分节点 $0, \ldots, \frac{n}{2} - 1$ 按升序与后半部分节点 $n - 1, \ldots, \frac{n}{2}$ 按降序交错来折叠。一般来说，折叠的 k 元 n 立方体是通过按折叠顺序组合 k 个折叠的 k 元 n - 1 立方体来构建的，如图 5.6 所示。

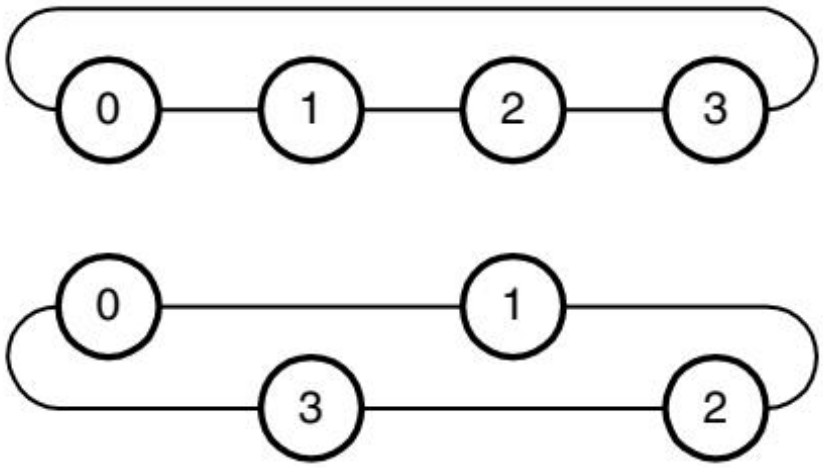


图 5.5 折叠环形网络：（a）未折叠的四元一维立方体；（b）折叠后缩短节点 3 到节点 0 的环绕连接。

当逻辑维度数超过物理维度数时，必须将多个逻辑维度映射到每个物理维度中。如果物理维度数为 q，一种直接的映射是将 $\frac{n}{q}$ 个逻辑维度折叠到每个物理维度中。

$$p_i = \sum_{j=0}^{\frac{n}{q}-1} k^j a_{i+jq}.$$

图 5.7 以 3 元 2 网格为例说明了这种映射。这里，网格的三行排成一条线。维度 0（逻辑 x）的通道长度为 1，而维度 1（逻辑 y）的通道长度为 l = k = 3。一般来说，以这种方式映射到 q 个物理维度的网格中，维度 i 的通道长度为

5.3 构建网格和环面网络

$$l_i = k^{\lfloor \frac{i}{q} \rfloor}.$$

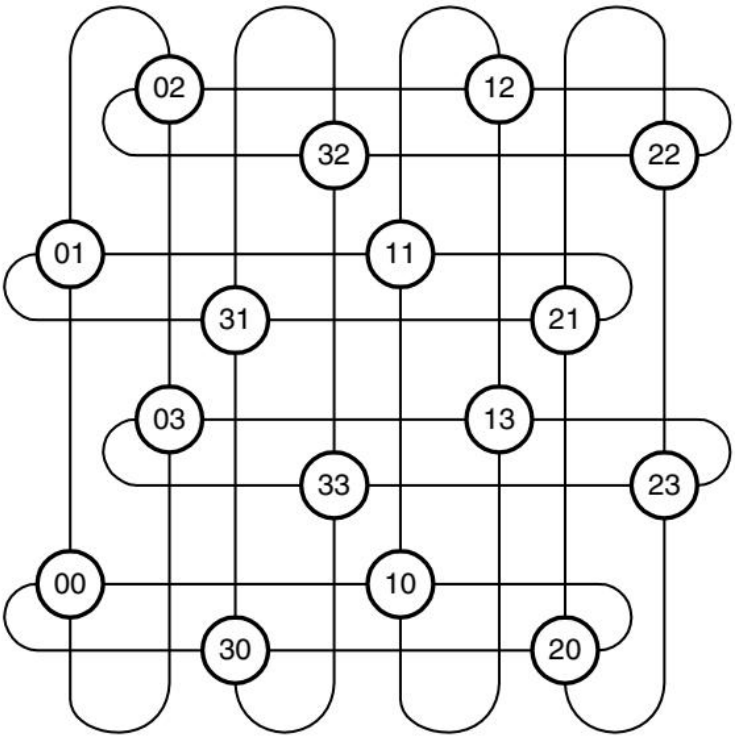


图 5.6 折叠式四元二维环面网络。

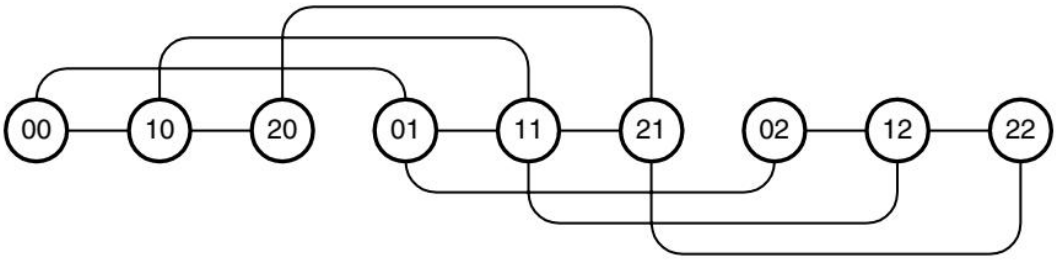


图 5.7 映射到单一物理维度的 3 元 2 网格。逻辑 x 通道为单位长度。逻辑 y 通道长度为 3。

这种将逻辑维度投影到物理维度的方法也可以用于折叠环面。在这种情况下，长度加倍。

$$l_i = 2k^{\lfloor \frac{i}{q} \rfloor}$$

5.4 快速立方体

由于环面网络的物理通道较短，通道延迟 t_c 通常由路由延迟 t_r 主导。这导致头部延迟 T_h 比直径更小的网络所能达到的要大。直径可以通过增加维度来减小。然而，如图 5.4 所示，这会缩窄通道宽度，导致串行化延迟 T_s 增加。

快速立方体网络是一种 k 元 n 立方体，增加了一些长（快速）通道 [45]。通过让必须在某个维度上长距离传输的数据包走快速通道，头部延迟可以降低到接近通道延迟的极限。由于快速通道的数量可以控制以匹配网络的二分宽度，因此可以在不增加串行化延迟的情况下实现头部延迟的降低。

一个 k 元 n 立方体网络通过在每个维度上每 i 个节点插入交换单元，并沿维度用长快速通道连接这些交换单元，从而转变为快速立方体，如图 5.8 所示（以一个维度为例）。如果一个消息在原始网络的维度 j 上的跳数为 H_j，并假设 H_j 能被 i 整除，那么在快速

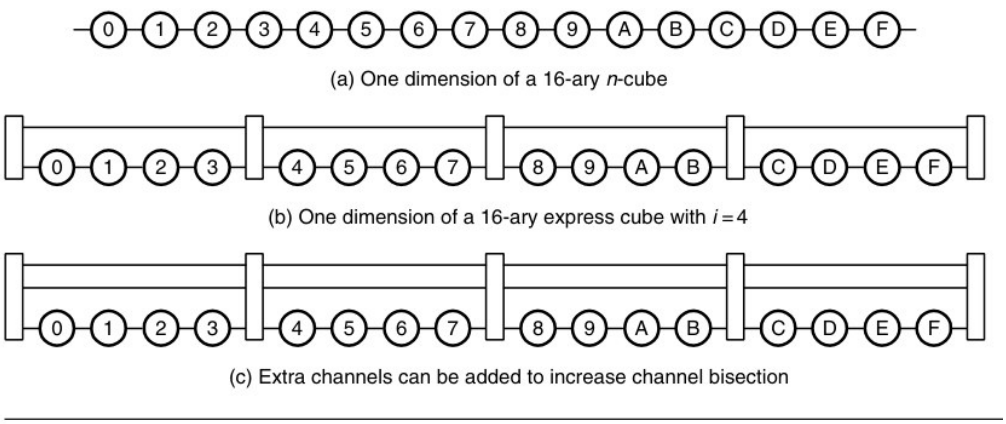
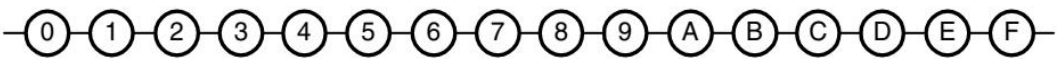
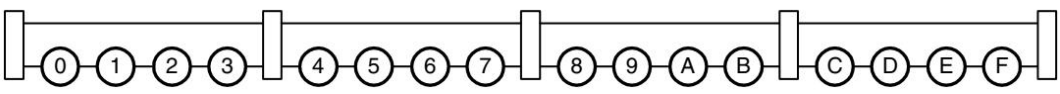


图 5.8 通过长通道构建的快速立方体网络



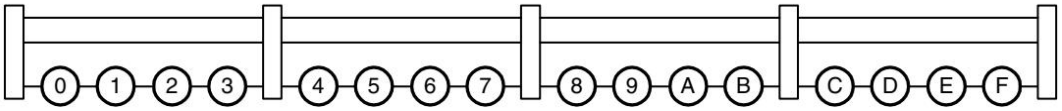
（a）16 元 n 立方体的一个维度

（a）16 元 n 立方体的一个维度



（b）16 元快速立方体的一个维度，i=4

（b）16 元快速立方体的一个维度，i=4



（c）可以增加额外通道以增加通道二分宽度

（c）可以增加额外通道以增加通道二分宽度

图 5.8 通过插入由长通道连接的交换单元来创建平面快速立方体：（a）16 元 n 立方体的一个维度，（b）通过每 i 个节点插入一个交换单元并用长快速通道连接这些交换单元，该网络转变为快速立方体，（c）可以通过改变交换单元之间的快速通道数量来控制通道二分宽度。

立方体的维度 j 上的跳数变为 6

$$H_{je} = \left(\frac{H_j}{i} + i \right).$$

对于非常长的维度，该方程的第一项占主导地位。在这种情况下，可以通过选择 $i = \frac{t_r}{t_c}$ 来平衡通道延迟和路由延迟。对于维度较短的网络，可以通过选择 $i = \sqrt{H_{\min}}$（其中 H_{min} 是不使用快速通道时的平均跳数）来平衡方程中的两项。

分层快速立方体构建的网络具有环面的局部性特性，其对数直径接近公式 3.1 的极限。如图 5.9 所示，分层快速立方体是递归构建的。首先，每 i 个节点插入一个 0 级交换单元。然后，每第 i 个 0 级交换单元升级为 1 级交换单元，依此类推，共 l 级。第 j 级的每个交换单元与其他第 j 级交换单元相隔 i^j 个节点，并连接到第 0 级至第 j 级的相邻交换单元。

假设 H_j 能被 i^l 整除，如果分组在传统立方体的维度 j 中的跳数为 H_j，则在分层快速立方体上，该跳数

5.4 快速立方体

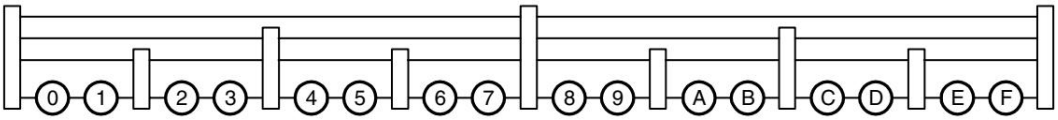


图 5.9 一个 i = 2 且 l = 3 的分层快速立方体。

变为 7

$$H_{jh} = \left(\frac{H_j}{i^l} + (i-1)l + 1 \right).$$

选择 i = 2 可以最小化本地节点引起的延迟。选择 $l = \log_i \frac{t_r}{t_c}$ 可以平衡长距离下的路由器和通道延迟。

总体而言，分层快速立方体的延迟在两个基本限制之和的小常数范围内：（1）遍历两个节点之间最小距离所需的最小通道延迟，以及（2）做出路由决策以选择目标节点

所需的对数延迟。当快速通道的数量变化时，快速立方体的吞吐量可以饱和引脚和二分带宽约束。尽管有这些优点，快速立方体在实践中很少使用。这主要是因为大多数实际网络没有足够高的基数来证明添加交换单元的复杂性是合理的。此外，随着技术的进步，t_r 和临界线长 l_c 都变小，减少了对长通道的需求。

5.5 案例研究：MIT J-Machine

MIT J-Machine [51，136] 如图 5.10 所示，展示了环面和网格网络如何自然地映射到三维。J-Machine 于 1991 年在 MIT 与英特尔合作构建，是一台消息传递多计算机。每个处理节点包含一个消息驱动处理器芯片 [53] 和三个外部 DRAM 芯片。当时，该机器因其网络的性能而引人注目，其具有任何同等规模并行计算机中最高的二分带宽和最低的延迟。该机器具有新颖的网络接口，允许在单条指令中直接从处理器寄存器发送短消息。8

图 5.10 一对 J-Machine。近处的机器已移除外壳。机器的上部容纳多达 1,024 个处理节点，连接成 $8 \times 8 \times 16$ 网格。这台机器仅安装了一半的处理节点板，有八个 8×8 的 64 处理器板。处理器阵列下方是用于图形、磁盘控制和局域网（LAN）接口的 I/O 卡。机器底部容纳一个包含 80 块磁盘的磁盘阵列。

J-Machine 既使用网络进行节点间消息通信，也用于 I/O。一组 I/O 节点沿机器的一侧排列，如图 5.10 所示。机器中的任何节点都可以通过向任何 I/O 设备发送消息与其通信。为 J-Machine 开发了用于图形（分布式帧缓冲区）、磁盘（SCSI 主机适配器）和网络（局域网（LAN）接口）的 I/O 设备。

J-Machine 处理节点连接成三维网格网络 [138]。网格到物理空间的自然映射使得机器构建非常密集。9 每块板包含 64 个节点，连接成 8×8 网格。每个处理节点尺寸为 2×3 英寸。相邻板通过分布在板面上的弹性连接器垂直连接。单个机箱中最多可连接 16 块板（图 5.10），形成 1,024 节点的 $8 \times 8 \times 16$ 网格。板边缘提供带状电缆连接器，允许多个机箱连接在一起形成更大的机器。路由器寻址将最大机器尺寸限制为 64 K 节点的 $32 \times 32 \times 64$ 网格；然而，从未构建过大于 1,024 节点的机器。

每个网络通道由 15 条线组成（9 条数据信号 10 和 6 条控制信号），以 32 MHz 运行，是 16 MHz 处理器速率的两倍。这提供了 288 Mbits/s 的通道有效载荷带宽。单个物理通道用于一对相邻节点之间的双向通信，节点以 flit 为单位仲裁通道的使用。根据机器中的位置，网络通道可以在 PC 板上（卡上 64 个节点之间的内部连接）、通过弹性连接器（两个相邻 64 节点卡之间的垂直连接）或通过带状电缆（相邻卡之间的水平连接）实现。

J-Machine 网络采用维序路由（第 8.4.2 节）和虫洞流控（第 13.2 节）。路由器集成在消息驱动处理器（MDP）芯片上（图 5.11）。维序路由使路由器能够清晰地划分为 X、Y 和 Z 部分。通过路由器的延迟，包括通道遍历，为每跳两个 32-MHz 路由器周期（一个 16 MHz 处理器周期）。

在均匀流量下，$8 \times 8 \times 16$ J-Machine 网络上 4 字消息的零负载延迟可以通过扩展公式 5.11 来处理这种混合基数情况来计算。序列化延迟为 16 个 31.25 ns 网络周期，平均跳数为 32/3，每跳两个网络周期，总延迟为 37 个 31.25 ns 网络周期（18.5 个处理器周期），即 1.16 μs。

该网络在均匀流量下的吞吐量可以用对分带宽界限来计算。将公式 5.2 应用于较长的 k = 16 维度，我们计算出 $B_C = 128$ 条通道穿过网络对分。然而，由于物理通道在两个方向上共享，实际的通道对分是这个数字的一半，$B_C = 64$。在均匀流量下，1,024 个节点中有一半通过这个对分发送数据包，通道负载为 $\gamma = 512/64 = 8$。然后我们计算吞吐量为 $\Theta = b/\gamma = 288/8 = 36$ Mbits/s 每节点。

为了与我们计算的性能数字进行比较，图 5.12（来自 [136]）显示了在 12.5 MHz 处理器时钟下运行的 512 节点 J-Machine 的测量延迟作为提供流量的函数。测得的零负载延迟为 23 个处理器周期，比我们计算的值（18.5 个周期）大 4.5 个周期，这是由于通过网络输入和输出接口的固定 4 周期延迟以及半个周期的向上取整。饱和对分流量为 6 Gbits/s，仅为我们计算的吞吐量 14.4 Gbits/s（64 通道 $\times$ 9 位 $\times$ 25 MHz）的 42%。这主要是由于两个因素。首先，四字消息包含一个两字报头，使其具有 50% 的报头开销。其次，机器中采用的虫洞流控制无法达到 100% 的通道利用率。

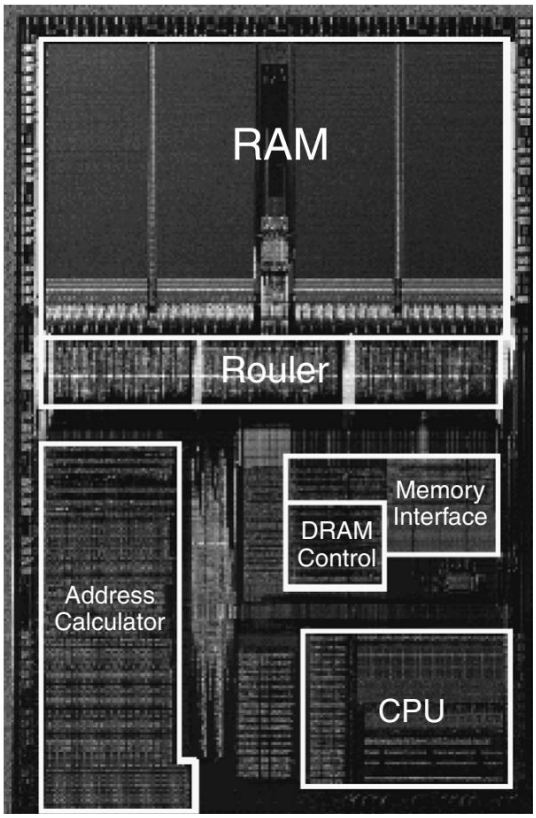


图 5.11 一个 MDP 芯片，采用 1.2 μ m CMOS 工艺实现，包括一个 32 位 RISC 处理器、一个三维维序路由器、一个网络接口、一个 DRAM 接口和 18 Kbytes 的片上存储器。路由器位于 RAM 正下方，分为 X、Y 和 Z 部分。

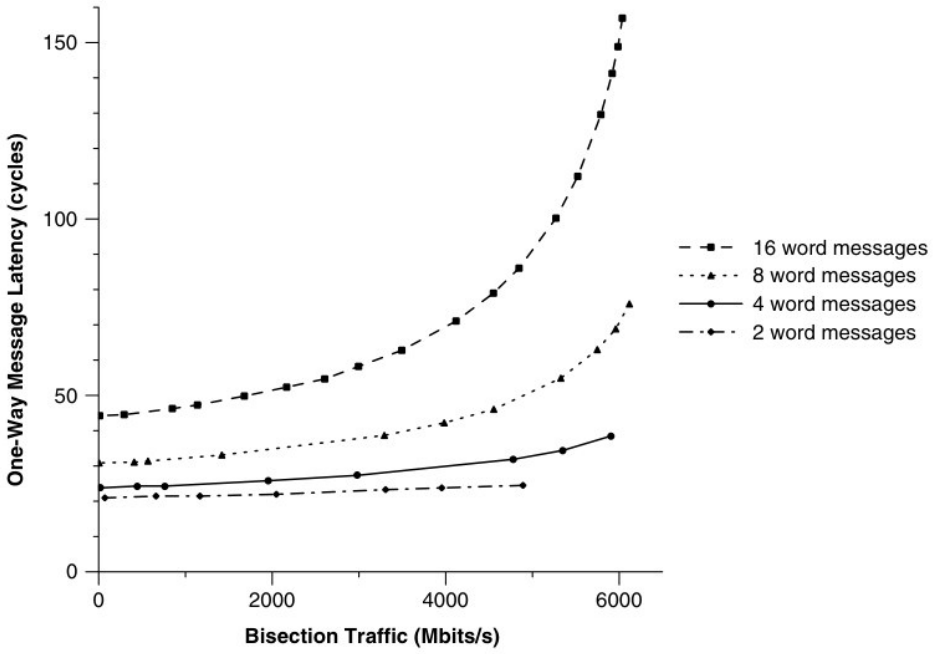


图 5.12 512 节点 J-Machine 的实测延迟与提供负载关系

5.6 文献注释

网格和环面长期以来一直是流行的网络拓扑，并被用于一些最早的并行计算机，包括 Solomon [172] 和 Illiac-IV [13]。二进制 n 立方体或超立方体网络由 Sullivan 和 Bashkow [179] 推广。超立方体被用于加州理工学院 Cosmic Cube [163]，许多商业并行计算机以 Cosmic Cube 为模型，包括 Intel iPSC 计算机 [155, 38] 和 NCUBE 系统 [140, 134]。1985 年，Dally 引入了基于物理约束（线对分和引脚数）比较网络拓扑的概念，并展示了在现实约束下低维环面的优势 [46]。Agarwal 后来改进了这一分析 [2]。此后，低维环面和网格网络成为并行计算机的标准 [56, 164, 138, 95, 162]。最近，环面网络已被应用于互联网路由器设计 [49]。Dally 开发了快速立方体以解决传统环面网络的大渐近路由延迟 [45]。然而，实际网络很少大到渐近性起作用的程度。许多环面网络的变体已经发表，涉及扭曲端到端连接或改变维度内连接的顺序。其中一些变体在直径上略有改进。参见例如 [119，165，33]。您将有机会在练习 5.6 中实验此类拓扑。

5.7 练习

5.1 比较蝶形和环面拓扑。比较 1,024 节点的蝶形和环面网络，其中节点带宽为 300 Gbits/s，对分带宽为 4Tbits/s。对于这两种网络，选择 k 和 n 的最小值，使得对分带宽饱和（为简单起见，仅考虑 $k^n = 1,024$ 的 k 和 n 组合）。对于数据包长度 L = 1,280，这些网络的串行化延迟是多少？平均跳数 H_{min} 是多少？忽略线延迟，每跳延迟为 $t_r = 12$ ns，两个网络的零负载延迟是多少？

5.2 4,096 节点环面中的权衡。检查 4,096 节点环面中 k 和 n 之间的权衡。对于 $k^n = 4,096$ 的每个 k 和 n 组合，理想吞吐量和平均零负载消息延迟是多少？假设每个节点有 120 个信号引脚，系统的对分宽度为 1,500 个信号，信号频率为 f = 2.5 GHz，数据包长度为 L = 512 位，路由器跳延迟为 20 ns。忽略线延迟（$T_w = 0$）。

5.3 三级封装层次。一个 256 节点环面需要在以下约束下封装在三级层次中：每个节点有 384 个信号引脚，1,200 个信号可以离开板，6,000 个信号可以穿过背板的中部。确定使网络带宽最大化的 k 和 n。如果多个 k 和 n 值达到此目标，选择零负载延迟最小的那个。解释如何通过使用节点、板和背板来封装此网络，同时最小化使用的板数。

5.4 单向环面中的通道负载。计算均匀流量下单向环面中的平均通道负载。这个负载大约比双向环面大多少倍？解释这种增加的来源。

5.5 略微非最小路由的数量。作为 x 和 y 维度中最小跳数 Δ_x 和 Δ_y 的函数，如果允许最多比最小路由多 1 跳的路由，二维环面中有多少条路由？最多比最小路由多 2 跳的路由有多少条？假设 k 为奇数。

5.6 双扭曲环面。图 5.13 所示的拓扑是双扭曲环面 [165]。假设最小路由，比较此拓扑的平均跳数 H_{min} 与标准 4 元 2 立方体的平均跳数。为什么扭曲会改变跳数？

5.7 环面布局中的线长。计算在平面中布局的 k 元 6 网格网络中消息遍历的平均线长。假设所有节点对齐在二维网格上，节点间距为 10 cm。为简单起见，测量节点中心之

间的距离。如果网络封装在三维中，第三维的节点间距也是 10 cm，平均距离如何变化？假设两种情况下的 k 均为偶数。

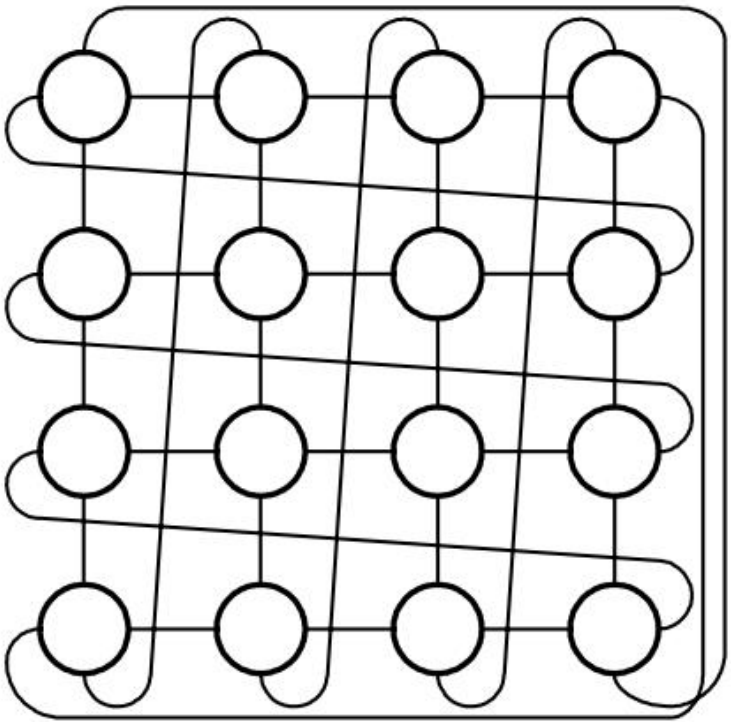


图 5.13 双扭曲环面拓扑。

5.8 立方体连接环拓扑。虽然高维环面具有较低的跳数，但其高度数要求导致通道狭窄，进而增加了串行化延迟。立方体连接环（CCC）拓扑在超立方体（2 元 n 立方体）中解决了这个问题。一个 n 阶 CCC 的构造方法是取一个 2 元 n 立方体，并将每个节点替换为一个由 n 个节点组成的环（图 5.14）。由此产生的拓扑具有固定的节点度数 3，与网络规模无关。图 3.16 展示了一个 3 阶 CCC。一个 n 阶 CCC 中有多少个节点，其平均最小跳数 H_{min} 是多少？（计算节点间的最小跳数很困难——如有必要可进行近似。）考虑设计一个 64 节点的网络，节点引脚数限制为 120 个，每个引脚的工作速率为 2.5 Gbits/s。比较使用 CCC 拓扑的该网络与优化尺寸的 k 元 n 立方体的零负载延迟。确保通道足够宽，以饱和每个节点的引脚带宽。使用消息大小 L = 1,024 位，每跳延迟 t_{r} = 20 纳秒，并忽略任何线路延迟。

图 5.14

从（a）2 元 n 立方体到（b）n 阶 CCC 所使用的节点变换。如图所示，2 元 n 立方体中的每个单一节点在 CCC 中被替换为一个由 n 个节点组成的环。CCC 中的每个节点具有固定的度数 6：环内 2 条双向连接，以及到另一个环的 1 条双向通道。

5.9 J-Machine 的封装。每个 J-Machine（第 5.5 节）64 节点电路板是一个 8 层板，包含 2 个电源层、2 个表面层、2 个 x 布线层和 2 个 y 布线层。每个布线层提供每线性英寸 20 个信号的信号密度。板之间有 16 个弹性连接器，每个连接器承载 60 个信号。在均匀流量、128 位消息下，使用相同的封装技术，在 J-Machine 相同的体积内你能实现的最高性能 1,024 端口蝶形网络的吞吐量和延迟是多少？

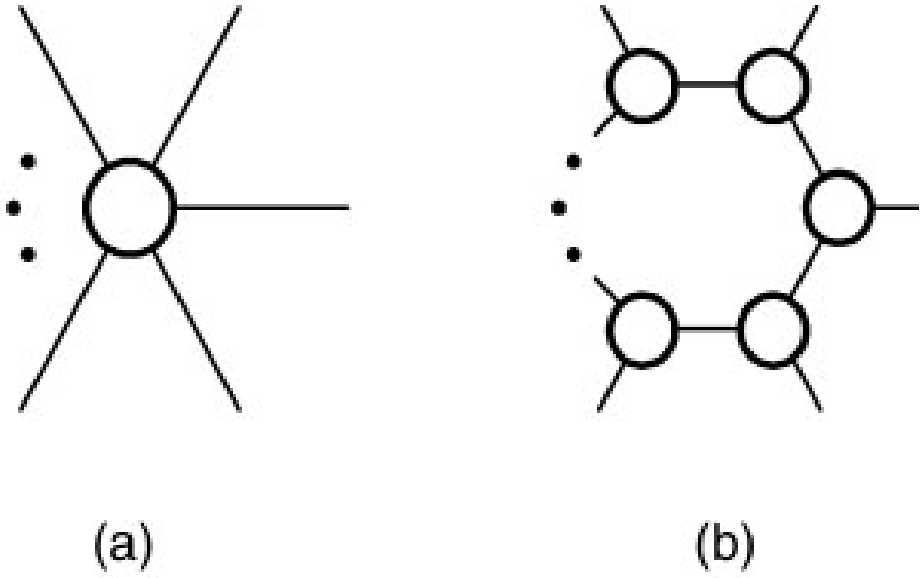
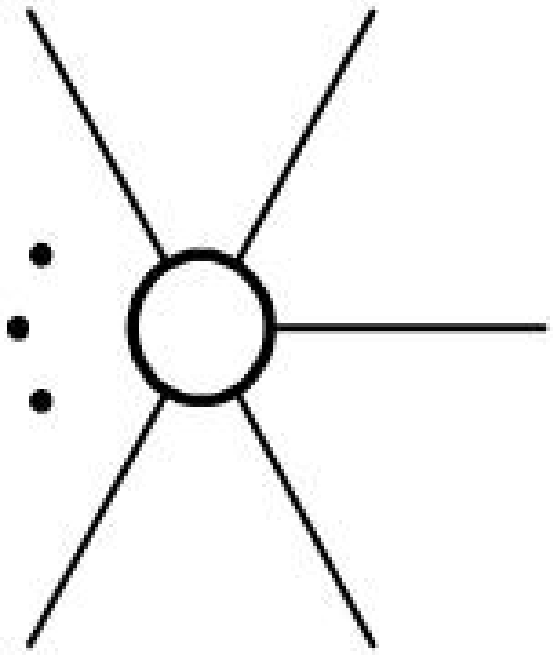
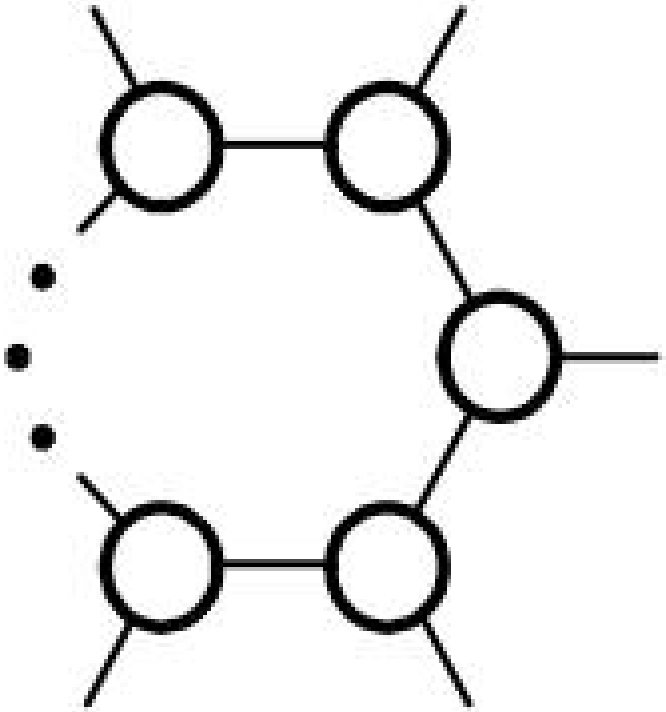


图 5.14 从二元 n 立方体到 n 阶立方体连接环（CCC）的节点变换



(a)



(b)

5.10 J-Machine 中双向信号传输的优势。J-Machine 使用了双向信号传输，在两个节点之间相反方向传输的逻辑通道共享一个物理通道。为了理解这样做的原因，比较一个 1,024 节点的 J-Machine 网络与将每个 9 位宽的双向通道替换为两个 4.5 位宽的单向通道的网络在吞吐量和延迟上的差异。（不必担心小数位宽。）

5.11 同时双向信号传输的优势。比较一个具有 9 位宽顺序双向通道（一次一个方向）的 1,024 节点 J-Machine 与使用同时双向通道（可以同时向两个方向发送信息）的同一网络的吞吐量和延迟。

5.12 将 J-Machine 网络视为环面。使用练习 5.9 中的相同封装密度假设，在均匀流量、128 位长消息下，使用相同封装在相同体积内你能实现的最高性能 1,024 节点环面网络的吞吐量和延迟是多少?

注释

- 1. 公式 5.3 至 5.5 适用于双向网络。单向环面的信道负载在练习 5.4 中研究。
- 2. 在双向网格或环面中，需要非最小路由以防止两个方向之间的负载不平衡将峰值信道负载（在一个方向上）增加到几乎 $4\gamma_{\text{uniform}}$。
- 3. 严格来说，网格中的一些节点的度数小于 $4n$，但在实践中不值得为这些边缘情况设计特殊部件。相反，所有节点的度数是恒定的，并且在网格边缘信道简单地保持未连接。

- 4. 如果使用同时双向信号传输，引脚可以在相反方向的信道之间共享，有效节点度数减半。（见练习 5.11。）
- 5. 这里我们假设每个节点在物理空间中的直径为单位长度，因此每个维度上连接物理相邻节点的通道长度均为单位长度。
- 6. 如果 H_j 不能被 i 整除，公式会变得更复杂，但结果相同。详见 [45]。
- 7. 这也假设 HjH_{j} 足够大以使用所有 ll 级。再次，对于 HjH_{j} 不能被 i^l 整除的更一般情况，请参见 [45]。
- 8. 我们将处理器-网络接口的讨论推迟到第 20.4 节，在那里我们讨论 M-Machine 的网络接口，它是 J-Machine 的后继。
- 9. 在练习 5.9 中，我们将探讨将其他拓扑映射到这种密集封装中的困难。
- 10. 该机器具有 36 位字长（32 位数据 + 4 位标签位），并在 4 个网络周期内通过网络发送一个字。
- 11. 我们在练习 5.10 中探讨这种双向通道的优势。

第六章　非阻塞网络

到目前为止，我们一直关注分组交换网络，但在本章中，我们将注意力转向电路交换网络。在电路交换网络中，特定源和目的地之间的连接首先被建立，然后保持，数据通过连接连续流动，最后被拆除。[1] 历史上，非阻塞的电路交换网络首先与电话系统相关联。一个人打给另一个人的电话代表一个单一的连接请求。要使通话接通，必须找到一条通过网络的未使用路径，然后分配给该通话。如果电话系统是非阻塞的，那么只要接收方的电话未占线，通话就总能成功。

更准确地说，如果一个网络能够处理所有作为输入和输出排列的电路请求，则称该网络为非阻塞的。也就是说，可以从每个输入到其选定的输出形成一条专用路径，而没有任何冲突（共享通道）。相反，如果一个网络无法在没有冲突的情况下处理所有此类电路请求，则它是阻塞的。[2] 在本章中，我们研究两种类型的非阻塞网络。首先，如果一个网络可以增量地建立任何排列，一次一个电路，而不需要重新路由（或重新排列）任何已建立的电路，则它是严格非阻塞的。如果任何未使用的输入可以连接到任何未使用的输出，而不改变任何其他流量所走的路径，则该网络是严格非阻塞的。

相比之下，如果一个网络可以为任意排列路由电路，但增量构建排列可能需要重新排列一些早期电路以允许建立后续电路，则它是可重排非阻塞的（或简称为可重排的）。可重排网络可以将任何未连接的输入连接到任何未连接的输出，但可能需要重新路由一些无关流量来进行此连接。

上述所有定义适用于单播流量，其中每个输入最多连接到一个输出。我们还考虑多播流量，其中来自单个输入的连接可以扇出到多个输出。如果一个网络可以在不影响其他流量的情况下将任何未连接的输出连接到任何输入（已连接或未连接），则它对于多播流量是严格非阻塞的。此外，如果一个网络可以将任何未连接的输出连接到任何输入，但可能必须重新路由其他流量来进行连接，则它对于多播流量是可重排的。

交叉开关和扩展比为 2：1 的 Clos 网络是单播流量严格无阻塞网络的例子。扩展比更大的交叉开关和 Clos 网络对于多播流量也是严格无阻塞的。Beneš 网络和无扩展的 Clos 网络是单播流量可重排网络的例子。

6.1　无阻塞网络与无干扰网络

对于分组交换应用，无阻塞网络的概念在很大程度上无关紧要，或者至少是过度的。因为分组交换机不会在连接或会话期间占用电路，所以只要满足与带宽和资源分配有关的两个约束，它们就可以在连接和分组流之间共享信道而不会产生干扰。首先，必须有足够的信道带宽来支持共享该信道的所有流量。如果信道负载 γ_{max} 小于信道带宽，则满足该带宽约束。其次，资源（缓冲区和信道带宽）的分配必须以这样的方式

进行：任何单个流拒绝向另一个流提供服务的时间不超过一个短暂的、预定的时间量。这种资源分配约束可以通过第 12 章中描述的合适的流控制机制来实现。

我们将满足这些标准的分组交换网络称为无干扰网络。这样的网络能够处理任意分组流量，并保证分组延迟的上界。流量既不超过任何网络信道的带宽容量，也不会导致流之间的耦合资源分配。

对于当今几乎所有的应用，当人们说他们想要一个无阻塞网络时，他们真正需要的是一个无干扰网络，这通常可以以相当低的成本实现。然而，为了历史起见，并且对于那些需要真正无阻塞来支持电路交换的情况，我们在本章的剩余部分对无阻塞网络进行简要综述。

6.2 交叉开关网络

一个 $n \times m$ 的交叉开关或交叉点开关直接连接 n 个输入到 m 个输出，没有中间级。实际上，这样的开关由 m 个 n：1 多路复用器组成，每个输出一个。许多交叉开关网络是方形的，即 m = n。其他是矩形的，m > n 或 m < n。

图 6.1 显示了一个 4×5 交叉开关的概念模型。有 4 条输入线和 5 条输出线。在输入线与输出线交叉的每个点——即每个交叉点——一个开关可选地将输入线连接到输出线。为了正确操作，每个输出必须连接到至多一个输入。然而，输入可以连接到多个输出。3 例如，在图 6.1 中，输入 1 驱动输出 0 和 3。

在某一时期，许多电话网络是使用由机械继电器组成的交叉开关实现的，这些继电器确实具有图 6.1 的结构。使用这种基于开关的交叉开关，输入和输出之间没有真正的区别。开关建立的连接是双向的。

然而，今天大多数交叉开关是用数字逻辑实现的，具有图 6.2 所示的结构。n 条输入线中的每一条连接到 m 个 n：1 多路复用器的一个输入。多路复用器的输出驱动 m 个输出端口。多路复用器可以用驱动输出线的三态门或线或门实现，模仿图 6.1 的结构，或者用逻辑门树实现更传统的多路复用器。

6.2 交叉开关网络

图 6.2 用 5 个 4：1 多路复用器实现的 4×5 交叉开关。每个多路复用器选择要连接到相应输出的输入。这里的多路复用器设置实现了与图 6.1 相同的连接：$\{1, 0, 3, 1, 2\} \rightarrow \{0, 1, 2, 3, 4\}$。

无论怎样实现，当我们在系统中描绘交叉开关时，为了避免每次都绘制整个原理图，我们采用图 6.3 所示的交叉开关符号。如果从上下文可以清楚地看出该框是交叉开关，我们将省略“X”，并将交叉开关描绘为带有输入和输出的简单框。

交叉开关对于单播和多播流量显然都是严格无阻塞的。任何未连接的输出都可以通过简单地闭合连接输入和输出的开关，或通过适当地设置该输出的多路复用器来连接到任何输入。许多其他网络非常努力地实现这种在交叉开关中很容易获得的特性。

如果交叉开关是平凡无阻塞的，那么为什么还要费心使用任何其他无阻塞网络呢？原因是成本和可扩展性。虽然在小配置中经济，但方形 $n \times n$ 交叉开关的成本随着 n^2 增

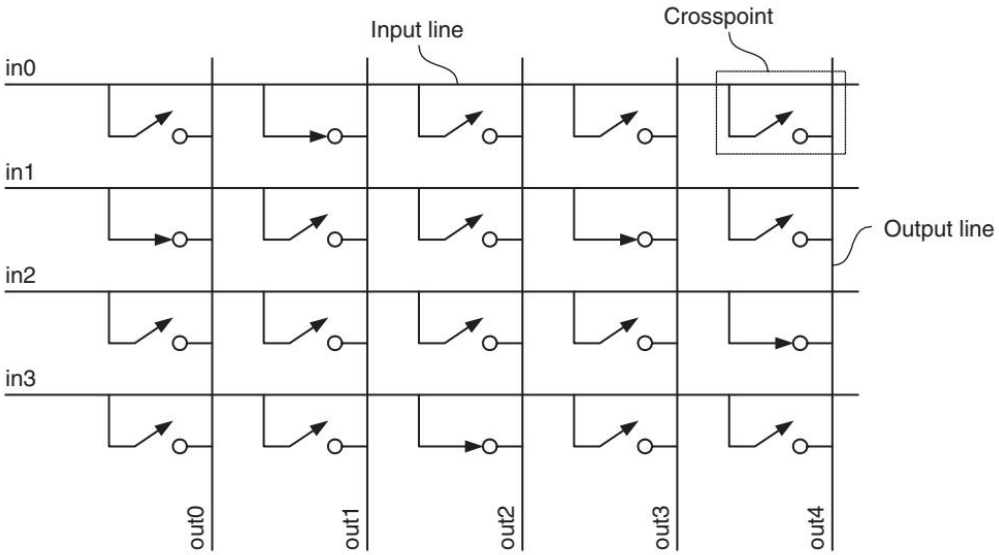


图 6.1 一个 4×5 交叉开关由 4 条输入线、5 条输出线和 20 个交叉点组成。每个输出可以连接到至多一个输入，而每个输入可以连接到任意数量的输出。该开关的输入 1，0，3，1，2 分别连接到输出 0，1，2，3，4。

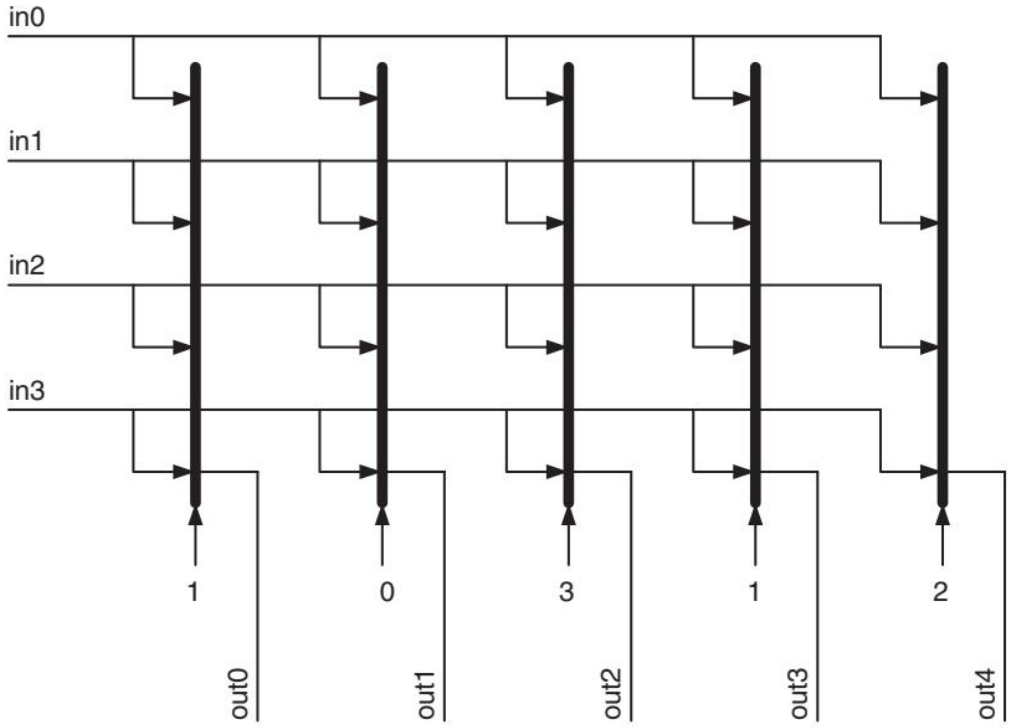


图 6.2 由五个四选一多路复用器构成的 4×5 交叉开关；实现图 6.1 中的连接

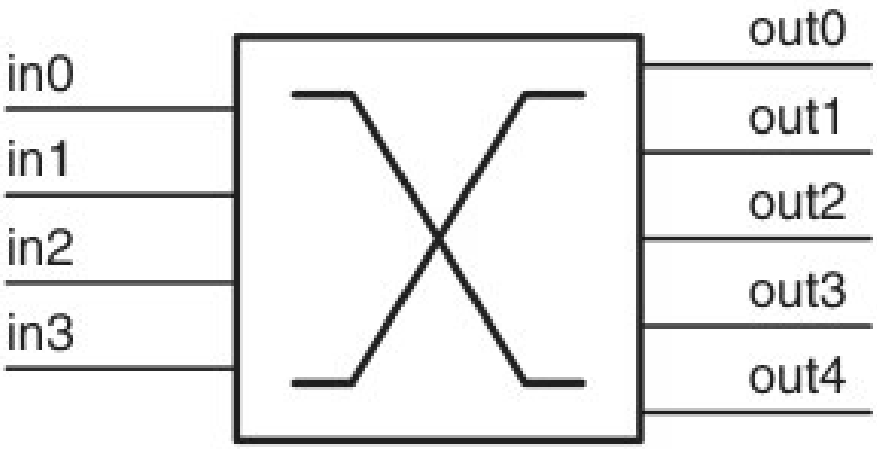


图 6.3 四输入五输出交叉开关的符号表示

长。随着 n 的增长，与成本以 $n \log n$ 增长的网络相比，交叉开关的成本迅速变得过高。此外，正如我们将在第 19 章中看到的，随着 n 变大，在分组交换机中调度交叉开关的问题变得越来越困难。

交叉开关的二次成本在开关本身的结构以及用于将几个较小的交叉开关组合成一个较大的交叉开关的构造中都很明显。从图 6.1 和 6.2 可以清楚地看出，需要 n^2 面积来布局输入和输出线的网格，需要 n^2 面积来包含 n^2 个交叉点，并且需要 n^2 面积来容纳 n 个多路复用器，每个多路复用器的面积与 $n.^4$ 成正比。

如图 6.4 所示，从小交叉开关构建大交叉开关的成本也是二次的。该图显示了如何使用 $n \times n$ 交叉开关的 2×2 阵列来构建 $2n \times 2n$ 交叉开关。由于高速信号需要点对点分配，因此需要在输入上使用 1：2 分配器来将输入信号扇出到每一行的交叉开关。互补操作在每一列中执行，其中 2：1 多路复用器选择要转发到每个输出的信号。一般来说，一个 $jn \times jn$ 交叉开关可以由 $j \times j$ 个 $j^2n \times n$ 交叉开关的阵列构建，其中 n 个 1：j 分配器用于扇出输入，n 个 j：1 多路复用器用于选择输出。

与互连网络中的大多数情况一样，实际交叉开关的成本很大程度上取决于其封装方式。完全封装在芯片内且不包含其他系统组件的交叉开关往往受引脚数量限制。也就是说，其规模受限于芯片上的引脚数 W_n，而非在芯片上实现交叉开关所需的面积。在这种情况下，交叉开关的有效成本在单芯片可封装的最大交叉开关范围内呈线性增长。然而，如果交叉开关既能容纳于一个芯片上，又连接该芯片上的其他组件，那么其二次方面积就变得重要，因为它可能不再受引脚限制。一旦交叉开关必须用多个芯片实现，如图 6.4 所示，其成本便随规模呈二次方增长。

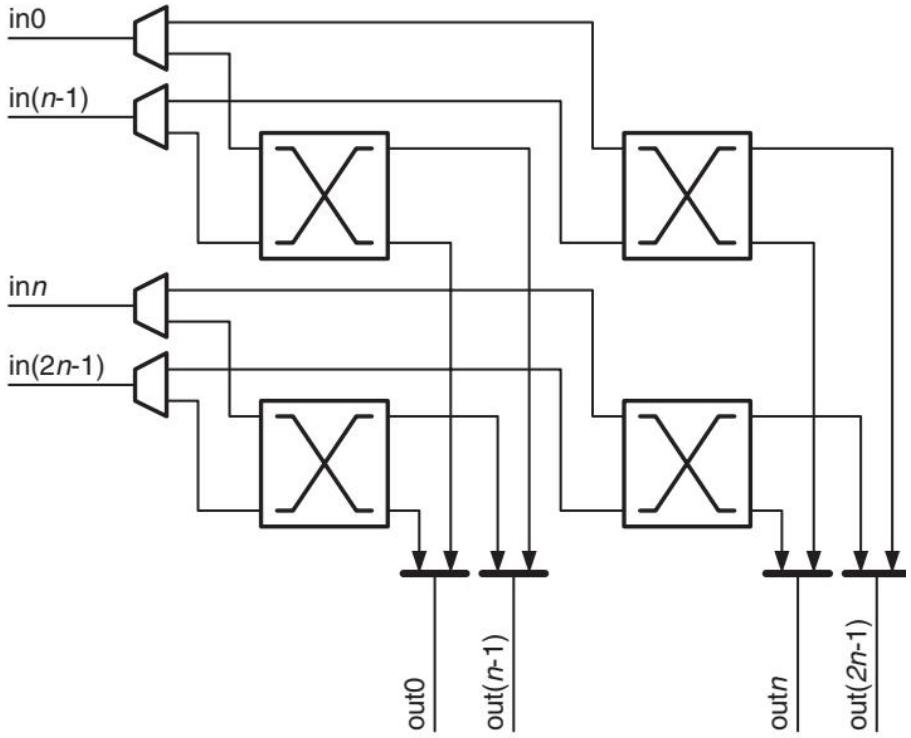


图 6.4 如图所示，一个 $2n \times 2n$ 交叉开关可以由 $4n \times n$ 交叉开关构建。每个子交叉开关处理四分之一交叉点阵列。输入端需要分配器以维持点对点连接，输出端需要多路复用器在上、下输入组之间进行选择。一般而言，一个 $jn \times jn$ 交叉开关可以由 $j^2n \times n$ 交叉开关以及 jn 个 1：j 分配器和 jn 个 j：1 多路复用器构建。

6.3 Clos 网络

6.3.1 Clos 网络的结构与特性

Clos 网络是一种三级 5 网络，其中每一级由若干交叉开关组成。对称 Clos 网络由三元组 (m, n, r) 表征，其中 m 是中间级开关的数量，n 是每个输入（输出）开关上的输入（输出）端口数，r 是输入和输出开关的数量。6 在 Clos 网络中，每个中间级开关都有一条来自每个输入开关的输入链路和一条通往每个输出开关的输出链路。因此，r 个输入开关是 $n \times m$ 交叉开关，用于将 n 个输入端口连接到 m 个中间开关；m 个中间开关是 $r \times r$ 交叉开关，用于将 r 个输入开关连接到 r 个输出开关；r 个输出开关是 $m \times n$ 交叉开关，用于将 m 个中间开关连接到 n 个输出端口。例如，图 6.5 展示了一个 $(3, 3, 4)$ Clos 网络，图 6.6 展示了一个 $(5, 3, 4)$ Clos 网络。在提及 Clos 网络的输入和输出端口时，我们将交换机 s 的端口 p 表示为 s。p。

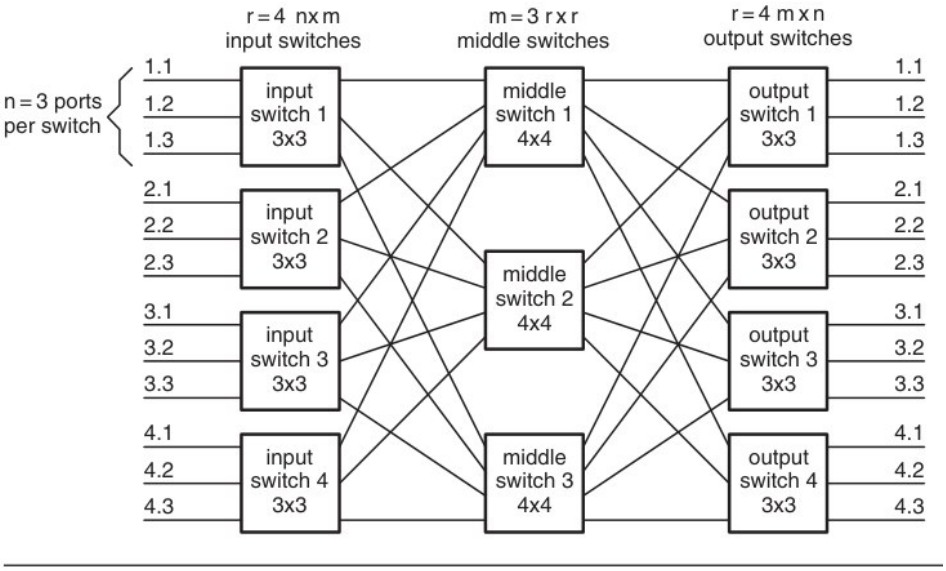


图 6.5 对称 Clos 网络的三级交换结构

通常有助于在三维空间中可视化 Clos 网络各级之间的全对全连接，如图 6.7 所示。输入和输出开关可以视为将流量水平地移向和移离垂直的中间开关。中间开关将流量从水平输入开关垂直移动到水平输出开关。这种交叉排列也是封装小型 Clos 网络的一种实用方式，因为它使各级之间的所有连接保持较短。

具有 N = rn 个终端的 (m, n, r) Clos 网络的特性由其拓扑决定。所有三级 Clos 网络都有 H = 4。该网络可以如图 6.7 所示水平或垂直二分，水平切割中间开关得到二分带宽 $B_C = mr$，垂直切割输入和输出开关得到 $B_C = 2nr = 2N$。在实践中，大多数网络通过将输入和输出开关并置并切割所有交换机间通道来封装，得到 $B_C = 2N$。输入和输出开关的度为 $\delta_{io} = n + m$，中间开关的度为 $\delta_m = 2r$。

6.3 Clos 网络

Clos 网络最有趣的性质，也是其非阻塞性质的来源，是它的路径多样性。对于具有 m

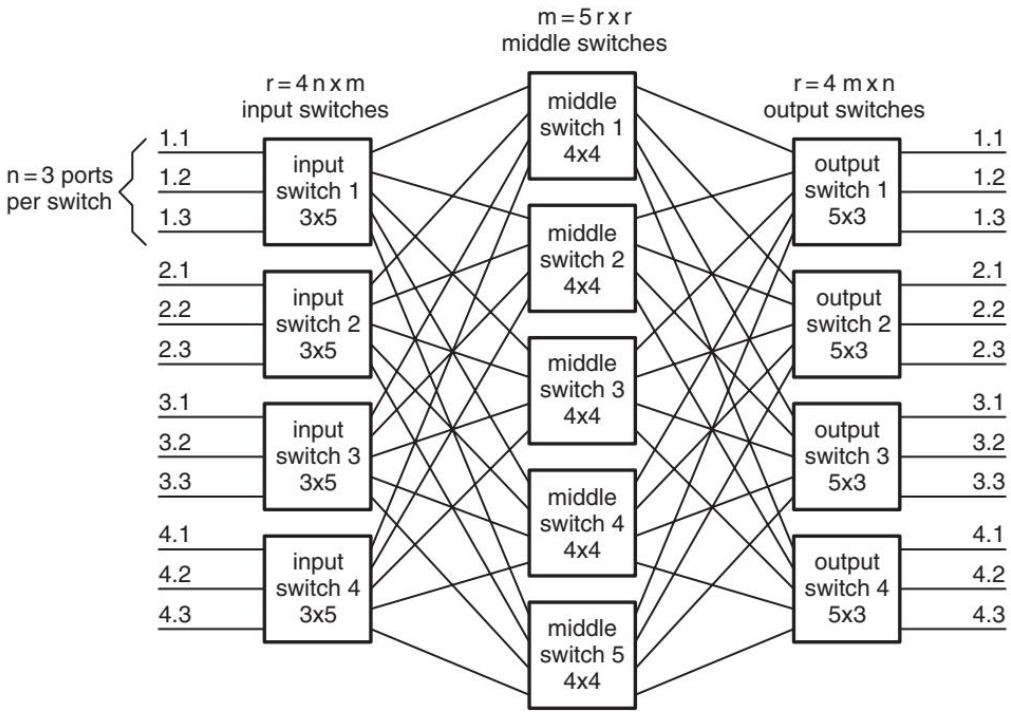


图 6.6 一个（5，3，4）Clos 网络。该网络对于单播流量是严格非阻塞的。

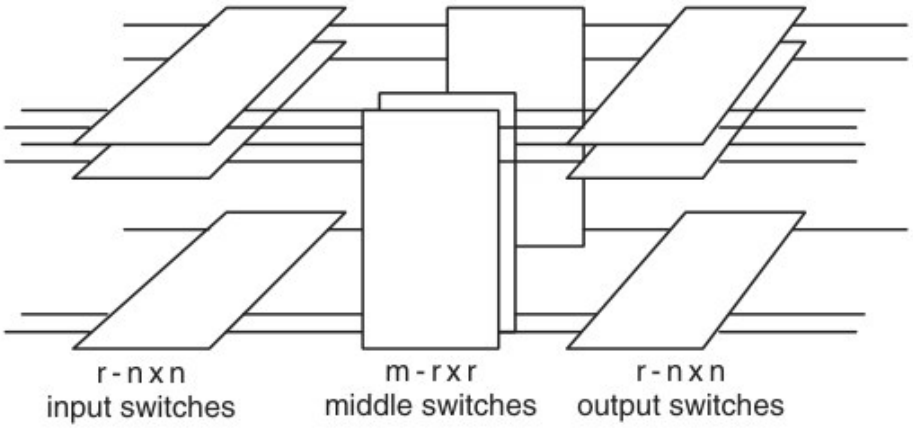


图 6.7 Clos 网络各级之间的全连接结构

个中间开关的 Clos 网络，从任意输入 a 到任意输出 b 有 $|R_{ab}| = m$ 条路由，每条路由经过一个中间级开关。

在 Clos 网络中为电路选择路由时，唯一自由的决定是在输入开关处，只要通往该中间开关的链路可用，就可以选择 m 个中间开关中的任意一个。中间开关必须选择通往输出开关的唯一链路（如果该链路忙，则路由无法进行）。类似地，输出开关必须选择所选的输出端口。因此，Clos 网络中的路由问题简化为将每个电路（或数据包）分配给一个中间开关的问题。

6.3.2 严格非阻塞 Clos 网络上的单播路由

定理一个 Clos 网络对于单播流量是严格非阻塞的，当且仅当 $m \geq 2n - 1$。6.1 证明考虑 N - 1 个呼叫的布置，其中除输入 a。i 外所有输入都已连接，除输出 b。j 外所有输出都已连接。网络中先前的呼叫已设置好，使得开关 a 上的呼叫所使用的中间开关与开关 b 上的呼叫所使用的中间开关不相交。不失一般性，我们可以假设 a 上的呼叫使用前 n - 1 个中间开关，b 上的呼叫使用后 n - 1 个中间开关。要将 a。i 连接到 b。j，我们必须找到一个当前未被 a 或 b 使用的中间开关。由于 a 使用了 n - 1 个中间开关，b 使用了不同的 n - 1 个中间开关，因此必须至少有 2n - 1 个中间开关，才能有一个开关可用于路由此呼叫。7 因此，图 6.6 中的 $(5, 3, 4)$ Clos 网络对于单播流量是严格非阻塞的，因为 m = 2n - 1 = 5。图 6.5 中的 $(3, 3, 4)$ Clos 网络不是严格非阻塞的，因为 m = 3 < 2n - 1 = 5。我们将在 6.3.3 节中看到，$(3, 3, 4)$ 网络是可重排的。

在严格无阻塞 Clos 网络上路由单播呼叫的算法如图 6.8 所示。为了从 a。i 到 b。j 路由一个单播呼叫，通过将 a 未使用的交换机列表与 b 未使用的交换机列表求交集来找到中间级交换机。根据上述计数论证，这个交集中总是至少有一个交换机。

考虑如图 6.9 所示，从输入 1.1 （1）到输出 3.3 （9）路由一条电路。在这种情况下，输入交换机 1 已经分别通过从 1.2 （2）和 1.3 （3）到 4.1 （10）和 2.3 （6）的路由阻塞了到中间交换机 1 和 2 的路径，如虚线所示。从中间交换机 4 和 5 到输出交换机 3 的路径也分别被从 3.1 （7）和 2.1 （4）到 3.1 （7）和 3.2 （8）的路由阻塞。因为输入交换机和输出交换机各自最多可以阻塞 n - 1 = 2 个中间交换机，所以最多有 2n - 2 = 4 个中间交换机被阻塞。当 m = 2n - 1 = 5 个中间交换机时，总是保证有一个中间交换机可用于处理新呼叫。在这种情况下，中间交换机 3 可用于路由新呼叫，如图中粗线所示。

对于一个更完整的路由示例，考虑排列 $\{5, 7, 11, 6, 12, 1, 8, 10, 3, 2, 9, 4\}$。也就是说，输入 1 （1.1）路由到输出 5 （2.2），输入 2 （1.2）到输出 7 （3.1），依此类推。我们还通过只考虑交换机而非端口，将排列简化为 $\{(2, 3, 4), (2, 4, 1), (3, 4, 1), (1, 3, 2)\}$。在寻找中间级分配时，我们只需要考虑交换机，一旦找到这些分配，调度第一级和最后一级就很简单，因为交叉开关元件是严格无阻塞的。输入交换机 1 连接到输出交换机 2、3 和 4；输入交换机 2 连接到输出交换机 2、4 和 1；依此类推。假设呼叫按顺序 $\{9, 6, 7, 8, 3, 12, 10, 5, 1, 11, 2, 4\}$ 应用。从输入 9 （3.3）到输出 3 （1.3）的呼叫首先应

be at least one switch in this intersection.

Consider routing a circuit from input 1.1 (1) to output 3.3 (9) as shown in Figure 6.9. In this case, input switch 1 has already blocked the paths to middle switches 1 and 2 with routes from 1.2 (2) and 1.3 (3) to 4.1 (10) and 2.3 (6), respectively, as shown with dotted lines. The paths from middle switches 4 and 5 to output switch 3 are also blocked with routes to 3.1 (7) and 3.2 (8) from 3.1 (7) and 2.1 (4), respectively. Because the input switch and output switch can each block at most $n - 1 = 2$ middle switches, there can be at most $2n - 2 = 4$ middle switches blocked. With $m = 2n - 1 = 5$ middle switches, there is always guaranteed to be one middle switch available to handle the new call. In this case, middle switch 3 is available to route the new call as shown with the bold lines in the figure.

For a more complete routing example, consider the permutation {5, 7, 11, 6, 12, 1, 8, 10, 3, 2, 9, 4}. That is, input 1 (1.1) routes to output 5 (2.2), input 2 (1.2) to output 7 (3.1), and so on. We also simplify the permutation to {(2, 3, 4), (2, 4, 1), (3, 4, 1), (1, 3, 2)} by considering only switches, not ports. We need to consider only switches when finding the middle stage assignments, and once these assignments are found, scheduling the first and last stages is trivial because the crossbar elements are strictly non-blocking. Input switch 1 has connections to output switches 2, 3, and 4; input switch 2 to output switches 2, 4, and 1; and so on. Suppose the calls are applied in the order {9,6,7,8,3,12,10,5,1,11,2,4}. The call from input 9

```
For each call (a,b)
    freeab = free(a) ∧ free(b)      middle switches available on both a and b
    middle = select(freeab)                                        pick one
    assign((a,b),middle)                                           route (a,b)
```

图 6.8 严格无阻塞 Clos 网络的单播路由算法

用，然后是从 6 （2.2）到 1 （4.3）的呼叫，依此类推。8

操作	**含义**
freeab = free(a) ∧ free(b)	输入交换机 a 和输出交换机 b 均可使用的中间交换机集合
middle = select(freeab)	从可用集合中选择一个中间交换机
assign((a,b),middle)	使用所选中间交换机为呼叫 (a, b) 建立路径

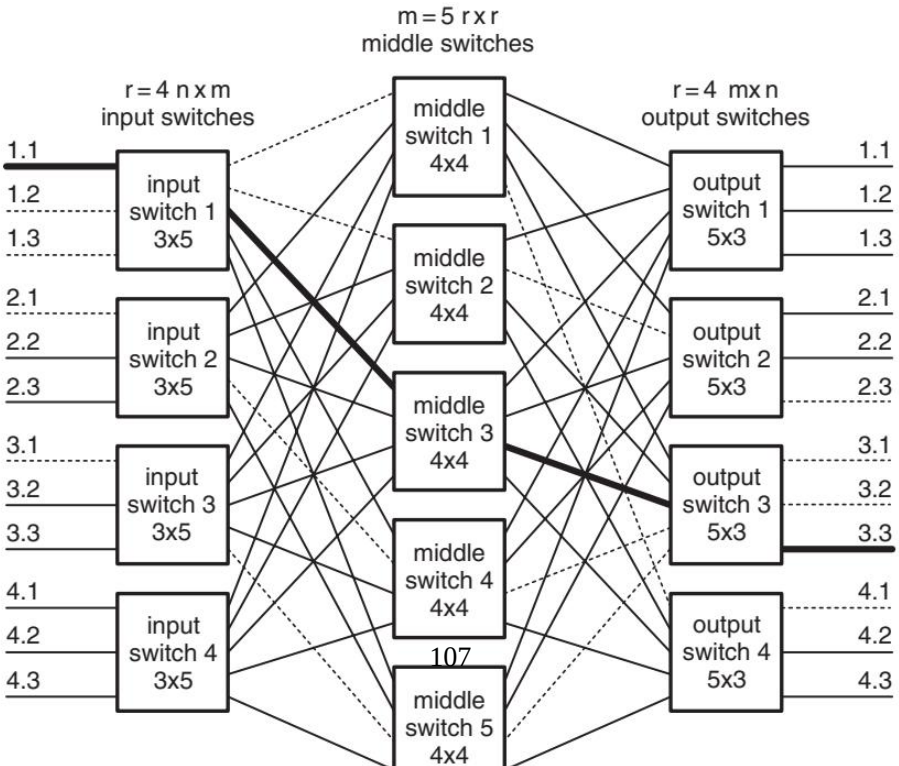


表 6.1 严格无阻塞 Clos 网络的呼叫路由示例：前三列列出输入、输出和中间级交换机，其余列给出各输入与输出交换机的空闲向量。

输入	输出	中间级	输入交换机空闲向量				输出交换机空闲向量			
			1	2	3	4	1	2	3	4
3	1	1	11111	11111	01111	11111	01111	11111	11111	11111
2	1	2	11111	10111	01111	11111	00111	11111	11111	11111
3	3	2	11111	10111	00111	11111	00111	11111	10111	11111
3	4	3	11111	10111	00011	11111	00111	11111	10111	11011
1	4	1	01111	10111	00011	11111	00111	11111	10111	01011
4	2	1	01111	10111	00011	01111	00111	01111	10111	01011
4	1	3	01111	10111	00011	01011	00011	01111	10111	01011
2	4	4	01111	10101	00011	01011	00011	01111	10111	01001
1	2	2	00111	10101	00011	01011	00011	00111	10111	01001
4	3	4	00111	10101	00011	01001	00011	00111	10101	01001
1	3	3	00011	10101	00011	01001	00011	00111	10001	01001
2	2	3	00011	10001	00011	01001	00011	00011	10001	01001

使用这种空闲集的位向量表示，我们通过将输入空闲向量 a（指示从 a 可用的交换机）与输出空闲向量 b（指示从 b 可用的交换机）进行 AND 运算，来为新呼叫从输入交换机 a 到输出交换机 b 选择中间交换机。结果向量中，每个可用于处理呼叫的中间交换机对应一个 1。因为两个空闲向量各自最多有两个 0，所以最多有四个 0，因此在 AND 结果中至少有一个 1。选择此向量中的一个 1 来确定承载新呼叫的中间交换机。例如，考虑表 6.1 中最后建立的呼叫，从输入 4（2.1）到输出 4（2.1）。在此呼叫之前，空闲向量如表倒数第二行所示。输入交换机 a = 2 的空闲向量是 10101（中间交换机 2 和 4 被阻塞），输出交换机 b = 2 的空闲向量是 00111（中间交换机 1 和 2 被阻塞）。我们将这两个空闲向量进行 AND 运算，得到组合空闲向量 00101，这表明交换机 3 和 5 可用于处理新电路。选择中间交换机 3。

路由严格无阻塞 Clos 网络的过程非常简单。然而，这种简单性是有代价的。严格无阻塞 Clos 网络几乎贵一倍，需要的中级交换机数量是相应可重排网络的 $\frac{2n-1}{n}$ 倍。因此，对于大多数应用，可重排网络更受青睐。

6.3.3 可重排 Clos 网络上的单播路由

定理 6.2 若 Clos 网络满足 $m \geq n$，则该网络是可重排的。

为了简化对 Clos 网络路由的推理，我们将一组要路由的电路表示为二分图。输入交换机是一组顶点，输出交换机是另一组顶点，每个呼叫由图中的一条边表示。路由问题变成此图中的边着色问题。每个中间交换机对应一种颜色，问题是为每条边分配一

种颜色（每个呼叫分配到一个中间交换机），使得没有顶点与两条相同颜色的边相关联（没有输入或输出多次使用同一个中间交换机）。

例如，考虑第 6.3.2 节中的路由问题。回想一下，就交换机而言，需要路由的电路集合为 {（2，3，4），（2，4，1），（3，4，1），（1，3，2）}。也就是说，输入交换机 1 有电路连接到输出交换机 2、3 和 4；输入交换机 2 连接到 2、4 和 1；依此类推。表示这组呼叫的二分图如图 6.10 所示。

图 6.11 展示了一种在可重排非阻塞网络上路由一组呼叫的算法。为了路由从输入交换机 a 到输出交换机 b 的呼叫，该算法首先寻找一个在 a 和 b 上都空闲的中间交换机。如果找到这样的空闲中间交换机，就将其分配，呼叫完成。至此，该过程与图 6.8 所示的严格非阻塞网络的过程相同。如果找不到共同的空闲中间交换机，则将一个在 a 上空闲的中间交换机 mida 分配给该呼叫，并将使用该交换机在 b 上的呼叫（c，b）移动到在 b 上空闲的中间交换机 midb。如果 midb 在 c 上已被使用，则算法循环，将（c，d）重命名为（a，b），以便在 mida 上重新路由（c，d），并根据需要重新安排更多呼叫。

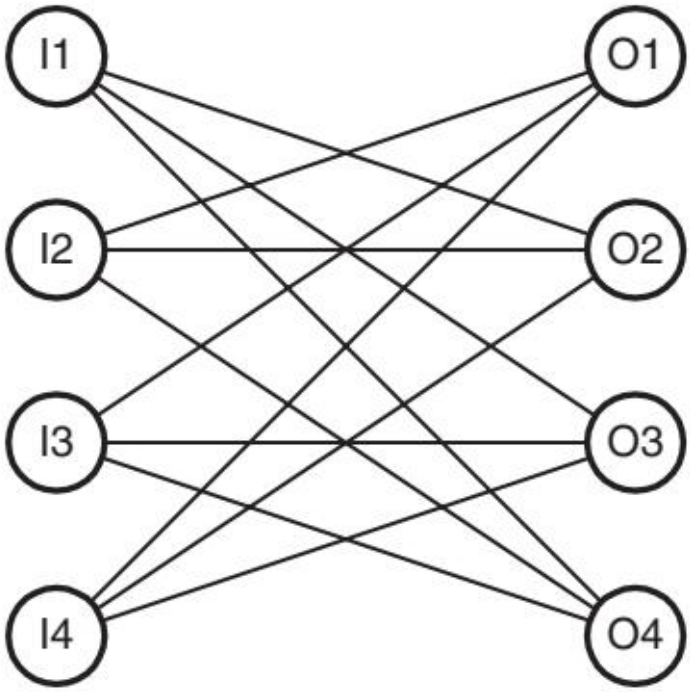


图 6.10 在 Clos 网络上需要路由的电路集合可以表示为二分图。

将循环算法应用于第 6.3.2 节的呼叫集和顺序的过程如表 6.2 所示。该算法能够在不进行重排的情况下建立呼叫，直到它尝试建立从输入 2 到输出 4 的呼叫（2，4）（表格第八行）。此时的情况如图 6.12（a）所示。输入交换机 2 正在使用中间交换机 2 路由到输出 1，而输出交换机 4 正在使用中间交换机 1 和 3 分别路由到输入交换机 1 和 3。因此，没有可用的中间交换机来处理从交换机 2 到交换机 4 的呼叫。循环算法将（a = 2，b = 4）路由到交换机 mida = 1 上。为了使该交换机可用，它将（c = 1，b = 4）重排到交换机 midb = 2。由于输入交换机 1 没有使用中间交换机 2 的呼叫，因此无需进一步重排。结果如图 6.12（b）所示。

```
For each call (a,b)
    freeab = free(a) ∧ free(b)          middle switches available on both a and b
    If freeab ≠ 0 then                  if there is one free on both a and b
        middle = select(freeab)                                   pick one
        assign((a,b),middle)                                      route (a,b)
    else
        mida = select(free(a))                                    middle switch for (a,b)
        midb = select(free(b))                                    middle switch for rearranged call
        do                                                        rearrange calls
            (c,b) = call(b,mida)                                  rearrange call using mida on b
            if(c,b) then unassign((c,b),mida)                     disconnect (c,b)
            assign((a,b),mida)                                    setup (a,b)
            if(c,b) then                                          if mida on b was used - rearrange (c,b)
                (c,d) = call(c,midb)                              get call using midb on c
                if(c,d) then unassign((c,d))                      disconnect call
                assign((c,b),midb)                                move (c,b) to midb
                if(c,d) then                                      if midb on c was in use
                    (a,b) = (c,d)                                 iterate to move this call
        while (a,b) ∧ (c,b)                                       while calls remain to be moved
```

图 6.11　Clos 网络的路径选择算法示例

当程序尝试建立从 a=4 到 b=3 的呼叫（表格第 11 行）时，会发生更大的重排，展示了算法的循环动作。尝试此呼叫之前的情况如图 6.13（a）所示。由于 I4 和 O3 之间没有共同的空闲中间交换机，该呼叫被建立在中间交换机 mida = 2 上，导致与呼叫（I3，O3）冲突，如图 6.13（b）所示。呼叫（I3，O3）是冲突链中的第一个呼叫，冲突链为（I3，O3），（I3，O1），（I2，O1），（I2，O4），和（I1，O4），如图 6.13（c）所示，必须在 mida=2 和 midb=1 之间翻转。将（I3，O3）翻转到 midb=1 会与（I3，O1）冲突。在算法的第二次迭代中，通过将（I3，O1）翻转到 mida=2 来解决此冲突，这又导致与（I2，O1）冲突，依此类推。翻转链中的所有呼叫解决了所有冲突，如图 6.13（d）所示。

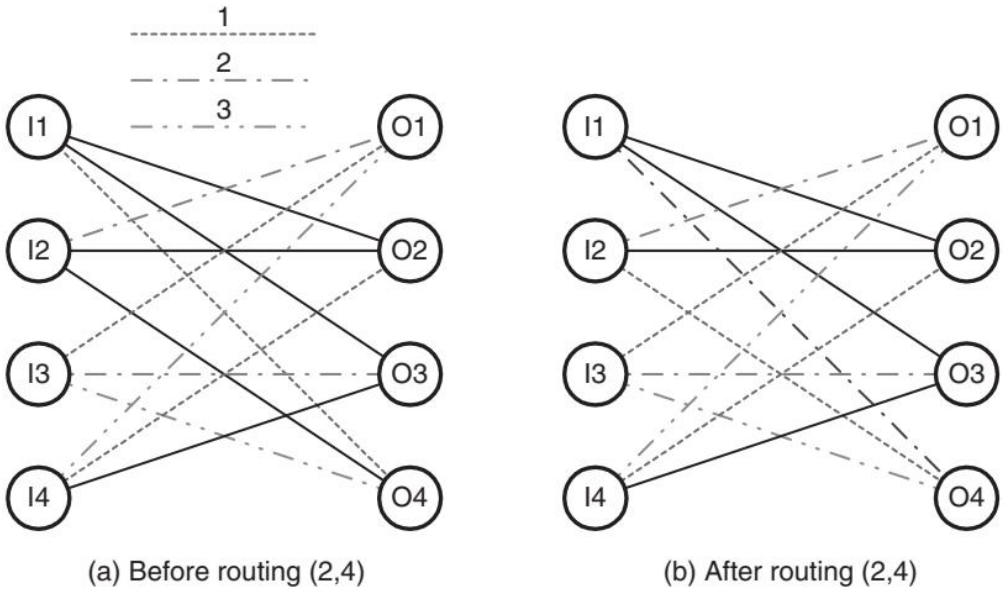


图 6.12　Clos 网络中通过重排中间级交换机完成呼叫路由的各个阶段

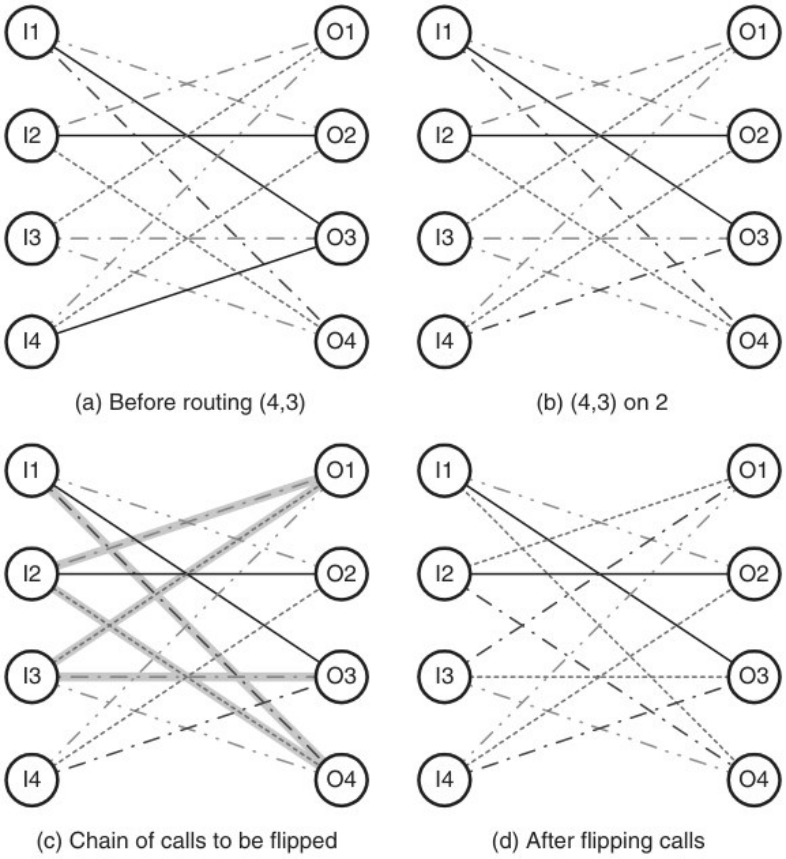


图 6.13 对图 6.10 的呼叫集合逐步进行 Clos 网络路由分配

表 6.2 Clos 网络循环路由算法的呼叫分配示例。前三个逻辑组依次记录输入／输出交换机、中间级新旧分配以及相应的空闲向量。

交换机		中间级		输入空闲向量				输出空闲向量			
入	出	新	旧	1	2	3	4	1	2	3	4
3	1	1		111	111	011	111	011	111	111	111
2	1	2		111	101	011	111	001	111	111	111
3	3	2		111	101	001	111	001	111	101	111
3	4	3		111	101	000	111	001	111	101	110
1	4	1		011	101	000	111	001	111	101	010
4	2	1		011	101	000	011	001	011	101	010
4	1	3		011	101	000	010	000	011	101	010
2	4	1		011	001	000	010	000	011	101	010
1	4	2	1	101	001	000	010	000	011	101	000
1	2	3		100	001	000	010	000	010	101	000
4	3	2		100	001	000	000	000	010	101	000

3	3	1	2	100	001	010	000	000	010	001	000
3	1	2	1	100	001	000	000	100	010	001	000
2	1	1	2	100	011	000	000	000	010	001	000
2	4	2	1	100	001	000	000	000	010	001	100
1	4	1	2	010	001	000	000	000	010	001	000
1	3	2		000	001	000	000	000	010	001	000
4	3	3	2	000	001	000	010	000	010	000	000
4	1	2	3	000	001	000	000	001	010	000	000
3	1	3	2	000	001	010	000	000	010	000	000
3	4	2	3	000	001	000	000	000	010	000	001
2	4	3	2	000	010	000	000	000	010	000	000
2	2	2		000	000	000	000	000	000	000	000

使用循环算法建立单个呼叫最多需要重排 2r - 2 个其他呼叫。

证明

这是正确的，因为循环算法在建立单个呼叫时，从不重复访问二分图的顶点。因此，它在访问最多 2r 个顶点后完成迭代。从访问第三个顶点开始重排呼叫。因此，最多重排 2r - 2 个呼叫。

我们通过归纳法证明没有顶点会被重复访问。我们知道顶点 a_1 和 b_1 不相同，因为 a 是输入交换机而 b 是输出交换机。假设对于 i < j，顶点 a_i 和 b_i 是唯一的。那么当我们访问节点 a_j 时，它也必须是唯一的，因为它是通过沿着从 b_{j-1} 出发的 mida 链接找到的，并且所有 $a_i, i < j$ 上的 mida 链接都已计入。在 a_1 上，mida 最初未连接。对于 $a_i, 1 < i < j$，mida 最初连接到 b_{i-1}，这与 b_{j-1} 不同。类似地，b_j 保证是唯一的，因为它是通过沿着从 a_j 出发的 midb 链接识别的，并且所有 $b_i, i < j$ 上的 midb 链接都已计入。因此，该链保证是无环的，因为不会重复访问任何顶点。由于总共有 2r 个顶点，算法保证在访问最多 2r 个顶点并重排最多 2r - 2 个呼叫（从呼叫（a_2, b_1）开始）后终止。

证明

（定理 6.2）当算法在调度 N = rn 个呼叫后终止时，它已经为任意单播流量生成了调度。因此，我们已经证明具有 $m \geq n$ 的 Clos 网络是可重排的。□

在重排现有呼叫 (c, d) 以建立新呼叫 (a, b) 时，希望将呼叫 (c, d) 切换到其新路由而不影响正在承载的流量。这种无中断切换可以通过同步承载呼叫 (c, d) 的输入、中间和输出交换机的切换来实现，使它们都沿旧路径传递呼叫的第 i 个单元，然后切换为沿新路径传递呼叫的第 $i + 1$ 个单元。这里的单元可以是位、字节或帧。重要的不是切换的粒度，而是它必须同步。

最后，前面的例子只考虑了每对节点之间至多有一条边的二分图。然而，一般情况下，我们需要允许二分多重图。多重图是可能包含平行边的图。例如，如果需要一个从输

入 1.1 到输出 3.3 的电路，同时还需要另一个从 1.2 到 3.1 的电路，就会出现这种情况。那么，就交换机而言，我们将有两个从交换机 I1 到交换机 O3 的电路，对应于二分多重图中从 I1 到 O3 的两条平行边。幸运的是，这种复杂性不影响本节的结果，并且循环算法在多重图上同样有效。

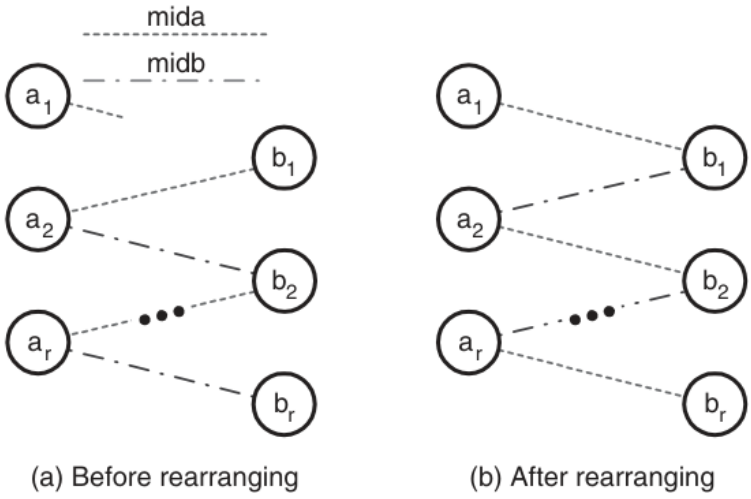


图 6.14 循环算法中顶点访问顺序的变化：(a) 重排之前；(b) 重排之后。

6.3.4 使用矩阵分解路由 Clos 网络

可重排 Clos 网络也可以利用矩阵分解进行路由。待路由的呼叫集合可以表示为矩阵 R，其中每个元素 x_{ij} 表示从输入交换机 i 到输出交换机 j 的呼叫数量。例如，前述路由示例所对应的矩阵为

$$R = \begin{bmatrix} 0111 \\ 1101 \\ 1011 \\ 1110 \end{bmatrix}.$$

这个矩阵可以分解为一组 m 个正矩阵之和，其中每个矩阵的每行和每列之和至多为 1。这个集合中的每个矩阵对应于一个中间级交换机的设置。例如，表 6.2 中所示的解决方案对应于以下矩阵分解：

$$\begin{bmatrix} 0111 \\ 1101 \\ 1011 \\ 1110 \end{bmatrix} = \begin{bmatrix} 0001 \\ 1000 \\ 0010 \\ 0100 \end{bmatrix} + \begin{bmatrix} 0010 \\ 0100 \\ 0001 \\ 1000 \end{bmatrix} + \begin{bmatrix} 0100 \\ 0001 \\ 1000 \\ 0010 \end{bmatrix} \tag{6.1}$$

在这种情况下，方程 6.1 右侧的三项是置换矩阵，对应于图 6.5 中三个中间级交换机的设置。如前一节所述，一个输入-输出交换机对之间可能有多个电路请求。这对应于原始请求矩阵中大于 1 的元素。

矩阵分解可用于路由单播或多播流量，并且可以应用于可重排和严格非阻塞 Clos 网络。分解算法的推导留作练习。

6.3.5 Clos 网络上的多播路由

在多播呼叫集 $C = \{c_1, \ldots, c_n\}$ 中，每个多播呼叫 $c_i = (a_i, \{b_{i1}, \ldots, b_{if}\})$ 指定一个输入端口 a_i 和 f 个输出端口 $b_{i1}, \ldots, b_{if}$ 之间的连接。呼叫的扇出 f 是连接的输出端口数量。呼叫集 C 受到约束，使得每个输入端口和输出端口最多包含在一个呼叫中。等价地，如果我们用交换机而不是端口来表示呼叫，每个输入交换机和输出交换机最多可以包含在 n 个呼叫中。

在 Clos 网络上路由多播流量比路由单播流量更困难，原因有三。首先，需要更多的中间交换机。要路由最大扇出为 f 的多播呼叫集，有 $m(f)$ 个中间交换机就足够了 9，如方程 6.5 所示。其次，循环算法不能应用于多播流量，除非所有扇出都在输入交换机中完成。中间交换机中的任何扇出都会导致冲突图中出现环，并使算法失败。最后，对于多播，我们有一个额外的自由度。除了选择特定的中间交换机外，我们还可以选择在哪里扇出呼叫——在输入交换机中、在中间交换机中，或部分在两者中。正如我们将看到的，正确利用这个额外的自由度是 Clos 网络中高效多播路由的关键。

为了说明多播问题，考虑在 n=3 和 r=4 的 Clos 网络上路由扇出为二的呼叫集 $C = \{(1, \{1, 2\}), (1, \{3, 4\}), (1, \{1, 3\}), (2, \{1, 4\}), (2, \{2, 4\}), (2, \{2, 3\})\}$ 的问题。如前所述，我们只显示对应于电路的交换机编号，而不是具体端口。

表 6.3 给出了该呼叫集的向量表示。该表为每个输入交换机有一列，为每个输出交换机有一列。表的第 i 行通过在对应于 a_i, b_{i1} 和 b_{i2} 的位置放置 1 来表示呼叫 $c_i = (a_i, \{b_{i1}, b_{i2}\})$。例如，表的第一行在输入交换机 1 的列中有 1，在输出交换机 1 和 2 的列中有 1，因此表示 $(1, \{1, 2\})$。

表 6.3 呼叫集合的冲突向量表示

Call	**Input**	**Output**						
1	2	3	4	1	2	3	4	
c_1	1				1	1		
c_2	1						1	1
c_3	1				1		1	
c_4		1			1			1
c_5		1				1		1
c_6		1				1	1	

两个呼叫 c_i 和 c_j 只有在它们的向量在同一列中没有 1 时才能共享同一个中间交换机。也就是说，c_i 和 c_j 可以映射到同一个中间级当且仅当 $c_i \wedge c_j = 0$。检查表 6.3 可以发现，每一对行都有非零交集。因此，除非通过在输入交换机中执行一些扇出来拆分它们，否则需要六个不同的中间级交换机来路由这六个呼叫。

在输入交换机中执行的扇出允许输出集跨多个中间级拆分。实际上，k 路输入扇出将

单个呼叫 $c_i = (a, B)$ 拆分为 k 个呼叫 $c_{i1} = (a, B_1), \ldots, c_{ik} = (a, B_k)$，其中 $B_1, \ldots, B_k$ 是不相交的，并且 $\sum_{j=1}^{k} B_j = B$。

例如，表 6.3 的呼叫集在表 6.4 中展示，其中对呼叫 c_3 和 c_6 应用了输入扇出。以这种方式拆分呼叫消除了许多成对冲突，并允许呼叫集在四个中间级交换机上路由，如表 6.5 和图 6.15 所示。

表 6.4 通过输入扇出拆分的呼叫集合

Call	Input	Output							
1	2	3	4	1	2	3	4		
c_1	1				1	1			
c_2	1						1	1	
c_{3a}	1				1				
c_{3b}	1						1		
c_4		1			1			1	
c_5		1				1		1	
c_{6a}		1				1			
c_{6b}		1					1		

表 6.5 将呼叫集合分配到四个中间级交换机

Middle Switch	Calls	Input	Output						
1	2	3	4	1	2	3	4		
1	c_1, c_{6b}	1	1			1	1	1	
2	c_2, c_{6a}	1	1				1	1	1
3	c_{3a}, c_5	1	1			1	1		1
4	c_{3b}, c_4	1	1			1		1	1

如果我们有 m 个中间交换机，那么我们的加速比是 $S = \frac{m}{n}$。为了路由一般的多播呼叫集，我们总是可以在输入级扇出 S。这样在中间级最多留下 $g = \lceil \frac{f}{S} \rceil$ 的扇出。每个要分配到中间交换机的拆分呼叫在其向量中至多有 $g + 1$ 个 1，一个用于输入交换机，g 个用于 g 个输出交换机。这个呼叫最多可以与 $(g + 1)(n - 1)$ 个其他呼叫冲突，在它有 1 的 $g + 1$ 列中每列有 n - 1 个呼叫。因此，如果满足以下条件，我们保证能够路由任意扇出为 f 的呼叫集：

$$m \geq (g + 1)(n - 1) + 1. \tag{6.2}$$

如果 f 能被 S 整除，我们可以将其重写为

$$S = \frac{f}{g} \geq \frac{(g + 1)(n - 1) + 1}{n}. \tag{6.3}$$

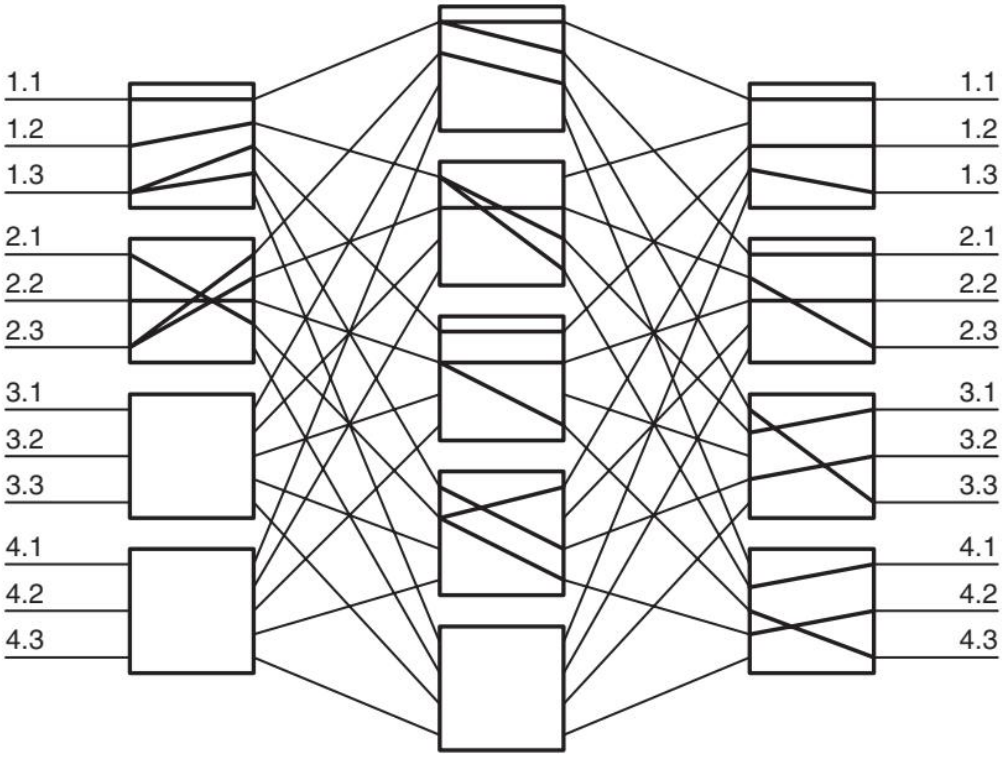


图 6.15 表 6.5 指定的路由。呼叫 c_3 和 c_6 的扇出分别在输入交换机 1 和 2 中执行。所有其他扇出在中间交换机中执行。所有六个双播呼叫在 4 个中间级交换机上路由。

在第一级交换机中为扇出拆分呼叫后，给定的输入列可能有多达 Sn 个 1。然而，我们仍然保证给定的呼叫在这一列中最多与 n 个呼叫冲突。这是因为从同一个主呼叫派生出的两个子呼叫不会冲突。

表 6.6 扇出与中间级数量的关系

n	S	m	f
2	2	4	4.0
2	3	6	12.0
2	4	8	24.0
2	5	10	40.0
3	2	6	3.0
3	3	9	9.0
3	4	1	18.0
3	5	1	30.0
4	2	8	2.7
4	3	1	8.0
4	4	1	16.0
4	5	2	26.7
48	2	96	2.0
48	3	144	6.1
48	4	192	12.3

48	5	24	20.4

因此，根据公式 6.3，给定一个有 m 个中间交换机的 Clos 网络，我们保证能够路由扇出为以下值的呼叫：

$$f \leq \frac{m(m-n)}{n(n-1)}. \tag{6.4}$$

表 6.6 给出了对于几个 n 和 S 值可以处理的最大扇出。注意，由于公式 6.4 中的 n - 1 项，对于给定的加速比，可处理的扇出随着 n 的增加而显著下降。例如，当加速比 S = 2 时，扇出从 n = 2 时的 4 下降到 n = 48 时的略高于 2。

如果我们对公式 6.4 求解 m，我们可以推导出以下表达式，用于计算扇出为 f 所需的中间交换机数量：

$$m \geq \frac{n + \sqrt{n^2 + 4n(n-1)f}}{2} = \frac{n + \sqrt{(4f+1)n^2 - 4fn}}{2}. \tag{6.5}$$

我们关于冲突呼叫数量的论证，公式 6.2 到 6.5，仅在 g 为整数时成立。在 $\frac{f}{S}$ 不是整数的情况下，我们必须向上取整到 $g = \lceil \frac{f}{S} \rceil$。

上述计算的界限可以通过图 6.16 所示的简单贪心算法实现。该算法首先拆分呼叫集 C 中的呼叫，创建一个最大扇出为 g 的拆分呼叫集 C_s。然后以贪心方式将 C_s 中的呼叫分配到中间交换机。对于每个呼叫 c_i，算法扫描中间交换机。对于每个中间交换机 m_j，检查 c_i 是否与已分配到 m_j 的呼叫冲突。如果没有冲突，则将 c_i 分配到 m_j，算法继续下一个呼叫。


```
C_s =                                                    initialize split call set empty
for each call c_i ∈ C                                    first split calls for first stage fanout
    split c_i into c_i1, ..., c_iS each with fanout ≤ g = ⌈f/S⌉
    add c_i1, ..., c_iS to C_s
for each call c_i ∈ C_s
    for each middle switch m_j ∈ M                       find first middle stage that doesn't conflict
        if c_i doesn t con  ict with calls already assigned to m_j
            assign c_i to m_j
            break                                        skip to next call c_i
```

图 6.16 在 Clos 网络中路由组播呼叫的贪心算法

为判定 c_i 是否与已分配到 m_j 的呼叫冲突，引入一个 $m \times 2r$ 的分配矩阵 A。矩阵的行对应中间级交换机，列对应输入级或输出级交换机，所有元素初始为 $\varnothing$。将拆分呼叫 $c_k = (a_k, \{b_{k1}, \ldots, b_{kg}\})$ 分配到 m_j 时，将对应的元素置为其父呼叫 $p(c_k) \in C$：

$$A_{m_j,a_k} = A_{m_j,b_{k1}} = \cdots = A_{m_j,b_{kg}} = p(c_k).$$

当检查呼叫 $c_i = (a_i, \{b_{i1}, \ldots, b_{ig}\})$ 是否可以分配到 m_j 时，只要相关矩阵元素 $A_{m_j,a_i}, A_{m_j,b_{i1}}, \ldots, A_{m_j,b_{ig}}$ 均为 $\varnothing$，或等于该呼叫的父呼叫 $p(c_i)$，便不存在冲突。

定理 6.4 贪心算法（图 6.16）能够达到式（6.2）给出的中间级数量界，即足以路由扇出为 f 的组播流量。

证明 使用反证法。假设拆分呼叫集合 C_s 中有一个呼叫无法分配给 $m \geq (g+1)(n-1)+1$ 个中间级交换机中的任何一个。那么，该呼叫必须与每个中间级交换机上至少一个已分配呼叫冲突，即一共至少与 m 个呼叫冲突。然而，根据前面的界限，它最多只能与 $(g+1)(n-1)$ 个呼叫冲突，产生矛盾。故定理成立。

由于算法可以不移动已经建立的呼叫，就能路由任意扇出为 f 的呼叫集合，因此直接得到以下结论。

定理 6.5 如果式（6.4）成立，则 (m,n,r) Clos 网络对于扇出为 f 的组播流量是严格非阻塞的。

6.3.6 多于三级的 Clos 网络

使用 $n \times n$ 交叉开关作为构建块，我们可以构建一个具有 n^2 个端口的可重排非阻塞 (n,n,n) 三级 Clos 网络。如果我们需要超过 n^2 个端口，我们可以将这个 $n^2 \times n^2$ 端口的 Clos 网络用于 Clos 网络的每个中间交换机，使用 $n \times n$ 交叉开关作为输入和输出交换机。这给了我们一个具有 n^3 个端口的五级 Clos 网络。一般来说，对于任何整数 i，我们可以通过使用 n^i 个 $n \times n$ 交叉开关作为第一级，n - 2i - 1 级 Clos 网络作为中间级，以及 n^i 个 $n \times n$ 交叉开关作为最后一级，来构建一个具有 n^{i+1} 个端口的 $2i+1$ 级可重排 Clos 网络。类似的构造可用于创建 $2i+1$ 级严格非阻塞 Clos 网络，其中使用 n^i 个 $n \times (2n-1)$ 输入交换机，以及 $2n-1$ 个 n^i 端口 2i - 1 级 Clos 网络作为中间交换机。

例如，图 6.17 展示了一个由 2×2 交叉开关组成的五级 $(2,2,4)$ Clos 网络。该网络中的两个 4×4 中间交换机本身都是 $(2,2,2)$ Clos 网络。

通过在多于三级的 Clos 网络上使用图 6.11 或图 6.16 的算法从外向内工作来路由流量。我们首先将每个呼叫分配到 n^i 个中间交换机之一。这将一个 $(2i+1)$ 级调度问题分解为 n 个 $(2i-1)$ 级调度问题。我们重复这个过程，直到在 i 步后达到调度 n^i 个 $n \times n$ 交叉开关的阶段。在第 j^{th} 步，我们调度 n^{j-1} 个 $(2i-2j+3)$ 级网络。

考虑在一个由 $n \times n$ 交叉开关构成的五级 Clos 网络（$i=2$）上调度单播呼叫。第一步是将每个呼叫分配到 n 个中间开关之一。这使用图 6.11 的算法来调度到一个 (n,n,n^2) Clos 网络。这给出了两个外部级的开关设置，并为用作中间开关的 n 个三级 Clos 网络中的每一个提出了路由问题。然后我们向内移动，对用作中间开关的每个 Clos 网络应用相同的过程。对于 n 个中间开关中的每一个，我们使用图 6.11 的算法将分配给这个 (n,n,n) Clos 中间开关的呼叫进行路由。这将每个呼叫分配到该 Clos 中的一个交叉开关（总体上是 n^2 个中间级交叉开关之一）。

图 6.17 的五级 Clos 网络对于单播流量是可重排非阻塞的。外部 $(2,2,4)$ 网络是可重排的，因此可以将每个呼叫分配到两个中间级开关之一。此外，每个 $(2,2,2)$ 中间级网

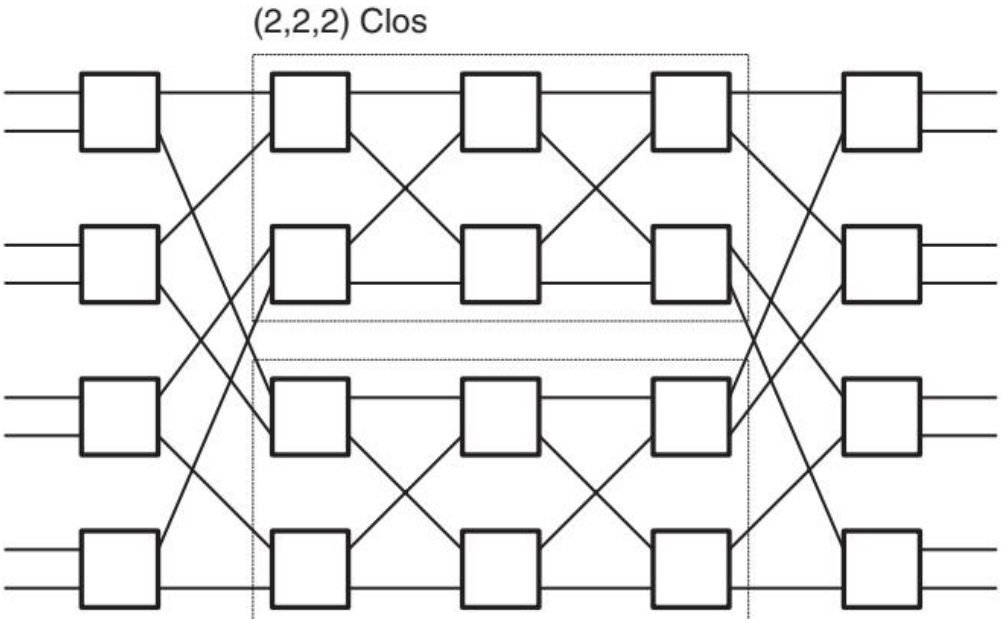


图 6.17 由两个 $(2,2,2)$ Clos 网络构成中间交换级的 $(2,2,4)$ 可重排 Clos 网络（Beneš 网络）

络都是可重排的，因此可以路由分配给它们的所有呼叫。

为了使五级 Clos 网络对于单播流量严格非阻塞，每个输入级都需要扩展。例如，一个与图 6.17 网络相当的严格非阻塞网络将需要一个 $(3,2,4)$ 外部网络，在输入级有四个 2×3 开关，在输出级有四个 3×2 开关，以及三个 4×4 中间级子网络。为了严格非阻塞，四个中间级子网络中的每一个都必须实现为 $(3,2,2)$ Clos 网络，具有两个 2×3 输入开关、两个 3×2 输出开关和三个 2×2 中间开关。五级严格非阻塞网络在中点处有九个 2×2 开关，而五级可重排网络只有四个 2×2 开关。这是因为第一级和第二级都需要扩展。如果在每个输入级使用两倍的扩展，10 一个 $2n+1$ 级严格非阻塞 Clos 网络需要总共 2^n 的扩展。

要在具有超过三级的 Clos 网络中路由多播流量，必须决定在哪里执行扇出。扇出可以在网络的任何级执行。对这个问题的完整讨论超出了本书的范围。然而，当在 $2n+1$ 级 Clos 网络上路由 f 个多播呼叫的扇出时，一个好的启发式是在前 $n+1$ 级的每一级上执行 $f^{\frac{1}{n+1}}$ 的扇出。

6.4 Beneš 网络

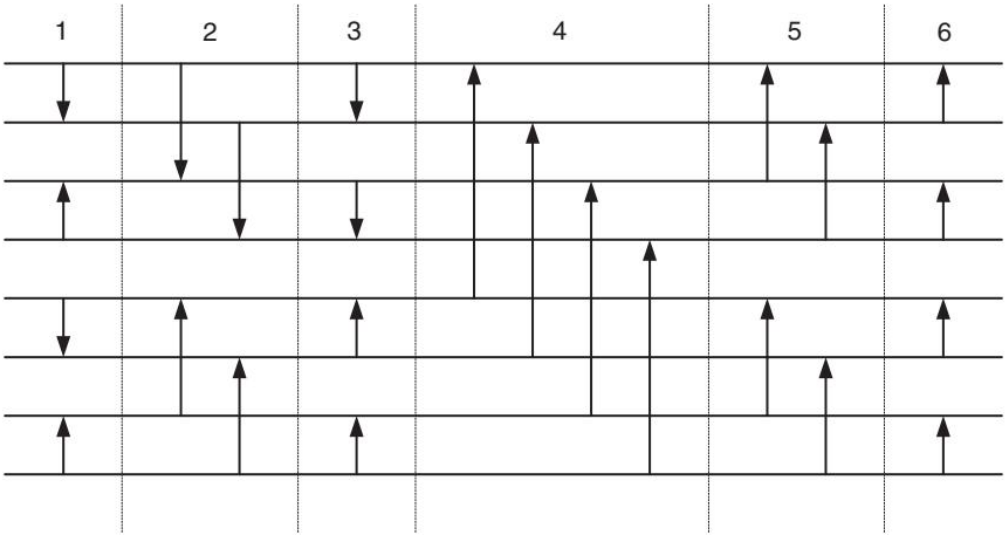
由 2×2 开关构成的 Clos 网络——例如图 6.17 的网络——也称为 Beneš 网络。这些网络之所以引人注目，是因为它们以可重排非阻塞的方式连接 $N=2^i$ 个端口所需的交叉点数量最少。如上所述，要用 2×2 开关连接 $N=2^i$ 个端口，需要 $2i-1$ 级 2^{i-1} 个 2×2 开关。这总共给出 $(2i-1)2^{i-1}$ 个开关。每个 2×2 开关有 4 个交叉点，交叉点总数为 $(2i-1)2^{i+1}$。

敏锐的读者会注意到，一个 $2i-1$ 级 Beneš 网络等效于两个 2 元 i 飞网络背靠背，并将相邻级融合。通常，Beneš 网络和其他多级 Clos 网络通过沿中间开关级折叠网络来

实现。在 $2i-1$ 级折叠 Clos 网络中，第 j 级和第 $2i-j$ 级位于同一位置并共享共同的封装。这种安排利用了网络的对称性来降低布线复杂性。

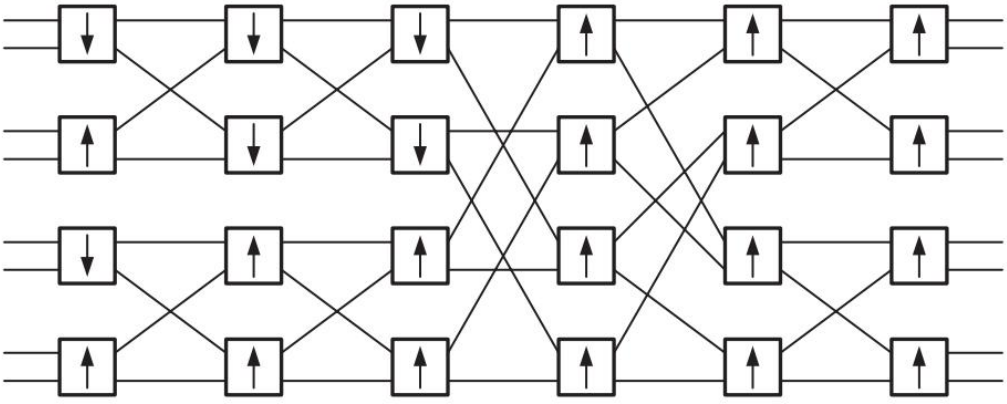
6.5 排序网络

一个 N 输入排序网络在其 N 个输入端接受一组带有唯一排序键的 N 条记录，并在其 N 个输出端按排序顺序输出这些记录。通过使用每条记录的输出地址作为其排序键，排序网络可以用作非阻塞网络。11 因为排序网络能够将排序键的任何排列排序为有序顺序，所以它能够路由任何排列。



（a）Batcher 双调排序

（a）Batcher 双调排序



（b）双调排序网络

（b）双调排序网络

图 6.18

Batcher 双调排序网络。（a）Batcher 双调排序通过将两个长度为 2^i 的有序序列视为长度为 2^{i+1} 的双调序列，然后将其合并为长度为 2^{i+1} 的有序序列来操作。（b）实现这种

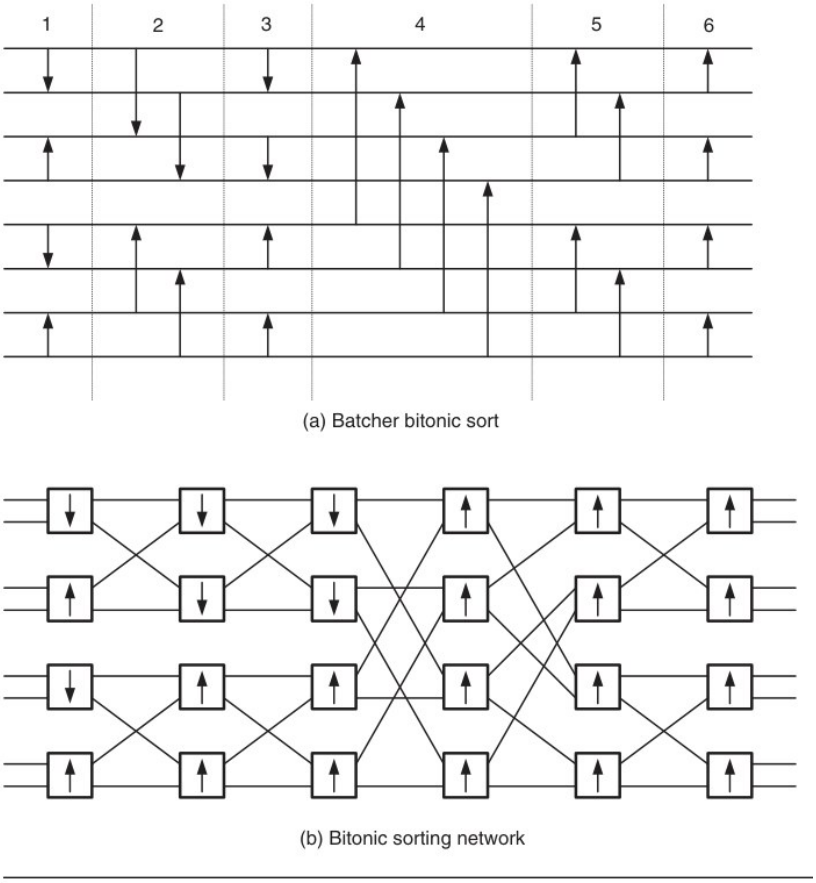


图 6.18 Batcher 双调排序网络及其合并过程

排序的排序网络由 2×2 开关构成，这些开关根据排序键的相对值进行路由。

图 6.18 显示了一个 Batcher 双调排序网络，并说明了该网络所基于的双调排序算法。该排序算法利用了双调序列的性质，双调序列在循环查看时最多有一个升序子序列和一个降序子序列（两个单调子序列形成一个双调序列）。如图 6.18（a）所示，双调排序算法从左侧的 2^n（此处为 8）个未排序搜索键开始，在右侧生成一个有序序列。在此图中，每个箭头表示一个操作，该操作比较左侧两条记录的键，并在必要时交换它们，以便具有最高键的记录位于右侧箭头的头部。第一阶段以交替方向对偶数和奇数键进行排序，使得第二阶段的输入由两个双调序列组成，一个来自输入 0–3，另一个来自输入 4–7。然后阶段 2 和 3 将每个 4 长双调序列合并为 4 长单调序列。结果是一个 8 长双调序列。然后阶段 4、5 和 6 将此双调序列合并为单调序列，排序完成。例如，表 6.7 说明了对八个数字序列的排序。

一般来说，一个 $N = 2^n$ 输入的比特 onic 排序需要 $S(N) = \frac{(\log_2 N)(1+\log_2 N)}{2}$ 个阶段，其中包括 $S(N/2)$ 个阶段将输入排序为两个长度为 2^{n-1} 的单调序列组成的比特 onic 序列，以及 $\log_2 N$ 个阶段将这个比特 onic 序列合并为一个长度为 N 的单调序列。

图 6.18（b）所示的排序网络直接源自该算法。这里，图 6.18（a）中的每个箭头被一个通过比较键进行路由的 2×2 开关所取代。

表 6.7 双调归并排序示例

Row	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
Input	4	6	2	7	3	1	5	0
Stage 1	4	6	7	2	1	3	5	0
Stage 2	4	2	7	6	5	3	1	0
Stage 3	2	4	6	7	5	3	1	0
Stage 4	2	3	1	0	5	4	6	7
Stage 5	1	0	2	3	5	4	6	7
Stage 6	0	1	2	3	4	5	6	7

一些异步传输模式（ATM）交换机将 Batcher 排序网络与蝶形网络结合使用（这通常称为 Batcher-榕树网络，其中榕树是蝶形网络的别名）。Batcher 网络按目标地址对输入信元进行排序。在 Batcher 的输出端，一个陷阱阶段检测发往同一输出的信元，并除一个外全部移除。剩余的按序排列且都发往不同输出的信元随后被混洗并输入到蝶形网络中。由于它们的顺序，蝶形网络保证能够路由它们。（见练习 6.4。）

6.6 案例研究：Velio VC2002（Zeus）梳理交换机

Velio VC2002 是一款单芯片时分复用（TDM）电路交换机（图 6.19）。VC2002 接受 72 路同步光网络（SONET）STS-48 2.488 Gbits/s 串行输入流，并产生 72 路 STS-48 输出流。每个流包含 48 路 51.83 Gbits/s 的 STS-1 流，以字节为基础复用。也就是说，每个 3.2 纳秒的字节时间或时隙，线路上承载来自不同 STS-1 的一个字节——首先是来

自通道 0 的一个字节，然后是通道 1，依此类推直到通道 47。经过 48 个时隙后，模式重复。TDM 交换机，也称为交叉连接或梳理交换机，在时间和空间上进行交换，将输入 STS-1 时隙映射到输出 STS-1 时隙。12

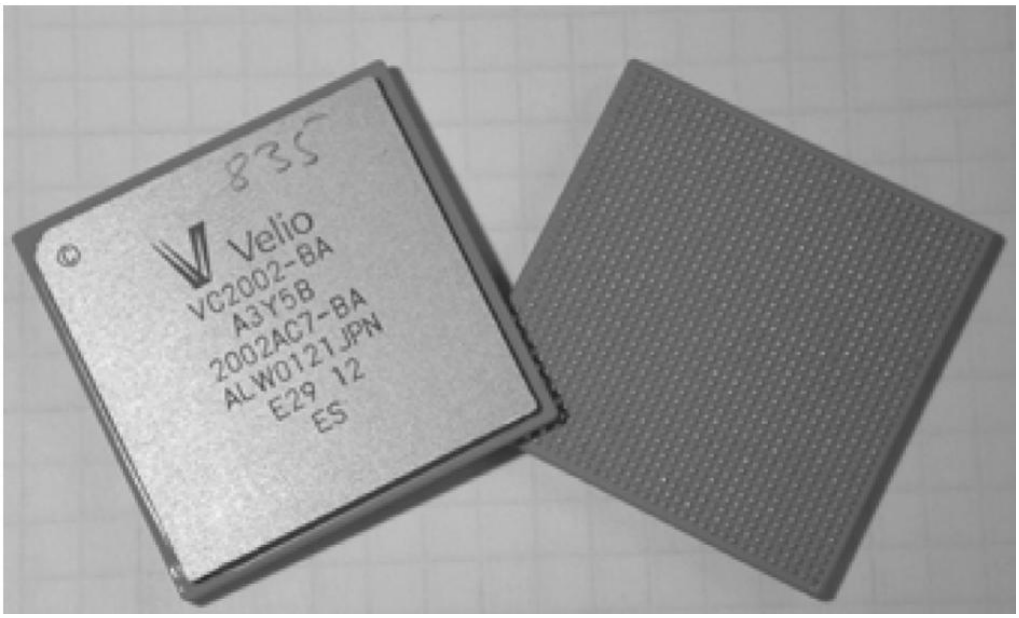


图 6.19 Velio VC2002 是一款单芯片 72×72 STS-48 梳理交换机。该芯片采用 37.5mm×37.5mm 1296 球球栅阵列封装。

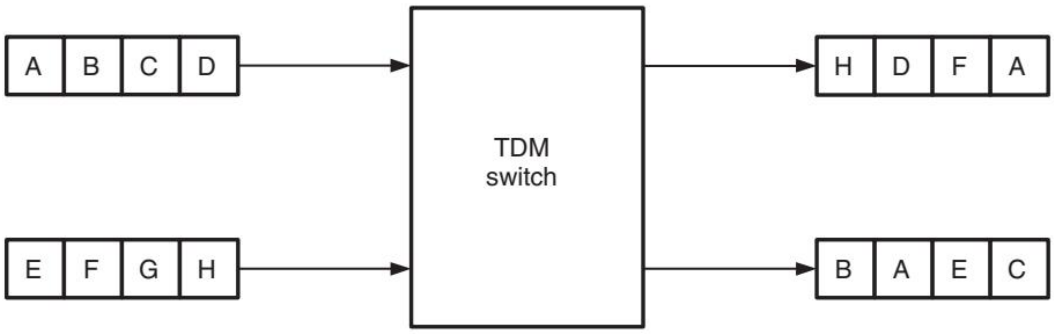


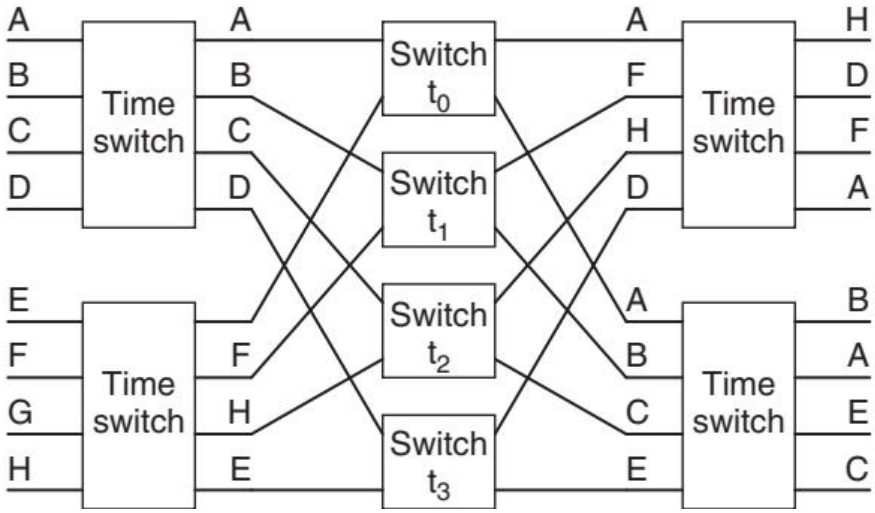
图 6.20 TDM 交换机接受并生成时分复用流，并将输入时隙的内容交换到输出时隙的内容中。例如，0.2 上的通道 C（输入 0，时隙 2）被交换到 1.3。

图 6.20 展示了两个输入和输出、每个有四个时隙的简化情况下的 TDM 交换示例。该示例包括单播连接——例如，0.1 上的 B（输入 0，时隙 1）被交换到 1.0（输出 1，时隙 0）——和多播连接——0.0 上的 A 被交换到 0.3 和 1.1。VC2002 执行类似的功能，但在 72 个输入、每个 48 个时隙上进行。

TDM 梳理交换机构成了大多数城域网和长途语音及数据通信网络的主体。13 这些网络大多实现为互连的 SONET 环集合（因为环的恢复特性）。梳理交换机充当连接多个环的交叉连接，以及将较慢的馈送复用到环上或从环上分下的分插复用器。

TDM 交换机的配置通常变化缓慢，除非在保护事件期间。新的 STS-1 电路以每分钟最多几个的速率进行配置。然而，当网络发生故障时（例如，挖掘机挖断光纤束），

VC2002 中的 3456 个连接中的很大一部分可能需要在几毫秒内切换以避免服务中断。TDM 交换机可以利用输入和输出流被复用的事实，在时域中实现 Clos 网络的第一和第三级，如图 6.21 所示。该图展示了如何在没有 8×8 交叉开关复杂性的情况下实现图 6.20 的 2 输入 4 时隙示例。取而代之的是，实现了一个三级 Clos 网络，其中 $m = 4$，$n = 4$，r = 2。$r = 2 \times 4(n \times m)$ 输入和输出交换机实现为时隙交换器（TSI），而 $m = 4 \times 2(r \times r)$ 中间交换机实现为单个时分复用 2×2 交换机。TSI 是一个 2n 项存储器。当一半项（n 项）按顺序被输入流填充时，通过以任意顺序读取另一半项（如实现输入交换机配置所需）来创建输出流。每次输入流填满 n 项时，过程重复，输出流读出这些项，而输入流填充另一半——这些项刚刚被读出。由于图 6.21（b）中 TSI 的输出是时分复用的，图 6.21（a）中的四个 2×2 交换机可以由单个交换机实现。在每个四个时隙中，交换机充当该时隙的中间级。



(a)

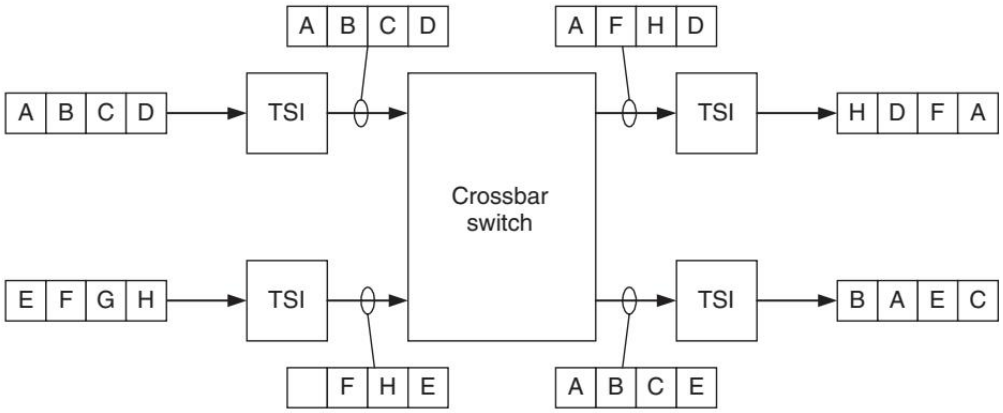


图 6.21 图 6.20 示例的时间-空间-时间（TST）TDM 交换机实现。（a）概念上，输入在时域中由两个 4×4 交换机交换，然后，在四个时隙 t_0 到 t_3 的每个中，两个输入交换机的输出由 2×2 空间交换机交换。最后，一对输出时间交换机重新排序输出。（b）输入和输出时间交换机由 TSI 实现。（a）中的四个 2×2 交换机由单个 2×2 交换机实现，在每个时隙上以不同的配置操作。

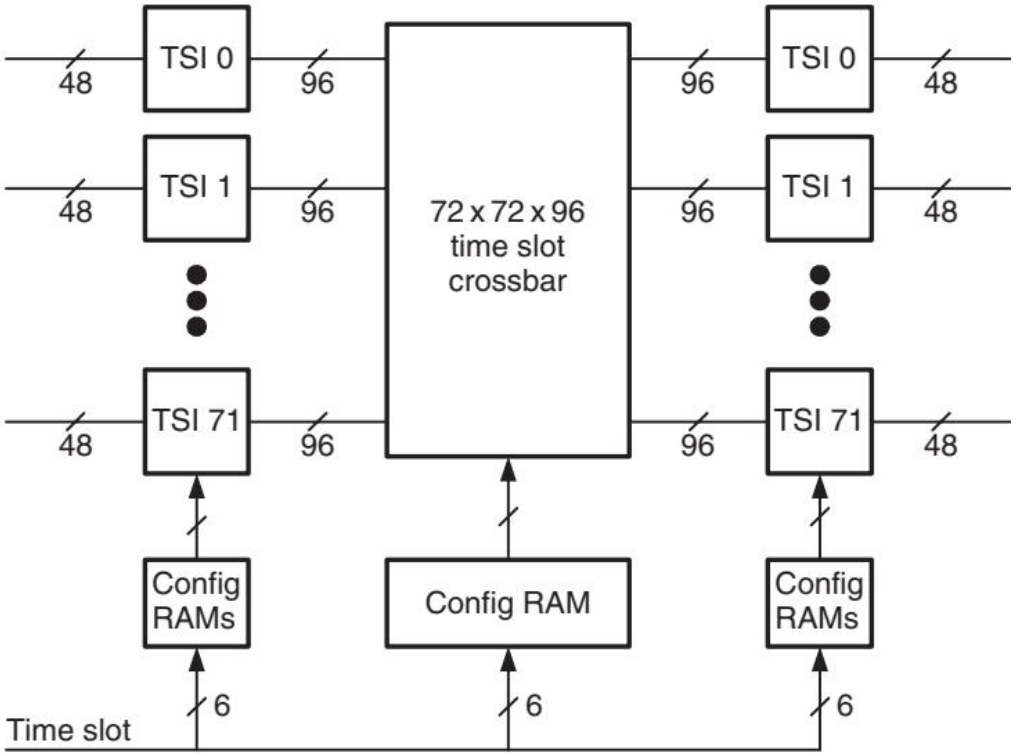


图 6.22 VC2002 电路交换机的结构：输入 TSI、空分交换网络与输出 TSI

图 6.22 展示了 VC2002 的框图。14 与图 6.21 的示例类似，VC2002 使用三级 TST Clos 网络。然而，在此情况下，Clos 的参数为 $(m, n, r) = (96, 48, 72)$。共有 72 个输入通道，每个通道有 48 个时隙。为了使交换机对单播流量严格无阻塞，对双播流量（扇出 = 2）可重排无阻塞，该交换机具有两倍的加速比。对于每个输入交换机的 48 个输入，有 96 个逻辑中间级。15 交换机由一组配置存储器控制，这些存储器由当前时间步寻址。对于每个时间步，配置存储器指定从每个输入 TSI 读取哪个位置（时隙）、哪个输入端口应连接到交换机的每个输出端口，以及从每个输出 TSI 读取哪个位置。

VC2002 的调度方法是为每个必须通过交换机的 STS-1 呼叫分配一个中间级交换机（时隙）。由于 VC2002 对单播流量严格无阻塞，中间级交换机的分配可以使用图 6.8 的相交法或图 6.11 的循环算法来执行。双播流量可以使用图 6.16 的贪心算法进行调度。或者，双播流量可以通过将每个双播呼叫拆分为两个单播呼叫，并将中间级交换机拆分为两个相等的子集，使用循环算法进行调度。每个子集上调度一个单播呼叫。由于每个子集有 m/2 = n 个交换机，该子集对于这些单播呼叫是可重排的。

在实践中，VC2002 可以通过如第 6.3.5 节所述拆分呼叫，以极低的阻塞概率处理任意扇出的多播呼叫。我们将高扇出呼叫的阻塞概率计算留作练习 6.5。

单个 VC2002 提供 180 Gbits/s 的 STS-1 整理带宽，足以交换 3,456 个 STS-1。如果需要更大的整理交换机，可以将多个 VC2002 连接在 Clos 网络中，如图 6.23 所示。该图展示了如何将 120 个 VC2002 连接在折叠 Clos 配置中，以交换高达 4.3 Tbits/s（1,728 个 STS-48 或 82,944 个 STS-1）。第一排的 72 个芯片每个都充当一个 $24 \times 48\times$ STS-48 第一级输入交换机和一个 $48 \times 24\times$ STS-48 第三级输出交换机。第二排的 48 个芯片每

个构成 $72 \times 72\times$ STS-48 中间级交换机。

图 6.23 的系统在逻辑上是一个五级 Clos 网络。由于每个芯片内部本身就是一个三级交换机，可能看起来我们创建了一个九级网络。然而，我们可以忽略除最外层 TSI 之外的所有 TSI，并将网络视为一个 TSSST 网络，其中包含一个 48×48 TSI，随后是一个 24×48 空分交换器，然后是一个 72×72 空分交换器、一个 48×24 空分交换器和一个 48×48 TSI。每条路径上的其他四个 TSI 保持直通配置。练习 6.6 探讨了将网络视为七级 TSTSTST 的可能性。

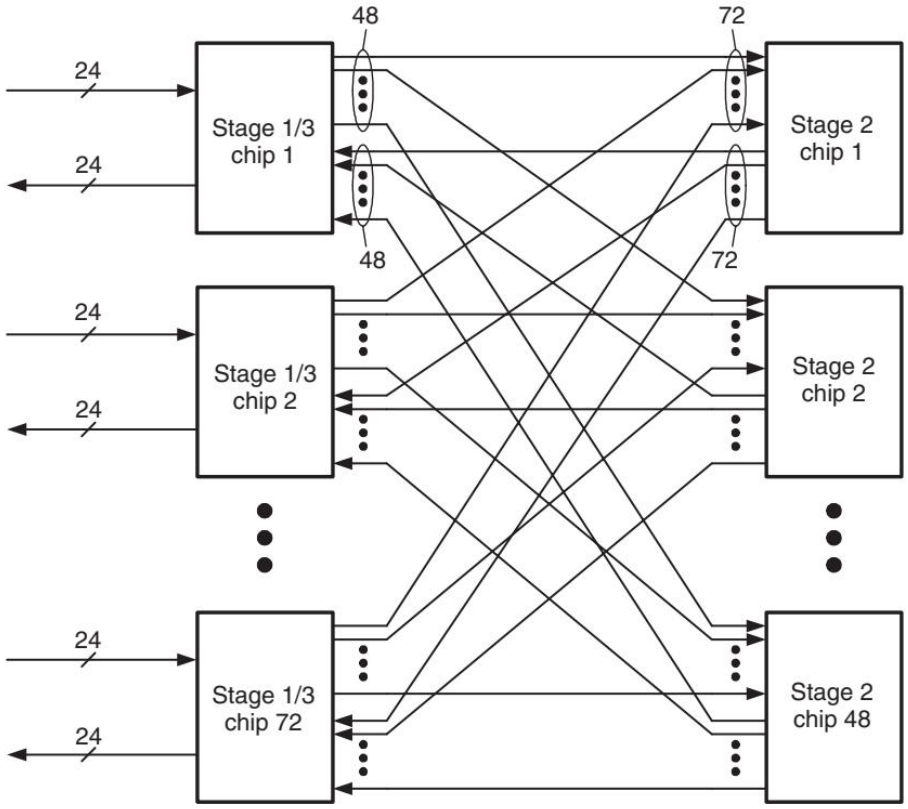


图 6.23 使用 120 个 VC2002 的折叠 Clos 网络提供 4.3 Tbits/s 的 STS-1 整理带宽。第一排包含 72 个芯片，每个芯片拆分其输入和输出，以同时提供一个 $24\times48\times$ STS-48 输入交换机和一个 $48\times24\times$ STS-48 输出交换机。第二排的 48 个芯片构成 $48 \times 72\times$ STS-48 中间交换机。

为了调度我们的逻辑五级网络，我们首先将中间三个空分交换级视为 48 个 $3,456 \times 3,456$ 空分交换器，每个时隙一个。我们通过调度这个 $(m, n, r) = (48, 24, 3456)$ Clos 网络，将每个呼叫分配到 48 个时隙之一。这个解决方案随后留下 48 个子问题，每个时隙一个，即将该时隙中的每个呼叫分配到 48 个物理中间级交换机之一。对于每个子问题，我们在由单个时隙的三个空分交换器形成的 $(m, n, r) = (48, 24, 72)$ Clos 网络上调度该中间级时隙中的所有呼叫。

6.7 文献注释

关于非阻塞网络的开创性论文由 Clos 于 1953 年发表 [37]。该论文介绍了 Clos 网络，并描述了其严格无阻塞的要求。Beneš、Slepian 和 Duguid 发现，小得多的 Clos 网络可以是可重排无阻塞的 [17，171，64]。用于寻找这些安排的循环算法可以追溯到 König，他证明了最大度为 Δ 的二分图可以使用 Δ 种颜色进行边着色 [103]。Cole 提出了几种更高效的边着色算法 [39，40]，矩阵分解由 Waksman 处理 [190]。Beneš 1965 年的经典著作 [18] 推导了实现可重排无阻塞网络所需的交叉点数量的界限，并引入了 Beneš 网络（Clos 网络的一种特殊形式）来实现这一界限。Batcher 1968 年的论文介绍了双调排序和 Batcher 排序网络 [14]。关于排序网络的详细论述可以在 Knuth [102] 中找到。Clos 网络中的多播首先由 Masson 和 Jordan 考虑 [120]，此后许多研究者进行了研究。最近的结果包括 Yang 和 Masson 的近最优多播调度算法 [196]。

6.8 练习题

6.1 一个 27 端口的 Clos 网络。使用 3×3 交叉开关，画出一个恰好有 27 个端口的可重排 Clos 网络。

6.2 最大规模的 Clos 网络。使用 $n \times n$ 交叉开关作为构建模块，你能用 k 级（k 为奇数）构建多大的 Clos 网络？假设你的 Clos 网络只需要对单播流量是可重排的。

6.3 循环算法的性能。实现图 6.11 中描述的循环算法。对一个 $(5, 5, 5)$ Clos 网络运行该算法：从空网络（无连接）开始，每次随机选择一个连接加入，直到所有输入和输出端口都被占用。我们知道每个连接所需的最大重排次数是 2r - 2 = 8，但在你的实验中平均需要多少次重排？现在修改原始算法，尝试进一步减少平均重排次数。解释你的修改，并报告使用原始算法和修改后算法所需的平均重排次数。

6.4 Batcher-banyan 网络。Batcher-banyan 网络是由 Batcher 级（其中呼叫按其输出端口排序）后接 banyan（蝶形）级构成的无阻塞网络。在两级之间，对有序呼叫执行混洗——离开 Batcher 网络端口 $\{a_{n-1}, a_{n-2}, \ldots, a_0\}$ 的呼叫进入 banyan 网络的端口 $\{a_{n-2}, \ldots, a_0, a_{n-1}\}$。证明 2 元 n 飞（banyan 级）总能无阻塞地路由一组有序地址的混洗。

6.5 概率性无阻塞多播。在使用贪心算法时，估计在 VC2002（第 6.6 节）上路由一组完整的随机分布且扇出为四的呼叫时发生阻塞的概率。

6.6 一个七级时间-空间交换网络。在图 6.23 的 120 芯片网络中，我们最初将该网络视为一个五级 TSSST 交换网络，并忽略了 VC2002 的内部加速。现在将该网络视为一个七级 TSTSTST 交换网络，其中第一级和第三级上的两个 TSI 都被使用。这如何影响网络能够路由的流量模式？这如何影响你对网络的调度方式？

6.7 可重排性的证明。使用霍尔定理（如下）证明 $m \geq n$ 的 Clos 网络是可重排无阻塞的。

定理 6.6

（霍尔定理）设 A 为一个集合，$A_1, A_2, \ldots, A_r$ 为 A 的任意 r 个子集。存在 $A_1, \ldots, A_r$ 的一组互异代表元 $a_1, \ldots, a_r$，使得 $a_i \in A_i$ 且 $a_i \neq a_j \forall i \neq j$，当且仅当 $\forall k, 1 \leq k \leq r$，$A_1, \ldots, A_r$ 中任意 k 个集合的并集至少有 k 个元素。

6.8 使用欧拉划分的图着色。欧拉划分 [40] 的思想可用于改进循环算法以寻找二分图边着色的性能。欧拉划分首先通过寻找二分图中的欧拉回路开始——欧拉回路是图中一条从同一节点开始并结束且恰好访问每条边一次的路径。然后，沿着路径，将交替的边划分到两个新的二分图中，这两个图具有与原始图相同数量的节点。例如，对于访问节点 $\{L_1, R_6, L_3, R_4, \ldots\}$ 的路径，其中 R 和 L 表示二分图的左侧和右侧，边 (L_1, R_6) 被放入第一个新图，(R_6, L_3) 放入第二个，(L_3, R_4) 放入第一个，依此类推。注意两个新图的度数恰好是原始图的一半。然后这些新图再次被划分，如此继续，直到我们剩下度数为 1 的图，每个图被分配一种唯一的颜色。

6.8 练习题

（a）对于一个所有节点度数均为 2^i（i 为某个整数）的二分图，证明欧拉回路总是存在。描述一个简单的算法来找到这样的回路。

（b）对于具有 V 个节点、每个节点度数为 2^i（i 为某个整数）的图，找出基于欧拉划分的边着色算法的渐近运行时间。将其与循环算法的运行时间进行比较。

切片与分割

在本拓扑部分的最后一章中，我们简要探讨一些封装拓扑的实用技术。我们从集中器和分配器开始。集中器将多个终端节点的流量合并到单个网络通道上。当来自任何一个终端的流量太小而无法充分利用网络通道时，可以使用它们。它们在合并来自许多突发性终端的流量时也很有效。当峰值与平均流量的比率很大时，使用集中器可以降低串行化延迟并构建更具成本效益的网络。

分配器与集中器相反。它们从单个节点获取流量，并以逐包的方式将其分配到多个网络通道上。当来自节点的流量太大而无法由单个网络通道处理时，使用分配器。它们增加了串行化延迟并降低了负载均衡，但在某些情况下，对于连接否则无法服务的节点仍然有用。

有三种方法可以将网络节点切片到多个芯片或模块上：位切片、维度切片和通道切片。在位切片中，一个 w 位宽的节点被划分为 k 个 $\frac{w}{k}$ 位宽的切片，每个切片封装在单独的模块中。每个切片包含路由器数据路径的 $\frac{w}{k}$ 位宽部分。控制信息必须分发到所有切片，以便它们一致行动。这需要额外的引脚来分发控制信息，并且需要延迟来允许这种分发的时间。

在维度切片中，网络节点被切片，使得整个通道（例如与某个维度相关的通道）包含在每个切片上。使用这种方法，必须在切片之间添加额外的数据通道，以承载从一个切片进入但必须从另一个切片离开的流量。

最后，采用通道切片时，一个 w 位宽的节点被划分为 k 个独立节点，每个节点具有 $\frac{w}{k}$ 位宽的通道。然而，与位切片方法不同，这些子节点之间没有连接。它们构成完全独立的网络。通常，在每个终端处使用分配器将流量分配到这些网络切片上。

注释

- 1. 第 12 章包括分组交换和电路交换的详细描述。
- 2. 对于任何曾收到“所有电路忙”录音的人来说，美国电话系统显然是阻塞的。
- 3. 这对大多数但不是所有交叉开关实现都成立。有些具有有限的扇出。
- 4. 矩形交叉开关的 $m \times n$ 面积。
- 5. 任何奇数级数的 Clos 网络都可以通过将中间级的开关替换为三级 Clos 网络，从三级 Clos 网络递归推导出来。
- 6. 在某些情况下，会使用非对称 Clos 网络，其中输入级和输出级的 rr 和 nn 不同。非对称 Clos 网络由五元组 (m, n_i, r_i, n_o, r_o) 描述。
- 7. 这种证明技巧通常被称为鸽巢原理，在广泛的调度问题中很有用。
- 8. 选择这个排列和呼叫顺序是为了对可重排的 (3,3,4)(3,3,4) 网络特别棘手。大多数呼叫序列更容易调度。
- 为说明该算法生成的冲突链无环，可将图 6.13(c) 的冲突链展开为图 6.14。图中 a_i 和 b_i 分别表示第 i 次迭代所涉及的输入和输出交换机。算法先建立呼叫 (a_1, b_1)，然后将呼叫 (a_2, b_1)（若存在）从中间交换机 a 转移到 b。此后每次迭代 i，将呼叫 (a_i, b_i) 从 b 转移到 a，再将 (a_{i+1}, b_i)（若存在）从 a 转移到 b。
- 9. 尽管我们没有关于所需中间交换机数量的界限，但有几个例子表明 n 是不够的。
- 10. 严格来说，每级所需的扩展是 $\frac{2n-1}{n}$，略小于 2。
- 11. 注意，必须向空闲输出发送虚拟记录才能使其工作。
- 12. 单个 51.83-Gbits/s STS-1 可能包含 28 个 1.5-Mbits/s T1 或 DS1 通道，每个通道又承载 24 个 64-Kbits/s DS0 通道，每个通道承载一个电话呼叫。
- 13. 尽管人们认为互联网是用分组路由器实现的，但这些路由器仅存在于边缘，并且在这些端点之间通过 TDM 电路连接。
- 14. VC2002 还在每个输入和每个输出上包含一个 SONET 成帧器。输入成帧器对齐输入流，并监控和终止 SONET 传输开销。输出成帧器生成传输开销。
- 15. 为了保持时钟速率可控，96 时隙的 72×72 交换机被实现为两个 48 时隙交换机，每个以 STS-48 字节速率（311 MHz）运行。

第七章　集中与切片

7.1　集中器与分配器

7.1.1　集中器

在某些应用中，将来自多个（例如 M 个）网络终端的负载合并到单个通道中更为可取。这 M 个终端随后对网络呈现为单个大型终端。执行这种合并的设备称为集中器。如图 7.1 所示（其中 M = 4），集中器在终端侧接受 M 条双向通道，并将它们合并为网络侧的单个双向通道。除了减少端口数量外，集中器还可能减少总带宽。集中器的集中因子是终端侧带宽与网络侧带宽之比，由 $k_C = \frac{Mb_T}{b_N}$ 给出，其中 b_T 是终端通道带宽，b_N 是集中器网络侧通道的带宽。

图 7.2 展示了集中器如何应用于网络。一个 8 节点环可以直接服务 8 个终端，如图 7.2（a）所示。在终端和网络之间放置 2：1 集中器，如图 7.2（b）所示，允许相同的 8 个终端由 4 节点环处理。

集中器通常用于合并具有突发流量特性的多个终端。共享集中器网络侧的带宽可以平滑突发源的负载，并更有效地利用通道带宽。

例如，考虑一个 N=512 节点的多计算机，其中每个节点向网络呈现平均 100 Mbits/s 的负载。然而，当节点发生缓存未命中时，它在 128 纳秒期间（L = 128 位）向网络呈现 1 Gbit/s 的瞬时负载。为了防止串行化延迟增加网络的内存访问时间，我们必须为每个处理器提供 1-Gbit/s 的通道进入网络。如果不使用集中器，我们将构建一个具有 1-Gbit/s 通道的 8 元 3 立方体来处理此负载。该网络中路由器的总引脚带宽（成本的良好估计）为 3Tbits/s。如果我们改为根据最坏情况流量下的平均带宽来确定网络规模，我们将使用 200 Mbits/s 通道，并导致串行化延迟增加 5 倍，从 128 纳秒增加到 640 纳秒。

对于这个 512 节点网络，更有效的方法是将 8 个节点组成一组，使用 8：1 集中器馈送到一个具有 1-Gbit/s 通道的 64 节点 4 元 3 立方体网络。一条 2-Gbits/s 通道将每个集中器连接到该网络的一个节点，以防止此链路成为瓶颈。1 集中后的节点现在具有 800 Mbits/s 的平均带宽。尽管集中节点的峰值负载为 8 Gbits/s，但很少达到此峰值。2 只有在极少数时间有超过两个节点同时传输，并且在这小部分时间内的额外延迟被网络直径的减小和串行化延迟的降低所抵消。如果我们使用 1-Gbit/s 通道来确定网络规模，总引脚带宽为 384 Gbits/s（比未集中网络少 8 倍）。另一方面，如果我们根据平均带宽来确定网络规模，我们将使用 800-Mbits/s 通道，串行化延迟为 160 纳秒（比未集中网络少 4 倍）。

7.1 集中器与分配器集中器还用于促进网络的封装。例如，如果 4 个终端封装在一个

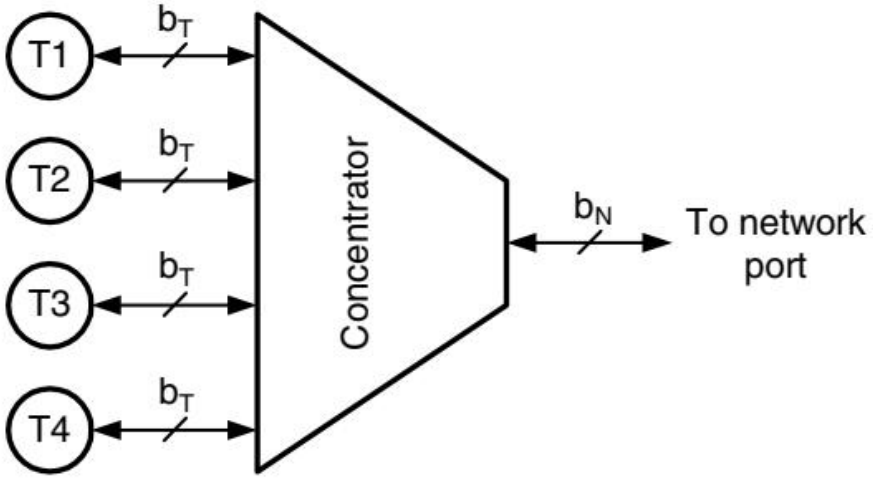


图 7.1 一个 4：1 集中器将来自 4 个终端节点 $T_1, \ldots, T_4$（每个带宽为 b_T）的流量合并到一条带宽为 b_N 的网络通道上。

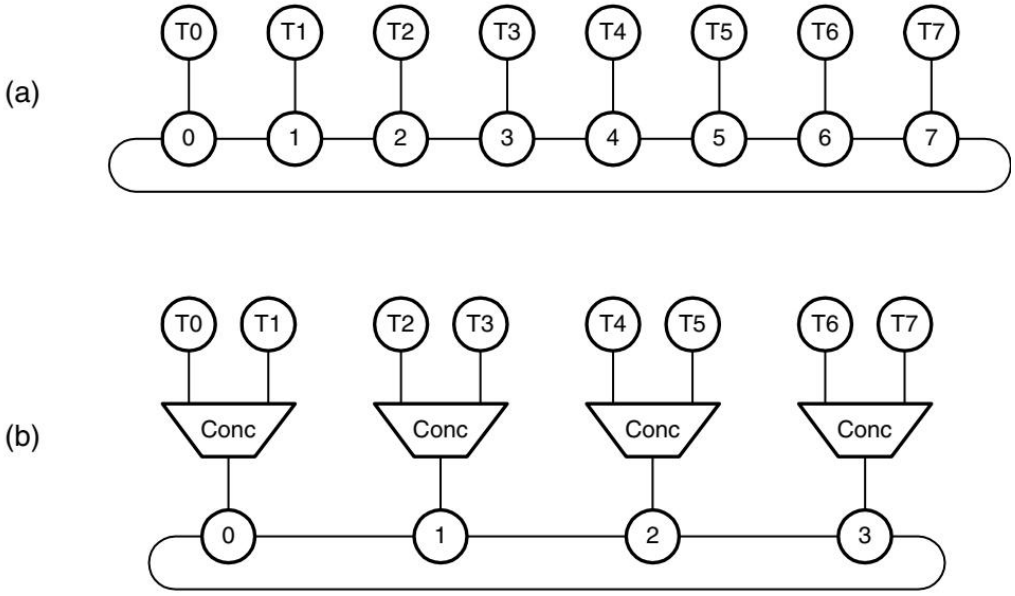


图 7.2 2：1 集中器的应用：（a）一个 8 元 1 立方体（环）网络连接 8 个终端。（b）相同的 8 个终端由前置四个 2：1 集中器的 4 元 1 立方体（环）网络连接。

模块（例如芯片）上，从封装的角度来看，通常方便使用集中器将来自 4 个终端的流量合并在一起，并将该模块视为单个网络终端。

7.1.2 分配器

分配器与集中器相反。如图 7.3 所示，分配器接受一条高带宽通道，并将其数据包分配到几条较低带宽的通道上。尽管乍看之下分配器似乎只是反向插入的集中器，但功能不同。反向通过集中器的每个数据包必须交付到特定终端，即其寻址的终端。另一方面，分配器可以任意将数据包分配到网络通道。分配可以是随机的、轮询的或负载均衡的。在某些情况下，为了保持顺序，同一类 3 的数据包总是分配到同一网络通道。分配器用于多种应用。它们可用于通过使用多个网络端口将更高带宽的模块（如更快的处理器或更高速率的线卡）与具有较低带宽的现有网络接口。在其他应用中，分配器用于提供容错。将负载分配到两条半速通道上，允许网络在一条通道故障时优雅地降级到一半性能。

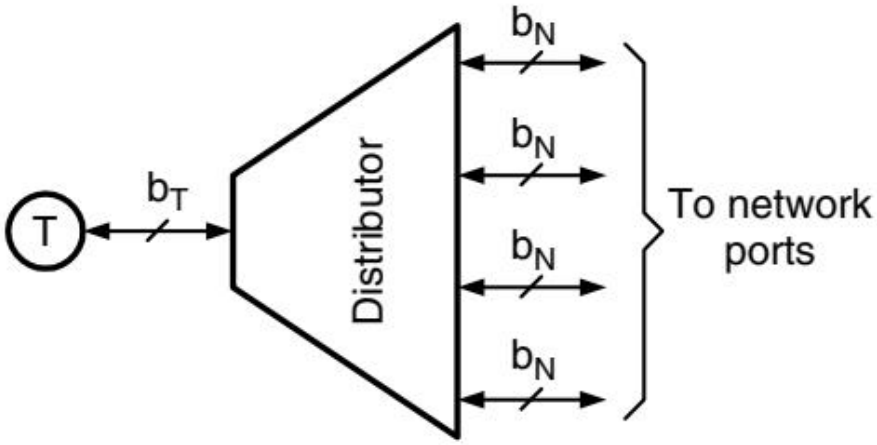


图 7.3 一个 1：4 分配器接受来自单个终端节点（带宽为 b_T）的数据包，并将数据包分配到 4 条网络通道（每条带宽为 b_N）。分配器可以均匀地将数据包分配到 4 条网络通道，或根据数据包长度或队列长度进行负载均衡。

当通道的带宽过高以致路由器难以轻松处理时，也会使用分配器。例如，如果时钟频率为 500 MHz（周期 2 ns），包大小为 8 字节，且路由器每 4 个周期最多处理 1 个包，则该路由器能处理的最大带宽为每 8 ns 8 字节，即 1 Gbyte/s。如果我们希望将 4-Gbyte/s 的端口连接到该网络，就需要使用 1：4 分配器将此流量分解为路由器能力范围内的较低带宽通道。在分解高带宽流量后，我们可以将较低带宽的通道插入一个网络的多个端口或并行网络中。（这称为通道切片——见第 7.2.3 节。）

在网络中添加分配器会从两方面对性能产生不利影响。首先，分配增加了串行化延迟。包在瓶颈链路上的串行化延迟为 $\frac{L}{b}$，其中 b 为最低带宽。因此，将包从带宽为 b_T 的终端链路分配到带宽为 b_N 的网络通道，会使串行化延迟增加 $\frac{b_T}{b_N}$。与串行化延迟相关的排队延迟也会成比例增加。其次，分配降低了负载均衡。分配器输出通道上的均衡永远无法完美。此外，如果分配器用于馈送并行网络，则在任何给定时刻，一个网络

的某条链路可能过载，而并行网络上对应的链路可能空闲。一般来说，从性能角度看，共享资源总是更好的。因此，分配会损害性能，但可能有助于可实施性或容错性。

7.2 切片与分割

有时，一个网络节点无法完全放在单个模块（芯片或板卡）上。这通常是由于引脚限制，但面积也可能是一个考虑因素（尤其是对于存储器）。在这种情况下，我们需要将路由器划分到多个芯片上，这一过程称为切片。路由器切片有三种方法：位切片、维度切片和通道切片。

7.2.1 位切片

位切片如图 7.4 所示，是将网络节点划分到多个芯片上的最直接方法。如果每个通道宽 w 位，我们希望将节点划分到 m 个芯片上，只需将每个通道的 $\frac{w}{m}$ 位放在每个芯片上。4 例如，图 7.4 显示了一个 8 位宽的二维节点被切成两个 4 位宽的模块。8 个通道中的每一个都被分成一对 4 位通道。若干控制线（ctl）用于在两个路由器位切片之间传递信息。

7.2 切片与分割

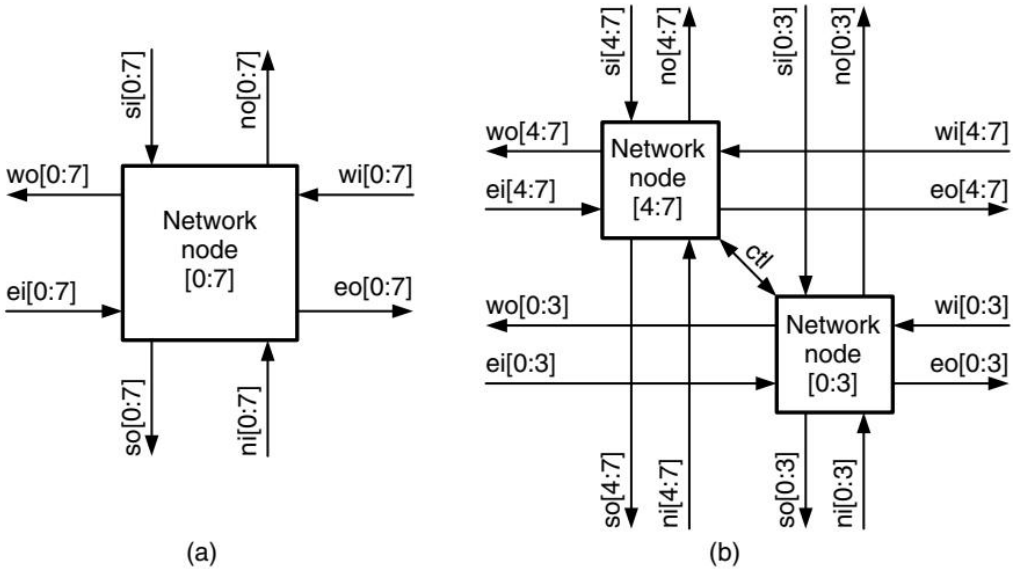


图 7.4 位切片：（a）用于二维环面网络的网络节点，通道宽度 w = 8 位。每个方向（n-北，s-南，w-西，e-东）都有一个 8 位输入通道（例如，ni [0：7] 是北输入）和一个 8 位输出通道（例如，wo [0：7] 是西输出）。（b）同一节点被切成 2 个封装，每个具有 w = 4 位宽的通道

位切片的难点在于控制分配和故障恢复。一个 flit 的一半位到达网络节点 $[4:7]$，另一半到达网络节点 $[0:3]$；然而整个 flit 必须作为一个单元进行交换。要将 flit 作为一个单元交换，两个切片必须具有相同且完整的控制信息。遇到的第一个问题是头部信息的分配。一种方法是将头部的所有相关位（指定路由、目的地、虚拟通道等）分配到两个切片。使用这种方法，头部的所有相关位必须通过两个芯片之间的控制通道。这

种分配的成本是引脚开销和延迟。引脚开销恰好是头部位与总 flit 位的比率。对于小 flit 尺寸，这种开销可能相当大，达到 25% 或更多。延迟开销也很显著。在典型路由器中，将头部从一个芯片传输到另一个芯片很容易使包延迟增加两个时钟周期。

通过在一个位切片上执行所有控制并将控制决策（而非头部信息）分配到其他切片，可以在一定程度上减少引脚带宽开销。然而，这种方法并不能减少延迟损失。芯片穿越仍然需要几个时钟周期。

位切片的另一个问题是错误检测和恢复。许多路由器通过计算一个函数（例如对整个 flit 所有位的循环冗余校验（CRC））来执行错误检测。在位切片路由器中，这种校验必须跨切片进行分区，并在切片之间交换中间结果。

如果跨位切片复制的控制状态发生分歧（例如由于瞬态故障），也会出现问题。一个健壮的切片路由器必须不断检查以确保该状态保持同步，并且必须能够在状态分歧时恢复。

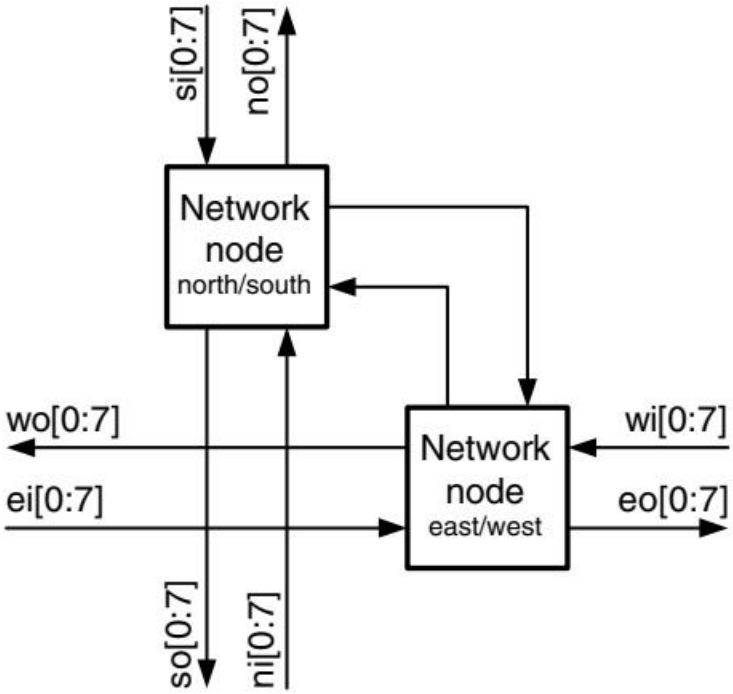


图 7.5 维度切片：一个二维节点被划分为 2 个一维节点，一个承载南北流量，一个承载东西流量。切片间通道处理在这些方向之间交叉的流量。

尽管控制分配和错误处理复杂，位切片在具有大 flit 的路由器中工作良好，因为这种开销可以分摊。位切片与允许控制流水线到数据之前的流控制方法（如 flit 预留流控制（第 13.4 节））结合时特别有吸引力。

7.2.2 维度切片

如果以将整个 flit 保持在单个切片中的方式对路由器进行切片，则可以避免位切片的控制和错误处理复杂性。维度切片通过在其端口 5 上划分路由器，同时保持每个端口在单个切片上完整，来实现这一目标。实际上，每个度为 d 的网络节点被划分为一个

由 m 个度为 $\frac{d}{m}+p$ 的节点组成的子网络，其中每个节点需要额外的 p 个端口用于切片间通信。

例如，图 7.5 展示了图 7.4（a）中宽度为 w=8 位的二维路由器被切分为两个一维路由器：一个处理南北向流量，另一个处理东西向流量。如果需要进一步划分，该路由器还可以被划分为四个独立的一维单向路由器，每个方向一个。

两个一维路由器之间的通道必须具有足够的带宽，以处理所有从南北向切换到东西向（或反之）的流量。需要切换方向的流量大小很大程度上取决于路由算法（第 8 章）。像维度顺序路由这样倾向于直线传输而非方向切换的算法，所需的跨路由器带宽远小于允许任意方向切换的算法。

7.2.3 通道切片

前面两种节点划分方法都需要分区之间的通信。位切片需要交换控制信息，维度切片需要分区之间的数据通路。通道切片通过将整个网络拆分为两个完全独立的网络，消除了分区之间的所有通信。并行网络之间唯一的通信发生在使用分配器的终端处。

图 7.6 展示了通道切片。图 7.4（a）中宽度为 w=8 位的网络被替换为两个完全独立的 4 位宽网络。两个 4 位宽子网络之间没有连接，除了在终端链路上使用分配器将流量分配到两个网络进行传输，并在目的地重新组合流量。

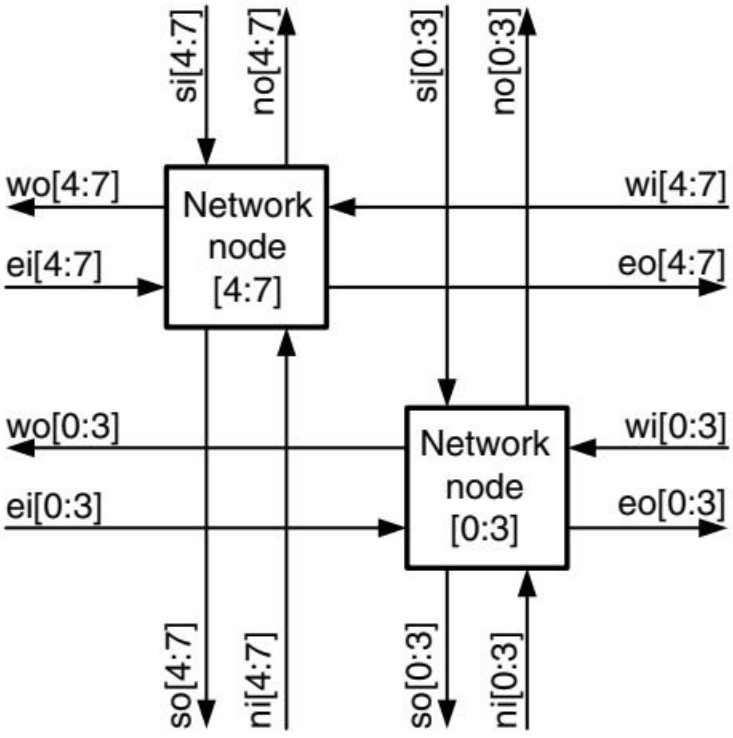


图 7.6 通道切片：一个宽度为 w=8 位的二维网络节点被划分为两个完全独立的宽度为 w=4 位的节点。两个 4 位宽节点之间没有连接，除了在终端链路上（未显示）。

7.3 多级网络的切片

多级网络的切片可用于在串行化延迟 T_s（取决于通道宽度）和头部延迟 T_h（取决于直径）之间进行权衡，从而通过平衡这两个分量来优化网络延迟。该技术类似于第 5.2.2 节中讨论的通过选择网络维度来平衡 T_s 和 T_h 以最小化环面延迟的方法。对多级网络进行通道切片还可以通过减少路由组件数量和引脚总数来降低总成本。

一个具有 w 位宽通道的 $N = k^n$ 节点 k 元 n 飞网络可以通过通道切片划分为 x 个具有 w/x 位宽通道的 k 元 n' 飞网络。例如，图 7.7 显示，通过将一个具有 2 位宽通道的二元 2 飞网络替换为两个具有串行（1 位宽）通道的 4 元 1 飞网络，可以降低成本和直径。这两个网络具有相同的吞吐量，并且需要引脚数相同（八个信号）的交换机。然而，通道切片后的 4 元飞网络比二元飞网络具有更小的直径，并且所需的交换机组件数量减半。

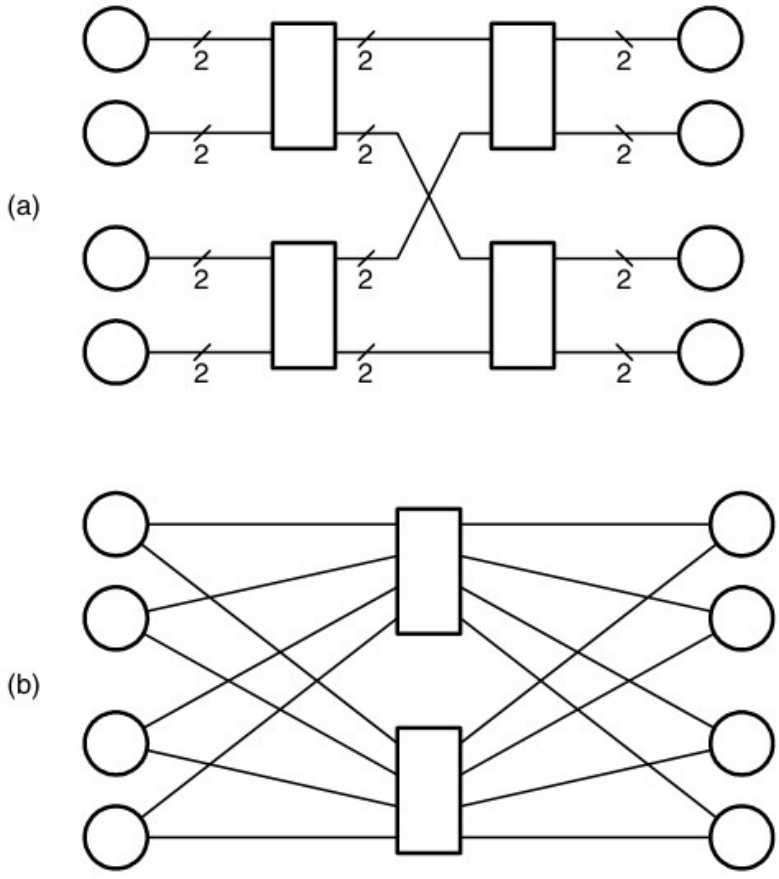


图 7.7　通过通道切片构建高基数交换机

对多级 k 元 n 飞网络进行通道切片可将网络直径从 $n + 1$ 减小到 $n' + 1$，其中

$$n' = \log_{xk} N = \log_{xk} k^n = \frac{n}{1 + \log_k x}.$$

这种切片的结果是，T_s 增加 x 倍，T_h 减少 $\frac{1}{1+\log_k x}$ 倍。使用 n' 的表达式并将导线延迟归入 t_r，得到

$$T_s = \frac{xL}{b},$$

$$T_h = t_r \left(\frac{n}{1 + \log_k x} \right).$$

例如，考虑一个 N = 4,096 节点的二元 12 飞网络，具有 w = 32 位宽通道，b=1 Gbit/s，$t_r = 20$ ns，消息长度 L=256 位。表 7.1 显示了当切片因子 x 从 1（无切片，32 位宽通道）以 2 的幂次增加到 32（网络切片为串行通道）时，直径 n'、通道宽度 w 以及延迟的两个分量 T_s 和 T_h 如何变化。延迟数据绘制在图 7.8 中。无切片时，二元立方体的延迟 T 主要由 T_h 决定——通过 $n' = 12$ 级、每级 20 ns，共 240 ns。当我们将网络切片为两个 16 位通道的网络（x = 2）时，现在可以用相同的引脚数实现基数为 4 的交换机，因此可以使用两个 $n' = 6$ 级的网络。这将 T_h 减半至 120 ns，同时将 T_s 加倍至 16 ns——这是一个很好的权衡。切片为 4 个 8 位通道的网络（x = 4）得到最小延迟 T = 112 ns，T_h 进一步降至 80 ns（$n' = 4$ 级）。超过这一点后，T_s 的增量增加（每次 x 加倍时 T_s 也加倍）大于 T_h 的增量减少（T_h 随 x 的倒数对数减少），总延迟增加。极端情况 x = 32 个并行位串行 $n' = 2$ 级网络，基数 k = 64 的交换机，在所有配置中延迟最大，主要由 $T_s = 256$ 决定。尽管这种配置延迟最高，但引脚成本最低，因为其直径 n' 最小。

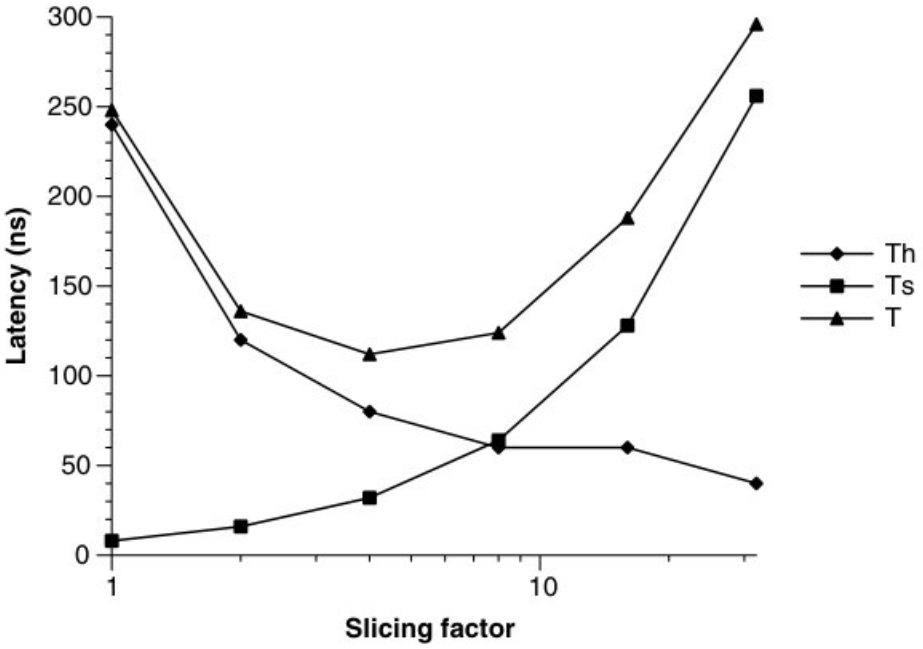
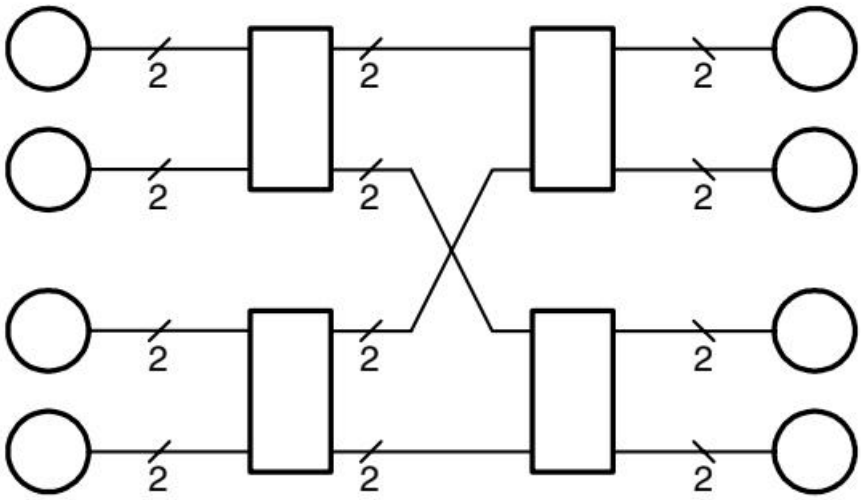
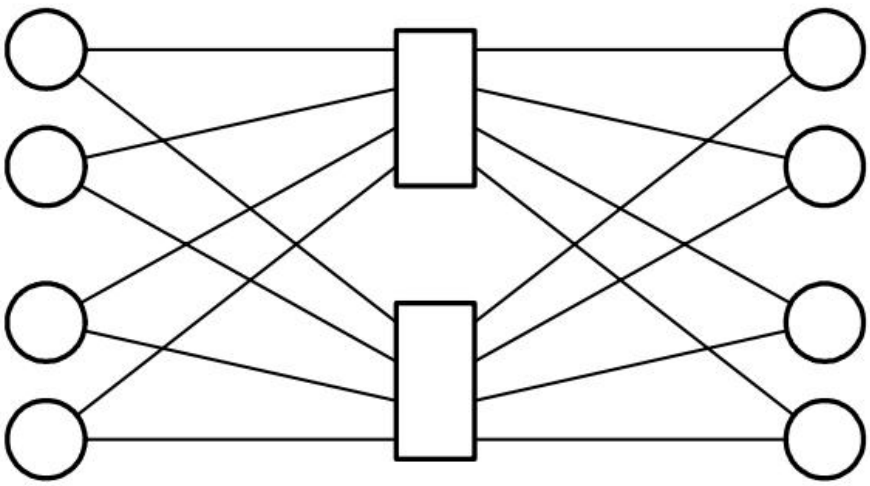


图 7.8 切片前后网络的总延迟、序列化延迟与首部延迟

图 7.7 对多级网络进行通道切片允许以相同的引脚数实现更高基数的交换机。这以更高的串行化延迟为代价降低了网络直径和成本。(a) 具有 w=2 通道的二元 2 飞网络直径为 3，使用具有 8 个信号的交换机。(b) 两个并行的 4 元 1 飞网络具有串行 $w = 1$ 通道，总带宽相同，交换机信号数相同，但直径为 2，且交换机数量是二元 2 飞网络的一半。



(a)



(b)

表 7.1 通道切片降低蝶形网络延迟

x	k	n’	w	T_s	T_h	T
1	2	12	32	8 ns	240 ns	248 ns
2	4	6	16	16 ns	120 ns	136 ns
4	8	4	8	32 ns	80 ns	112 ns
8	16	3	4	64 ns	60 ns	124 ns
16	32	3	2	128 ns	60 ns	188 ns
32	64	2	1	256 ns	40 ns	296 ns

我们已经在蝶形网络的背景下描述了通道切片的使用，但它也可以应用于其他多级网络，包括 Clos 网络和 Batcher 网络。我们在练习 7.5 中探讨 Clos 网络的切片。此外，虽然我们描述了使用通道切片的多级网络切片，我们也可以使用位切片来切片这些网络。我们在练习 7.6 中探讨这种替代方案。

7.4 案例研究：Tiny Tera 中的位切片

Tiny Tera 是一种快速分组交换机，最初在斯坦福大学设计，后来由 Abrizio 商业化 [125]。6 正如其名称所示，它提供 1 Tbit/s 的总带宽。7 Tiny Tera 交换机的高层架构如图 7.9 所示。如图所示，Tiny Tera 围绕一个 32×32、8 位宽的交叉开关组织。端口卡在交叉开关和外部网络的物理接口之间提供输入和输出缓冲。每个端口卡与一个中央调度器单元通信，该单元负责计算交叉开关的配置。

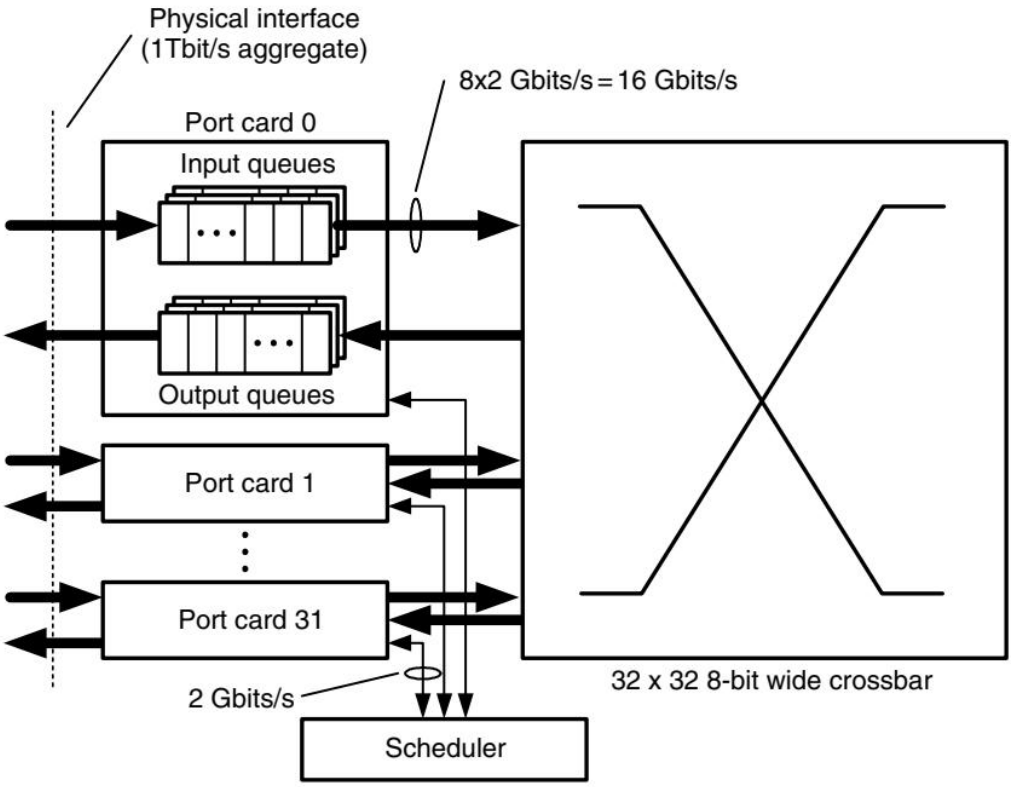


图 7.9 Tiny Tera 架构使用 32 个端口卡，每个端口卡具有 32 Gbits/s 的带宽，馈入一个 32×32 的 8 位宽交叉开关，实现 1 Tbit/s 的总带宽。一个集中式调度器与端口卡通信以计算交叉开关调度，这些调度被中继回端口卡。

配置被中继回端口卡，并在分组传递到交叉开关时附加到分组上。

通过考虑交叉开关所需的引脚数量，构建交叉开关的挑战变得显而易见。首先，端口卡和交叉开关之间的信号是差分信号，工作速率为 2 Gbits/s。交换机的每个端口发送和接收 16 Gbits/s 的数据，因此这需要总共 16 个信号，即每个端口 32 个引脚。由于交叉开关有 32 个端口，它总共需要 1,024 个高速引脚。虽然具有此引脚数量的封装是可用的，但我们尚未考虑电源和接地引脚，或每个高速链路消耗的功率量。因此，基于其苛刻的要求，设计者选择对 Tiny Tera 交叉开关进行位切片。

图 7.10 显示了交换机设计者选择的位切片方法。交叉开关被切片成八个 32×32 的 1 位宽交叉开关。切片将引脚数减少到每芯片 128 个高速引脚，更易于管理，并且功耗也分散在这 8 个芯片上。虽然这里未显示，但切片贯穿端口卡本身，其中输入和输出队列被位切片并分布在几个 SRAM 上。

切片设计不仅缓解了 Tiny Tera 中的许多封装问题，还提高了灵活性。例如，可以添加额外的交叉开关切片以为交换机提供部分加速或提高可靠性。如果采用单芯片交叉开关，任何加速或冗余的增加都需要另一个完整的交叉开关。

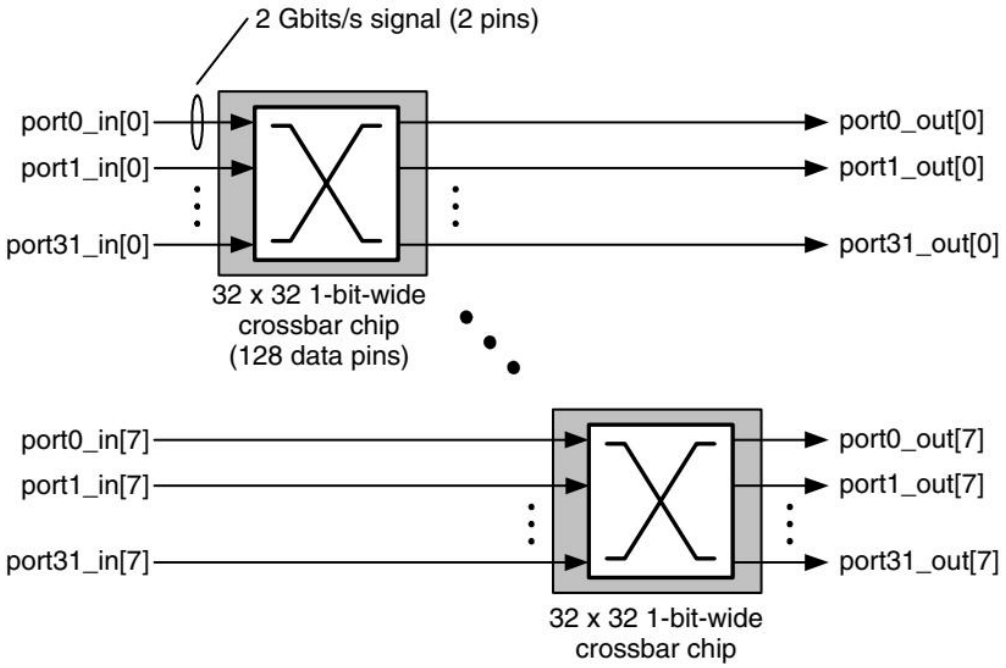


图 7.10 Tiny Tera 交叉开关的位切片实现。8 位宽接口被切成 8 个切片，每个交叉开关芯片仅需 128 个引脚。

7.5 文献注释

切片、集中和分布有着悠久的历史。集中长期以来一直用于电话网络中，以解决大多数电话大部分时间处于挂机状态的事实。J-Machine 路由器 [138]（第 5.5 节）采用维度切片，所有三个切片在一个芯片上。Cray T3D [95] 使用了类似的组织，每个切片在一个单独的 ECL 门阵列上。位切片交叉开关被用于 MARS 加速器 [3] 的网络以及 Tiny Tera（第 7.4 节）[125]。Avici TSR [49] 中使用了具有六个 4 位切片的位切片 3-D 环面。

7.6 练习

7.1 环面中的集中。你需要连接 4,096 个节点，其峰值带宽为 10 Gbits/s，平均带宽为 500 Mbits/s。将你自己限制在二分带宽固定为 2.56Tbits/s 的环面网络，比较替代网络在集中器、维度和基数方面的差异。假设每个集中器和路由器节点封装在单独的芯片中，每个芯片的总引脚带宽可达 100 Gbits/s。哪种拓扑提供最低的引脚带宽？哪种在不增加额外串行化延迟的情况下提供最低的引脚带宽？

7.2 从线卡分发流量。使用分配器，建议如何连接 64 个 40-Gbits/s 的线卡，使用由不超过 10 Gbits/s 的通道组成的环面网络。底层网络应支持所有节点跨二分发送的最坏

情况流量。与通道可以以 40 Gbits/s 运行的情况相比，这种安排是否改变了二分带宽?
7.3 切片蝶形网络。考虑将具有 w = 2 位宽通道的基数 4 蝶形节点使用位切片、维度（端口）切片和通道切片划分为两个模块。微片为 64 位，其中 16 位是头部信息。画出每种划分的草图，标记所有通道宽度和每芯片所需的信号数量，并定性比较每种情况下的延迟。
7.4 子信号切片。可以选择通道切片因子，使得产生的通道小于一个信号宽。为了实现这些通道，可以将几个通道复用到单个物理信号上。例如，如果通道被切片到半个信号，两个通道将共享一个物理信号。这种级别的切片能否减少零负载延迟? 解释为什么能或不能。
7.5 Clos 网络的通道切片。对于一个 N=256 的可重构（m=n）Clos 网络，由具有 32 个信号的交换机组成，求使延迟最小的通道切片因子 x。每个节点需要 8 Gbits/s 的带宽，L=128 位，$t_r = 20$ ns，f=1 Gbit/s。
7.6 4,096 节点蝶形网络的位切片。考虑一个具有表 7.1 第一行网络参数的网路。求使零负载延迟最小的该网络的位切片。假设每个位切片中重复一个 32 位头部。（切片之间不需要控制信号。）此外，在经历任何路由器延迟 t_r 之前，必须接收到完整的头部。
7.7 Tiny Tera 中的头部分发。在位切片的 Tiny Tera 交换机中，交叉开关配置由集中式调度器计算，然后重新分发到输入端口。然后，这个头部信息被复制并附加到每个输出分组切片上。然而，对于端口 A，离开 A 的分组不包含其目的端口，而是包含写入 A 的端口的地址（也就是说，配置不是通过将分组写入何处来描述，而是通过从何处接收分组来描述）。为什么这会导致更高效的编码? 提示：Tiny Tera 支持多播流量。

注释

- 1. 如果引脚允许，我们可以将集中器与节点路由器结合，并完全消除此通道。
- 2. 如果我们假设 8 个节点的访问模式是独立分布的泊松过程，则所有 8 个节点同时传输的概率为 10^{-8}。
- 3. 例如，IP 路由器中来自同一流的包，或多处理器中访问同一缓存块的包。
- 4. 当然，这假设 w 能被 m 整除。
- 5. 更准确地说，这种方法应称为端口切片，因为它是按端口进行划分的。该方法最初应用于立方体和网格网络时是按维度划分的——因此得名维度切片。
- 6. 我们对 Tiny Tera 细节的讨论基于斯坦福大学的学术设计。
- 7. 分组交换机的带宽通常被宣传为输入和输出带宽之和（总带宽）。更准确地说，该交换机具有 500 Gbits/s 的吞吐量。

第八章　路由基础

路由涉及在特定拓扑中选择从源节点到目的节点的路径。一旦我们有了拓扑，或者说网络的路线图，路由就是下一个逻辑步骤：在地图上选择一条能带我们到达目的地的路线。拓扑决定了网络的理想性能，而路由是决定这种潜力能实现多少的两个关键因素之一。我们将在第 12 章和第 13 章讨论另一个关键因素：流控。

用于网络的路由算法至关重要，原因有几个。一个好的路由算法即使在非均匀流量模式（如置换流量）下也能平衡网络通道上的负载。通道负载越平衡，网络的吞吐量就越接近理想值。令人惊讶的是，许多已经构建并正在使用的路由器在平衡负载方面做得很差。相反，每对节点之间的流量遵循单一的、预定的路径。正如您所料，非均匀流量模式可能会在这种类型的路由算法下引起大的负载不平衡，导致次优的吞吐量。然而，这些路由选择至少可以部分解释，因为这些路由器中的大多数都是为了优化任何路由算法的第二个重要方面而设计的：短路径长度。

一个设计良好的路由算法还尽可能保持路径长度短，减少跳数和消息的整体延迟。可能不会立即明显的是，通常，最小路由（总是选择最短路径）与平衡负载和最大化吞吐量是矛盾的。事实上，对于 oblivious 路由算法，为了改善所有流量模式下的负载平衡，我们被迫增加所有消息的平均路径长度。反之亦然。这种权衡存在于 oblivious 算法中，因为它们没有将当前流量模式纳入路由算法。我们将在第 9 章更详细地探讨这些算法。

另一方面,一个聪明的设计者可能会提出一种提供“两全其美”的方法。与其像 oblivious 算法那样选择独立于流量模式的算法，为什么不适应于当前的流量条件呢？然后，对于“容易”的流量模式（如均匀流量），我们可以最小化地发送流量，但对于“困难”的非均匀流量模式，我们可以采用非最小路由。这个简单的想法构成了自适应路由算法的基础，我们将在第 10 章探讨。这些算法的潜在优势是实现负载平衡和局部性（短路径长度）。然而，我们将看到，实际设计问题使实现这一目标具有挑战性。

路由算法的另一个重要方面是它在网络中存在故障时工作的能力。如果一个特定的算法被硬连线到路由器中，并且链路或节点发生故障，整个系统就会失败。然而，如果算法可以重新编程或适应故障，系统可以在仅略微损失性能的情况下继续运行。显然，这对于高可靠性要求的系统至关重要。最后，路由与网络的流控相互作用，通常需要仔细设计两者以避免死锁和/或活锁（第 14 章）。

我们的路由讨论从下面的一个简短示例开始，然后讨论路由分类和确定性路由算法的介绍。我们在第 9 章继续讨论确定性和 oblivious 路由，然后在第 10 章讨论自适应路由。最后在第 11 章讨论路由机制。

8.1 一个路由示例

考虑图 8.1 所示的 8 节点环形网络上的路由问题。如果我们排除回溯，或重新访问网络中的节点，这里的路由决策是二元的。对于从 s 发送到 d 的每个分组，我们可以选择从 s 开始到 d 结束的顺时针或逆时针方向发送分组。即使使用这种简单的拓扑和只有二元决策，也有许多可能的路由算法。以下是一些：

贪婪：始终沿环上最短方向发送分组。例如，始终从 0 到 3 沿顺时针方向路由，从 0 到 5 沿逆时针方向路由。如果两个方向的距离相同，则随机选择一个方向。

均匀随机：为每个分组随机选择一个方向，选择任一方向的概率相等。

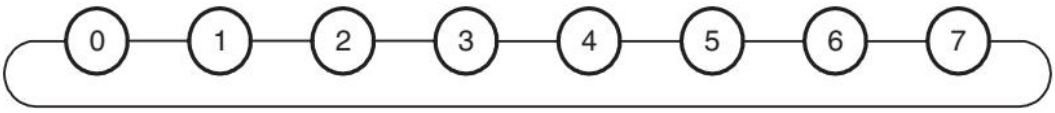


图 8.1 一个 8 节点环形网络。

加权随机：为每个分组随机选择一个方向，但以概率 $1 - \Delta/8$ 加权短方向，以 $\Delta/8$ 加权长方向，其中 Δ 是源和目的地之间的（最小）距离。

自适应：将数据包发送到本地通道负载最低的方向。我们可以通过测量服务该通道的队列长度或记录过去 T 个时隙内它传输了多少数据包来近似负载。注意，由于我们禁止回溯，此决策仅在源节点应用一次。

哪种算法能提供最佳的最坏情况吞吐量？绝大多数人选择贪婪算法。1 也许这说明了人性的某些方面。然而，事实证明，贪婪算法在这种拓扑上并不能提供最佳的最坏情况吞吐量。

为了了解贪婪路由如何给我们带来麻烦，考虑一种龙卷风流量模式，其中每个节点 i 向 $i + 3 \mod 8$ 发送一个数据包，如图 8.2 所示。上述 4 种路由算法在 8 节点环上的龙卷风流量下的性能总结在表 8.1 中。使用贪婪路由算法，所有

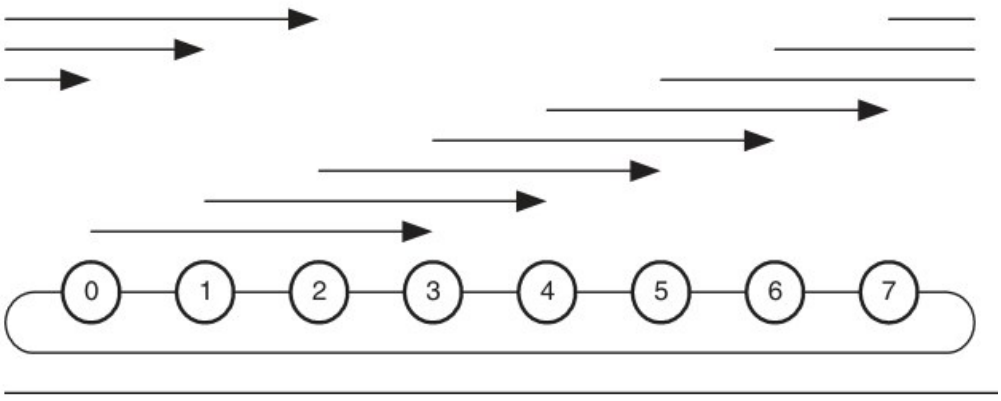


图 8.2 八节点环网络中的龙卷风流量模式

图 8.2 8 节点环网络上的龙卷风流量。使用贪婪路由时，所有流量沿环顺时针方向移动，使逆时针通道空闲。

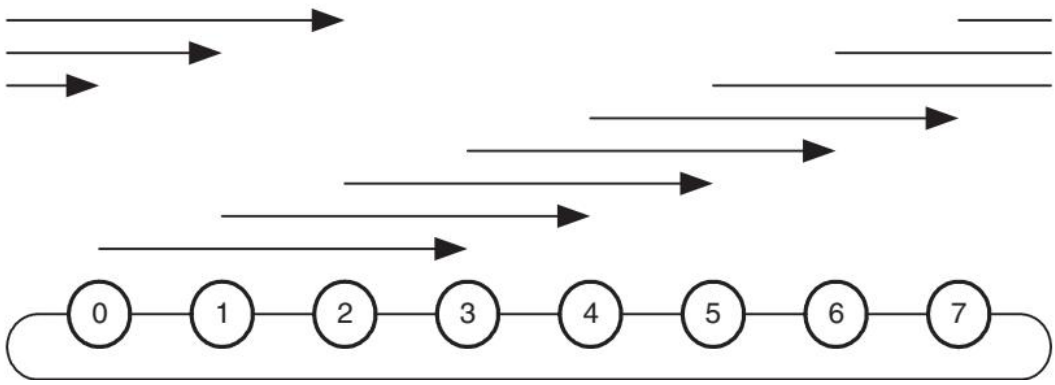


表 8.1 环网络不同路由算法的吞吐量

Algorithm	吞吐量 on Tornado
Greedy	0.33
Random	0.40
Weighted random	0.53
Adaptive	0.53

8.1 一个路由示例流量沿环顺时针方向路由，使所有逆时针通道空闲，并给顺时针通道加载 3 个单位的流量——即 $\gamma = 3$——这使每个终端的吞吐量为 $\Theta = b/3$。使用随机路由时，逆时针链路成为负载为 $\gamma = 5/2$ 的瓶颈，因为一半的流量沿逆时针方向经过 5 条链路。这给出的吞吐量为 2b/5。对随机决策加权后，5/8 的流量经过 3 条链路，3/8 的流量经过 5 条链路，两个方向的负载均为 $\gamma = 15/8$，从而给出 8b/15 的吞吐量。自适应路由，在关于如何实现自适应性的某些假设下，将在稳态下匹配这种完美的负载平衡，给出与加权随机路由相同的吞吐量。

这个例子展示了路由函数的选择如何显著影响负载平衡。然而，最坏情况吞吐量只是设计者可能希望优化的几个指标之一。正如人们所料，不同的指标可能导致关于这四种算法中哪一种最合适的不同的结论。我们在练习 8.1 中探讨其中一些。

8.2 路由算法的分类

我们根据路由算法如何在从源节点 x 到目的节点 y 的可能路径集合 R_{xy} 中进行选择来对它们进行分类。

确定性路由算法总是选择 x 和 y 之间的相同路径，即使存在多条可能路径（$|R_{xy}| > 1$）。这些算法忽略底层拓扑的路径多样性，因此在平衡负载方面表现很差。尽管如此，它们在实践中非常常见，因为它们易于实现且易于实现无死锁。

oblivious 算法（包括确定性算法作为子集）在不考虑网络当前状态的任何信息的情况下选择路由。例如，在 R_{xy} 中的所有路径上均匀分配流量的随机算法就是一种 oblivious 算法。

自适应算法根据网络状态进行调整，在做出路由决策时使用这种状态信息。此信息可

能包括节点或链路的状态（开启或关闭）、网络资源的队列长度以及历史通道负载信息。

上一节的龙卷风示例包含了所有这三种路由类型的示例。环上的贪婪算法是确定性路由的一个例子。s 和 d 之间的所有数据包沿环的相同方向传输。均匀和加权随机路由方案是 oblivious 路由的例子。它们在环周围的方向之间进行选择，而不考虑网络状态。最后，自适应算法根据初始跳的通道负载做出决策。

在这些定义中，我们描述了 R_{xy} 中的路由集合上的每种路由算法——从源到目的的最小或最短路径路由。

因此这些算法被称为最小路由。正如我们已经看到的，包含非最小路由通常很重要，在这种情况下，路由函数从所有最小和非最小路由的集合 R'_{xy} 中选择路径。这些算法被称为非最小路由。再次，从我们环上的简单例子来看，贪婪算法是最小路由，而随机和自适应算法是非最小路由。

8.3 路由关系

将路由算法表示为路由关系 R 和选择函数 ρ 是有用的。R 返回一组路径（或用于增量路由算法的通道），ρ 在这些路径（或通道）之间进行选择以选择要采用的路由。通过这种算法划分，与通道依赖和死锁相关的问题涉及关系 R，而与自适应性相关的问题涉及选择函数 ρ。我们在第 14 章中详细讨论死锁。

根据我们的算法是增量的，以及它是基于节点还是基于通道，我们以三种不同方式定义 R：

$$R : N \times N \mapsto \mathcal{P}(P) \tag{8.1}$$

$$R : N \times N \mapsto \mathcal{P}(C) \tag{8.2}$$

$$R : C \times N \mapsto \mathcal{P}(C) \tag{8.3}$$

其中 $\mathcal{P}(X)$ 表示集合 X 的幂集，或所有子集的集合。这种表示法允许我们反映路由关系可能返回多个路径或通道的事实，其中一个由选择函数选择。

当路由关系的输出是完整路径时，如关系 8.1——我们的三个路由关系中的第一个，路由算法被称为一次性路由。这个名称准确反映了路由算法的使用方式。当数据包在源节点 x 注入网络并 destined to 节点 y 时，评估路由关系：$U = R(x, y)$。由于 U 可能是一组路由，选择其中一个并分配给数据包。当然，U 不必包括所有可能的路由 R'_{xy} 甚至所有最小路由 R_{xy}，在确定性路由算法的情况下，它只返回一个（$|U| = 1$）。一旦选择了路由，它就会与数据包一起存储。正如我们将在第 11 章中看到的，一次性路由最小化了每个数据包评估路由关系所花费的时间，但这种优势伴随着在数据包内携带路由的开销。

另一种方法是增量路由，其中关系返回一组可能的通道。增量路由不是一次性返回整条路径，而是每经过一跳就对路由关系求值一次。关系的输出用于选择数据包接下来要走的通道。例如，在关系 8.2（路由关系的第二种形式）中，关系的输入是数据包的当前节点 w 及其目的地 y。对关系求值会得到一组通道 D = R（w，y），其中 D 的每个元素都是从 w 出发的输出通道，即 D ⊆ C_{Ow}。然后使用选择函数从 D 中选出数据包接下来要使用的通道。这个增量过程会重复进行，直到数据包到达最终目的地。关系 8.3 是第三种关系，它也是增量的，使用方式类似。唯一的区别在于，函数的输入是数据包先前使用的通道及其目的地。

与一次性路由相比，增量路由不需要随数据包携带路由信息，因此没有相关开销，但路由关系可能需要多次求值，这可能会增加数据包的延迟。另一个重要点是，增量算法无法实现一次性路由所能实现的所有路由策略。这是因为我们利用数据包的历史信息（如果有的话）很少或根本没有用来计算下一跳。例如，使用一次性算法，我们可以为二维网格设计一种路由算法，使得数据包只能通过特定节点进行垂直或水平路由。（没有数据包在该节点从水平维度转向垂直维度。）然而，对于第二种路由关系（关系 8.2）来说，这是不可能的，因为无法区分数据包是从垂直通道到达的还是从水平通道到达的。当然，第三种关系可以缓解这个问题，但它仍然无法涵盖许多一次性算法。（见练习 8.2。）

路由关系的第三种形式（关系 8.3）也是增量的，但它基于数据包的当前通道 c 而不是当前节点 w 来做出路由决策。在这种情况下，路由关系接收当前通道 c 和目的节点 y，并返回一组通道 D = R（c，y）。将决策基于数据包到达节点 w 时所经过的通道 c 而不是 w 本身，提供了恰好足够的历史信息来解耦通道之间的依赖关系，这对于避免死锁很重要（第 14 章）。

无论我们使用哪种形式的 R，除非路由是确定性的，否则它都会返回一组可能的路径或通道。选择函数 ρ 用于从该集合中选择将要使用的元素。如果 ρ 在选择时不使用任何网络状态信息，则路由是无意识的。另一方面，如果 ρ 基于输出通道的可用性进行选择，则路由是自适应的。

8.4 确定性路由

最简单的路由算法是确定性的——它们将每个数据包从源 x 到目的地 y 都沿完全相同的路由发送。例如，确定性路由算法的路由关系是一个函数 R：$N \times N \mapsto P$。正如我们在第 8.1 节中所看到的，这种缺乏路径多样性的特点可能导致网络中严重的负载不均衡。事实上，对于每一种确定性路由算法，都存在一种流量模式会导致严重的负载不均衡。因此，对于关注最坏情况的设计者来说，这些算法不会是首选。然而，确定性算法仍然有其优点。

许多早期网络采用确定性路由，因为它实现起来非常简单且成本低廉。可能令人惊讶的是，确定性路由至今仍出现在网络中。对于不规则拓扑尤其如此，因为设计良好的随机或自适应算法更加困难。对于几乎[2]任何拓扑，选择最小确定性路由函数都是有

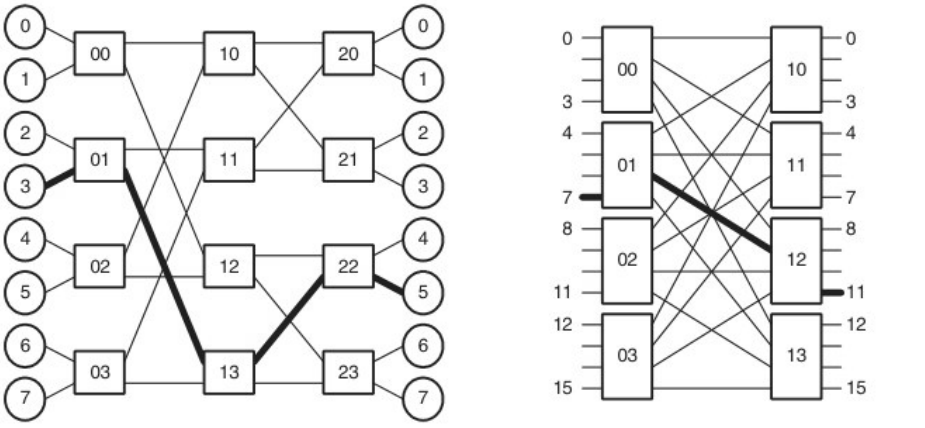
意义的。因此，至少路径长度会很短。对于某些拓扑，简单的确定性方法实际上可以像任何其他最小路由算法（包括自适应算法）一样实现负载均衡（练习 9.2）。最后，对于特定源-目的对之间消息顺序很重要的网络，确定性路由通常是提供这种顺序的一种简单方法。例如，这对于某些缓存一致性协议很重要。

在本节中，我们描述两种最流行的确定性路由算法：蝶形网络中的目的地标签路由，以及环面和网格中的维序路由。

8.4.1 蝶形网络中的目的地标签路由

在 k 元 n 飞网络中（见第 4.1 节），目的地址被解释为一个 n 位的 k 进制数，直接用于路由数据包。地址的每一位依次用于在路由的每一步选择输出端口，就好像地址本身就是根据源路由表确定的路由头。这就是我们在第 2 章的简单路由器中采用的路由方式。

图 8.3 展示了目的地标签路由的两个例子。从源 3 到目的地 5 的路由在图 8.3（a）的二元 3 飞网络中以粗线显示。从左到右，网络的每一级使用二进制目的地址 101 的一位来选择输出。最高位的 1 在交换的第一级选择下方的输出，0 在第二级选择上方的输出，最低位的 1 在最后一级选择下方的输出。



(a) Routing from 3 to 5 in a 2-ary 3-fly　　(b) Routing from 7 to 11 in a 4-ary 2-fly

图 8.3　目的标签路由的两个实例

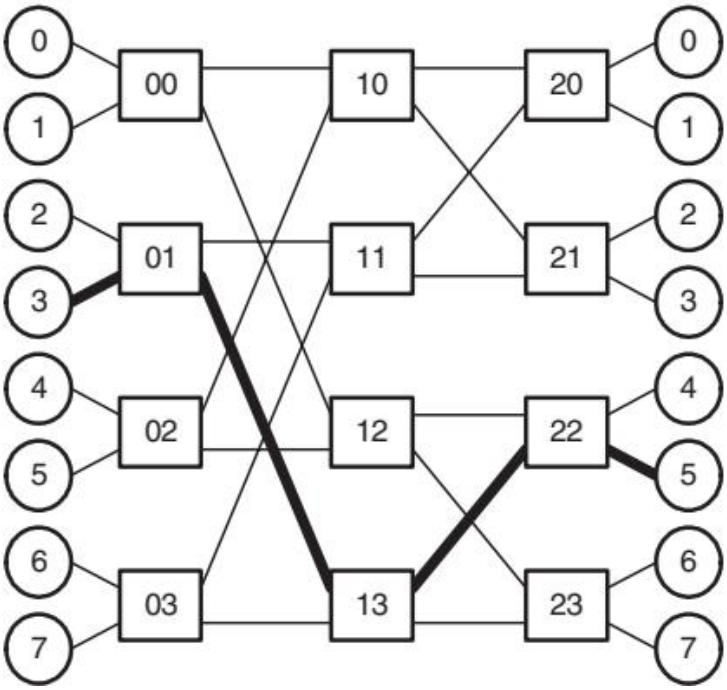
（a）在 2 元 3 飞网络中从 3 到 5 的路由

（b）在 4 元 2 飞网络中从 7 到 11 的路由

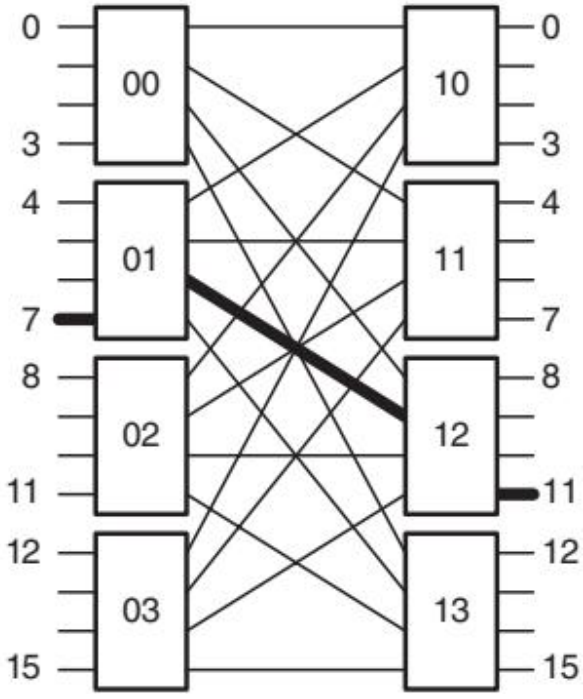
图 8.3 目的地标签路由的两个例子：（a）在 2 元 3 飞网络中从源 3 到目的地 5 的路由。二进制目的地址 $5 = 101_2$ = 下、上、下，选择路由。（b）在 4 元 2 飞网络中从 7 到 11 的路由。解释为四进制数字的目的地址 $11 = 1011_2 = 23_4$，选择路由。

8.4 确定性路由

回顾我们从 3 路由到 5 的描述，我们实际上从未使用源节点的地址。事实上，从任何源节点出发，使用相同的 101 交换机端口模式，都会路由到目的地 5，与源节点无关。也不难说服自己，这一事实对所有可能的目的地都成立。因此，k 元 n 飞网络中的目



（a）在 2 元 3 飞网络中从 3 到 5 的路由



（b）在 4 元 2 飞网络中从 7 到 11 的路由

的地标签路由仅取决于目的地地址，而非起始位置。
图 8.3 (b) 展示了更高基数蝶形网络中的一个示例路由。该图中的粗线显示了四元（基数 4）2 飞网络中从节点 7 到节点 11 的路由。与二进制网络一样，从左到右，目的地地址的各位数字决定了网络每一级的输出端口。然而，对于四元网络，目的地地址被解释为四元数 $11 = 1011_2 = 23_4$。每个路由器的输出端口从顶部开始编号，从零开始。目的地地址 23_4 选择第一个路由器的端口 2（从上数第三个）和第二个路由器的端口 3（底部）。与前一个例子一样，无论起始点如何，这种端口选择都会选择目的地 11。

8.4.2 立方网络中的维序路由

维序路由或 e-cube 路由是直接 k 元 n 立方网络（环面和网格）中目的地标签路由的类似物。与目的地标签路由一样，目的地地址的数字（解释为基数 k 的数）被逐一用于指导路由。然而，每个数字不是用于在给定级选择输出端口，而是用于在给定维度中选择一个节点。与蝶形网络不同，立方网络可能需要多跳来解决每个地址数字，然后才能继续下一个数字。
作为维序路由的一个例子，考虑一个数据包在图 8.4 所示的 6 元 2 立方中从节点 s = 03 传输到节点 d = 22。由于环面的每个维度可以顺时针或逆时针方向遍历，e-cube 路由的第一步是计算每个维度上的最短或首选方向。为了找到首选方向，我们首先为源地址和目的地地址的每个数字 i 计算相对地址 Δ_i：

$$\begin{aligned} m_i &= d_i - s_i \bmod k \\ \Delta_i &= m_i - \begin{cases} 0 \text{if} m_i \le k/2, \\ k \text{otherwise} \end{cases} . \end{aligned}$$

然后可以用它来计算我们的首选方向：

$$D_{T,i} = \begin{cases} 0, & \text{若} |\Delta_i| = k/2, \\ \text{sign}(\Delta_i), & \text{其他情况。} \end{cases} \tag{8.4}$$

其中 T 表示该函数适用于环面。在讨论首选方向为零的情况之前，我们回到我们的例子。
根据上述公式，相对地址为

$$\begin{aligned} m &= (2,2) - (0,3) \bmod 6 = (2,5) \\ \Delta &= (2,5) - (0,6) = (2,-1). \end{aligned}$$

因此，我们的首选方向是

$$D = (+1,-1).$$

一旦计算出首选方向向量，数据包就一次在一个维度上路由。在每个维度内，数据包沿首选方向前进，直到在该维度上到达与目的地相同的坐标。在图 8.4 的例子中，数

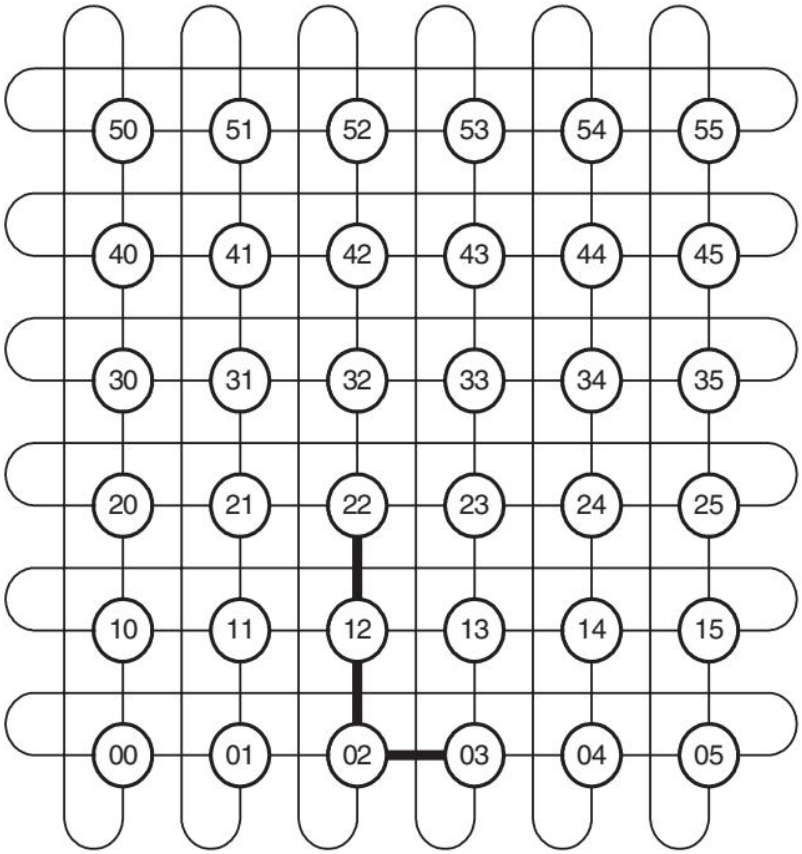


图 8.4 6 元 2 立方中维序路由的一个例子。数据包从节点 s = 03 路由到节点 d = 22，首先在 x 维度路由，然后在 y 维度路由。

据包从节点 s = 03 开始，在 x 维度上沿负方向（地址递减）移动。经过一跳后，在节点 02，它已经到达 x 维度上的正确坐标，因此开始在 y 维度上沿正方向路由。数据包再经过两跳到达目的地节点 22。

现在考虑相同路由问题，但目的地稍微移动到 d = 32。按照相同步骤，我们发现 $D = (0, -1)$。在 x 维度上的路由保持不变，但对于 y 维度，首选方向是 $D_y = 0$。在这种情况下我们如何路由数据包？通过将目的地节点移动到 32，在 y 维度上沿正方向或负方向路由都需要三跳。因此，为了平衡负载，重要的是流量在两个方向上均匀分布。一个简单的方法是放弃确定性算法，随机将流量均匀分配到正 y 和负 y 方向。3 通过直觉或从公式 8.4 也可以很容易地验证，首选方向为零只可能在 k 为偶数时发生。

到目前为止，我们主要关注环面，但维序路由在网格中同样适用。缺少环绕通道简化了首选方向的选择，在这种情况下，首选方向也是唯一有效的方向：

$$D_{M,i} = \begin{cases} +1 \text{if} d_i > s_i \\ -1 \text{otherwise.} \end{cases}$$

尽管负载均衡特性普遍较差，维序路由在网格和环面网络中得到了广泛应用，原因有两个。首先，它非常容易实现。特别是，它允许路由器按维度切片或跨维度分区。其次，它通过防止维度之间任何通道依赖循环，简化了死锁避免问题。然而，在一个维度内仍可能发生死锁。（见第 14 章。）

8.5 案例研究：Cray T3D 中的维序路由

图 8.5 展示了一个 Cray T3D [95，161]，它在 3-D 环面中连接多达 2,048 个 DEC Alpha 处理单元。T3D 是一个共享内存多处理器。每个处理单元包括本地内存，但可以通过环面网络转发加载和存储操作来访问所有其他处理单元的本地内存。每对处理单元通过一个网络接口共享一个路由器。

T3D 网络使用维序路由，并采用维度切片（第 7.2.2 节）路由器实现，如图 8.6 所示。该路由器在三个相同的 ECL 门阵列上实现，分别路由 x、y 和 z 维度。整体设计紧密遵循 J-Machine 路由器（第 5.5 节）的组织结构。这种分区之所以可行，是因为维序路由。我们在练习 8.6 中考虑一种不同的分区。

当数据包从网络接口到达时，x 路由器会检查该数据包，以确定它必须在 +x 维度、-x 维度上路由，还是如果已经到达目的 x 坐标，则继续前往 y 路由器。假设数据包沿 +x 方向（通过 xpOut 通道）转发。在每个后续的 x 路由器中，路由器检查数据包是否位于正确的 x 坐标。当数据包到达正确的坐标时，它被转发到 y 路由器。否则，它继续沿 +x 方向移动。

每个 T3D 路由器通道的带宽为 300 MB/s，并通过模块之间的线垫传输。每个通道有 16 个数据位和 8 个控制位，工作频率为 150 MHz，与原始 Alpha 21064 处理器相同。线垫是一束线缆，手动连接到板边连接器以实现环面拓扑。它被称为垫，因为它类似于不规则编织的织物。每个数据和控制信号作为差分 ECL 信号通过线垫中的双绞线

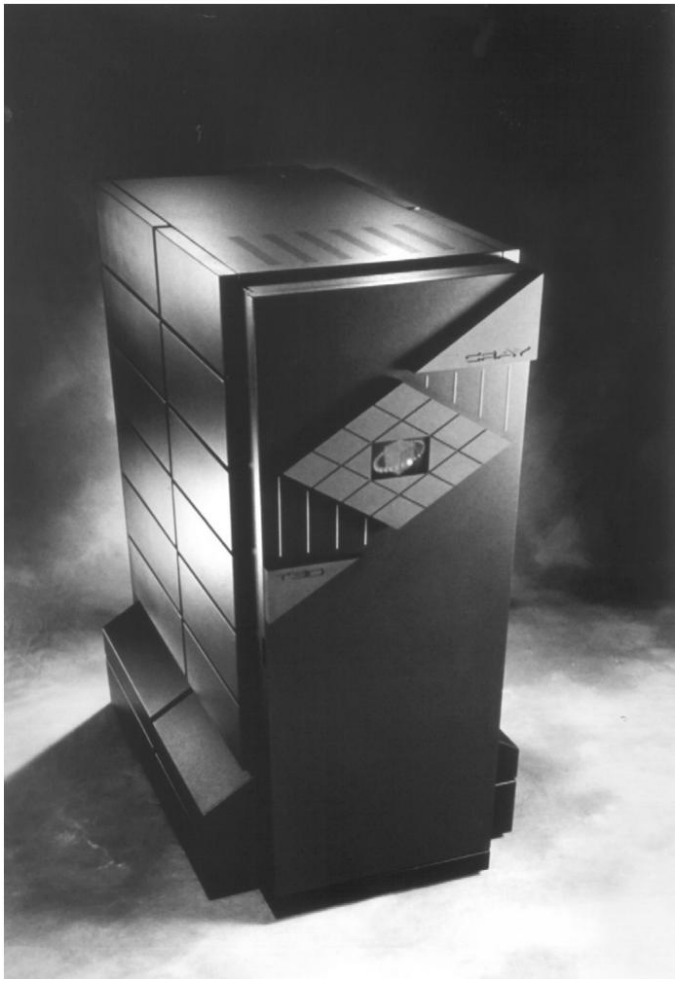


图 8.5 一个 Cray T3D 在共享内存的 3 维环面中连接多达 2048 个 DEC Alpha 处理器。

传输。

8.5 案例研究：Cray T3D 中的维序路由

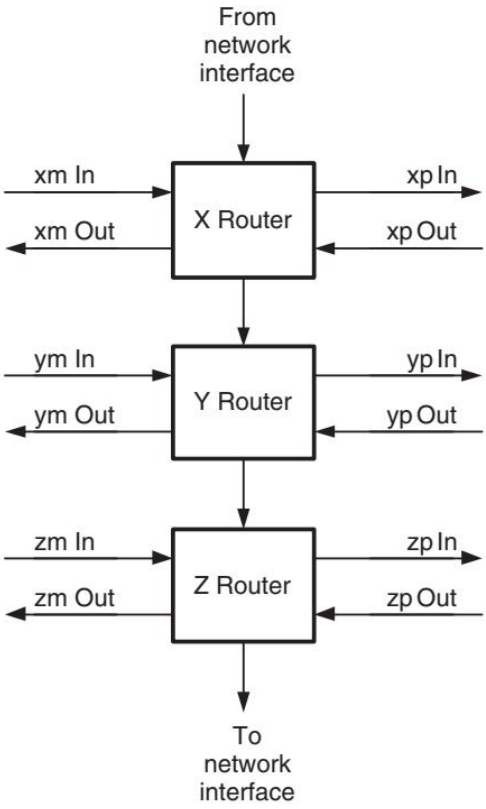


图 8.6 一个 T3D 路由器被划分到三个相同的 ECL 门阵列芯片上，分别对应 x、y 和 z 维度。

Cray T3D 包含一组仅以 x 和 z 维度连接到网络的 I/O 节点。这使得环面网络有些不规则。这似乎合理，因为消息总是从 x 开始并以 z 结束，但当发送消息的节点与 I/O 节点的地址仅在 y 维度不同时会发生什么？为了使维序路由仍然有效，这些节点被赋予两个地址。我们在练习 8.8 中探讨这个问题。

8.6 文献注释

龙卷风流量的路由问题和加权随机解决方案由 Singh 等人 [168] 描述。路由关系的不同形式及其在分析死锁中的重要性已由 Dally [57] 和 Duato [60，61，62] 描述。蝶形网络中的目的地标签路由最早由 Lawrie [110] 描述，环面网络中的 e-cube 路由归功于 Sullivan 和 Bashkow [179]。对于度数为 δ 的网络，Borodin 和 Hopcroft [28] 以及 Kaklamanis 等人 [91] 表明，对于任何确定性路由算法，某些流量模式可以导致至少 $\Omega(\sqrt{N}/\delta)$ 的通道负载。

8.7 练习

8.1 路由算法之间的权衡。重新考虑第 8.1 节中的路由算法和网络。你会选择哪种算法来优化以下方面：
（a）最小消息延迟。
（b）均匀流量下的最佳吞吐量。
（c）在许多排列流量模式下的最高平均吞吐量。
将你的选择限制为每个不同标准对应一种算法，并为你的选择辩护。
8.2 增量路由的局限性。描述一种可以使用关系 8.1 中的基于路径的关系来指定，但无法用关系 8.2 和 8.3 中的任一增量形式表达的路由算法。
8.3 增量路由和一次性路由的头部比特。目的地标签路由可以实现为增量算法或一次性算法。计算实现每种方法需要与数据包一起存储的比特数。一种方法需要更少的比特吗？这种关系在一般拓扑中的最小路由中成立吗？这与拓扑的路径多样性有何关系？
8.4 环中的回溯。假设我们在第 8.1 节的路由示例中允许回溯。是否有可能开发出一种比加权随机算法具有更好最坏情况吞吐量的算法？如果是，给出这样的算法；否则，解释为什么不存在这样的算法。
8.5 带额外级的蝶形网络中的路由。描述一种目的地标签路由的确定性扩展，以处理具有一个或多个额外级的 k 元 n 飞。提出一种简单的方法将随机化引入此算法以改善负载均衡。
8.6 Cray T3D 中的方向序路由。假设你重新排列图 8.6 中 Cray T3D 路由器通道上的标签，使得第一个路由器处理 +x 和 +y，第二个路由器处理 +z 和-x，第三个路由器处理-y 和-z。描述一种可以与此分区一起工作的路由算法。记住，一旦数据包到达三个路由器中的每一个，它就不能再返回之前的路由器。
8.7 方向序路由的优势。考虑你为练习 8.6 推导的路由算法。该算法与维序路由相比有哪些优势？
8.8 T3D 中到 I/O 节点和从 I/O 节点的路由。通过向 3 立方体添加额外的 x 和/或 z 坐标，I/O 节点仅以 x 和 z 维度添加到 T3D 网络中。例如，假设你有一个 64 节点的 4 元 3 立方体，节点地址从 $(0,0,0)$ 到 $(3,3,3)$。可以添加一个地址为 $(4,0,0)$ 或 $(0,0,4)$ 的 I/O 节点。解释如何为每个 I/O 节点分配一对地址，以便始终可以使用维序路由从机器内部的任何节点路由到 I/O 节点，以及从 I/O 节点路由到任何内部节点。
8.9 环面中半圈流量的负载均衡。对于维序路由，我们讨论了当节点恰好位于环面环的半圈时出现的负载均衡问题。如果在这种半圈情况下我们总是选择正方向而不是进行负载均衡，这将如何影响 k 为偶数的 k 元 n 立方体上均匀流量的吞吐量？在这种情况下最坏情况的吞吐量是多少？以容量分数表示你的结果。提出一种在保持确定性算法的同时改善半圈情况下负载均衡的方法。重新计算均匀和最坏情况的吞吐量。
8.10 CCC 中的最小路由。为练习 5.8 中描述的一般 CCC 拓扑设计一种接近最小的路由算法。选择简单性，而不是在所有情况下找到精确的最小路由，但要确保算法生成

的任何路径都不大于直径 $H_{max} = 2n + \lfloor n/2 \rfloor - 2$，如 [128] 所示。评论你的路由算法在均匀流量下的负载平衡。

注释

- 1. 这个环上的路由问题在 2002 年作为博士资格考试题，超过 90% 的考生最初选择了贪婪算法。
- 2. 一个例外是那些在均匀流量下最小路由并非最优的奇特拓扑，如练习 3.3 所示。
- 3. 在练习 8.9 中，我们探讨了不均衡此负载的代价以及实现这种均衡的确定性方法。

第九章　无关路由

Oblivious 路由，即我们在不考虑网络状态的情况下路由数据包，实现简单且分析简单。虽然添加有关网络状态的信息可能潜在地提高路由性能，但它也增加了相当大的复杂性，如果不小心处理，可能会导致性能下降。

Oblivious 路由的主要权衡是在局部性和负载平衡之间。通过首先将每个数据包发送到随机节点，然后从那里直接发送到其目的地，Valiant 的随机路由算法（第 9.1 节）可以精确地平衡任何流量模式的负载。然而，这种负载平衡是以破坏流量模式中的任何局部性为代价的——即使是最邻近的流量也不会比最坏情况的流量有更好的性能。另一方面，最小 oblivious 路由（第 9.2 节）保留了局部性，并且通常提高了网络在所有流量模式下的平均情况吞吐量。然而，在环面网络上，任何最小算法给出的最坏情况吞吐量最多是 Valiant 算法的一半。我们还介绍了负载平衡的 oblivious 算法，这些算法提供了最小算法和 Valiant 方法之间的中间点。

分析 oblivious 路由算法很直接，因为它们给出了线性通道负载函数。这种线性使得从流量模式 Λ 计算通道负载 γ 变得容易，从而计算路由算法在任何流量模式下的理想吞吐量。我们将在第 9.4 节中看到，这种线性使得计算 oblivious 路由算法的最坏情况流量模式相对容易。

9.1　Valiant 的随机路由算法

使用 Valiant 算法，可以在几乎任何拓扑 1 上为任何流量模式平衡负载，其中从 s 发送到 d 的数据包首先从 s 发送到随机选择的中间终端节点 x，然后从 x 发送到 d。2 两个阶段中的每一个都可以使用任意路由算法，但一般来说，在均匀流量下平衡负载的路由算法效果最好。因此，对于环面和网格网络，维度顺序路由是一个合适的选择，而对于蝶形网络，目的地标签路由效果很好。然后，无论原始流量模式如何，Valiant 算法的每个阶段都表现为均匀随机流量。因此，Valiant 算法将任何流量模式的负载降低到随机流量负载的两倍或网络容量的一半。

9.1.1　环面拓扑上的 Valiant 算法

Valiant 算法在 k 元 n 立方体网络上以牺牲局部性为代价提供了良好的最坏情况性能。两个随机阶段中的每一个都将每个数据包在 n 个维度中的每个维度上平均发送距离 $k/4$，两个阶段的总跳数为 $nk/2$。因此，每条链路平均负载为 $\gamma = k/4$，吞吐量为 $4b/k$。这种吞吐量几乎是最优的，正如龙卷风流量模式所证明的那样。在龙卷风流量下，每个数据包必须经过 $H = n(k/2 - 1)$ 跳（对于偶数 k）。在这种模式下，任何路由算法的通道负载至少为

$$\gamma \geq \frac{H_{\min} N}{C} = \frac{n(k/2-1)N}{2nN} = \frac{k}{4} - \frac{1}{2}.$$

随着 k 的增加，这个理想通道负载接近 Valiant 算法引起的通道负载，因此 Valiant 算法是渐近最优的。

相比之下，任何最小算法在龙卷风流量下，顺时针通道的通道负载至少为 $\gamma = k/2 - 1$，逆时针通道为 0。由于这种糟糕的负载平衡，吞吐量最多为 $\frac{b}{(k/2-1)} \approx 2b/k$。因此，没有最小路由算法能够实现超过 Valiant 算法大约一半的最坏情况吞吐量。

这种在最坏情况流量模式（如龙卷风）上的良好性能是以牺牲局部性和开销为代价的，局部性被随机化破坏，开销由两阶段路由增加。例如，最邻近流量通常每个数据包需要一跳，但被减少为两个随机流量阶段，将跳数从 H = 1 增加到 nk/2，并降低吞吐量。将 Valiant 算法应用于随机流量会将吞吐量减半，并将延迟加倍，因为将一个随机流量阶段替换为两个阶段。第 9 章 Oblivious 路由

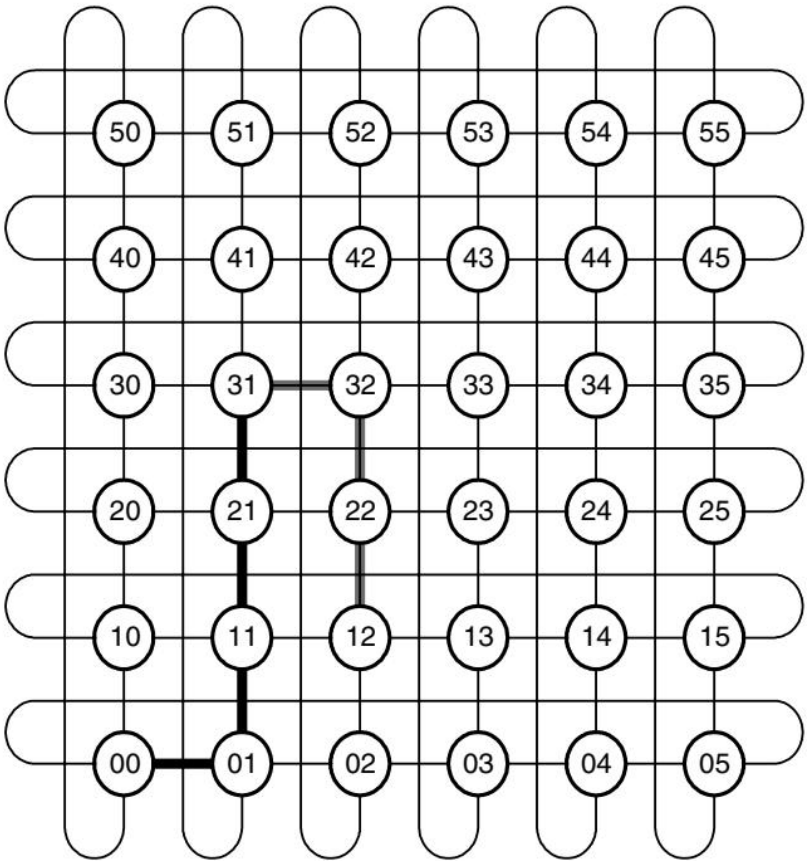


图 9.1 在 6 元 2 立方体上随机路由（Valiant 算法）的示例。数据包从 s = 00 路由到 d = 12，分两个阶段。在第一阶段，数据包被路由到随机选择的中间节点 x = 31，如深色粗线所示。第二阶段将数据包从 x = 31 传递到 d = 12，如灰色粗线所示。两个阶段都使用维度顺序（e-cube）路由。

图 9.1 显示了使用 Valiant 算法在二维基数 6 环面（6 元 2 立方体）中将数据包从节点 s = 00 传递到节点 d = 12 的示例。数据包通过随机选择的中间节点 x = 31 进行路由。在第一阶段，数据包使用维度顺序路由从 s = 00 到 x = 31，需要 4 跳，如深色粗线所示。然后，在第二阶段，数据包从 x = 31 路由到 d = 12，再需要 3 跳。随机路由需要

7 跳到达目的地，而最小路由算法可以在 3 跳内到达。

9.1.2 间接网络上的 Valiant 算法

将 Valiant 算法应用于 k 元 n 飞网络可以消除某些流量模式造成的瓶颈，如第 4.4 节所述。实际上，Valiant 算法的两遍路由等价于在逻辑上复制蝶形网络——从而得到一个 Beneš 网络，其中前 n 级与后 n 级共享硬件。在这种情况下，Valiant 算法是一种路由所得到的 Beneš 或 Clos 网络（如果 $k \neq 2$）的方法，即通过为每个数据包随机选择要使用的中间级交换机。Clos 网络的这种随机路由会产生均衡良好的平均负载，但由于选择每个中间节点的数据包数量的变化，可能会导致瞬时不平衡。（见练习 9.5。）流控制机制必须具有足够深的缓冲区，以平均这些瞬态负载变化。或者，自适应路由可以避免这种统计上的负载变化。

9.1 Valiant 随机路由算法

如果要在间接网络上使用 oblivious 路由，有利的做法是折叠网络，使 Valiant 算法的第一遍和第二遍以相反方向经过蝶形网络的节点。这种折叠安排消除了在遍之间转发数据包时需要将相应的源节点和目的节点连接在一起的需求。更重要的是，它通过允许数据包在到达能够到达最终目的节点的交换机时立即终止第一阶段的路径，实现了高效的最小 oblivious 路由。

9.2 最小 oblivious 路由

最小 oblivious 路由试图在不放弃局部性的情况下实现随机路由的负载均衡，方法是将路由限制为最小（最短路径）。非最小 oblivious 路由算法可以选择 R'_{xy} 中的任何路径来将数据包从 x 路由到 y，而最小 oblivious 路由将其选择限制在 R_{xy} 中的路径。对于分层拓扑，最小 oblivious 路由效果极佳——它在保持局部性的同时提供了良好的负载均衡。我们在第 9.2.1 节中探讨了胖树网络的一个例子。然而，在其他网络中，需要非最小路由来平衡负载，正如我们在环面中的龙卷风流量中所看到的那样。

9.2.1 折叠 Clos（胖树）上的最小 oblivious 路由

图 9.2 显示了在具有集中功能的 16 节点折叠 Beneš 网络（第 6.4 节）上进行最小 oblivious 路由的示例。这种类型的网络通常称为胖树。该图显示了沿左边缘的 16 个终端节点。每个终端连接到一个 2：1 集中器（第 7.1.1 节），该集中器标有与其两个终端节点匹配的地址模板。然后，每个集中器连接到 8 端口 radix-2 折叠 Clos 网络（Beneš 网络）的一个终端。Beneš 网络的每个节点都标有与该节点左侧终端可到达的所有节点匹配的地址模板。设置为 X 的模板位表示不关心，与 0 和 1 都匹配。例如，节点 00XX 匹配从 0000 到 0011，即节点 0 到 3。

为了将数据包从节点 s 路由到节点 d，数据包被路由到 s 和 d 的随机选择的最近公共祖先 x，然后从 x 路由到 d。例如，要从 s = 1 路由到 d = 6，交换机节点 0XXX-A 和

0XXX-B 是 s 和 d 的两个最近公共祖先。数据包可以通过 0XXX-A 经由深色粗体路径路由，或者通过 0XXX-B 经由灰色粗体路径路由。无论哪种情况，该路由都是最小路由——从 s = 1 到 d = 6 需要六跳。

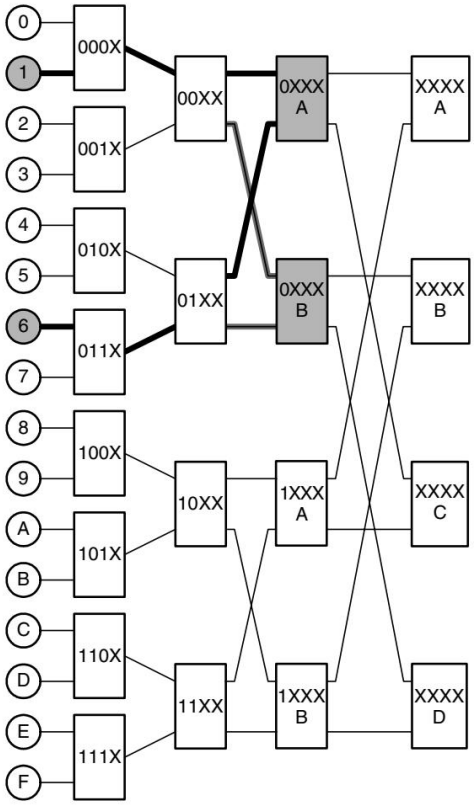


图 9.2 折叠 Clos 网络（胖树）上的最小 oblivious 路由。从源节点 s = 1 到目的节点 d = 6 的数据包首先被路由到 s 和 d 的随机选择的最近公共祖先，即交换机 0XXX-A 或 B，然后从这个公共祖先交换机路由到 d = 6。由于网络是折叠的，所有通道都是双向的。

该路由也可以增量构建。最初，数据包向右前进，在每个交换机节点的右上端口和右下端口之间做出随机路由决策，直到到达地址模板与 d 匹配的交换机节点。回顾示例，标记为 0XXX 的节点是第一个与 d = 6 = 0110 匹配的交换机节点。一旦匹配，数据包就改变方向并向左路由，使用部分目的标签路由来选择到 d 的唯一路径。目的标签路由仅使用目的地址中与交换机模板中的 X 对应的那些位，因为其他地址位在公共祖先处已经解析。在这种情况下，路由的第二阶段使用 d 的低三位 110，给出从 0XXX-A 或 B 到终端 d = 6 的下、下、上的路由。9.2 最小 oblivious 路由通过随机选择公共祖先节点，从而在路由的第一阶段随机路由，来自给定源/目的对的流量在公共祖先上以及到这些祖先的路径上精确平衡。在这个最小子网之外路由没有任何好处，该子网右侧以公共祖先节点为界。从 s 到 d 的每条路由都必须以从 s 到公共祖先 x_1 的路径开始，并且必须以从公共祖先 x_2（可能与 x_1 相同）到 d 的路径结束。在这个子网之外的任何路由都只是消耗其他交换机和通道上的带宽，而不会改善子网内的负载均衡。

Thinking Machines CM-5（第 10.6 节）使用的算法与此处描述的算法几乎相同，不同之处在于第一阶段的路由是自适应的而非 oblivious 的。

9.2.2 环面上的最小 oblivious 路由

Valiant 算法的最小版本可以在 k 元 n 立方体拓扑上实现，方法是将中间节点 x 限制在 s 和 d 之间的最小象限内。最小象限是包含 s 和 d 作为角节点的最小 n 维子网。图 9.3 给出了从 s = 00 到 d = 21 的最小无关路由示例（与图 9.1 中 6 元 2 立方体上的 s 和 d 相同）。第一步是计算相对地址 $\Delta = (2, 1)$，如第 8.4.2 节所述。Δ 的大小给出了最小象限的尺寸（在此例中，x 维度为 2 跳，y 维度为 1 跳）。首选方向向量给出了最小象限相对于节点 s 的位置。在此例中，$D = (+1, +1)$，因此最小象限位于 s = 00 的右上方。

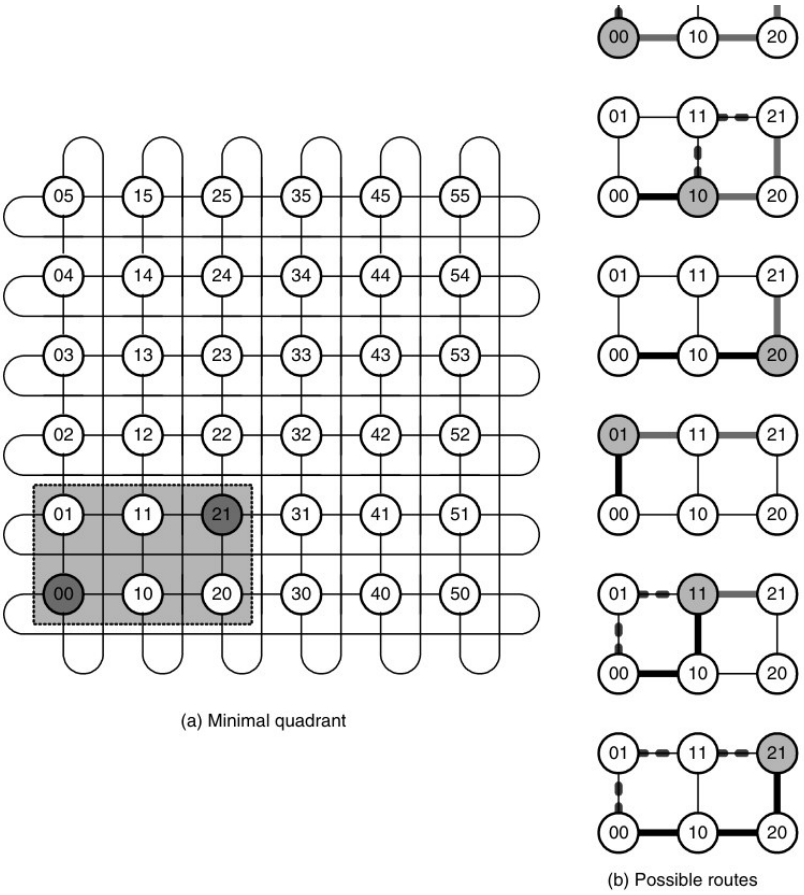
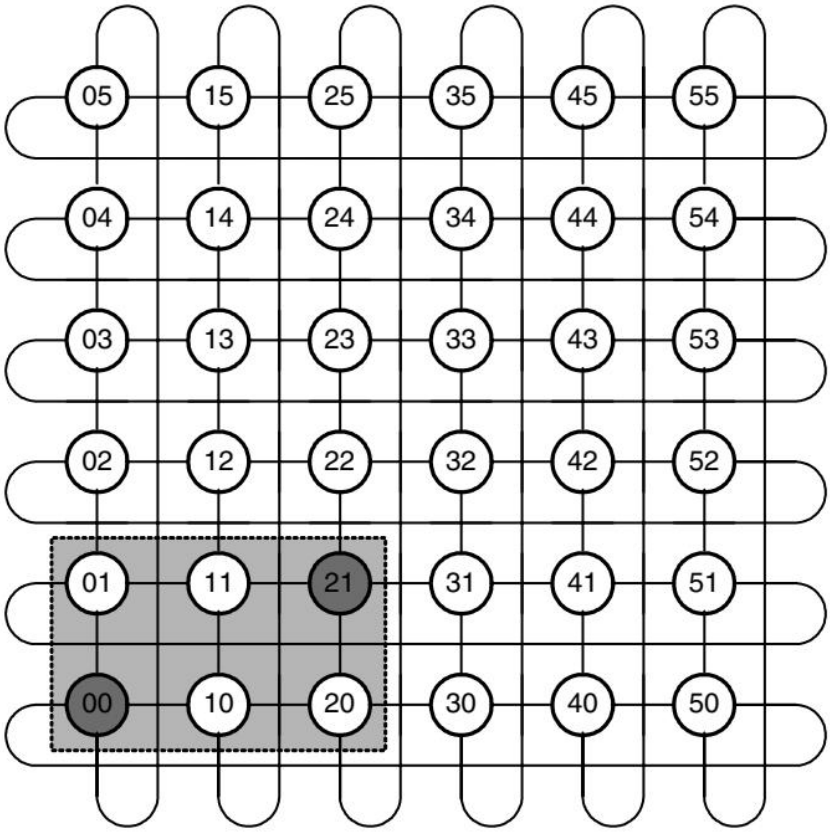


图 9.3 六元二维环面网络中的最小无关路由

一旦确定了最小象限，就从该象限内选择一个中间节点 x。然后使用 e-cube 路由将数据包从 s 路由到 x，再从 x 路由到 d。在此例中，x 有六种可能，每种可能由图 9.3（b）中的阴影节点示出。每条路由在中间节点之前的部分用粗实线表示，在中间节点之后的部分用灰色实线表示。注意，源节点和目的节点本身也可以被选为中间节点。

虽然存在六个可能的中间节点，但在此例中只有三条可能的路由——对应于可以在三个不同点进行 y 方向跳步。负载在这些路由上分布不均。从 02 进行 y 跳步的路由出现了四次，而从 00 进行 y 跳步的路由只出现了一次。这种不均衡可以通过随机化维度遍历的顺序来部分纠正。图 9.3（b）中的灰色虚线显示了当两个阶段中路由与 x 优先路由不同时，首先在 y 方向进行路由所采用的路径。在 x 优先和 y 优先路由之间随

机选择可以平衡最小象限两端 y 通道上的负载（在 12 条路由中各有 5 条使用），但中间通道仅在 12 条路由中的 2 条被使用。

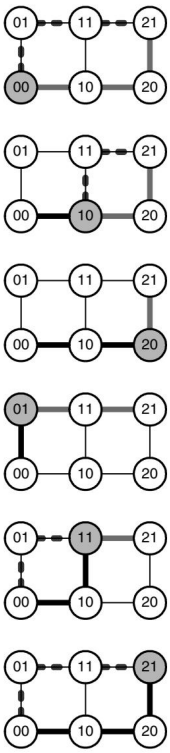


（a）最小象限

（a）最小象限
（b）可能的路由
图 9.3 在 6 元 2 立方体上从 s = 00 到 d = 21 的最小无关路由。（a）路由被限制在最小象限内——以阴影显示。（b）使用 x 优先路由时，有六条可能的路由，对应于六个可能的中间节点。粗实线显示到中间节点的路由，灰色实线显示从中间节点到目的地的路由。灰色虚线显示如果使用 y 优先路由所采用的路由。
环面上的最小无关路由在保持局部性方面表现优异。最近邻流量在该路由算法下保持局部性，随机流量也不会因吞吐量减半而负担过重。不幸的是，这种局部性是以最坏情况性能为代价实现的。如上所述，诸如龙卷风流量之类的模式将导致严重的负载不均衡。

9.3 负载均衡的无关路由

通过随机选择要路由的象限，可以在完全随机路由（Valiant 算法）和环面上的最小无关路由之间取得折中。通过按距离对象限的选择进行加权（如第 8.1 节所述），我们可以精确地平衡龙卷风流量的负载。在每个维度 i 中，我们以概率 $\frac{k-\Delta_i}{k}$ 选择短方向



（b）可能的路由

$D'_i = D_i$，并以概率 $\frac{\Delta_i}{k}$ 选择长方向 $D'_i = -D_i$。一旦选择了方向向量 D'，我们就在其选择的象限中按照最小无关路由进行路由。我们在该象限中随机选择一个中间节点，路由到该中间节点，然后从该中间节点路由到目的地。对于每个路由阶段，路由的维度顺序是随机选择的。3 两个阶段的路由始终按照 D' 指定的方向进行。

这种负载均衡的无关路由方法是在局部性和最坏情况吞吐量之间的折中。对于局部流量，它优于 Valiant 算法，因为它更频繁地沿短方向路由，但其性能不如最小路由，因为它将一些数据包路由通过非最小象限。尽管负载均衡的无关路由对于龙卷风流量表现良好，但其最坏情况吞吐量远低于 Valiant 算法。

9.4 无关路由的分析

由于无关路由算法选择路径与网络状态无关，从 s 到 d 发送单位负载（即 $\lambda_{sd} = 1$）在通道 c 上引起的负载 $\gamma_{c(sd)}$ 与任何其他节点对之间发送的负载无关。这一性质使我们能够通过将流量矩阵每个元素的贡献相加，来求解给定算法和流量模式下通道上的负载：

$$\gamma_c = \sum_{i,j} \lambda_{ij} \gamma_{c(ij)}. \tag{9.1}$$

例如，在图 9.3 中，当使用两个阶段均为 x 优先路由从 $s_1 = 00$ 路由到 $d_1 = 21$ 时，通道（10，20）在六条可能路由中的两条中被使用，因此我们有 $\gamma_{(10,20)(00,21)} = 1/3$。当从 $s_2 = 10$ 路由到 $d_2 = 31$ 时，通道（10，20）在六条可能路由中的四条中被使用，因

此 $\gamma_{(10,20)(10,31)} = 2/3$。
现在，考虑一个只有两个非零元素的流量矩阵 $\lambda_{(00,21)} = \lambda_{(10,31)} = 1$。我们可以通过将流量矩阵的两个非零元素引起的分量相加来计算 $\gamma_{(01,02)}$：

$$\gamma_{(10,20)} = \gamma_{(10,20)(00,21)} + \gamma_{(10,20)(10,31)} = 1/3 + 2/3 = 1.$$

使用无关路由时，通道负载是线性的。我们将流量模式引起的通道总负载计算为流量模式各个元素引起的负载的叠加。通道负载的这种线性使我们能够通过找到使最大通道负载最高的流量矩阵来求解最坏情况流量模式。
为了计算路由算法在给定流量模式下达到的吞吐量，我们将流量矩阵归一化，使行和列之和均为 14，然后计算比例因子 Θ，使得流量矩阵 $\Lambda' = \Theta\Lambda$ 恰好使关键通道饱和。我们的原始流量矩阵 Λ 引起最大通道负载 $\gamma_{\max}(\Lambda)$，这可能大于通道带宽 b。因此，吞吐量和我们的比例因子由 $\Theta = \frac{b}{\gamma_{\max}(\Lambda)}$ 给出。根据线性关系，该比例因子给出的最大通道负载等于通道容量：$\gamma_{\max}(\Lambda') = \Theta\gamma_{\max}(\Lambda) = b$。
为了简化对最坏情况流量模式的搜索，我们首先观察到所有最坏情况流量模式都是置换。要理解这一点，请注意所有归一化流量矩阵都可以表示为置换的叠加：

$$\Lambda = \sum_i w_i \Pi_i, \quad \text{s.t.} \sum_i w_i = 1. \tag{9.2}$$

这个和式中的一个置换，比如 Π_{max}，相比其他置换会产生最大的最大信道负载：$\forall i$, $\gamma_{\max}(\Pi_{\max}) \geq \gamma_{\max}(\Pi_i)$。根据线性性，我们可知

$$\gamma_{\max}(\Lambda) = \sum_i w_i \gamma_{\max}(\Pi_i) \leq \gamma_{\max}(\Pi_{\max}). \tag{9.3}$$

因此，一个置换总是可以成为最坏情况流量模式。
我们可以通过构造一个二分图来找到使给定信道 c 的负载最大的置换，如图 9.4 所示。图的左侧为每个源节点有一个顶点，右侧为每个目的节点有一个顶点。从每个源节点 s 到每个目的节点 d 有一条边，标记为 $\gamma_{c(sd)}$，即从 s 到 d 的单位流量在 c 上引起的负载。任何置换 Π 对应于该图的一个匹配，其中当且仅当 $\pi_{sd} = 1$ 时存在从 s 到 d 的边。根据线性性，该置换在 c 上的总负载 $\gamma_c(\Pi)$ 是匹配的权重，即匹配中所有边权重的总和。因此，通过计算该图上的最大权重匹配，我们可以找到在 c 上产生最大负载的置换。然后我们对网络中的每个信道重复此过程，以找到在所有信道上产生最大负载的置换。

9.4 无关路由分析

如上所述计算路由算法的最坏情况流量模式，通常得到的吞吐量远低于第 3.2 节中描述的标准流量模式的最差情况。表 9.1 显示了四种路由算法：e-cube、Valiant、最小无关（第 9.2.2 节）和负载均衡无关（第 9.3 节）在五种标准流量模式以及该特定算法的最坏情况流量模式下的吞吐量。每种算法的最坏情况流量模式不同，并且对于最小无关和负载均衡路由算法，其吞吐量均显著低于标准模式。

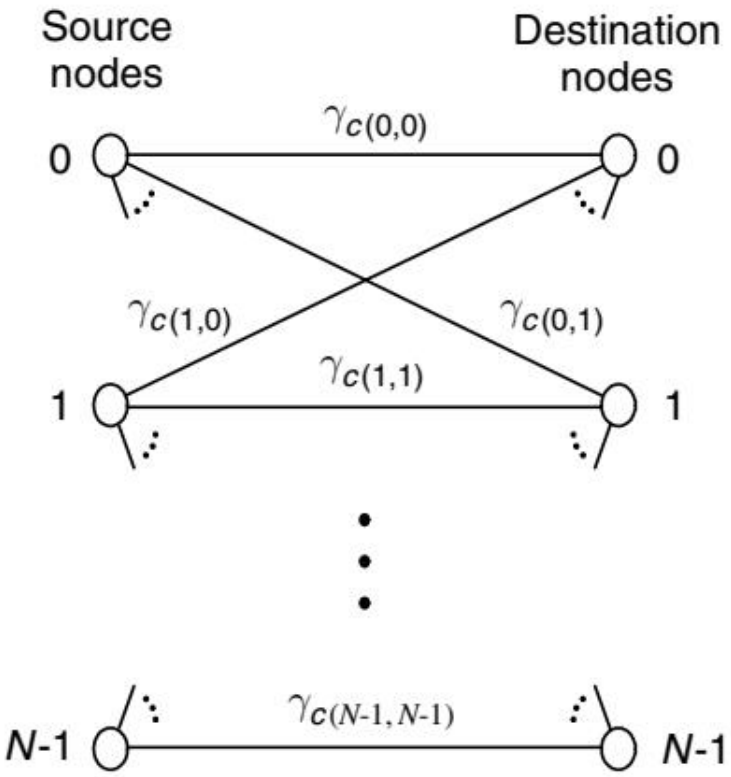


图 9.4 为了找到使信道 c 上负载最大的置换，构造一个二分图，左侧顶点对应每个源节点，右侧顶点对应每个目的节点。从每个源节点 s 到每个目的节点 d 的边标记为 $\gamma_{c(sd)}$。该图的一个匹配描述了一个置换 Π，匹配的权重即为该置换在 c 上引起的负载 $\gamma_c(\Pi)$。因此，图的最大权重匹配给出了使 c 上负载最大的置换。

表 9.1　多种流量模式下四种路由算法的吞吐量

Load	e-cube	Valiant	Minimal	Load-Balanced
Nearest neighbor	4.00	0.50	4.00	2.33
Uniform	1.00	0.50	1.00	0.76
Bit complement	0.50	0.50	0.40	0.42
Transpose	0.25	0.50	0.54	0.57
Tornado	0.33	0.50	0.33	0.53
Worst-case	0.25	0.50	0.21	0.31

通过模拟几种可疑的流量模式来测试路由算法（这是上述方法开发之前的做法）可能会对最坏情况吞吐量给出非常误导的结果。不幸的是，由于自适应路由算法的信道负载函数是非线性的，该方法不能应用于自适应路由。目前，在特定流量模式上进行模拟是我们评估自适应路由算法的唯一方法。

9.5　案例研究：Avici 太比特交换路由器（TSR）中的无关路由

Avici 太比特交换路由器（TSR）（图 9.5）是一种可扩展的互联网路由器，它使用三维环面互连网络作为交换结构，将输入线卡连接到输出线卡 [34，49，50]。每个机柜包含 40 个 5 Gbits/s 线卡，连接为 $2 \times 4 \times 5$ 折叠环面网络。最多可组合 8 个机柜，形成总规模高达 320 个节点（$8 \times 8 \times 5$）的机器，总带宽为 1.6 Tbits/s。每个线卡提供 5 Gbits/s（全双工——5 Gbits/s 输入和 5 Gbits/s 输出）的接口带宽，通常连接到 OC-48 或 OC-192（2 个卡）的 SONET 上的分组（POS）链路。

图 9.6 显示了一个具有单个 OC-48 POS 接口的线卡。卡底部的 6 个芯片（带散热器）组成一个位切片环面路由器。路由器宽 24 位，6 个芯片上各 4 位。每条网络链路宽 28 位（24 位数据加 3 位控制和 1 位时钟），工作频率为 400 MHz，数据速率为 9.6 Gbits/s（1.2 Gbytes/s）。5 每个路由器连接到 12 条这样的链路，6 个方向各一条输入和一条输出。

Avici TSR 在许多方面都是一款值得注意的路由器。在第 15.7 节中，我们将研究它如何为每个输出使用虚拟网络来提供服务保证——特别是保证发往一个输出的流量不会干扰发往另一个输出的流量。在本节中，我们将仅关注 TSR 用于平衡网络链路负载的无关路由算法。

TSR 网络必须提供不间断路由，即使在部分填充的路由器中或当一个或多个线卡发生故障或被移除时出现的不规则网络中也是如此。为了便于在这种不规则的部分环面拓扑中进行路由，TSR 采用源路由（第 11.1.1 节）。发送节点或源以路由方向字符串（例如 +x，-y，+x，+z，$-y, -x$）的形式确定确切路由。该字符串的每个字符指定从源到目的地的路由中的一跳。例如，+x 指定第一跳沿正 x 方向。软件进程通过可能不规则的拓扑找到每个源 s 和每个目的地 d 之间的路由，并将这些路由存储在表中。每个

分组在其源节点 s 查询该表以找到到达其目的地 d 的路由。该路由被附加到分组头部，并在沿途的每一步用于选择下一个方向。

9.5 Avici 太比特交换路由器中的 oblivious 路由

图 9.5 Avici TSR 是一种可扩展的互联网路由器，它使用三维环面互连网络作为其交换结构。每个机柜容纳 40 块线卡，每块线卡提供高达 5 Gbits/s 的接口容量，排列成 $2 \times 4 \times 5$ 的子环面。可以添加额外的机柜，将机器扩展到 $8 \times 8 \times 5$ 的环面。

由于输入 SONET 链路上没有背压，TSR 必须能够路由所有流量模式而不会使其任何内部交换结构通道过载——即使存在对抗性流量。为了满足这一要求，TSR 通过使用 oblivious 路由来平衡最坏情况流量下网络通道上的负载。每个从源线卡 s 到目的线卡 d 穿越网络的数据包会从路由表中存储的 24 条到 d 的路由中随机选择一条。来自同一流的数据包通过基于流标识符进行随机路由选择来保持顺序，使得流中的所有数据包选择相同的路由。

为了平衡从 s 到 d 的最小象限中 3 条链路上的负载，d 的路由表中的 24 条路由包括 3 个初始方向各 8 条路由。例如，如果 $s = (0, 0, 0)$ 且 $d = (3, 2, 4)$，在完全填充的 $8 \times 8 \times 5$ 系统中，最小方向是 +x、+y 和 -z。路由表将包括以这 3 个初始方向各开始的 8 条路由。

路由过程的细节如图 9.7 所示。数据包进入线卡 s 并输入到路由查找和分类过程。该过程为数据包选择目的线卡 d 并计算数据包的流标识符 f。属于同一流的所有数据包将具有相同的流标识符。6 流 ID 被哈希以给出路由选择器 $r = \text{hash}(f) \in [0, 23]$。然后

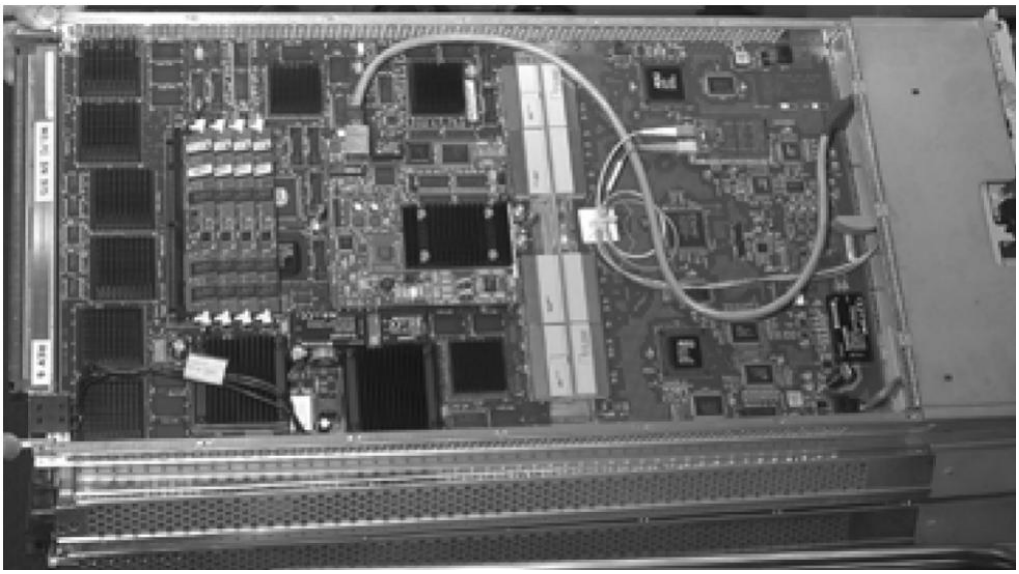


图 9.6 单个 OC-48 （2.488 Gbits/s）SONET 线卡分为两部分。上部子卡是接口特定的。下部主卡是接口无关的。板底部带有散热片的六个芯片组成一个 24 位宽的环面路由器，每个芯片四位切片。

路由表由 $24d + r$ 索引，以选择从 s 到 d 的 24 条路由之一。所选路由被附加到数据包头并用于路由。

使用表驱动的 oblivious 路由算法进行负载平衡的关键是构建一个好的表。如果每个 (s, d) 对只存储少量 N_R 条路由，则必须谨慎选择这些路由。一种方法是通过一次生成一条路由来增量构建路由表。每次生成路由时，它会为其使用的每个通道增加一个单位的负载。每条路由的生成方式是从最小路径中选择路径负载最低的路径——路径中所有通道的负载总和。我们在练习 9.3 和 9.4 中探讨这种路由生成。

9.6 文献注释

随机路由最早由 Valiant [187] 描述。第 9.2.2 节中描述的最小 oblivious 路由算法归功于 Nesson 和 Johnsson [135]。第 9.3 节的负载平衡 oblivious 路由算法由 Singh 等人 [168] 提出。Towles 和 Dally 描述了 oblivious 路由的最坏情况分析方法 [185]。

9.7 练习

9.1 均匀流量下的负载平衡路由。找出均匀流量下 k 元 n 立方体上负载平衡路由算法（第 9.3 节）引起的通道负载的一般表达式。

9.2 维序路由的最坏情况最优性。构造一个论证，说明为什么维序路由在二维环面中的最小路由算法具有最优的最坏情况吞吐量（在常数加性项内最优）。这个论证对 n > 2 成立吗?

9.3 Avici TSR 中的路由生成。编写一个程序，为任意拓扑网络构建源路由表，类似于 Avici TSR 的路由表。使用第 9.5 节末尾讨论的方法构建表。假设每个 (s, d) 对有

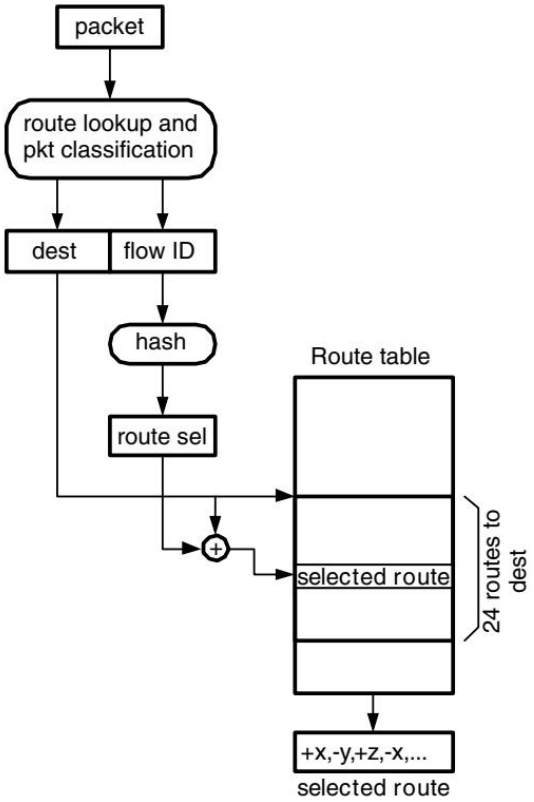


图 9.7 每个源节点 s 上的 TSR 路由表为每个目的节点 d 存储 24 条路由，3 个初始方向各 8 条路由。为了保持流内数据包的顺序，特定数据包要使用的路由通过哈希该数据包的流标识符来选择。

$N_R = 8$ 个条目。通过生成多个随机排列并确定这些排列的最大链路负载来测试你的表。现在改变 N_R，并确定负载不平衡如何随每个 (s, d) 对的路由数量变化。

9.4 Avici TSR 中的迭代路由生成。执行与练习 9.3 相同的分析，但这次编写一个迭代路由生成程序。在确定初始路由集后，使用该初始集的通道负载生成第二组路由。重复此过程 N_I 次迭代。这种迭代算法的性能与单遍算法相比如何?

9.5 胖树中的瞬时不平衡。假设我们在批处理模式下对 1,024 节点的折叠 4 元 5 飞（胖树）使用 Valiant 算法。对于每一批，每个源终端选择一个随机中间目的终端。设 $f(x)$ 为选择中间终端 x 的源终端数量。在所有中间终端 x 上，$\max(f(x))$ 的期望值是多少?

注释

- 1. Valiant 算法可以应用于任何连通拓扑——每个终端对之间至少有一条路径的拓扑。例如，如果输出终端连接到相应的输入终端，则蝶形网络是连通的。
- 2. 这实际上是 Valiant 原始工作的简化，原始工作需要针对每种拓扑进行专门化，并允许比这里提出的更严格的分析。
- 3. 只需在维度的 nn 种旋转中随机选择一种，而不必在维度的所有 n!n! 种排列中选择。

- 4. 使用归一化流量矩阵时，每个源发送一个单位的流量，每个目的地接收一个单位的流量。这种行和列之和均为单位值的矩阵通常称为双随机矩阵。
- 5. 后来的 TSR 系统具有更高带宽的线卡和更高带宽的交换结构链路。
- 6. 在 IP 中，流由具有相同源地址和端口以及目的地址和端口的数据包组成。

第十章　自适应路由

自适应路由

自适应路由算法利用网络状态信息（通常是队列占用情况）在备选路径中选择以传递数据包。由于路由依赖于网络状态，自适应路由算法与流控机制紧密耦合。这与确定性和 oblivious 路由形成对比，后者的路由算法和流控机制在很大程度上是正交的。

理论上，一个好的自适应路由算法应当优于非自适应路由算法，因为它利用了非自适应算法无法获取的网络状态信息。然而在实践中，许多自适应路由算法的最坏情况性能很差。这主要是由于大多数实用自适应路由算法的局部性。由于它们在做出路由决策时仅使用局部网络状态信息（例如本地队列长度），因此其路由方式虽然平衡了局部负载，却常常导致全局失衡。

实用自适应路由的局部性还导致对流量模式变化的响应延迟。在决策点能够感知拥塞之前，决策点与拥塞点之间的队列必须完全填满。具有强硬背压的流控方法（例如使用浅队列）更适合与自适应路由配合使用，因为它能更快地适应远端拥塞。

10.1　自适应路由基础

通过考虑一个简单的 8 节点环的情况，可以说明自适应路由涉及的许多问题，如图 10.1 所示。节点 5 正在向节点 6 发送连续的数据包流，使用了通道（5，6）上的所有可用带宽。

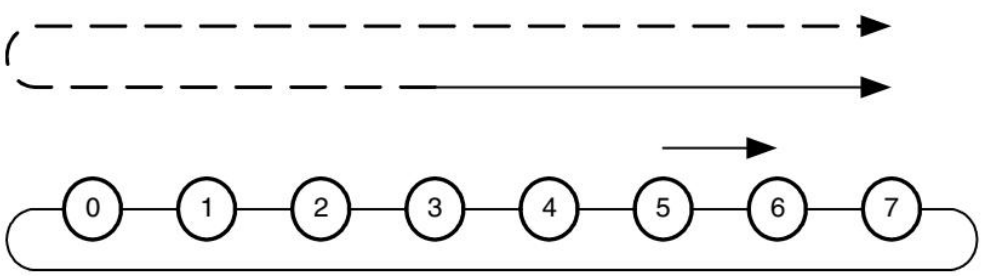


图 10.1　8 节点环上的自适应路由。节点 3 希望向节点 7 发送一个数据包，可以在经过节点 4 到 6 的实线路径和经过节点 2 到 0 的虚线路径之间选择。节点 3 无法得知节点 5 和 6 之间的同时通信。

与此同时，节点 3 希望向节点 7 发送一个数据包。它可以选择顺时针路由（用实线箭头表示）或逆时针路由（用虚线箭头表示）。

对于你——读者（拥有对网络状态的全局、瞬时知识）来说，显然节点 3 的路由器应选择逆时针路由以避免通道 $(5,6)$ 上的争用。然而，节点 3 的典型路由器并不知道通

道 $(5,6)$ 上的争用。该争用影响节点 5 上的队列，在没有其他通信的情况下，并不影响节点 3 上的队列。

自适应路由算法如何感知网络状态？这是理解自适应路由的关键问题。我们可以将这个问题分解为涉及空间和时间的子问题：算法使用局部信息还是全局信息？算法使用当前信息还是历史信息？当然，这些问题的答案并非二元的。相反，在局部信息与全局信息之间以及信息的时效性之间存在一个连续谱。

几乎所有自适应路由器都使用基于 flit 或基于数据包的流控（见第 12 章），并使用当前节点上 flit 或数据包队列的状态来估计本地链路的拥塞。它们没有关于网络其他地方链路状态的直接信息。因此，在图 10.1 中，如果节点 3 向节点 7 发送一个孤立的数据包，其队列不会反映通道（5，6）上的拥塞，因此节点 3 只能随机选择路由。

路由器能够通过背压间接感知网络其他地方的拥塞。当一个节点上的队列填满时，背压信号会阻止前一个节点的传输，从而导致该节点上的队列也被填满。背压沿与流量相反的方向通过网络向后传播。然而，背压仅在存在路由进入拥塞的流量时传播。在没有流量的情况下，背压不会传播，因此没有关于远端拥塞的信息。

例如，在图 10.1 的情况下，节点 4 和 5 上的输入队列必须完全填满，节点 3 才能感知到通道（5，6）上的拥塞。图 10.2 说明了这种情况，其中通道上方的每个点表示该通道目的节点输入缓冲区中的一定数量的数据包。

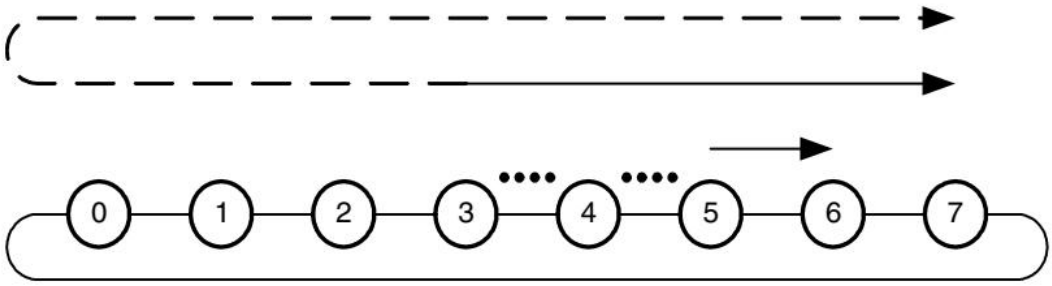


图 10.2 只有在节点 4 和 5 上的输入缓冲区完全填满数据包（如图中圆点所示）后，节点 3 才能感知到通道（5，6）上的拥塞。

这个例子说明了为什么自适应路由在强硬流控下表现更好。1 假设每个输入队列只能容纳 F = 4 个数据包。那么节点 3 在仅发送 8 个数据包后就会感知到拥塞并开始沿另一方向路由。网络将相对较快地实现负载均衡，只有前 8 个数据包会承受走拥塞路径的较高延迟。另一方面，如果输入队列的容量为 F = 64 个数据包，那么节点 3 检测到拥塞所需的时间将延长 16 倍，并且会有 16 倍多的数据包承受拥塞路径的较高延迟。在轻微的负载失衡下，拥塞信息传播回源节点所需的时间甚至更长。例如，如果一条通道仅过载 10%，那么必须通过拥塞路径发送 10 个数据包，才能使该通道之前的输入缓冲区中积压一个数据包。在这种情况下，源节点感知到拥塞可能需要极长的时间，并且会有许多次优路由的数据包。

图 10.1 的例子还说明了自适应路由的信息时效性问题。假设就在节点 3 感知到通道 $(5,6)$ 上的拥塞时，从 5 到 6 的流量停止，并被从 1 到 0 的数据包流取代。那么节点 3

会错误地开始将数据包路由到新拥塞的通道 $(1, 0)$。问题在于节点 3 是基于网络状态的历史信息采取行动。它在 HF 个数据包之前感知到通道 $(5, 6)$ 的状态，其中 H = 2 是到拥塞通道源节点的跳数，F 是输入缓冲区的容量。

在比环更复杂的拓扑中，自适应路由决策在每一步都做出，而不仅仅是在源节点。然而，拥塞信息的局部性仍然导致次优路由，如图 10.3 所示。该图展示了一个好的局部决策如何导致坏的全局路由。在这种情况下，一个数据包从 s = 00 传输到 d = 23。所采用的路径用灰色高亮显示。初始跳是向北到节点 01。在节点 01，向北的链路略有拥塞（拥塞程度用线条的粗细表示），因此数据包接下来向东移动到 11。在这次向东移动之后，所有向北的路径都高度拥塞，因此数据包被迫经过两条高度拥塞的链路。全局最优路由应该是在 01 处向北，仅经过一条略有拥塞的链路。

10.1 自适应路由基础

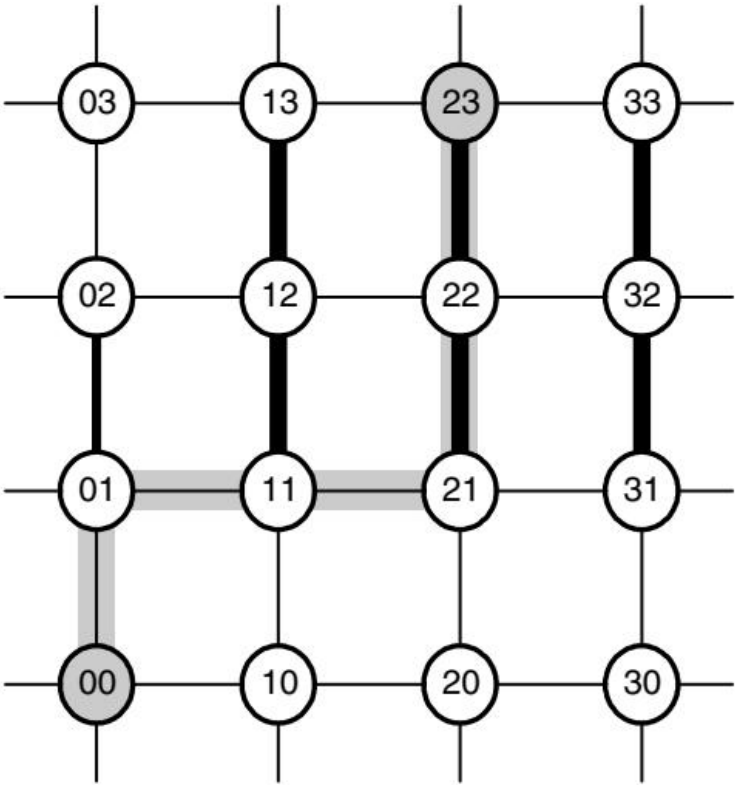


图 10.3 局部最优决策导致全局次优路由。一个数据包从 s = 00 到 d = 23，沿灰色高亮的路由传输。为了避免（01，02）上的轻微拥塞，数据包从 01 路由到 11，之后被迫经过两条高度拥塞的链路。

10.2 最小自适应路由

最小自适应路由算法在从源 s 到目的 d 的最小（最短）路由中进行选择，并在每一跳利用网络状态信息做出路由决策。在每一跳，路由函数生成一个有效输出向量，该向量标识当前节点的哪些输出通道将使当前数据包更接近其目的地。然后利用网络状态（通常是队列长度）选择其中一个有效输出通道作为下一跳。

例如，在图 10.3 的节点 01 处，四个输出通道（+x、-x、+y、-y）中有两个将使数据包更接近目的地，因此有效输出向量为（1，0，1，0）。在两个有效输出中，+x 的拥塞程度低于 +y，因此数据包通过 +x 路由到节点 11。

最小自适应路由擅长局部平衡通道负载，但在全局负载平衡方面表现不佳。图 10.4 中从 00 到 12 的路由说明了自适应路由如何避免局部拥塞。在节点 01 处，+x 和 +y 都是有效方向。路由器选择通道（01，11）以避免拥塞通道（01，02），从而局部平衡节点 01 的输出负载。图 10.3 说明了局部自适应路由无法避免全局拥塞。如上所述，选择经过通道（01，11）而不是轻微拥塞的通道（01，02）的局部决策导致下游出现更严重的拥塞。

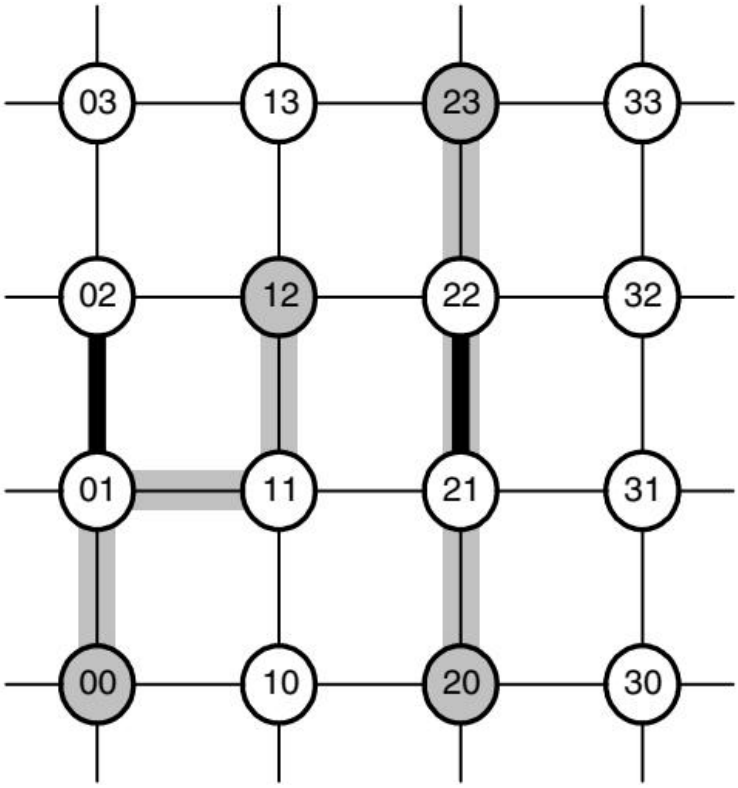


图 10.4　最小自适应路由避免局部拥塞的实例及其局限性

与任何最小路由算法一样，最小自适应路由算法无法为没有最小路径多样性的源-目的对（|Rsd| = 1）避免拥塞。图 10.4 中从 20 到 23 的路由说明了这种情况。每一跳只有一个有效方向 +y，因此数据包无法避开拥塞通道（21，22）。我们将在下面看到非最小自适应路由如何避免此类瓶颈。

虽然本节中的所有示例都涉及环面网络，但最小自适应路由可以应用于任何拓扑。例如，图 9.2 中的折叠 Clos 可以通过自适应向右路由直到遇到 s 和 d 的共同祖先，然后确定性地向左路由到达 d。在这种情况下，在路由的向右阶段所有输出都是有效的，但在向左阶段只有一个输出是有效的。事实上，这正是 Thinking Machines CM-5 的数据网络所使用的路由方法（第 10.6 节）。

10.3 全自适应路由

采用非最小或全自适应路由时，我们不再限制数据包沿最短路径到达目的地。数据包可以被引导到增加与目的地距离的通道上，以避开拥塞或故障通道。

例如，图 10.5 展示了自适应路由如何避免图 10.4 中从 20 到 23 的路由上的拥塞。在节点 21 处，数据包被引导到节点 31，将到目的地的距离从 2 跳增加到 3 跳，以避开拥塞通道（21，22）。将数据包沿这样的非有效通道引导通常称为误路由。

典型的全自适应路由算法优先考虑有效输出，因此在没有拥塞的情况下数据包被路由向目的地，但允许在非有效输出上路由以增加路径多样性。一种可能的算法如下：对于给定的数据包，如果存在队列长度小于某个阈值的有效输出，则将数据包路由到队列长度最短的有效输出。否则，将数据包路由到队列长度最短的输出，无论是否有效。一些算法限制第二步，以避免选择将数据包送回其刚到达节点的通道（无掉头），假设经过一个通道然后反向经过该通道显然是适得其反的。

虽然全自适应路由提供了额外的路径多样性以避开拥塞，但除非采取措施保证进展，否则可能导致活锁（见第 14.5 节）。当数据包在网络中无限期传输而从未到达目的地时，就会发生活锁。在全自适应路由中，如果数据包至少一半时间被误路由到非有效通道上，就可能发生这种情况。图 10.6 说明了这种活锁的示例。一个从 00 到 03 的数据包在 02 处遇到拥塞，被误路由到 12，在那里遇到更多拥塞，再次被误路由到 11。这开始了一个循环：数据包前进两步（从 11 到 02），然后后退两步（从 02 到 11）。

为了避免活锁，全自适应路由算法必须包含某种机制来保证随时间推进。一种方法是仅允许固定次数的误路由。例如，在误路由 M 次后，算法恢复为最小自适应路由。这保证如果数据包从距离目的地 H 跳开始，它将在经过至多 H+2M 个通道后被送达。另一种替代方案是允许数据包每 H’ > 1 个有效跳误路由一跳。因为这种方法需要 H’ + 1 跳才能将到目的地的距离减少 H’ - 1，所以保证在 H（（H’+1）/（H’-1））跳后送达数据包。另一种方法（练习 10.3 采用的混沌路由）不限制送达数据包所需的跳数，而是通过概率论证说明数据包最终会被送达。

除了可能引起活锁外，全自适应路由算法还提出了导致死锁的新可能性。我们将这些问题的讨论推迟到第 14 章。

10.4 负载均衡自适应路由

自适应路由算法很难在互连网络的信道上实现全局负载均衡，因为它们通常基于完全局部的信息做出路由决策。克服这个问题的一种方法是采用混合路由算法，其中使用第 9.3 节的方法无关地选择要路由到的象限。然后在该象限内使用无回溯的自适应路由将数据包传递到其目的地。象限的无关选择在全球范围内平衡负载，而所选象限内的自适应路由执行局部平衡。

这种混合算法实现了非常好的负载均衡，因此具有非常好的最坏情况性能。不幸的是，它在本地流量模式上的性能不如纯自适应算法（最小或全自适应），因为与第 9.3 节的

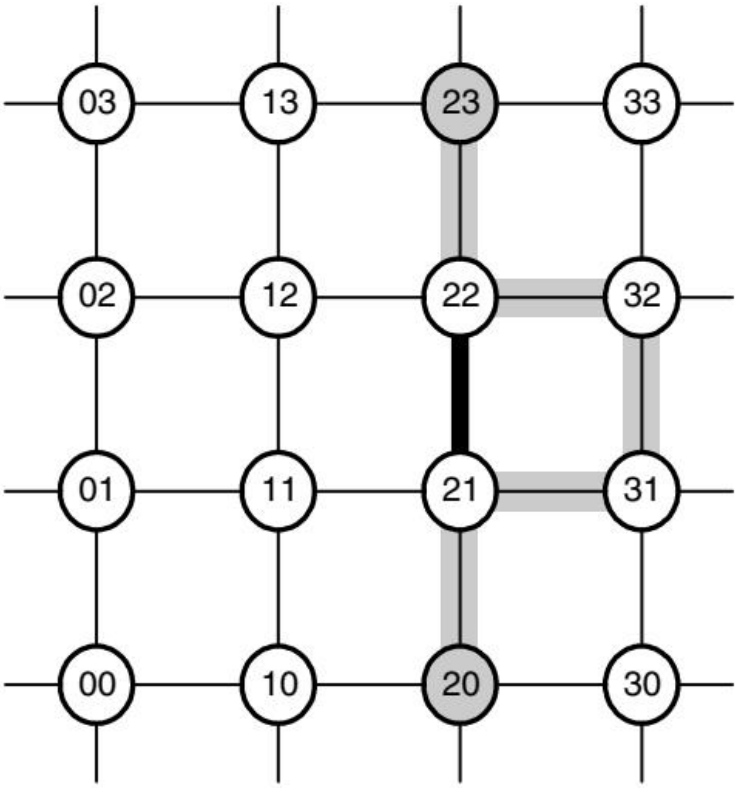


图 10.5 全自适应路由可能误路由数据包以避免拥塞。在节点 21，一个目的地为 23 的数据包被误路由到 31，以避免链路（21，22）上的拥塞。

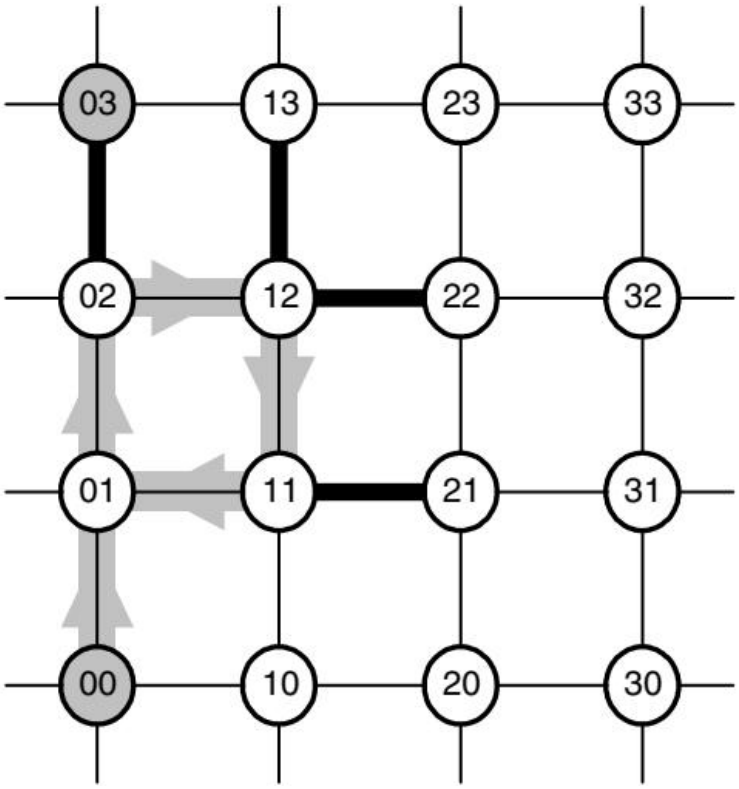


图 10.6 完全自适应路由中可能出现的活锁示例

无关算法一样，它让一些数据包绕远路通过网络。尽管这种路由算法不是最小的，并且一些数据包走了远路，但数据包总是朝着目的地前进。一旦选择了路由象限，到达目的地所需的跳数 H 就确定了，数据包总是恰好经过 H 跳传递。因此，负载均衡自适应路由不存在活锁问题。

10.5 基于搜索的路由

到目前为止，我们将注意力限制在既贪婪又保守的路由策略上。它们之所以贪婪，是因为它们不回溯。一旦占用了信道，就会一直保持。它们之所以保守，是因为它们只沿一条路径发送数据包，而不是同时在多条路径上广播。

非贪婪路由的一种方法是将路由问题视为搜索问题。指示数据包搜索到目的地的最佳路径。这可能涉及数据包在发现路径被阻塞或拥塞时进行回溯，或者沿多条路径广播头部，然后通过这些路径中的最佳路径传输数据。

由于既慢又大量占用资源，这种基于搜索的路由算法在实践中很少用于实际路由数据包。然而，它们可用于离线查找网络中的路径以构建路由表。

10.6 案例研究：Thinking Machines CM-5 中的自适应路由

图 10.7 展示了 Thinking Machines Connection-Machine CM-5 的照片 [114，199]。这台机器是制造的最后一台连接机，也是 Thinking Machines 制造的第一台（也是唯一一台）多指令多数据（MIMD）机器。早期的 CM-1 和 CM-2 是位串行、单指令多数据（SIMD）并行计算机。CM-5 最多由 16K 个处理节点组成，每个节点包含一个 32-MHz 的 SPARC 处理器和一个 4 宽度的向量单元。该机器包括三个独立的互连网络：数据网络、控制网络和诊断网络。从许多角度来看，CM-5 都是一台有趣的机器，包括在电影《侏罗纪公园》中的客串亮相；然而，我们将重点关注其数据网络。

如图 10.8 所示，CM-5 数据网络采用折叠 Clos 拓扑，与处理器进行双工连接，并在前两级交换机中采用 2：1 集中。图中每个信道在每个方向上为 20 Mbytes/s（4 位宽，40 MHz），并采用差分信号。2 图中的每个交换机是采用 1 μm CMOS 标准单元技术实现的单芯片 8×8 字节宽路由器。3 连接网络前两级的信道使用背板实现。更高级别的信道使用长度为 9 英尺或 26 英尺的电缆实现。

10.6 案例研究：Thinking Machines CM-5 中的自适应路由

每个处理节点通过一对信道连接到两个独立的交换机，从而提供每节点 40 Mbytes/s 的总接口带宽。这种双工连接使网络具有单点容错能力。如果连接到处理节点的单个路由器发生故障，处理节点可以通过其第二信道继续发送和接收消息。每个处理器通过内存映射接口将消息注入网络。消息最多可包含 5 个 32 位字的数据。4 连接到一组 4 个处理元件的两个一级交换机在逻辑上充当单个节点，并共同连接到 4 个不同的二级交换机中的每一个。类似地，图中的 4 个二级交换机共同连接到 8 个不同的三级交

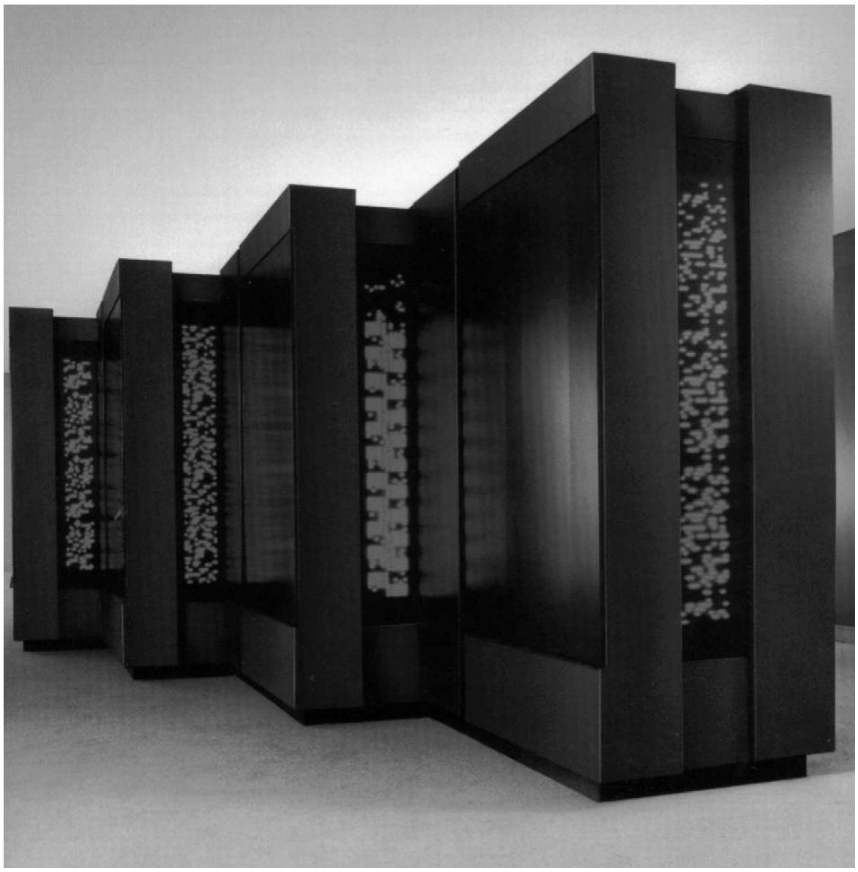


图 10.7 一台 Thinking Machine CM-5 最多包含 16K 个处理节点。每个节点集成了一个 32-MHz 的 SPARC 处理器和一个向量浮点单元。节点通过折叠 Clos（胖树）网络连接。

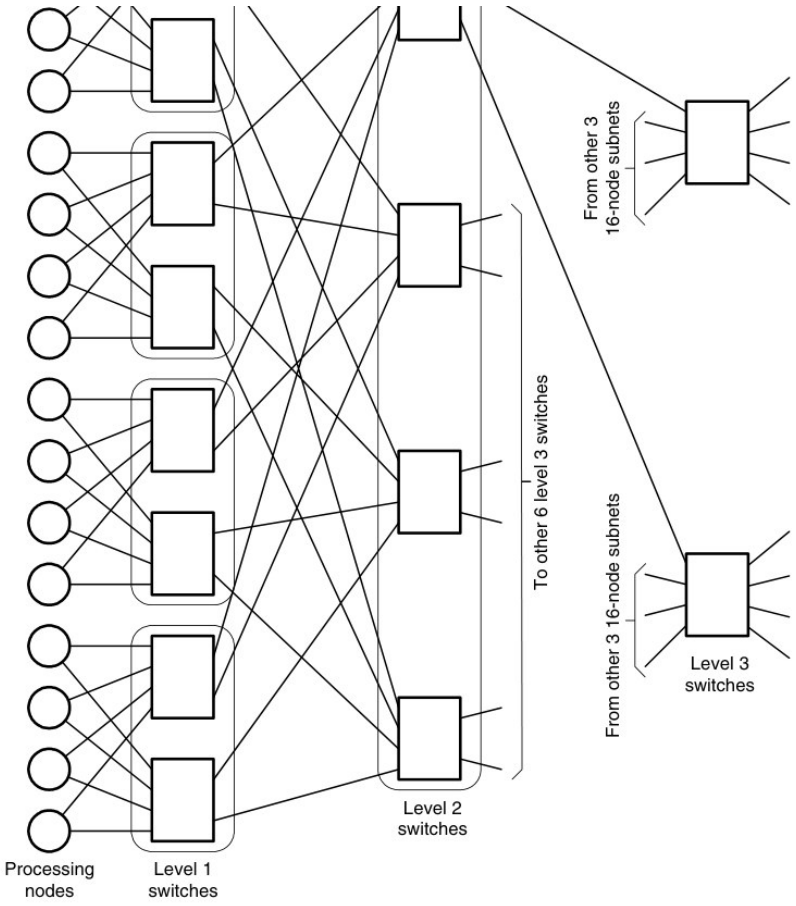
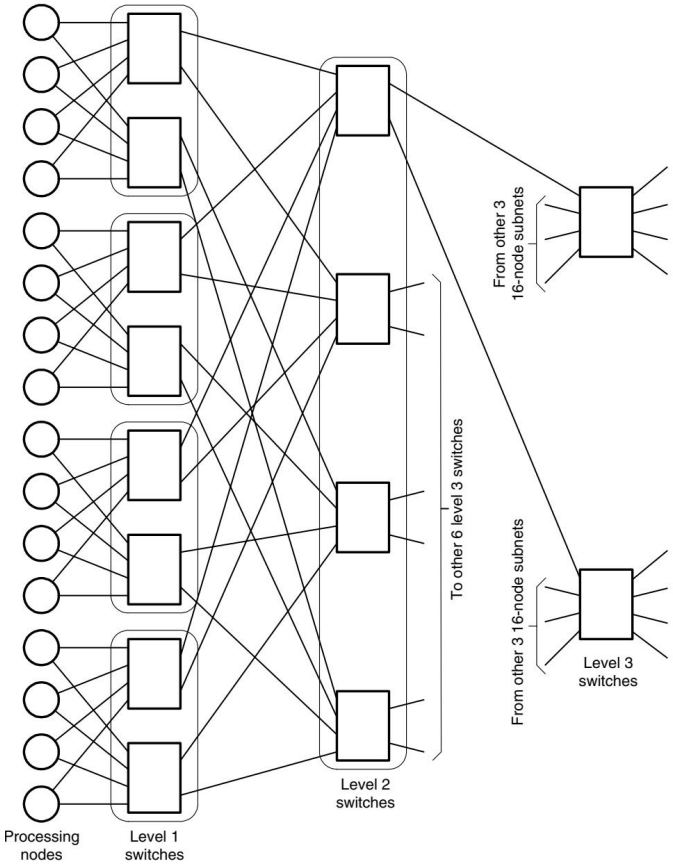


图 10.8 CM-5 使用 8×8 交换机组成的折叠式 Clos（胖树）网络



CM-5 拓扑是由 8×8 交换机组成的折叠 Clos（胖树）网络，每个交换机有 4 个上游（右）端口和 4 个下游（左）端口。每个处理节点连接到 2 个第一级交换机。为了提供净 4：1 的集中，前 2 级交换机仅使用 4 个上游端口中的 2 个。从第三级交换机开始，所有 4 个上游端口都被使用。

换机中的每一个（仅显示了其中 2 个）。拓扑结构的设计使得第 i 级的交换机仅通过向下游（向左）发送消息即可访问 4^i 个节点。

CM-5 按照第 9.2.1 节所述的方式路由消息，不同之处在于其上游路由是自适应的，而非 oblivious 的。5 从节点 s 到节点 d 的消息分两个阶段路由。消息首先被向上游（向右）路由，直到到达一个作为 s 和 d 共同祖先的交换机。这种上游路由是自适应的，消息在空闲的上游链路中随机选择。到达共同祖先之后，消息使用目的标签路由，沿着到 d 的唯一路径确定性地向下游（向左）路由。

CM-5 中的消息具有图 10.9 所示的格式。消息被划分为 4 位 flit。只要发送节点有信用，每个周期通过 4 位数据通道传送一个 flit。消息的第一个 flit 是高度 flit，它指定此消息的高度 h：消息必须传播到网络中多高（多靠右）才能到达 s 和 d 的共同祖先。高度 flit 之后是 $\lceil h/2 \rceil$ 个路由 flit，每个包含两个 2 位的路由字段。每个路由字段指定下游路由的一步。消息中其余的 flit 与有效载荷有关，与路由无关。

上游路由阶段由消息头的高度字段 h 控制。当上游消息进入每个路由器时，将 h 与该路由器的层级 l 进行比较。如果 l < h，则上游阶段继续，随机选择一条空闲的上游链路来转发消息。如果所有链路都忙，则消息被阻塞，直到某条链路变为空闲。当消息到达 l = h 的路由器时，就找到了 s 和 d 的共同祖先，下游路由阶段开始。

在下游路由的每一步，消耗前导路由 flit r 的一个路由字段，然后高度 h 递减。每一跳递减 h 有两个有用的目的。首先，这保持了不变式：头部 flit 之后始终有 $\lceil h/2 \rceil$ 个路由 flit，因为我们在消耗路由 flit 的同时递减 h。其次，h 的最低有效位用于选择 r 中的哪个路由字段来进行输出端口选择。如果 h 为偶数，则使用 r 的左侧路由字段来选择下游输出端口，并递减 h。如果 h 为奇数，则使用 r 的右侧路由字段来选择下游端口，递减 h，并丢弃 r。在下一跳，h 再次为偶数，路由继续使用下一个路由 flit 的左侧路由字段。当消息到达目的地时，h = 0，并且所有路由 flit 都已消耗完毕。

上游路由的自适应性受 CM-5 所采用的 flit 级阻塞流控（第 13.2 节）支配。为了调节通道上的流量，CM-5 路由器采用了一种开关流控的变体（第 13.3 节）。当输入端口缓冲区有空间时，接收路由器向发送路由器发送一个令牌。发送方可以立即使用此令牌发送一个 flit，但不能存储令牌（如基于信用的流控中那样）。如果没有要发送的 flit，令牌就会过期。当缓冲区满时，不发送令牌，流量被阻塞。每个 CM-5 输出端口都有一个足够大的缓冲区，可以容纳一条 5 个字的消息（后来版本中为 18 个字的消息）。回想一下，在上游路由阶段，数据包被随机分配到一个空闲的上游输出端口。如果一个上游端口的输出缓冲区为空，则该端口被视为空闲，有资格分配给新消息。如果没有空闲的上游端口，数据包就在原地被阻塞——使其输入缓冲区保持忙，进而阻塞更下游的数据包。由于路由器会等到输出缓冲区可以接受整个消息后才将消息分配到该输出端口，因此消息永远不会在路由器的交换机处被阻塞。

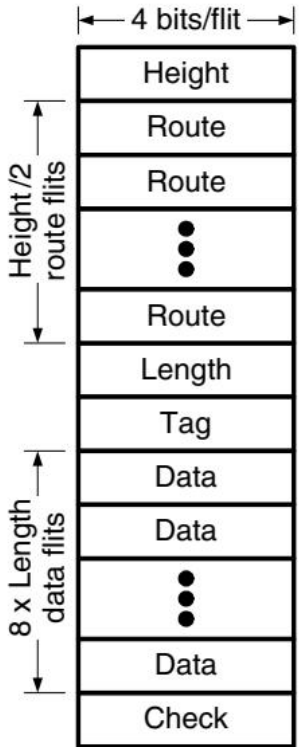


图 10.9 CM-5 消息的格式。路由由高度字段和下行路由字段控制。4 位高度字段指示消息应向上游传播多远才能到达目的地的祖先。下行路由字段是目的地址的后缀，指定从该祖先到目的节点的路径——每层两位。

10.7 文献注释

自适应路由的许多发展都与避免死锁和活锁所需的流控机制密切相关。早期的自适应路由算法包括 Linder 和 Harden [118]、Chien [36]、Aoki 和 Dally [48] 以及 Allen 等人 [8] 的算法。Duato 的协议 [61] 促成了一系列自适应路由算法，包括 Cray T3E [162] 中使用的算法。Chaos 路由（练习 10.3）由 Konstantinidou 和 Snyder [104] 提出，并由 Bolding [26] 进一步探索。胖树上的最小自适应路由用于 CM-5，由 Leiserson [114] 描述。Boppana 和 Chalasani [27] 对几种路由方法进行了比较，并表明在某些指标上，实用的自适应算法可能被确定性算法击败。

10.8 练习

10.1 网格中最小自适应路由的优势。在 4×4 网格网络中，找到一个置换流量模式，使得最小自适应路由（第 10.2 节）优于最小 oblivious 路由（第 9.2 节）。计算两种算法在稳态下的 γ_{max}。（假设已有足够的时间让背压信息在网络中传播。）

10.2 比较最小自适应路由和负载均衡自适应路由。找到一个置换流量模式，使得负载均衡自适应路由（第 10.4 节）优于最小自适应路由（第 10.2 节），并找到另一个流量模式，使得最小自适应路由优于负载均衡自适应路由。

10.3 Chaos 路由的无活锁特性。Chaos 路由是一种偏转路由方案。如果多个数据包争

用同一通道，路由器会随机将该通道授予其中一个争用的数据包。任何在此分配中失败的数据包都会被错误路由到任意空闲输出端口。（此端口可能是非生产性的。）由于路由器具有相同数量的输入和输出端口，因此总是可以将传入数据包匹配到某个输出。通过展示数据包在 T 个周期内未到达目的地的概率随着 T 的增加而趋近于零，解释为什么这种方法在概率上是无活锁的。

10.4 胖树中的自适应与 oblivious 路由。考虑一个由 8×8 交叉开关构建的 256 节点折叠 Clos（胖树）网络，该网络使用丢弃流控。哪种算法的丢弃概率更低？两种丢弃概率如何随流量模式变化？

10.5 CM-5 网络中的最坏情况流量。为 CM-5 网络的随机 oblivious 路由算法（第 9.2.1 节）找到一个最坏情况流量模式。比较自适应路由和 oblivious 路由在此流量模式下的吞吐量。

10.6 模拟。在 8 元 2 立方体网络中，探索自适应路由的缓冲区深度与响应时间之间的权衡。每隔 T 个周期在两个流量置换之间交替，并绘制平均数据包延迟随时间变化的曲线。每个节点的缓冲量如何影响该曲线的形状？

本页故意留空

注释

- 1. 在第 12 章中，我们将看到如何使用虚拟通道以较浅的队列深度提供高性能——每个队列仅需容纳足够的 flit 来覆盖信用环路的往返延迟。
- 2. 除了每个方向的 4 个数据位外，每个方向还有一个流控制位，用于在数据包被阻塞时施加背压。
- 3. 每侧有 4 个连接，乍看之下图 10.8 中的交换机像是 4×4 交换机。然而，重要的是要记住，与每条链路都是单向的蝶形网络不同，图 10.8 中交换机的每个连接都是双向的。因此，这些交换机中的每一个都是具有 8 个输入和 8 个输出的 8×8 交叉开关。
- 4. 这种最多 5 个字的消息长度被认为是一个限制，后来版本的机器允许最多 18 个字的消息。
- 5. 我们在练习 10.4 和 10.5 中探讨这两种方法的性能差异。

第十一章　路由机制

术语“路由机制”指的是用于实现任何路由算法（确定性、oblivious 或自适应）的机制。许多路由器在源端或沿路径的每一跳使用路由表来实现路由算法。每个目的地只有一个表项时，表仅限于确定性路由，但通过为每个目的地提供多个表项，可以实现 oblivious 和自适应路由。表的替代方案是算法路由，其中专用硬件在运行时计算数据包的路由或下一跳。然而，算法路由通常仅限于简单的路由算法和规则拓扑。

11.1　基于表的路由

回顾第 8.3 节中的关系 8.1 至 8.3，它们描述了任何路由算法

$$R : N \times N \mapsto \mathcal{P}(P)$$
$$R : N \times N \mapsto \mathcal{P}(C)$$
$$R : C \times N \mapsto \mathcal{P}(C).$$

路由关系的三种形式中的任何一种都可以使用表来实现。每对输入的关系值存储在表中，表由输入索引。例如，对于路由关系的第一种形式，每对节点的路径集合存储在表中，表由源节点和目的节点索引。特定节点上只需要存储该节点所需的那部分表。例如，在节点 x 处，只需要存储源节点（或当前节点）x 的那部分表。

基于表的路由的主要优点是通用性。在容量限制下，路由表可以支持任何拓扑上的任何路由关系。使用基于表的路由的路由芯片可以通过简单地重新编程表的内容而用于不同的拓扑。

在本节的剩余部分，我们将探讨基于表的路由的两个极端。在源表路由中，我们通过在源端查找整个路由来直接实现一次性路由关系。在节点表路由中，我们通过沿路径的每个节点查找逐跳路由关系来执行增量路由。当然，存在一个连续的范围，其中每次表查找返回路径的一部分。

11.1.1　源路由

在源路由中，数据包的所有路由决策完全在源终端通过查找预先计算的路由表来完成。每个源节点包含一个路由表，至少每个目的地一个。为了路由数据包，使用数据包目的地索引表以查找适当的路径或路径集。如果返回一个集合，则使用附加状态以 oblivious 或自适应方式从集合中选择一条路径。然后，这条路径被添加到数据包的前面，并用于沿着所选路径快速引导数据包通过网络，无需进一步计算。由于其速度、简单性和可扩展性，源路由是确定性和 oblivious 路由最广泛使用的路由方法之一。源表路由通常不用于实现自适应路由，因为它不提供利用路径中间跳的网络状态信息的

机制。

表 11.1 显示了图 11.1 的 4×2 环面网络中节点 00 的源路由表。该表包含网络中每个目的地的两条路径。表中的每条路径指定了从该源到特定目的地的路径，作为端口选择器的字符串，沿路径的每个通道一个。本例中的端口选择器来自一个五符号字母表（NEWSX），编码为 3 位二进制数。前四个符号选择路由器的四个输出端口之一（按罗盘方向），最后一个符号（X）指定路由器的出口端口。

表 11.1 四乘二环面网络节点 00 的源路由表

目的端	Route 0	Route 1
00	X	X
10	EX	WWWX
20	EEX	WWX
30	WX	EEEX
01	NX	SX
11	NEX	ENX
21	NEEX	WWNX
31	NWX	WNX

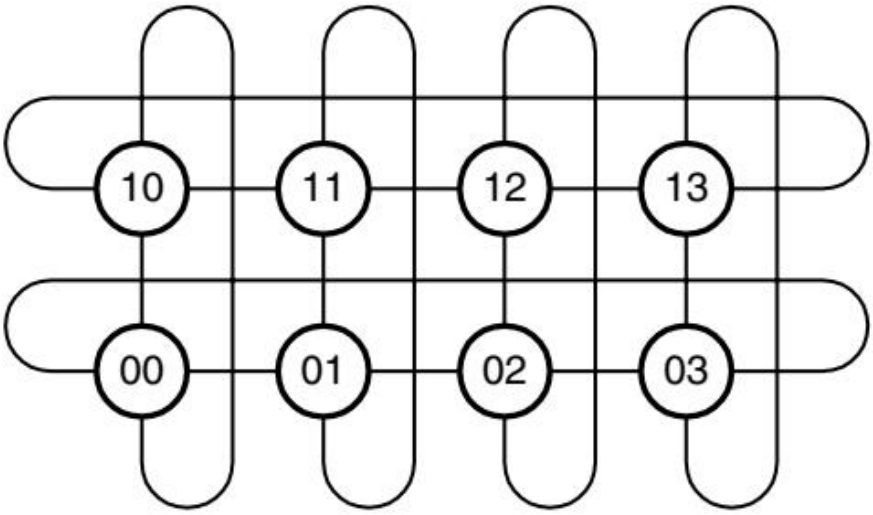


图 11.1 表 11.1 的源路由示例网络。

例如，考虑将数据包从节点 00 路由到图 11.1 网络中的节点 21。在源端，路由表（表 11.1）用数据包的目的地 21 索引，以确定预先计算的路由。此查找返回两条路径：NEEX 和 WWNX，源端任意选择第一条路径 NEEX。然后，这个路由字符串被逐个符号地消化，以引导数据包通过网络。第一个字符 N 选择节点 00 处路由器的北输出端口，并被剥离，将字符串的剩余部分 EEX 传递到节点 01。在节点 01，选择东端口，并将 EX 传递到节点 11。在节点 11 再次选择东端口，并将最后一个字符 X 传递到节点 21。在节点 21，X 符号选择路由器的出口端口，将数据包引导到节点 21 的输入队列中。

与本章其余部分描述的许多替代方案相比，源路由具有几个优点。最重要的优点是速度。在源端进行初始表查找和路径选择后，不再花费时间在路由上。当每个数据包到达路由器时，它可以立即选择其输出端口，无需任何计算或内存引用。每跳延迟中的路由延迟（见第 16 章）分量为零。除了速度快之外，源路由还导致简单的路由器设计。每个路由器中不需要任何路由逻辑或表。

源路由与拓扑无关。它可以在任何强连通拓扑中路由数据包，仅受路由器端口数量、源表大小和路由最大长度的限制。例如，使用图 11.1 网络中的四端口路由器，可以连接一个任意度的四端口网络，并使用源路由在其中路由数据包。

使用源路由的路由器可用于任意大小的网络，因为对网络大小、表大小和路由长度的唯一限制由源端决定。这是有利的，因为它允许单一的路由器设计用于任意大小的网络，而不会让小网络承担与最大网络成比例的成本。支持大型网络的开销被转移到终端，而在小型网络中无需提供。11.1 基于表的路由

206 CHAPTER 11 *Routing Mechanics*

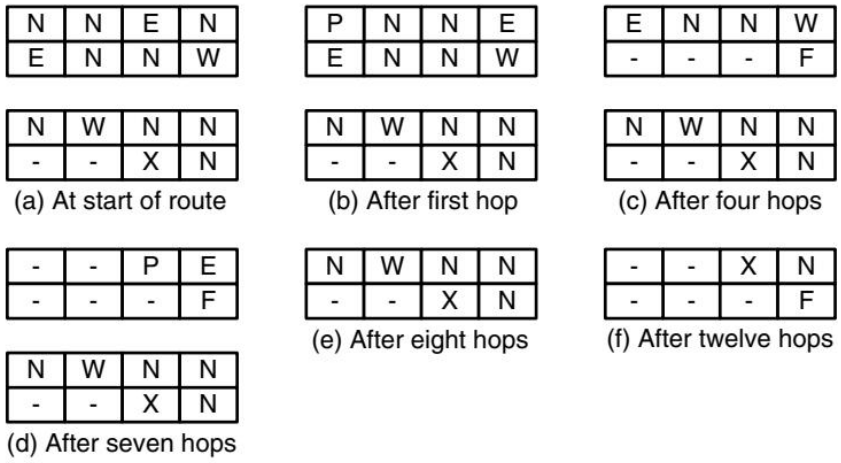


图 11.2 源路由的任意长度编码。

为了使路由器提供任意可扩展性，它必须能够处理任意长度的路由。虽然路由按定义是以出口端口选择符 X 结尾的任意长度字符串，但需要一种编码来将这些任意长度字符串打包成固定长度的路由 phit 和 flit。

图 11.2 展示了一种编码任意长度源路由的方法。此示例假设路由器具有 16 位 phit、32 位 flit 和最多 13 个路由器端口。每个 16 位 phit 中打包四个 4 位端口选择符，每个 flit 中有 8 个选择符。为了方便打包，我们引入两个新的端口选择符号 P 和 F，分别表示 phit 继续和 flit 继续。

图 11.2（a）展示了一个 13 跳路由（NENNWNNNENNWNN）编码到两个 flit 中。在每个 phit 中，端口选择符从右到左解释。原始路由中没有继续选择符。这些在路由的后续步骤中被移入，如图 11.2（b–f）所示。在走完第一跳向北后，前导 flit 右移四位以丢弃第一个端口选择符，并将 phit 继续选择符 P 移入最高有效位，如图 11.2（b）所

示。四跳后，路由器遇到一个最右侧端口选择符为 P 的 phit。这个 P 指示路由器剥离前导 phit，并将 flit 继续 phit（F）附加到当前 flit 的末尾（图 11.2[c]）。八跳后，路由器遇到 flit 继续端口选择符，指示它丢弃当前 flit 并从下一个 flit 开始继续处理端口选择符（图 11.2[e]）。以这种方式跨多个 phit 和 flit 继续处理，直到遇到出口端口选择符 X。通过继续跨越任意数量的 flit 和 phit，可以以这种方式编码任意长度的路由。

使用继续端口选择符使我们能够处理任意长度的路由，同时一次只处理一个 phit。因为每个阶段的前导 phit 输出仅取决于该阶段的前导 phit 输入，所以它可以在第二个 phit 到达之前生成并输出。

为了处理任意长度的路由，源终端中的路由表必须设计为高效存储任意长度字符串。这可以通过使用单级间接寻址来实现，如图 11.3 所示。用目的地索引表返回一个或多个指针，每个指针指向一个可变长度路由字符串。对于图中所示的示例，目的地选择表的第四行，该行包含两个指针，分别对应两条路由。第二个指针定位路由 NENEX 的起始位置。注意所有路由都以 X 符号终止。作为另一条路由后缀的路由可以由指向第一条路由字符串的指针表示，无需额外存储。例如，表第二行的第一条路由定位字符串 NEX，它是第四行第二条路由的后缀。这种组织方式需要额外的一次内存访问来查找路由。然而，对于除最短路由外的所有路由，它更节省存储空间，因为它避免了将所有路由向上舍入到最大长度相关的存储浪费。一种折衷方案是将路由的前几跳存储在表本身中，并附带指向路由其余部分的指针。这种组织方式避免了短路由的额外内存引用，同时保留了长路由间接寻址的效率。

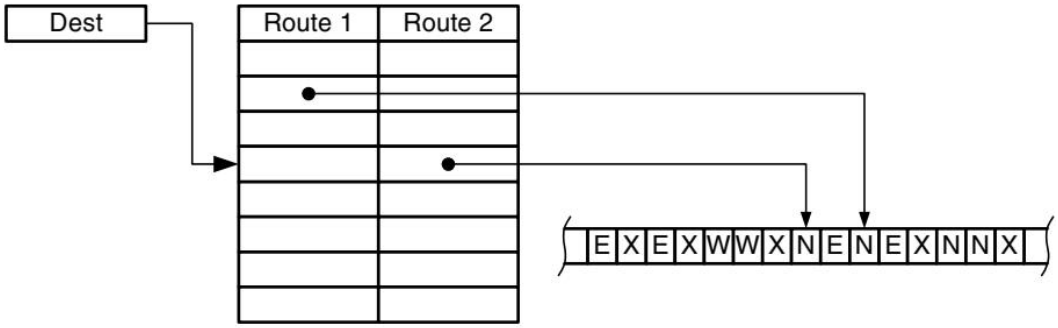


图 11.3 采用单级间接寻址组织的路由表，以允许高效编码可变长度源路由。目的地索引一个指针表，指针指向连接源路由字符串。上面的指针指向的源路由是下面指针索引的源路由的后缀。

源路由表可以为每个目的地关联多条路由。例如，表 11.1 为每个目的地存储两条路由。如上文关于 oblivious 和自适应路由所述，在给定源和目的地之间提供多条路由有几个优点：

容错：如果某些路由是边不相交的，即使节点之间的一个或多个通道发生故障，两个节点之间的连接也能保持。

负载均衡：这对节点之间的流量分布在多条网络路径上，在广泛的流量模式下平衡网络中的负载。

分发：当用于索引表的目的地是逻辑目的地（例如服务而非特定服务器）时，多条路由实际上可能通向多个目的地。在这种情况下，路由表用于在提供特定服务的服务器

之间分发流量。

当路由表为每个目的地存储多条路由时，需要一种路由选择方法来选择每个数据包要遵循的路由。当数据包顺序不是要求时，使用伪随机数生成器可以很好地将流量随机分布到各条路由上。如果路由和表条目之间存在一一对应关系，并且随机数均匀分布，则每条路由被选中的可能性相同，流量将在各路由之间平衡。通常，不同目的地具有不同数量的路由，而表存储固定数量的路由，因此通常不可能安排一一对应。对于任意数量的路由，通过使用非常大量的表条目，可以尽可能接近地近似负载均衡。如果有 M 条路由和 N 个表条目，则每条路由存储在 k 或 $k+1$ 个条目中，其中 $k = \lfloor N/M \rfloor$，两条路由之间的负载差异以 $\frac{1}{N}$ 为界。

另一种方法是将路由概率直接编码为与每条路由一起存储的二进制字段。然后，逻辑电路可以将这些概率与随机数结合，以在路由之间进行选择。这种方法允许编码任意概率分布，并且比复制表项在存储效率上要高得多。

在某些情况下，我们需要保持通过网络传输的某些数据包的顺序。如果协议无法在组装过程中重新排序数据包，则可能需要保持消息中数据包的顺序。此外，某些协议——例如缓存一致性协议——依赖于保持消息顺序以确保正确性。必须保持顺序的一组数据包称为流，流中的每个数据包通常由流标识符标记。这可能是数据包头中标识协议和数据包类型的字段的组合。通过散列流标识符来生成路由选择器，而不是使用随机数，我们可以确保流中的所有数据包遵循相同的路由，从而（假设采用 FIFO 排队）保持顺序。然而，如果流分布不均匀，这种方法可能会使选择特定路由的概率发生偏差。第 9.5 节描述了在 Avici TSR 的随机化、基于表的路由中如何对流进行排序。

11.1.2 节点表路由

基于表的路由也可以通过将路由表存储在路由节点中而不是终端中来实现。节点表路由更适合自适应路由算法，因为它允许使用每跳网络状态信息在路由的每个阶段从多个可能的下一跳中进行选择。使用节点表而不是源表显著减少了保存路由表所需的总存储量，因为每个节点只需要保存到每个目的地的路由的下一跳，而不是整个路由。然而，这种存储的经济性是以显著的性能损失和路由限制为代价的。在这种安排下，当数据包到达路由器时，必须执行表查找才能选择数据包的输出端口。与源路由器一样，在查找过程中，除了目的地之外，还可以使用数据包到达的输入端口来选择表项。在路由的每一步而不是仅在源端执行查找，显著增加了数据包通过路由器的延迟。可扩展性也受到损害，因为每个节点必须包含一个足够大的表，以保存到最大可能网络中所有目的地的下一跳。此外，使用这种方法，不可能为来自两个不同源节点 x 和 y、目的地为同一节点 z、通过同一通道到达中间节点 w 的数据包提供从 w 到 z 的两条不同路由。而使用源路由则很容易做到这一点。

表 11.2 显示了图 11.1 网络的一组可能的节点路由表。每个节点的表显示在节点编号下的两个相邻列中。每个目的地有一行。对于每个节点和目的地，列出了两个可能的下一跳，以允许路由中的一些自适应性。例如，如果一个数据包在节点 01，其目的地

是节点 13，则它可以沿 N 或 E 方向路由。注意，一些下一跳实际上将数据包移得离目的地更远。这些错误路由以粗体显示。

活锁（见第 14.5 节）可能是节点表路由的一个问题。可以将节点表加载为引导数据包进入无休止循环的条目。例如，考虑一个经过节点 00、目的地为节点 11 的数据包。如果条目 $(00 \to 11)$ 为 N，将数据包导向 10，而条目 $(10 \to 11)$ 为 S，则数据包将在 00 和 10 之间循环，永远无法到达目的地。这不是源路由的问题，因为固定长度的源路由意味着数据包在到达目的地之前有有限的跳数。通过仔细构建路由表，可以在节点表路由中避免这种情况。

节点表路由促进了路由的本地自适应。如果一个节点的输出链路变得拥塞或故障，该节点可以在不与其他节点协调的情况下本地重定向流量。如果路由表为每个目的地包含多个条目，如表 11.2 的情况，这可以通过简单地基于拥塞或可用性偏置输出端口的选择来实现。例如，如果表 11.2 中节点 00 的 N 端口变得拥塞，则目的地为节点 10 的数据包将选择 S 作为其下一跳，而不是 N。

表 11.2 图 11.1 中 4×2 环面网络的节点路由表。每个单元格列出两个候选下一跳；X 表示本节点。

目的端	来源节点							
To	00	01	02	03	10	11	12	13
00	XX	WN	WE	EN	SN	SW	SW	SE
01	EN	XX	WS	EW	SW	SN	SW	SW
02	EW	EN	XX	WN	SW	SE	SN	SW
03	WN	EW	EN	XX	SE	SE	SE	SN
10	NS	NW	NW	NE	XX	WN	WE	EN
11	NE	NS	NW	NW	ES	XX	WN	EW
12	NE	NE	NS	NW	EW	EN	XX	WN
13	NW	NE	NE	NS	WS	EW	EN	XX

表 11.3 八元三维立方体网络中节点 444_8 的分层路由表。X 表示目的地址的无关位。

目的地址	**下一跳**	**说明**
100 100 100	X	本节点
100 100 0XX	W	正西方向的节点
100 100 11X	E	正东方向的节点
100 100 101	E	东侧相邻节点
100 0XX XXX	S	南方节点
100 11X XXX	N	北方节点

100 101 XXX	N	北面一行的节点
0XX 0XX 0XX	W	下方、南方和西方的八分区
0XX 0XX 1XX	S	下方、南方和东方（含相等位置）的八分区
0XX 1XX XXX	D	下方与北方（含相等位置）的四分区
11X XXX XXX	U	上方至少两层的半空间
101 XXX XXX	U	紧邻的上一层

在非常大的网络中，节点表可以按层次组织，以进一步降低存储需求。[1]在这种表中，每个附近的节点都有自己的下一跳条目，小群远程节点共享条目，更大的远距离节点群共享条目，依此类推。分层路由的一种方法是使用内容可寻址存储器（CAM）而不是 RAM 来保存每个节点中的路由表。例如，表 11.3 显示了 8 元 3 立方体网络中节点 444_8 的基于 CAM 的路由表。该表的每一行由一个目的地组和该组的下一跳组成。目的地组被编码为 9 位地址（x、y 和 z 各 3 位），可以在一个或多个位中包含“无关”符号（X）。允许“无关”符号的 CAM 通常被称为三态 CAM（TCAM）。

根据目的地中 X 的数量，它可以编码单个节点或一维、二维或三维的连续节点组。例如，表中的第一个条目编码单个节点，即节点 100 100 100（444），即包含此表的节点。下一行编码此节点以西的一维节点数组（440 到 443）。二维节点数组（从 500 到 577）由表的最后一个条目编码，表中的几个条目编码三维节点组。

实际上，基于 CAM 的节点路由表是一种可编程逻辑阵列，它根据目的地址计算选择输出端口的逻辑函数。每个选择给定输出端口（例如 W）的表项，在用于选择该输出的逻辑方程中充当一个乘积项（或蕴含项）。对于 W 输出，方程中有两个蕴含项，分别对应表中的第二行和第八行。因此，通过使用 CAM，可以用对应于该输出函数的最小和之积实现的较少行数替换给定输出的所有表项，从而压缩任意节点路由表。

节点表路由是大多数广域分组路由器采用的方法。这是因为它适合使用分布式算法来计算网络连通性和路由。然而，在互连网络中，节点位于同一地点，节点表路由通常不如源路由，因为延迟增加，除非需要自适应。

源路由和节点表路由代表了基于表的路由器空间中的两个极端。节点表路由器仅在表中存储路由的下一跳，而源路由器存储整个路由。当然，我们可以在这两个极端之间构造任意数量的点，其中表存储路由的下 $M > 1$ 跳。这些中间解决方案提供了局部自适应性和两个极端之间的跳延迟。然而，迄今为止它们尚未在实践中使用，因为它们缺乏对称性——采用混合方法时，分组必须在不同节点执行不同的操作。

11.2 算法路由

我们可以使用算法来计算路由关系，而不是将其存储在表中。为了提高速度，该算法通常实现为组合逻辑电路。这种算法路由牺牲了基于表的路由的通用性。该算法特定于一种拓扑和该拓扑上的一种路由策略。然而，算法路由在面积和速度方面通常比基

[1]例如 SGI Spider 芯片采用了这种方法，参见文献 [69] 和习题 11.4。

于表的路由更高效。算法路由电路可以简单到用于目的标签路由的数字选择器（第 8.4.1 节），它提取目的地址的一位数字。例如，第 2.5 节的简单路由器就采用了这种电路。

图 11.4 显示了用于二维环面路由的算法路由电路。该电路接受一个路由头，包括每个维度的符号位（sx 和 sy）以及相对地址（x 和 y），这些地址指定了在符号位指定的方向上每个维度到目的地的跳数。相对地址输入到零检查器，生成信号 xdone 和 ydone。如果特定维度中的相对地址为零，则分组在该维度上的路由完成。然后，符号位和完成信号输入到一个由五个与门组成的阵列，这些与门确定哪些方向是有效的，即它们将使分组更接近目的地。例如，左起第二个门确定如果 xdone = 0（表示我们在 x 维度上的路由尚未完成）且 sx = 0（表示我们正在 +x 方向上行进），则 +x 方向是有效的。5 位有效方向向量为每个使分组向目的地前进的方向设置一个位。该向量输入到路由选择块，该块选择其中一个方向进行路由，并输出选定的方向向量。

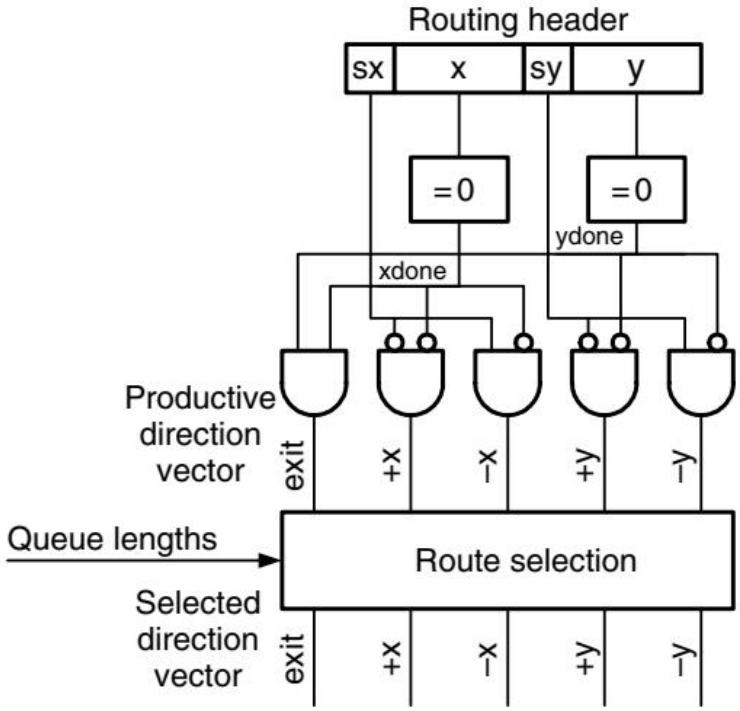


图 11.4 算法路由逻辑示例。该电路从分组路由头接受方向位 sx 和 sy 以及距离字段 x 和 y，并生成有效方向向量（指示哪些通道使分组向目的地前进）和选定方向向量（一个独热向量，选择其中一个有效方向进行路由）。

路由选择块决定该电路执行的路由类型。最小 oblivious 路由器可以通过随机选择有效方向向量中的一个活动位作为选定方向来实现。将这种随机选择替换为基于各自输出队列长度的选择，就得到了最小自适应路由器。如果所有有效方向的队列长度都超过某个阈值，允许选择选择一个非有效方向，就得到了全自适应路由器。

11.3 案例研究：IBM Vulcan 网络中的 Oblivious 源路由

IBM SP1 和 SP2 系列消息传递并行计算机基于采用 oblivious 源路由的多级互连网络。图 11.5 显示了 512 处理器的 IBM SP2 的照片。图中的每个机柜或框架容纳 16 台 IBM RS6000 工作站（处理节点）和一个 32 端口交换机模块。

SP1 和 SP2 都使用基于 8×8 交换机芯片的互连网络，通过源路由进行控制。交换机上的八个双向端口每个方向宽 8 位，工作频率为 50 MHz。除了每个方向上的八条数据线外，还有两条控制线：tag 和 token。tag 线标识有效的 flit，而 token 线沿相反方向运行，将信用（用于流控制）返回给发送节点。通过交换机的延迟为五个 20 ns 周期。

图 11.5 512 处理器的 IBM SP2 系统。每个机柜或框架容纳 16 台 RS6000 工作站和一个 32 端口交换机板。

八个交换机芯片以两级双向网络的形式布置在 32 端口交换机模块上，如图 11.6 所示。2 该模块在两侧各提供 16 个双向通道。从交换机模块传输出去的所有通道均以差分 ECL 信号形式承载。在 SP 机器的底层，16 个处理器模块连接到交换机模块一侧的 16 个端口。这 17 个模块一起封装在一个机柜或框架中。

规模超过 16 个处理器的 SP 机由机架和交换模块组成。小型 SP 机最多可支持 80 个处理器（5 个机架），通过将机架直接互连来构建，如图 11.7 所示，展示了 3 机架和 5 机架的配置。更大的机器则将机架作为折叠 Clos 网络的前 2 级，并使用交换模块提供 16×16 的中间级和/或 32 端口的最终级。例如，图 11.8 展示了如何用 32 个机架和 16 个额外的交换模块构建一台 512 处理器的机器。使用两级交换模块可以构建最多 8K 处理器的机器。11.3 IBM Vulcan 网络中的无关源路由

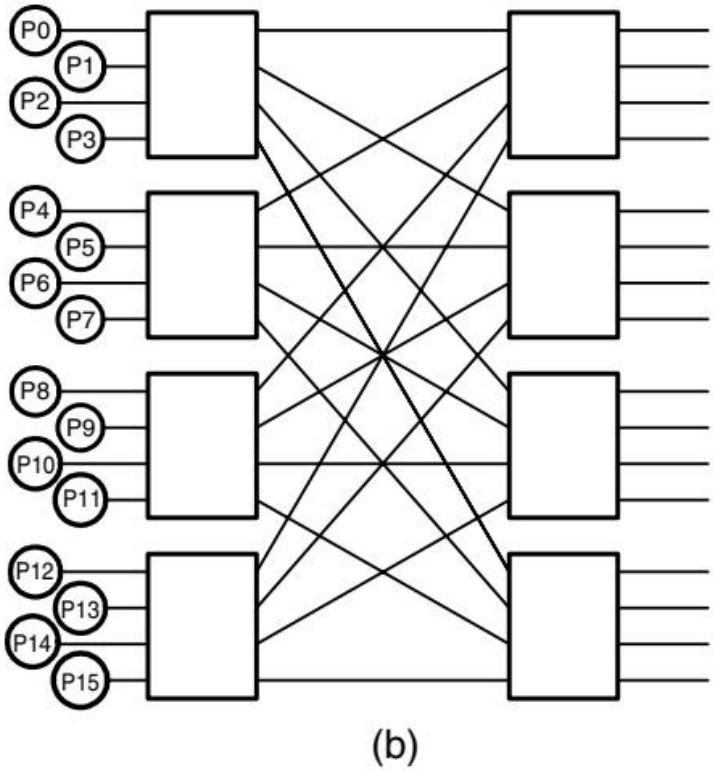


图 11.6 （a）一个 32 端口交换模块由八个 8×8 交换机组成，排列成两排。所有链路都是双向的。（b）每个机架包含 16 个处理节点和 1 个交换模块。

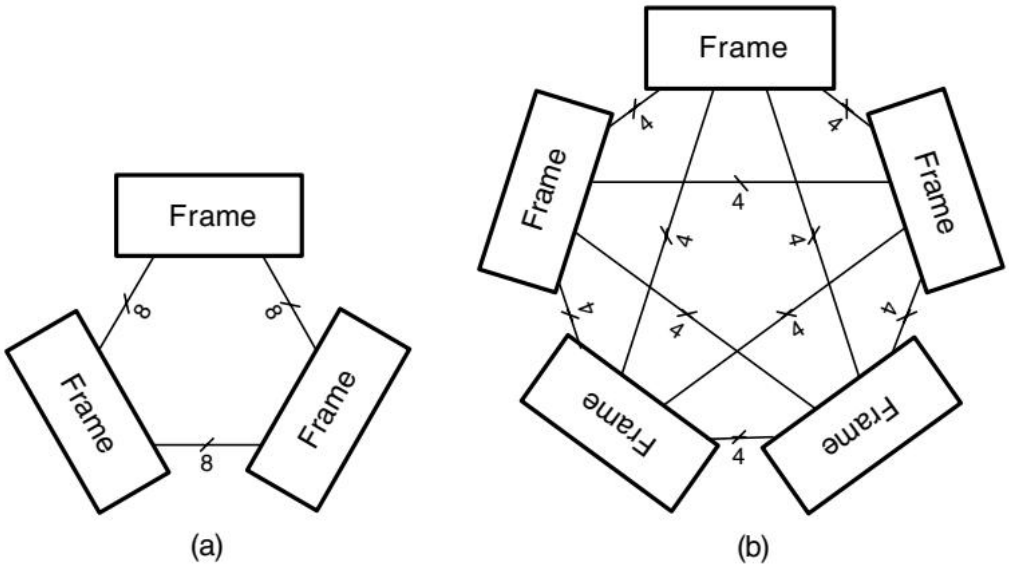


图 11.7 小型 SP 机通过直接连接机架实现。（a）48 处理器（3 机架）配置。（b）80 处理器（5 机架）配置。

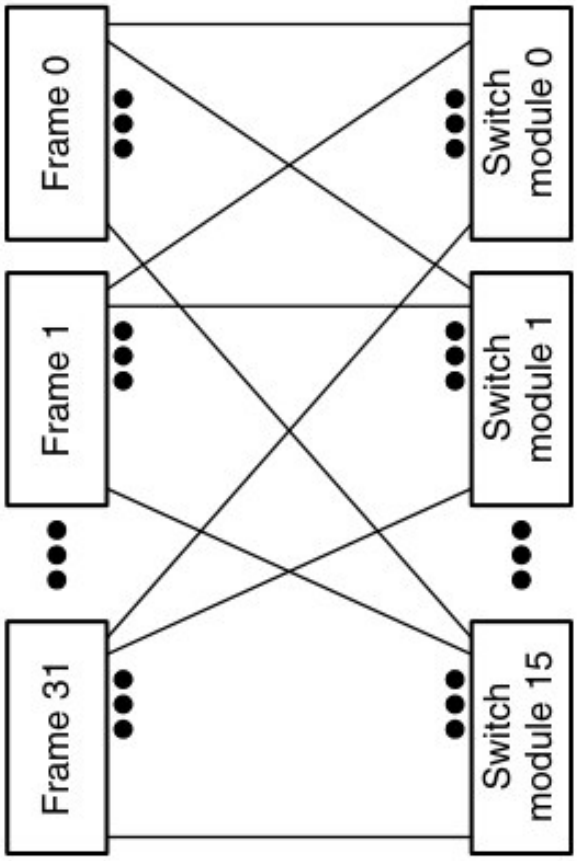


图 11.8 大型 IBM SP 系统：由处理器机框和交换模块构成的折叠式 Clos 网络

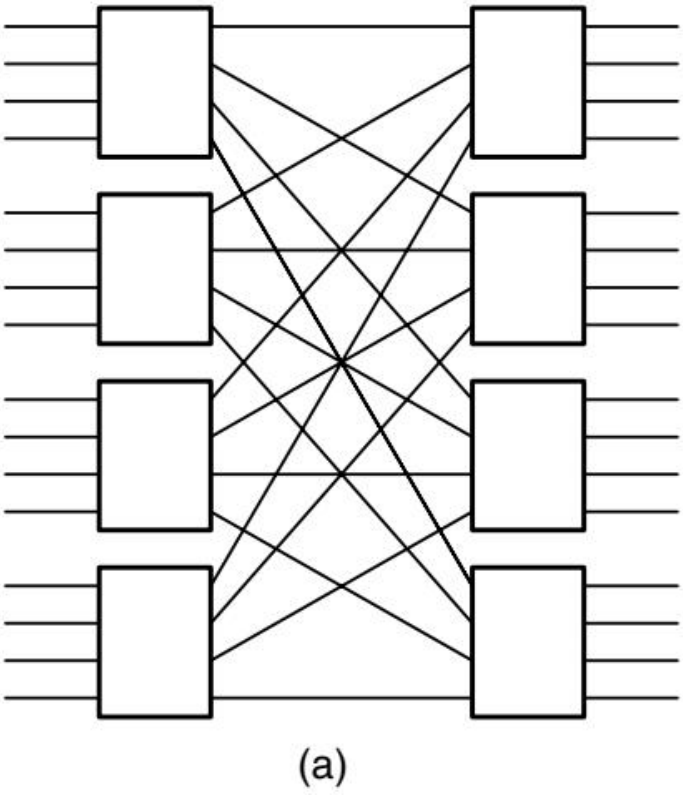


图 11.6 （a）一个 32 端口交换模块由八个 8×8 交换机组成，排列成两排。所有链路都是双向的。（b）每个机架包含 16 个处理节点和 1 个交换模块。

每个节点包含一个源路由表（如第 11.1.1 节所述）。该表包含到网络中每个目的地的四条路由。如图 11.9 所示，当数据包到达需要路由时，它使用数据包的目的地和 2 位数据包计数器的当前值来索引该表。数据包计数器在下一个数据包使用之前递增。从表中读取一个 8 字节的路由描述符。第一个字节包含描述符中有效路由 flit 的数量 N（0-7）。

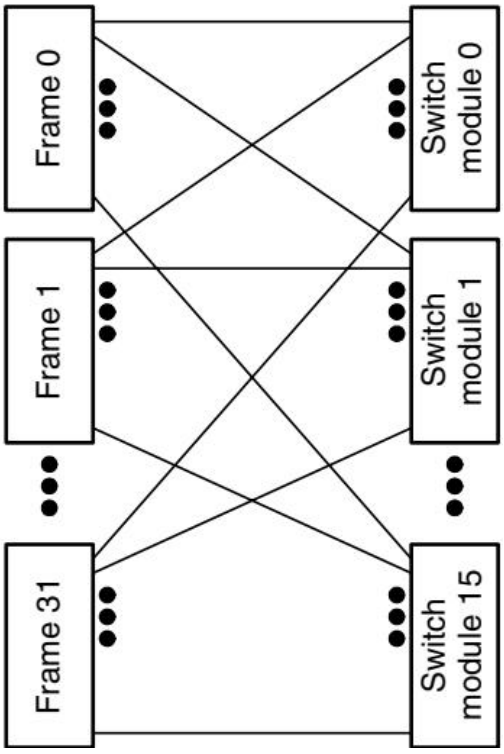
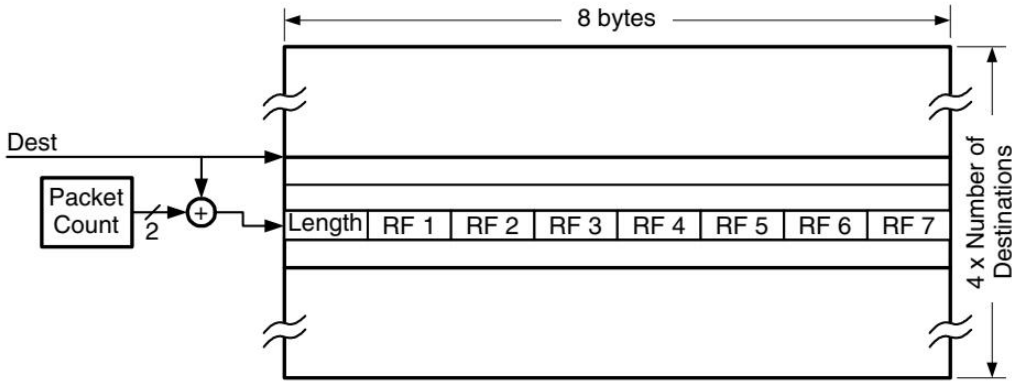


图 11.9 IBM SP 交换网络的结构与扩展方式（参见原书图 11.9）



更大的 SP 机通过将处理器机架连接到实现折叠 Clos 互连网络的交换模块来实现。这里，一个 512 节点（32 机架）的机器通过 16 个交换模块（32 端口）连接。

IBM SP1/SP2 的源路由表。每个目的地有 4 条路由。每条路由在表中占用 8 个字节。长度字节指示路由中应包含多少个路由 flit（RF）字节。每个 RF 字节编码路由的 2 跳，

如图 11.10 所述。

接下来的 N 个字节包含路由 flit，每个 flit 编码路由的 1 或 2 跳。最大长度的路由描述符包含 7 个路由 flit，编码最多 14 跳。这 N 个路由 flit 被添加到数据包的前面，在路由过程中使用。SP 数据包如图 11.10 所示。在一个初始长度 flit 之后，数据包包含一个或多个路由 flit，随后是数据 flit。每个路由器检查第一个路由 flit。如果选择位 S 为零，路由器使用 flit 的上跳字段（位 4-6）选择输出端口并设置 S。如果 S 为 1，则使用第二跳字段（位 0-2），丢弃路由 flit，并减少长度。通常，

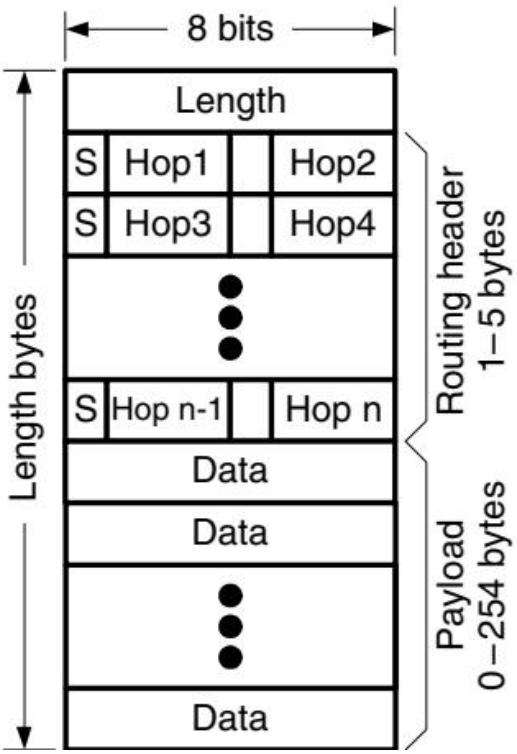


图 11.10 IBM SP1/SP2 的数据包格式。数据包被划分为 8 位 flit。每个数据包以一个长度 flit 开始，后跟一个或多个路由 flit。数据包的其余部分包含有效载荷数据。长度 flit 给出数据包的总长度（以 flit 为单位），包括长度 flit 和路由 flit。每个路由 flit 指定用于两跳的输出端口（8 个中的 1 个）。选择位 S 在 flit 内的两跳之间进行选择。

S 位最初为 0。然而，为了编码具有奇数跳的路由，其中一个路由 flit（通常是最后一个）的 S 位必须最初设置为 1。SP 数据包沿着源确定的路由向目的地前进，每两跳消耗一个路由 flit。当它到达目的地时，它只包含长度 flit 和数据 flit——所有路由 flit 都已被丢弃。长度 flit 随着每个路由 flit 的丢弃而递减，因此在任何时间点它都反映数据包的当前长度。

路由表的构建方式旨在平衡网络链路上的负载，并提供边不相交路径 [1]。对于每个节点，到每个目的地的第一条路由通过为网络找到最小生成树来确定，假设每条边最初具有单位成本。然后，通过为每条边在路由中每次使用时增加一个小的量 ϵ 来调整边权重。然后计算新的最小生成树，并用于生成到每个目的地的第二条路由。对第三条和第四条路由重复该过程。将 IBM SP 网络与 CM-5（第 10.6 节）和 Avici TSR（第 9.5 节）的网络进行比较是很有趣的。在大型配置中，SP 网络的拓扑与 CM-5 几乎相同。

两者都使用折叠 Clos 拓扑由 8×8 端口交换机组成。拓扑上的主要区别在于 SP 网络中没有集中，而 CM-5 网络有四倍的集中。在这两种常见拓扑中，两种网络采用了非常不同的路由方法。CM-5 在上行链路上使用自适应路由来动态平衡负载。相比之下，SP 网络使用多个源路由进行无关路由，以静态平衡负载。我们在练习 11.9 中研究这种权衡。3 虽然 CM-5 和 SP 具有共同的拓扑但路由不同，而 TSR 和 SP 具有非常相似的路由但将此路由方法应用于非常不同的拓扑。TSR 和 SP 都使用无关源路由，其中源表维护到每个目的地的多个替代路由：SP 中为 4 条，TSR 中为 24 条。它们的不同之处在于，SP 通过使用数据包计数器选择源路由，而 TSR 通过对流标识符进行哈希来选择源路由——这使相关的数据包保持在同一路由上，从而保持顺序。两者都以静态平衡网络通道负载的方式在路由表中构建替代路由。有趣的是，这种无关源路由技术适用于像不规则 3-D 环面和折叠 Clos 网络这样不同的拓扑。源路由的一个主要优势是其拓扑独立性。

11.4 文献注释

基于表的路由因其灵活性和简单性而在实现中广受欢迎。如第 11.3 节所述，IBM SP1 和 SP2 网络 [178，176] 使用源路由表，Avici TSR（第 9.5 节）也是如此。然而，对于较大的网络，源表的大小可能变得令人望而却步。许多网络，如 SGI Spider [69]、InfiniBand [150] 和互联网，出于这个原因采用节点表。此外，许多节点表实现使用分层路由，如 McQuillan [127] 所介绍并由 Kleinrock 和 Kamoun [100] 详细分析的那样，以进一步减少表的大小。SGI Spider 中使用的分层路由机制在练习 11.4 中考虑。另一种减少表大小的方法是区间路由（见练习 11.5），它由 Santoaro 和 Khatib [158] 开发，由 van Leeuwen 和 Tan [188] 扩展，并已在 INMOS Transputer [86] 中实现。如 Pei 和 Zukowski [147] 以及 McAuley 和 Francis [122] 所述，CAM 在路由表的实现中很常见。由 Fredkin [68] 引入的 Trie 树在路由表中也很受欢迎，尤其是在 IP 路由器中。Pei 和 Zukowski [147] 描述了 Trie 树的硬件设计，Doeringer 等人 [59] 给出了 Trie 树的扩展，以便在 IP 查找中高效使用。

11.5 练习

11.1 源路由的压缩。在本章介绍的源路由中，每个端口选择符（N、S、E、W 和 X）用三位编码。提出一种替代编码，以减少表示源路由所需的平均长度（以位为单位）。根据环面上可能出现的“典型”路由来证明你的编码的合理性。此外，在以下路由上比较原始的每符号三位编码与你的编码：

(a) NNNNNEEEX

(b) WNEENWWWWWX

(c) SSEESSX

11.2 基于 CAM 的路由表。考虑一个 8 元 2 立方体拓扑中节点 34 处的节点表。如果

存储在 RAM 中，需要多少个表项？如果改为存储在 CAM 中，可以减少多少？列出 CAM 的表项。

11.3 最小自适应路由选择逻辑。画出最小自适应路由下路由选择模块（图 11.4）所需的逻辑。设计逻辑以选择具有最短下游队列的有效路径。

11.4 SGI Spider 中的分层路由。SGI Spider 使用具有两层的分层节点表：元表和本地表。网络中每个节点的 ID 被分为元地址（高位）和本地地址（低位）。为了计算下一跳，数据包首先检查其元地址是否与当前节点的元地址匹配。如果不匹配，则从元表中检索下一跳，该表以数据包目的地的元地址为索引。如果元地址匹配，则从本地表中检索下一跳，该表以数据包的本地地址为索引。从概念上讲，元地址用于将数据包移动到网络的正确“区域”，而本地地址将数据包移动到其确切目的地。如果在这种方法中使用节点 ID 的两位作为元地址，二维环面网络的路由表可以小多少？

11.5 网格网络中的线性区间路由。区间路由旨在通过用不重叠的区间标记节点的出边来减小路由表的大小。图 11.11 显示了一个线性区间标记的示例：每条通道都标有一个区间 $[x, y]$，表示所有目的地为节点 d 且满足 $x \leq d \leq y$ 的数据包都应沿着这条通道转发。例如，要从节点 3 路由到节点 1，数据包首先沿着节点 3 的西向通道转发，因为目的节点落在西向通道的区间内。类似地，下一跳将把数据包从节点 1 带到其目的地节点 0。为 4×4 网格网络找到一个线性区间标记。（将数字 0 到 15 分配给目的节点。）

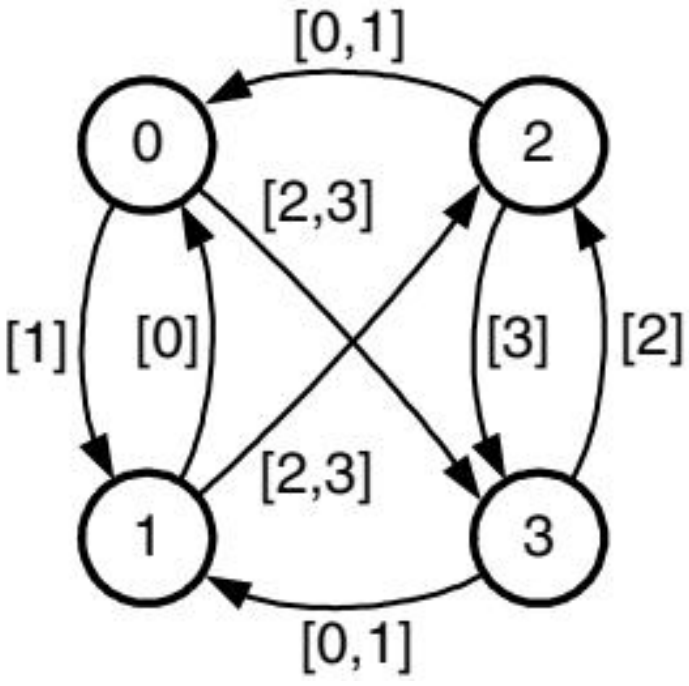


图 11.11 一个拓扑和线性区间标记。数据包通过选择其区间包含数据包目的节点的出边通道进行逐跳路由。

11.6 IBM SP2 的路由生成。编写一个程序来计算 512 节点 IBM SP2 网络中的生成树。使用这些树生成一个路由表，每个目的地有四个节点。

11.7 生成树路由的负载均衡。使用你在练习 11.6 中生成的路由表，计算多个随机排列

遍历机器时的通道负载。负载有多均衡?
11.8 IBM SP 网络上的自适应路由。后续版本的 SP 网络采用了自适应源路由，允许在上行链路上进行类似 CM-5 的自适应，然后在下行链路上进行源路由。解释这可能如何工作。开发一种支持这种自适应源路由并适合 SP 网络通用框架的路由编码。
11.9 模拟。在 512 节点 SP 网络上比较类似 CM-5 的自适应路由与类似 IBM SP 的 oblivious 路由的性能。

注释

- 1. 例如，在 SGI Spider 芯片 [69] 中就是这样做的。参见练习 11.4。
- 2. 为了实现容错，每个交换模块包含该 32 端口交换配置的 2 个副本（共 16 个交换芯片）。一个副本执行交换，而另一个副本对其进行影子操作以检查错误。
- 3. 后续版本的 SP 机器在上行链路上使用了自适应源路由。

第十二章　流控制基础

流控制决定了如何将网络资源（如通道带宽、缓冲区容量和控制状态）分配给穿越网络的数据包。良好的流控制方法以高效的方式分配这些资源，使网络达到其理想带宽的高比例，并以低且可预测的延迟交付数据包。1 另一方面，糟糕的流控制方法通过让资源闲置并对其他资源进行非生产性工作来浪费带宽。这导致网络（如我们在第 2 章中检查的网络）仅实现理想带宽的一小部分，并且具有高且可变的延迟。

可以将流控制视为资源分配问题或争用解决问题。从资源分配的角度来看，当每个数据包从源到目的地前进时，必须以通道、缓冲区和状态的形式为其分配资源。同样的过程可以视为解决争用。例如，同时到达路由器不同输入的两个数据包可能都希望同一个输出。在这种情况下，流控制机制解决这种争用，将通道分配给一个数据包，并以某种方式处理另一个被阻塞的数据包。

最简单的流控机制是无缓冲的，它们不暂时存储被阻塞的分组，而是丢弃或错误路由这些分组。复杂度和效率的下一级是电路交换，其中只有分组头被缓冲。在电路交换中，分组头先于任何分组有效载荷在网络中传输，沿途预留适当的资源。如果分组头无法立即在特定节点分配资源，它就在该节点等待，直到资源变为空闲。一旦整个路径（或电路）被预留，数据就可以通过该电路发送，直到通过释放信道来拆除电路。所有这些流控机制都相当低效，因为它们浪费昂贵的信道带宽来避免使用相对廉价的存储空间。

通过在数据等待获取网络资源时对其进行缓冲，可以实现更高效的流控。缓冲在时间上解耦了相邻信道的分配。这种解耦减少了对分配的约束，从而带来更高效的运行。这种缓冲可以以分组为单位进行，如存储转发和直通流控，也可以以更细粒度的 flit 为单位进行，如虫洞或虚拟信道流控。通过将大的可变长度分组分解为更小的固定大小的 flit，可以大大减少任何特定节点所需的存储量。以 flit 为单位分配资源还有助于在网络中为每个物理信道创建多个虚拟信道，这可以缓解阻塞并提高吞吐量。

本章我们开始讨论流控，首先讨论流控问题（第 12.1 节）、无缓冲流控（第 12.2 节）和电路交换（第 12.3 节）。在第 13 章中，我们通过讨论缓冲流控方法继续探索流控。在这两章中，我们只处理网络中的资源分配。正如我们将在第 14 章中看到的，为了确保网络保持无死锁，对分配还有额外的约束。在端点处管理资源（特别是缓冲存储器）还有一个相关问题。这种端到端流控采用类似的原则，但此处不予讨论。

12.1　资源与分配单元

要穿越互连网络，消息必须被分配资源：信道带宽、缓冲容量和控制状态。图 12.1 展示了网络单个节点中的这些资源。当分组到达节点时，它必须首先被分配一些控制状

态。根据流控方法的不同，每个信道可能有一个控制状态，或者，如果一个输入可以同时被多个分组共享，则可能有多组状态。控制状态跟踪节点内分配给分组的资源以及分组穿越节点的状态。要前进到下一个节点，分组必须被分配节点输出信道上的带宽。在一些网络中，分配输出（或前向）信道上的带宽也会分配反方向的反向信道上的带宽。反向信道通常用于携带确认信息并传达来自接收节点的流控信息。最后，当分组到达节点时，它被暂时保存在缓冲区中，同时等待信道带宽。所有流控方法都包括控制状态和信道带宽的分配。然而，一些方法（我们将在第 12.2 节讨论）不分配缓冲区。

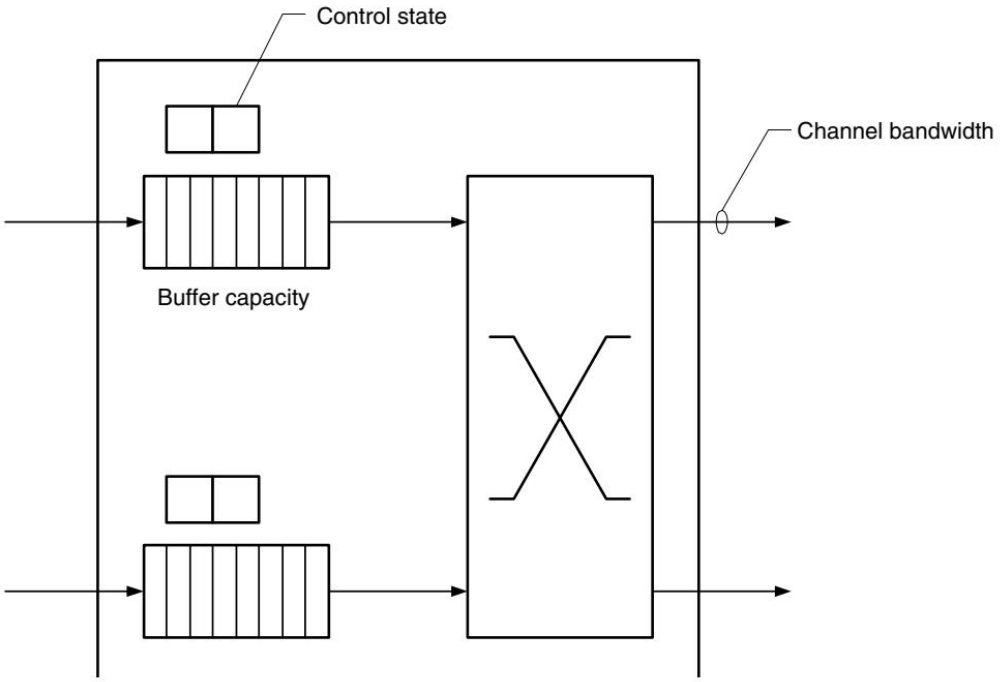


图 12.1 流控方法在一个网络节点内分配的资源：控制状态记录信道和缓冲区对分组的分配以及分组穿越节点的当前状态。信道带宽将分组的 flit 从该节点推进到下一个节点。缓冲区在分组的 flit 等待信道带宽时保存它们。

为了提高这种资源分配的效率，我们将消息划分为分组以分配控制状态，并划分为流控单元（flit）以分配信道带宽和缓冲容量。

图 12.2 显示了网络资源分配的单元。在顶层，消息是从源终端传递到目的终端的逻辑上连续的比特组。由于消息可能任意长，资源不直接分配给消息。相反，消息被划分为一个或多个具有受限最大长度的分组。通过限制分组的长度，资源分配的大小和持续时间也受到限制，这对于流控机制的性能和功能通常很重要。

分组是路由和排序的基本单位。控制状态分配给分组。如图 12.2 所示，分组由消息的一段加上前置的分组头组成。分组头包括路由信息（RI），如果需要，还包括序列号（SN）。路由信息用于确定分组从源到目的地的路由。如第 8 章所述，路由信息可以包括目的字段或源路由等。如果分组在传输中可能乱序，则需要序列号来重新排序消息的分组。例如，如果不同分组在源和目的地之间遵循不同路径，就可能发生这种情况。如果可以保证分组保持顺序，则不需要序列号。

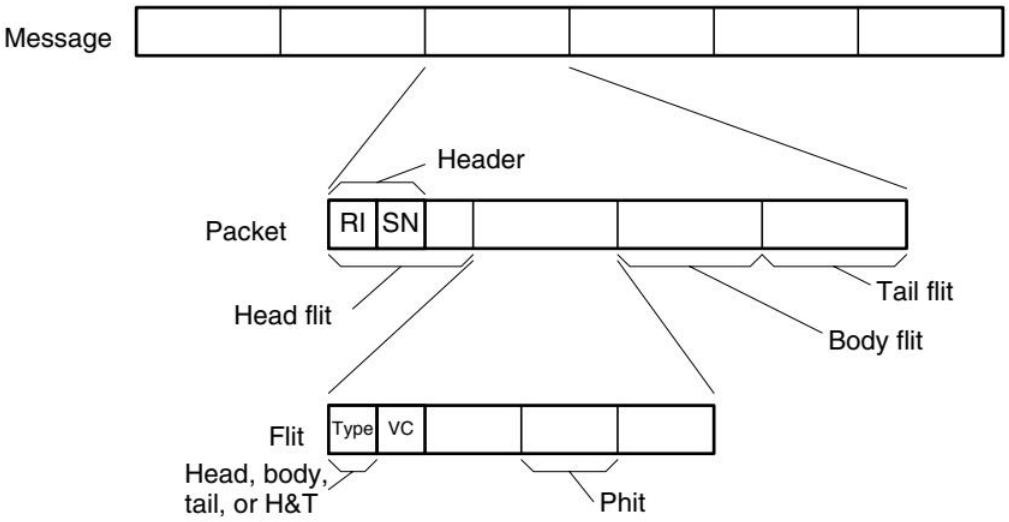


图 12.2 资源分配单元。消息被划分为分组以分配控制状态。每个分组包括路由信息（RI）和序列号（SN）。分组进一步划分为 flit 以分配缓冲容量和信道带宽。除了分组中携带的信息外，flit 不包含路由或排序信息，但可能包含虚拟信道标识符（VCID）以记录分组到控制状态的分配。

一个包可以进一步划分为流量控制数字（flow control digit）或微片（flit）。微片是大多数流量控制方法所使用的带宽和存储分配的基本单位。微片不携带路由和排序信息，因此必须遵循相同的路径并保持顺序。然而，在单个物理信道上可能同时有多个包传输的系统中，微片可以包含虚拟信道标识符（VCID），以标识该微片属于哪个包。

微片在包中的位置决定了它是头微片、体微片、尾微片还是它们的组合。头微片是包的第一个微片，携带包的路由信息。头微片之后是零个或多个体微片以及一个尾微片。在非常短的包中，可能没有体微片；在极端情况下，即包只有一个微片时，头微片也可能同时是尾微片。当包穿越网络时，头微片为该包分配信道状态，而尾微片则释放它。体微片和尾微片没有路由或排序信息，因此必须沿着头微片的路由前进并保持顺序。

微片本身又细分为一个或多个物理传输数字（physical transfer digit）或 phit。phit 是在单个时钟周期内通过信道传输的信息单位。尽管没有以 phit 为单位分配资源，但链路层协议必须解释信道上的 phit 以找到微片之间的边界。

为什么要费心将包分解成微片？人们可以以包为单位进行所有分配：信道状态、缓冲区容量和信道带宽。事实上，我们将研究几种正是这样做的流量控制策略。然而，这些策略在选择包大小时面临相互冲突的约束。一方面，我们希望使包非常大，以摊销路由和排序的开销。另一方面，我们希望使包非常小，以实现高效、细粒度的资源分配并最小化阻塞延迟。引入微片消除了这种冲突。我们可以通过使包相对较大来实现低开销，同时通过使微片非常小来实现高效的资源利用。

关于大小没有硬性规定。然而，phit 的大小通常在 1 位到 64 位之间，典型值为 8 位。微片通常范围从 16 位（2 字节）到 512 位（64 字节），典型值为 64 位（8 字节）。最后，包通常范围从 128 位（16 字节）到 512 千位（64 千字节），典型值为 1 千位（128

字节)。按照这些典型大小，一个 64 位的微片包含八个 8 位的 phit，而一个 1 千位的包包含十六个 64 位的微片。

12.2 无缓冲流量控制

最简单的流量控制形式不使用缓冲，仅用于为竞争的包分配信道状态和带宽。在这些情况下，流量控制方法必须执行仲裁，以决定哪个包获得其请求的信道。仲裁之后，获胜的包通过该信道前进。仲裁方法还必须决定如何处理未获得所请求目的地的任何包。由于没有缓冲区，我们不能保留失败的包直到它们的信道变为空闲。相反，我们必须要么丢弃它们，要么错误路由它们。

例如，考虑图 12.3（a）中的情况。两个包 A 和 B 到达一个无缓冲网络节点，并且都请求输出信道 0。图 12.3（b）展示了类似于第 2 章中使用的丢弃流量控制方法如何处理这种冲突。在这种情况下，A 赢得仲裁并通过输出链路前进。另一方面，包 B 输掉仲裁并被丢弃。任何用于将包 B 推进到这一点的资源（如信道带宽）都被浪费了。包 B 必须从其源重新传输，我们假设源有一个缓冲副本，重复已经花费的努力将包送到网络中的这一点。此外，还需要某种确认机制来通知源 B 何时被成功接收或何时需要重新传输。或者，包 B 可能被错误路由到另一个输出，如图 12.3（c）所示。在这种情况下，必须有足够的路径多样性和适当的路由机制来将包 B 从这一点路由到其目的地。

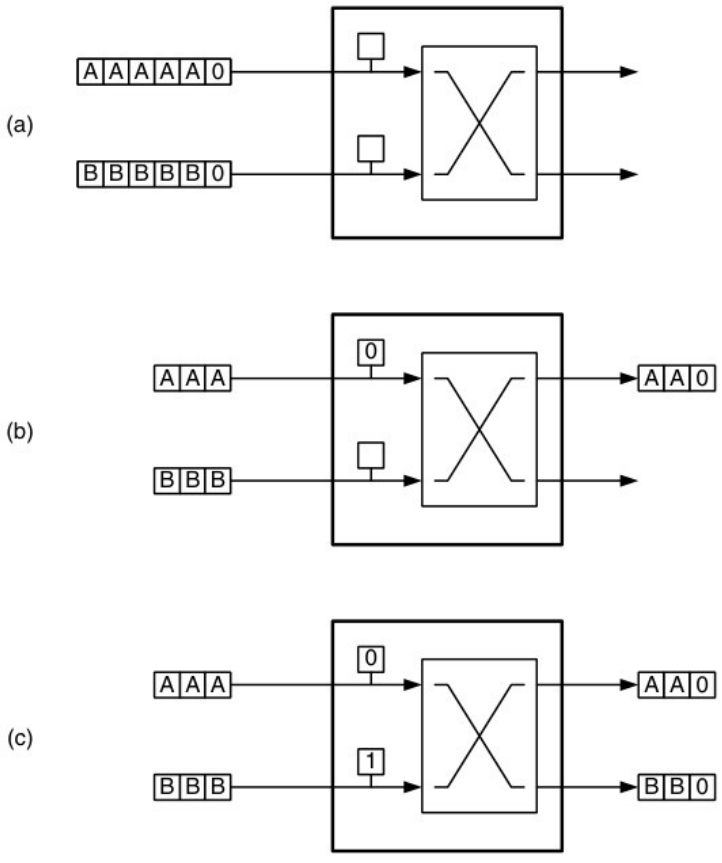


图 12.3 无缓冲流控：两个数据包竞争同一输出通道时的丢弃与处理

图 12.4 显示了图 12.3（b）中丢弃流量控制策略的时空图，使用显式否定确认或 nack（N）。该图类似于甘特图，垂直轴显示资源（信道）的利用率，水平轴显示时间。图中显示了一个五微片包沿着四跳路由发送。垂直轴显示四个信道（0-3）的正向（F）和反向（R）方向。水平轴显示微片周期（0-17）。在图中，包的第一次传输无法分配信道 3 并被丢弃。一个 nack 触发包的重新传输，这次传输成功。

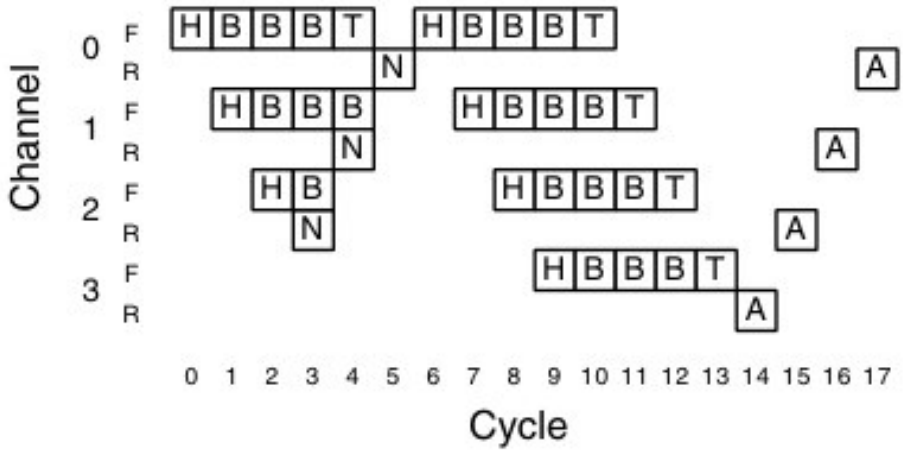


图 12.4 采用显式否定确认的丢弃式流控时空图

包传输从微片周期 0 开始，包的头微片（H）穿过信道 0。体微片（B）在周期 1 到 3 跟随，尾微片（T）在周期 4 跟随。在这种情况下，尾微片不会释放信道 0，因为包必须保持对信道的所有权，直到它收到确认（肯定或否定）。在周期 1 和 2 期间，头微片分别穿过信道 1 和 2。然而，在穿过信道 2 之后，头微片遇到争用，无法获得信道 3，并被丢弃。为了发出此失败信号，信道 2 远端的路由器在周期 3 沿信道 2 的反方向发送一个 nack。nack 分别在周期 4 和 5 沿信道 1 和 0 的反方向传播，在周期 5 结束时到达源。当 nack 到达每个节点时，它释放该节点持有的资源，使其可供其他包使用。例如，在周期 4，nack 到达信道 1 的近端。此时，包释放信道 1 的正向和反向方向。源端收到 nack 后，数据包从周期 6 开始重传。重传的数据包能够获取到达目的地所需的所有四个通道。头微片在周期 9 到达目的地，尾微片在周期 13 到达。尾微片被接收后，在周期 14 沿反向通道发送确认（A），并在周期 17 到达源端。当确认到达每个节点时，它释放该节点持有的资源。

我们在第 2 章实现的丢弃流控不使用显式否定确认。相反，它使用超时来检测数据包何时被丢弃，如图 12.5 所示。和之前一样，数据包在第一次传输时未能获取通道 3。然而，在这种情况下，不发送 nack。相反，数据包继续通过通道 0 到 2 传输。在这些通道上，尾微片在离开节点时释放数据包持有的资源。因此，通道 0、1 和 2 分别在周期 5、6 和 7 变为空闲。

在超时时间过去而源端未收到确认后，它从周期 12 开始重传数据包。这次数据包在周期 15 到 19 期间成功被接收。确认在周期 20 发送，在周期 23 到达。由于尾微片通过后不持有任何资源，确认必须竞争反向通道，并且自身也可能被丢弃。在这种情况

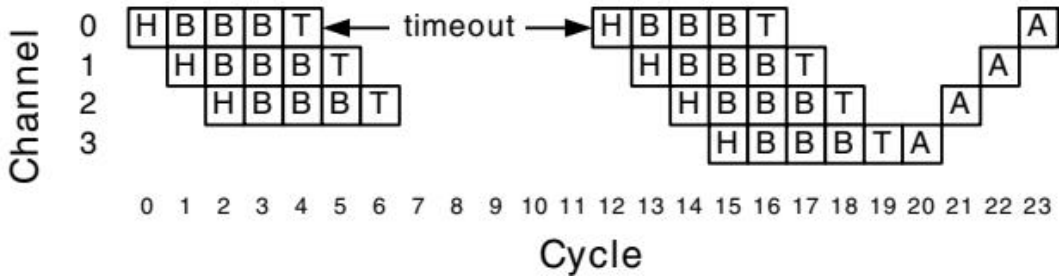


图 12.5 展示不带显式 nack 的丢弃流控的时空图：一个 5 微片包从周期 0 开始传输，并继续通过通道 0、1 和 2。该数据包在周期 3 无法获取通道 3，此时被丢弃。然而，前面的通道继续传输该数据包，直到尾微片被接收。超时在周期 12 触发数据包的重传。数据包的尾微片在周期 19 被接收，并在周期 20 开始发送确认。

下，即使数据包第一次被正确接收，它也会被重传。接收端需要某种机制（通常采用序列号）来删除这种重复的数据包，确保每个数据包恰好被接收一次。

丢弃流控虽然简单，但效率非常低，因为它使用宝贵的带宽传输最终被丢弃的数据包。第 2 章给出了计算丢弃流控（不带显式 nack）吞吐量的方法。

虽然如图 12.3（c）中的误路由不会丢弃数据包，但它通过将数据包发送到错误方向而浪费带宽。在某些情况下，这会导致不稳定性，即当提供的流量达到某一点后，网络吞吐量下降。误路由也只适用于具有足够路径多样性的网络，使得数据包在被误路由后仍能到达目的地。例如，蝶形网络不能使用误路由，因为一次错误的跳转将阻止数据包到达其目的地。在像环面这样确实具有足够路径多样性的网络中，使用误路由时活锁是一个问题。如果数据包误路由过于频繁，它可能永远无法接近其目的地。任何涉及误路由的流控策略都应包括某种可证明的向前进展保证，以确保每个数据包最终被送达。

12.3 电路交换

电路交换是一种无缓冲流控形式，其工作方式是首先分配通道以形成从源到目的地的电路，然后沿此电路发送一个或多个数据包。当不需要再发送数据包时，电路被释放。如图 12.6 所示，该过程涉及四个阶段。在第一阶段（周期 0–4），请求（R）从源传播到目的地并分配通道。在此示例中，没有遇到竞争，请求无延迟地到达目的地。电路分配后，在第二阶段（周期 6–10）将确认（A）传回源。一旦收到确认，电路建立，可以处理任意数量和大小的数据包，无需进一步控制。在示例中，发送了两个四微片包，每个后面跟着三个空闲周期。最后，当不需要再发送数据时，发送尾微片（T）以释放通道（周期 26–30），将这些通道释放以供其他电路使用。

电路交换与丢弃流控的不同之处在于，如果请求微片被阻塞，它被保持而不是丢弃。这种情况如图 12.7 所示，与图 12.6 相同，只是请求在能够分配通道 3 之前延迟了四个周期。在此期间，头微片被阻塞。它被保持在通道 3 近端的路由器中，并且每个周期重新仲裁以访问通道 3。最终（在此情况下，在周期 7），头微片获取通道 3 并继续

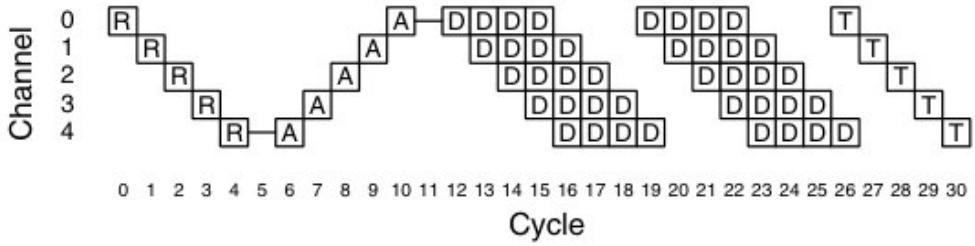
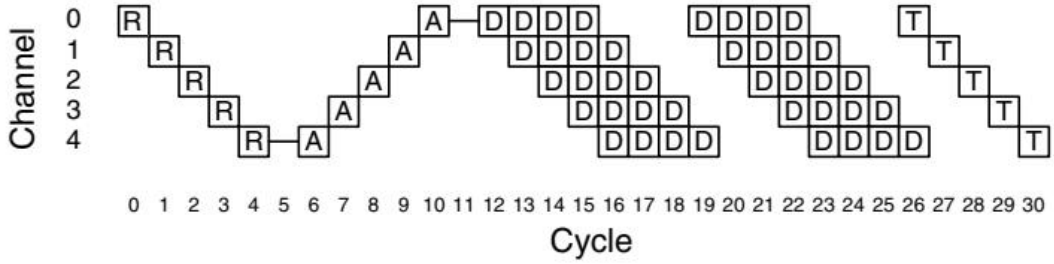


图 12.6 五跳路径上采用电路交换传送两个四 flit 数据包的时空图

分配电路。因此，与丢弃流控相比，电路交换的优点是从不通过丢弃数据包来浪费资源。由于它在每一跳缓冲头部，它总是向前进展。然而，电路交换确实有两个弱点，使其不如缓冲流控方法有吸引力：高延迟和低吞吐量。



时空图展示了使用无竞争的电路交换在 5 跳路由上传输两个 4 微片数据包的过程。在本例中, $t_{r} = 1$ 个周期，且 D = 0。传输分 4 个阶段进行。首先，向目的地发送请求（R），以获取路由每一跳的信道状态。其次，当请求到达目的地时，沿反向信道向源端返回确认（A）。第三，当确认到达源端时，数据微片（D）通过预留的信道发送。只要信道（电路）保持开放，就可以发送额外的数据包。最后，尾部微片（T）在通过时释放预留的信道。

时空图展示了使用无竞争的电路交换在 5 跳路由上传输两个 4 微片数据包的过程。在本例中，$t_r = 1$ 个周期，且 D = 0。传输分 4 个阶段进行。首先，向目的地发送请求（R），以获取路由每一跳的信道状态。其次，当请求到达目的地时，沿反向信道向源端返回确认（A）。第三，当确认到达源端时，数据微片（D）通过预留的信道发送。只要信道（电路）保持开放，就可以发送额外的数据包。最后，尾部微片（T）在通过时释放预留的信道。

从图 12.6 和图 12.7 的时空图可以看出，使用电路交换的单个数据包的零负载延迟为

$$T_0 = 3Ht_r + \frac{L}{b},$$

忽略线路延迟。第一项反映了建立信道（不包括竞争）并传递头微片所需的时间，第二项是串行化延迟，第三项是飞行时间，最后一项是竞争时间。该方程中的头部延迟是公式 3.11 中给出的三倍，因为从源到目的地的路径必须遍历三次才能交付数据包：每个方向一次用于建立电路，然后再一次用于交付第一个微片。对于单个短数据包，这三次遍历意味着延迟的显著增加。2

12.3 电路交换

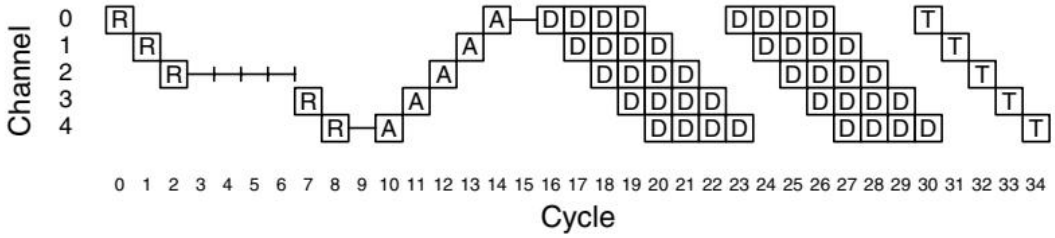


图 12.7 带竞争的电路交换时空图。在本例中，$t_r = 1$ 个周期，且 D = 0。该情况与图 12.6 相同，只是请求在能够分配信道 3 之前被阻塞了 4 个周期（周期 3–6）。

电路交换的吞吐量也会受到影响，因为信道被预留的时间比其活跃的时间更长。在发送单个数据包的情况下，每个信道处于忙状态（被电路占用）的时间为

$$T_{b0} = 2Ht_r + \frac{L}{b}.$$

在 $2Ht_r$ 的建立时间内，信道处于空闲状态。它不能被分配给其他电路，而当前电路也尚未准备好在其上发送数据。因此，这段时间所代表的信道带宽就损失了。对于短时电路，这是一个显著的开销。3 电路交换的优点是实现非常简单。路由器的逻辑与第 2 章中描述的丢弃流控路由器只有轻微差别。每个输入端增加一个寄存器，以便在发生竞争时保存请求，并增加一条反向路径。

12.4 文献注释

电路交换起源于电话通信，但在现代互连网络中并不常用。丢弃流控虽然效率低下，但简单易行，曾用于 BBN Butterfly（第 4.5 节）及其后续产品 BBN Monarch（第 23.4 节）。错路路由数据包，也称为偏转路由或热土豆路由，由 Baran [12] 提出，并用于 HEP 多处理器 [174] 和 Tera 计算机系统 [9]。

12.5 练习题

12.1 带显式 nack 的丢弃流控。考虑第 12.2 节讨论并在图 12.4 中展示的带显式 nack 的丢弃流控技术。计算使用该流控方法的网络吞吐量上限（作为容量的比例）。假设最大数据包长度为 F 个微片，平均跳数为 H_{avg}，均匀流量，以及对称拓扑。

12.2 丢弃流控的超时间隔。考虑第 12.2 节讨论并在图 12.5 中展示的带超时的丢弃流控技术。假设最大数据包长度为 F 个微片，网络的每一跳需要一个微片时间（周期），给出用这些参数和网络直径表示的最小超时间隔的表达式。

12.3 带丢弃流控和超时的活锁。在带超时机制的丢弃流控中，用于确认的反向信道没有被预留，因此确认本身可能因竞争而被丢弃。解释与此相关的活锁问题，并建议一个简单的解决方案。

12.4 乐观电路交换。乐观电路交换技术可以通过在建立电路时推测性地随头部一起发送数据来降低零负载延迟。如果头部被阻塞，则丢弃数据并沿部分预留的电路发回

nack。否则，数据可以在电路建立后立即交付到目的地。画出乐观电路交换在推测数据被丢弃和未被丢弃两种情况下的两个时空图。假设 $t_r = 1$ 个周期，D = 0。乐观电路交换能减少信道被预留但空闲的周期数吗？如果能，减少多少？
12.5 预留电路交换。考虑一种类似于电路交换的流控方法，但请求消息为每个信道预留未来固定的时间段（例如，从第 15 个周期开始预留 10 个周期）。在路径上的每个路由器处，如果请求可以被容纳，则进行预留。如果请求不能被容纳，则发送一个 nack，取消该连接的所有先前预留，并重试请求。如果请求到达目的地，则向源端发送确认，确认所有预留。画出一个时空图，展示预留电路交换相对于传统电路交换的优势。
12.5 练习题
本页故意留空

注释

- 1. 这假设路由方法能很好地平衡流量负载，并在接近最小距离的路径上路由分组。
- 2. 在没有竞争的情况下，可以通过使用乐观电路交换来消除这一缺点。见练习 12.4。
- 3. 在没有竞争的情况下，可以通过让电路建立请求为将来使用预留信道而不是立即占用信道来消除部分开销。见练习 12.5。

第十三章　缓冲式流控制

在网络中添加缓冲区可以显著提高流控效率。这是因为缓冲区将相邻信道的分配解耦。没有缓冲区时，两个信道必须在连续的周期内分配给一个数据包（或微片），否则数据包必须被丢弃或错路。数据包无处可去。添加缓冲区为我们提供了一个地方来存储数据包（或微片），同时等待第二个信道，从而允许第二个信道的分配延迟而不会带来麻烦。

一旦在互连网络中引入缓冲区，流量控制机制就必须同时分配缓冲区和通道带宽。此外，我们还可以选择分配这些资源的粒度。如图 13.1 所示，我们可以按微片或包为单位分配缓冲区或通道带宽。如图所示，大多数流量控制机制以相同的粒度分配这两种资源。如果同时以包为单位分配通道带宽和缓冲区，则称为包缓冲区流量控制，包括存储转发流量控制和直通流量控制，将在 13.1 节进一步描述。

		Channel allocated in units of	
		Packets	Flits
Buffer allocated in units of	Packets	Packet-buffer flow control	See Exercise13.1
	Flits	Not possible	Flit-buffer flow control

图 13.1　按通道带宽分配粒度与缓冲分配粒度划分的缓冲式流控方法

如果同时以微片为单位分配带宽和缓冲区，则称为微片缓冲区流量控制，将在 13.2 节描述。以微片而非包为单位分配存储有三个主要优点：减少了路由器正确操作所需的存储量；从拥塞点向源端提供了更强的背压；能够更有效地利用存储。

图 13.1 中的非对角线条目并不常用。首先考虑将通道分配给包但将缓冲区分配给微片的情况。这行不通。在通过通道传输包之前，必须分配足够的缓冲来容纳整个包。将缓冲区分配给包而将通道带宽分配给微片的情况是可能的，但不常见。练习 13.1 更详细地探讨了这种流量控制类型。

本章详细探讨带缓冲的流量控制。首先描述包缓冲区流量控制（13.1 节）和微片缓冲区流量控制（13.2 节）。接着在 13.3 节介绍管理缓冲区和信号背压的方法。最后介绍微片预留流量控制（13.4 节），它将包的控制部分和数据部分分离，以提高分配效率。

13.1 包缓冲区流量控制

如果在路由节点中添加缓冲区，我们可以更有效地分配通道带宽。将微片（或包）存储在缓冲区中，可以将输入通道对微片的分配与输出通道对微片的分配解耦。例如，一个微片可以在第 i 个周期通过输入通道传输，并存储在缓冲区中等待 j 个周期，直到第 $i+j$ 个周期成功分配输出通道。如果没有缓冲区，第 i 个周期到达的微片将不得不在第 $i+1$ 个周期传输、被误路由或丢弃。添加缓冲区可以避免因丢弃或误路由包而浪费通道带宽，也避免了电路交换固有的空闲时间。因此，带缓冲的流量控制可以使通道利用率接近 100%。

缓冲区和通道带宽可以分配给微片或包。我们首先考虑两种将这两种资源都分配给包的流量控制方法：存储转发和直通。在 13.2 节中，我们将看到将缓冲区和通道带宽分配给微片而非包如何实现更有效的缓冲区使用，并能减少争用延迟。

在存储转发流量控制中，路由上的每个节点等待一个包被完全接收（存储），然后将该包转发到下一个节点。在包被转发之前，必须为其分配两种资源：通道远端的包大小的缓冲区和通道的独占使用权。一旦整个包到达某个节点并获得这两种资源，该包就被转发到下一个节点。在等待获取资源期间，如果资源不能立即获得，不会有通道被空闲占用，当前节点上只占用一个包缓冲区。

图 13.2 显示了存储转发流量控制的时间-空间图。该图展示了一个 5 微片 1 的包在无争用的 4 跳路由上转发。在路由的每一步，整个包通过一个通道转发后，再继续到下一个通道。

存储转发流量控制的主要缺点是其非常高的延迟。由于包在一个节点被完全接收后才能开始移动到下一个节点，因此在每一跳都会经历串行化延迟。因此，一个包的总延迟为

$$T_0 = H\left(t_r + \frac{L}{b}\right).$$

直通流量控制 2 克服了存储转发流量控制的延迟惩罚，它在收到头部并获取资源（缓冲区和通道）后立即转发包，而不等待整个包被接收。与存储转发流量控制一样，直通流量控制以包为单位分配缓冲区和通道带宽。不同之处仅在于每一跳的传输都尽可能早地开始，而不等待整个包被接收。

图 13.3 显示了直通流量控制在无争用（图 13.3[a]）和有争用（图 13.3[b]）情况下，通过 4 跳路由转发一个 5 微片 3 包的时间-空间图。该图展示了只要缓冲区和通道可用，每一跳的传输在收到头部后立即开始。

在第一种情况下，资源立即可用，头部一收到就被转发。在第二种情况下，数据包必须等待三个周期才能获取通道 2。在此期间，通道 1 上的传输继续进行，数据被缓冲。通过尽快传输数据包，直通流控将延迟从跳数与串行化延迟的乘积降低为这些项之和：

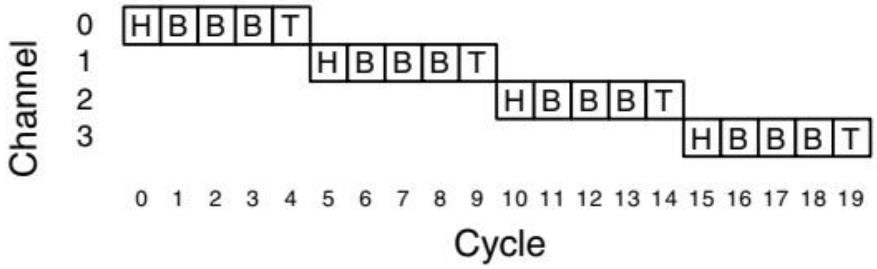


图 13.2 存储转发流量控制的时间-空间图，用于通过 4 个通道发送一个 5 微片的包。在此示例中，$t_r = 1$ 个周期，D = 0。在每个通道，为包分配一个缓冲区，整个包通过该通道传输后再继续到下一个通道。

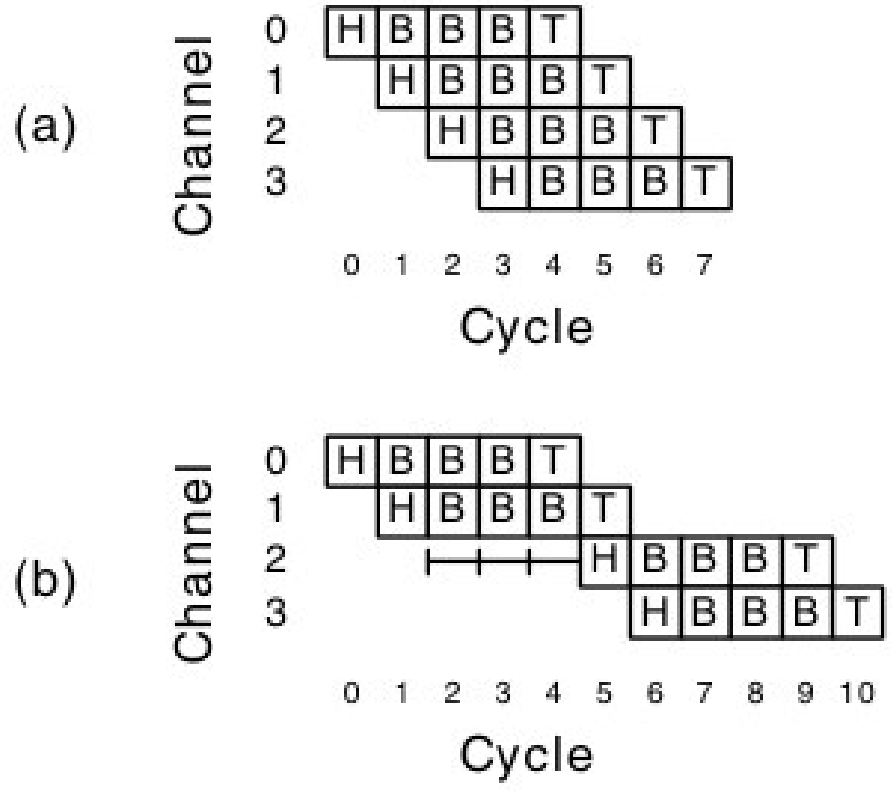


图 13.3 直通流控经 4 条通道发送一个 5-flit 数据包的时空图（$t_r = 1$，$D = 0$）：(a) 无争用时每个 flit 收到后即可转发；(b) 通道 2 的分配延迟 3 个周期，但通道 1 的传输仍然继续。

$$T_0 = Ht_r + \frac{L}{b}.$$

至此，直通流控似乎是一种理想的方法。它通过使用缓冲区解耦通道分配，实现了非常高的通道利用率。它还通过尽快转发数据包实现了非常低的延迟。然而，直通方法或任何其他基于数据包的方法有两个严重缺点。首先，通过以数据包为单位分配缓冲区，它使缓冲区存储的使用效率非常低。正如我们将看到的，通过以 flit 为单位分配缓冲区，我们可以更有效地利用存储。当我们需要多个独立的缓冲区组以减少阻塞或提供死锁避免时，这一点尤为重要。其次，通过以数据包为单位分配通道，竞争延迟增加。例如，一个高优先级数据包与一个低优先级数据包冲突时，必须等待整个低优先级数据包传输完毕才能获取通道。在下一节中，我们将看到以 flit 而非数据包为单位分配资源如何导致更高效的缓冲区使用（从而更高的吞吐量）和减少的竞争延迟。

13.2 Flit 缓冲流控

13.2.1 虫洞流控

虫洞流控 4 的操作类似于直通，但通道和缓冲区分配给 flit 而非数据包。当数据包的头部 flit 到达节点时，它必须获取三个资源才能沿路由转发到下一节点：数据包的虚拟通道（通道状态）、一个 flit 缓冲区和一个 flit 的通道带宽。数据包的体 flit 使用头部 flit 获取的虚拟通道，因此只需获取一个 flit 缓冲区和一个 flit 的通道带宽即可前进。数据包的尾部 flit 像体 flit 一样处理，但在通过时还会释放虚拟通道。

虚拟通道保存协调数据包 flit 在通道上处理所需的状态。至少，该状态标识当前节点下一跳的输出通道以及虚拟通道的状态（空闲、等待资源或活动）。虚拟通道还可以包括指向当前节点上缓冲的数据包 flit 的指针以及下一节点上可用的 flit 缓冲区数量。图 13.4 展示了一个使用虫洞流控通过节点传输 4-flit 数据包的示例。图 13.4（a–g）逐周期说明了转发过程，图 13.4（h）显示了总结整个过程的时空图。当头部 flit 到达时，输入虚拟通道最初处于空闲状态（I）（a）。上方输出通道忙，分配给了下方输入（L），因此输入虚拟通道进入等待状态（W），头部 flit 被缓冲，同时第一个体 flit 到达（b）。数据包再等待两个周期以获取输出虚拟通道（c）。在此期间，第二个体 flit 被阻塞，无法通过输入通道转发，因为没有缓冲区可容纳该 flit。当输出通道变得可用时，虚拟通道进入活动状态（A），头部 flit 被转发，第二个体 flit 被接受。数据包的其余 flit 在后续周期中通过通道前进（e、f 和 g）。当数据包的尾部 flit 被转发时，它重新分配输入虚拟通道，使其返回空闲状态（g）。

与直通流控相比，虫洞流控更有效地利用缓冲区空间，因为每个虚拟通道只需要少量 flit 缓冲区。5 相比之下，直通流控需要几个数据包的缓冲区空间，通常至少比虫洞流控多一个数量级的存储。然而，这种缓冲区空间的节省以一定的吞吐量为代价，因为虫洞流控可能在数据包中途阻塞通道。

（c - 2 个周期）

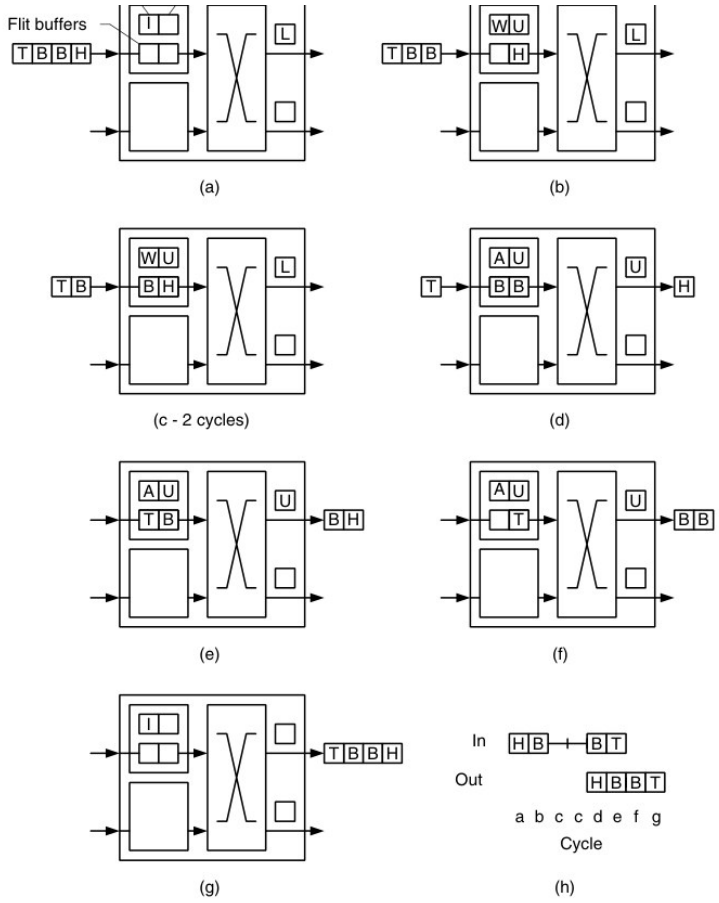
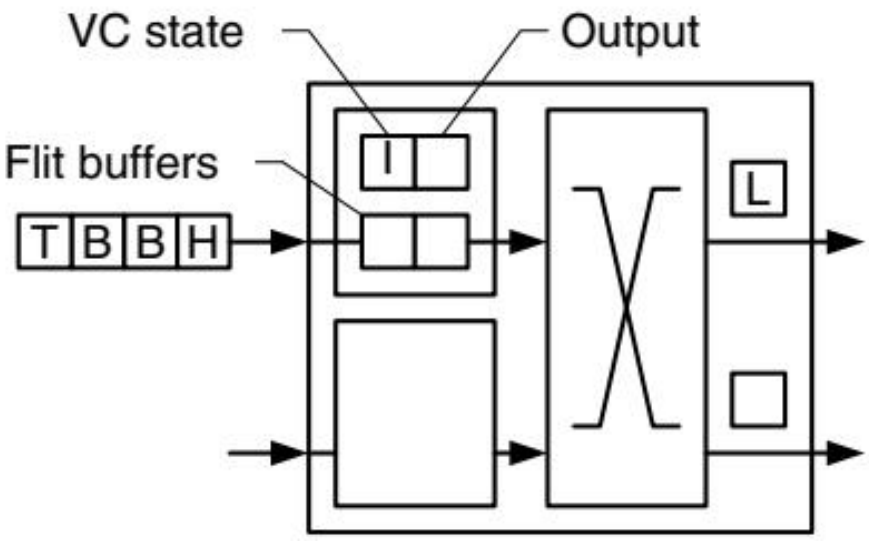
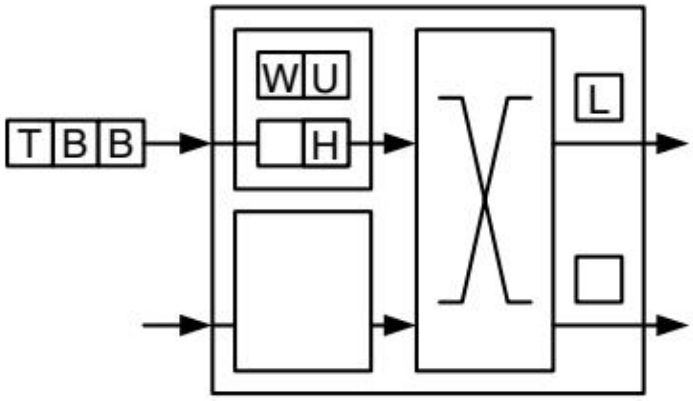


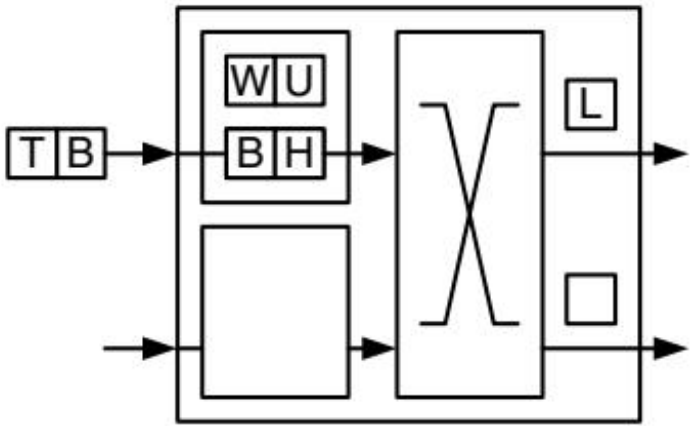
图 13.4 虫洞流控：四 flit 数据包穿越路由器的逐步过程与时空图



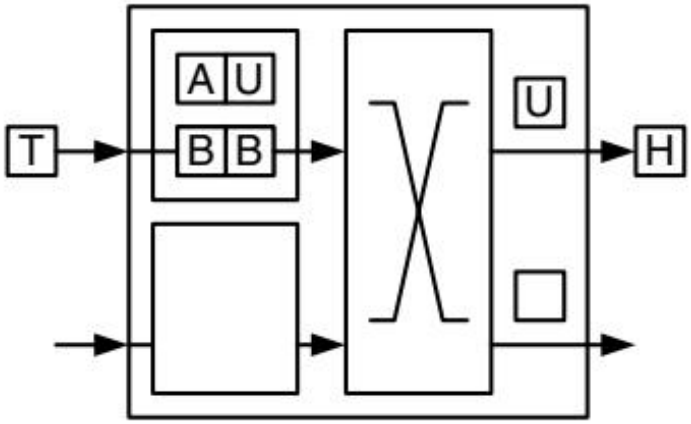
(a)



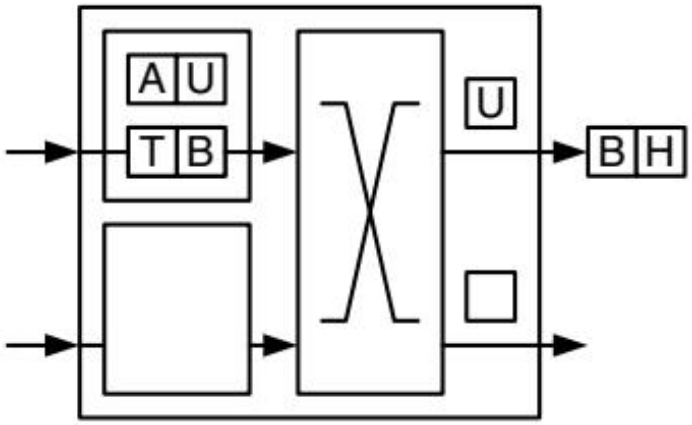
(b)



（c - 2 个周期）

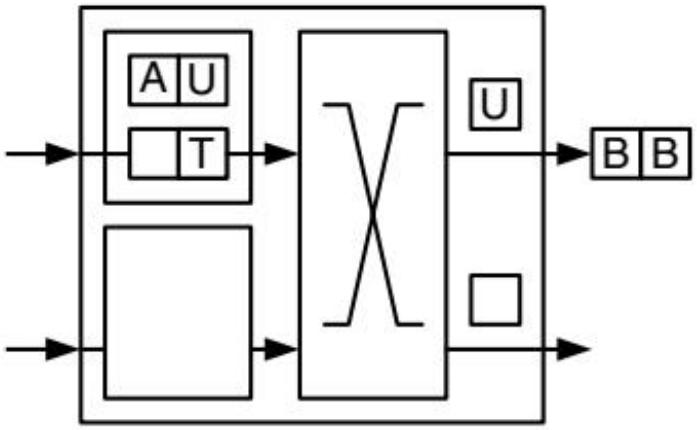


(d)



(e)

(e)



(f)

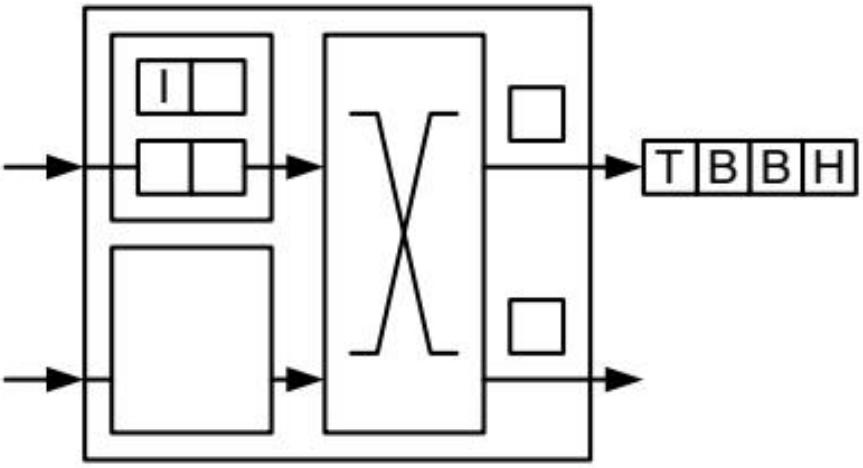
(f)

(g)

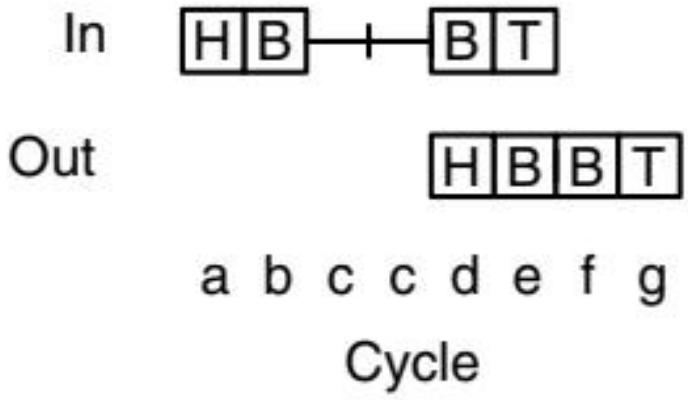
(h)

虫洞流控：（a–g）一个 4-flit 数据包从上方输入端口到上方输出端口通过节点的路由过程，（h）显示此过程的时空图。在（a）中，头部到达节点，而虚拟通道处于空闲状态（I），期望的上方（U）输出通道忙——分配给了下方（L）输入。在（b）中，头部被缓冲，虚拟通道处于等待状态（W），同时第一个体 flit 到达。在（c）中，头部和第一个体 flit 被缓冲，而虚拟通道仍处于等待状态。在这种持续两个周期的状态下，输入通道被阻塞。第二个体 flit 无法传输，因为它无法获取 flit 缓冲区。在（d）中，输出虚拟通道变得可用并分配给此数据包。状态转为活动（A），头部被传输到下一节点。体 flit 在（e）和（f）中跟随。在（g）中，尾部 flit 被传输并释放虚拟通道，使其返回空闲状态。

虫洞流控中可能出现阻塞，因为信道被一个包独占，但缓冲区是按微片逐个分配的。当某个微片无法获得缓冲区时（如图 13.4（c）所示），信道便进入空闲状态。即使还



(g)



(h)

有另一个包可能利用这条空闲信道的带宽，它也无法使用，因为被阻塞的包占用了与该链路关联的唯一虚拟信道。尽管我们按微片逐个分配信道带宽，但只有属于一个包的微片才能使用这些带宽。

13.2.2 虚拟信道流控

虚拟信道流控将多个虚拟信道（信道状态和微片缓冲区）与一条物理信道关联起来，从而克服了虫洞流控的阻塞问题：当一个包阻塞时，它允许其他包使用原本会闲置的信道带宽。与虫洞流控一样，到达的头微片必须分配一个虚拟信道、一个下游微片缓冲区和信道带宽才能前进。该包后续的体微片使用头微片分配的虚拟信道，并且仍然必须分配微片缓冲区和信道带宽。然而，与虫洞流控不同的是，这些微片并不保证能获得信道带宽，因为其他虚拟信道可能正在竞争通过同一链路传输其所属包的微片。虚拟信道允许包绕过被阻塞的包，从而利用原本空闲的信道带宽，如图 13.5 所示。该图展示了一个二维单向环面网络的 3 个节点，其中包 B 从北侧进入节点 1，获取了从节点 1 到节点 2 的信道 p，然后被阻塞。第二个包 A 从西侧进入节点 1，需要向东路由到节点 3。图 13.5（a）展示了每条物理信道仅使用一个虚拟信道的虫洞路由情形。在这种情况下，包 A 在节点 1 被阻塞，因为它无法获取信道 p。物理信道 p 和 q 都处于空闲状态，因为包 B 被阻塞且未使用带宽。然而，包 A 无法使用这些空闲信道，因为它无法获取包 B 在节点 2 上持有的信道状态。

图 13.5（b）展示了每条物理信道配置两个虚拟信道的相同情形。每个小方块代表一个完整的虚拟信道状态：信道状态（空闲、等待或活跃）、输出虚拟信道和一个微片缓冲区。在这种情况下，包 A 能够获取节点 2 上的第二个虚拟信道，从而继续前往节点 3，利用在每节点仅一个虚拟信道时原本空闲的信道 p 和 q。

图 13.5 所示的情形类似于单车道上的一辆停下的汽车（包 B）等待左转进入拥堵的支路（节点 2 的南出口）。第二辆车（包 A）被第一辆车堵在后面，即使前方道路（信道 p 和 q）畅通也无法前进。添加一个虚拟信道几乎等同于在道路上增加一条左转车道。有了左转车道，到达的汽车（包 A）就能越过等待转弯的汽车（包 B），继续沿道路行驶。而且，正如添加虚拟信道不会增加物理信道的带宽一样，左转车道也不会增加交叉口之间道路的宽度。

正如这个简单例子所示，虚拟信道流控将信道状态的分配与信道带宽解耦。这种解耦防止了先获取信道状态然后阻塞的包一直占用信道带宽而不使用。这使得虚拟信道流控能够实现比虫洞流控高得多的吞吐量。事实上，在总缓冲区空间相同的情况下，虚拟信道流控也优于直通流控，因为将缓冲区空间分配为多个短的虚拟信道微片缓冲区比分配为单个大的直通包缓冲区更高效。

图 13.6 展示了一个两个虚拟信道上的包必须共享单条物理信道带宽的例子。这里，包 A 和包 B 分别从输入 1 和输入 2 到达，并且都获取了与同一输出物理信道关联的虚拟信道。两个包的微片在输出信道上交错传输，每个包获得信道上一半的微片周期。由于包以全速率到达输入并以半速率离开，微片缓冲区（每个容量为三个微片）被填满，

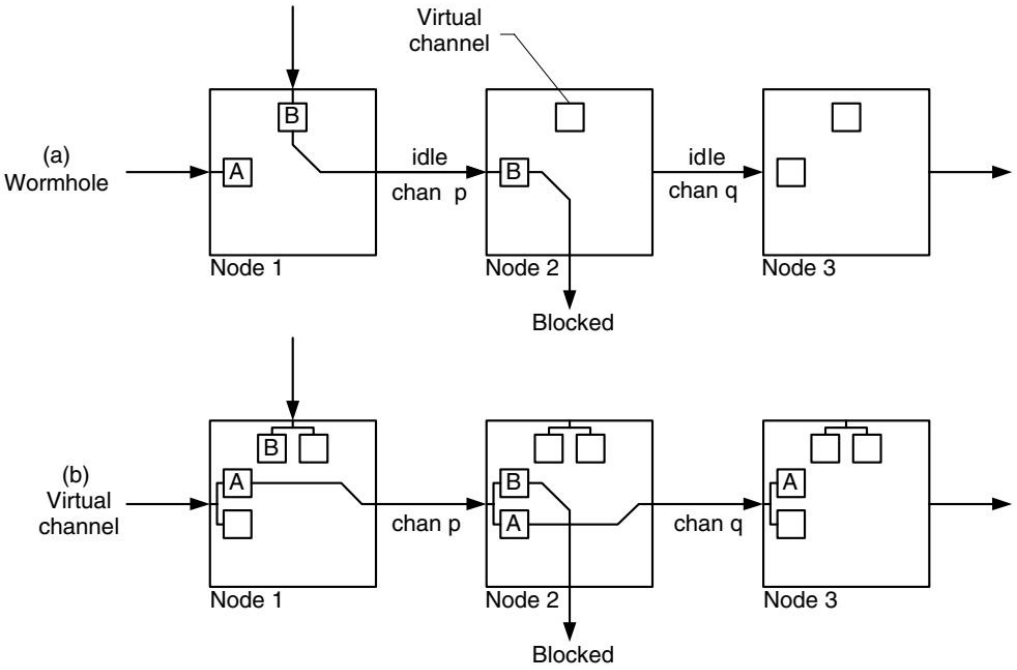


图 13.5 虫洞流控与虚拟通道流控在阻塞条件下的差异

迫使输入也节流到半速率。然而，输入上包之间的间隙并不意味着物理信道空闲，因为另一个虚拟信道可以使用这些周期。

当两个虚拟信道上的包在单条物理信道上交错传输微片时（如图 13.7 所示），这条物理信道成为瓶颈，影响这些包在上游和下游的带宽。在下游信道（节点 2 的南侧和东侧），头微片以全速率传播，分配虚拟信道，但体微片每隔一个周期才跟随一次。空闲周期可供其他包使用。类似地，一旦节点 1 上的微片缓冲区填满，阻塞会限制每个包在紧邻上游的信道上每隔一个周期才能发送微片。这种带宽降低可能导致上游节点的缓冲区填满，将阻塞和带宽降低进一步向上游传播。对于长包，这种阻塞可能一直回溯到源节点。与下游情况一样，上游信道上的空闲周期可供其他包使用。

公平带宽仲裁在通道上交织竞争分组的 flit（图 13.6），导致比赢家通吃带宽分配更高的平均延迟。赢家通吃带宽分配将全部带宽分配给一个分组，直到它完成或被阻塞，然后再服务其他分组（图 13.8）。在我们的两个分组示例中，使用交织时，两个分组都看到争用延迟，这实际上使它们的串行化延迟加倍。然而，使用赢家通吃带宽仲裁时，首先一个分组获得全部带宽，然后另一个分组获得全部带宽。结果是，一个分组没有争用延迟，而另一个分组看到的争用延迟等于第一个分组的串行化延迟——与交织时相同。当然，如果分组 A 在完成传输之前被阻塞，它将放弃通道，分组 B 将使用空闲周期。使带宽仲裁不公平可减少虚拟通道流控的平均延迟，而不会造成吞吐量损失。

图 13.9 的框图说明了为实现虚拟通道而复制的状态。6 该图显示了一个具有两个输入端口、两个输出

与图 13.6 相同，数据包 A 和 B 同时到达两条共享同一物理通道的虚拟通道。不过，在“赢家通吃”仲裁下，A 会先传送完全部微片，然后才释放物理通道供 B 使用。因此，A 的延迟减少了 7 个周期，而 B 的延迟不受影响。

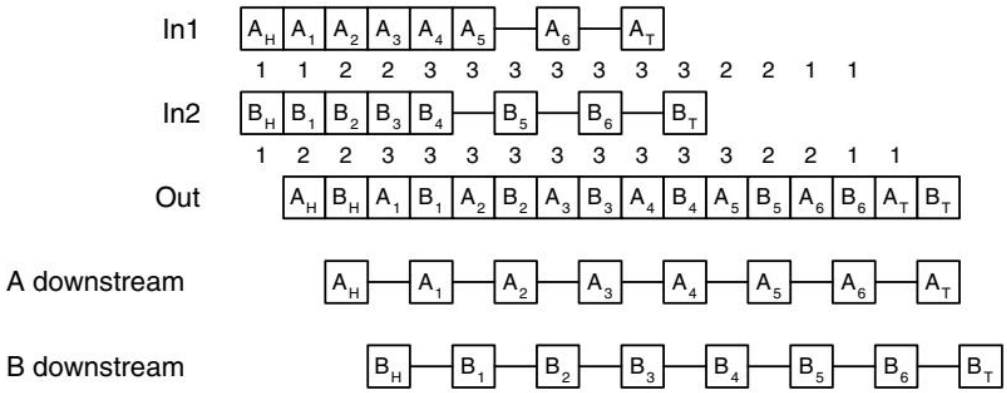


图 13.6 两个虚拟通道在单一物理通道上交织它们的 flit。分组 A 在输入 1 到达的同时，分组 B 在输入 2 到达，并且两者请求相同的输出。两个分组都获取了与输出通道关联的虚拟通道，但必须在逐个 flit 的基础上竞争带宽。在公平带宽分配下，两个分组的 flit 交织在一起。到达的 flit 下方的数字显示了对应虚拟通道输入缓冲区中的 flit 数量，该缓冲区的容量为 3 个 flit。当输入缓冲区满时，到达的 flit 被阻塞，直到离开的 flit 腾出空间。在下游链路上，每个分组的 flit 每隔一个周期才可用。

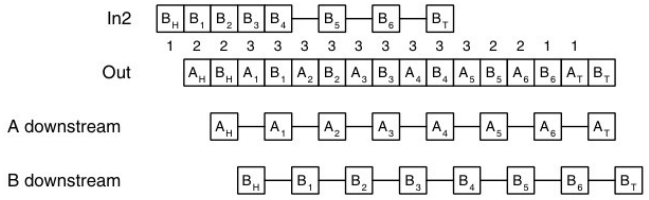


Figure 13.6 Two virtual channels interleave their flits on a single physical channel. Packet *A* arrives on input 1 at the same time that packet *B* arrives on input 2, and both request the same output. Both packets acquire virtual channels associated with the output channel but must compete for bandwidth on a flit-by-flit basis. With fair bandwidth allocation, the flits of the two packets are interleaved. The numbers under the arriving flits show the number of flits in the corresponding virtual channel's input buffer, which has a capacity of 3 flits. When the input buffer is full, arriving flits are blocked until room is made available by departing flits. On downstream links, the flits of each packets are available only every other cycle.

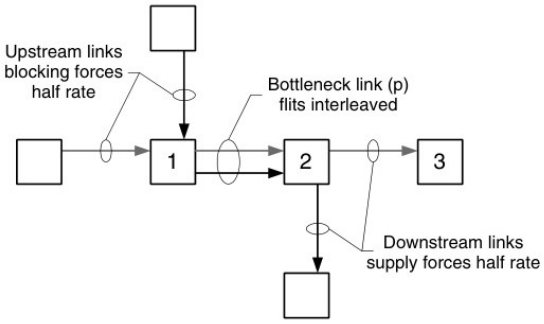
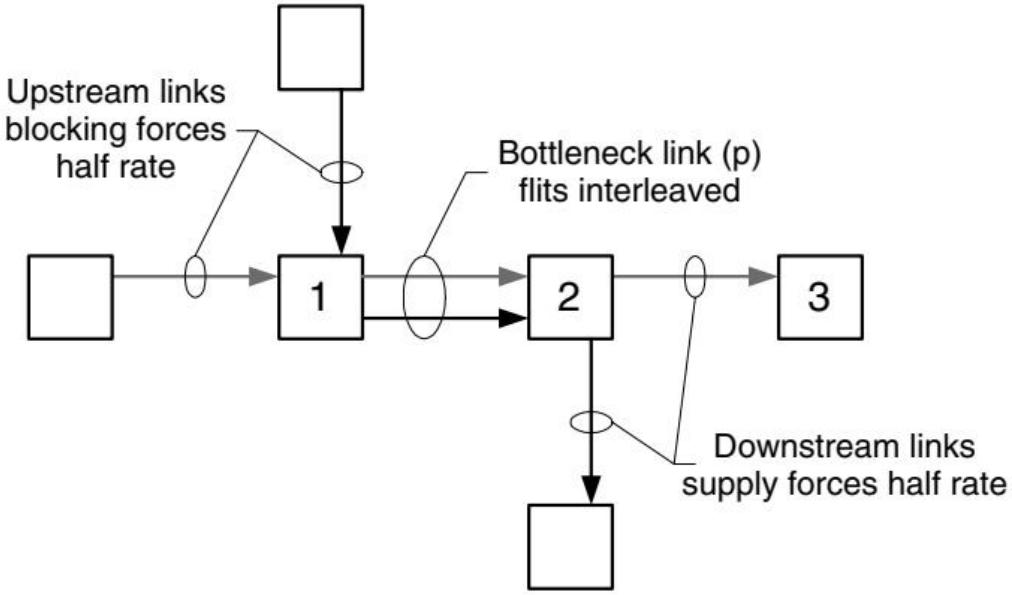


图 13.7 两个虚拟通道共享瓶颈链路带宽时的上游与下游影响



两个虚拟通道共享瓶颈链路的带宽会降低上游和下游的带宽。分组 A（灰色）和分组 B（黑色）共享瓶颈链路 p 上的带宽。每个分组获得链路带宽的 50%。拓扑和路由与图 13.5 相同。瓶颈链路输出的 flit 供应将这些分组限制为使用瓶颈下游链路带宽的不超过 50%。此外，一旦节点 1 上虚拟通道的 flit 缓冲区被填满，瓶颈通道上游的带宽因阻塞而降低到 50%。在其他虚拟通道上传输的分组可以使用瓶颈留下的空闲带宽。

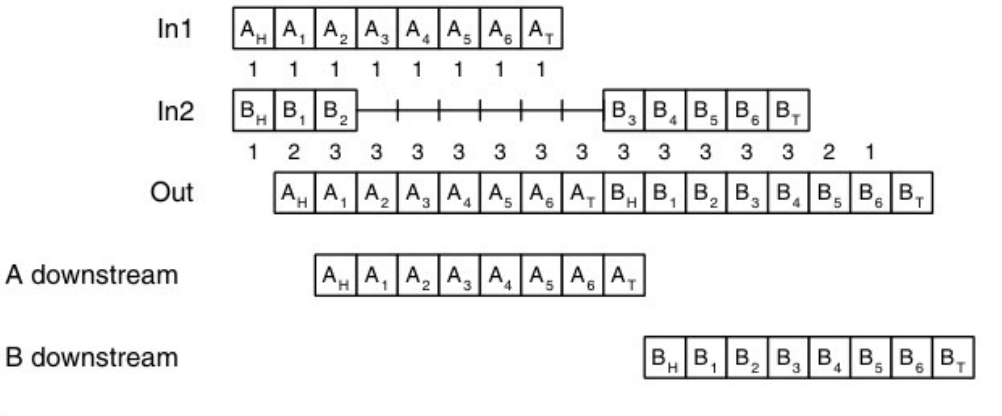


图 13.8　赢者独占仲裁下两个虚拟通道的数据包传输

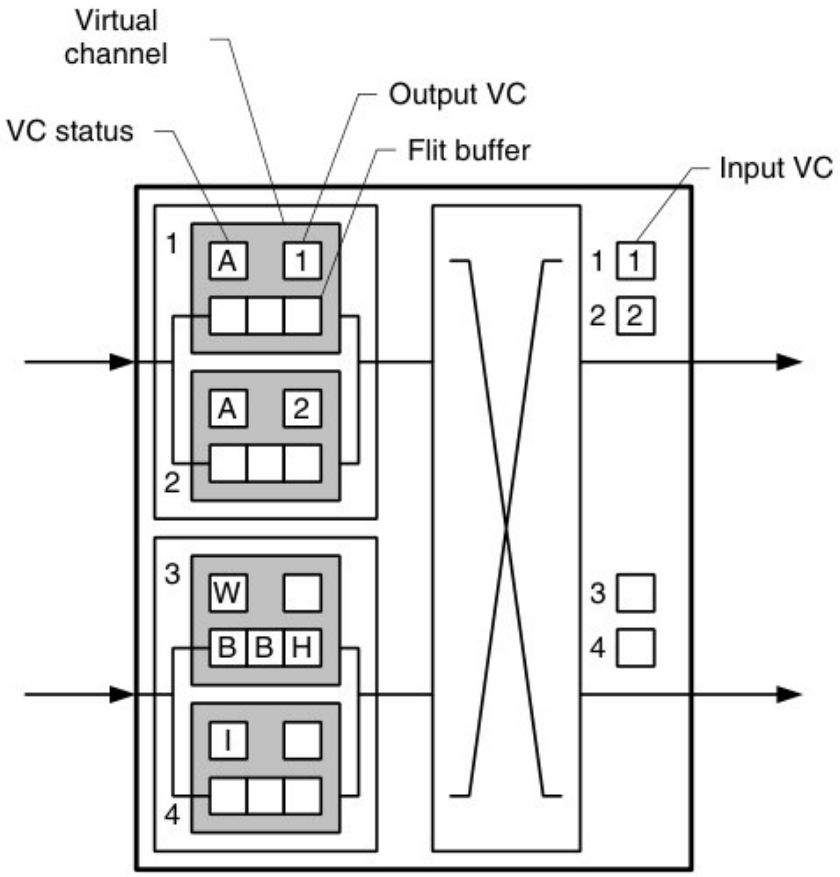
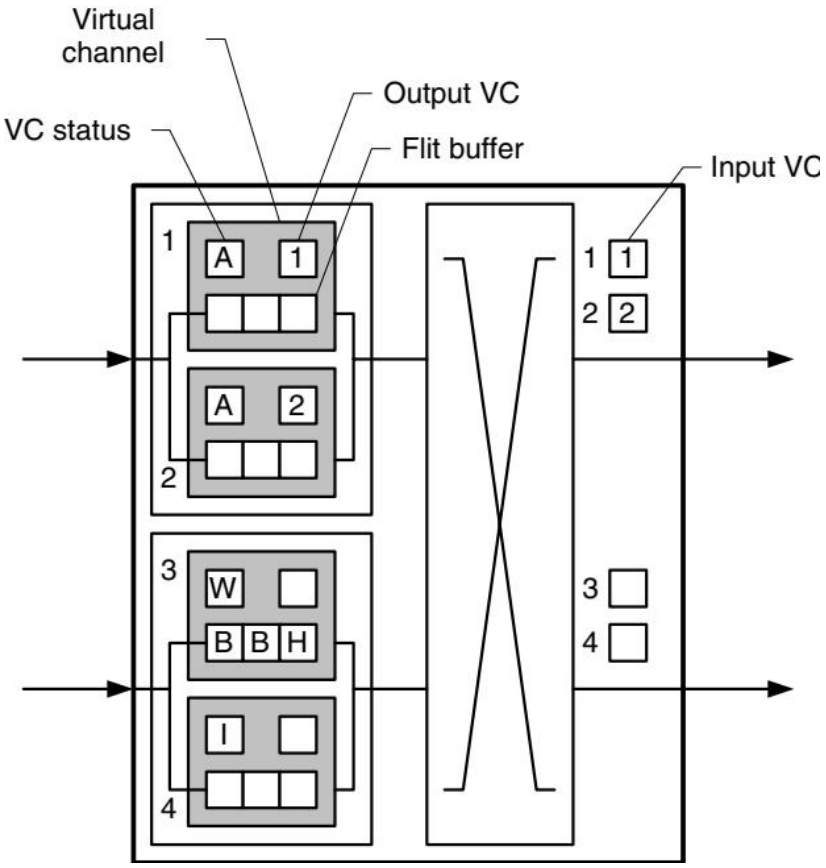


图 13.9 虚拟通道路由器框图及每条虚拟通道关联的状态

In1	AH	A1	A2	A3	A4	A5	A6	AT
1	1	1	1	1	1	1	1	
In2	BH	B1	B2					
1	2	3	3	3	3	3	3	
Out	AH	A1	A2	A3	A4	A5	A6	AT
B3	B4	B5	B6	BT				
3	3	3	3	3	3	3	2	
1								
A downstream	AH	A1	A2	A3	A4	A5	A6	AT
B1	B2	B3	B4	B5	B6	BT		
B downstream	BH	B1	B2	B3	B4	B5	B6	BT



虚拟通道路由器的框图，显示了与每个虚拟通道关联的状态。每个输入虚拟通道包括通道状态、分配给当前分组的输出虚拟通道以及 flit 缓冲区。每个输出虚拟通道标识分配给它的输入虚拟通道（如果有）。在此配置中，输入虚拟通道 1 处于活动状态，将分组转发到输出虚拟通道 1。类似地，输入虚拟通道 2 将分组转发到输出虚拟通道 2。与下部物理通道关联的输入虚拟通道 3 正在等待分配输出虚拟通道。输入虚拟通道 4 空闲，等待分组的到达。

端口，每个物理通道有两个虚拟通道。每个输入虚拟通道包括所有通道状态的完整副本，包括状态寄存器（空闲、等待或活动）、指示分配给当前分组的输出虚拟通道的寄存器以及 flit 缓冲区。7 对于每个输出虚拟通道，单个状态寄存器记录输出虚拟

通道是否被分配，如果已分配，则记录分配给它的输入虚拟通道。该图显示了一种配置，其中与上部输入端口关联的输入虚拟通道 1 和 2 处于活动状态，将分组转发到与上部输出端口关联的输出虚拟通道 1 和 2。输入虚拟通道 3 正在等待分配输出虚拟通道，输入虚拟通道 4 空闲。虚拟通道流控在两个维度（虚拟通道和每个虚拟通道的 flit 数）上组织缓冲区存储，为我们分配缓冲区提供了自由度。给定每个路由器输入的固定 flit 缓冲区数量，我们可以决定如何在这两个维度上分配这些缓冲区。图 13.10 说明了每个输入有 16 个 flit 缓冲区存储的情况。此存储可以组织为具有 16 个 flit 存储的单个虚拟通道、两个 8-flit 虚拟通道、四个 4-flit 虚拟通道、八个 2-flit 虚拟通道或十六个 1-flit 虚拟通道。一般来说，使虚拟通道的深度超过覆盖往返信用延迟所需的 flit 数量（以便在单个虚拟通道运行时实现全吞吐量）几乎不会带来性能提升。（参见第 16.3 节。）因此，当增加可用的缓冲区存储时，通常更好的做法是添加更多虚拟通道，而不是向每个虚拟通道添加 flit。8

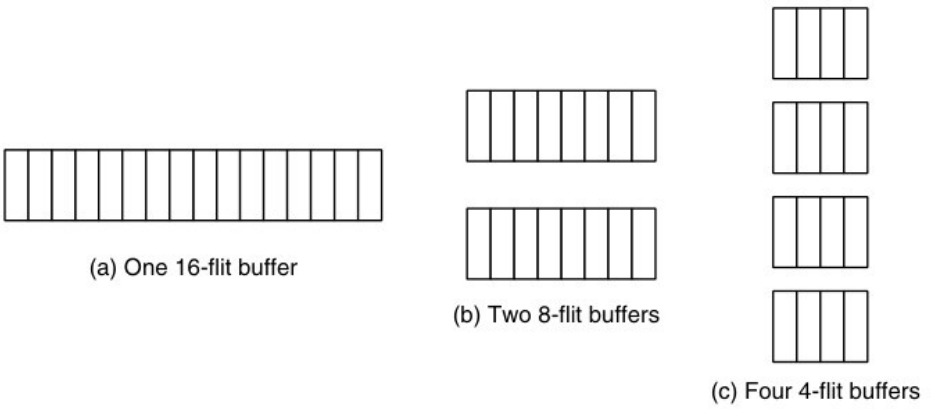
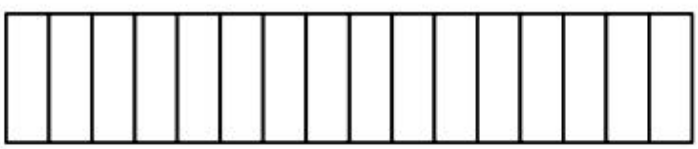
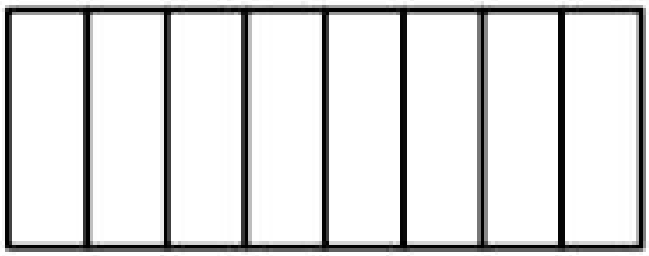


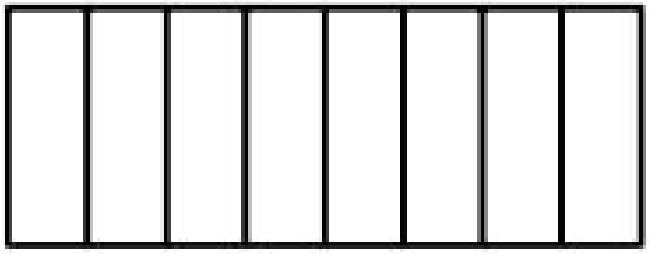
图 13.10 将输入端口的 16 flit 缓冲空间划分为不同数量的虚拟通道

虚拟通道是互连网络中的瑞士军刀 ™。它们是一种实用工具，可用于解决各种各样的问题。如上所述，它们允许活跃数据包绕过被阻塞的数据包，从而提高网络性能。正如我们将在第 14 章中看到的，它们被广泛用于避免互连网络中的死锁。它们既可以用来对抗网络本身的死锁，也可以用来对抗高层协议中依赖关系导致的死锁。通过将不同类别的流量分配到不同的虚拟通道，并优先将带宽分配给高优先级流量类别，它们可以在互连网络中提供多个优先级或服务级别。甚至可以通过隔离具有不同目的地的流量，使网络互不干扰。

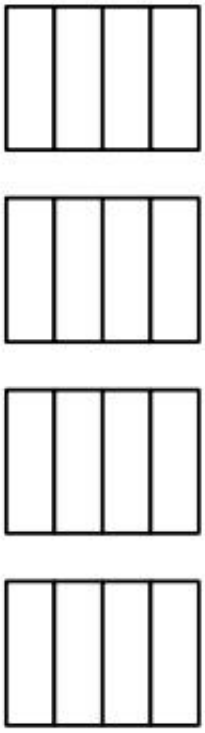


（b）两个 8 微片缓冲区





（b）两个 8 微片缓冲区



（c）四个 4 微片缓冲区

（c）四个 4 微片缓冲区

图 13.10

如果输入端口有 16 个微片的缓冲区存储空间，则可以将该存储空间安排为单个具有 16 微片队列的虚拟通道，或 2 个具有 8 微片队列的虚拟通道，或 4 个具有 4 微片队列的虚拟通道。

13.3 缓冲区管理与背压

所有使用缓冲的流控方法都需要一种手段来传达下游节点缓冲区的可用性。这样，上游节点就能确定何时有缓冲区可用于容纳下一个要传输的微片（或对于存储转发或直通而言的数据包）。这种类型的缓冲区管理通过在上游节点必须停止发送微片时（因为所有下游微片缓冲区都已满）通知它们来提供背压。目前常用的低级流控机制有三种来提供这种背压：基于信用的、开/关的和确认/否认的。我们依次考察每一种。

13.3.1 基于信用的流控

在基于信用的流控中，上游路由器维护一个计数器，记录下游每个虚拟通道中空闲微片缓冲区的数量。然后，每当上游路由器转发一个微片，从而消耗一个下游缓冲区时，它就递减相应的计数。如果计数达到零，则所有下游缓冲区都已满，在缓冲区变得可用之前不能再转发微片。一旦下游路由器转发了一个微片并释放了相关缓冲区，它就向上游路由器发送一个信用，使缓冲区计数递增。

图 13.11 展示了基于信用的流控的时间线。就在时间 t_1 之前，通道下游端（节点 2）的所有缓冲区都已满。在时间 t_1，节点 2 发送一个微片，释放一个缓冲区。一个信用被发送到节点 1，以表示该缓冲区可用。该信用在时间 t_2 到达，经过短暂的处理间隔后，导致在时间 t_3 发送一个微片，并在时间 t_4 被接收。短暂间隔后，该微片在时间 t_5 离开节点 2，再次释放一个缓冲区并向节点 1 发送一个信用。

从时间 t_1 发送信用到时间 t_5 为同一缓冲区发送信用之间的最小时间就是信用往返延迟 t_{crt}。这个延迟包括往返线路延迟和两端的额外处理时间，是任何路由器的关键参数，因为它决定了流控机制所能支持的最大吞吐量。如果该虚拟通道上只有一个微片缓冲区，则每个微片在发送前都必须等待该单一缓冲区的信用。这将把通道的最大吞吐量限制为每个 t_{crt} 不超过一个微片。这对应于比特率 $\frac{L_f}{t_{crt}}$，其中 L_f 是一个微片的比特长度。如果该虚拟通道上有 F 个微片缓冲区，则可以在等待原始缓冲区的信用之前发送 F 个微片，从而给出每 t_{crt} F 个微片的吞吐量，即每秒 $\frac{FL_f}{t_{crt}}$ 比特。因此，我们看到，为了防止低级流控限制带宽为 b 的通道上的吞吐量，我们需要：

$$F \geq \frac{t_{\text{crt}} b}{L_f}. \tag{13.1}$$

图 13.11 基于信用的流控时间线。在时间 t_1，通道节点 2 端的所有缓冲区都已满，节点 2 发送一个微片，释放一个缓冲区。同时，该缓冲区的信用被发送到节点 1，并在

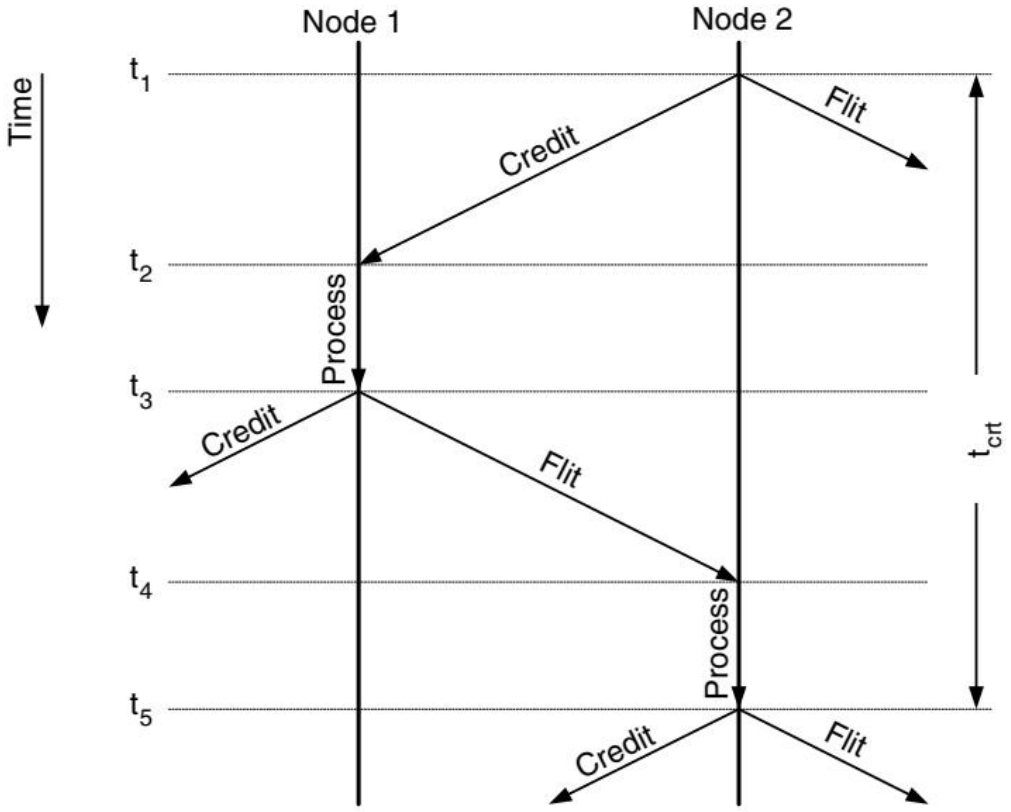


图 13.11 基于信用的流控时序：信用从节点 2 返回节点 1 后释放并重新使用缓冲区

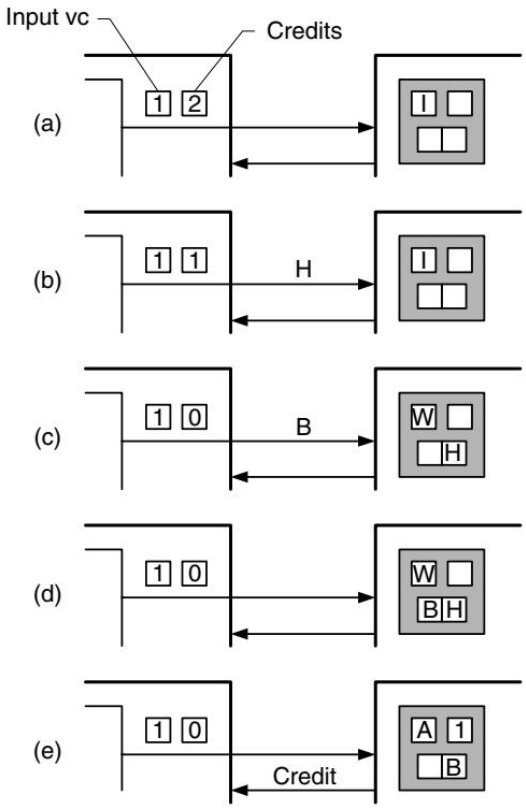
t_2 被接收。经过短暂的处理延迟后，节点 1 在 t_3 发送一个微片以占用信用所授予的缓冲区，该微片在 t_4 被接收。到达后不久，该微片在 t_5 离开节点 2，再次释放缓冲区，从而为节点 1 生成另一个信用。同一缓冲区的连续信用之间的最小时间是信用往返时间 t_{Crt}。

图 13.12 说明了基于信用的流控机制。序列从图 13.12（a）开始。上游节点 1 对下游节点 2 上所示的虚拟通道有两个信用。在图 13.12（b）中，节点 1 发送头微片并将其信用计数递减到 1。头微片在图 13.12（c）中到达节点 2，同时，节点 1 发送数据包的第一个体微片，将其信用计数递减到零。由于没有信用，节点 1 在图 13.12（d）体微片到达时以及（e）节点 2 发送头微片时无法发送任何微片。发送头微片释放一个微片缓冲区，节点 2 向节点 1 发送一个信用，以授予其使用该空闲缓冲区的权限。该信用在图 13.12（f）中到达，使节点 1 上的信用计数递增，同时体微片被向上游发送，并且第二个信用由节点 2 向下游发送。由于信用计数非零，节点 1 能够转发尾微片并通过清除输入虚拟通道字段来释放虚拟通道（图 13.12[g]）。尾微片在图 13.12（h）中由节点 2 转发，返回一个最终信用，使节点 1 上的计数在图 13.12（i）中回到 2。

基于信用的流量控制的一个潜在缺点是微片与信用之间的一一对应关系。每向下游发送一个微片，最终都要向上游返回一个相应的信用。这需要大量的上游信令，特别是对于小微片，可能带来很大的开销。

节点 1

节点 2



13.3.2 开/关流量控制

在某些情况下，开/关流量控制可以大大减少上游信令的数量。使用这种方法时，上游状态是一个单一的控制位，表示是否允许上游节点发送（开）或不发送（关）。只有当需要改变此状态时，才向上游发送信号。当控制位为开且空闲缓冲区数量低于阈值 F_{off} 时，发送关信号。如果控制位为关且空闲缓冲区数量升至阈值 F_{on} 以上，则发送开信号。

图 13.13 展示了开/关流量控制的时间线。在时间 t_1，节点 1 发送一个微片，导致节点 2 上的空闲缓冲区数量低于 F_{off}，从而在 t_2 触发关信号的传输。在 t_3，关信号到达节点 1，经过短暂的处理延迟后，在 t_4 停止发送微片。在 t_1 和 t_4 之间的时间 t_{rt} 内，发送了额外的微片。必须设置下限，使得

$$F_{\text{off}} \geq \frac{t_{rt} b}{L_f} \tag{13.2}$$

以防止这些额外的微片溢出剩余的微片缓冲区。

在发送若干微片后，节点 2 清空足够的缓冲区，使空闲缓冲区计数在 t_6 升至 F_{on} 以上，并向节点 1 发送开信号。节点 1 在 t_7 收到此信号，并在 t_8 恢复发送微片。由开信号触发的第一个微片在 t_9 到达。为了防止节点 2 在 t_6 发送开信号和 t_9 收到微片之间耗尽要发送的微片，必须设置开限制，使得

$$F - F_{\text{on}} \geq \frac{t_{rt} b}{L_f}. \tag{13.3}$$

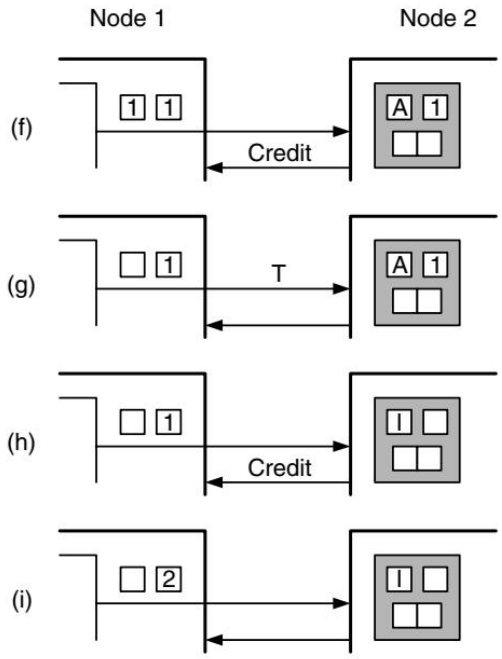


图 13.12 展示基于信用的流量控制操作的序列。每个子图代表一个微片时间。在（a）中，节点 1 对于节点 2 上虚拟通道的两个下游微片缓冲区有两个信用。在（b）和（c）中，节点 1 使用这两个信用发送了两个微片，一个头微片和一个体微片。这些微片分别在（c）和（d）中到达并被缓冲。此时节点 1 没有信用，无法发送更多微片。头微片的信用在（e）中返回，在（f）中增加信用计数，并导致尾微片在（g）中发送。体信用在（f）中返回，尾信用在（h）中返回，使节点 1 在（i）中再次拥有两个信用（对应于节点 2 上的两个空微片缓冲区）。

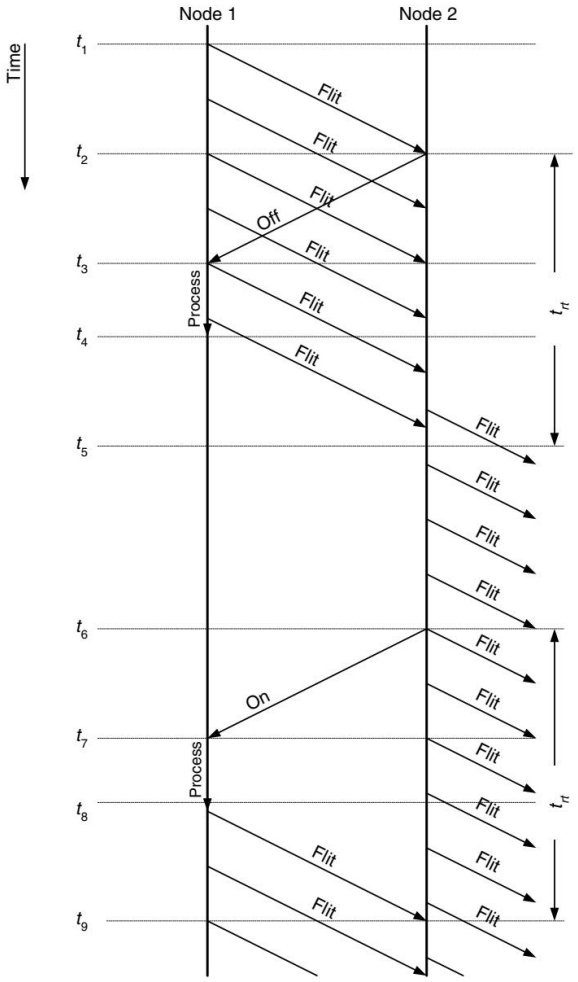


图 13.13 开/关流量控制的时间线。在 t_1 发送的微片导致节点 2 上的空闲缓冲区计数低于其限制 F_{off}，进而导致节点 2 向节点 1 发送关信号。在收到此信号之前，节点 1 发送额外的微片，节点 2 将这些微片缓冲在发送关信号时空闲的 F_{off} 个缓冲区中。在 t_6，节点 2 上的空闲缓冲区计数升至 F_{on} 以上，导致节点 2 向节点 1 发送开信号。在 t_6 发送此开信号与 t_9 收到来自节点 1 的下一个微片之间的间隔内，节点 2 从 t_6 时已满的 $F - F_{on}$ 个缓冲区中发送微片。

开/关流量控制要求缓冲区数量至少为 $\frac{t_{rt}b}{L_f}$ 才能工作，否则方程 13.2 无法满足。要以全速运行，需要此数量的两倍缓冲区，以便方程 13.3 也能满足（$F_{on} \geq F_{off}$）：

$$F \geq F_{\text{on}} + \frac{t_{rt}b}{L_f} \geq F_{\text{off}} + \frac{t_{rt}b}{L_f} \geq \frac{2t_{rt}b}{L_f}.$$

在缓冲区数量充足的情况下，开/关流量控制系统可以在很少的上游信令下运行。

13.3.3 确认/否认流量控制

基于信用的流量控制和开/关流量控制都要求在缓冲区变空（触发信用或开信号）与微片到达占用该缓冲区之间有一个往返延迟 t_{rt}。确认/否认流量控制将此缓冲区空闲时间的最小值降至零，平均空闲时间降至 $t_{rt}/2$。不幸的是，没有净收益，因为缓冲区被额外保留 t_{rt} 时间等待确认，使得确认/否认流量控制在缓冲区利用方面不如基于信用的流量控制高效。它在带宽利用方面也效率低下，因为它发送微片只是为了在没有可用缓冲区时丢弃它们。

使用确认/否认流量控制时，上游节点不保留指示缓冲区可用性的状态。上游节点乐观地在微片可用时发送它们。9 如果下游节点有可用缓冲区，它接受微片并向发送确认（ack）。如果微片到达时没有可用缓冲区，下游节点丢弃微片并发送否认（nack）。上游节点保留每个微片直到收到确认。如果收到否认，则重传该微片。

在多个微片可能同时传输的系统中，确认或否认与微片之间的对应关系通过排序来维持——确认和否认按照与其对应的微片相同的顺序接收。然而，在这样的系统中，否认一个微片会导致下游节点乱序接收微片。下游节点然后必须通过保留后续微片直到较早的微片成功重传来重新排序微片。

图 13.14 的时间线展示了确认/否认流量控制。在时间 t_1，节点 1 向节点 2 发送微片 1，不知道没有可用缓冲区。此微片在时间 t_2 到达，此时节点 2 向节点 1 发送否认。同时，节点 1 开始向节点 2 发送微片 2。在时间 t_3，节点 1 收到否认，但在处理之前开始向节点 2 发送微片 3。它必须等到微片 3 的传输在 t_4 完成后，才能重传微片 1。微片 1 最终在 t_5 在微片 2 和 3 之后被接收。节点 2 必须缓冲微片 2 和 3 直到收到微片 1，以避免乱序发送微片。

由于其在缓冲区和带宽方面的低效，确认/否认流控很少被使用。相反，基于信用的流控通常用于缓冲区数量较少的系统，而开/关流控则被大多数拥有大量微片缓冲区的系统所采用。

13.4 微片预留流控

虽然传统的虫洞网络大大降低了数据包通过互连网络的延迟，但理想化的路由器行为可能与流水线硬件实现存在显著差异。流水线将微片路由的各个阶段分解为若干更小的步骤，这增加了跳数时间。考虑这些流水线延迟和传播延迟，可以准确了解缓冲区利用率。

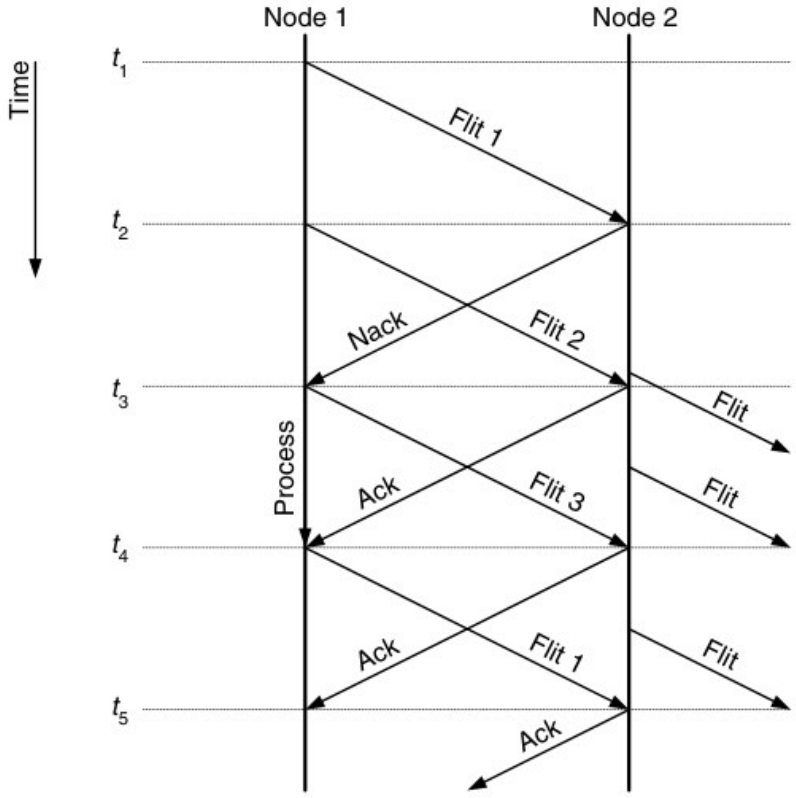


图 13.14 确认／否定确认（ack/nack）流控的时间线

图 13.15 展示了一个采用基于信用流控的虫洞网络的缓冲区利用率示例。最初，一个微片从当前节点发送到数据包路由中的下一个节点。该微片占用缓冲区，直到被转发到下一跳；然后一个信用沿反向通道发送回当前节点，这需要线传播时间 $T_{w,credit}$。该信用由当前节点处理，经历信用流水线延迟，然后可供另一个微片使用。一旦这个微片收到信用，它必须经过微片流水线才能访问通道。最后，微片在 $T_{w,data}$ 时间内通过通道传播并占用缓冲区。如图所示，实际的缓冲区使用只占整体时间的一小部分。回收信用并发出另一个微片以占用缓冲区所需的剩余时间称为周转时间。较低的缓冲区利用率会降低网络吞吐量，因为可用于绕过阻塞消息和吸收流量变化的缓冲区更少。微片预留流控可以在实际实现中将周转时间减少到零，并隐藏微片流水线延迟。

微片预留通过分离控制网络和数据网络，隐藏了与流水线路由器实现相关的开销。控制微片跑在数据微片前面，以预留网络资源。当数据微片到达时，它们已经被分配了出站虚拟通道，可以以很小的开销继续前进。预留还简化了信用的传递，使缓冲区可以实现零周转时间。当然，并不总是能够提前预留资源，特别是在严重拥塞的情况下。在这些情况下，数据微片只是在路由器处等待，直到资源被预留——这与标准虫洞路由器的行为相同。

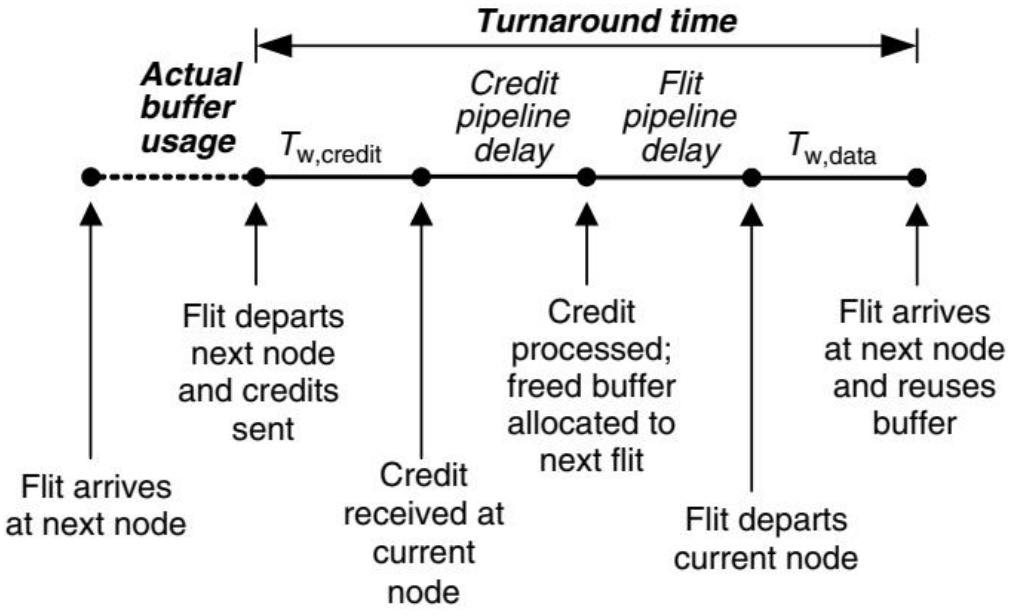


图 13.15 典型虫洞路由器中实际延迟对缓冲区利用率的图示。随着缓冲区周转时间的增加，缓冲区实际使用的时间比例减少。

图 13.16 展示了一个微片预留数据包。如图所示，微片的控制部分与数据部分分离。控制头微片持有与典型虚拟通道流控头微片相同的信息：类型字段（控制头）、虚拟通道和路由信息。一个额外的字段 t_{d0} 表示到第一个数据微片的时间偏移。这个数据偏移将控制微片与其关联的数据微片耦合起来。例如，当一个控制微片在时间 t = 3 到达节点，且 $t_{d0} = 2$ 时，路由器知道关联的数据微片将在时间 $t = 3 + t_{d0} = 5$ 到达数据网络。通过知道数据的未来到达时间，路由器可以在数据实际到达之前开始为其预留资源。由于路由器知道特定数据微片何时到达，数据微片本身不需要控制字段。

由于数据包可以包含任意数量的数据微片，而控制头微片只有有限数量的数据偏移字段（本例中为一个），因此可以在头微片之后跟随额外的控制体微片。控制体微片类似于典型虫洞路由器中的体微片，它们包含额外的数据偏移字段。例如，在图 13.16 中，控制体微片有两个额外的数据偏移字段，允许每个控制体微片对应两个数据微片。

13.4.1 微片预留路由器

图 13.17 展示了一个微片预留路由器。在我们描述路由器中模块的详细操作之前，先讨论路由器的基本操作。控制微片到达路由器并立即传递到路由逻辑。当控制头微片到达路由逻辑时，计算其输出虚拟通道（下一跳）并与传入的虚拟通道关联。当后续控制体微片到达该虚拟通道时，它们被标记为与控制头微片相同的输出通道。所有控制微片然后通过交换机到达其输出端口。到目前为止，控制微片的处理与标准虚拟通道路由器中的微片相同。

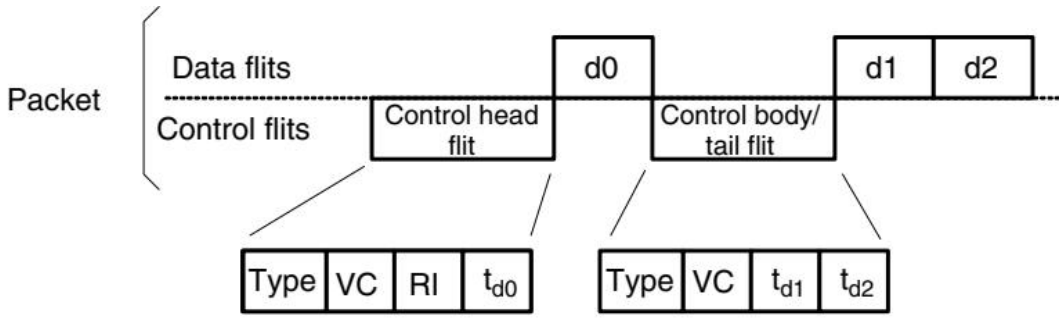


图 13.16 一个微片预留数据包，展示了控制微片和数据微片之间的分离。控制微片通过到达时间的偏移与特定数据微片关联（例如，对于在时间 t 到达的控制头微片，其关联的数据微片 d0 在时间 $t + t_{d0}$ 到达）。

控制微片到达其输出端口后，经过输出调度器，然后被转发到下一跳。输出调度器负责为控制微片的每个关联数据微片（由数据偏移字段给出）在下一跳预留缓冲区空间。下一路由器的缓冲区使用计划保存在输出预留表中，该表由传入的信用不断更新。一旦输出调度器分配了控制微片的所有关联数据微片，并在输出预留表中标记了这些预留，控制微片就被转发到其下一跳。

输入预留表在路由器内部建立控制微片与其数据微片之间的关联。当控制微片通过路由逻辑时，其目的地也会在输入预留表中标记。此外，控制微片的每个数据偏移字段也会在此表中标记。一旦输出调度器为某个微片安排了出发时间，该信息就会回传给与该微片到达的输入端口相关联的输入调度器。这样，输入调度器就能知道哪些周期已安排微片出发，从而指示其缓冲区何时将变为空闲。

13.4.2 输出调度

输出调度器负责确定与每个控制微片相关联的每个数据微片的未来出发时间。数据微片要出发，必须预留物理输出通道，并且输出调度器还必须确保下一跳路由器中有足够的缓冲区空间来存储该数据微片。为了跟踪未来输出通道的利用率和下一跳缓冲区

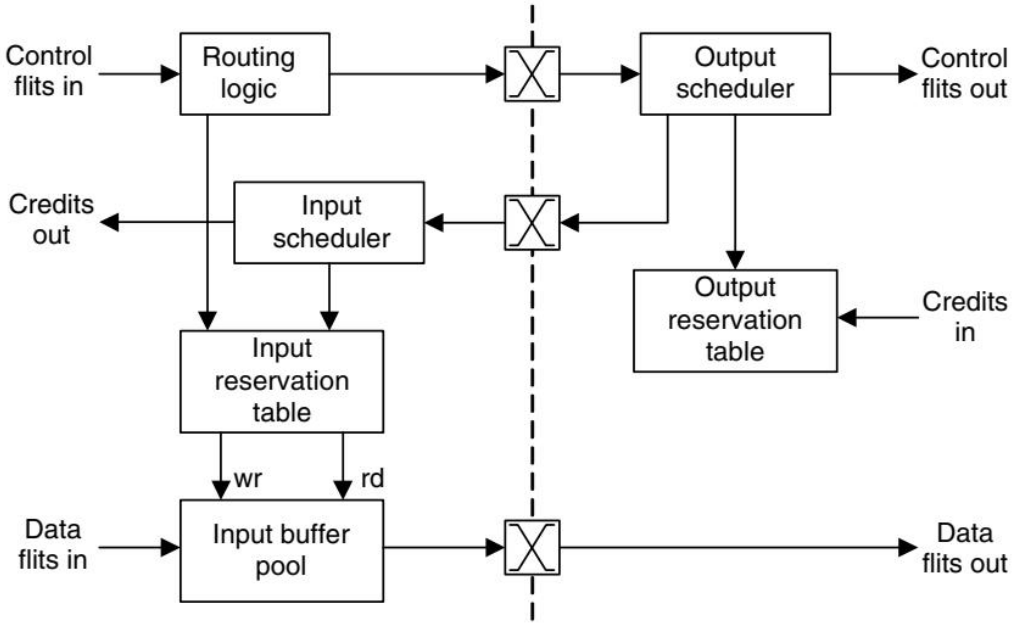


图 13.17 微片预留流控架构。为简单起见，仅显示单个输入和单个输出（由虚线分隔的每输入和每输出结构）。

的可用性，输出调度器维护输出预留表（图 13.18）。该表维护未来多个周期的通道使用情况和缓冲区可用计数。

Output channel \ Time		8	9	10	11	12	13	14	15	16	17
East channel	Channel busy			X							
	Free buffers on next node	2	1	1	0	1	2	3	4	4	4

(a)

Output channel \ Time		8	9	10	11	12	13	14	15	16	17
East channel	Channel busy			X		X					
	Free buffers on next node	2	1	1	0	0	1	2	3	3	3

(b)

图 13.18 输出预约表在调度前后的状态

输出预留表的一个示例状态如图 13.18（a）所示。一个新的控制微片在时间 t = 0 到达，并确定其数据微片的到达时间。在此示例中，单个新数据微片由 $t_{d0} = 9$ 表示。10 因此，该数据微片将在周期 $t + t_{d0} = 9$ 到达，目的地为东输出。此信息被转发到输出调度器和输入预留表。（输入预留表将在下一节讨论。）

输出调度器尝试为新数据微片寻找可用的出发时间。周期 10 不可用，因为输出通道已被标记为忙。周期 11 也不可用，但在此情况下，是因为下一跳没有空闲缓冲区。第一个可用周期是 12，因此输出调度器更新输出预留表，将周期 12 的通道标记为忙（图

13.18[b])。此外，该微片占用下一节点的一个缓冲区，因此从周期 12 开始，空闲缓冲区数量递减。11 一旦来自下一路由器的信用返回，该缓冲区最终将被加回输出预留表计数，这也在下一节讨论。

现在输出预留已完成，预留信息被传递给输入调度器。一旦与控制微片相关联的所有数据微片都被预留，控制微片就被转发到下一跳。然而，其数据微片的相对时间可能在输出调度过程中发生了变化，因此其数据偏移字段必须更新。如果本例中的控制微片离开路由器，然后在 t = 2 时到达下一路由器，则其 t_{d0} 字段变为 10。这是因为输出调度器使数据微片相对于控制微片额外延迟了一个周期，其出发时间为周期 12——因此，$t_{d0} = 12 - 2 = 10$。

13.4.3 输入调度

输入调度器和输入预留表组织数据微片通过微片预留路由器的移动。通过访问传入控制微片的数据偏移字段，路由逻辑在输入预留表中标记数据微片的到达时间及其目的端口（图 13.19）。一旦输出调度器为数据微片预留了输出时间，该预留信息就被回传给输入调度器。输入调度器用相应数据微片的出发时间和输出端口更新输入预留表。此外，输入调度器向上一路由器发送信用，以指示数据微片的缓冲区何时再次可用。

Input channel \ Time		8	9	10	11	12	13	14	15	16	17
West channel	Flit arriving?			X							
	Buffer in			5							
	Departure time			+2							
	Buffer out					5					
	Output channel					E					

图 13.19 输入预约表及 flit 缓冲区的调度示例

继续上一节的示例，当新数据微片的到达时间（周期 9）在路由逻辑中被解码时，此信息被传递给输入预留表（图 13.19）。在周期 9 到达的数据微片在周期 10 开始时被锁存，因此到达及其输出通道在表中标记。此时表中的其余条目仍未确定。

当新数据微片的预留信息从输出调度器返回时，微片的出发时间被存储在输入预留表中。此时，向上一节点发送信用，指示在周期 12 出发。尽管微片预留保证到达的数据微片可以被缓冲，但实际的缓冲区位置直到数据微片到达前一个周期才被分配。分配后，表中的缓冲区输入和缓冲区输出字段被写入。

13.5 文献注释

直通流控由 Kermani 和 Kleinrock[94] 提出。直通是适用于小数据包网络的一种有吸引力的解决方案，并已在 Alpha 21364[131] 等近期网络中使用。Dally 和 Seitz 引入了虫洞流控，并首次在 Torus Routing Chip[56] 上实现。这些思想由 Dally 进一步完善，他还引入了虚拟通道流控 [44, 47]。虫洞和虚拟通道流控都已出现在众多网络中——例如，Cray T3D[95] 和 T3E[162]、SGI Spider[69] 以及 IBM SP 网络 [178, 176]。微片预留流控由 Peh 和 Dally[145] 开发。

13.6 练习题

13.1 包缓冲、微片通道流控。考虑一种流控方法，它像直通流控那样为数据包分配缓冲区，但像虚拟通道流控那样为微片分配通道带宽。这种方法有哪些优点和缺点？构造一个例子，并画出时间-空间图，说明该流控方案与纯直通流控在事件顺序上不同的情况。
13.2 基于信用的流控开销。计算基于信用的流控开销占数据包长度 L 的比例。假设微片大小为 L_f，虚拟通道数为 V。
13.3 共享缓冲池的流控。考虑一个路由器，其微片缓冲区在交换机的所有输入之间共享。也就是说，在任何时刻，存储在每个输入处的微片可以占用该共享缓冲区的一部分。为确保公平性，该共享缓冲区的一部分被静态分配给特定输入，但剩余部分则动态划分。描述一种缓冲区管理技术，用于将该缓冲区的状态传达给上游节点。共享缓冲区的引入是否增加了信令开销?
13.4 单端口输入缓冲池。考虑一个微片预留路由器，其输入缓冲池是单端口的（每周期一次读或写）。描述如何修改第 13.4 节中描述的路由器，以及路由器中是否需要额外的状态（如果需要，是什么状态）。
13.5 微片预留中的同步问题。微片预留路由器中的微片数据仅通过其相对于控制信息的到达来标识。然而，在准同步系统中，路由器之间的时钟可能偶尔“滑动”，导致插入额外的传输延迟周期。如果忽略，这些额外的周期将改变与特定控制微片相关联的数据。提出一种简单的方法来解决这个问题。

注释

- 1. 在存储转发流量控制中，包不被划分为微片。此处显示划分是为了与其他流量控制方法的表述保持一致。
- 2. 这种方法通常称为虚拟直通。此处缩短名称以避免与虚拟通道流量控制混淆。
- 3. 与存储转发一样，直通不将包划分为微片。此处显示划分是为了保持一致。
- 4. 历史上，这种方法被称为“虫洞路由”。然而，它是一种流控方法，与路由无关。因此，我们称这种方法为“虫洞流控”。

- 5. 应提供足够的缓冲区以覆盖两个节点之间的往返延迟和流水线延迟。这在 13.3 节和 16.3 节中有更详细的讨论。
- 6. 虚拟通道路由器的内部组织在第 16 章中更详细地描述。
- 7. 正如我们将在第 17.1 节中看到的，与单个输入关联的虚拟通道的 flit 缓冲区通常组合成单个存储阵列，并且在某些情况下从同一池中动态分配存储。
- 8. 这不是一条硬性规则。虚拟通道流控的性能分析将在第 23 章中更详细地讨论。
- 9. 由于这种行为，确认/否认流量控制通常被称为乐观流量控制。
- 10. 为简单起见，在本示例中我们假设线路传播时间为零。
- 11. 由于输出调度器不知道微片从下一路由器缓冲区出发的时间，最初假设缓冲区被占用，直到另行通知。出于同样的原因，示例中的数据微片即使在输出通道空闲时也不能在周期 10 出发，因为在该时间点之后的每个时间步都必须有一个空闲缓冲区。

第十四章　死锁与活锁

当一组代理（通常是数据包）由于相互等待释放资源（通常是缓冲区或通道）而无法取得进展时，互连网络中就会发生死锁。如果等待代理的序列形成一个循环，则网络发生死锁。作为一个简单的例子，考虑图 14.1 所示的情况。穿越电路交换网络的连接 A 和 B 各自持有两个通道，但在获得第三个通道（目前由另一连接持有）之前无法继续前进。然而，两个连接都无法在完成传输之前释放对方所需的通道。这些连接发生死锁，并将保持此状态直到某种干预。死锁可能发生在各种资源上。在此例中，资源是物理通道。它也可以是虚拟通道或共享数据包缓冲区。

死锁对网络是灾难性的。在一些资源被死锁的数据包占用后，其他数据包会阻塞在这些资源上，使网络运行瘫痪。为了防止这种情况，网络必须使用死锁避免（保证网络不会死锁的方法）或死锁恢复（检测并纠正死锁）。几乎所有现代网络都使用死锁避免，通常通过对相关资源施加顺序并要求数据包按顺序获取这些资源来实现。

一种密切相关的网络病症是活锁。在活锁中，数据包继续在网络中移动，但它们没有朝着目的地取得进展。例如，当允许数据包在网络中采用非最小路由时，这就会成为一个问题——必须通过确定性或概率性保证来确保数据包偏离目的地的误路由次数是有限的。

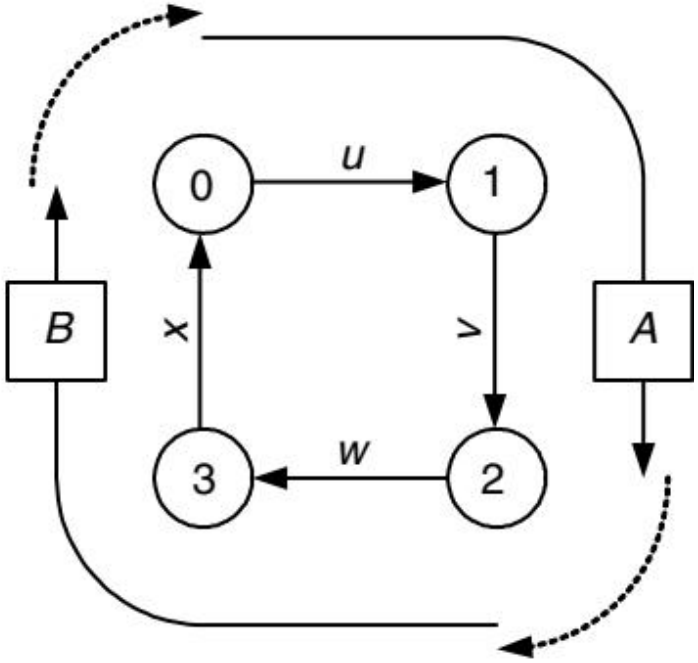


图 14.1　电路交换网络中的死锁。连接 A 持有通道 u 和 v，但在获得通道 w 之前无法取得进展。同时，连接 B 持有通道 w 和 x，但在获得通道 u 之前无法取得进展。两个连接都不会释放对方所需的通道。因此，它们发生死锁。

14.1 死锁

14.1.1 代理与资源

参与死锁的代理和资源因所采用的流控类型而异，如表 14.1 所示。对于电路交换，如图 14.1 所示，代理是连接，资源是物理通道。当连接建立时，它获取物理通道，并且在连接完成之前不会释放任何通道。每个连接可以无限期地持有多个通道，即从源到目的地的路径上的所有通道。对于包缓冲流控方法（如存储转发或虚拟直通），代理是数据包，资源是数据包缓冲区。当数据包的头部在网络中传播时，它必须在每个节点获取一个数据包缓冲区。在任何给定时间点，一个数据包可以无限期地只持有一个数据包缓冲区。每次数据包获取新的数据包缓冲区时，它会在一个短暂的、有限的时间后释放旧的数据包缓冲区。对于微片缓冲流控方法，代理仍然是数据包，但资源是虚拟通道。当数据包的头部前进时，它在每个节点分配一个虚拟通道（控制状态和若干微片缓冲区）。它可以无限期地持有多个虚拟通道，因为如果数据包阻塞，每个虚拟通道中的缓冲区空间不足以容纳整个数据包。

表 14.1 不同流控方式的死锁代理与资源

流控方式	等待主体	资源	持有数量
电路交换	连接	物理通道	多个
数据包缓冲流控	数据包	数据包缓冲区	单个
微片缓冲流控	数据包	虚拟通道	多个

14.1.2 等待与持有关系

代理和资源通过等待和持有关系相关联。例如，考虑图 14.1 的情况。该情况的等待和持有关系如图 14.2（a）所示。连接 A 持有（虚线箭头）通道 u 和 v，并等待（实线箭头）w。类似地，连接 B 也持有两个通道并等待第三个通道。如果一个代理持有一个资源，那么该资源就在等待该代理释放它。因此，每个持有关系都会在相反方向上引发一个等待关系：$holds(a,b) \Rightarrow waitfor(b,a)$。将持有边重新绘制为相反方向的等待边，得到图 14.2（b）的等待图。该图中的循环（阴影部分）表明该配置发生死锁。

图 14.2 中的环路由代理和资源之间交替出现的边构成。从代理指向资源的边表示该代理正在等待该资源。方向相反的边表示该资源被所指的代理持有（因此，该资源正在等待该代理释放）。

当以下情况出现时，这样的环路就会存在，从而发生死锁：

1. 代理在等待访问另一个资源时持有资源且不释放。

2. 等待中的代理之间存在环路，即存在一组代理 $A_0, \ldots, A_{n-1}$，其中代理 A_i 持有资源 R_i，同时等待资源 $R_{(i+1 \mod n)}$，对于 $i = 0, \ldots, n-1$。

图 14.2 图 14.1 死锁示例的等待和持有关系。（a）连接 A 和 B 各自持有两个通道（虚线箭头）并等待第三个通道（实线箭头）。（b）每个持有关系都隐含一个方向相反的等待关系。仅使用等待关系重新绘制该图，揭示了导致死锁的等待环路（阴影部分）。

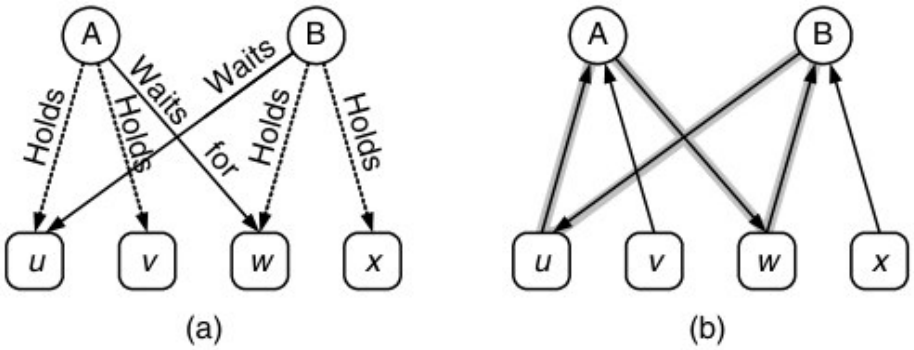
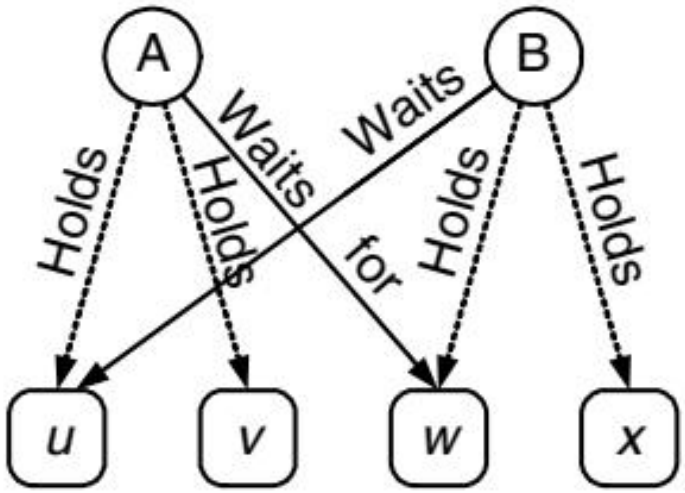
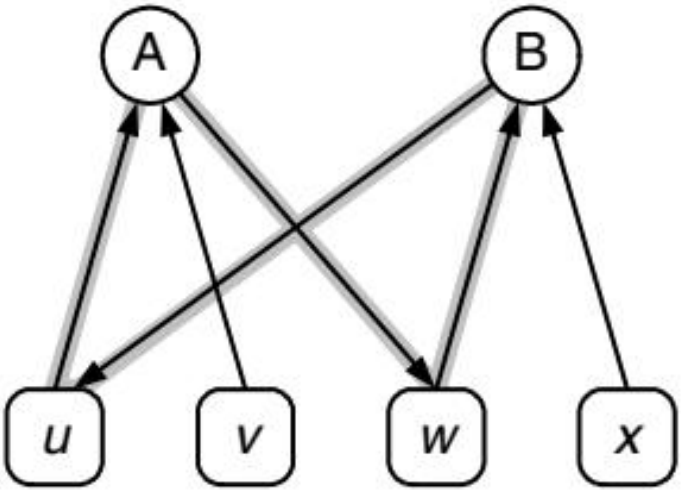


图 14.2 死锁示例中的等待关系与占有关系



(a)



(b)

14.1 死锁

14.1.3 资源依赖

要使两个资源 R_i 和 R_{i+1} 在等待图中相隔两条边，持有资源 R_i 的代理 A_i 必须有可能无限期地等待资源 R_{i+1}。只要持有 R_i 的代理有可能等待 R_{i+1}，我们就说存在从 R_i 到 R_{i+1} 的资源依赖，并记为 $R_i \succ R_{i+1}$。在图 14.1 的示例中，我们有 $u \succ v \succ w \succ x \succ u$。注意，资源依赖（$\succ$）是一种传递关系。如果 $a \succ b$ 且 $b \succ c$，那么 $a \succ c$。

这种资源依赖的环路如图 14.3 的资源（通道）依赖图所示。该图对每个资源（在此情况下为每个通道）都有一个顶点，顶点之间的边表示依赖关系——例如，为了表示 $u \succ v$，我们画一条从 u 到 v 的边。

由于图 14.1 的示例涉及电路交换，资源是物理通道，因此我们的资源依赖图是物理通道依赖图。对于分组缓冲流控，我们将使用分组缓冲依赖图。类似地，对于微片缓冲流控，我们将使用虚拟通道依赖图。

资源依赖图中的资源依赖环路（如图 14.3 所示）表明死锁有可能发生。要实际发生死锁，需要代理（连接）实际上以某种方式获取某些资源并等待其他资源，从而在等待图中产生环路。资源依赖图中的环路是死锁的必要但不充分条件。避免死锁的一种常见策略是消除资源依赖图中的所有环路。这使得等待图中不可能形成环路，从而不可能使网络死锁。

14.1.4 一些示例

考虑图 14.1 的四节点环形网络，但使用分组缓冲流控，每个节点有一个分组缓冲区。在这种情况下，代理是分组，资源是分组缓冲区。这种情况下的分组缓冲依赖图如图 14.4（a）所示。驻留在节点 0 缓冲区（B_0）中的分组在获取 B_1 之前不会释放该缓冲区，因此我们有 $B_0 \succ B_1$，依赖图在这两个缓冲区之间有一条边。

分组缓冲区依赖图中的环路表明该网络存在死锁的可能性。要在此情况下实际构造死锁情形，需要四个分组 $P_0, \ldots, P_3$，每个持有一个缓冲区并等待下一个。该死锁配置的等待图如图 14.4（b）所示。每个缓冲区 B_i 等待持有它的分组 P_i 释放它。每个分组又等待缓冲区 B_{i+1} 以沿环前进。在分组缓冲流控下，每个分组一次只能持有一个缓冲区，而缓冲区依赖图中的环路长度为四，因此我们无法用少于四个分组在此等待图中构造环路。

现在考虑相同的四节点环形网络，但使用微片缓冲流控，每个物理通道有两个虚拟通道。我们假设一个物理通道的任一虚拟通道中的分组可以选择下一个物理通道的任一虚拟通道进行等待。一旦分组选择了某个虚拟通道，它将继续等待该虚拟通道，即使另一个虚拟通道变为空闲。1 这种情况下的虚拟通道依赖图如图 14.5（a）所示。由于持有一条链路的任一虚拟通道的分组可以等待下一条链路的任一虚拟通道，因此该图中所有相邻通道之间都有边。

图 14.5（b）显示了该微片缓冲网络的一个死锁配置的等待图。情况与图 14.2 类似，但

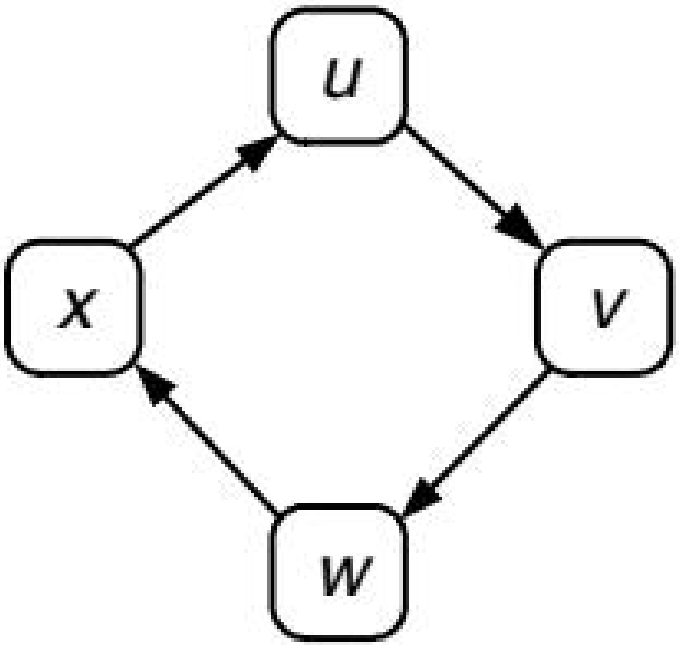


图 14.3 图 14.1 示例的资源（通道）依赖图

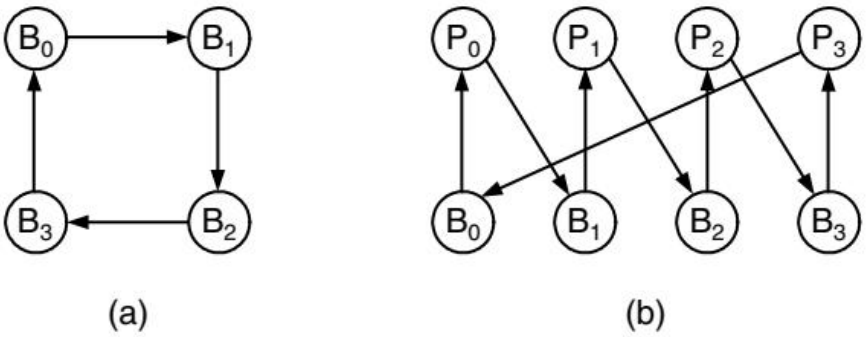


图 14.4 分组缓冲流控的依赖图和等待图。（a）对于图 14.1 的网络，使用分组缓冲流控且每个节点一个分组缓冲区时的资源（分组缓冲区）依赖图。（b）四个分组持有四个分组缓冲区的死锁配置的等待图。

用分组和虚拟通道代替了连接和物理通道。分组 P_0 持有虚拟通道 u0 和 v0，并等待 w0。同时，P_1 持有 w0 和 x0，并等待 v0。“1”虚拟通道完全未被使用。如果允许分组 P_0 使用 w_0 或 w_1，则该配置不会构成死锁。当允许分组在每一跳使用任何未占用的虚拟通道时，该网络的死锁配置需要四个分组。生成此配置留作练习 14.1。

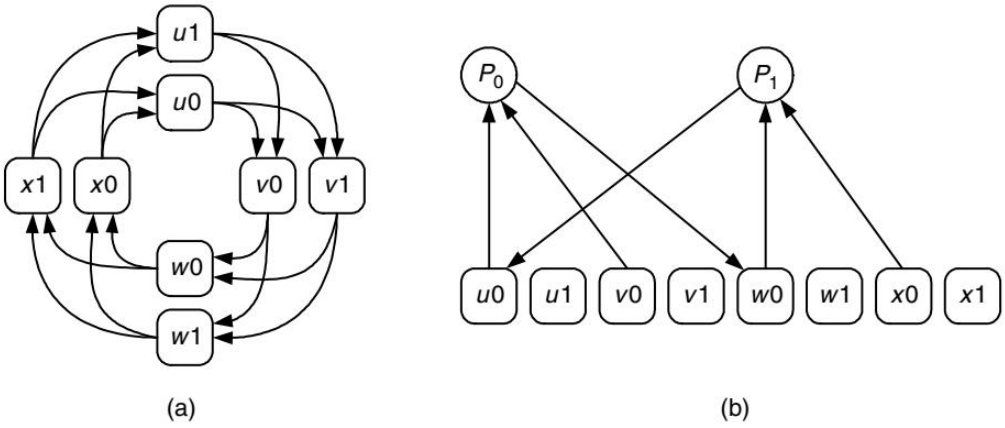


图 14.5 微片缓冲流控的依赖图和等待图。（a）图 14.1 网络在使用微片缓冲流控且每条物理通道有两个虚拟通道时的资源（虚拟通道）依赖图。（b）两个数据包各持有两个虚拟通道的死锁配置的等待图。

14.1.5 高层（协议）死锁

死锁可能由网络外部的依赖引起。例如，考虑图 14.6 所示的情况。上层网络通道正在等待服务器从网络中移除请求数据包。而服务器由于缓冲区有限，在低层通道接受来自服务器输出缓冲区的应答数据包之前，无法接受请求数据包。实际上，由于外部服务器的存在，上层通道正在等待低层通道。等待图中的这条边并非源于网络本身，而是源于服务器。由包含此类外部边的等待环引起的死锁通常称为高层死锁或协议死锁。

例如，在共享内存多处理器中，这种外部等待边可能由每个节点的内存服务器引起，该服务器接受内存读写请求数据包，按请求读写本地内存，并将响应数据包发送回请求节点。如果服务器内部缓冲区有限，情况就与图 14.6 所示完全一致，进入服务器的通道可能不得不等待离开服务器的通道。通过使用不同的逻辑网络（采用不相交的资源集——例如，单独的虚拟通道或数据包缓冲区）来处理请求和应答，可以消除这些外部等待边的影响。在缓存一致的共享内存机器中，情况可能更加复杂，单个事务在返回最终应答之前可能依次经过两个或三个服务器（目录、当前所有者、目录）。这里，在事务的每一步通常采用单独的逻辑网络。使用这些单独的逻辑网络来避免协议死锁是资源排序的一种特殊情况，如第 14.2 节所述。

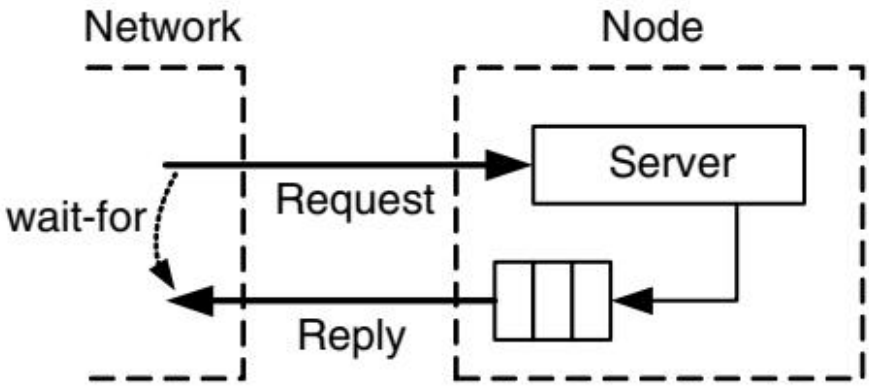


图 14.6　请求-应答系统中的隐式资源依赖。

14.2　死锁避免

可以通过消除资源依赖图中的环来避免死锁。这可以通过对资源施加偏序，然后坚持代理按升序分配资源来实现。因此避免了死锁，因为任何环必须包含至少一个持有较高编号资源并等待较低编号资源的代理，而有序分配不允许这种情况。虽然偏序足以消除环并因此消除死锁，但为简单起见，我们通常通过对资源编号来施加全序。

虽然所有死锁避免技术都使用某种形式的资源排序，但它们在本资源排序所施加的限制如何影响路由方面有所不同。在某些方法中，资源可以按顺序分配，而对路由没有限制。在另一些方法中，所需资源的数量减少了，但代价是禁止某些原本会违反资源排序的路由。

使用数据包缓冲流控时，我们有一个优势：每个节点通常有许多数据包缓冲区。类似地，在使用微片缓冲流控的系统中，每条物理通道通常有许多虚拟通道。由于每个物理单元有多个资源，我们可以通过将同一物理单元上的不同资源（例如，节点上的不同数据包缓冲区）分配到顺序中的不同位置来实现排序。在电路交换中，资源是物理单元（通道），因此每个通道只能在排序中出现一次。因此，为了在电路交换中对资源排序，我们别无选择，只能限制路由。

14.2.1　资源类

距离类：对资源（虚拟通道或数据包缓冲区）排序的一种方法是将资源分组为编号的类，并限制资源的分配，使得数据包按升序从类中获取资源。强制升序资源分配的一种方法是要求距离源 i 跳的数据包从类 i 分配资源。在源端，我们将数据包注入资源类 0。在每一跳，数据包获取下一个最高类的资源。在这种系统中，持有类 i 的数据包缓冲区的数据包只能等待类 $i+1$ 的缓冲区（图 14.7）。2 类似地，持有类 i 的虚拟通道的数据包只能等待编号更高的类中的虚拟通道。数据包在网络中传输时，就资源类而言只向上行进。因为持有类 i 的资源（数据包缓冲区或虚拟通道）的数据包永远不能直接或间接地等待同一或更低编号类中的资源，所以资源依赖图中不存在环，因此不会发生死锁。

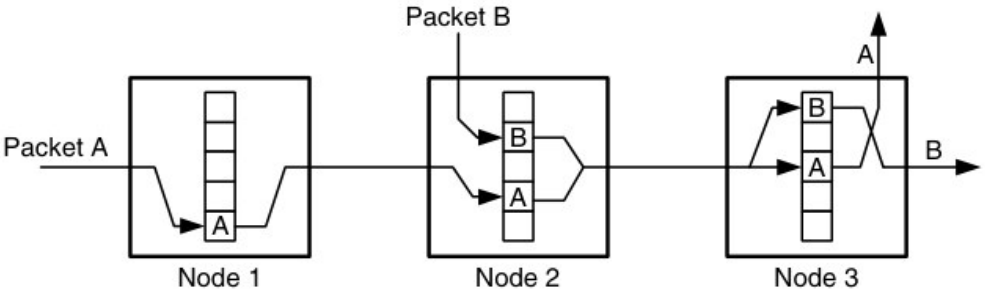


图 14.7 数据包在多个缓冲区类别中路由的示例

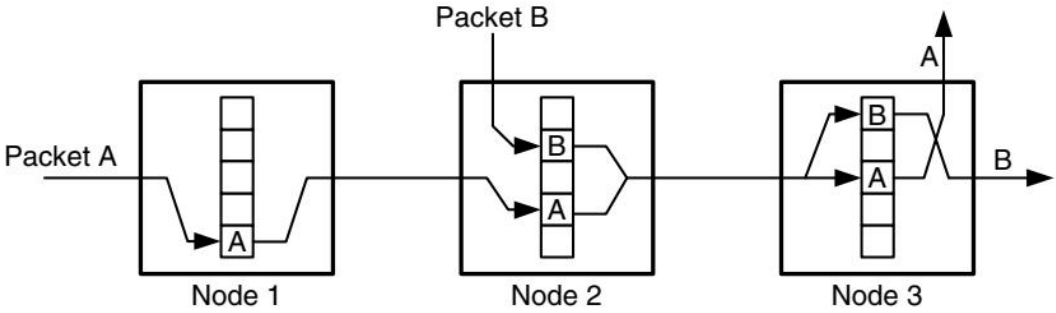
作为距离类的具体示例，图 14.8 展示了一个使用基于距离的缓冲区类的四节点环形网络。每个节点 i 有四个缓冲区 B_{ji}，每个缓冲区保存到目前为止已传输 j 跳的数据包。缓冲区 B_{ji} 中的数据包要么被递送到本地节点 i，要么被转发到节点 i 上的缓冲区 $B_{j+1,i+1}$。这种缓冲区结构导致一个由四个螺旋组成的无环缓冲区依赖图，从而避免了死锁。

为了强制仅向上行进的资源分配规则，每个数据包在分配下一个资源时需要记住其先前的资源类。因此，对于使用距离类的数据包缓冲流控，路由关系的形式为:

$$R : Q \times N \rightarrow Q$$

其中 Q 是网络中所有缓冲区类的集合。类似的关系也用于在具有微片缓冲流控的网络中分配虚拟通道。这种逐跳路由关系使我们能够表达缓冲区类的上坡使用。

距离类提供了一种非常通用的方法，可以在任何拓扑中对资源进行排序。然而，这样做的代价相当大——它们需要与网络直径成比例的报文缓冲区（或虚拟通道）数量。对于某些网络，我们可以利用拓扑来显著减少缓冲区类的数量。例如，在环形网络中，我们可以仅提供两类资源来对资源进行排序。3



通过多个缓冲区类路由报文的示例。每个节点包含五个缓冲区类，最低类位于缓冲区的底部，最高类位于顶部。当报文 A 和 B 在网络中前进时，它们的缓冲区类会增加。

图 14.9 展示了如何通过使用日界线缓冲区类在环中使缓冲区依赖无环。每个节点 i 有两个缓冲区，一个“0”缓冲区 B_{0i} 和一个“1”缓冲区 B_{1i}。在源节点 s 注入的报文最初被放入缓冲区 B_{0s}，并一直留在“0”缓冲区中，直到它到达节点 3 和 0 之间的日界线。越过日界线后，报文被放入“1”缓冲区 B_{10}，并一直留在“1”缓冲区中，直到

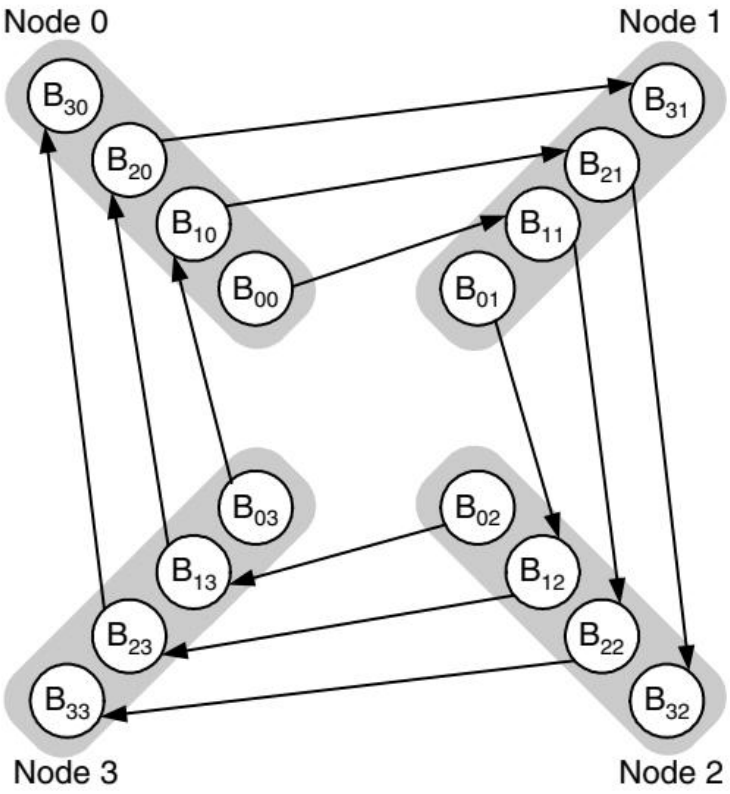


图 14.8 在四节点环形网络上使用的距离类别及对应缓冲区

到达目的地。根据报文是否已越过日界线来划分两个缓冲区类的使用，实际上将缓冲区依赖的循环转换为无环螺旋。因此避免了死锁。

日界线类也可以应用于微片缓冲流控。图 14.10 展示了将日界线类应用于四节点环的虚拟通道依赖图，每个物理通道有两个虚拟通道。每个物理通道 c 有两个虚拟通道 c0 和 c1。所有报文开始使用“0”通道，仅当它们在节点 3 越过日界线时才切换到“1”通道。为了基于输入虚拟通道限制输出虚拟通道的选择，这种方法要求路由函数具有以下形式

$$R : C \times N \rightarrow C$$

其中 C 在这里表示虚拟通道的集合。这里限制虚拟通道的选择，取图 14.5 的循环通道依赖图，并通过移除若干边使其无环。

重叠资源类：根据距离或日界线限制资源类的使用，同时使资源依赖图无环，可能会导致显著的负载不均衡。例如，在图 14.10 中，在均匀业务下，使用虚拟通道 v0 的报文（5 条路由）将多于使用 v1 的报文（1 条路由）。这种负载不均衡会对性能产生不利影响，因为一些资源可能闲置，而另一些则过载。类似地，使用距离类时，并非每条路由都会使用最大跳数，因此编号较高的类往往利用率较低。

减少负载不均衡的一种方法是重叠缓冲区类。例如，使用日界线类时，假设我们有 32 个报文缓冲区。我们可以如图 14.11（a）所示，为类“0”和“1”各分配 16 个缓冲区。然而，更好的方法是如图 14.11（b）所示，为每个类各分配一个缓冲区供其独占使用，

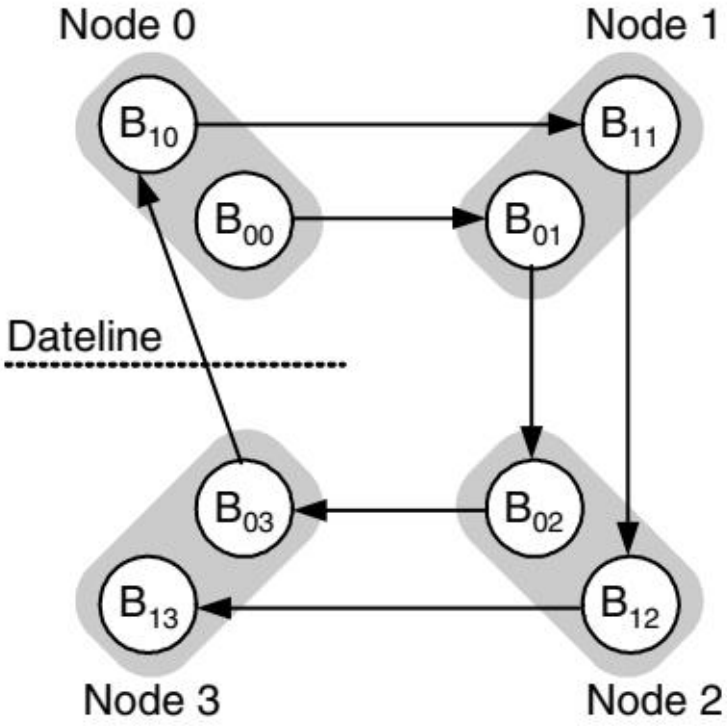


图 14.9 使用日界线缓冲区类时，环中的每个节点 i 有两个缓冲区 B_{1i} 和 B_{0i}。在节点 s 注入的报文从缓冲区 B_{s0} 开始，并一直留在“0”缓冲区中，直到它到达节点 3 和 0 之间的日界线。越过日界线后，报文被放入缓冲区 B_{10}，并一直留在“1”缓冲区中，直到到达目的地。

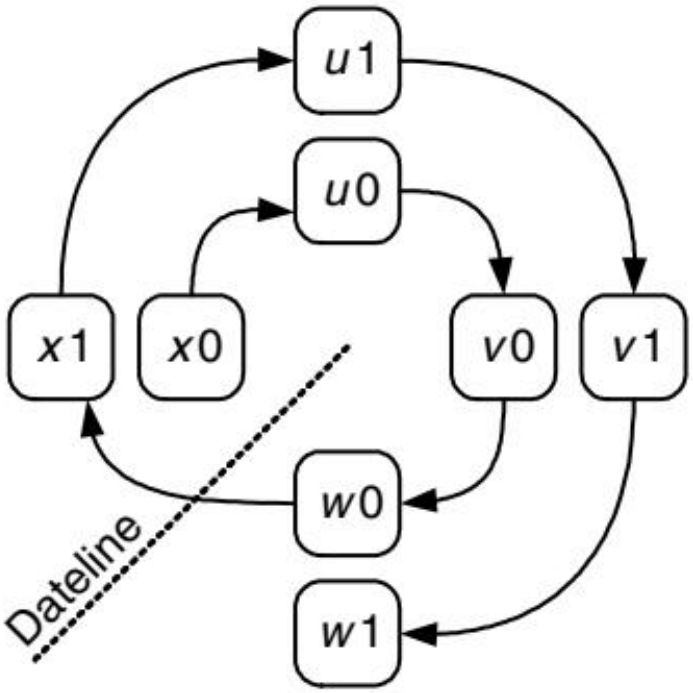


图 14.10 虚拟通道划分为日界线类。四节点环上的每个物理通道 c 被划分为两个虚拟通道 c0 和 c1。所有报文开始在“0”虚拟通道上路由，当它们在节点 3 越过日界线时切换到“1”虚拟通道。

并允许其余 30 个缓冲区由任一类使用。这种方法通过允许大多数缓冲区被需要任一类别的报文使用，从而减少了负载不均衡。

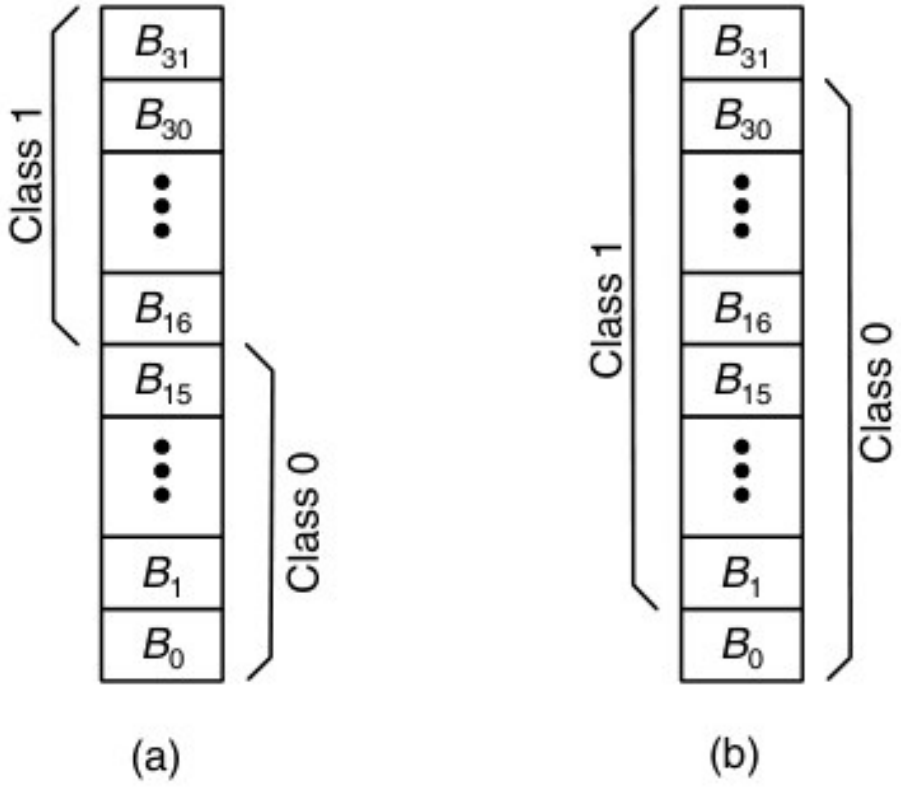


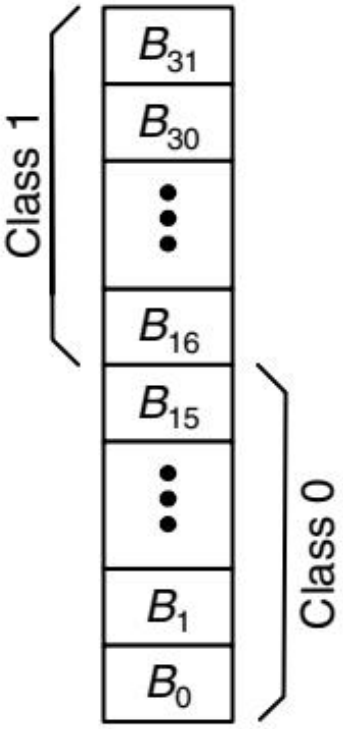
图 14.11 将 32 个缓冲区划分为两类的两种方法

然而，在重叠类时，重要的是绝不允许报文在重叠区域中等待一个忙碌的资源。也就是说，报文不能选择一个属于两个类的忙碌缓冲区——比如 B_{11}——然后等待 B_{11}。如果这样做，它可能正在等待另一个类的报文，从而导致死锁。为了避免重叠类下的死锁，报文在适当类别的空闲缓冲区可用之前，不得选择特定的缓冲区进行等待。通过等待类，报文是在等待该类中的任何缓冲区变为空闲，因此最终会被该类中的一个独占缓冲区满足。如果非独占缓冲区更早可用，这可以提升性能，但不会改变等待独占缓冲区的正确性。

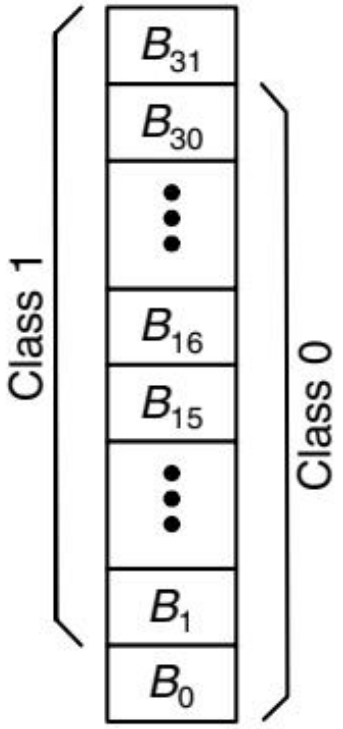
14.2.2 受限物理路由

尽管将网络的资源结构化为类允许我们创建一个无死锁的网络，但在某些情况下，这可能需要大量资源来确保没有循环资源依赖。一种替代方案是限制路由函数，而不是结构化资源以适应所有可能的路由。对路由施加适当的限制可以消除资源之间足够的依赖关系，使得结果依赖图无环，而不需要大量资源类。

图 14.11 将 32 个缓冲区划分为 2 类的两种方法。（a）每个类分配 16 个缓冲区，无重叠。（b）每个类分配 31 个缓冲区，重叠 30 个缓冲区。只要重叠不完全，这些类在避免死锁方面仍然是独立的。


B_{31}
B_{30}
B_{16}
B_{15}
B_1
B_0
Class 1
Class 0

(a)


B_{31}
B_{30}
B_{16}
B_{15}
B_1
B_0
Class 1
Class 0

(b)

维序（e-立方）路由：在 k 元 n 网格中，保证无死锁的最简单的路由限制之一是采用维序路由。（见第 8.4.2 节。）例如，考虑一个二维网格。在第一维 x 中，沿 +x 方向传输的数据包只能等待 +x、+y 和-y 方向的通道。类似地，-x 方向的数据包只等待-x、+y 和-y 方向的通道。在第二维中，+y 方向的数据包只能等待其他 +y 通道，而-y 方向的数据包只等待-y 通道。这些关系可用于枚举网络中的通道，从而保证无死锁。

图 14.12 展示了一个 3×3 网格中维序路由的枚举示例。向右的通道首先编号，因此其值向右递增。然后分别对向左、向上和向下的通道编号。现在，任何经过网络的维序路由都沿着编号递增的通道前进。一旦选择了固定的维序，类似的枚举方法适用于任意数量的维度。

转向模型：虽然维序路由提供了一种限制路由算法以防止 k 元 n 网格网络中循环依赖的方法，但网格网络中限制路由算法的更通用框架是转向模型。在转向模型中，可能的死锁循环是根据产生它们所需的特定转向来定义的。我们将考虑二维情况下的该模型，尽管它可以扩展到任意数量的维度。如图

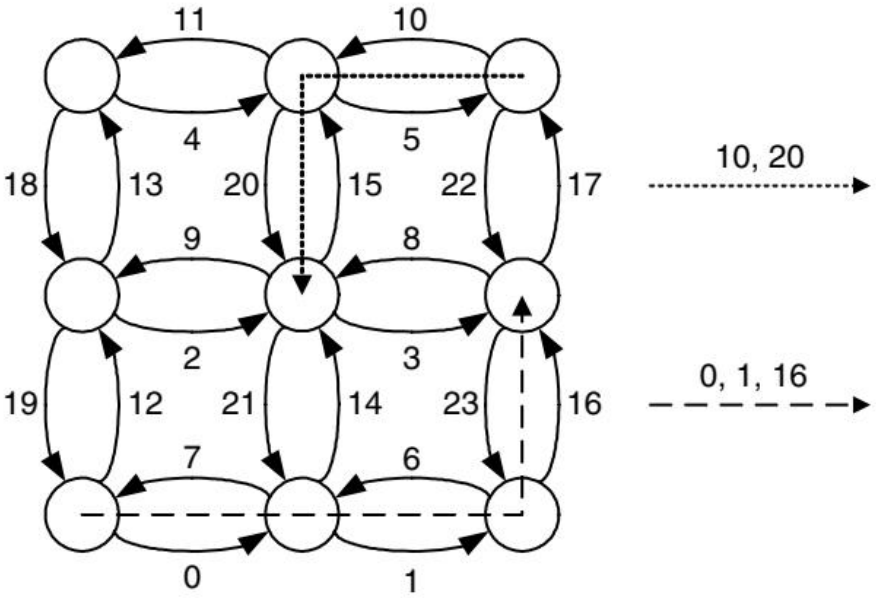


图 14.12　维序路由中 3×3 网格的枚举。还展示了两条路由的通道顺序。

如图 14.13（a）所示，二维路由算法中有八个转向（+x 到 +y、+x 到-y、-x 到 +y 等），它们可以组合成两个抽象循环。通过观察，必须从这两个循环中各消除至少一个转向以避免死锁。例如，维序路由在每个循环中消除两个转向——即从任何 y 维到任何 x 维的转向。

我们可以通过首先从第一个抽象循环中消除一个转向，来探索一组限制比维序路由更少的路由函数。图 14.13（b）展示了消除从北到西的转向（即从 +y 到-x 的转向）。将其与第二个循环中的转向消除相结合，产生了三种无死锁路由算法，如图 14.13（c）所示。第四种可能的消除不是无死锁的，如练习 14.3 所探讨。

三种选择各自产生不同的路由算法。当消除从南到西的转向时，生成西向优先算法。在西向优先路由中，数据包必须先完成所有向西的跳步，才能向任何其他方向移动。

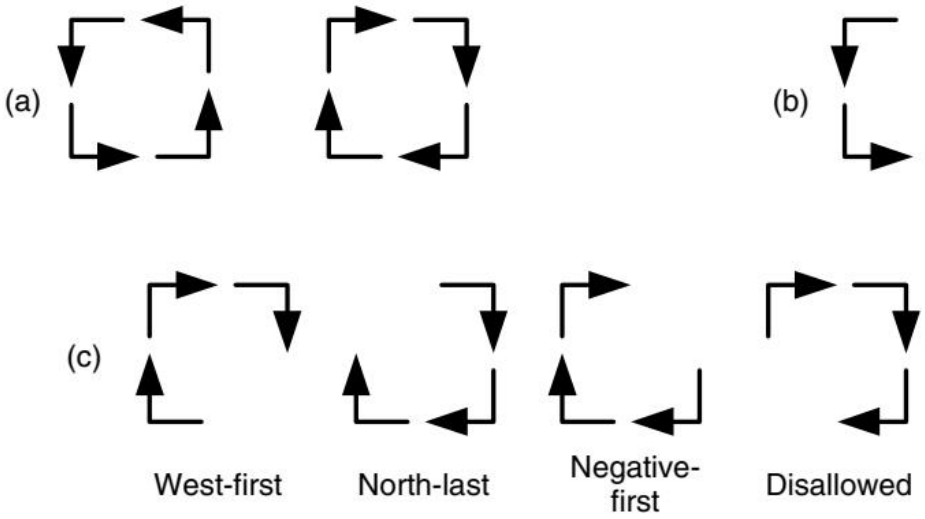


图 14.13 二维网格网络的转向模型。(a) 两个抽象循环。(b) 如果消除从北到西的转向，(c) 通过在另一个抽象循环中消除另一个转向，可以创建三种可能的路由算法。

在从西向转向后，它可以沿除西向以外的任何方向路由。消除从北到东的转向则产生北向最后路由。在北向最后路由中，数据包可以在除北向以外的方向之间自由移动。一旦数据包转向北向，它必须继续向北直到目的地。最后，消除从东到南的转向得到负向优先路由。在负向优先路由中，数据包必须完全沿负方向（南和西）移动，然后才能转向正方向（北和东）。一旦进入正方向，数据包就停留在那里直到到达目的地。乍一看，转向模型似乎在做不同的事情——通过限制转向而不是强加资源顺序来打破死锁。事实并非如此。相反，通过限制转向，转向模型对网络通道施加了全序，但与维序路由所诱导的顺序不同。图 14.14 展示了西向优先转向模型对 3×3 网格网络通道所施加的顺序。向西的通道首先编号。

其余通道然后从西到东编号，每列中的北/南通道按方向顺序编号。

转向模型和维序路由给出了限制路由算法以实现无死锁的两种可能性。然而，这些技术仍然存在缺点。限制路由函数降低了网络的路径多样性，这可能削弱网络的性能和容错能力。在维序路由的情况下，多样性降为零。此外，这些技术无法消除诸如环面等拓扑中固有的通道循环。

14.2.3 混合死锁避免

在前面的章节中，我们看到无死锁网络可以通过拆分网络资源并枚举这些资源，或者适当地限制数据包从源到目的地所走的路径来设计。然而，这两种方法都有一些缺点。缓冲区类方法需要每个节点大量的缓冲，而限制路由算法会导致路径多样性降低，并且不能用于所有拓扑。与许多设计问题一样，最实用的死锁避免解决方案结合了两种方法的特征。

环面路由：我们可以使用维序路由在环面中实现无死锁路由，方法是对每个维度应用日期线类。实际上，日期线类将环面变成了网格——从资源依赖的角度来看——而维

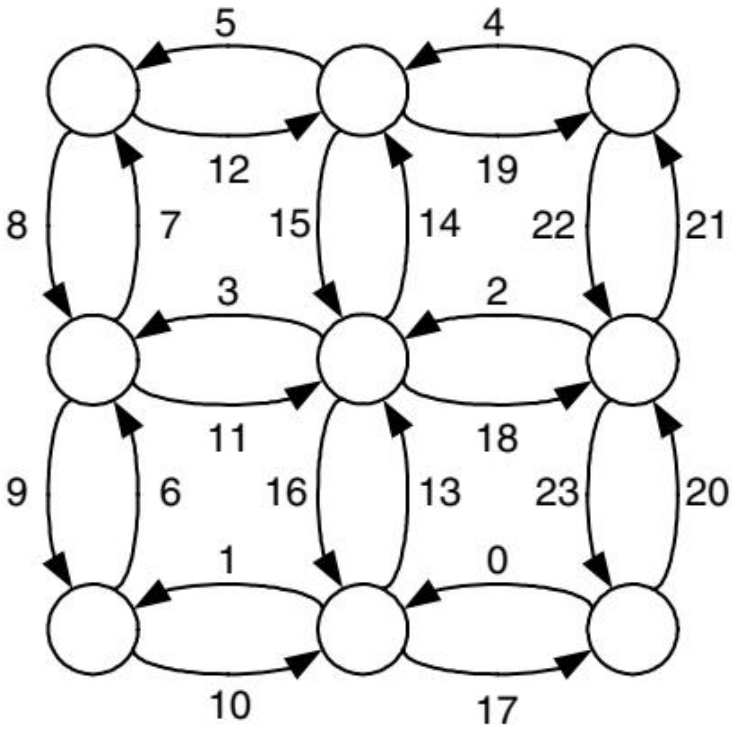


图 14.14 由西向优先转向模型诱导的通道排序。

序路由在所得网格中是无死锁的。例如，在微片缓冲流控制中，我们为每个物理通道提供两类虚拟通道。当数据包注入网络时，它使用虚拟通道 0。

如果数据包在每个维度上跨越了预定义的数据线，它就会被转移到虚拟通道 1。当数据包在某个特定维度上完成路由后，它总是使用虚拟通道 0 进入下一个维度。4 这一过程持续到数据包被弹出为止。

打破依赖关系，然后用虚拟通道重新连接路径的技术可以推广到许多情况。回顾第 14.1.5 节中描述的路由协议死锁，请求和回复之间的依赖关系可能导致潜在的死锁。解决这个问题的一种常用技术是将请求和回复分配到不同的虚拟通道。这样，只要用于路由请求和回复的底层算法是无死锁的，请求-回复协议就不会引入死锁。

平面自适应路由：另一种将虚拟通道与受限物理路由结合的路由技术是平面自适应路由。在平面自适应路由中，路由函数允许有限的自适应性，同时仍避免通道依赖图中的循环。该算法首先在 k 元 n 维网格中定义自适应平面。一个自适应平面由两个相邻的维度 i 和 $i+1$ 组成。在一个自适应平面内，可以使用任何最小自适应路由算法。通过将平面的大小限制为二维，避免死锁所需的虚拟通道数量与网络大小和维度数量无关。

图 14.15 以 k 元 3 维网格为例说明了这一点。最小自适应算法可以在最小子立方体内的任意路径上进行路由。然而，随着 n 的增长，这需要指数数量的虚拟通道来避免死锁。平面自适应路由允许在平面 A_0（由维度 0 和 1 定义）内进行自适应路由，然后在平面 A_1（由维度 1 和 2 定义）内进行自适应路由。这种限制仍然允许较大的路径多样性，但只使用常数数量的虚拟通道。

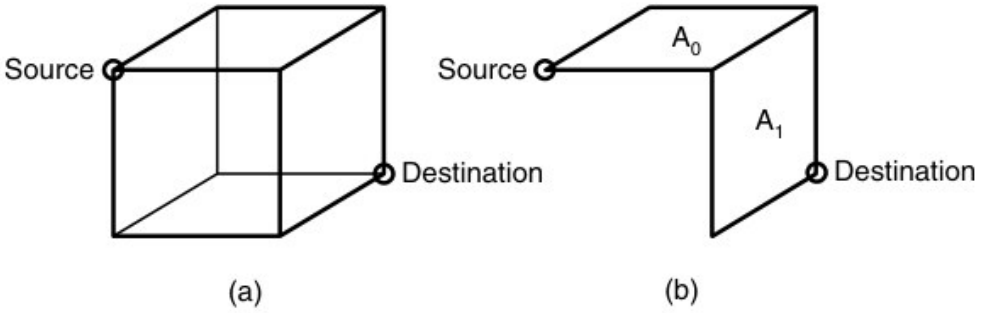
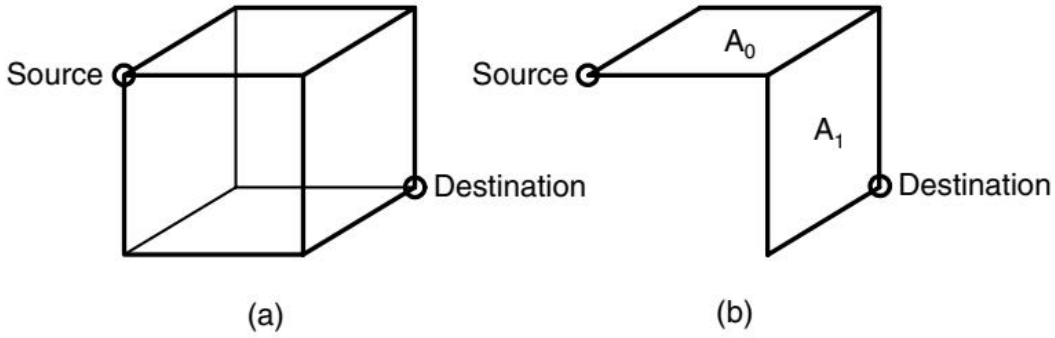


图 14.15 三维网格中的全部最短路径与平面自适应路由允许路径的对比

对于一般的平面自适应路由，每个通道被分为三个虚拟通道，记为 $d_{i,v}$，其中 i 是虚拟通道所在的维度，v 是虚拟通道 ID。自适应平面 A_i 包含虚拟通道 $d_{i,2}, d_{i+1,0}$ 和 $d_{i+1,1}$，其中 $i = 0, \ldots, n-2$。路由从 A_i 开始，i=0。使用任何最小自适应路由算法来路由数据包，直到数据包所在位置的 i^{th} 维度与其目的地的 i^{th} 维度匹配。然后 i 递增，下一个自适应平面中的路由开始。这些步骤持续进行，直到 A_{n-2} 中的路由完成。在数据包到达目的地之前，可能需要在 n^{th} 维度中完成最小路由。



（a）k 元 3 维网格中源节点和目的节点之间所有最小路径的集合（b）平面自适应路由中允许的路径子集。

为了确保 i^{th} 自适应平面内的无死锁路由，该平面内的虚拟通道被分为递增和递减子集。递增子集是 $d_{i,2^+}$ 和 $d_{i+1,0}$，递减子集是 $d_{i,2^-}$ 和 $d_{i+1,1}$，其中“+”和“–”分别表示在某个维度上增加或减少地址的通道。需要在 i^{th} 维度上增加地址的数据包仅在递增网络中路由，反之亦然。这种方法在练习 14.7 中被验证为无死锁的。

14.3 自适应路由

在前面的章节中，我们修改了网络和路由算法，以消除资源依赖图中的循环依赖。其中一些技术可以自然地表达为自适应路由算法。例如，使用西向优先路由的数据包可以在向西方向移动任意跳数，然后再向其他方向路由——这个决策可以根据网络条件自适应地做出。一些缓冲类方法也可以很容易地结合自适应性。

然而，本节的重点是自适应路由算法和 oblivious 路由算法之间的一个重要区别：5 自适应路由算法可以在其资源依赖图中存在循环，同时保持无死锁。这一结果允许设计无死锁网络，而不必像 oblivious 情况那样对路径多样性进行显著限制。

14.3.1 路由子函数与扩展依赖

在通道依赖图存在循环的情况下保持无死锁的关键思想是为潜在循环中的每个数据包提供一条逃生路径。只要逃生路径是无死锁的，数据包就可以在网络中更自由地移动，可能创建循环的通道依赖。然而，逃生路径的存在确保如果数据包陷入困境，仍然存在一条无死锁的路径到达目的地。

一个例子：考虑一个二维网格网络，它使用 flit 缓冲流控制，每个物理通道有两个虚拟通道。我们用 $xydv$ 表示一个通道——即其节点、方向和虚拟通道类别（例如，10e0 是节点 10 上东方向的虚拟通道类别 0）。虚拟通道之间的路由受以下规则限制。

1. 所有路由都是最小的。

2. 虚拟通道 xyd1 中的数据包允许路由到链路另一端的任何虚拟通道——任何方向、任何虚拟通道。例如，00e1 可以路由到节点 10 上的八个虚拟通道中的任何一个。

3. 虚拟通道 xyd0 中的数据包允许路由到链路另一端任何方向的虚拟通道 1。例如，00e0 可以路由到 $10d1$，其中 d 是四个方向中的任意一个。

4. 位于虚拟通道 xyd0 中的数据包允许按维度顺序（先 x，后 y）路由到链路另一端的虚拟通道 0。例如，00e0 可以路由到 10e0 或 10n0，以及节点 10 上的四个“1”类通道。通道 00n0 只能路由到 01n0 和节点 01 上的四个“1”类通道。

简而言之，只要源或目的虚拟通道属于“1”类，路由就是不受限制的。然而，当源和目的虚拟通道都属于“0”类时，路由必须按维度顺序进行。

这组路由规则保证了无死锁，即使虚拟通道依赖图中存在环。图 14.16 展示了一个环的例子，该图显示了采用这些路由规则的网络中四个节点的虚拟通道依赖图。为了避免图形过于杂乱，图中只画出了部分依赖边。图中显示了一个依赖环，包含虚拟通道 00e0、10n0、11w1 和 01s1。根据上述四条路由规则，这个环的四条边都是合法路由。尽管存在这个依赖环，这种路由安排仍然是无死锁的，因为存在逃逸路径。例如，假设数据包 A 占用虚拟通道 00e0 和 10n0，数据包 B 占用虚拟通道 11w1 和 01s1。如果 A 等待 11w1 而 B 等待 00e0，就会形成死锁配置。然而，这些数据包不必等待循环资源，因为它们有其他选择。数据包 A 可以选择 11n0 而不是 11w1。只要（1）潜在环中至少有一个数据包具有无环选项，并且（2）数据包不会承诺等待繁忙资源，死锁配置就不会发生。在这个例子中，我们保证环中的每个数据包都存在无环选项，因为它们总是可以退回到“0”通道上的维度顺序路由，而如第 14.2.2 节所述，这种路由保证是无环的。

间接依赖：现在考虑我们的二维网格示例，但不加路由限制 1——也就是说，允许非最小路由。没有这个限制，网格就不再保证无死锁，因为间接依赖可能导致沿逃逸通道形成环。

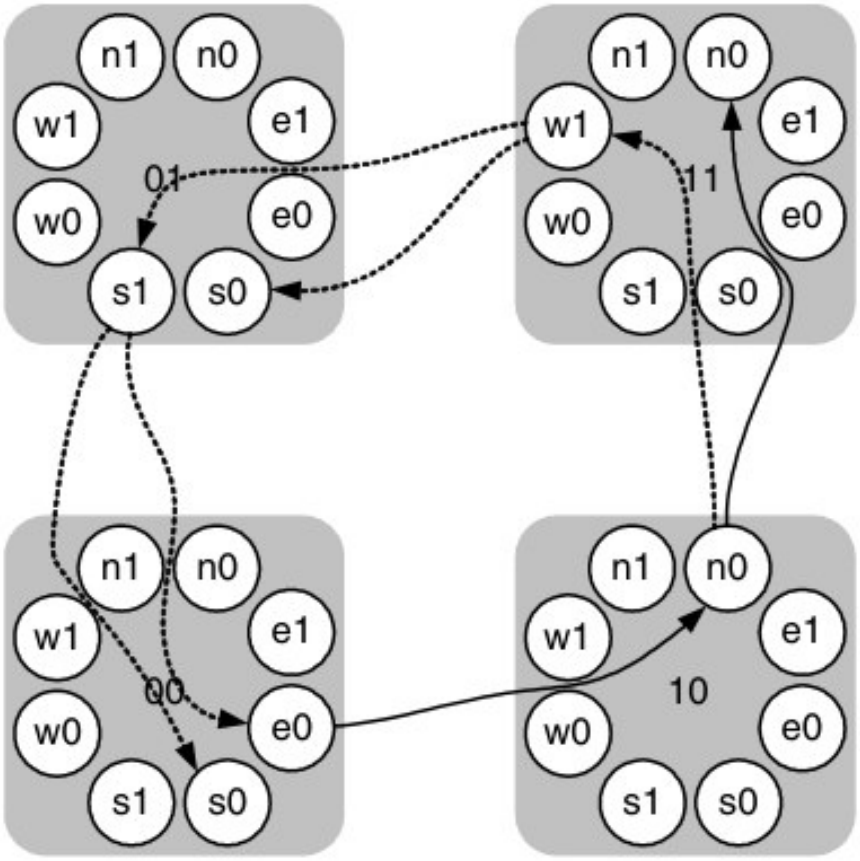
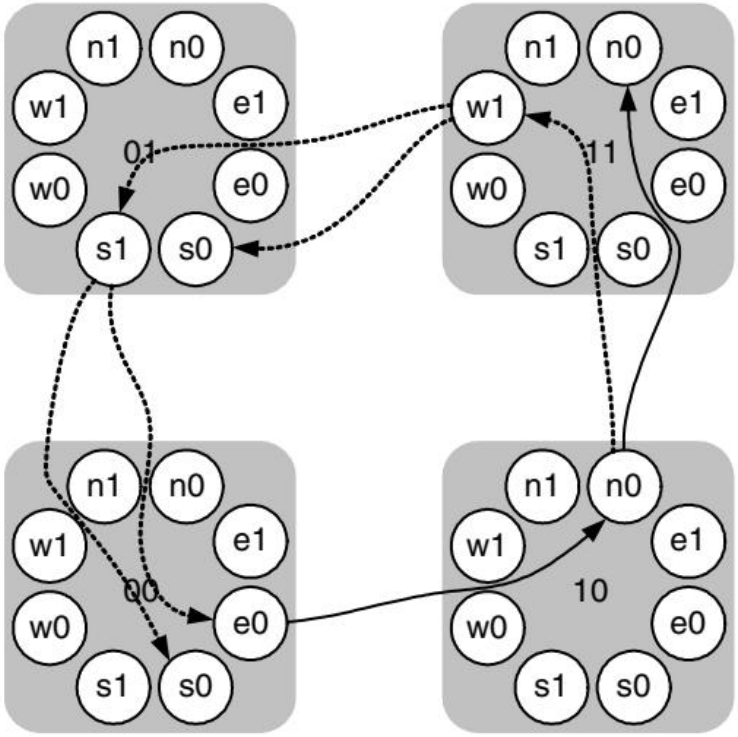


图 14.16 采用逃逸路径的循环虚拟通道依赖关系

14.3 自适应路由



带有逃逸路径的循环虚拟通道依赖示例。该图显示了二维网格网络中四个节点的虚拟通道依赖图，这些节点在“0”通道内必须遵循维度顺序路由。虚拟通道依赖图中存在一个环，包含虚拟通道 00e0、10n0、11w1 和 01s1。然而，不可能发生死锁，因为在三个点上数据包可以自由选择不在环上的另一个虚拟通道——即逃逸路径。

例如，在图 14.16 中，假设数据包 A 正路由到节点 12（未画出），当前占用通道 00e0。数据包 B 也路由到节点 12，但进行了非最小路由，因此占用了通道 10n0、11w1 和 01s1。尽管错误路由是在“1”通道（11w1 和 01s1）上进行的，但它对“0”通道造成了间接依赖——从 10n0 到 00e0。这种间接依赖在“0”通道上形成了一个环。如果所有“1”通道因循环路由而阻塞，“0”通道就不再能够排空网络。数据包 A 正在等待数据包 B 释放 10n0，然后才能沿其维度顺序路由前进。同时，数据包 B 正在等待数据包 A 释放 00e0，以便按维度顺序路由。因此，本应作为无死锁逃逸路径的“0”通道，由于间接依赖而陷入了死锁。

我们可以通过坚持以下原则来避免间接依赖：访问逃逸通道（“0”通道）a 的数据包，如果 $b \succ a$，则不允许通过任何通道（逃逸通道或其他通道）路由到逃逸通道 b。对于我们的二维网络示例，强制执行这种顺序的一种简单方法是坚持非逃逸通道（“1”通道）上的路由是最小路由。

间接依赖仅对使用微片缓冲流控的网络值得关注。使用数据包缓冲流控时，一个数据包一次只能占用一个缓冲区。因此，占用逃逸缓冲区的数据包不可能通过某些非逃逸缓冲区等待另一个逃逸缓冲区。一旦获取了非逃逸缓冲区，初始逃逸缓冲区就会被释放。

形式化推导：既然我们已经介绍了逃逸通道和间接依赖的概念，我们就可以更形式

化地描述自适应路由的死锁避免。如果在虚拟通道集合 C 上有一个自适应路由关系 $R : C \times N \to \mathcal{P}(C)$, 我们可以在虚拟通道子集 $C_1 \subseteq C$ 上定义一个路由子关系 $R_1 \subseteq R$, 使得 R_1 是连通的——即任何源 s 都可以使用 R_1 将数据包路由到任何目的地 d。如果路由子关系 R_1 在其扩展通道依赖图中没有环，则整个路由关系 R 是无死锁的。R 中但不在 R_1 中的路由称为补子关系 $R_C = R - R_1$。当包含路由关系的参数时 $R(c, d)$, 它们表示数据包的当前通道 c 和数据包的目的地 d。

回到我们的例子，R 是完整的路由规则集（上述规则 1 到 4），C 是所有虚拟通道的集合（上述“0”和“1”通道），R_1 是逃逸通道的规则集（上述规则 4），C_1 是逃逸虚拟通道的集合（上述“0”通道）。此外，R_C 是 R 允许但 R_1 不允许的路由集（上述规则 1 到 3）。

R_1 的扩展资源依赖图以 C_1 中的虚拟通道为顶点，以 C_1 中通道之间的直接依赖和间接依赖为边。[6] 简而言之，这就是我们的标准依赖图加上间接依赖的扩展。我们更精确地将扩展依赖图中的两类边定义为:

直接依赖——这与无死锁避免中考虑的信道依赖相同，适用于 oblivious 路由。如果存在节点 x 使得 $c_j \in R_1(c_i, x)$，则存在从 c_i 到 c_j 的信道依赖。也就是说，如果 R_1 中的一条路由依次使用两个信道 c_i 和 c_j，则这两个信道之间存在直接依赖（或简称依赖），我们将其记为 $c_i \succ c_j$。例如，在图 14.16 中，存在从 00e0 到 10n0 的依赖，因为任何在第 0 行路由到第 1 列的数据包都将依次使用这两个虚拟信道。

间接依赖——间接依赖的产生是因为我们对微片缓冲流控的假设允许数据包同时占用任意数量的信道。在这种情况下，当到达节点 x 的路径使用信道 $c_i \in R_1$，紧接着通过 R_C 使用若干信道 $c_1, \ldots, c_m$，最后通过信道 $c_j \in R_1$ 进行路由时，就产生了依赖。例如，在图 14.16 中，使用非最小路由时，存在从 10n0 到 00e0 的间接依赖，因为持有 10n0 的数据包可以在 R_C 中经由 11w1 和 01s1 误路由到节点 00，在那里 R_1 的维度顺序路由要求使用信道 00e0。使用非最小路由时，这种依赖将不存在，因为想要使用 00e0 的数据包将不被允许在 11w1 上沿另一个方向路由。实现时有时通过简单地禁止从 R_1 到 R_C 的转换来消除间接依赖——一旦数据包使用了路由子函数 R_1，它就必须在其路由的剩余部分继续使用 R_1。

Duato [61] 证明的一个关键结果是，无环的扩展信道依赖图意味着无死锁网络。

对于互连网络，如果存在一个连通且其扩展信道依赖图中无环的路由子关系 R_1，则自适应路由关系 R 是无死锁的。

虽然我们对扩展依赖的讨论集中在虫洞流控上，但这些思想也可以应用于存储转发和直通网络。关键区别在于扩展依赖更简单，并且依赖是在缓冲区之间而非信道之间形成的。简化是因为数据包在阻塞时不能占用任意数量的信道，这消除了间接依赖，只留下直接依赖。如果存储转发网络中 R_1 的缓冲区依赖图是无环的，则网络是无死锁的。路由子关系可以独立于 R 进行检查，从而允许非常灵活地定义 R，只要 R_1 是无死锁的。

14.3.2 Duato 的无死锁自适应算法协议

虽然 Duato 的结果为设计无死锁自适应路由函数提供了理论基础，但它并未直接揭示如何利用这些思想来创建实际的网络设计。在本节中，我们描述应用 Duato 结果的最常见技术，该技术允许完全自适应、最小路由。

Duato 协议包含三个步骤，可用于帮助设计虫洞和存储转发网络的无死锁自适应路由函数：

1. 底层网络设计为无死锁。这可以包括添加虚拟资源以打破任何循环依赖，例如环面中的循环依赖。

2. 为网络中的每个物理资源创建一个新的虚拟资源。对于虫洞网络，这些资源是（虚拟）信道；对于存储转发网络，缓冲区是资源。然后，原始虚拟资源集使用步骤 1 中的路由关系。这是逃逸关系 R_1。新的虚拟资源使用路由关系 R_C。

3. 对于数据包缓冲流控，对 R_C 没有限制。对于微片缓冲流控，必须构造 R_C，使得 R_1 的扩展依赖图保持无环。

Duato 协议最常见的用途是创建最小自适应路由算法，其中维度顺序路由作为路由子函数 R_1。例如，在具有虫洞流控的 3 元 2 网格中，维度顺序路由用于 R_1。然后 R_C 可以包含所有最小路径，且 $R = R_1 \cup R_C$。所得的扩展依赖图是无环的（图 14.17）。

14.4 死锁恢复

到目前为止，我们专注于消除导致死锁的条件的技术。这些方法需要受限的路由函数或额外的资源来打破循环依赖。然而，处理死锁的另一种方法不是避免它，而是从中恢复。当设计无法容纳避免死锁所需的额外资源或性能下降时，这种方法很有吸引力。当然，此类技术依赖于死锁不频繁发生，并且考虑的是平均情况性能而非最坏情况性能。

任何死锁恢复算法都有两个关键阶段：检测和恢复。在检测阶段，网络必须意识到它已进入死锁配置。准确判断网络是否死锁需要在资源等待图中找到循环。解决这个问题既困难又昂贵，因此大多数实用的检测机制是保守的。保守检测总能正确识别死锁，但也可能标记非死锁的网络状况（误报）。误报的引入会降低性能，但使实现更可行，同时仍能正确检测所有死锁。这种检测通常通过超时计数器完成：每个网络资源关联一个计数器，当资源取得进展（即数据通过资源发送）时重置。然而，如果计数器达到预定阈值，则认为资源死锁，并启动恢复阶段。

14.4.1 回退恢复

在回退恢复中，发生死锁的数据包或连接会从网络中移除。例如，这种技术可以应用于电路交换。如果一组部分建立的电路形成了等待环，它们就发生了死锁。这种状态会持续到某个通道的超时计数器达到阈值。然后，任何等待该通道的电路都会被拆除（从网络中移除）并重试。这打破了等待依赖关系，从而消除了死锁。为了确保源端重

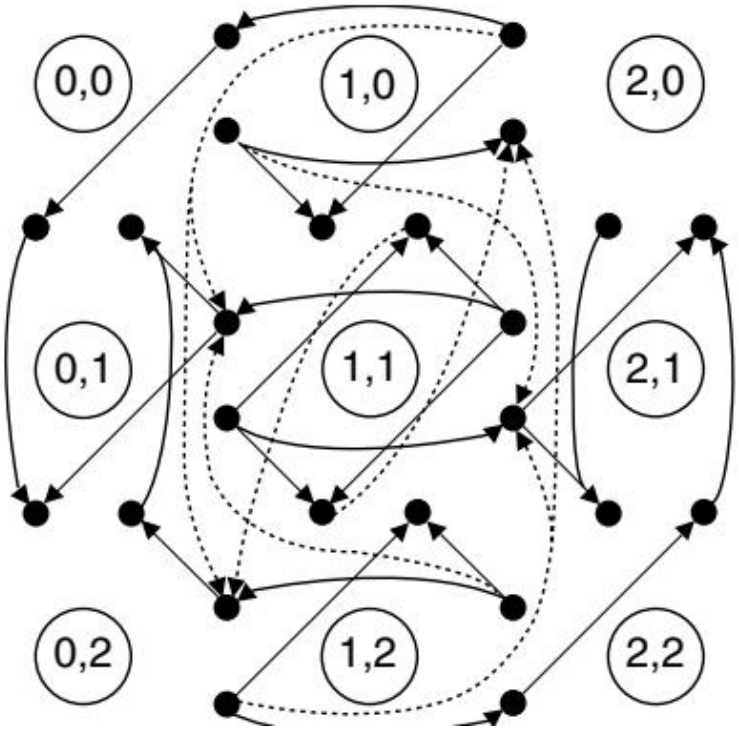


图 14.17 3 元 2 网格的扩展通道依赖图。直接（间接）依赖关系用实线（虚线）表示。网络通道由实心圆表示，可以表示为其他依赖关系组合的依赖关系未显示。

试电路，可以向源端发送 Nack，或者源端可以重试任何挂起时间超过阈值的电路。在这里，与许多回退恢复技术一样，必须特别注意活锁问题，以确保重试的数据包最终能够成功。

无压缩路由对基于虫洞的网络采用了类似的恢复思路。数据包末尾会附加额外的空 flit，以便源节点在发送最后一个 flit 时能够确保目的端至少已收到一个 flit。这是可行的，因为虫洞网络通常只有少量缓冲——在无压缩路由中，任何数据包的最小长度大致等于从源到目的路径上的缓冲区数量。

当一个数据包启动时，源节点开始超时计数。每当一个新的 flit 被网络接受后，该计数就会重置。如果在最后一个 flit 注入网络之前超时计数达到阈值，整个数据包就会从网络中移除并重试。一旦最后一个 flit 注入完毕，源端就可以保证数据包头部已经到达目的端。这意味着该数据包已经分配了一条通往目的端的（虚拟）通道路径，不会再被阻塞。这种方法的主要成本是必须为数据包添加额外的填充。如果发送大量短数据包，最大吞吐量可能会显著降低。

14.4.2 渐进恢复

渐进恢复在不将死锁数据包从网络中移除的情况下解决死锁状况。由于不会因发送然后移除数据包而浪费网络资源，渐进恢复技术具有潜在更高的性能。此外，与重发数据包相关的活锁问题也被消除了。

主流的渐进恢复方法在硬件中实现 Duato 协议的思路。例如，DISHA [186] 架构在网

络每个节点上提供一个共享逃生缓冲区。当检测到疑似死锁状况时，共享逃生缓冲区被用作死锁中数据包的排流通道。与路由子功能一样，使用逃生缓冲区的路由被设计为无死锁的。

Duato 算法的硬件实现是否合适取决于特定设计中资源的相对成本。DISHA 是在假设虚拟通道和缓冲区是昂贵网络资源的前提下设计的，这在某些应用中确实如此。然而，在其他应用中，引入集中式资源（此处为共享逃生缓冲区）的影响可能超过额外虚拟通道的影响。

14.5 活锁

与死锁不同，活锁的数据包会继续在网络中移动，但永远无法到达目的地。这主要是非最小路由算法可能错误路由数据包所带来的问题。如果无法保证数据包被错误路由的最大次数，数据包可能会无限期地留在网络中。丢弃流控技术也可能导致活锁。如果数据包每次重新进入网络时都被丢弃，它可能永远无法到达目的地。

避免活锁主要有两种技术：确定性和概率性避免。在确定性避免中，每个数据包会添加少量状态以确保其进展。该状态可以是一个错误路由计数，记录数据包被错误路由的次数。一旦计数达到阈值，就不允许再错误路由。这种方法在非最小自适应路由中很常见。类似的方法是在每个数据包中存储基于年龄的优先级。当数据包之间发生冲突时，优先级最高（最老）的数据包获胜。在偏转路由或丢弃流控中使用时，数据包将在有限时间后成为网络中优先级最高的数据包。这防止了进一步的偏转或丢弃，数据包将直接前往其目的地。

概率性避免通过保证数据包在网络中停留 T 个周期的概率随着 T 趋于无穷大而趋近于零来防止活锁。例如，我们可能希望在采用偏转路由和单 flit 数据包的 2 元 k 网格中避免活锁。7 数据包距离其目的地的最大跳数始终为 $H_{\max} = 2(k-1)$。然后我们为数据包的历史写一个字符串，其中 t 表示朝向目的地的路由决策，d 表示偏转（例如 tddtdtt。。。）。如果字符串中 t 的数量减去 d 的数量曾经超过 H_{max}，那么我们知道数据包必定已到达其目的地。只要数据包朝向目的地路由的概率始终非零，这种情况发生的概率就趋近于一。因此，只要我们能始终保证数据包在每一跳都有非零的机会朝向其目的地移动，我们的网络就是无活锁的。

14.6 案例研究：Cray T3E 中的死锁避免

Cray T3E [162] 是 T3D（第 8.5 节）的后续产品。它围绕 Alpha 21164 处理器构建，单机柜机器最多可扩展至 272 个节点。最多可互连 8 个机柜，形成多达 2,176 个节点的液冷配置。与 T3D 一样，T3E 的拓扑是三维环面网络。对于

14.5 案例研究：Cray T3E 中的死锁避免对于 2,176 节点的机器，基础拓扑是 8，32，8 元 3 立方体，这占了 2,048 个节点。然后在 z 维度的一半环中添加额外的冗余/操作系统节点，将其基数扩展到 9，使节点总数达到 2,176。

T3E 中额外的延迟容忍度允许从用于实现 T3D 路由器的快速 ECL 门阵列退后一步，每个 T3E 路由器都实现为 CMOS ASIC。额外的延迟容忍度还允许 T3E 设计者使用自适应路由，以提高吞吐量，优于 T3D 的维度顺序路由，并且通过路由算法绕过故障链路或节点的能力提高了容错性。

T3E 的路由中采用了与 Duato 协议（第 14.3.2 节）相同的方法来融入自适应性。T3E 网络使用直通流控制，每个节点有足够的缓冲来始终存储最大的数据包。这确保了在扩展通道依赖图中永远不会产生间接依赖。因此，路由子函数无死锁就足以保证整个网络无死锁。

T3E 中使用的路由子函数称为方向顺序路由：数据包首先在递增的 x 维度（+x）路由，然后在 +y、+z、-x、-y，最后是-z。数据包头中的三个位指示每个数据包是沿递增还是递减方向穿越维度。例如，在所有维度上递增的数据包将按 +x、+y、然后 +z 路由，而在除 y 维度外的所有维度上递增的数据包将按 +x、+z、然后-y 路由。暂时忽略由环面拓扑引起的维度内循环，方向顺序路由很容易证明是无死锁的。概念上，通道依赖图中的任何循环都必须包含从递增通道（+）到递减通道（–）的路由，反之亦然。虽然允许从递增通道到递减通道的路由，但反之则不成立。因此，方向顺序路由对于网格是无死锁的。

T3E 还允许通过使用初始和最终跳来轻微变化方向顺序路由。数据包在网络中采取的初始跳可以在任何递增方向，而最终跳可以在所有其他跳完成后在-z 方向。添加初始和最终跳提高了路由算法的容错性，如图 14.18 所示。如图所示，节点 21 和节点 22 之间的故障通道阻塞了默认的方向顺序路由。然而，通过在-y 方向采取初始跳，数据包成功地从节点 20 路由到节点 03。

现在考虑环面的环绕通道，T3E 采用了第 14.2.3 节所述的日期线方法：任何在维度内穿越预定日期线的数据包必须从虚拟通道（VC）0 开始，并在日期线之后过渡到虚拟通道 1。不穿越日期线的数据包可以使用虚拟通道 0 或虚拟通道 1，但任何特定数据包必须选择一个。在 T3E 中，为这些数据包选择虚拟通道的目标是平衡虚拟通道之间的负载。也就是说，虚拟通道的最优分配恰好平衡了穿越特定物理通道的每个虚拟通道的数据包平均数量。由于可能的 VC 分配空间如此之大，T3E 中使用模拟退火算法来找到这个虚拟通道平衡问题的近似解。

在日期线方法中，需要两个虚拟通道来打破环面维度内的死锁，另外两个用于为请求和回复消息创建单独的虚拟网络。这消除了图 14.6 中所示的高级死锁。最后，一个虚拟通道用于最小自适应路由，这解释了 T3E 网络中使用的五个 VC。

14.7 文献注释

早期关于无死锁互连网络的工作确定了枚举网络资源并按递增顺序遍历这些资源的技术 [129，70，78，57]。Linder 和 Harden[118] 开发了一种方法，使任意自适应路由无死锁，但代价是虚拟通道的数量随维度数量呈指数增长。Glass 和 Ni 开发了转向模型，允许有限的自适应性同时保持无死锁 [73]。在另一种方法中，Dally 和 Aoki 允许

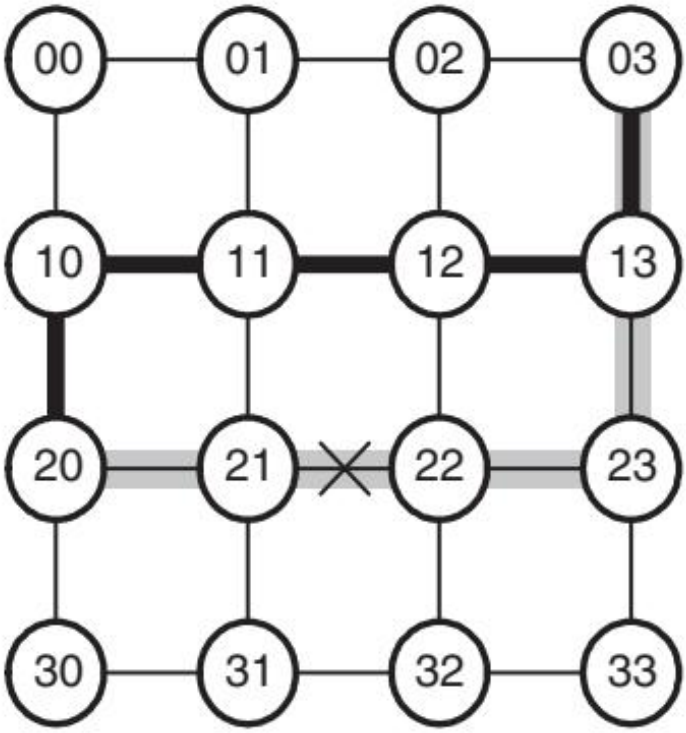


图 14.18 方向顺序路由前增加初始跳所获得的容错性

通过少量维度反转实现一些非最小自适应性 [48]。

Duato 引入了扩展通道依赖图 [60]，并在 [61] 和 [62] 中完善和扩展了这些结果。Duato 设计无死锁自适应路由算法的方法已被用于多个网络，如可靠路由器 [52]、Cray T3E 和 Alpha 21364[131]。

其他更具体的无死锁路由算法包括第 14.2.3 节所述的平面自适应路由 [36]。在不规则网络拓扑中，可以使用 up*/down* 算法 [70] 避免死锁，例如在 DEC AutoNet[159] 中采用。Gravano 等人引入了 *-通道和其他算法 [76]，它们使用与 Duato 类似的思想将自适应性融入环面。Kim 等人描述了无压缩路由 [98]，它采用电路交换思想，而不是本章重点关注的基于分组交换的方法。虽然大多数系统选择死锁避免，但 Anjan 和 Pinkston 描述了 DISHA 死锁恢复方案 [186]。

Bolding [25] 专门在环面网络中研究了由许多死锁避免方案引起的虚拟通道失衡问题。除了 Scott 和 Thorson 在 Cray T3E 案例研究中描述的模拟退火方法外，Cray 持有的美国专利还涵盖了一些额外的虚拟通道负载均衡方法 [30]。

Schwiebert [160] 表明，即使存在循环通道依赖，也有可能实现无死锁的 oblivious 路由算法。然而，目前这种行为示例需要不寻常的路由函数和拓扑结构。

14.8 练习

14.1 死锁配置。考虑图 14.5 的示例，但采用这样的准则：数据包在每一跳可以使用任一虚拟通道，以先变为空闲者为准。为该网络的一个死锁配置绘制等待图。

14.2 简单路由算法的无死锁性。判断以下 oblivious 路由算法在二维网格中是否无死锁。（证明通道依赖图是无环的或包含环。）
（a）随机维度顺序：所有数据包均以最小方式路由。一半数据包在 Y 维度之前完全在 X 维度中路由，另一半数据包则在 X 之前路由 Y。
（b）较少随机化的维度顺序：所有数据包均以最小方式路由。最小方向在 X 和 Y 两个维度均递增的数据包总是先路由 X 后路由 Y。最小方向在 X 和 Y 两个维度均递减的数据包总是先路由 Y 后路由 X。所有其他数据包随机选择先 X 后 Y 或反之。
（c）二维网格的有限转向路由：所有路由限制为最多包含三个右转且无左转。
14.3 转向模型中的复合环。解释为什么图 14.13（c）中的第四种转向消除选项不会产生无死锁的路由算法。
14.4 禁止转向的必要数量。利用转向模型的思想，找出在 k 元 n 网格中路由时为确保无死锁而必须消除的转向数量的下界。
14.5 枚举转向模型通道。以类似于图 14.14 所示的方式对 3×3 网格网络的通道进行编号，但针对北最后路由限制。
14.6 平衡虚拟通道利用率。在第 14.2.3 节中，虚拟通道被用于消除环形网络中的循环依赖。然而，所讨论的简单日期变更线方法在虚拟通道之间的平衡性较差。例如，从节点 4 到 1 的虚拟通道 1 仅被从节点 2 路由到节点 1 的数据包使用，而从节点 1 到 2 的虚拟通道 1 则完全未被使用。描述一种形式为 $C \times N \mapsto C$ 的新路由算法，该算法能更好地平衡虚拟通道，并且也是无死锁的。
14.7 平面自适应路由的无死锁性。证明平面自适应路由（第 14.2.3 节）在 k 元 n 网格中是无死锁的。
14.8 容错平面自适应路由。容错平面自适应路由（FPAR）扩展了基本的平面自适应路由算法，以避免故障（失效的节点或通道）。
（a）为了允许最终维度中的故障节点，FPAR 添加了自适应平面 A_{n-1}，其中包含先前未使用的通道 d_{n-1，2}、d_{0，0} 和 d_{0，1}。然后，路由从自适应平面 A_{0} 进行到 A_{n-1}，如同平面自适应路由一样，但增加了额外的平面。证明自适应平面之间不存在循环依赖。
（b）在平面 A_{i} 中路由时，如果 d_{i} 或 d_{i+1} 被故障阻塞，则选择一个非故障通道，只要该通道也是最小的。可能发生的情况是，数据包已完成在（i+1）维中的路由，而 d_{i} 中存在故障。在这种情况下，需要在 d_{i+1} 中进行误路由——如果数据包正在 d_{i+1} 中路由，则继续沿相同方向；否则，在 d_{i+1} 中选择任意方向进行误路由。这种误路由持续进行，直到找到 d_{i} 中的第一个非故障通道，并采用该通道。然后，数据包继续在 d_{i} 中路由，直到第一次有机会“纠正”d_{i+1} 中的误路由。在 d_{i+1} 中的距离再次减小到零后，恢复正常路由。此过程的细节很复杂，但本质上它试图简单地绕过故障区域（图 14.19）。相反的情况，即在 d_{i} 中完成路由而 d_{i+1} 中存在故障，不会发生，因为一旦在 d_{i} 中完成路由，i 就递增。证明 FPAR 在 k 元 n 网格中的单个自适应平面 A_{i} 内是无死锁的。是否由此可以得出整个路由算法也是无死锁的?

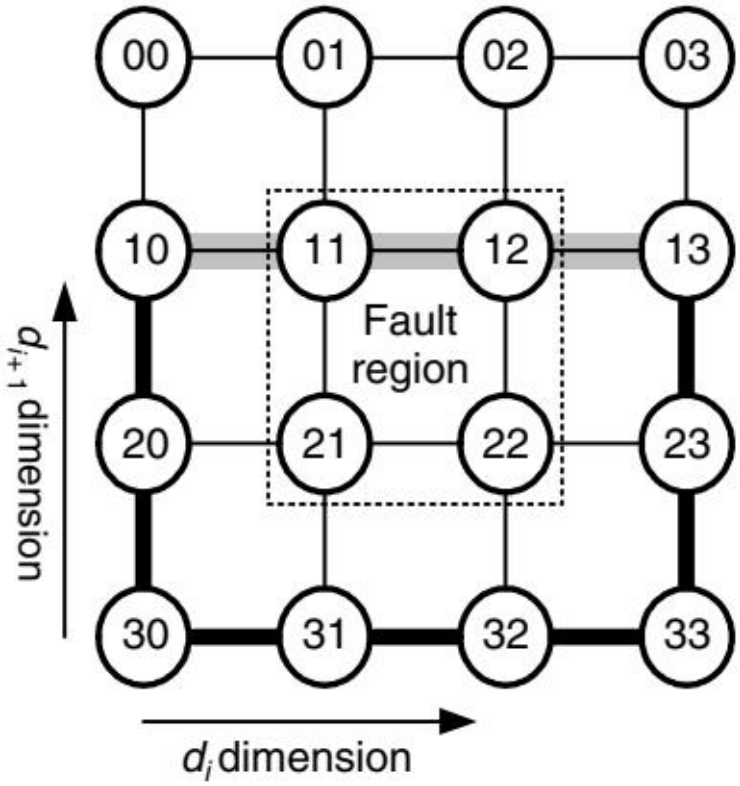


图 14.19　容错平面自适应路由中误路由的一个示例。节点和通道的故障区域（由虚线框表示）阻塞了从节点 10 到 13 在 A_{i} 平面中的最小路径，并且（i+1）维中的最小距离为零。因此，需要在 d_{i+1} 中进行误路由以绕过故障区域。

14.8 练习

14.9 Cray T3E 中路由的死锁分析。通过枚举通道，证明方向顺序路由（第 14.6 节）在 k 元 3 网格中是无死锁的。这种枚举是否也证明 Cray T3E 中使用的初始和最终跳变也是无死锁的？如果不是，请创建一个新的通道枚举来证明。现在考虑 T3E 路由算法的进一步扩展，允许在任意正（+）维度中以任意顺序进行任意数量的初始跳变。这种扩展是否也是无死锁的？

14.10 随机维度遍历顺序的虚拟通道成本。您为 k 元 n 网格设计了一种维度顺序路由的随机变体，其中遍历维度的顺序是完全随机的。这给出了 n！种可能的维度遍历顺序。当然，如果每个遍历顺序都有自己的虚拟通道，则路由算法是无死锁的。然而，对于高维网络，n！个虚拟通道可能成本高昂。您能否找到一种使用更少虚拟通道的无死锁虚拟通道分配方案？

14.11 可变大小数据包的概率性活锁避免。解释为什么在偏转路由网络中，关于无活锁的概率论证不一定适用于多 flit 数据包网络。假设数据包以虫洞流控方式在网络中流水线传输，且每个输入端口有一个 flit 的缓冲。构造一个多 flit 数据包和偏转路由导致活锁的场景。

注释

- 1. 我们将数据包选择第一个空闲虚拟通道时死锁配置的构造留作练习 14.1。
- 2. 我们可以允许报文等待任何比它当前持有的缓冲区类更高的缓冲区类。然而，如果这样做，我们无法保证它不会在上坡过程中跳过太多类而耗尽类。
- 3. 我们将在第 14.2.3 节中看到如何扩展此方法以处理任意环面网络。
- 4. 这种简单的分配会导致虚拟通道利用的不平衡。更有效的分配方法在练习 14.6 中探讨。
- 5. 实际上，一些 oblivious 路由算法可以具有循环依赖并保持无死锁，但这在一般情况下并不成立 [160]。
- R1R_{1}
- RCR_{C}
- 6. 在某些 R1R_{1} 和 RCR_{C} 共享信道的路由算法中，还必须将交叉依赖添加到扩展依赖图中。这种信道共享在实践中几乎从未发生，我们在此不讨论交叉依赖。
- 7. 对于多 flit 数据包，这个问题变得更加复杂。参见练习 14.11。

第十五章　服务质量

此前，我们关注的是提高资源分配的效率，例如实现更高的带宽和更低的延迟。然而，即使有完美的路由和流控，仍然存在对特定资源的请求超过其容量的情况。在这种情况下，我们的注意力从高效分配资源转向根据某种服务策略公平地分配资源。在本章中，我们既关注网络客户端请求的典型服务，也关注提供这些服务的机制。广义上讲，我们将这些主题称为提供 QoS。

15.1　服务类别与服务合约

在互连网络的某些应用中，将网络流量划分为多个类别有助于更有效地管理数据包的资源分配。不同类别的数据包可能有不同的需求——有些类别对延迟敏感，而另一些则不然。有些类别可以容忍延迟但不能容忍抖动。有些类别可以容忍丢包，而另一些则不能。此外，不同类别的数据包可能具有不同的重要性级别。维持生命支持系统运行的数据包将优先于娱乐系统中传输数字音频的数据包。

基于类别分配资源使我们能够优先处理服务，使更重要的类别获得更高水平的服务，并定制服务，使资源分配能够考虑每个数据包类别的特定需求。通过优先服务，我们可以在缓冲区和通道分配中给予某一类别的数据包比低类别数据包严格的优先级，或者为某一类别提供更大比例的整体资源池。通过定制资源分配，属于需要低延迟类别的数据包可以优先于对延迟不敏感的数据包。另一类可以容忍延迟但不能容忍抖动的数据包可以被调度为其延迟大但可预测。可以容忍丢失的类别的数据包将在不能容忍丢失的类别的数据包之前被丢弃。了解每个类别的优先级和需求，使我们能够比所有数据包接受完全相同服务时更有效地分配资源。

流量类别分为两大类：保证服务类别和尽力服务类别。保证服务类别保证一定水平的性能，只要它们注入的流量遵守一组限制。网络和客户端之间存在服务合约。只要客户端遵守限制，网络将提供性能。协议的客户端通常限制客户端可以注入的流量——即最大提供吞吐量。作为将提供的流量保持在某一水平以下的交换，合约的网络侧可以指定保证的丢失率、延迟和抖动。例如，我们可以保证，只要 A 在任何 100 纳秒期间注入不超过 1 千比特，A 类数据包中 99.999% 将无丢失地交付，且延迟不超过 1 微秒。

相比之下，网络对尽力服务数据包不做强有力的保证。根据网络的不同，这些数据包可能有任意延迟甚至被丢弃。1 网络将只是尽最大努力将数据包交付到其目的地。

互连网络中的服务类别类似于包裹递送中的服务类别。保证服务数据包就像联邦快递®（它们保证您的包裹在一夜之间到达，只要它适合他们的信封并且不超过指定重量），而尽力服务数据包就像美国邮政服务中的邮件（您的包裹可能会在几天内到达，但没

有保证）。网络中可能有多个尽力服务流量类别，就像邮件有多个包裹递送类别（如一等、三等，等等）。

在给定的服务类别中，特别是在尽力服务类别中，通常假定流之间是公平的。流只是对构成类别的数据包所做的最小级别的区分。一个流可能是由特定源生成的所有数据包、前往共同目的地的那些数据包，或由网络客户端上运行的应用程序发送的所有数据包。人们期望同一类别的两个流将看到大致相同水平的服务：相似的数据包丢失水平、相似的延迟等。如果一个流的所有数据包都以低延迟交付，而另一个流的所有数据包都被丢弃，那么网络是不公平的。我们将在下面更精确地定义公平性的概念。

15.2 突发性与网络延迟

如前所述，服务保证除了速率请求外，还可能包含延迟和抖动约束。为了实现这些保证，必须考虑特定流量流的突发性。从概念上讲，非突发流以规则的模式发送数据。图 15.1（a）显示了一个非突发流，以三分之二数据包/周期的速率发送。相比之下，突发流倾向于以更大的团块发送数据，而不是随时间平滑发送。例如，图 15.1（b）显示了一个速率为三分之一数据包/周期的突发流。

这两种流共享一条 1 包/周期信道的结果如图 15.1（c）所示。首先，由于与突发流的交互，非突发流的抖动从 1 个周期增加到了 2 个周期。此外，突发流在通过信道时被延迟了多达 4 个周期。很容易验证，降低第二条流的突发性既能减少其自身的延迟，也能减少第一条流的抖动。这个简单的例子表明，流的延迟和抖动保证会极大地受到与其共享资源的其他流的影响。为了量化这些影响，我们首先介绍一种表征突发流的技术。接着给出了一个受两条突发流影响的简单网络元件的延迟分析。

15.2.1 （σ，ρ）调节流

在进行 QoS 保证时，一种常见且强大的流建模方式是使用两个参数：σ 和 ρ。ρ 参数简单地表示流的平均速率，而 σ 项则刻画突发性。对于任意长度为 T 的时间区间，注入到（σ，ρ）调节流中的比特数小于或等于 $\sigma + \rho$T。也就是说，比特数只能以最大突发尺寸 σ 超过平均数量 ρT。我们已经在第 15.1 节中看到了流的（σ，ρ）描述：只要 A 在任何 100 纳秒周期内注入不超过 1 千比特，A 类数据包就在 1 微秒内交付。该流的速率为 ρ = 1 千比特/100 纳秒 = 10 Gbps，σ 至多为 1 千比特。

用 σ 和 ρ 来表达流的性质不仅有用，而且还可以控制任何特定流的这些参数。这种控制可以通过（σ，ρ）调节器实现，如图 15.2 所示。该调节器由一个输入队列组成，用于缓冲未调节的输入流。只有当令牌队列中有可用令牌时，该队列才会得到服务。对于从输入队列服务的每个字节，从令牌队列中移除一个令牌。为了控制输出流的速率 ρ，以 ρ 的速率将令牌插入令牌队列。然后，输出流中的突发量由令牌队列的深度 σ 设定。使用（σ，ρ）调节器控制流可以减少突发流对网络中其他流的影响。2

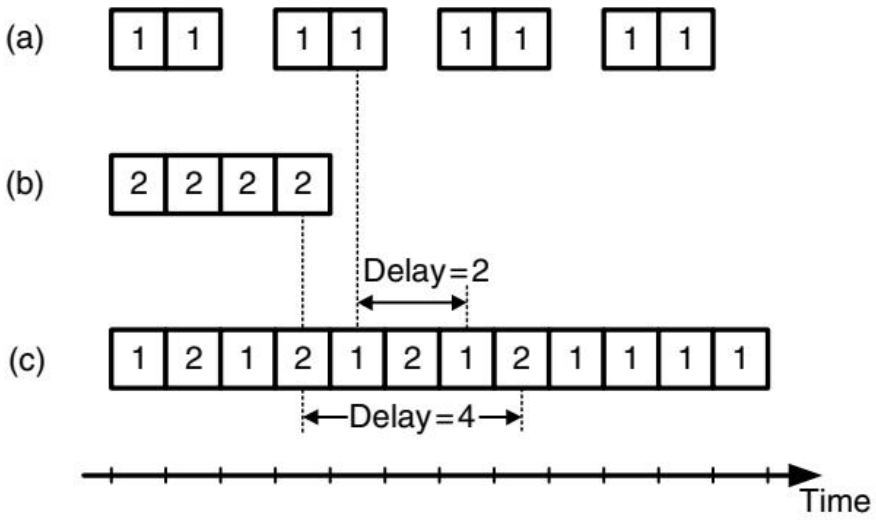


图 15.1 （a）非突发流和（b）突发流（c）共享同一信道的示例。非突发流的抖动增加到 2 个周期，突发流的延迟为 4 个周期。

15.2.2 计算延迟

假设流具有（σ，ρ）特征，就可以计算出网络延迟的确定性边界。作为一个例子，我们关注一个简单网络元件的延迟：一个带排队的两输入多路复用器（图 15.3 顶部）。该多路复用器接受两条受调节的输入流，记为（σ_1，ρ_1）和（σ_2，ρ_2），它们被多路复用到单一输出上。假设两个输入和输出信道的带宽均为 b。

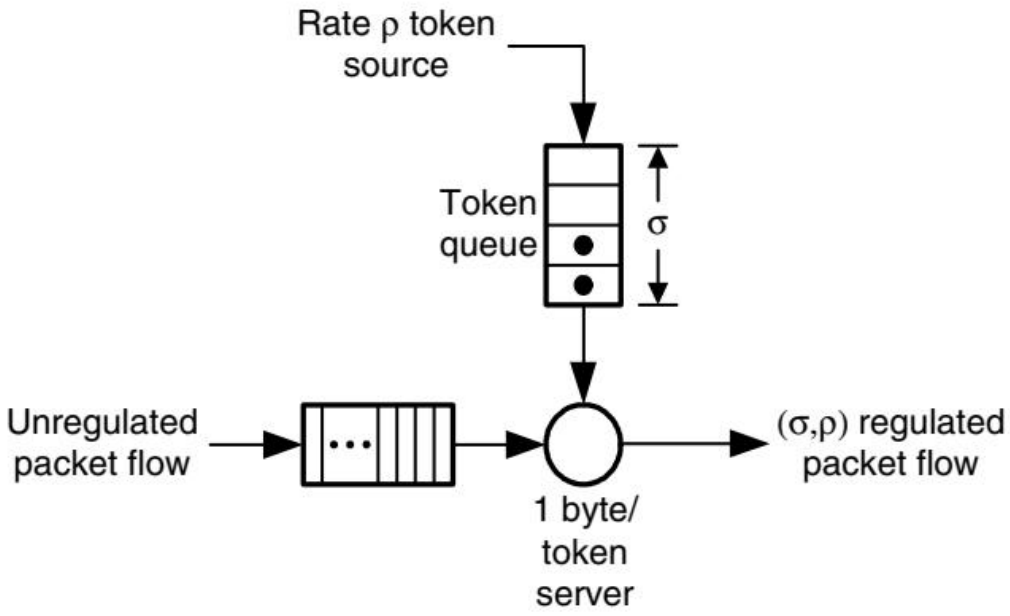


图 15.2 一个（σ，ρ）调节器。

最后，我们关于数据包被选择输出的方式的唯一假设是多路复用器是工作 conserving 的。也就是说，如果多路复用器包含任何数据包，输出信道就永远不会空闲。

为了使系统稳定且最大延迟有定义，$\rho_1 + \rho_2 < b$ 就足够了。这个条件也意味着多路复

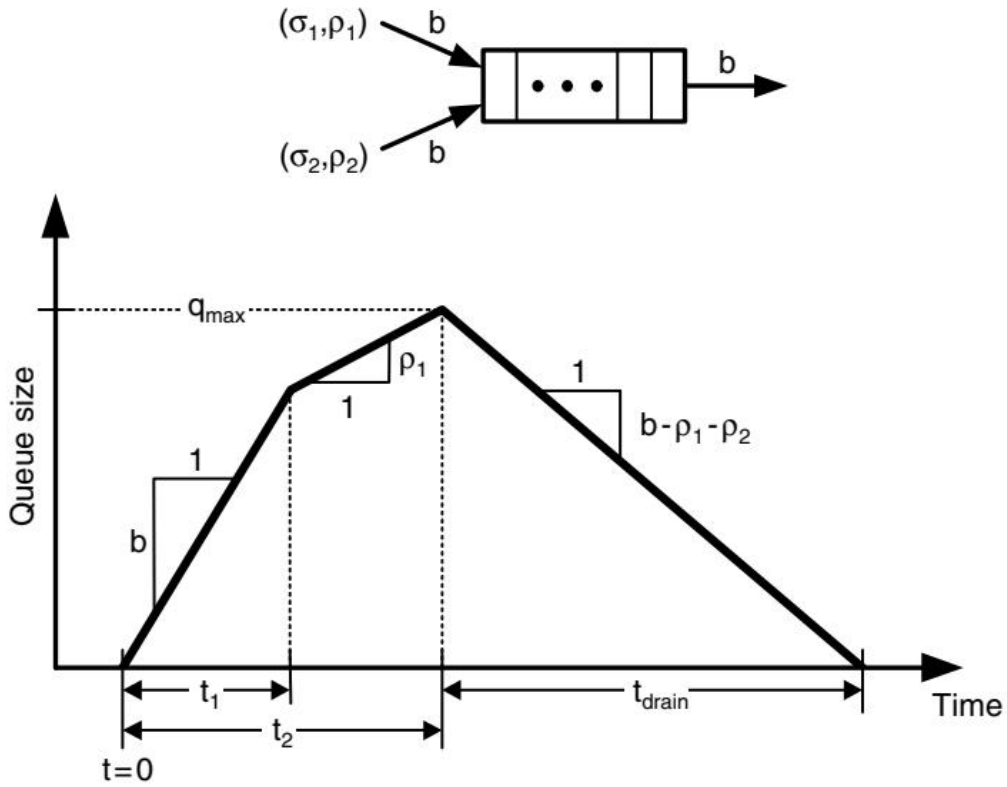


图 15.3 在两输入多路复用器（顶部所示）处，在旨在最大化队列非空间隔的对抗性输入下的队列大小。递增的斜率代表输入中两个或一个输入以峰值速率发送的部分，仅受注入信道带宽限制。在两个输入突发都耗尽后，队列缓慢排空。

用器队列有时会为空，这引出了关于数据包延迟的一个简单观察：任何数据包通过多路复用器的最大延迟至多是队列可能非空的最大时间。因此，我们确定最大数据包延迟的策略将是找到两条输入流的对抗性行为，使得多路复用器队列在尽可能长的时间间隔内保持非空。

我们的对抗策略总结在图 15.3 所示的队列时间与时间的关系图中。该策略分为三个阶段。最初，假设输入队列为空。第一阶段从时间 t = 0 开始，两条输入流同时以速率 b 发送数据包，这是信道约束所允许的最大值。这以速率 b 填充多路复用器的队列，因为输入信道以总速率 2b 注入数据包，并且由于多路复用器是工作 conserving 的，它以速率 b 从队列中排出数据包。因此，队列大小的净增长速率为 b，这反映在图 15.3 第一线段的斜率中。这一阶段持续到其中一个输入无法在不违反其（σ，ρ）约束的情况下维持速率 b 为止。不失一般性，我们假设第一条流在时间 t_1 首先达到这一点。根据我们对（σ，ρ）流的定义，这发生在 $bt_1 = \sigma_1 + \rho_1 t_1$ 时。重写，

$$t_1 = \frac{\sigma_1}{b - \rho_1}.$$

在第二阶段，第一条流只能以速率 ρ_1 发送，以便不违反其（σ，ρ）约束。第二条流继续以速率 b 发送，给出净注入速率 ρ_1 + b 和排出速率 b。因此，队列在第二阶段仍然增长，但增长率较小，为 ρ_1，如图所示。类似地，这一阶段在 t_2 时结束，此时第二条流无法再以速率 b 发送：

$$t_2 = \frac{\sigma_2}{b - \rho_2}.$$

在第三阶段开始时，两个输入流都已耗尽它们的突发量，只能分别以稳态速率 ρ_1 和 ρ_2 发送。这使得队列大小的减少速率为 $b - \rho_1 - \rho_2$。以这个速率，队列在 $t_{\text{drain}} = q_{\text{max}}/(b - \rho_1 - \rho_2)$ 之后变为空，其中 q_{max} 是该阶段开始时的队列大小。队列大小就是第一阶段和第二阶段之后净量的总和：

$$q_{\text{max}} = bt_1 + \rho_1(t_2 - t_1) = \sigma_1 + \frac{\rho_1\sigma_2}{b - \rho_2}.$$

根据我们之前的论证，我们知道延迟 D 必须受这个非空区间长度的限制。

$$D_{\text{max}} = t_2 + t_{\text{drain}} = \frac{\sigma_1 + \sigma_2}{b - \rho_1 - \rho_2}.$$

虽然我们尚未严格论证我们对输入行为的选择给出了 D_{max} 的最大可能值，但可以证明事实确实如此 [42]。直观地说，任何在队列重新排空之前耗尽两个输入流全部突发量的对手都会给出最大可能的非空区间。此外，我们立即启动两个突发的策略最大化了队列的大小，这也将多路复用器所需的总缓冲量限制为 q_{max}。类似的技术可以应用于各种各样的基本网络元件以确定它们的延迟。我们在练习中分析额外的网络元件。

15.3 保证服务的实现

实现保证服务有广泛的可能性。我们从聚合资源分配开始，其中没有特定资源分配给任何流；相反，网络根据其聚合资源使用情况接受来自客户端的请求。这种聚合方法在硬件成本方面是廉价的，但不能提供最紧的延迟界限。通过在空间或时间和空间上预留特定资源，可以获得更低的延迟。这些方法的额外成本是存储资源预留所需的硬件。

15.3.1 聚合资源分配

实现服务保证的最简单方法是要求流量类别 C 的聚合需求 Λ_C 小于一个界限。符合此界限的流量然后被保证不会使网络饱和，因此可以保证具有特定延迟特性的无损传递。这是提供保证服务的最简单方法。因为没有为单个流预留特定资源，所以支持聚合分配几乎不需要额外的硬件。3 然而，由于类别 C 中的所有（可能是突发的）输入流混合在一起，产生的输出流变得更加突发。因此，聚合资源分配给出了我们将描述的方法中最宽松的延迟界限。

使用聚合分配，请求可以由网络客户端显式提供，或者可以是隐式的。例如，在分组交换网络中，网络可能能够接受任何不会超额订阅网络任何输入或输出端口的资源分配集合。如果一个端口需要发起或接收的总流量超过其带宽，则该端口被超额订阅——这对应于请求矩阵 Λ_C 的行和或列和。

现在，为了了解突发性如何影响聚合资源分配，考虑图 15.4 中所示的带有一个额外级的 2 元 2 飞。为了平衡负载，该网络使用随机路由算法，其中所有流量从源路由到网络中间级的一个随机交换节点，然后再路由到其目的地。该图还显示了两个流：从节点 0 到节点 1 的突发流（实线）和从 2 到 0 的非突发流（虚线）。因为聚合资源分配不为流预留特定资源，所以无法防止这两个流之间的耦合。这使得在这个例子中更难实现对非突发流的低抖动要求。此外，使用随机路由引入了更多的突发性。之前，我们考虑了流量流在时间上的突发性，但随机路由也引入了空间上的突发性——路由平均上平衡负载，但瞬时负载可能不平衡。这进一步复杂化了对低抖动或延迟的保证。

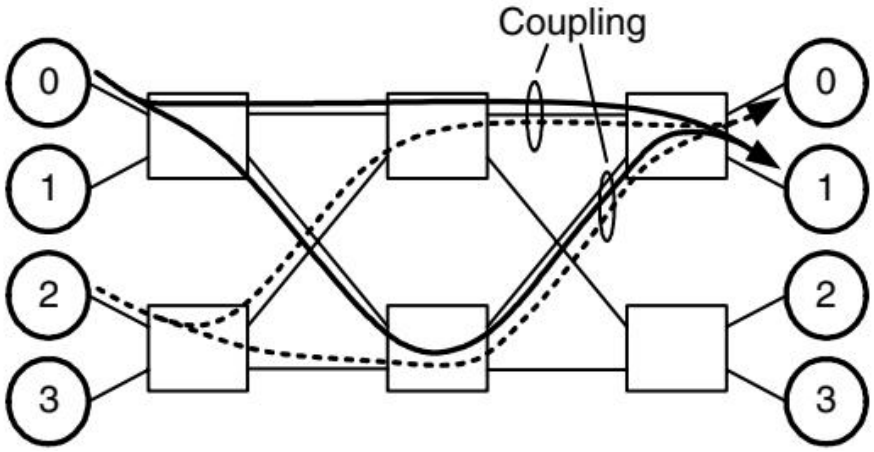


图 15.4 聚合资源分配下突发流与非突发流的耦合

15.3 保证服务的实现

考虑到这些因素并使用第 15.2.2 节的延迟结果，我们可以计算这种聚合资源分配的延迟界限。本例中的两个流都受 (σ, ρ) 调节，对于突发流，$\rho_1 = 0.75$ Gbps，$\sigma_1 = 1,024$ 比特，对于非突发流 $\rho_2 = 0.75$ Gbps 和 $\sigma_2 = 64$ 比特。通道带宽为 b = 1 Gbps，最大分组长度为 L = 128 比特。

跟踪从 0 到 1 通过系统的流，它在第一阶段被随机路由算法分割。我们假设路由算法将流分成两个子流，速率均为 $\rho_1/2$。虽然我们不能假设子流的突发性也减半，因为路由发生在分组粒度上，但这种突发性可以由 $(\sigma_1 + L)/2$ 上界。（见练习 15.2。）

使用这些结果，我们知道来自分配的子流受 $(\sigma_1', \rho_1/2)$ 调节，其中

$$\sigma_1' \leq \frac{\sigma_1 + L}{2} = \frac{1024 + 128}{2} = 576 \text{ bits}.$$

类似地，对于第二个子流

$$\sigma_2' \leq \frac{\sigma_2 + L}{2} = \frac{64 + 128}{2} = 96 \text{ bits}.$$

因此，在分割第二个流时，由于分组粒度限制，其突发性实际上增加了。

这两条流中的任意一条在第二阶段输出通道上复用时，都会产生第一个延迟。利用第 15.2.2 节的结果，我们知道该延迟至多为

$$D_{\max} = \frac{\sigma_0' + \sigma_1'}{b - (\rho_0 + \rho_1)/2} = \frac{\sigma_1 + \sigma_2 + L}{2b - \rho_1 - \rho_2}.$$

当 $\rho_0 + \rho_1 < 2b$ 时，上式成立。代入示例中的值，

$$D_{\max} = \frac{576 + 96}{1 - 0.375 - 0.375} = \frac{672 \text{ bits}}{0.25 \text{ Gbps}} = 2.688 \ \mu\text{s}.$$

在没有任何额外信息的情况下，抖动可能高达 D_{max}，因为数据包可能无延迟地通过多路复用器。类似的计算给出了子流在网络最后阶段合并后到达目的地之前所经历的延迟。

15.3.2 资源预留

在需要对延迟和抖动提供更强保证的情况下，可能需要预留特定资源，而不是依赖聚合分配。当然，这会带来更大的硬件开销，因为这些预留信息也必须存储在网络中。我们介绍两种预留方法：虚电路，即在空间上预留资源；以及时分复用，即在空间和时间上同时预留资源。

使用虚电路时，每条流都被分配一条特定的网络路径。例如，异步传输模式（ATM）就采用了这种预留技术（第 15.6 节）。虚电路的使用解决了若干延迟和抖动的来源。首先，由于资源是在空间上分配的，因随机路由等因素导致的资源使用变化被完全消除。第二个优点是，可以对流进行路由以避免与其他流耦合。考虑第 15.3.1 节中带有

额外级的 2 元 2 飞示例。通过控制其路由，非突发流（虚线）可以绕过突发流（实线），从而改善其抖动（图 15.5）。

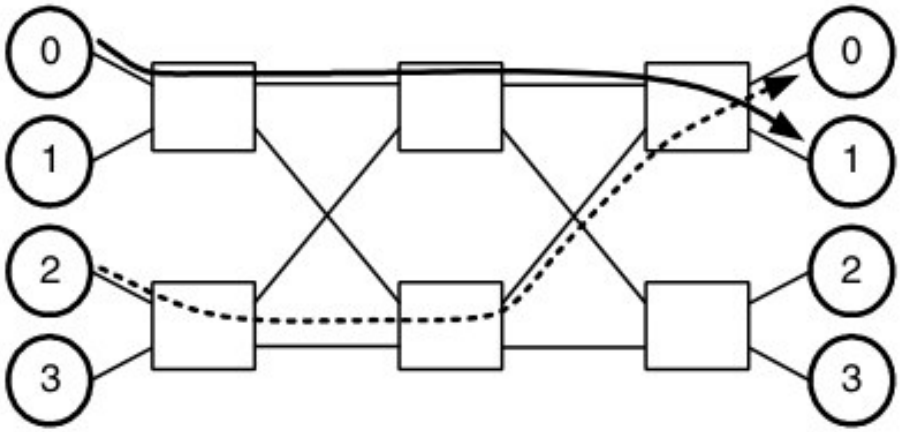


图 15.5 采用虚电路资源预留隔离突发流与非突发流

当需要极其严格的保证时，时分复用（TDM）提供了最严格的控制。为了避免流随时间共享资源而引入的变化，TDM 在时间和空间上“锁定”特定流所需的所有资源。由于不允许其他流访问这些资源，它们始终可供所分配的流使用，从而易于维持保证。TDM 实现将时间划分为固定数量的小时隙。时隙的大小和数量决定了资源分配的粒度。例如，时间可以被划分为 32 个时隙，每个时隙等于单个 flit 的传输时间。如果通道带宽为 1 Gbyte/s，流可以按 32 Mbytes/s 的倍数分配带宽。如果一条流需要 256 Mbytes/s，则它需要为其所需的每个资源请求 32 个时隙中的 8 个。

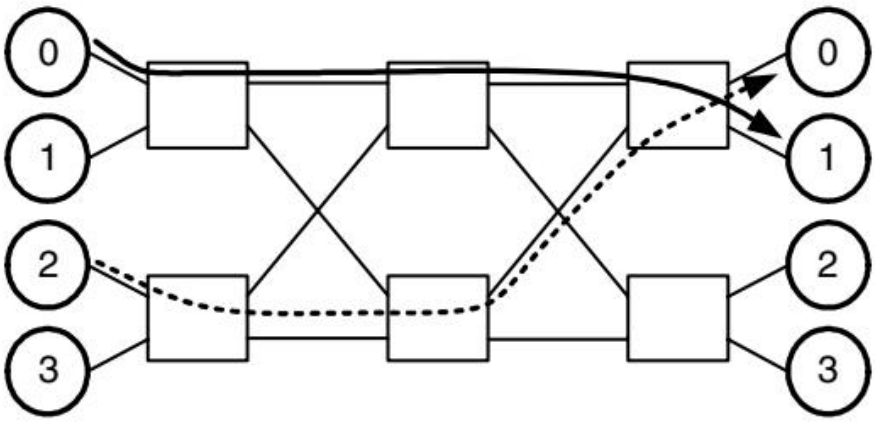
图 15.6 重新审视了 2 元 2 飞示例，其中流已使用 TDM 进行分配。尽管某些通道承载两条流，但由于四个时隙中的每一个都分配给唯一的流，因此流之间是隔离的。通过时隙分配，一条流可以与突发流共享资源，而不会增加自身的突发性。这种时隙分配既可以在所需连接预先已知的情况下离线计算，也可以在连接随时间添加和移除的情况下在线计算。无论哪种情况，寻找“最优”

分配通常是 NP 难的，因此大多数实际实现求助于启发式方法。练习 15.5 中探讨了一种启发式方法。

如图 15.6 所示，可以为网络中的每个资源关联一个时间轮，以存储其分配调度。然后，时间轮表中的指针指示当前时隙以及该时隙内资源的所有者。在此示例中，未使用的时隙标记为“BE”，表示该资源可用于尽力而为流量。已使用的时隙与通道的资源分配相匹配。随着时间推进，指针递增到下一个表项，当到达表底部时回绕到顶部。

15.4 尽力而为服务的实现

实现尽力而为服务时，关键的服务质量问题是在所有尽力而为流之间提供公平性。当资源未完全预先预留时，保证服务的流内部也可能出现类似问题。我们提出两种公平性定义：延迟公平性和吞吐量公平性，并讨论它们的实现问题。



虚电路资源预留下的两条流。通过选择独立的网络路径，避免了突发流（实线）和非突发流（虚线）之间的耦合。

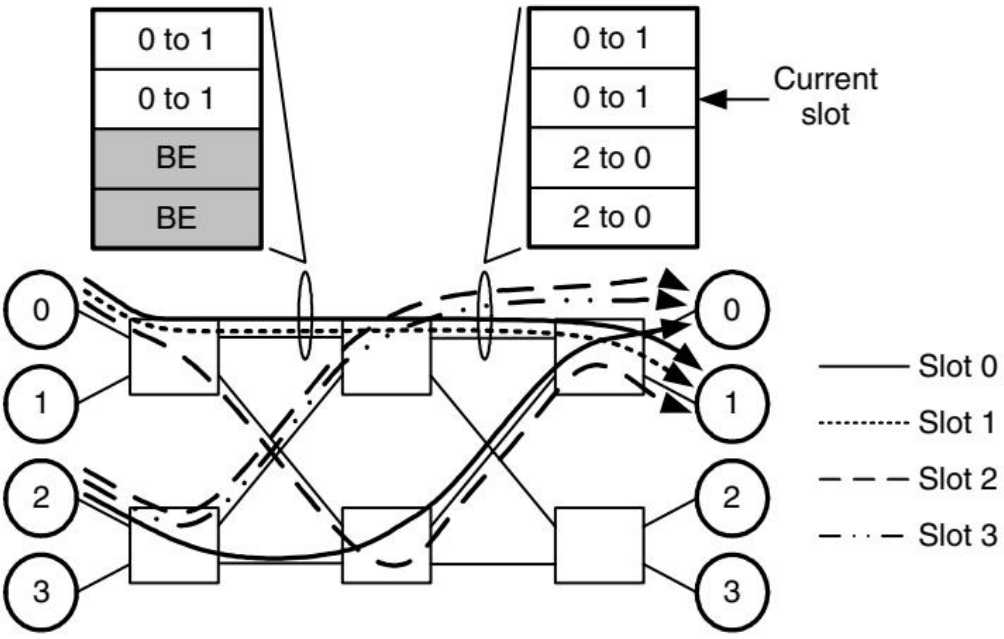


图 15.6 4 时隙 TDM 网络中的流分配，以及来自网络两个通道的时间轮。

15.4.1 延迟公平性

基于延迟的公平性的目标是为竞争相同资源的流提供相等的延迟。为了了解这种方法的实用性，考虑一个汽车离开拥挤停车场的例子，如图 15.7 所示。停车场的每一列都标有一个字母，并包含一排等待转入出口行（通向停车场出口）的汽车。汽车标有

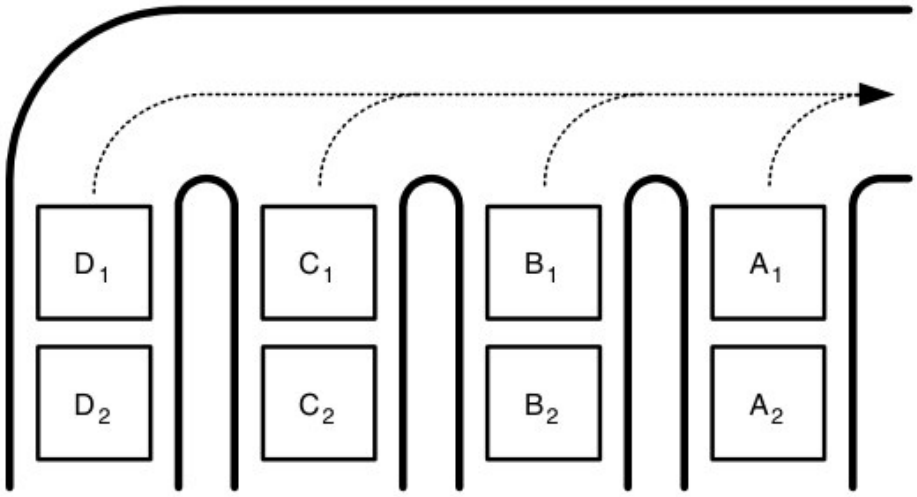
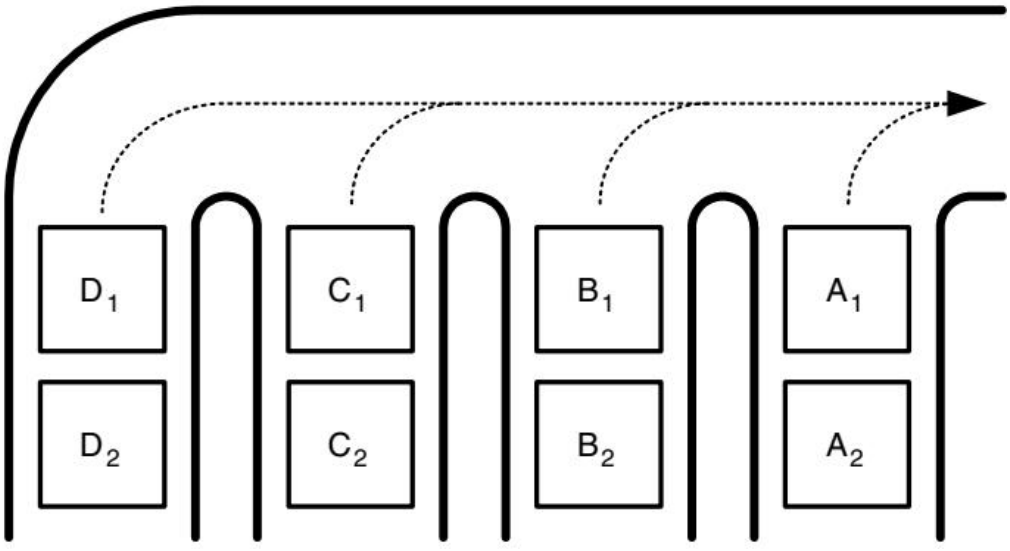


图 15.7 停车场模型：多个等待队列竞争唯一共享出口



$$\text{LF}: A_1B_1A_2C_1A_3B_2A_4D_1A_5\dot{s}$$

$$\text{AB}: A_1B_1C_1D_1A_2B_2C_2D_2A_3\dot{s}$$

图 15.7 一个有 4 列汽车等待进入单一共享出口的停车场。汽车标有其所在列以及进入停车场的相对时间。还显示了在局部公平（LF）和基于年龄（AB）仲裁下离开汽车的顺序。

它们的列以及它们开始离开停车场的相对时间。例如，D_2 比 A_4 先开始离开。这类似于网状网络垂直通道中排队等待访问该网络共享水平通道的数据包。我们将假设每 5 秒有一辆汽车可以离开停车场。

标准的驾驶礼仪规定，在每个合流点，来自任一入口的汽车交替获得该合流点的通行权。我们称之为局部公平仲裁策略。如虚线所示，我们的停车场示例有三个合流点。现在考虑在局部公平策略下离开停车场的汽车顺序。最右列的第一辆车 A_1 首先离开，然后是 B_1 。由于右侧合流点的局部公平策略，接下来是 A_2 离开，然后是 C_1 ，依此类推。尽管 D_1 是最早等待离开停车场的汽车之一，但它必须等待 8 辆车，即 40 秒，才能最终离开。到那时，A 列已有 4 辆车离开。显然，局部公平仲裁下的延迟分布是不公平的。事实上，如果这个例子扩展到包含 24 列，最后一列的第一辆车 X_1 将不得不等待超过一年才能离开停车场！当然，这是假设停车场中有相对大量的汽车。

为了解决这个问题，我们可以将仲裁策略替换为一种延迟公平的策略。如果一个资源的最早请求者总是被优先服务，那么这种仲裁就是延迟公平的。对于我们的停车场例子，我们可以简单地使用每辆车的相对起始时间在汇合点做出决策——在汇合点，两辆车中较早的那辆先走。这会产生一个更好的离开顺序：在任一列有两辆车离开之前，每一列都有一辆车离开。对于网络，我们将这种策略称为基于年龄的仲裁。当多个数据包竞争一个资源时，最老的（以自注入网络以来的时间衡量）先获得访问权。

15.4 尽力服务的实现

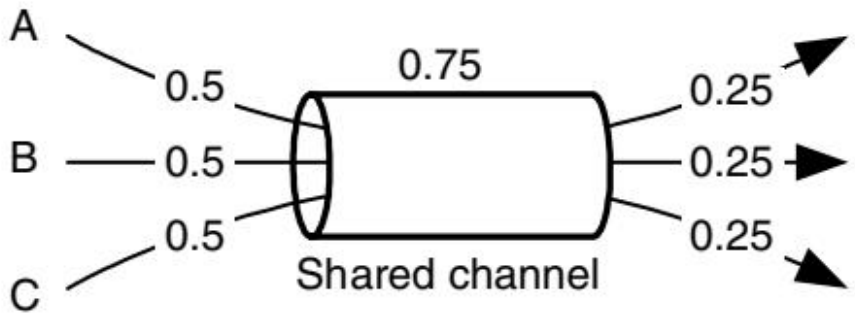
虽然基于年龄的仲裁极大地提高了网络的延迟公平性，但它通常仅作为真正延迟公平网络的局部近似来使用。这一警告的出现是因为高优先级（较老）的数据包可能会被低优先级（较年轻）的数据包阻塞，这被称为优先级反转。正如我们将看到的，吞吐量公平性中也会出现类似的问题，两者都可以通过构建一个无干扰网络来解决，我们将在第 15.5 节中讨论。

15.4.2 吞吐量公平性

延迟公平性的另一种替代方案是吞吐量公平性，它旨在为竞争同一资源的流提供相等的带宽。图 15.8（a）用三个数据包流穿过网络中的单个共享通道说明了这一想法。如图所示，每个流以每周期 0.5 个数据包的速率请求，但通道总共只能支持每周期 0.75 个数据包。自然地，吞吐量公平的仲裁将简单地在三个流之间划分可用带宽，使得每个流通过通道获得每周期 0.25 个数据包。

当每个流的速率不再相等时，这个例子变得更加复杂。图 15.8（b）显示了速率已改为每周期 0.15、0.5 和 0.5 个数据包的情况。在这种情况下，许多合理的公平性定义可能导致不同的分配，但用于吞吐量的最常见的公平性定义是最大最小公平性。如果一个分配不能在不减少具有相等或更少分配的流的分配的情况下增加任何流的分配，则

该分配是最大最小公平的。图 15.8（b）中显示的结果分配是最大最小公平的。
在算法上，最大最小公平性可以通过以下过程实现。对于 N 个流，设 b_i 为第 i 个流请求的带宽，其中 $0 \leq i < N$。带宽请求也按 $b_{i-1} \leq b_i$ 排序，对于 0 < i < N。然后使用以下递推关系分配带宽：



(b)

(a)

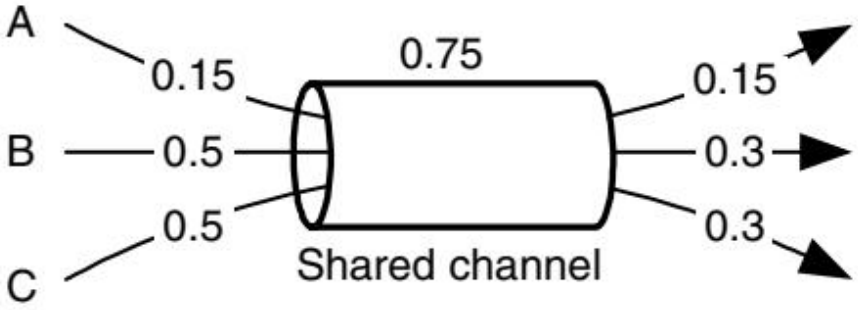


图 15.8 三个流共享单个通道的吞吐量公平带宽分配：（a）流具有相等请求时的分配，（b）不等请求时的最大最小公平分配。

$$
\begin{aligned}
R_0 &= b, \\
a_i &= \min\left[b_i, \tfrac{R_i}{N-i}\right], \\
R_{i+1} &= R_i - a_i,
\end{aligned}
$$

其中 b 是资源的总带宽，R_i 是调度 i 个请求后可用带宽量，a_i 是分配给请求 i 的带宽量。该算法首先满足最小的请求，每个请求的任何多余带宽均匀分配给剩余的较大请求。
最大最小公平性可以在硬件中通过将每个请求资源的流分离到单独的队列来实现。然后，队列以轮询方式服务。任何空队列都被简单跳过。这种实现通常被称为公平排队。虽然在概念上并不困难，但几个实际问题可能使实现复杂化。例如，如果数据包长度不等，则需要额外的工作，而加权公平排队增加了对某些流加权以获得更高优先级的能力。正如我们在延迟公平性中看到的，真正的吞吐量公平性还需要在网络中的每个资源处维护每流结构。

15.5 资源分离

为了满足服务和公平性保证，我们通常需要隔离不同类别的流量。有时，我们还需要区分一个类别内的流。为简洁起见，在本节中我们将类别和流统称为类别。使用 TDM 等预留技术时，实现这一点的成本在于存储资源预留所需的表。当资源动态分配时，问题更加复杂。理想情况下，算法可以全局调度资源，使得类别之间不会相互影响。然而，互连网络是分布式系统，因此全局方法不切实际。相反，本地算法（如公平排队和基于年龄的仲裁）必须生成资源分配，并且硬件资源必须分离类别以防止一个类别的行为影响另一个类别。在介绍用于隔离流量类别的无干扰网络之前，我们讨论树饱和，这是一种可能由不良隔离引起的重要网络病态。

15.5.1 树饱和

当一个资源接收到不成比例的高流量时，网络中可能会出现一个或多个热点。热点就是任何负载高于平均资源的资源。这种现象首先在共享内存互连中观察到：一种常见的同步构造是锁，其中多个处理器连续轮询单个内存位置以获取锁并访问共享数据结构。如果特定锁位于网络中的一个节点，而许多其他节点同时访问此锁，很容易看出目标节点如何被请求淹没并成为热点。在 IP 路由器应用中，到特定输出的流量的随机波动或路由表的瞬时错误配置都可能导致一个或多个输出端口过载，从而形成热点。当然，促进资源共享是任何互连网络的重要功能，在共享锁的例子中，网络的流控最终会对请求锁的节点施加反压。这是网络的预期行为。然而，热点的一个可能出乎意料的影响是它们对不请求热点资源的网络流量的影响。当数据包在热点资源处被阻塞时，就会发生树饱和（图 15.9）。最初，与热点资源相邻的通道因请求压垮资源而阻塞，形成树的第一层。这一效应继续下去，距离资源两跳的通道等待第一层中阻塞的通道，依此类推。由此产生的阻塞资源模式形成树状结构。

如图 15.9 所示，一个数据包完全可能请求饱和树中的通道，但从不请求热点资源。如果没有充分的资源分离，这些数据包可能会在饱和树中等待通道时被阻塞，即使它们不需要访问过载的资源。树饱和是互连网络中普遍存在的问题，并不局限于目的节点过载。例如，当网络通道的负载超过其容量时，也会发生同样的效应。共享内存系统中树饱和的定量效应在练习 15.4 中探讨。

15.5.2 无干扰网络

为了实现两个类别 A 和 B 之间的隔离，A 和 B 之间不能有任何共享资源可以被 A（B）无限期持有，以至于 B（A）无法中断该资源的使用。满足这一定义的网络称为无干扰网络。例如，使用虚拟通道流控时，无干扰网络会为网络中的每个类别提供一个虚拟通道。然而，物理通道不必复制，因为它们每个周期都会重新分配，不能被单个类别无限期持有。这种划分也适用于网络输入端的缓冲区，每个客户端需要为每个类别提供单独的注入缓冲区。

虽然无干扰网络提供了满足服务和公平性保证所需的类别间分离，但其实现可能代价

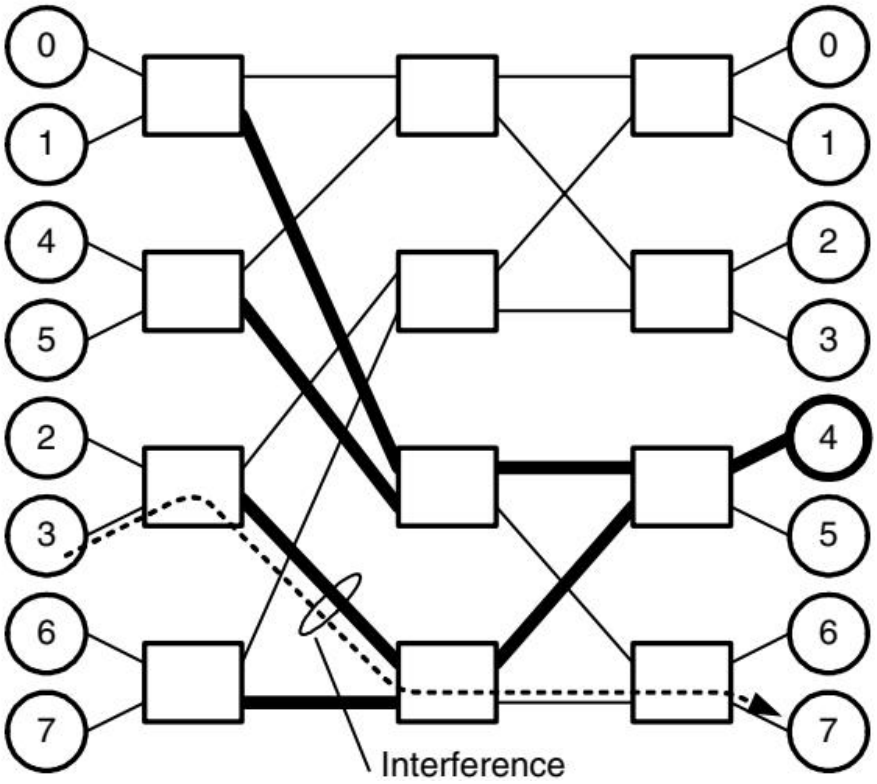


图 15.9 2 元 3 飞中的树饱和。目的节点 4 过载，导致通向它的通道阻塞。这些阻塞的通道又阻塞更多通道，形成树状模式（粗线）。然后，由于树饱和的通道，发往节点 7 的消息会发生干扰。

高昂。例如，考虑将互连网络用作互联网路由器的交换结构，我们要求发往一个输出的流量不干扰发往另一个输出的流量。在这种情况下，我们为每个输出提供一个单独的流量类别，并为每个类别提供完全独立的虚拟通道和注入缓冲区。即使在中等规模的路由器中，以这种方式提供无干扰也可能需要使用数百个虚拟通道，这对应于网络路由器中显著的复杂性（第 16 章和第 17 章）。4 为此，设计者应仔细选择需要真正隔离的类别数量。在许多情况下，可能可以合并类别而不会显著降低服务，从而降低硬件复杂性。

15.6 案例研究：ATM 服务类别

异步传输模式（ATM）是一种网络技术，旨在支持多种流量类型，特别强调语音和视频等多媒体流量，但具有足够的灵活性以高效地容纳尽力而为流量 [154]。ATM 的典型应用包括互联网和校园网骨干，以及企业内部的语音、视频和数据综合传输。

ATM 是基于连接的，因此在源-目的对之间发送任何数据之前，必须建立虚拟电路（第 15.3.2 节），以在连接源和目的的路径上预留网络资源。虽然连接是基于电路的，但 ATM 网络中的数据传输和交换是基于数据包的。ATM 的一个不寻常的特点是所有数据包（称为信元）都是固定长度的：53 字节。这减少了打包延迟并简化了路由器设计。

每个 ATM 连接根据五种基本服务类别之一进行表征：

- 恒定比特率（CBR）——恒定比特率连接，例如实时未压缩语音。

- 可变比特率，实时（VBR-rt）——低延迟和抖动都至关重要的突发连接，例如用于电话会议的压缩视频流。
- 可变比特率（VBR）——类似于 VBR-rt，但没有严格的延迟和抖动约束。
■ 可用比特率（ABR）——带宽需求大致已知，并有可能动态调整。
■ 未指定比特率（UBR）——尽力而为流量。

除 UBR 外，服务类别还需要除类别类型本身之外的附加参数来指定所需服务的参数。例如，VBR-rt 所需的 (σ, ρ) 调节流由持续信元速率（SCR）和突发容限（BT）参数指定。ATM 还提供了许多其他参数来进一步描述流的性质，例如最小和峰值信元速率（MCR 和 PCR）以及信元丢失率（CLR）。

虽然 ATM 交换机可以使用类似 TDM 的机制来提供 CBR 流量，但高效支持其他流量类型需要动态分配资源。大多数交换机提供吞吐量公平性，并且还必须分离资源以确保虚拟电路之间的隔离。

15.7 案例研究：Avici TSR 中的虚拟网络

在 9.5 节中，我们介绍了 Avici TSR 及其网络，并考察了它使用无关源路由来平衡负载的方式。在本节中，我们进一步研究这台机器，并探讨它如何利用虚拟通道来实现无干扰网络。

Avici TSR 为每一对目的节点使用单独的虚拟通道，使网络中的流量完全无干扰 [50]。也就是说，发往一个目的节点 A 的流量与发往任何其他目的节点 $B \neq A$ 的流量在网络中不共享任何缓冲区。因此，如果某个目的节点 B 过载并将流量回压到网络中，填满缓冲区并导致树饱和，这种过载不会影响发往 A 的数据包。由于发往 A 和 B 的数据包不共享缓冲区，A 的数据包与过载隔离。发往 A 和 B 的数据包确实共享物理通道带宽。然而，这不是问题，因为流量被分散以平衡交换结构通道上的负载，并且稳态下到 B 的网络负载不会超过节点 B 的输出带宽。

图 15.10 说明了这个问题。过载的目的节点 B 在所有通往 B 的链路上回压流量，如深色箭头所示。这是树饱和的一种形式（15.5.1 节）。这些链路上 B 可用的所有虚拟通道和 flit 缓冲区都已满，阻塞了进一步的流量。现在考虑一个沿灰色路径路由到节点 A 的数据包。如果该数据包与发往 B 的数据包共享资源，它将在链路 a、b 和 c 上等待虚拟通道或 flit 缓冲区而被阻塞，并无限期延迟。发往 A 的数据包在链路 d 上不会被阻塞，因为它与发往 B 的数据包方向相反，因而使用不同的虚拟通道和 flit 缓冲区集合。发往 B 的数据包消耗的带宽不是问题。假设节点 B 最多消耗一个单位的带宽，并且负载在传入路径上均匀分布，那么链路 c 上只有 $\frac{1}{12}$ 单位的带宽（链路 a 上 $\frac{1}{20}$）被发往 B 的流量消耗。这留下了充足的带宽来处理发往 A 的数据包。问题在于发往 B 的数据包占用了所有虚拟通道和 flit 缓冲区，并且由于回压而非常缓慢地释放这些资源。这个问题类似于在橄榄球比赛前试图开车去体育场（节点 B）附近的杂货店（节点 A）。你就像发往节点 A 的数据包一样，被所有前往比赛的车阻塞，等待道路清空。15.7 案例研究：Avici TSR 中的虚拟网络解决方案是为机器中的每个目的节点提供单独的虚

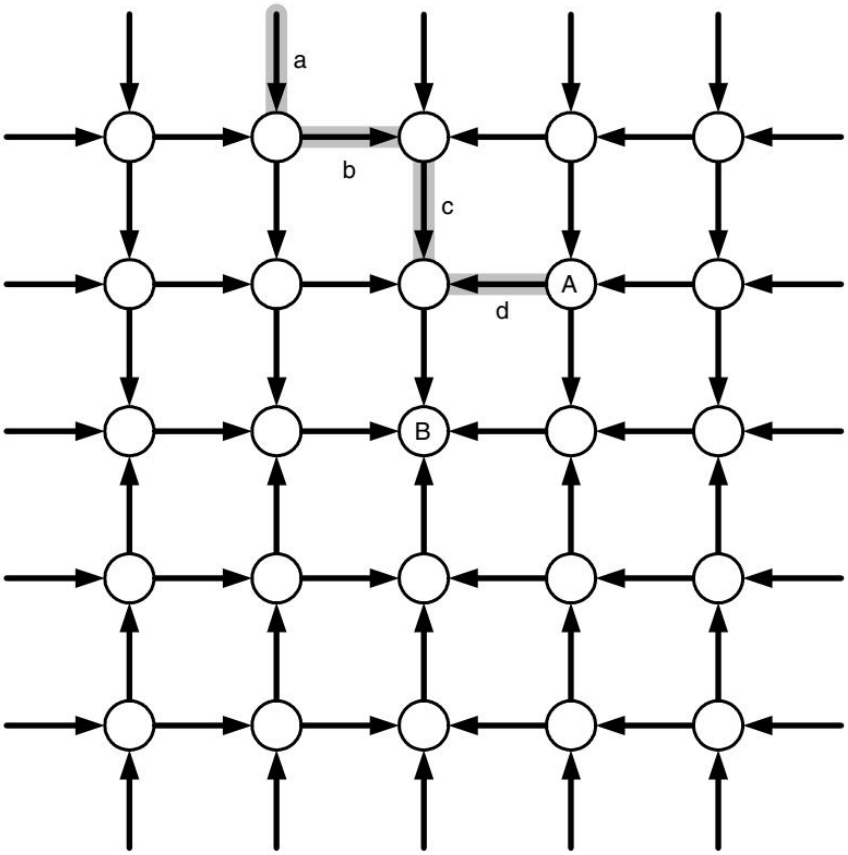


图 15.10 二维网格网络的一个片段，显示因过载目的节点 B 的树饱和而阻塞的链路。沿灰色路径 (a, b, c, d) 路由到节点 A 的数据包，除非资源保持分离，否则在除链路 d 外的所有链路上都会遇到发往 B 的数据包的干扰。

拟网络。Avici TSR 实现了这一点，并且通过为每个物理通道提供 512 个虚拟通道，为两类流量提供差异化服务。对于每个目的节点 d，TSR 在通往该目的节点的每条链路上保留两个虚拟通道。一个虚拟通道保留给普通流量，第二个虚拟通道保留给高级流量。还为每个目的节点和服务类别提供单独的源队列，以便源队列中不会发生干扰。通过这种安排，发往 A 的流量不会与发往 B 的流量竞争虚拟通道或缓冲区。发往 A 的流量仅与 B 共享物理通道带宽，除此之外不共享任何资源。因此，发往 A 的数据包能够无干扰地前进。回到我们的驾驶类比，就好像为每个目的地在道路上提供了一条单独的车道。等待前往体育场的汽车在体育场车道中排队，但不会干扰前往杂货店的汽车，后者在自己的车道中前进。互连网络的伟大之处在于，我们能够通过复制廉价资源（少量缓冲）同时共享昂贵资源（带宽）来提供这种隔离。不幸的是，这无法用道路和汽车来实现。

为了仅使用 512 个虚拟通道支持多达 512 个节点和 2 类流量，TSR 利用了一个事实：在所有维度上彼此距离最远的两个节点的最小路由不共享任何物理通道。因此，这些节点可以在每个物理通道上使用相同的虚拟通道号，而不会有干扰的危险。图 15.11 以 8 节点环为例说明了这种情况。沿最小路由前往节点 X 的数据包仅使用箭头所示的链路方向。沿最小路由前往节点 Y 的数据包仅使用相反方向的链路（未显示）。由于前往这两个节点的最小路由不共享任何通道，它们可以安全地使用相同的虚拟通道号，而不会实际共享虚拟通道，因此也不会有干扰的危险。

15.8 文献注释

Peterson 和 Davie [148] 对与 QoS 相关的一般问题以及实现问题（特别是与互联网等大规模网络相关的问题）进行了更详细的讨论。Kumar 等人 [107] 讨论了 QoS 对路由器设计的一般影响。Cruz 详细介绍了 (σ, ρ) 流及其在计算网络延迟中的效用 [42, 43]。Jaffe [88] 涵盖了网络中公平性的早期定义。吞吐公平性由 Nagle [132] 引入，并由 Demers 等人 [58] 扩展以考虑若干实际因素。真正的最大最小公平性实现起来可能很昂贵，存在几种著名的近似方法，例如 Golestani 的停等排队 [74] 和 Kim 与 Chien 提出的旋转组合排队 [97] 算法，后者专门针对互连网络设计。Yum 等人描述了 MediaWorm 路由器 [198]，它使用虚拟时钟算法 [200] 的扩展来提供基于速率的服务。其他几个商业和学术路由器也集成了不同级别的 QoS，例如 SGI SPIDER [69]、Tandem ServerNet [84] 和 MMR [63]。树饱和由 Pfister 和 Norton 提出，他们提出了合并缓冲区以整合共享内存系统中对同一内存地址的请求 [151]。一种著名的分离不同流资源的技术是虚拟输出排队，由 Tamir 和 Frazier [181] 涵盖。

15.9 练习题

15.1 突发性示例。验证图 15.1 所示的合并示例符合第 15.2.2 节通用多路复用器模型给出的延迟界限。

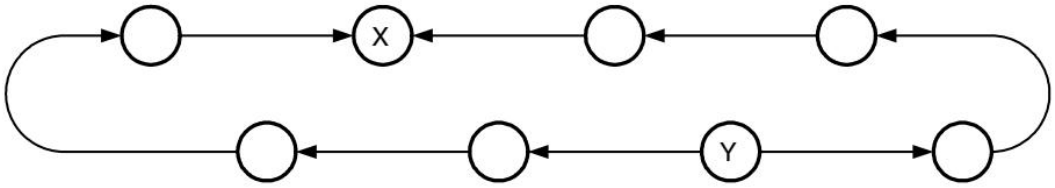


图 15.11 在这个 8 节点环上，两个彼此最大距离的目的地 X 和 Y 可以在最小路由上共享一组虚拟通道而不产生干扰。所有通向 X 的链路都背离 Y，反之亦然。

15.2 等分流的突发性。描述如何将一个以 (σ, ρ) 表征的流分成两个流，每个流的速率为 $\sigma/2$。证明两个新子流中任一个的突发性界限为 $(\sigma + L)/2$。
15.3 先到先服务多路复用器。将第 15.2.2 节的方法应用于具有先到先服务（FIFO）服务规则的多路复用器。假设多路复用器的两个输入分别是以 (σ_1, ρ_1) 表征的流和以 (σ_2, ρ_2) 表征的流，该多路复用器的最大延迟是多少?
15.4 树饱和的影响。考虑一个具有 p 个节点的系统，每个节点包含一个处理器和一个内存模块。每个节点以速率 r 生成固定长度的内存请求，其中比例为 h 的请求指向一个“热”内存位置，其余比例 1 - h 均匀分布在所有其他内存位置。进入热内存模块的请求总速率是多少? 假设一旦热模块饱和，树饱和会阻塞所有请求，p = 100，h = 0.01，系统中总内存带宽的可用比例是多少?
15.5 模拟。考虑在 4×4 网格网络中预留 TDM 流。假设时间被划分为 T 个时隙，各个流需要其中一个时隙。此外，流以贪心方式增量调度：给定特定源-目的对之间的流请求，流沿着成本最小的路径调度。路径的成本定义为沿其任何通道使用的 TDM 时隙的最大数量。然后调度的质量由支持它所需的时隙数（最大拥塞通道）决定。使用这种启发式方法调度一组随机连接，并将其与使用公式 3.9 的优化问题得到的拥塞下界进行比较。

注释

- 1. 一些网络保证不会丢弃任何数据包，即使是尽力服务数据包。
- 2. 在实践中，调节器的令牌队列可以通过一个饱和信用计数器来实现，该计数器以速率 ρ 递增，并在达到计数 σ 时饱和。只有当信用计数器非零时才能发送数据包，并且每次发送数据包时计数器递减。
- 3. 通常，采用聚合分配的网络会在每个网络输入处使用硬件来执行准入控制。该硬件仅在流量符合聚合需求界限时才允许流量进入网络。网络内部不需要额外的硬件。
- 4. Avici TSR [49] 中使用的环面网络以这种方式在多达 1,024 个类别之间提供无干扰。

第十六章　路由器体系结构

路由器架构

路由器由寄存器、开关、功能单元和控制逻辑组成，它们共同实现路由和流控制功能，这些功能是缓冲和转发在途中的 flit 到达目的地所必需的。尽管存在许多路由器组织，但在本章中，我们研究典型虚拟通道路由器的架构，并探讨路由器设计中的问题和权衡。

现代路由器在 flit 级别进行流水线处理。头 flit 经过执行路由和虚拟通道分配的流水线阶段，所有 flit 通过开关分配和开关遍历阶段。如果给定流水线阶段无法在当前周期内完成，则发生流水线停顿。这些停顿会停止停顿之前的所有流水线阶段的操作，因为 flit 必须保持顺序。

大多数路由器使用信用来分配缓冲区空间。当 flit 在通道上向下游传输时，信用向上游流动以授予对刚刚腾出的缓冲区的访问权。在缓冲区数量有限的路由器中，信用处理的延迟可能对性能产生重大影响。信用还影响死锁特性。如果在所有信用返回之前分配了虚拟通道，则会在被分配通道的数据包与仍在下游缓冲区中的任何数据包之间创建依赖关系。

16.1　基本路由器架构

16.1.1　框图

图 16.1 显示了典型虚拟通道路由器的框图。这些模块可以根据功能大致分为两组：数据路径和控制平面。路由器的数据路径处理数据包有效载荷的存储和移动，由一组输入缓冲区、一个开关和一组输出缓冲区组成。其余模块实现路由器的控制平面，负责协调数据包通过数据路径资源的移动。对于我们的虚拟通道路由器，控制模块执行路由计算、虚拟通道分配和开关分配。我们还将输入控制状态与输入缓冲区关联，形成输入单元，输出类似。

数据包的每个 flit 到达路由器的输入单元。输入单元包含一组 flit 缓冲区，用于保存到达的 flit 直到它们可以被转发，并且还维护与该输入链路关联的每个虚拟通道的状态。通常，维护五个状态字段来跟踪每个虚拟通道的状态，如表 16.1 所述。

要开始推进数据包，必须首先执行路由计算以确定数据包可以转发到的输出端口（或多个端口）。1 给定其输出端口，数据包向虚拟通道分配器请求输出虚拟通道。一旦确定了路由并分配了虚拟通道，数据包的每个 flit 通过使用开关分配器在开关和输出通道上分配一个时隙并在该时隙期间将 flit 转发到适当的输出单元，从而通过该虚拟通道转发。2 最后，输出单元将 flit 转发到数据包路径中的下一个路由器。与输入单元

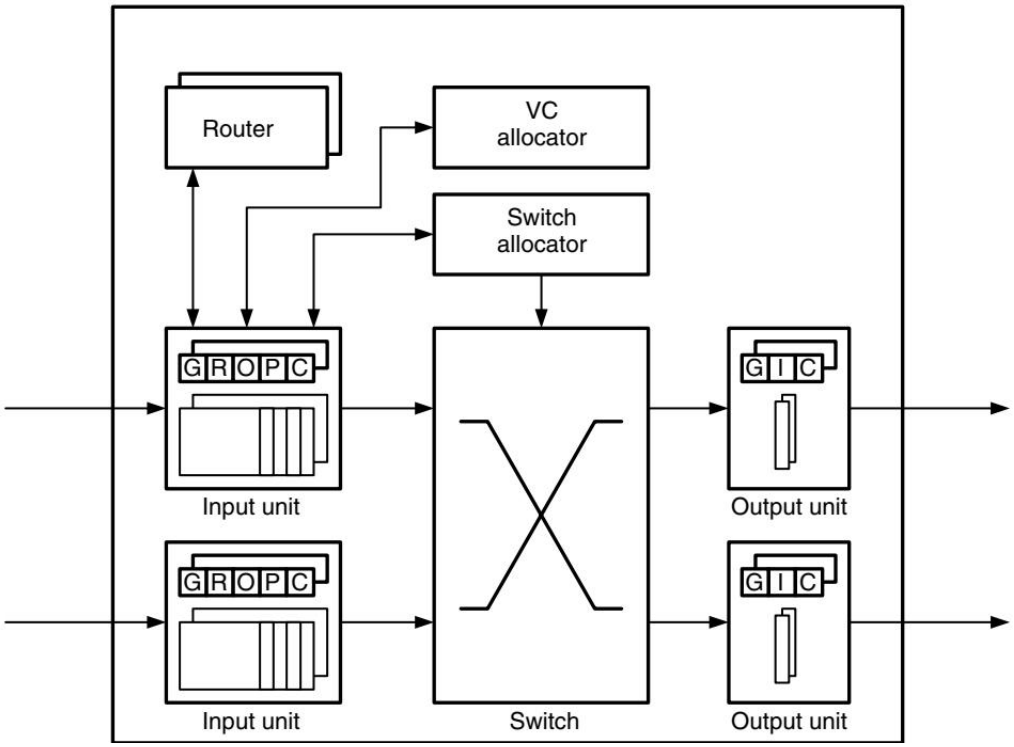


图 16.1 虚拟通道路由器框图。数据路径由每个输入端口的一个输入单元、每个输出端口的一个输出单元以及连接输入单元到输出单元的开关组成。三个模块做出决策并执行分配：数据包由路由器分配输出端口，由虚拟通道分配器分配输出虚拟通道。然后每个 flit 由开关分配器分配一个开关时隙。

一样，几个状态字段包含每个输出虚拟通道的状态（表 16.2）。第 16 章路由器架构

表 16.1 输入虚拟通道的状态字段

字段	名称	说明
G	全局状态	虚拟通道可能处于空闲（I）、路由计算（R）、等待输出虚拟通道（V）、活动（A）或等待信用（C）状态。
R	路由信息	路由计算完成后，本字段保存为该数据包选择的输出端口。
O	输出虚拟通道	虚拟通道分配完成后，本字段保存输出端口 R 上分配给该数据包的输出虚拟通道。
P	缓冲区指针	指向输入缓冲区中微片队列首、尾的指针；还可利用两者推导该虚拟通道的缓冲微片数量。
C	信用计数	输出端口 R 上输出虚拟通道 O 的信用数量，即下游可用微片缓冲区的数量。

路由器的控制以两种不同的频率运行：包速率和微片速率。路由计算和虚拟通道分配每个包执行一次。因此，虚拟通道的 R、O 和 I 状态字段每个包更新一次。另一方面，交换机分配以每个微片为基础执行。类似地，P 和 C 状态字段以微片频率更新。

表 16.2　输出虚拟通道的状态字段

字段	名称	说明
G	全局状态	虚拟通道可处于空闲（I）、活动（A）或等待信用（C）状态。
I	输入虚拟通道	当前正在向本输出虚拟通道转发微片的输入端口及输入虚拟通道。
C	信用计数	下游节点可用于容纳本虚拟通道所发送微片的空闲缓冲区数量。

16.1 基本路由器架构

16.1.2　路由器流水线

图 16.2 显示了一个甘特图，说明了典型虚拟通道路由器的流水线。为了前进，每个头微片必须经过路由计算（RC）、虚拟通道分配（VA）、交换机分配（SA）和交换机遍历（ST）等步骤。该图反映了一个路由器，其中每个步骤占用一个时钟周期——每个步骤都有自己的流水线阶段。3 图 16.2 显示了数据包在没有任何停顿的情况下通过流水线阶段的情况。正如我们稍后将讨论的，停顿可能发生在任何阶段。

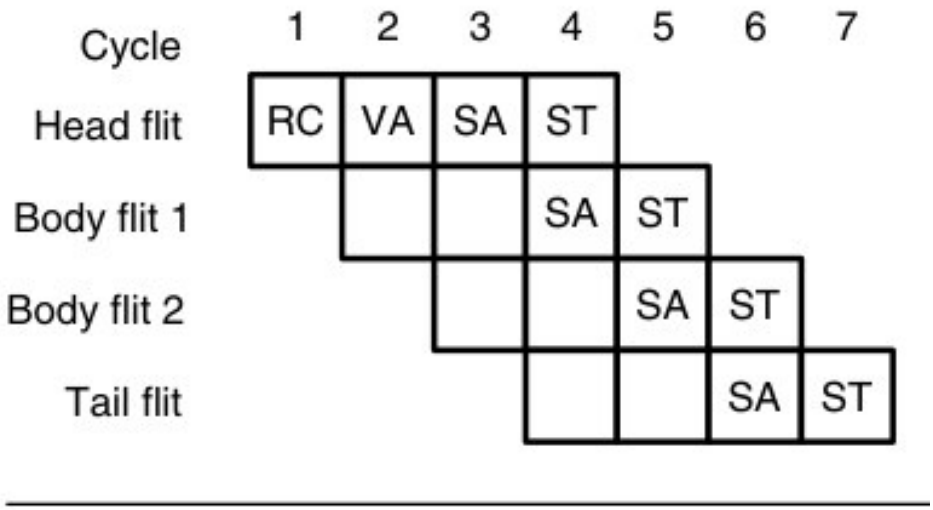
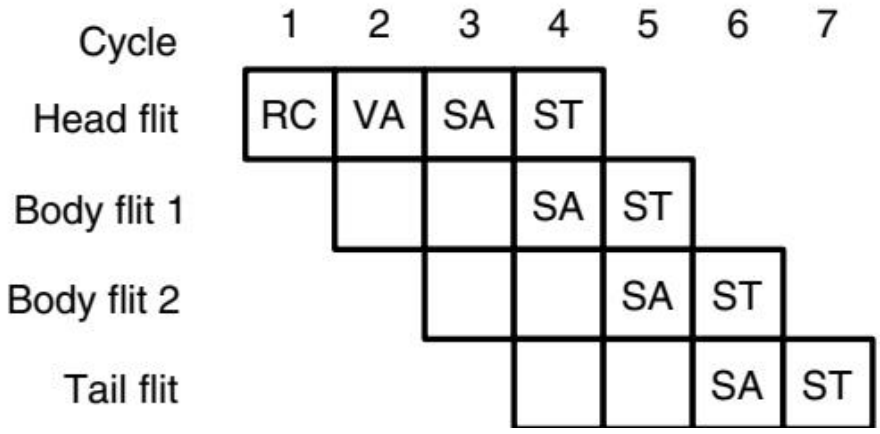


图 16.2　数据包的流水线化路由：路由计算、虚拟通道分配、交换分配与交换穿越

路由过程在数据包的头微片于周期 0 到达路由器时开始。数据包被引导到输入端口的一个特定虚拟通道，如头微片的字段所示。此时，目标虚拟通道的全局状态（G）处于空闲状态。头微片的到达导致虚拟通道在周期 1 开始时转换到路由（G = R）状态。在周期 1 期间，路由器块使用头微片的信息来选择输出端口（或多个端口）。此计算的结果更新虚拟通道状态向量的路由字段（R），并在周期 2 开始时将虚拟通道推进到等待输出虚拟通道（G = V）状态。与 RC 并行，第一个体微片到达路由器。

在周期 2 期间，头微片进入 VA 阶段，第一个体微片通过 RC 阶段，第二个体微片到

达路由器。在 VA 阶段，保存在虚拟通道状态向量的 R 字段中的路由计算结果被输入到虚拟通道分配器。如果成功，分配器在 R 字段指定的输出通道上（或其中一个）分配一个输出虚拟通道。VA 的结果更新输出虚拟通道字段（O）



数据包的流水线路由。数据包的每个微片经过路由计算（RC）、虚拟通道分配（VA）、交换机分配（SA）和交换机遍历（ST）阶段。RC 和 VA 阶段仅对头微片执行计算（每个包一次）。体微片通过这些控制阶段而不进行计算。SA 和 ST 阶段对数据包的每个微片进行操作。在没有停顿的情况下，数据包的每个微片比前一个微片晚一个周期进入流水线，并且每个周期通过一个阶段。

输入虚拟通道状态向量，并将虚拟通道转换到活动（G = A）状态。分配的结果还将所选输出虚拟通道状态向量的全局字段更新为活动（G = A）状态，并设置输出虚拟通道状态向量的输入（I）字段以标识成功的输入虚拟通道。从此时起直到尾微片释放通道，输出虚拟通道的 C（信用）字段也反映在输入虚拟通道的 C 字段中。

为了死锁分析的目的，VA 阶段是创建从输入虚拟通道到输出虚拟通道的依赖关系的点。在单个输出虚拟通道分配给数据包后，输入虚拟通道将不会被释放，直到数据包能够将其全部内容移动到下一个节点上输出虚拟通道的缓冲区中。

在周期 3 开始时，所有每包处理都已完成，所有剩余控制是 SA 阶段的逐微片交换机分配。头微片开始此过程，但处理方式与其他微片没有区别。在此阶段，任何包含缓冲微片（由 P 指示）并且下游缓冲区可用（C > 0）的活动虚拟通道（G = A）为其输入虚拟通道到包含其输出虚拟通道的输出单元的交换机争取一个单微片时隙。根据交换机的配置，此分配可能不仅涉及对交换机输出端口的竞争，还涉及与同一输入单元中的其他虚拟通道对交换机输入端口的竞争。如果交换机分配投标成功，则更新指针字段以反映头微片在虚拟通道输入缓冲区中的离开，信用字段递减以反映远端的一个缓冲区已分配给离开的头微片，并生成一个信用以通知前一个路由器此输入虚拟通道上有一个缓冲区可用。

成功的交换机分配如图 16.2 在周期 3 所示，头微片在周期 4 期间遍历交换机（ST 阶段）。由于我们假设交换机否则为空，头微片可以在周期 5 开始无竞争地遍历到下一个路由器的通道。

在头微片被分配虚拟通道时，第一个体微片处于 RC 阶段；当头微片处于 SA 阶段时，

第一个体微片处于 VA 阶段，而第二个体微片处于 RC 阶段。然而，由于路由和虚拟通道分配是按包进行的功能，体微片在这些阶段无事可做。体微片不能跳过这些阶段直接进入 SA 阶段，因为它们必须保持顺序并位于头微片之后。因此，体微片从到达时起，直到进入 SA 阶段，都只是存储在虚拟通道的输入缓冲区中。在没有停转的情况下，需要三个输入缓冲区位置来在任何时刻容纳流水线中的三个体微片。我们将在下文看到，需要更大的缓冲区来防止流水线因信用停转而产生气泡。

当每个体微片到达 SA 阶段时，它像头微片一样为交换机和输出通道进行竞标。这里的处理是按微片进行的，体微片与头微片没有区别。当交换机时隙分配给每个微片时，虚拟通道更新 P 和 C 字段，并生成信用以反映其离开。

最后，在第 6 个周期，尾微片到达 SA 阶段。尾微片的分配方式与头微片和体微片相同。当尾微片在第 6 个周期成功调度时，该包在第 7 个周期开始时释放虚拟通道：将输出虚拟通道状态设为空闲（G = I），如果输入缓冲区为空，则将输入虚拟通道状态设为空闲（G = I）。如果输入缓冲区不为空，下一个包的头微片已经在缓冲区中等待。在这种情况下，状态直接转换为路由（G = R）。

16.2 停转

在前面的讨论中，我们假设了理想情况：包和微片在流水线中无延迟地前进——分配总是成功，通道、微片和信用在需要时可用。在本节中，我们研究表 16.3 中列出的六种情况，其中这种理想行为被停转打断。前三种停转是包停转——流水线包处理功能中的停转。包停转阻止虚拟通道进入其 R、V 或 A 状态。一旦虚拟通道有包处于活动（A）状态，包处理就完成了，只会发生微片停转。微片停转发生在微片未能成功完成交换机分配的周期中。这可能是由于缺少微片、缺少信用，或者仅仅是在交换机时隙竞争中失败。

表 16.3 路由器流水线停顿类型

停顿类型	P/F	状态	说明
VC Busy	P	$\neg I$	相邻数据包停顿：前一个数据包的尾微片尚未完成交换机分配，下一个数据包的头微片就已到达。新头微片需在缓冲区等待；前一尾微片完成交换机分配后，新包才进入路由状态。
Route	P	R	路由计算未完成；输入虚拟通道保持 R 状态，并在下一个周期重试。
VC Allocation	P	V	虚拟通道分配失败；输入虚拟通道保持 V 状态，并在下一个周期重试分配。
Credit	F	A	当前没有可用信用；收到下游虚拟通道返回的信用之前，微片不能尝试交换机分配。
Buffer Empty	F	A	输入缓冲区没有可用微片，可能是上游停顿所致；只有新微片到达后才能尝试交换机分配。

Switch	F	A	交换机分配已尝试但未成功；微片留在分配流水线阶段，在下一周期重试。

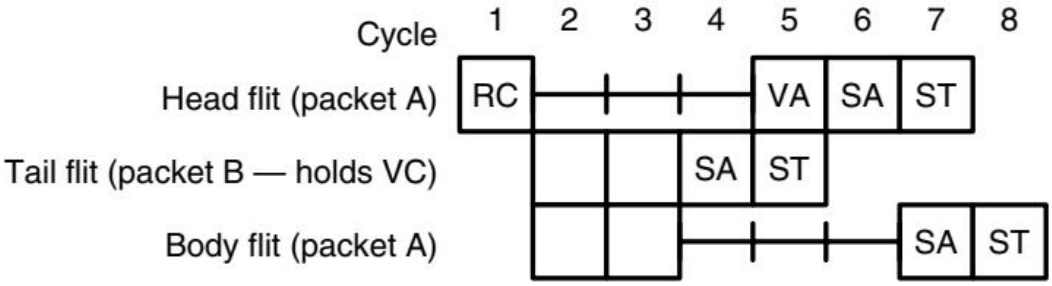


图 16.3 虚拟通道分配停转——包停转的一个例子。包 A 在第 1 个周期到达，但直到第 5 个周期，在包 B 释放 V 之后，才能分配虚拟通道 V。在等待虚拟通道期间，流水线停转了三个周期。

作为包停转的一个例子，图 16.3 展示了虚拟通道分配停转。包 A 的头微片在第 1 个周期完成路由计算，并从第 2 个周期开始竞标输出虚拟通道。不幸的是，与其路由兼容的所有输出虚拟通道都在使用中，因此该包及其头微片未能获得输出虚拟通道，无法取得进展。竞标每个周期重复但未成功，直到包 B 的尾微片在第 4 个周期完成交换机分配并释放输出虚拟通道（该通道是 A 可以使用的）。在第 5 个周期，包 A 成功完成虚拟通道分配，并获取了 B 刚刚释放的通道。它继续在第 6 个周期进行交换机分配，在第 7 个周期进行交换机遍历。注意，由于 A 的头部在 B 的尾部完成交换机分配之前无法执行虚拟通道分配，因此在转发 B 的尾微片和 A 的头微片之间，输出虚拟通道上有一个空闲周期。A 的体微片也会停转，因为第一个体微片在头微片完成此阶段之前无法进入交换机分配。然而，只要有足够的缓冲区空间（在本例中为六个微片）来容纳到达的头微片和体微片，直到它们能够开始交换机遍历，这种虚拟通道停转就不一定会减慢输入通道上的传输。

图 16.4 展示了交换机分配停转，这是微片停转的一个例子。头微片和第一个体微片无延迟地通过流水线。然而，第二个体微片在第 5 个周期未能分配交换机。该微片在停转一个周期后，于第 6 个周期成功重新尝试分配。此微片之后的所有体微片也会延迟。如上所述，如果提供了足够的缓冲区空间来容纳到达的微片，这种停转不一定会减慢输入通道上的传输。信用停转具有相同的时序图，但不是交换机分配失败，而是由于缺少信用而根本不尝试交换机分配。图 16.5 展示了缓冲区空停转。头微片和第一个体微片无延迟地前进，但第二个体微片直到第 6 个周期才到达路由器。第二个体微片跳过 RC 和 VA 阶段（它通常会在输入缓冲区中等待这些阶段），并在到达后的第 7 个周期直接进入 SA 阶段。第二个微片之后的体微片也可以直接进入 SA 阶段。

16.2 停转

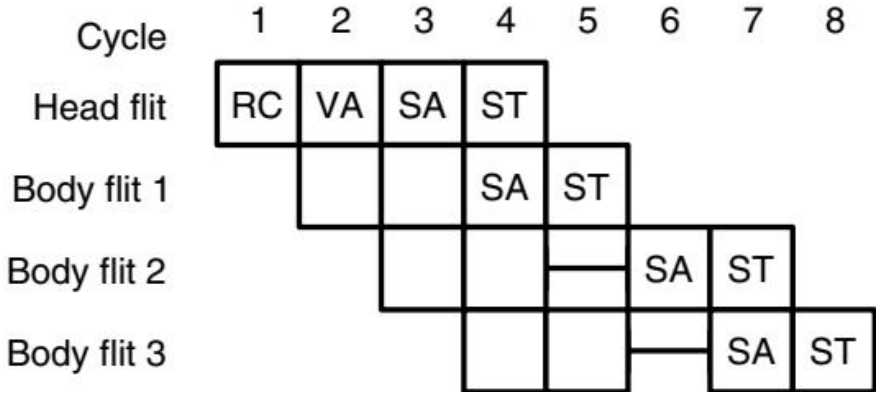


图 16.4 交换机分配停转——微片停转的一个例子。这里，包的第二个体微片在第 5 个周期未能分配通过交换机的请求连接，因此无法继续。该请求在第 6 个周期成功重试，导致一个周期的停转，延迟了体微片 2 和所有后续微片。

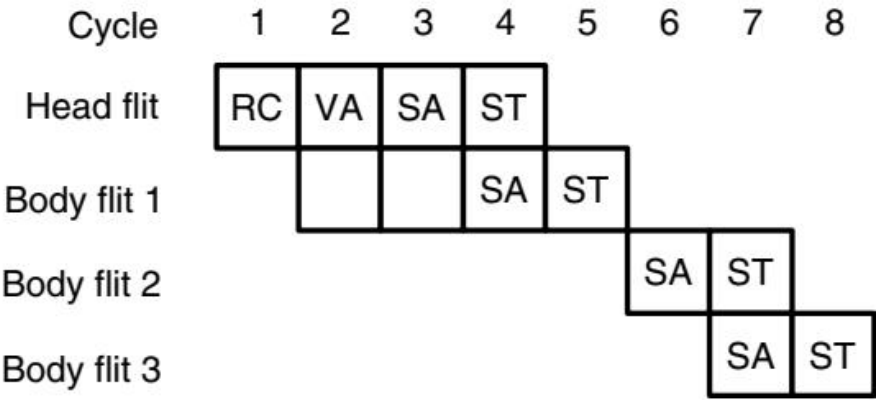


图 16.5 缓冲区空停顿——另一种类型的微片停顿。这里，体微片 2 延迟了三个周期，在第 6 周期到达而不是第 3 周期。它离开路由器仅延迟了一个周期；然而，另外两个周期被绕过空的 RC 和 VA 流水线级所吸收。

16.3 用信用关闭环路

当微片离开链路发送端或上游侧的 SA 级时，会为其分配一个缓冲区。为了重用这个缓冲区，在同一个微片离开链路接收端或下游侧的 SA 级后，会通过反向通道返回一个信用。当这个信用到达链路上游侧的输入单元时，缓冲区就可以重用了。因此，代表缓冲区的令牌可以看作是在一个环路中旅行：从链路上游侧的 SA 级开始，随微片向下游移动，到达链路下游侧的 SA 级，然后作为信用返回上游。缓冲区仅在微片位于上游侧的前三个流水线级（RC、VA 和 SA）期间被该微片占用。环路中的其余时间代表缓冲区空闲的开销期。

信用环路延迟 t_{crt}（以微片时间表示）给出了信道以全带宽运行而不发生信用停顿时，上游侧所需的微片缓冲区数量的下界。由于每个微片需要一个缓冲区，并且在它们的令牌遍历信用环路之前缓冲区无法回收，如果缓冲区少于 t_{crt} 个，那么在第一个信用返回之前缓冲区就会耗尽。以微片时间表示的信用环路延迟由下式给出：

$$t_{crt} = t_f + t_c + 2T_w + 1, \tag{16.1}$$

其中 t_f 是微片流水线延迟，t_c 是信用流水线延迟，T_w 是单向线延迟。4 如果每个虚拟通道可用的缓冲区数量为 F，则信道上的占空比为

$$d = \min\left(1, \frac{F}{t_{crt}}\right). \tag{16.2}$$

只要存在足够的微片缓冲区来覆盖往返延迟（如前面公式 13.1 所述），占空比将为 1（达到全带宽）。当微片缓冲区数量低于此数值时，信用停顿将导致一个虚拟通道的占空比成比例降低。当然，其他虚拟通道可以利用因信用停顿而空闲的物理信道带宽。

图 16.6 说明了信用环路的时序以及因缓冲区不足可能导致的信用停顿。该图显示了一个 6 微片数据包（1 个头微片后跟 5 个体微片）穿过远端有 4 个可用微片缓冲区的信道时的时序。上游流水线级显示为白色，下游流水线级显示为灰色阴影。为简单起见，微片时间等于一个时钟周期。假设每个方向穿过物理链路需要两个时钟周期，标记为 W1 和 W2——因此 $T_w = 2$ 个周期。信用传输在上游节点的流水线级 CT 中进行，响应信用更新信用计数在下游节点的流水线级 CU 中进行。

图 16.6 中的流水线具有 $t_f = 4$ 个周期的微片延迟和 $t_c = 2$ 个周期的信用延迟（CT 和 CU 级）。当 $T_w = 2$ 个周期时，公式 16.1 给出总往返信用延迟为 $t_{crt} = 11$ 个周期。因此，从头微片在第 1 周期处于 SA 级到体微片 4 重用头微片的信用进入 SA 级，共有 11 个周期。公式 16.1 中的最后一项（那个 1）是由于 SA 级在微片的前向路径中出现了两次。仅有 $F = 4$ 个缓冲区时，公式 16.2 预测占空比将为 4/11，这是正确的。

16.4 重新分配通道

正如信用允许微片缓冲区被回收一样，尾微片的通过使得输出虚拟通道可以被回收。然而，当我们回收一个输出虚拟通道时，它可能对网络的死锁特性产生重大影响。

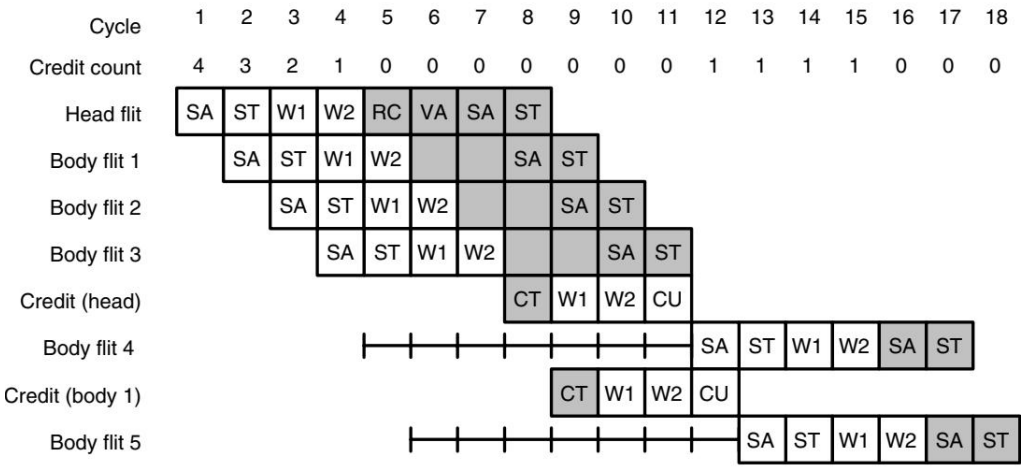


图 16.6 信用停顿：当微片在第 1 周期处于上游 SA 级时，为其分配一个缓冲区。该缓冲区不能被重新分配给另一个微片，直到头微片离开下游 SA 级，释放缓冲区，并且反映空闲缓冲区的信用在第 11 周期传播回上游信用更新（CU）级。体微片 4 使用这个信用在第 12 周期进入 SA 级。

当数据包 A 的尾微片进入 ST 级时，分配给 A 的输出虚拟通道 x 可以由虚拟通道分配器分配给不同的数据包 B，如图 16.3 所示。数据包 B 可以来自任何输入虚拟通道：与 A 相同的输入虚拟通道、同一输入上的不同虚拟通道，或不同输入上的虚拟通道。然而，此时将 x 分配给 B 会创建从 B 到 A 的依赖关系。因为 B 将位于虚拟通道 x 的输入缓冲区中 A 的后面，如果 A 被阻塞，B 将不得不等待 A。如果我们不是在上游路由器的尾微片进入 ST 级时重新分配输出虚拟通道 x，而是等到尾微片进入下游路由器的 ST 级，我们就可以消除这种依赖。通过等待 A 腾空下游路由器的输入缓冲区，数据包 B 确保它不会分配一个前面有被阻塞数据包的输出虚拟通道。这种等待对于某些路由算法（特别是自适应路由算法）的死锁特性至关重要，这些算法假设当虚拟通道被分配时，它是空的。在实践中，上游路由器不知道 A 何时进入下游路由器的 ST 级，而是等到来自尾微片的信用返回，使信用计数达到最大值，然后再重新分配虚拟通道。当许多背靠背的短数据包穿过单个虚拟通道而共享同一物理信道的其他虚拟通道空闲时，这种等待可能导致相当大的带宽损失。

图 16.7 展示了重新分配输出虚拟通道的三种方案。三个子图（a）到（c）都显示了一个两微片包 A，其后跟着包 B 的头微片。子图（a）还显示了来自 A 的尾微片的信用量。其他子图中未显示信用量。

图 16.7 将输出虚拟通道（OVC）重新分配给新包。(a）最保守的方法是等到虚拟通道的下游微片缓冲区完全为空，即来自尾微片的信用量到达时。这避免了在当前包和占用下游缓冲区的包之间建立依赖关系。(b）如果能够承受建立依赖关系，我们可以在前一个包的尾微片完成 SA 级后立即重新分配虚拟通道。我们不需要等待其信用量返回。然而，除非复制状态，否则新包在下游路由器上无法进入 RC 级，直到尾微片完成本路由器上的 SA 级。这是因为每个虚拟通道的全局状态一次只能处理一个包。(c）如果扩展全局状态以允许两个包同时位于虚拟通道上，新包在旧包仍活跃时进行路由，我们可以避免第二次停顿，代价是增加一些控制复杂度。

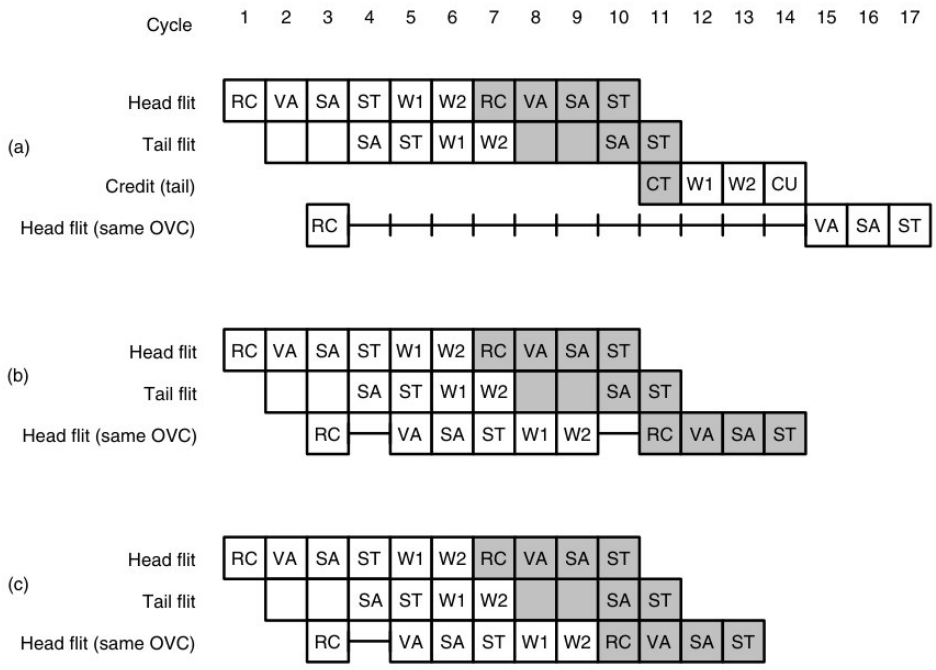


图 16.7 将输出虚拟通道重新分配给新数据包的不同策略

子图（a）展示了在重新分配虚拟通道之前等待来自尾微片的信用量的情况。在这种情况下，B 的头微片在第 2 周期到达，在第 3 周期完成 RC 级，然后进入虚拟通道停顿 12 个周期，直到第 15 周期最终完成 VA 级。这里的延迟是 $t_{crt}+1$，因为 VA 级的延迟被加到正常的信用量循环中。

图 16.7 的子图（b）和（c）展示了在尾微片进入 ST 级后立即重新分配输出虚拟通道的情况，如图 16.3 所示。在两个子图中，包 B 在仅一个停顿周期后就能在第 5 周期完成 VA 级。

子图（b）和（c）在链路下游侧输入虚拟通道重新分配的时机上有所不同。在子图（b）中，包 B 直到第 11 周期才被允许完成 RC 级，此时 A 的尾微片已进入下游 ST 级，完全释放了输入虚拟通道。这导致一个周期的虚拟通道忙停顿，但略微简化了路由器逻辑，因为虚拟通道从处理 A 到处理 B 的过渡是干净的。VC 状态在第 7 至 10 周期反映 A，从第 11 周期起反映包 B。

子图（c）展示了一种更激进的方法，它消除了虚拟通道忙停顿，代价是更复杂的虚拟通道控制逻辑，可以同时处理处于不同阶段的两个包。这里，我们允许 B 在尾微片完成 SA 级之前进入 RC 级。在这种情况下，输入虚拟通道同时处理包 A 和 B，因此必须跟踪两者的状态（例如，在状态向量中使用两个 G 字段）。此外，如果 B 完成 RC 级，但 A 的尾部未能通过开关分配，则 R 状态字段将属于 B，而 O、P 和 C 字段属于 A。这种情况可能持续几个周期。如果发生这种情况，B 会遇到虚拟通道忙停顿，但发生在 VA 级而不是 RC 级。

16.5 推测与前瞻

互连网络的延迟与流水线深度直接相关。因此，有利的做法是将流水线减少到尽可能少的级数，只要这不会增加每一级的延迟。我们可以通过使用推测来减少级数，使我们能够并行执行原本顺序执行的操作。我们还可以通过提前执行计算来将其从关键路径中移除，从而减少级数。

我们可以通过推测性地操作路由器来减少路由器中的流水线级数，而不必在每个级中塞入更多逻辑。例如，我们可以推测我们将被授予一个虚拟通道，并与虚拟通道分配并行执行开关分配，如图 16.8 所示。这将头微片的流水线减少到三级，并导致体微片在 RC 级期间仅空闲一个周期。一旦完成包的路由计算，我们就可以获取结果虚拟通道集，并同时竞标这些虚拟通道之一和相应物理通道上的开关时隙。5 如果包未能分配虚拟通道，流水线停顿，并且 SA 和 VA 在下一周期重复。如果包分配了虚拟通道，但开关分配失败，流水线再次停顿。然而，下一周期仅需要开关分配，并且由于包已分配输出虚拟通道，分配不再是推测性的。

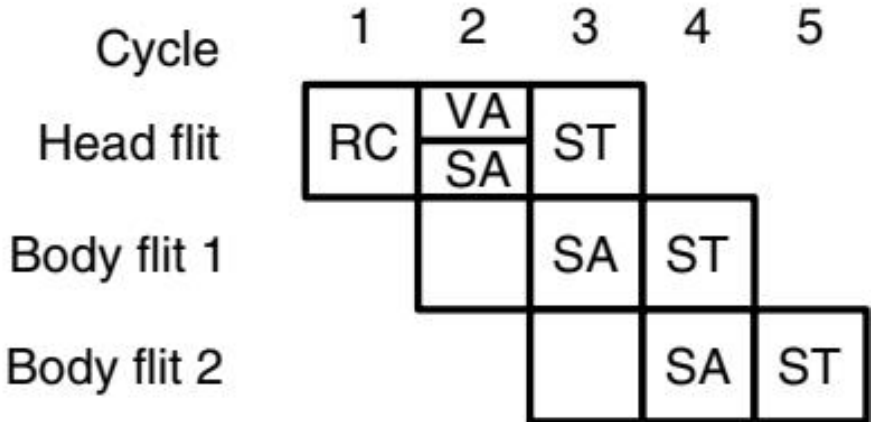


图 16.8 虚拟通道分配与交换分配并行进行的推测式路由器流水线

为了保证推测不会损害性能，我们可以保守地推测。我们通过在开关分配中给予非推测性请求优先权来做到这一点。这保证了本可用于非推测性请求的开关周期永远不会因为被授予了后来未能分配虚拟通道的推测性请求而空闲。

在实践中，推测工作得非常好，在流水线延迟重要的情况下几乎总是成功。当路由器负载较轻时，流水线延迟很重要。在这种情况下，几乎所有虚拟通道都是空闲的，推测几乎总是成功。当路由器负载较重时，排队延迟主导流水线延迟，因此推测的成功与否对性能影响不大。

我们可以进一步推进推测，将交换遍历与两个分配步骤并行地推测执行，如图 16.9 所示。在这种情况下，我们将 flit 转发到请求的输出，而不等待交换分配。为此，我们要求交换机具有足够的加速比，或者至少是输入加速比，以便它能接受来自所有输入的推测请求。（见第 17.2 节。）当推测的 flit 穿越交换机时，交换分配并行进行。6 分配在 flit 到达交换机另一侧的同时完成，分配结果选择获胜的 flit。这种双重推测能否在不影响周期时间的情况下完成，取决于交换机和分配器的细节。然而，在许多路由

器中，虚拟通道分配器是关键路径，交换分配和交换输出选择可以串行执行而不会延长时钟周期。

我们已经将流水线缩短到两个周期。能否将其缩短到单个周期？我们可以尝试第三次应用推测，将路由计算与其他三个功能并行执行。然而，这将非常低效，因为在分配通道、分配交换周期或将 flit 转发通过交换机之前，我们确实需要知道 flit 的目的地。

16.5 推测与前瞻

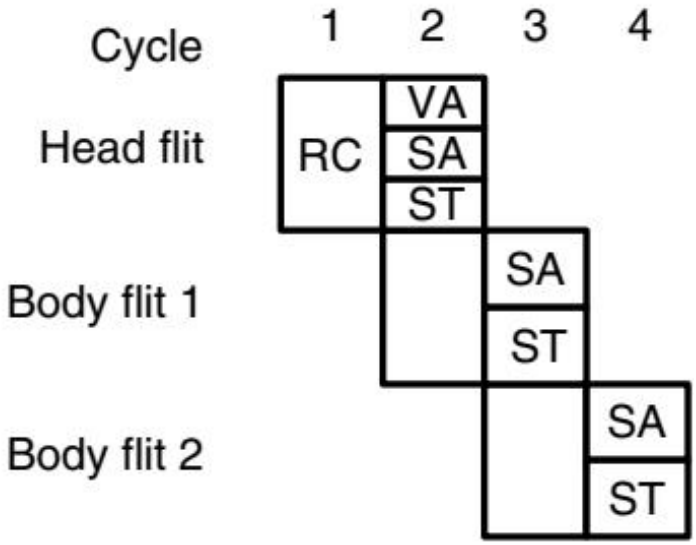


图 16.9 推测式路由器流水线，VA、SA 和 ST 全部并行执行。

与其对路由阶段的结果进行推测，我们可以提前执行这项工作，以便在需要时结果可用。为此，当我们在穿越给定路由器时——比如路由的第 i 跳——我们不是为当前路由器执行路由计算，而是为下一个路由器，即路由的第 $i+1$ 跳的路由器执行路由计算，并将计算结果随头 flit 一起传递。因此，当数据包到达每个路由器时，它已经选择了离开该路由器所使用的通道（或通道集），VA 和 SA 操作可以立即使用此信息开始。在与路由器任何流水线阶段并行的同时，执行下一跳的路由计算，记为 NRC，因此当 flit 到达时，该路由器也将已选择其输出通道。

图 16.10 显示了采用这种一跳路由前瞻的两种流水线。左侧，路由前瞻应用于我们的非推测四阶段流水线，将其缩短为三个阶段。这里，我们将 NRC（下一跳路由计算）放入 VA 阶段。我们本可以轻松地将其放入 SA 阶段，甚至让它跨越两个阶段而不损失性能。右侧，路由前瞻应用于图 16.9 的双重推测流水线，将其缩短为单个周期。7 对于延迟至关重要的应用，我们无法做得比这更好了。

16.6 Flit 与信用编码

到目前为止，我们讨论了 flit 和信用在路由器之间的通道中传输，但没有描述如何完成的细节。我们如何区分 flit 和信用？我们如何知道每个 flit 从哪里开始？flit 或信用内的信息如何编码？在本节中，我们将简要研究这些问题。

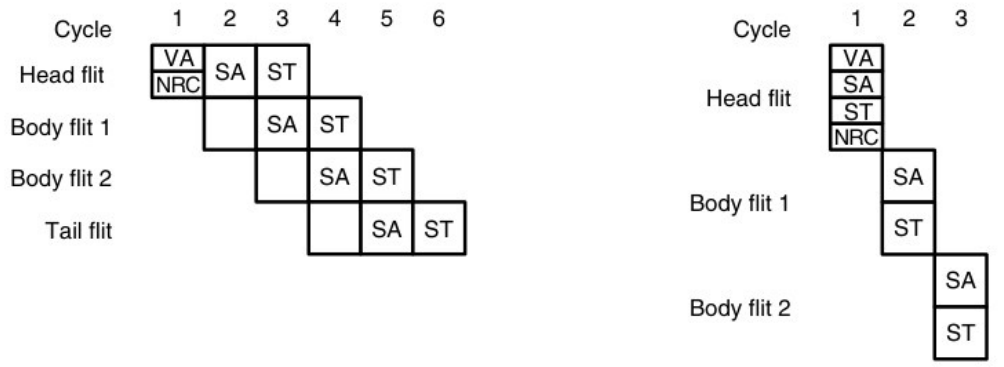
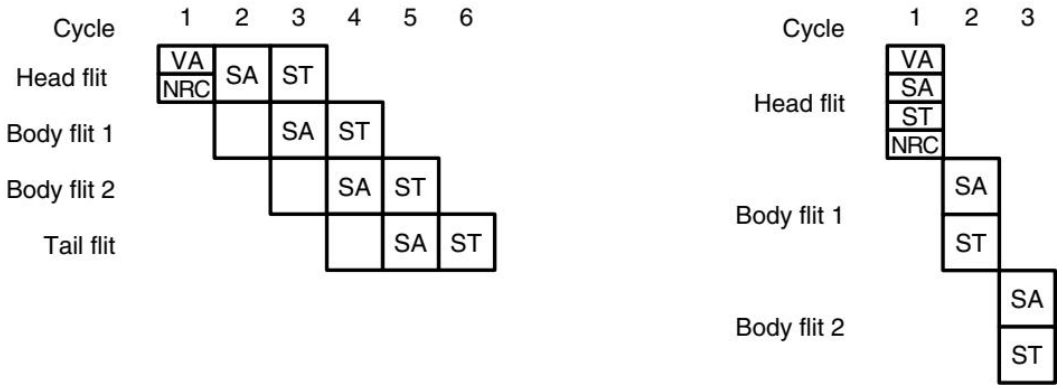


图 16.10 预先路由计算：提前一跳计算路由以重叠流水线阶段



首先，我们解决区分 flit 和信用的问题。最简单的方法是通过完全独立的通道发送 flit 和信用，如图 16.11（a）所示。flit 通道通常比信用通道宽得多（如较粗的线所示），因为它携带的信息多得多。虽然简单，但拥有单独的信用通道通常效率很低。信用通道上经常有浪费的过剩带宽，并且将信用串行化以通过窄通道传输会增加信用延迟。通常更有效的是在单个物理通道上携带 flit 和信用，如图 16.11（b）所示。一种方法是在每个 flit（用于前向通道）中包含一个信用（用于反向通道）。我们称之为将信用搭载在 flit 上，就像 flit 让信用搭便车一样。这种方法简单，并利用了这样一个事实：尽管大小不同，flit 和信用具有相同的速率——平均每个 flit 时间需要一个信用。或者，flit 和信用可以在线上以 phit 级别进行多路复用。这样做的好处是，在 flit 和信用速率不同的瞬态期间，所有带宽可以专用于 flit 或所有带宽可以专用于信用，而不是固定分配。然而，如果信用不能恰好放入整数个 phit 中，这种方法可能会有一些碎片损失。为了在 phit 级别多路复用 flit 和信用，我们需要解决如何知道每个 flit 和信用从哪里开始以及如何知道它是 flit 还是信用的问题。这通过 phit 级协议完成，该协议可能使用如图 16.12 所示的编码。这里，每个 phit 指示它是开始一个信用、开始一个 flit 还是空闲。因为我们知道信用和 flit 的长度，我们不需要在类型字段中编码 flit 中间；它可以被推断出来，并且我们可以将延续 phit 的 phit 类型字段用于额外的有效载荷信息。如果信用搭载在 flit 中，我们可以省去信用开始编码，并将 phit 类型字段减少到单个比特。其他链路级协议，例如基于成帧的协议，可以完全消除此字段，但会牺牲一些灵活性。

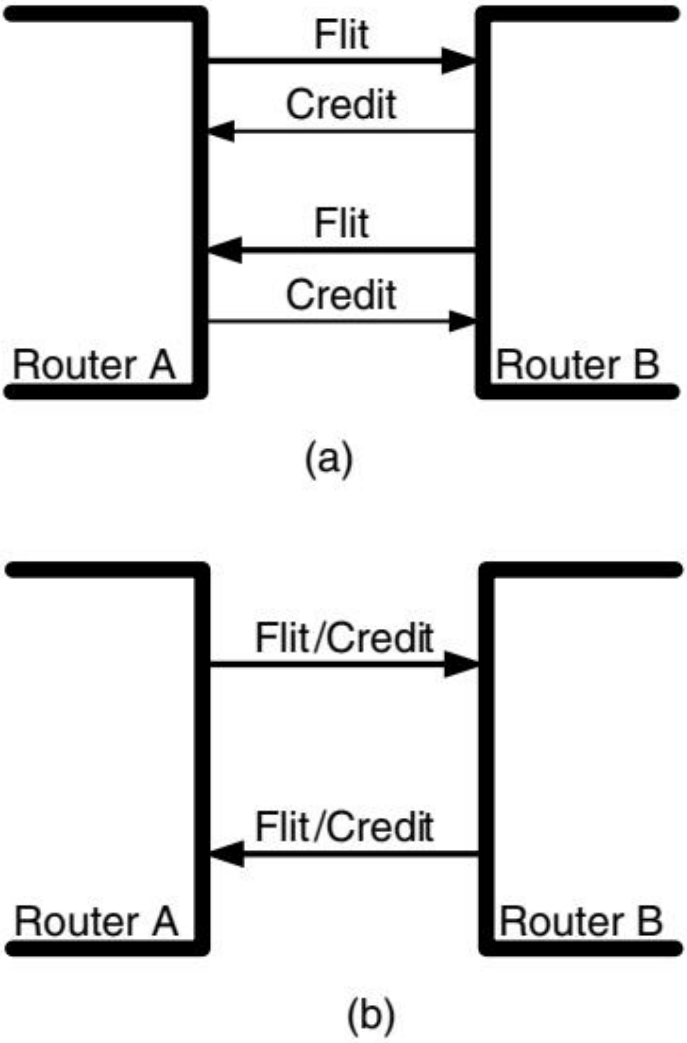


图 16.11 Flit 和信用可以（a）通过单独的物理通道发送，或（b）共享一个通道。如果它们共享一个通道，信用可以搭载在每个 flit 内，或者可以使用 phit 级协议与 flit 多路复用。

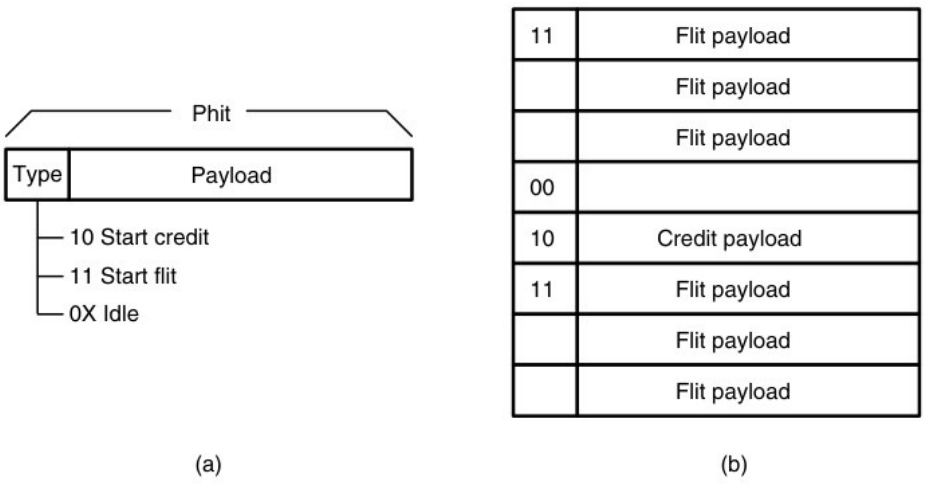


图 16.12 phit 的类型字段、有效载荷字段及 flit 与 credit 复用示例

既然我们知道了每个 flit 和 credit 的起始位置，接下来就来看看它们的内容。如图 16.13 所示，所有 flit 都包含一个虚拟通道标识符，用于指示该 flit 关联的是哪个虚拟通道；一个类型字段，用于标识 flit 的类型是头 flit 还是非头 flit（我们也可以编码是否为尾 flit）；以及一个校验字段，用于在载荷之外检查 flit 中的错误。头 flit 还会增加一个路由信息字段，供路由函数为数据包选择输出虚拟通道。如果 credit 搭载在 flit 上，则头 flit 和非头 flit 都会增加一个 credit 字段。无论是搭载还是单独发送，credit 都包含一个虚拟通道标识符，用于指示接收该 credit 的虚拟通道，以及一个校验字段。credit 还可以可选地包含一个类型字段，例如用于编码该虚拟通道的开关流控信息或其他反向通道信息。

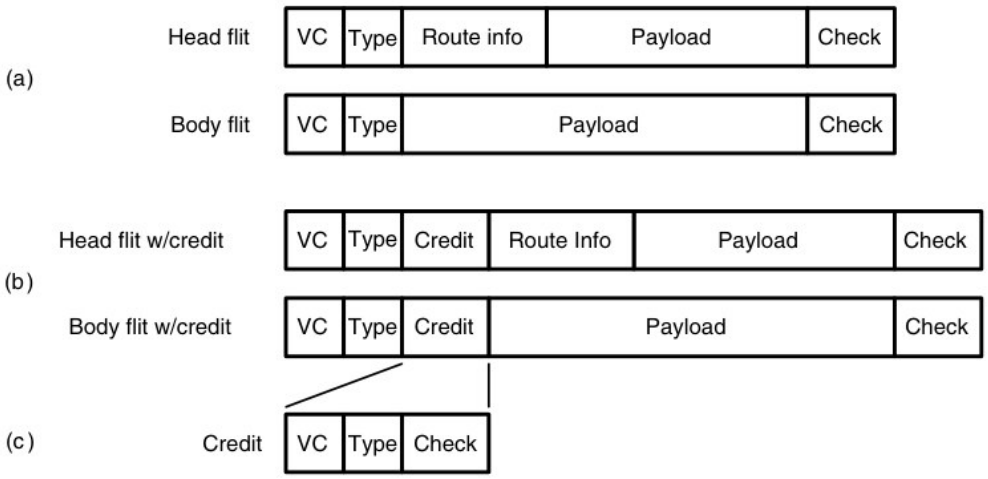


图 16.13 首部 flit 与主体 flit 的格式及虚拟通道标识字段

粗略浏览图 16.12 和 16.13 可能会让人觉得我们在做重复的工作。毕竟，phit 编码已经标识了虚拟通道（credit 或 flit）和类型（起始或空闲）。那么为什么还需要在 flit 级别重复这些信息呢？答案是开销。例如，考虑一个具有 16 位 phit、16 字节 flit、16 个虚拟通道和搭载 credit 的多计算机路由器。如果我们在每个 16 位 phit 中放入 4 位虚拟通道字段和 2 位类型字段，开销将达到 37.5%。再加上 8 位校验字段，就只剩下 2 位用于信息。而如果在 phit 上使用 1 位类型字段，并在 flit 上总共使用 14 位用于虚拟通道、类型和校验位，总开销仅为 17%。

16.7 案例研究：Alpha 21364 路由器

Alpha 21364 是 Alpha 微处理器架构系列中的最新成员 [89，131]。这款拥有 1.52 亿个晶体管、主频 1.2 GHz 的 21364 将 Alpha 21264 处理器核心与缓存一致性硬件、两个内存控制器和一个网络路由器集成在单个芯片上。这使得 21364 可以作为大型共享内存系统的单芯片构建模块。该路由器具有四个外部端口，总网络带宽为 22.4 GB/s，最多可将 128 个处理器连接成二维环面网络。21364 路由器的架构与本章介绍的基本路由器架构高度吻合：它采用数据包在路由器中的深度流水线来满足 1.2 GHz 的工作频

率，并利用推测来弥补部分流水线带来的延迟。

图 16.14 展示了 21364 路由器架构的高层视图。该路由器采用直通流控，而非虫洞流控。尽管这一设计决策要求每个输入单元至少能够存储一个完整的数据包，但这对于 21364 来说是可行的，因为其一致性协议中使用的最大数据包长度为 76 字节。每个输入单元都包含足够的缓冲来存放几个最大长度的数据包，路由器总共有 316 个数据包缓冲区。这样的数据包存储量足以让路由器拥有多个虚拟通道，这些虚拟通道既用于避免死锁，也用于实现自适应路由。

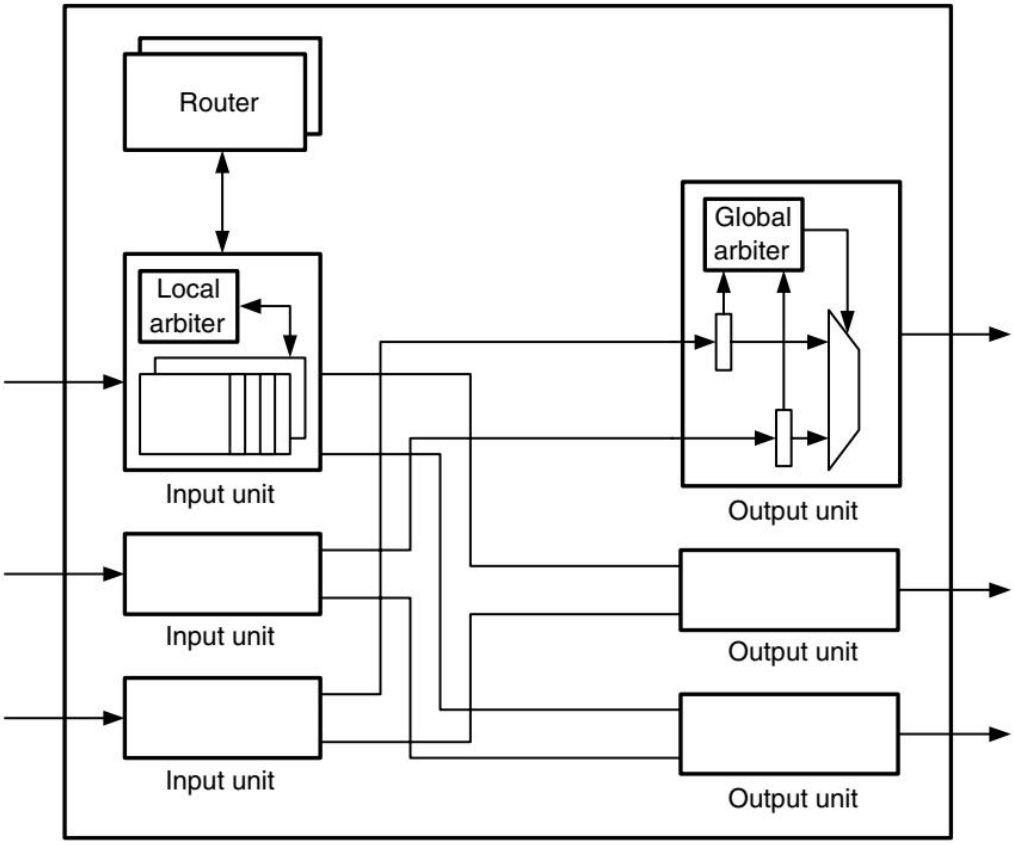


图 16.14 Alpha 21364 路由器架构的简化视图。尽管实际实现有 8 个输入端口和 7 个输出端口，为简单起见，这里仅显示一个 3×3 路由器。21364 的交叉开关为每个输出端口实现为一个多路选择器，如图所示。此外，交换机分配器分为两个不同的单元：本地仲裁器和全局仲裁器。

如路由器流水线图（图 16.15）所示，路由和缓冲占据了数据包第一个 phit 的前四个流水线周期——我们考虑的是 phit 流水线，因为这是一个直通路由器。路由发生在 RC 阶段，随后是一个传输/线延迟（T）周期。头部解码和输入单元状态更新占用另一个周期（DW），之后数据包的载荷最终被写入输入队列（WrQ）。在第一个 phit 存入输入缓冲的同时，交换机分配可以开始。

由于 Alpha 21364 采用直通流控，通常分开的虚拟通道分配和交换机分配步骤可以合并为一次分配。虽然在通用的虚拟通道路由器中，每个输出可以有多个活动虚拟通道，但直通路由器可以通过只允许每个输出有一个活动虚拟通道来简化设计。这确保了任何虚拟通道分配同时也是有效的交换机分配，对于 21364 的设计，我们将这个合并的 VA 和 SA 步骤简称为 SA。一旦使用某个虚拟通道的数据包完全传输到下游，相应的

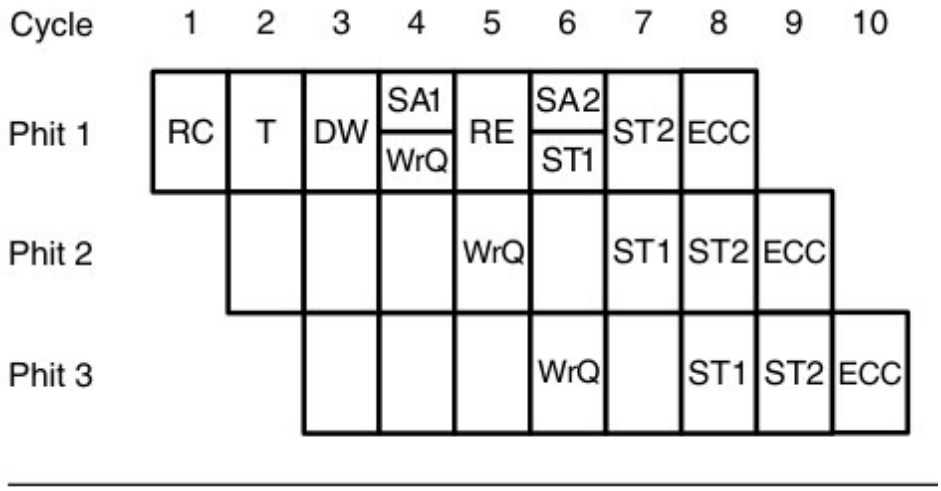


图 16.15 Alpha 21364 路由器的流水线阶段：全流水线延迟为 13 个周期；在 1.2 GHz 下约为 10.8 ns，包括同步、驱动器、接收器及引脚间传播所需周期。直通流控中 flit 与分组等长，因此图示的是 phit 级流水线。

输出就可用于其他等待的数据包。当然，这种方法在虫洞流控下可能会违背虚拟通道的初衷，因为每个输出上活动的单个虚拟通道可能会被阻塞。然而，直通确保下游路由器始终有足够的缓冲来存储整个数据包，从而避免了数据包在占用输出的同时被阻塞的可能性。

为满足严苛的时序要求，21364 的交换机分配被分为两个阶段，分别称为本地仲裁器和全局仲裁器。（在 19.3 节中，我们将更详细地讨论这些可分离的分配器。）首先，每个输入单元中的本地仲裁器在所有已就绪的等待数据包中选择一个数据包。当一个数据包的输出端口可用且其下游路由器有空闲的数据包缓冲区时，该数据包被视为就绪。本地仲裁占用一个流水线周期（SA1），在下一个周期中，由本地仲裁器选出的每个数据包的头部信息被读取并传输到正确的输出（RE）。这些头部信息足以启动全局仲裁周期（SA2）。由于本地仲裁决策并不协调，很可能多个数据包被选中发往同一个输出。全局仲裁器通过为每个输出选择一个数据包来解决这一冲突。与全局仲裁决策并行，由本地仲裁器选出的每个数据包所对应的数据被推测性地从输入缓冲区读出并传输到输出（ST1）。这种传输是推测性的，因为这些数据包仍可能在全局仲裁中失败，但如果它们胜出，读取数据并将其传输到输出所需的周期就被隐藏了。一旦全局仲裁结果已知，交换机传输在阶段 ST2 完成。最后一个阶段（ECC）在将每个 phit 传输到下游路由器之前，为其附加一个 7 位纠错码。尽管图 16.15 中未显示，路由器内部的反向通道将推测性交换机传输的结果传回输入单元。如果推测成功，该数据包的后续 phit 可以继续发送。如果失败，推测性的 phit 在交换机内部被丢弃，必须由输入单元重新发送。21364 深度流水线设计和推测性交换机传输的结果是极低的每跳延迟（10.8 纳秒），同时保持峰值吞吐量的 70% 至 90%[131]。

16.8 文献注释

本章介绍的基本虚拟通道路由器架构可追溯到 Torus 路由芯片 [56]。多年来的改进可见于 Reliable Router[52]、Cray T3D[95] 和 T3E[162] 中的路由器、IBM SP 系列 [176] 以及 SGI SPIDER 芯片 [69]。SPIDER 芯片是第一个提前一步计算路由的路由器。使用推测来降低路由器延迟的方法最早由 Peh 和 Dally 描述 [146]。

16.9 练习

16.1 单虚拟通道的简化。如果每个物理通道只有一个虚拟通道，流水线和全局状态会如何变化？描述每个流水线阶段和全局状态的每个字段。
16.2 电路交换路由器的架构。画出电路交换路由器的架构草图（如图 12.6 所示）。全局状态是什么？流水线是什么？处理每个电路和每个 flit 的步骤是什么？
16.3 确认/否认流控对流水线的影响。假设使用确认/否认流控而不是基于信用的流控。这会如何改变路由器流水线？信用停顿会发生什么变化？
16.4 激进虚拟通道重分配带来的依赖关系。考虑基于 Duato 协议的二维网格网络简单最小自适应路由算法：VC 0 保留给维序路由（路由子功能），VC 1 用于自适应路由。图 16.7 中所示的哪些重分配策略对于该路由算法是无死锁的？
16.5 窄通道的 flit 格式。对于图 16.13 所示的 flit 字段，设 VC 字段需要 4 位，类型信息需要 2 位，信用字段需要 8 位，路由信息需要 6 位。如果输入通道每个时钟周期接收 8 位，那么路由开始前需要多少个周期？如果是，给出相应的 flit 格式。如果采用路由前瞻，虚拟通道分配能否在第一个周期开始？

注释

- 1. 在某些情况下，当路由简单或带宽需求较低时，路由器可以复制，每个输入单元拥有自己的路由器。这种复制对于两个分配器来说是不可能的，因为它们分配的是共享资源。
- 2. 当交换机的输出加速比为 1（交换机带宽等于输出带宽）时，交换机和输出通道可以一起调度而不会损失性能。当交换机具有输出加速比时，输出单元通常包含一个 FIFO，以将交换机调度与输出调度解耦。
- 3. 情况并非总是如此。在一些路由器中，这些步骤中的两个或更多可以合并为一个流水线阶段，而在其他路由器中，单个步骤本身可以在多个阶段中进行流水线处理。
- 4. 尽管我们通常以秒或有时以周期来表示这些延迟，但为方便起见，这里我们假设它以微片时间（每个可能占用多个周期）为单位表示。

- 5. 如果 RC 级产生的可能路由集跨越多个物理通道，我们可以通过选择一个来再次推测。
- 图 16.10 路由前瞻。通过提前一跳执行路由计算，我们可以将其与任何其他流水线阶段重叠。左侧显示应用于我们的标准四阶段流水线，右侧显示应用于图 16.9 的流水线，将其缩短为单个阶段。
- 7. 这并不一定意味着每跳一个周期的延迟。虽然通过路由器只有一个周期，但许多系统将需要额外的一个周期（或更多）来穿越通道。

第十七章　路由器数据通路组件

在本章中，我们探讨构成路由器数据通路的组件设计：输入缓冲区、交换机和输出单元。输入缓冲区在 flit 等待虚拟通道、交换机带宽和通道带宽时暂存它们。输入缓冲区可以是集中式的、按物理通道分区的，或按虚拟通道分区的。集中式缓冲区在跨物理通道和虚拟通道分配存储时提供最大的灵活性，但由于带宽不足往往不切实际。在每个缓冲区分区内，存储可以使用环形缓冲区静态分配给每个虚拟通道缓冲区，或使用链表动态分配。

交换机是路由器的核心，也是 flit 从输入端口到输出端口实际交换发生的地方。集中式缓冲区本身可作为交换机，由缓冲区的输入多路复用器和输出解复用器执行该功能。共享总线也可以作为交换机，其优点是不需要复杂的交换机分配——每个输入在获得总线时向所有输出广播。然而，总线对内部带宽的利用效率低下。大多数高性能路由器使用交叉点交换机。交叉点交换机通常提供一定的额外带宽（加速），以简化分配问题，并防止交换机分配成为性能瓶颈，从而使昂贵的通道带宽闲置。

17.1　输入缓冲区组织

输入缓冲区是现代路由器中最重要的结构之一。流控协议在这些缓冲区中分配空间，以容纳等待通道带宽离开这些路由器的 flit。缓冲区为到达的 flit 提供空间，这样当数据包因流水线停顿或虚拟通道或物理通道争用而暂时延迟时，输入通道无需减速。

17.1.1　缓冲区划分

输入缓冲区的划分方式与交换机的设计紧密相关。输入缓冲区可以在整个路由器范围内合并，也可以按物理输入通道划分，或者按虚拟通道划分，如图 17.1 所示。如果使用单个存储器来保存路由器上的所有输入缓冲区，如图 17.1（a）所示，则不需要交换机。然而，在这种情况下并没有真正节省资源。相反，实际上存在两个交换机：一个将输入多路复用到存储器中，另一个将输出从存储器中分发出去。1 中央存储器组织的主要优点是能够在输入端口之间动态分配内存的灵活性。不幸的是，这种方法有两个严重的缺点。首先，单个存储器的带宽可能成为高性能路由器中的严重瓶颈。例如，在一个具有 $\delta_i = 8$ 个输入端口的路由器中，中央存储器带宽必须至少是端口带宽的 $2\delta_i = 16$ 倍，以便在每个 flit 时间写入 8 个 flit 并读取 8 个 flit。要获得高带宽需要非常宽的存储器，通常有许多 flit 宽，这导致了第二个问题。路由器延迟增加，因为需要将 flit 反序列化以形成足够宽的字写入共享存储器，然后在输出端重新序列化 flit。通过为每个输入端口提供单独的存储器，可以避免中央缓冲存储器的带宽和延迟问题，2 如图 17.1（b）所示。虽然在这种安排下总存储器带宽仍然是端口带宽的 $2\delta_i$ 倍，

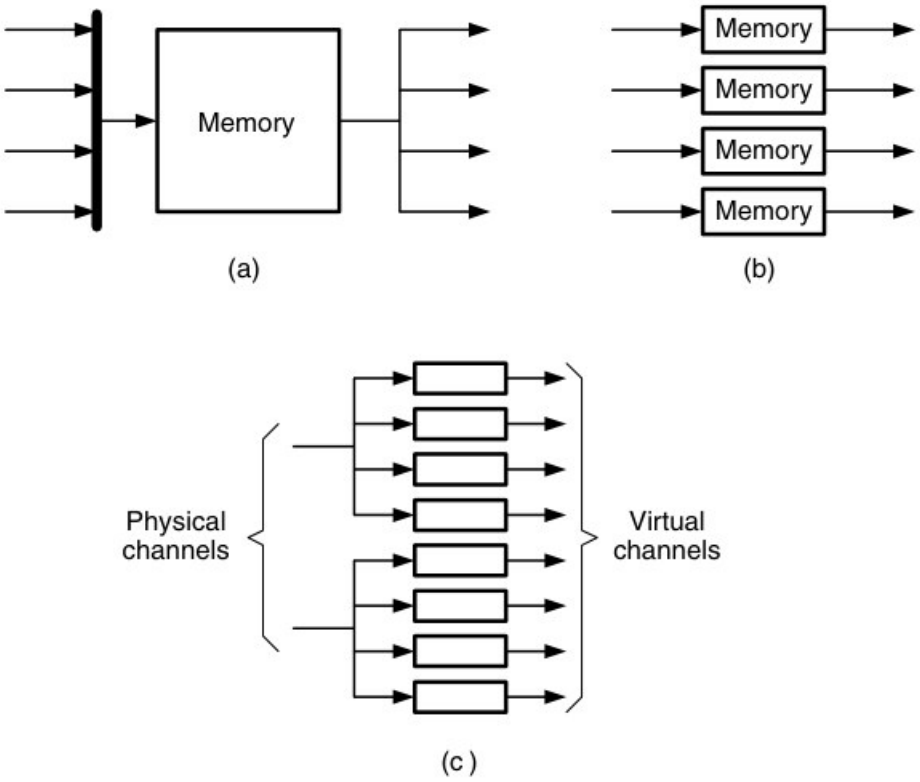
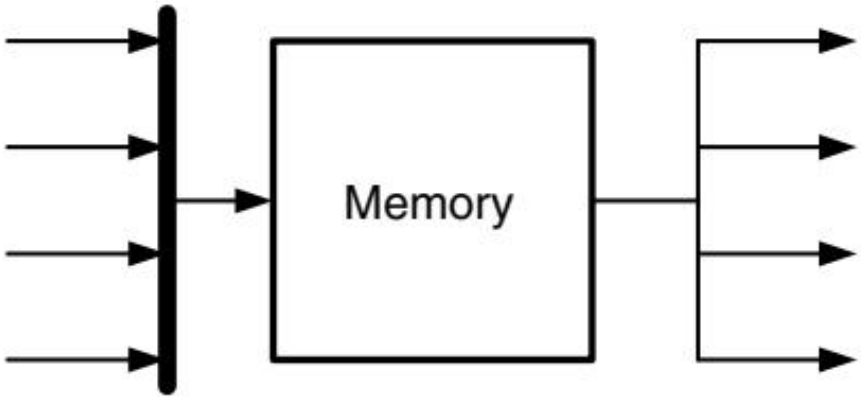


图 17.1 flit 缓冲组织：各输入端口共享中央存储器等方案

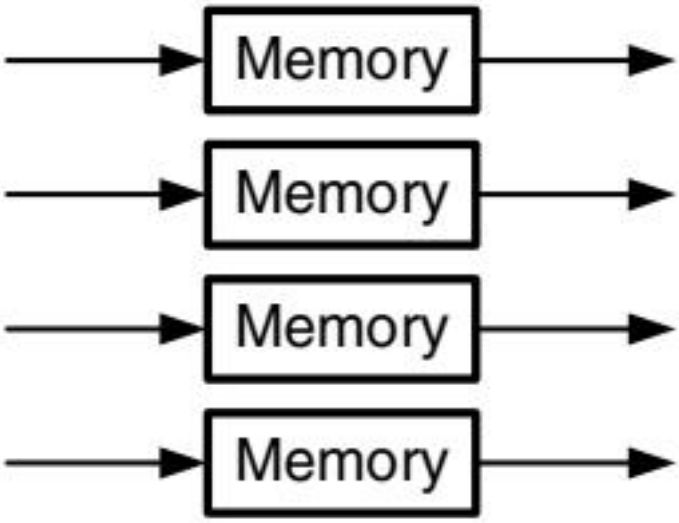
但每个单独的存储器只需要输入端口带宽的两倍，使得实现更加可行，并避免了宽存储器的延迟惩罚。这种安排使得缓冲区可以在与特定物理通道相关联的虚拟通道之间共享，但不能跨物理通道共享。

可以进一步细分 flit 缓冲区，为每个虚拟通道提供单独的存储器，如图 17.1（c）所示。这具有通过允许交换机在单个周期内从每个物理通道访问多个虚拟通道来实现交换机输入加速的优点。例如，假设一个路由器有 $\delta_i = 8$ 个输入端口，每个端口有 V=4 个虚拟通道。按虚拟通道组织的缓冲区将向 32×8 交换机提供 $\delta_i V = 32$ 个输入。如第 17.2 节将讨论的，以这种方式在交换机上提供输入加速会增加路由器的吞吐量。然而，如此精细地划分输入缓冲区可能导致较差的内存利用率，因为空闲虚拟通道的缓冲区内存空间无法分配给繁忙的虚拟通道。此外，与这么多小存储器相关的开销相当大。拥有几个较大的存储器要高效得多。

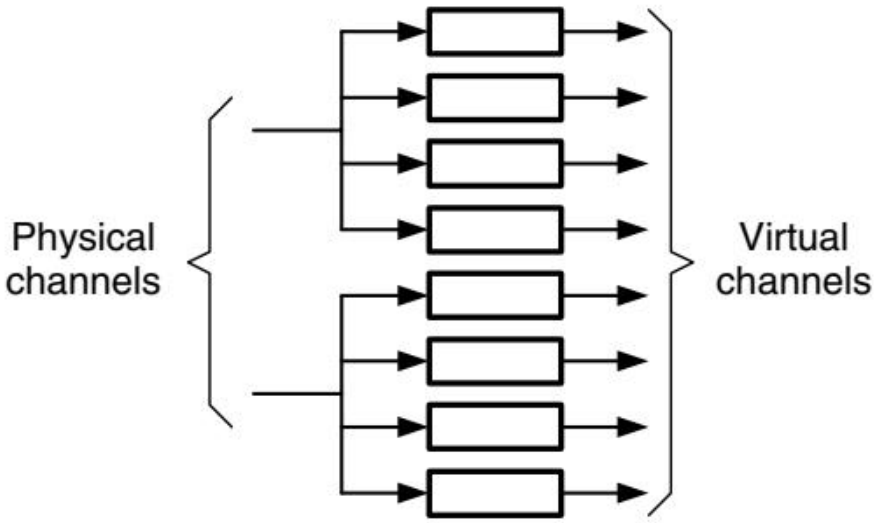
Flit 缓冲区组织。（a）中央存储器在路由器的所有输入端口之间共享。这允许在通道之间灵活分配缓冲区空间，并且存储器也充当交换机。然而，单个中央存储器的带宽可能成为瓶颈，这使得该组织不适合高性能路由器。（b）为每个输入物理通道提供单独的缓冲存储器。这种组织提供的带宽随端口数量扩展，并且需要具有与物理通道数量相等的输入端口数量的交换机。（c）为每个虚拟通道提供单独的缓冲存储器。这使得交换机能够在单个周期内从同一物理通道上的多个虚拟通道获取 flit。然而，它通常会导致显著的缓冲区碎片化，因为空闲虚拟通道的存储器无法被繁忙的虚拟通道共享。


Memory

(a)


Memory
Memory
Memory
Memory

(b)


Physical channels
Virtual channels

(c)

在如图 17.1（b）所示的按物理通道缓冲区和图 17.1（c）所示的按虚拟通道缓冲区之间，一个良好的折衷是在每个物理通道上提供一个具有少量输出端口的缓冲区，如图 17.2（a）所示。这种方法为我们提供了按物理通道缓冲区的缓冲区共享和规模经济优势，同时仍然允许交换机加速。通过使用分区的物理通道缓冲区而不是多端口物理通道缓冲区，可以避免图 17.2（a）中多端口存储器的开销，而性能损失非常小，如图 17.2（b）所示。采用这种方法，与一个物理通道相关联的 V 个虚拟通道被划分到少量 B < V 个按物理通道的缓冲区中。例如，在图中所示的 B = 2 的情况下，偶数虚拟通道将存储在一个缓冲区中，奇数虚拟通道存储在另一个缓冲区中。这种组织在缓冲区分配（存储不能跨分区共享）和交换机调度（交换机每个分区只能选择一个虚拟通道）方面牺牲了一些灵活性。然而，在实践中，分区缓冲区的性能几乎与多端口缓冲区相同，而成本（芯片面积）显著降低。

17.1 输入缓冲区组织

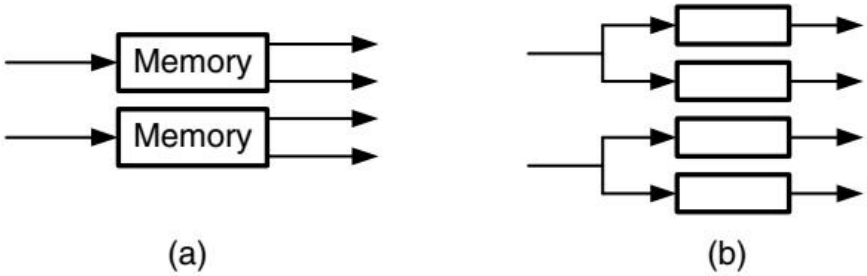


图 17.2 每个物理通道多个缓冲区端口。(a) 通过在每个物理通道缓冲区上提供多个输出端口，可以在不增加按虚拟通道缓冲区开销的情况下实现交换机输入加速。（b）一种成本更低且性能几乎相同的方法是将虚拟通道划分到多个单端口缓冲区中。例如，所有偶数虚拟通道存储在一个缓冲区中，所有奇数通道存储在另一个缓冲区中。

17.1.2 输入缓冲数据结构

在任何缓冲组织方式中，我们都需要维护一种数据结构，以跟踪微片和包在存储器中的位置，并管理空闲存储器。此外，在多个虚拟通道缓冲共享单一存储器的缓冲组织方式中，我们还需要一种机制来在虚拟通道之间公平地分配空间。

表示微片缓冲的两种常用数据结构是：环形缓冲和链表。环形缓冲管理更简单，但需要固定的缓冲空间划分。链表的开销更大，但允许在不同缓冲之间自由分配空间。

在对这两种缓冲的描述中，我们用术语缓冲或微片缓冲来指代用于保存与一个虚拟通道相关联的所有微片的整个结构，例如，用术语单元或微片单元来指代用于保存单个微片的存储。

环形缓冲如图 17.3 所示。固定的 FIRST 和 LAST 指针标记了存储器内缓冲的边界。这些指针可以通过放入寄存器来配置。然而，一旦缓冲被使用，它们就不能被改变。缓冲的内容位于 Head 指针处及其下方，以及 Tail 指针上方，可能环绕到缓冲的末尾，如图 17.3（b）所示。

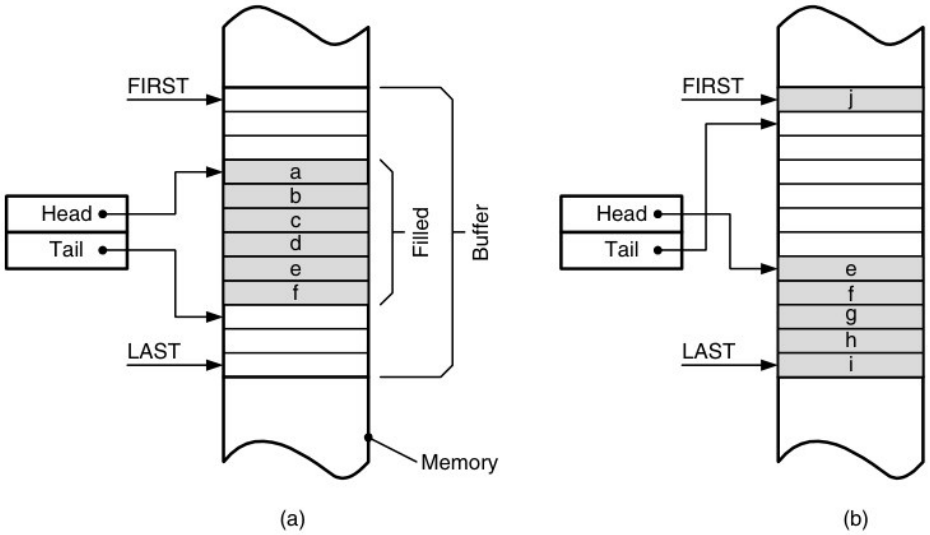
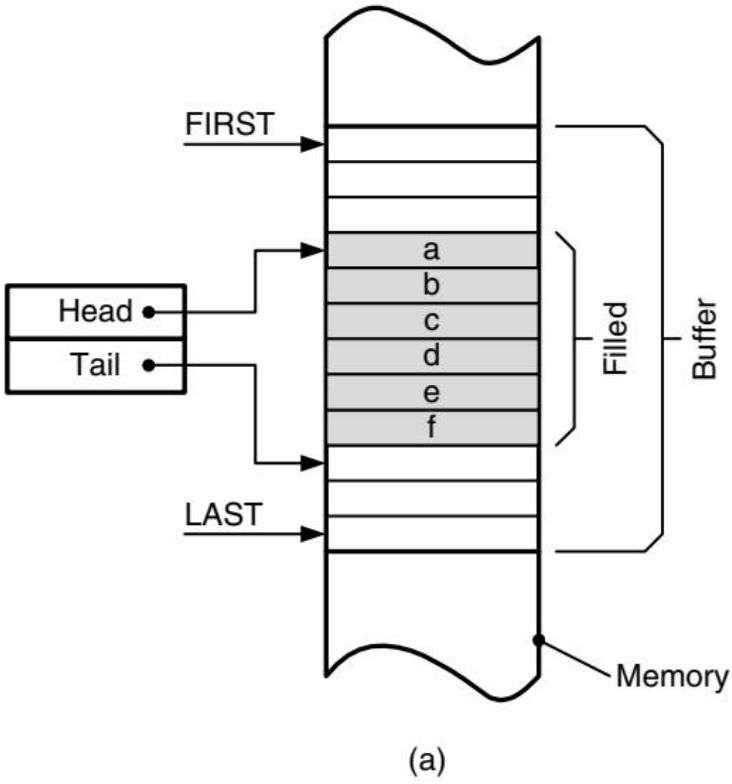


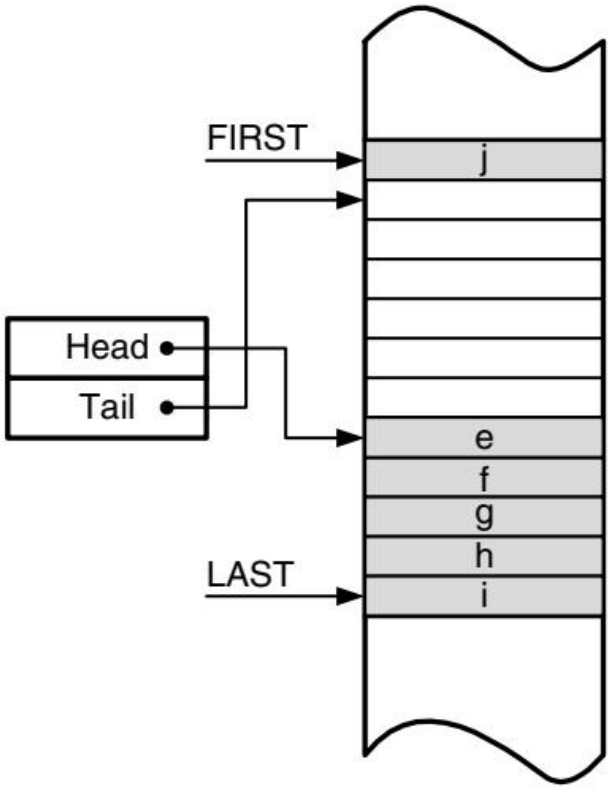
图 17.3 环形缓冲区中 flit 的入队、出队与回绕示例

通过将元素存储在 tail 指针所指示的单元中，并以环形方式递增 tail 指针，将元素添加到缓冲中。在“C”代码中，此操作表示为：




```
mem[Tail] = new_flit;
Tail = Tail + 1;
if(Tail > LAST) Tail = FIRST;
if(Tail == Head) Full = 1;
```

此代码的第三行通过当 tail 指针到达缓冲末尾时将其环绕回缓冲起始位置，使缓冲成



(b)

为环形。

类似地，通过读取 head 指针处的值并以环形方式递增 head 指针，从缓冲中移除元素：

```
flit = mem[Head];
Head = Head + 1;
if (Head > LAST) Head = FIRST;
if (Head == Tail) Empty = 1;
```

两个代码序列的最后一行分别检测缓冲何时为满和空。这些情况发生在 head 和 tail 指针相等时。区别在于，当插入操作导致指针相等时发生满条件，而当移除操作导致指针相等时发生空条件。满和空指示由微片缓冲外部的逻辑使用，以避免在缓冲满时发出插入请求，以及在缓冲空时发出移除请求。

在典型实现中，环形缓冲数据结构在逻辑中实现，该逻辑在单个时钟周期内执行上述代码序列的所有四行。事实上，许多实现的结构使得缓冲可以在单个时钟周期内同时插入一个元素和移除一个元素。

如果缓冲的大小是 2 的幂，并且对齐到其大小的倍数（例如，16 微片缓冲从 16 的倍数开始），那么指针的环绕可以通过仅阻止从指示缓冲内位置的位到指示存储器内缓冲的位的进位来执行，而无需比较。例如，如果微片缓冲的大小为 16 个单元，则指针的第 0 到 3 位选择缓冲内的单元，第 4 位及以上标识缓冲。因此，如果我们阻止从第 3 位到第 4 位的进位，则从缓冲的最后位置递增指针（缓冲 B 的 BF）将导致指针环绕到缓冲 B0 的第一个位置。

链表数据结构如图 17.4 所示，在将空闲存储分配给缓冲方面提供了更大的灵活性，但会带来额外的开销和复杂性。为了管理具有链表的存储器，每个单元都增加一个指针字段，以指示列表中的下一个单元。为了便于管理列表所需的多个指针操作，指针通常保存在与微片单元本身不同的存储器中，并且该指针存储器通常是多端口的。对于存储在存储器中的每个缓冲，一个 head 和 tail 指针指示缓冲内容链表的第一个和最后一个单元。所有不属于某个缓冲的单元都链接在一个空闲列表上。最初，存储器中的所有单元都在空闲列表上，所有微片缓冲的 head 和 tail 指针均为 NULL。为了进行存储器分配，我们还为每个缓冲保留一个计数寄存器，指示该缓冲中的单元数，以及一个保存空闲列表上单元数的计数寄存器。

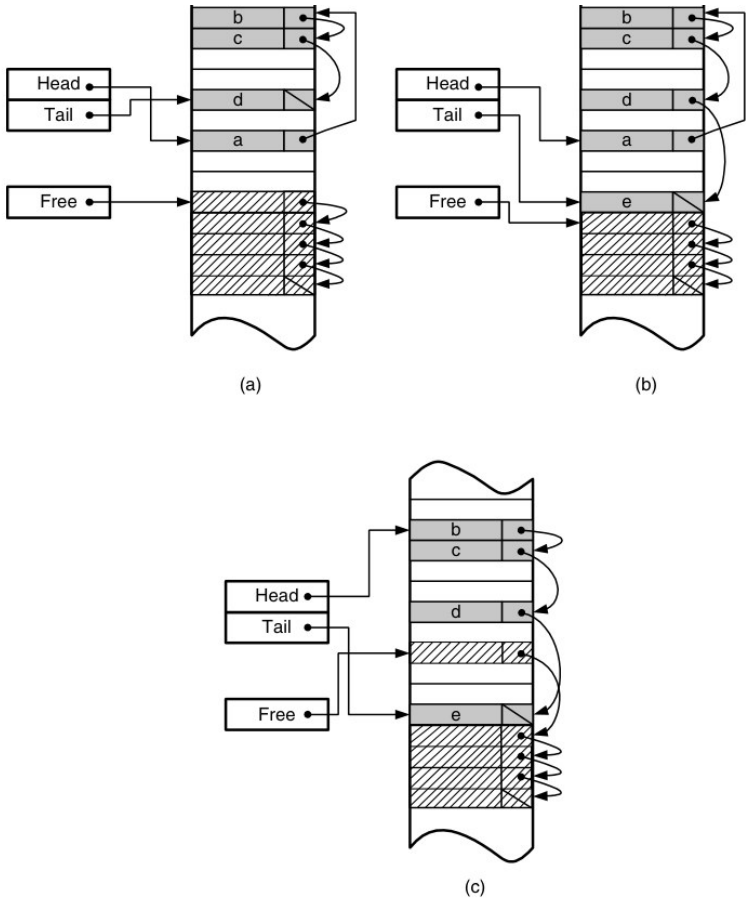


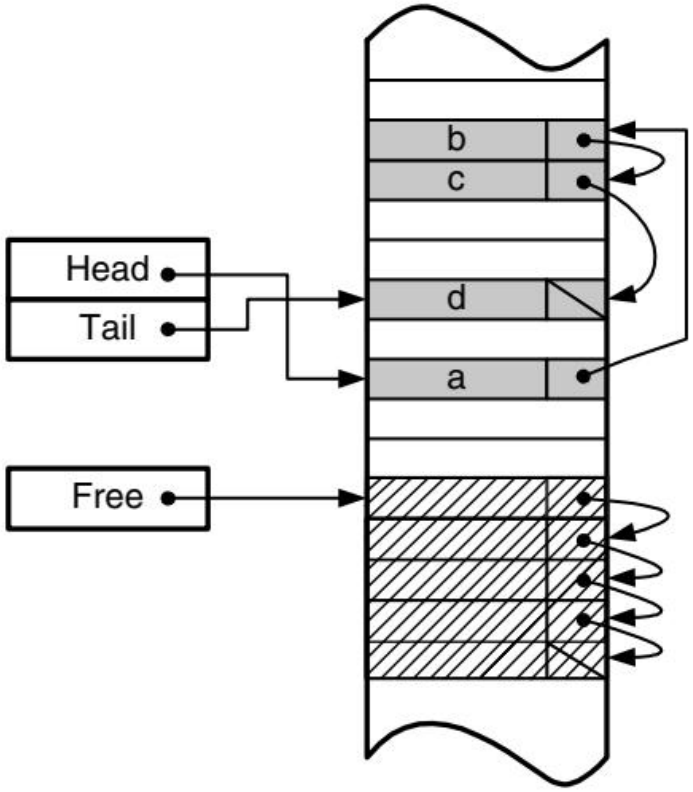
图 17.4 采用空闲链表管理 flit 缓冲单元的链表结构

将微片插入缓冲需要以下操作：

```
if(Free != NULL) {
mem[Free] = new_flit ; // store flit into head of free list
if(Tail != NULL)
    ptr[Tail] = Free ;    // link onto buffer list - if
Tail = Free ;    // it exists
Free = ptr[Free] ;    // unlink free cell
ptr[Tail] = NULL ;    // optional - clear link to free list
if(Head == NULL)    // if buffer was empty
    Head = Tail ;    // head and tail both point at this cell
```

```
} else ERROR ;    // no cells left on free list
```

像这样将微片插入链表可以在单个周期内完成，但需要一个双端口微片缓冲。我们必须写入旧 tail 单元的指针，并读取空闲列表头部（新 tail 单元）的指针。然而，这两个操作之间没有依赖关系，因此如果我们有足够的端口，它们可以同时进行。如果我们想清除新 tail 单元的指针，使其不再指向空闲列表（这对于一致性检查是个好主意——见下文——但并非绝对必要），我们需要在至少一个端口上进行读-修改-写操作。

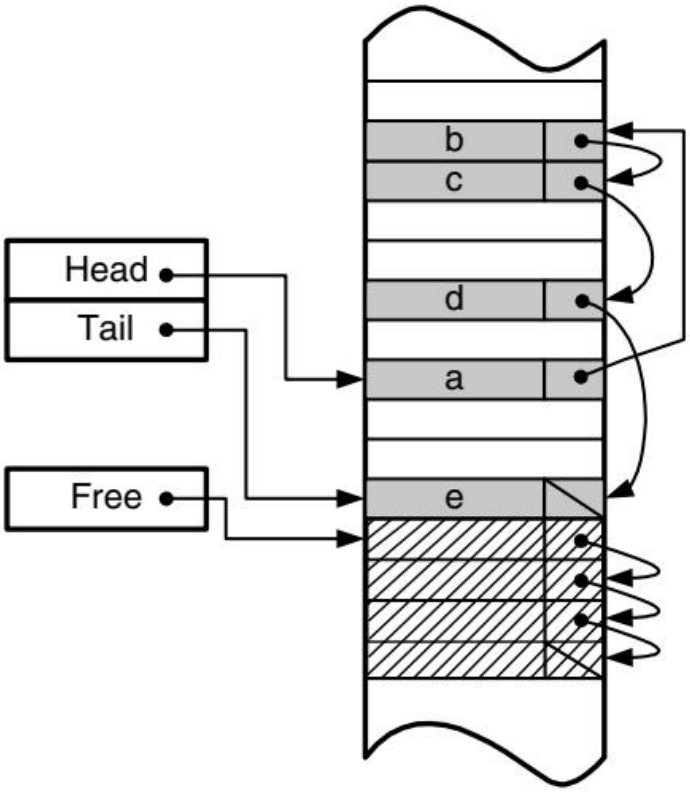


(a)

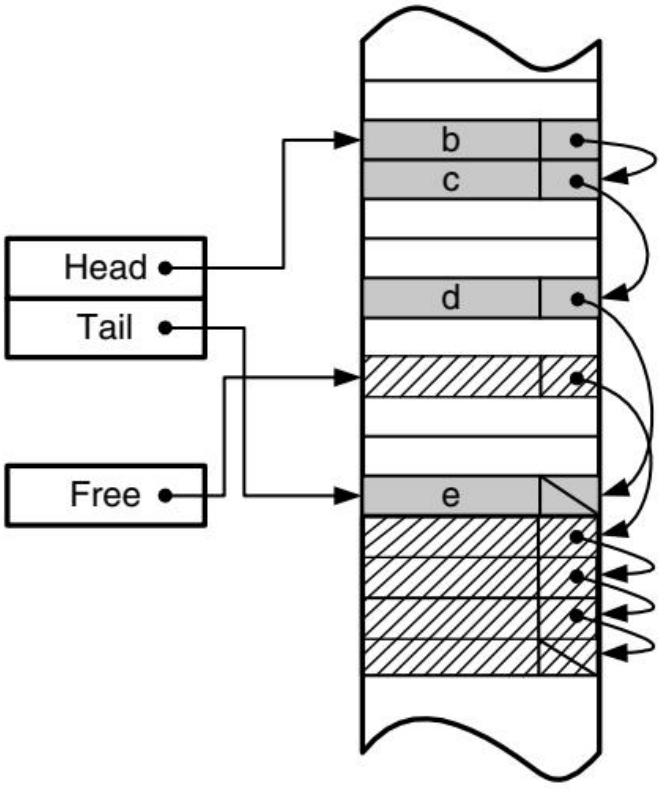
链表数据结构将每个缓冲维护为单元的链表。空闲单元链接在一起形成一个空闲列表。（a）一个微片缓冲包含链表中的四个单元。Head 指向列表中的第一个单元，Tail 指向列表中的最后一个单元。Free 指向五个空闲单元的链表。（b）通过从空闲列表中移除一个单元、填充它并将其链接到缓冲列表中，将微片 e 添加到列表中。（c）从列表中移除微片 a，并将其单元返回到空闲列表。

从缓冲中移除微片的过程如下：

```
if (Head != NULL) {
    flit = mem[Head] ; // get the flit
    Next = ptr[Head] ; // save pointer to next flit in buffer
    ptr[Head] = Free ; // put old head cell on the free list
    Free = Head ;    //
    if (Head == Tail) { // if this was the last cell - empty list
    Head = NULL ;
    Tail = NULL ;
```



(b)



(c)

```
} else {    // otherwise, follow the link to the
    Head = Next ;    // next cell
}
else ERROR ;    // attempt to remove from empty buffer
```

如这段代码所示，移除微片要容易得多。它可以在单个周期内完成，使用支持读-修改-写操作的单端口指针存储器，以读取 head 单元中的链接并将其链接到空闲列表中。使用链表结构的主要代价在于指针内存操作的复杂性：每次插入或删除都需要多次指针内存操作，而且错误处理的复杂度也随之增加。指针本身的实际存储开销很小。例如，将一个 8 位指针与一个典型的 8 字节 flit 关联起来，开销仅为 12.5%。

处理链表数据结构中出现的位错误，比处理循环缓冲区中的此类错误要复杂得多。在循环缓冲区中，内存中的位错误只影响缓冲区的内容，而不影响其结构。循环缓冲区头指针或尾指针中的位错误，最多只会导致从缓冲区中插入或删除有限数量的 flit。对于循环缓冲区，通过重置缓冲区的状态（设置 Head = Tail = FIRST），总是可以将错误限制在单个缓冲区内。

对于链表缓冲区，flit 指针字段中的单个位错误可能造成严重得多的后果。空闲链表中的指针发生位错误，会断开许多空闲存储 flit 的连接，导致分配失败，而缓冲区计数器却显示仍有许多空闲单元可用。如果该错误使指针指向一个已分配给某个 flit 缓冲区的单元，那么将该单元分配给另一个缓冲区，既会破坏原缓冲区的链，又会导致两个缓冲区共享链上的一段序列。分配给缓冲区的单元中出现位错误，也会产生类似的后果。

关键在于，链表结构中的位错误所造成的影响是全局性的，会波及内存中的所有缓冲区。因此，必须在错误的影响尚未破坏内存中所有缓冲区之前，及时发现并控制这些错误。

链表缓冲区的错误控制方法结合了条目防御性编码和后台监控。防御性编码首先为指针本身添加奇偶校验，以便在指针被使用之前检测出单个位错误。每个 flit 单元都标记有一个类型字段，指示其是空闲还是已分配，并利用该字段进行一致性检查。在已分配的单元中存储缓冲区编号（或至少其哈希值），使我们能够检查单元是否在缓冲区之间发生了交换。最后，在每个单元中存储一个序列号（或至少其低几位），使输入单元能够检查单元是否发生了乱序。插入和删除操作会检查其指针的有效性、所读取单元的类型和缓冲区编号是否符合预期，以及所删除单元的序列号是否按顺序排列。如果检测到异常，则禁用受影响的缓冲区，并触发错误处理例程（通常由软件实现）。后台监控过程通过检查未被访问单元的一致性，对防御性编码形成补充。这可以检测出链表中间位置的错误，否则这些错误会一直潜伏，直到很久以后才暴露。监控过程会遍历每个缓冲区链表，从头到尾检查缓冲区中的单元数是否与计数一致、缓冲区中的每个单元是否已分配、每个单元是否属于该缓冲区，以及是否存在奇偶校验错误。对空闲链表也进行类似的检查。这种链表遍历通常还辅以标记-清除垃圾回收过程，以检测那些以某种方式脱离了任何链表的单元。在大多数实现中，后台监控器本身是一个硬件过程，利用缓冲区内存的空闲周期来执行这些一致性检查。当检测到错误时，

禁用受影响的缓冲区，并调用软件错误处理例程。

17.1.3 输入缓冲区分配

如果我们使用链表数据结构来构建输入缓冲区，就可以在共享内存的不同缓冲区之间动态分配存储空间。这种动态内存分配通过为繁忙的虚拟通道分配更多内存、为空闲的虚拟通道分配更少内存，有可能更高效地利用内存。许多研究表明，当虚拟通道数量很大时，负载在各通道之间的分布并不均匀。一些通道空闲，而另一些则过载。因此，为繁忙通道分配更多内存、为空闲通道分配更少内存是有利的。然而，如果不谨慎处理，动态缓冲区分配可能会抵消虚拟通道的优势。如果允许一个贪婪的虚拟通道分配所有存储空间然后阻塞，那么其他虚拟通道将无法取得进展。这反过来又会在虚拟通道之间产生依赖关系，并可能引发死锁。

为了控制输入缓冲区分配，我们在每个虚拟通道缓冲区的状态中添加一个计数寄存器，用于跟踪该通道缓冲区中的 flit 数量。另一个计数寄存器跟踪空闲链表上剩余的单元数。利用这些计数，我们可以实现多种不同的缓冲区分配策略。策略的输入是计数的当前状态，输出则指示是否允许每个虚拟通道分配额外的缓冲区。

例如，一种简单的策略是为每个虚拟通道缓冲区保留一个 flit 的存储空间。这可以保证我们不会因为将所有存储空间分配给单个虚拟通道然后使其阻塞而无意中导致死锁。每个虚拟通道至少有一个 flit 缓冲区，因此能够继续取得进展。为实现这一策略，我们允许虚拟通道向其缓冲区添加 flit，条件是：（a）其缓冲区当前为空（每个虚拟通道至少获得一个 flit），或（b）空闲 flit 缓冲区的计数大于计数为零的虚拟通道缓冲区的数量。

为了简化这一策略的管理，我们将空闲计数分为保留单元计数和浮动单元计数。保留单元计数按虚拟通道维护，当缓冲区变为空时递增，当第一个 flit 插入空缓冲区时递减。当向至少还有一个额外单元的缓冲区插入或移除单元时，我们递增或递减共享的浮动计数。通过这种安排，条件（b）可以通过检查浮动单元的计数是否为零来检验。

尽管这一简单策略确实保持了我们的死锁特性，但它仍可能导致缓冲区分配严重失衡。纠正这种失衡的一种方法是使用滑动限制分配器，它由两个参数 r 和 f 表征。参数 r 是为每个缓冲区保留的单元数，且必须至少为 1 以避免死锁。参数 f 是剩余单元（为每个缓冲区保留 r 个之后）中可以分配给任意一个缓冲区的比例。

与固定预留方案一样，我们通过维护两个空闲单元计数器来实现滑动限制分配器，一个用于预留单元 N_r，一个用于浮动单元 N_f。如果 $N_b < r$，则允许插入到计数为 N_b 的缓冲区 b，此时缓冲区来自预留池，我们递减 N_r；或者如果 $r \le N_b < fN_f + r$，则缓冲区来自浮动池，我们递减 N_f。

该策略的激进程度由参数 f 设定。当 f = 1 时，单个缓冲区可以分配整个空闲池，该策略退化为上述预留策略。当 $f = \frac{1}{B}$ 时，其中 B 是内存中的缓冲区数量，每个缓冲区最多可以分配池的 $\frac{1}{B}$，该策略等价于固定分区。在这两个极端之间存在更有用的 f 值。例如，当 f = 0.5 时，任何缓冲区最多可以使用剩余浮动池的一半；当 $f = \frac{2}{B}$ 时，任何

缓冲区使用的空间不能超过固定分区所分配空间的两倍。

17.2 交换机

交换机是路由器的核心。正是在这里，数据包和微片被实际导向它们期望的输出端口。交换机的主要设计参数是其加速比——即交换机提供的带宽与支持所有输入和输出全吞吐量所需的最小交换机带宽之比。正如我们将看到的，为交换机增加加速比简化了交换机分配的任务，并带来更高的吞吐量和更低的延迟。我们可以独立地设置交换机输入侧和输出侧的加速比。

给定加速比的交换机可以通过多种不同方式实现。如果一个微片时间涵盖多个时钟周期，我们可以按时间进行交换（如总线那样），而不是或附加于按空间进行交换。此外，在某些情况下，我们由一系列较小的交换机构成一个交换机，从而自身形成一个小型网络。然而，除了最简单的情况外，多级交换机会引发大量困难的分配和流控问题。

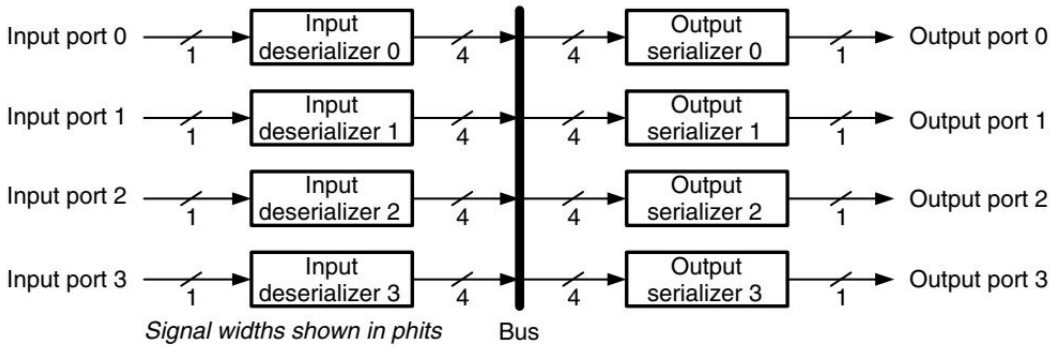


图 17.5 实现为总线的 4×4 交换机。每个输入解串器累积 phit，获取总线周期，然后通过总线并行地向一个或多个输出串行器发送 P 个 phit。输出串行器随后将各个 phit 发送到输出端口。

17.2.1 总线交换机

当一个微片时间包含的内部时钟周期（phit 时间）数多于端口数时，可以使用总线作为交换机，如图 17.5 所示。这种总线交换机的时序如图 17.6 所示。交换机的每个输入端口接收一个微片的 phit 并累积它们，直到至少有 P 个 phit，其中 P 是输入交换机端口的数量。一旦一个输入端口有 P 个 phit，它就仲裁总线。当它获得总线时，它向任意组合的输出单元广播这 P 个 phit（可能是一个完整的微片）。接收的输出单元随后解串这 P 个 phit，并一次一个地将它们发送到下游逻辑。为了在不因碎片化而损失带宽的情况下正常工作，每个微片的 phit 数必须是 P 的倍数。

图中展示了一个总线交换机有四个输入端口和四个输出端口的情况，每个微片长度为四个 phit，并且微片在输入和输出上是对齐的（即所有微片的第一个 phit 都在同一个时钟周期）。在这种情况下，每个输入端口累积一个完整的微片，然后通过总线发送它。因为每个输入端口在其总线周期内可以访问所有输出端口，所以不需要复杂的交换机分配。（见第 19 章。）每个输入端口在获得总线时保证可以访问它期望的输出，因

为它可以访问所有输出。这也意味着多播和广播可以无需额外努力即可完成（暂时忽略与多播相关的流控问题）。在图 17.6 中，使用了固定的总线仲裁方案。输入端口 0 在 phit 周期 0（每个微片的第一个 phit）获得总线，输入端口 1 在 phit 周期 1 获得总线，依此类推。这种仲裁极其简单。

对于总线交换机，加速比大于 1 没有意义。因为输入端口在其总线周期内总是获得其输出端口，所以它总是可以处理刚到达的输入微片，加速比为 1。额外的加速比会增加成本而不提高性能。

17.2 交换机

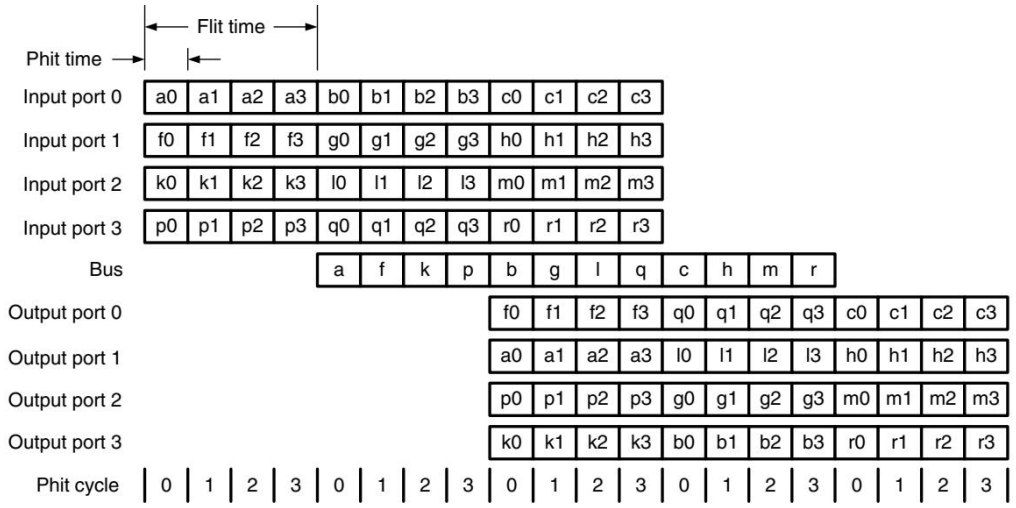


图 17.6 4×4 总线交换机的时序图，其中每个微片由四个 phit 组成。微片 a、f、k 和 p 在第一个微片时间内一次一个 phit 地到达四个输入端口；微片 b、g、l 和 q 在第二个微片时间内到达；微片 c、h、m 和 r 在第三个微片时间内到达。当第 i 个微片到达时，第 $(i-1)$ 个微片通过总线广播传输，每个 phit 时间传输一个微片，而第 $(i-2)$ 个微片正在输出端口上重新串行化。

总线交换机的优点是极其简单。但这种简单性以浪费端口带宽和增加延迟为代价。总线带宽等于路由器的总带宽 Pb，其中 P 是端口数，b 是每端口带宽。然而，为了与该总线接口，每个输入解串器必须具有 Pb 的输出带宽，每个输出串行器必须具有 Pb 的输入带宽。因此，从输入解串器输出、进入输出串行器的总带宽为 P^2b，因为在任何给定周期内，只有 $\frac{1}{P}$ 的带宽（至少在输入侧）是活跃的。

如果路由器实现在单个芯片上，这种额外带宽虽有一定代价，但并非严重限制。3 图 17.7 展示了如何将总线线路直接布设在输入解串器和输出串器之上，从而降低额外带宽的成本。实际上，这种总线交换机的布线复杂度恰好是同等带宽交叉开关的两倍：总线交换机需要 $2P \times P$ 条 phit 宽的总线，而交叉开关只需 $P \times P$ 条。总线交换机中的每个串行器或解串器还需要 2P 个 phit 的存储空间，而交叉开关则是无状态的。

336 第 17 章路由器数据通路组件

总线交换机的延迟和粗粒度是其比额外带宽更严重的限制。如图 17.6 所示，总线交换机必须以至少 P 个 phit 为单位进行交换。如果 flit 尺寸小于此值，总线交换机就不可行。此外，由于需要对输入进行解串并对齐输出，总线交换机的延迟为 2P 个 phit 时

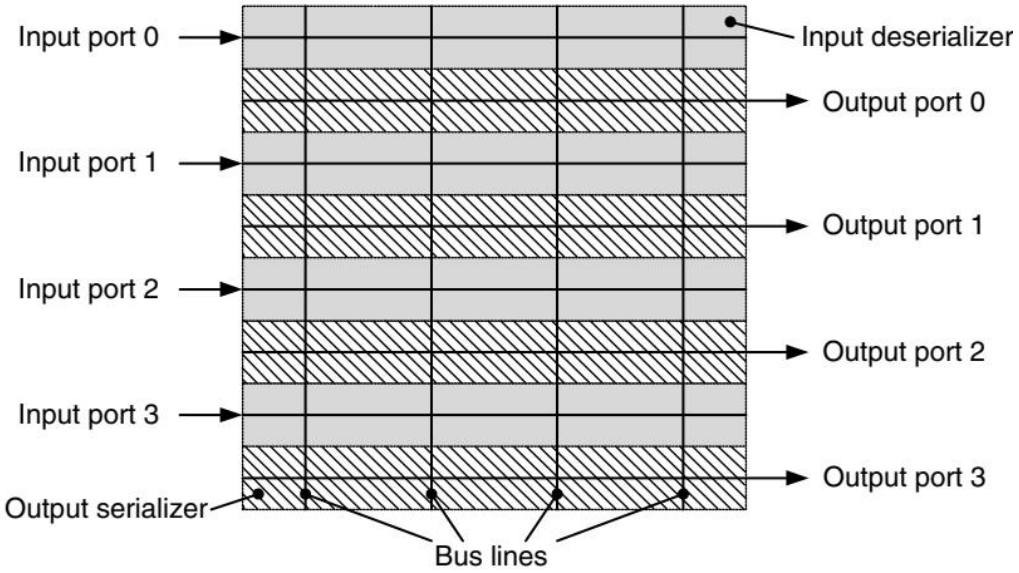


图 17.7 4×4 总线交换机的布局与布线图。phit 宽的输入和输出线路水平走线。每条输入线馈入一个输入解串器（阴影部分），而每条输出线由一个输出串行器（斜线部分）驱动。4-phit 宽的总线垂直布设在串行器和解串器之上。

间。4 相比之下，交叉开关的延迟仅为单个 phit 时间。

在直接总线交换机和完全交叉开关之间的折中方案是多总线交换机。图 17.8 展示了一个具有两条总线的输出分区交换机。输出 0 和 1 共享总线 1，输出 2 和 3 共享总线 2。通过将每条总线提供的输出数量减半，我们也将每个周期必须在总线上传输以保持输出忙碌的 phit 数量减半。因此，串行化延迟减半。这种延迟的降低以更复杂的分配为代价。我们不能再使用固定分配，而必须解决一个匹配问题，为每个周期分配输入端口和输出端口。（见第 19 章。）

图 17.8 所示的多总线配置等效于一个 2-phit 宽的 4×2 交叉点开关（四个 2-phit 宽的输入交换到两条 2-phit 宽的总线上）。这个 4×2 开关的每个输出驱动两个输出串行器，这些串行器从总线接收 2-phit 宽的数据，并一次一个 phit 地输出这些数据。在极端情况下，如果我们将每个输出放在单独的总线上，多总线交换机就等效于一个 $P \times P$ 交叉点。

我们既可以构建输入分区多总线交换机，也可以构建输出分区多总线交换机，这将在练习 17.4 中进一步探讨。此外，如练习 17.5 所述，我们可以同时在输入和输出上对交换机进行分区。

内存交换机，如图 17.1（a）所示，是一种在总线的输入侧和输出侧之间用内存隔开的总线交换机。这些内存交换机可以按照练习 17.6 所述在输入或输出或两者上进行分区。

17.2.2 交叉开关

回顾第 6.2 节，一个 $n \times m$ 交叉开关或交叉点开关可以将 n 个输入中的任何一个连接到 m 个输出中的每一个。我们用图 17.9 所示的符号表示交叉开关。交叉开关的结构

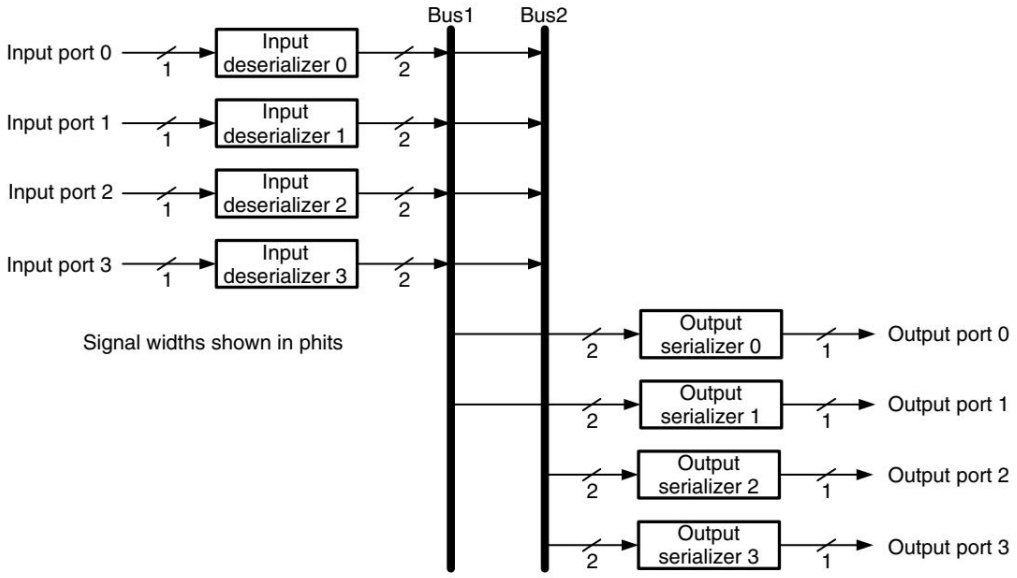


图 17.8 输出分区多总线交换机：四个输入可以一次驱动两个 phit 到任一条总线上。输出 0 和 1 从总线 1 接收两个 phit 的包，而输出 2 和 3 从总线 2 接收。这种组织以更复杂的分配为代价降低了延迟和额外带宽。

在第 6.2 节中详细讨论，此处不再重复。

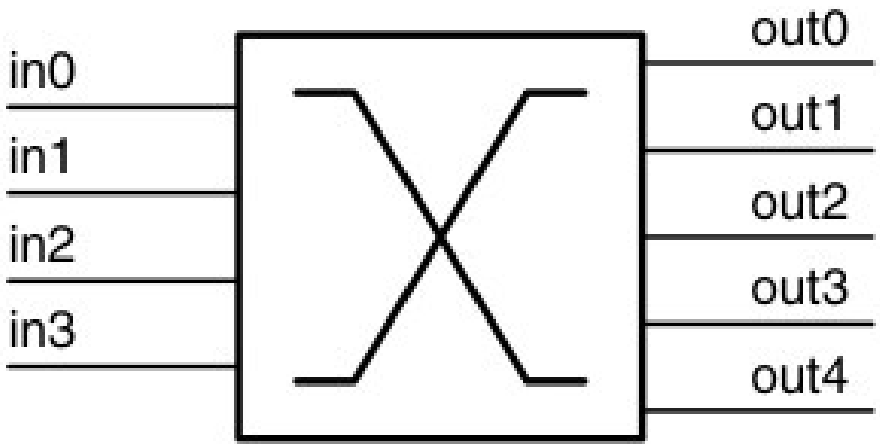


图 17.9 四输入五输出交叉开关的符号表示

在路由器数据通路中使用交叉开关作为交换机时，主要问题是加速比：提供的带宽与所需带宽之比。我们可以在交叉开关的输入、输出或两侧提供加速。这种加速可以在空间上（额外的输入）或时间上（更高带宽的输入）提供。5 图 17.10 和 17.11 展示了具有输入加速（图 17.10[b]）、输出加速（图 17.11[a]）以及输入输出同时加速（图 17.11[b]）的交换机示例。

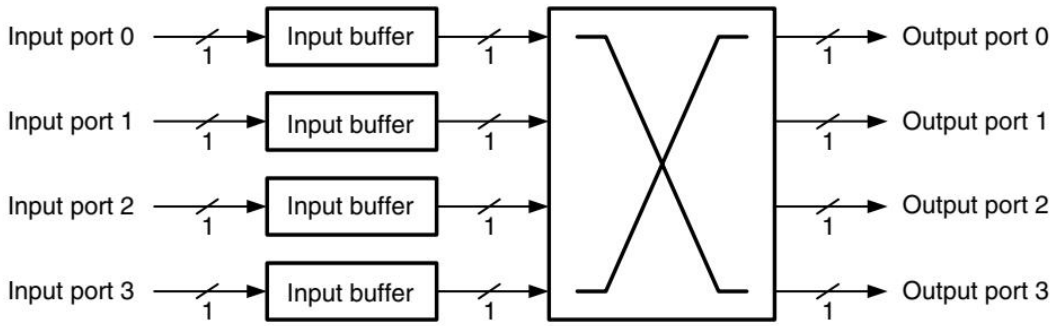
为交叉开关提供加速可以简化交换机的分配任务，或者用简单的分配器获得更好的性能。6 特别是在 flit 尺寸小且延迟要求低的情况下，构建具有更多加速的交换机通常比构建一个以较少加速达到所需性能的分配器更便宜。交换机分配是以无冲突的方式

将交叉开关输入端口和输出端口分配给输入缓冲区中等待的 flit 子集的过程。分配的主题在第 19 章详细讨论。这里我们假设使用简单的随机可分离分配器来比较具有不同加速量的交换机的性能。

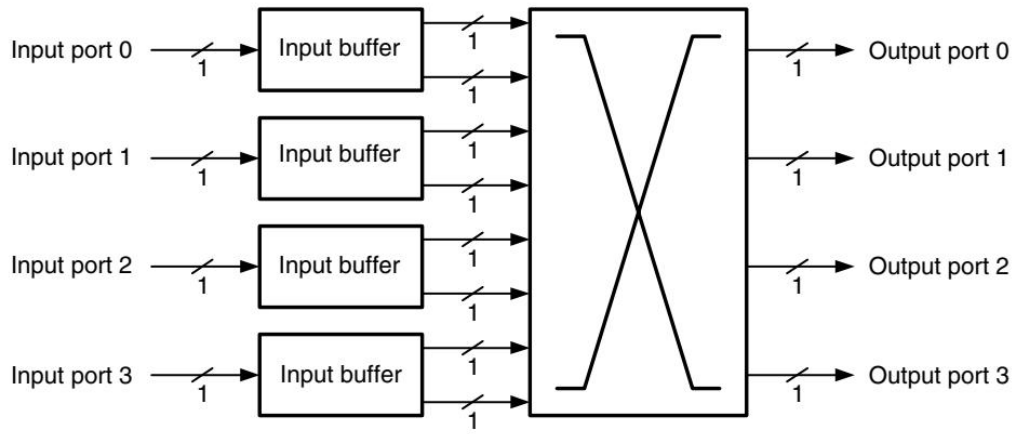
随机可分离分配器的工作方式是：首先让每个输入缓冲区从所有等待的 flit 中随机选择一个 flit，以便在交叉开关的每个输入端口上转发。如果流量均匀，每个输出成为每个选定的输入 flit 目的地的概率相等。然后，每个输出端口从竞标该输出的输入端口上的 flit 中随机选择一个 flit。

假设我们的 k 输入、k 输出路由器具有输入加速 s 且无输出加速；因此，交叉开关有 sk 个输入。使用随机可分离分配器，只要某个输入为该输出选择了一个 flit，输出就会转发一个 flit。交换机的吞吐量 Θ 是在分配第一阶段选择的 sk 个 flit 中至少有一个是发往给定输出的概率。这个概率由下式给出

$$\Theta = 1 - \left(\frac{k-1}{k}\right)^{sk}. \tag{17.1}$$



(a)



(b)

图 17.10 交叉开关输入加速比。（a）中所示的 4×4 交换机具有输入加速比 1——输入数量与输出数量匹配，且均为单位带宽。（b）中的交换机具有输入加速比 2——输入数量是输出数量的两倍——从而简化了分配问题。

图 17.12 绘制了 4 输出交换机的吞吐量与输入加速比的关系曲线。使用可分离分配器时，无加速比的交换机仅达到容量的 68%，而输入加速比为 2 时达到容量的 90%，加速比为 3 时容量提升至 97%。

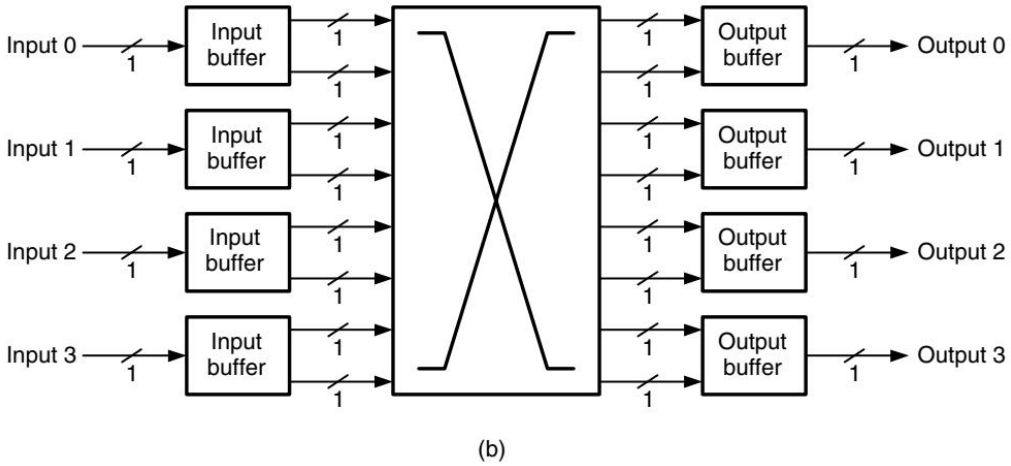


图 17.11 交叉开关输出加速比。（a）输出加速比为 2 的 4×4 交换机。（b）加速比为 2 的交换机——同时具有输入加速比 2 和输出加速比 2。

当此情况下交换机的输入加速比达到 4（或一般情况下的 k）时，使用分配器就没有意义了。此时，每个输入缓冲区的 k 个交叉开关输入可以各自专用于单个输出端口，使得每个交叉开关输入处理单个输入-输出端口对之间的 flit。这种情况与上文第 17.2.1 节中描述的总线交换机完全相同，无需分配即可实现 100% 吞吐量。由于通过简单的仲裁和加速比 k 即可实现 100% 吞吐量，因此永远没有必要制造输入加速比大于 s = k 的交换机。

具有输出加速比的交换机可以通过反向操作的可分离分配器实现与具有相同输入加速比的交换机完全相同的加速效果。每个输出端口选择一个发往该输出端口的等待 flit，然后向持有该 flit 的输入端口发出请求。在实践中，这很难实现，因为它需要将等待输入 flit 的信息传递给输出端口。因此，并且为了消除输出缓冲的需要，通常首选输入加速比而非输出加速比。

图 17.11 交叉开关输出加速比。（a）输出加速比为 2 的 4×4 交换机。（b）加速比为 2 的交换机——同时具有输入加速比 2 和输出加速比 2。

同时具有输入和输出加速比的交换机可以实现大于 1 的吞吐量。当然，它无法持续保持这一吞吐量，因为输入和输出缓冲区是有限的。如果输入和输出加速比分别为 s_i 和 s_o，并假设 $s_i \geq s_o$，我们可以将其视为总加速比为 s_o，且相对于总加速比的输入加速比为 s_i/s_o 的交换机。因此，此情况下的吞吐量由下式给出

$$\Theta = s_o \left(1 - \left(\frac{k-1}{k}\right)^{\frac{s_i k}{s_o}}\right).$$

最常见的是，交叉开关以较小的总加速比（1.2 到 1.5 之间）运行，这样无需复杂的分配即可实现 100% 吞吐量，并具有低延迟，然后将输入端口加倍以进一步简化分配。

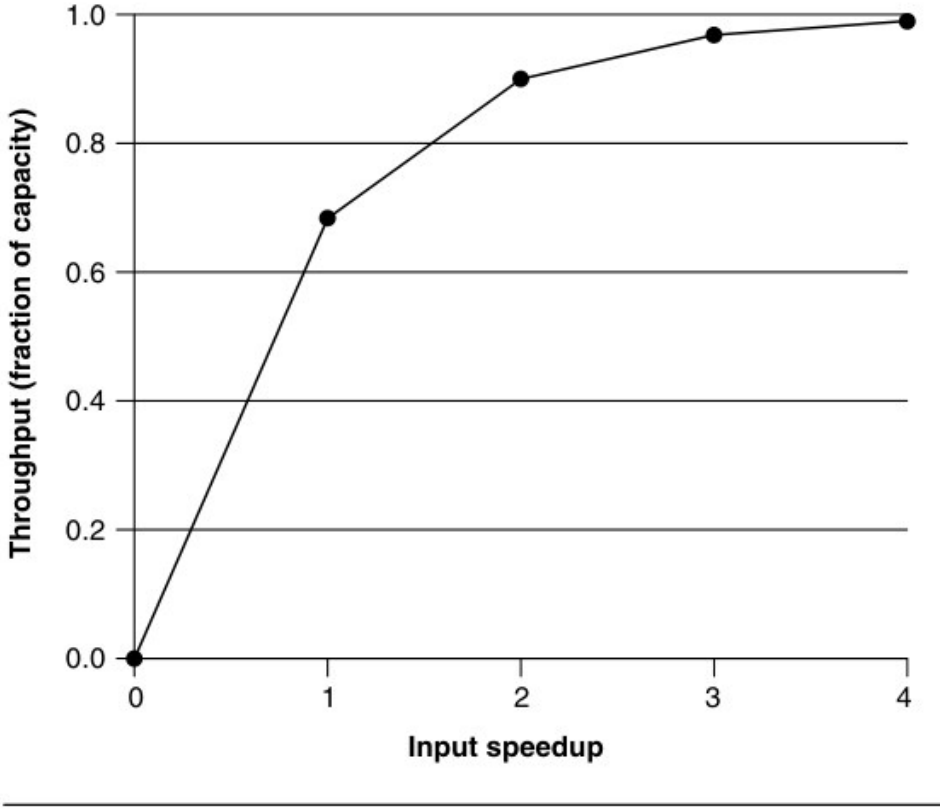
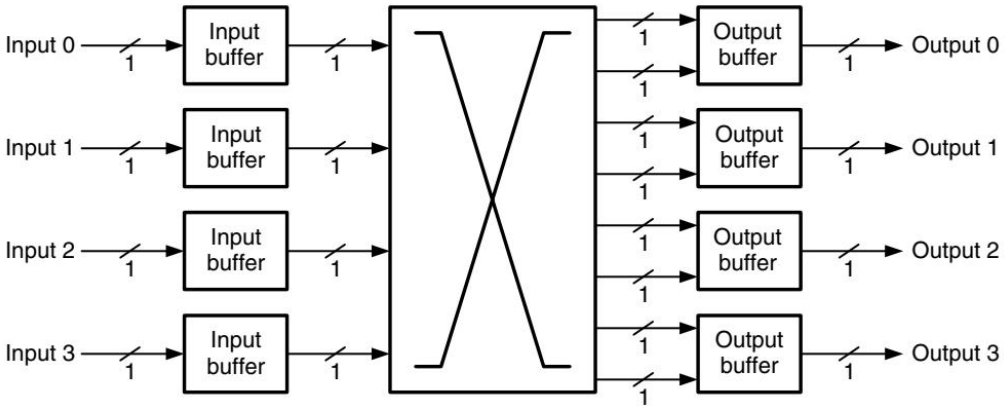


图 17.12 交换机输入加速比对吞吐量的影响



17.2.3 网络交换机

交换机可以实现为较小交换机的网络。例如，为 3-D 环面网络实现维序路由器所需的 7 输入 ×7 输出交换机可以用三个 3 × 3 交换机实现，如图 17.13 所示。这将实现交换机所需的交叉点数量从 49 减少到 27。该交换机的每个维度可以通过分离正负方向进一步划分，如图 17.14 所示。然而，这种额外的划分将交叉点数量增加到 33。
划分交换机的优点在于减少交叉点的总数并增强逻辑的局部性，从而减少线长。7 然而，缺点是，如果一次性分配整条路径，则划分后的交换机需要更复杂的控制，或者在划分之间需要 flit 缓冲区来在 flit 仲裁下一个交换机时保持 flit。此外，根据网络、流量和路由的不同，子路由器之间内部链路上的负载可能会大于通道本身的负载。

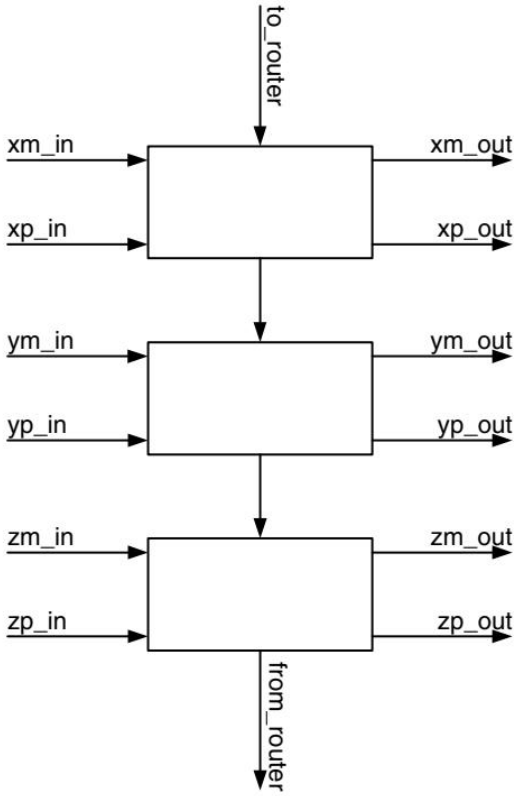


图 17.13 通过用三个 3 × 3 交换机实现受限的 7 × 7 交换功能，可以将维序路由器划分为更小的路由器。

通常，应避免将交换机划分为交换机网络，除非有必要使每个划分适应封装层次结构的某一级别。

17.3 输出组织

图 16.1 中输出单元的数据通路主要由一个 FIFO 缓冲区组成，以匹配输出通道的速率与交换机的速率。对于没有输出加速比的交换机，不需要数据通路。当 flit 离开交换机时，可以直接放置到输出通道上。然而，对于具有输出加速比（时间或空间）的交换机，需要一个 FIFO 缓冲区来在交换机和通道之间暂存 flit。

无需在虚拟通道之间划分输出缓冲区存储。单个 FIFO 缓冲区可以在多路复用到输出物理通道上的所有虚拟通道之间共享，而不会在虚拟通道之间引入依赖关系。不存在依赖关系，因为输出 FIFO 从不会阻塞。每个 flit 时间，它都会从 FIFO 头部取出 flit（如果有）并将其放到通道上，无论下游 flit 缓冲区的状态如何。背压不会影响输出缓冲区，而是在路由器流水线的 SA 阶段限制交换机输入侧的流量。

17.3 输出组织

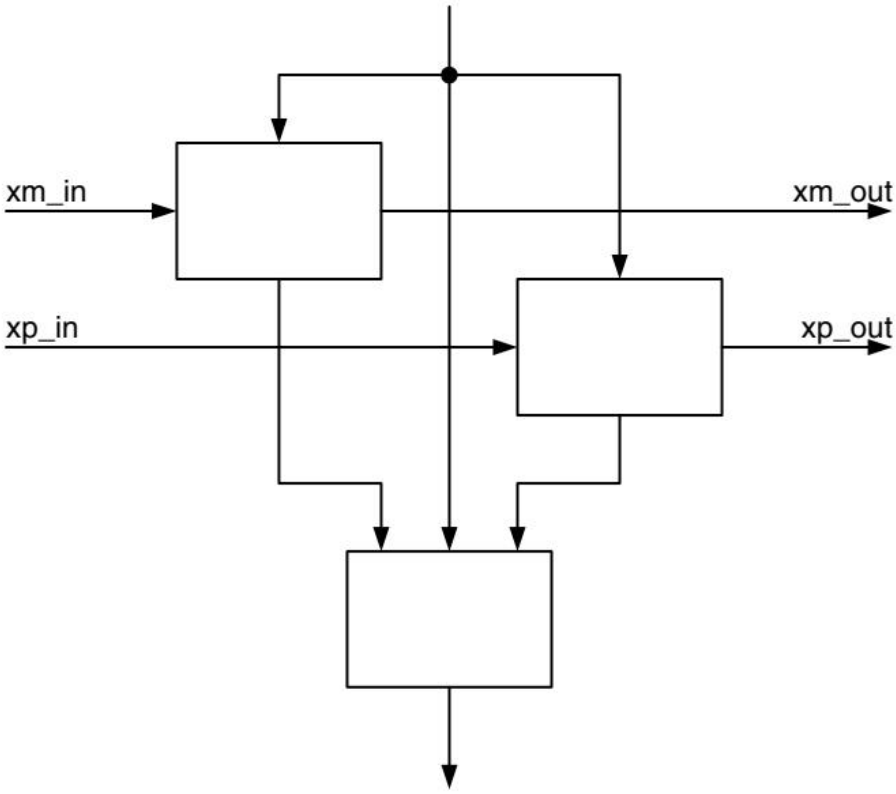


图 17.14 维序路由器可以进一步划分，将每个 3×3 开关分为两个 2×2 开关和一个 3：1 多路复用器。

尽管从通道到输出缓冲区没有背压，但必须从输出缓冲区到开关分配器有背压以防止缓冲区溢出。当缓冲区占用超过阈值时，输出缓冲区向开关分配器发出信号，停止向该输出端口发送流量，直到通道可以将缓冲区排空到阈值以下。

输出缓冲区不必很大。通常 2 到 4 个 flit 的缓冲就足以匹配开关和通道之间的速度，更大的缓冲区会消耗面积而不提高性能。

17.4 案例研究：IBM Colony 路由器的数据通路

IBM Colony 路由器是 IBM SP 互连网络家族中的第三代路由器 [177]。前一代 Vulcan 路由器在第 11.3 节的案例研究中讨论过，Colony 路由器采用了类似的内部架构和相同的多级网络拓扑。Colony 还包括若干增强功能，如更高的信号速率、更快的内部周期时间，以及对自适应和组播路由的支持。其结果是该路由器可以扩展以实现大规模互连网络。例如，Colony 路由器用于 ASCI White，这是一台拥有 8192 个处理器的

IBM SP 超级计算机，曾在 2000 年至 2001 年间是世界上最快的计算机。

Colony 内部架构的概览如图 17.15 所示。8×8 交换元件采用混合数据通路组织，既使用共享的中央存储器来存储和交换数据包，也使用输入排队的旁路交叉开关来交换数据包。任何到达交换机输入端口的数据包都请求访问旁路交叉开关。如果交叉开关所请求的输出端口可用，数据包立即通过路由器流水线传输，从而最小化延迟。只要网络负载低，数据包就有很大机会获得旁路交叉开关的访问权，其总体延迟与输入排队路由器非常接近。

如果输入端口未能获得旁路交叉开关的访问权，并且数据包的一个块（定义为 8 个 flit）已到达输入端口，则该数据包请求访问中央存储器。此请求保证最终会被服务。由于交换机的所有 8 个输入端口都可能被阻塞并需要访问中央存储器，因此进入存储器的带宽必须等于路由器的总输入带宽。虽然中央存储器在概念上可以用 16 端口 SRAM 实现（路由器的每个输入和输出各一个端口），但高端口数的 SRAM 昂贵且速度慢，因此对于高带宽路由器设计是不切实际的。相反，Colony 路由器的设计者将中央存储器实现为 8-flit 宽（一个块）的 2 端口 SRAM。然后，SRAM 的读端口和写端口在时间上复用：每个输入（输出）端口向 SRAM 传输整个 8-flit 块的数据。如果多个输入（输出）端口在同一周期想要访问中央存储器，它们按最近最少使用顺序得到服务。对宽的分体 SRAM 进行时间复用使得中央存储器的实现变得可行，但也将交换机的粒度增加到 8-flit 块。如果数据包长度不是 8 个 flit 的倍数，交换机的很大一部分带宽可能会被浪费。

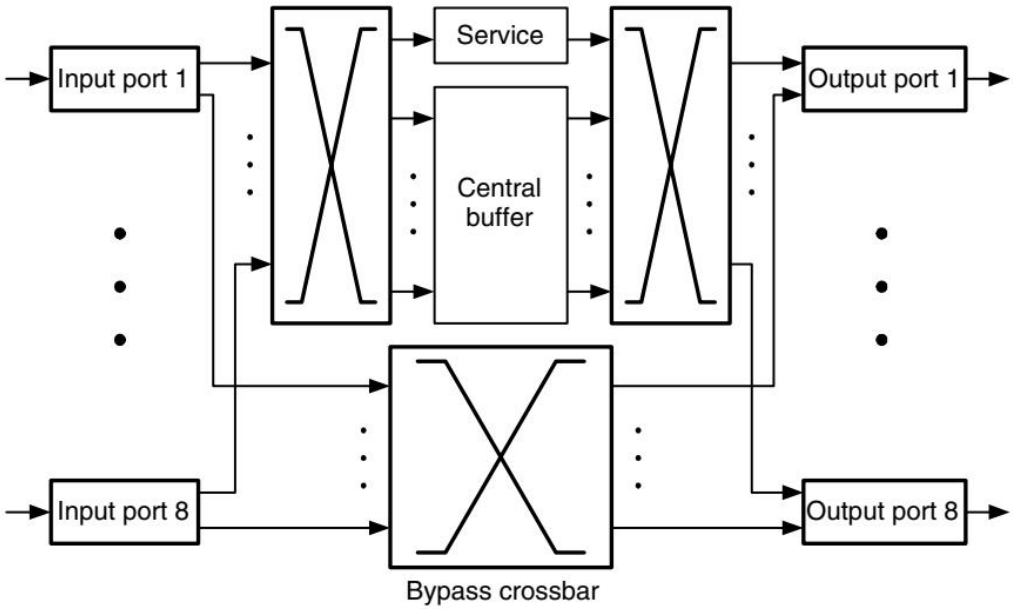


图 17.15 IBM Colony 路由器的架构。8×8 交换机围绕一个共享的中央缓冲区组织，该缓冲区在路由器端口之间进行时间复用。旁路交叉开关允许数据包直接通过路由器，在争用较低时减少延迟。服务块发起并执行网络服务命令，如诊断和错误报告。

17.4 案例研究：IBM Colony 路由器的数据通路作为进一步的优化，宽 SRAM 被分成四个 2-flit 宽的 SRAM。不是每个端口在同一周期访问所有 SRAM 体，而是访问在时间

上错开。例如，输入一在一个周期传输块的第一个 flit，在下一个周期传输第二个 flit，依此类推。8 这种方法允许从端口到 SRAM 的连接使用通过交叉开关交换的 1-flit 宽通道来实现。输入和输出端口访问中央存储器需要一个这样的交叉开关，如图 17.15 所示。如果体访问不在时间上错开，所需的布线量将大大增加——每个端口需要 8 倍宽的通道来连接到中央存储器。

Colony 路由器中央存储器的使用情况通过几个链表结构来跟踪。存储在中央存储器中的每个数据块有两个额外字段用于维护这些链表。第一个字段是下一个数据包指针，每个输出维护一个指向发往该输出的数据包列表的头指针（FirstP）和尾指针（LastP）。由于数据包可能由多个块组成，第二个字段包含下一个块指针。类似地，也为块列表维护头指针（FirstC）和尾指针（LastC）。一个空闲列表跟踪中央存储器中所有可用的块，并维护指向列表头部的指针（Free）。通过同时跟踪空闲列表中的第二个条目（FreeList），块列表可以支持每周期一次读和一次写，而只需要一个 2 端口 SRAM。（见练习 17.8。）

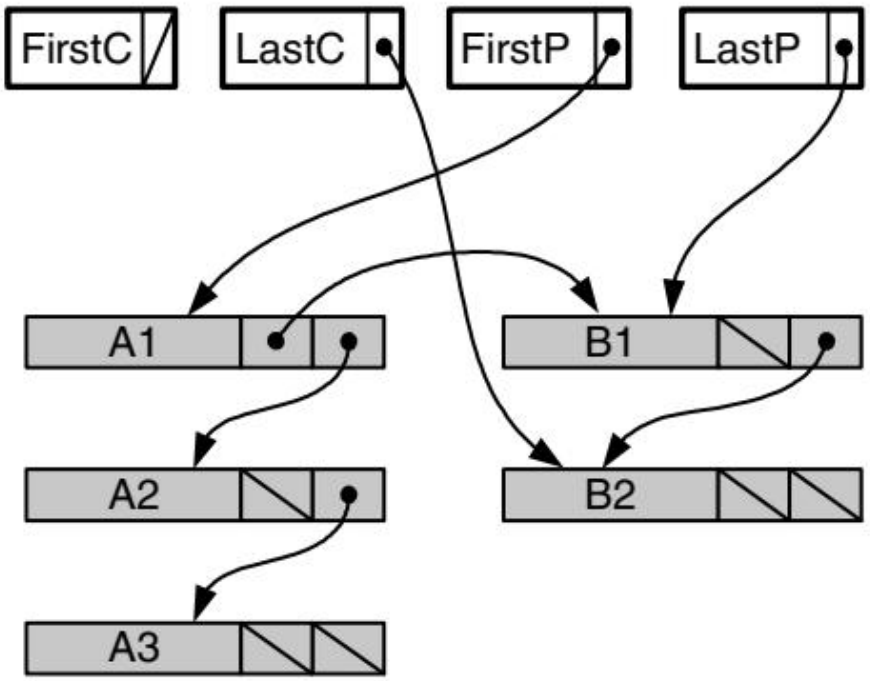
路由器一个输出的这些链表示例如图 17.16 所示。最初，两个数据包存储在中央缓冲区中，如图 17.16（a）所示。在下一个周期，图 17.16（b），数据包 A 的第一个块被读取，数据包 B 的一个块被写入存储器。读取首先检查块列表的头部 FirstC，在这种情况下为 NULL。这表明下一个块应从新数据包中获取，本例中为数据包 A。因此，通过跟随 FirstP，找到新数据包的第一个块并从存储器中读取。FirstC 设置为所读块的下一个块指针，FirstP 更新为指向存储器中的下一个数据包。然后释放的块被添加到空闲列表。写操作首先从空闲列表中取出一个块并存储该块的数据。由于此写入是针对数据包 B 的，写操作只需跟随 LastC，并从 B2 添加一个下一个块指针到 B3。（见图 17.16（b）。）如果写入的是新数据包，则会跟随并更新 LastP。

17.5 文献注释

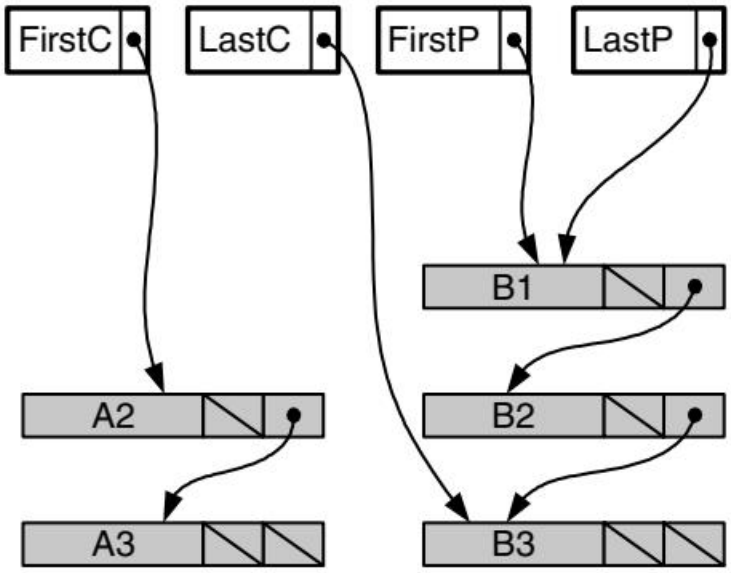
Tamir 和 Frazier [182] 给出了动态缓冲区管理方法的详细实现，Stunkel 等人 [177] 也是如此。许多早期的互连网络，如 J-Machine [138] 和 Cray T3D [95]，按维度划分其交换机以简化实现。Nojima 等人 [137] 提出了一种最大划分设计。其他网络，如 Torus Routing Chip [56]、SGI SPIDER [69] 和 Cray T3E [162]，选择了具有不同加速量的基于交叉开关的交换机。共享内存方法包括 Katevenis 等人 [92] 的设计以及案例研究中详述的 IBM Colony 交换机 [177]。Sivaram 等人 [170] 提出的 HIPIQS 架构将输入排队方法与加速和动态缓冲区管理相结合。最后，Ahmadi 和 Denzel [5] 以及 Tobagi [184] 都对本章未涵盖的不同数据通路组织提供了出色的综述。

17.6 练习

17.1 环形缓冲区管理硬件。绘制一个寄存器传输级图，展示从环形缓冲区同时插入和移除 flit 所需的逻辑。特别注意确定满和空状态的逻辑。可选地，用 Verilog 编写代码。



(a)



(b)

图 17.16 Colony 路由器中央存储器中使用的链表数据结构示例。链表中的每个条目有三个字段，在图中从左到右显示：块数据、下一包和下一块。为简单起见，仅显示一个输出端口的链表。

17.2 链表缓冲区管理硬件。绘制一个示意图，展示在单周期内从链表 flit 缓冲区插入或移除 flit（二者之一）所需的逻辑。可选地，用 Verilog 编写代码。
17.3 基于总线的交换机设计。假设您有一个路由器，P = 7 个输入端口和 P = 7 个输出端口，flit 大小为 8 phit。解释如何为此路由器构建高效的总线交换机。总线每个周期需要承载多少 phit?
17.4 输入划分的基于总线的交换机。绘制一个双总线输入划分交换机的框图，类似于图 17.8，但有两个输入驱动每条总线，每个输出从两条总线接收。允许每个输出在任意给定周期内仅从这两条总线之一读取。描述一种分配此类交换机的方法。现在考虑输出可以同时从两条总线接收的情况。这会如何改变您的分配方法?
17.5 双重划分的基于总线的交换机。绘制一个具有八个端口的输入和输出划分交换机的图——将输入和输出都划分为两组，并使用四条独立总线进行交换。解释此交换机的分配。
17.6 划分存储器交换机。将总线划分推广到图 17.1（a）所示的存储器交换机组织。如果输出被划分为两组，需要多少个独立的存储器？这些存储器的宽度是多少?
17.7 带缓冲的交叉开关。对练习 17.6 中的存储器交换机递归应用划分。即，对于 P 输入交换机，将输入划分为 P 组。输出同样如此。此交换机中需要多少个独立的存储器? 这些存储器的宽度是多少?
17.8 跟踪两个空闲列表条目的优势。为 IBM Colony 链表结构编写插入和删除的伪代码，使用两个空闲列表条目（存储指向空闲列表第一个和第二个条目的指针）。使用三个独立的存储器（数据、包指针和块指针），每个都有一个读端口和一个写端口。提示：考虑三种不同的情况：仅插入、仅删除以及同时插入和删除。

注释

- 1. 通过解复用到共享存储器并多路复用出共享存储器进行的交换类似于总线交换机（第 17.2.1 节）执行的交换，不同之处在于存储器将交换机输入侧和输出侧的时序解耦。

- 2. 当然，在这两点之间还有各种可能性——例如，在每对输入端口之间共享一个存储器。

- 3. 然而，如果试图将总线交换机的串行器和解串器构建在独立的芯片上，引脚带宽的成本将高得令人望而却步。

- 4. 这个延迟可以降低到 $P+1+a$ 个 phit 时间，其中 a 是在输出 flit 未对齐的总线交换机上进行总线仲裁时损失的周期数。如果所有 P 个端口都在竞争，a 的平均值为 $P/2$。

- 5. 空间加速比总是整数值（不可能有半个交换机端口）。然而，时间加速比可以是非整数值。例如，你可以让一个交叉开关在 3 个周期内传递一个 flit，而路由器每 4 个周期处理一个 flit，从而获得 1.33 的加速比。

- 6. 在其他上下文中，加速比也用于补偿因向数据包或 flit 添加报头，或在将数据包分割成 flit 时因分片而产生的开销。在这种情况下，我们假设交换 flit 时不增加额外开销，因此严格使用加速比来简化分配。
- 7. 这就是在第 7.2.2 节中引入这种划分类型作为维度切片或端口切片的动机。
- 8. 双 flit（解）串行化缓冲区将交换机的 1-flit 数据通路宽度与 SRAM 的 2-flit 宽度相匹配。

第十八章　仲裁

仲裁

虽然路由器的数据通路由缓冲区和交换机组成，但路由器的控制通路主要由仲裁器和分配器组成。如第 16 章所述，我们使用分配器将虚拟通道分配给数据包，并将交换机周期分配给 flit。本章讨论仲裁器，它解决对单个资源的多个请求。仲裁器本身不仅有用，而且是分配器的基本构建模块，分配器将多个请求者与多个资源匹配。分配器将在第 19 章讨论。

每当一个资源（如缓冲区、通道或交换机端口）被多个代理共享时，就需要一个仲裁器来一次将一个代理分配给该资源。图 18.1（a）显示了一个 n 输入仲裁器的符号，用于在一组代理（例如连接到该输入端口的虚拟通道（VC））之间仲裁对资源（例如交叉开关交换机的输入端口）的使用。每个有 flit 要发送的虚拟通道通过断言其请求线来请求访问输入端口。例如，假设有 n = 8 个 VC，并且 VC 1、3 和 5 断言其请求线 r_1, r_3 和 r_5。然后仲裁器选择这些 VC 之一——比如 VC i = 3——并通过断言授权线 g_3 将输入端口授予 VC 3。VC 1 和 5 仲裁失败，必须稍后通过重新断言 r_1 和 r_5 再次尝试。大多数情况下，这些线将保持高电平，直到 VC 成功并获得授权。

18.1　仲裁时序

授权的持续时间取决于应用。在某些应用中，请求者可能需要多个周期不间断地访问资源。在其他应用中，每个周期重新仲裁可能是安全的。为了适应这些不同的应用，我们可以构建仲裁器，发出单周期、固定周期数或直到资源释放的授权。

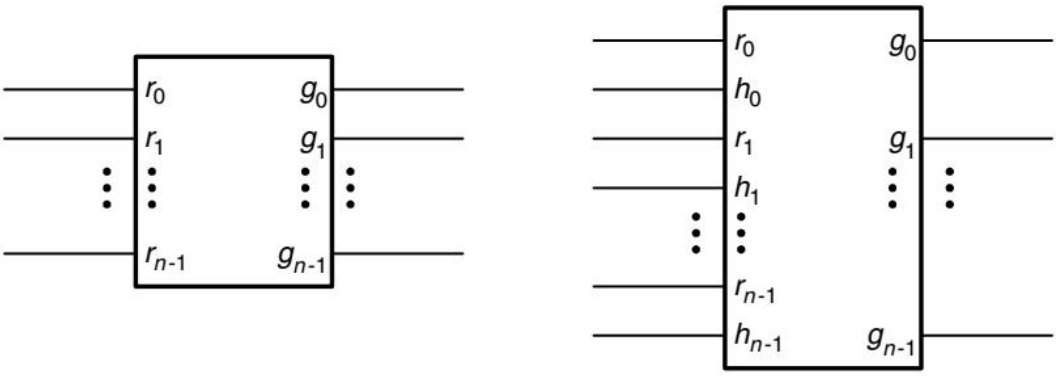


图 18.1　仲裁器符号。（a）仲裁器接受 n 条请求线 $r_0, \ldots, r_{n-1}$，在断言的请求线中进行仲裁，选择一条 r_i 进行服务，并断言相应的授权线 g_i。（b）为了允许授权保持任意时间而请求之间没有间隙，向仲裁器添加了一条保持线。

例如，在一个单周期处理 flit 的交换机分配中使用的仲裁器，只会授予某个虚拟信道

一个周期的交换机端口使用权。在这种情况下，每个周期都会进行一次新的仲裁，每次仲裁授予交换机一个周期的使用权——通常但不总是紧接仲裁之后的那个周期。图 18.2（a）说明了这种情况。在这里，VC 0 和 VC 1 都在周期 1 通过置位 r_0 和 r_1 来请求交换机。VC 0 赢得了这次仲裁，在周期 2 获得了所请求的交换机端口，并在同一周期撤销了它的请求。VC 1 保持其请求为高，由于没有竞争，在周期 3 获得了交换机端口。如果两个 VC 都保持它们的请求线为高，并且仲裁器是公平的（见第 18.2 节），那么交换机端口将在它们之间逐周期交替，如图中周期 6 到 9 所示。

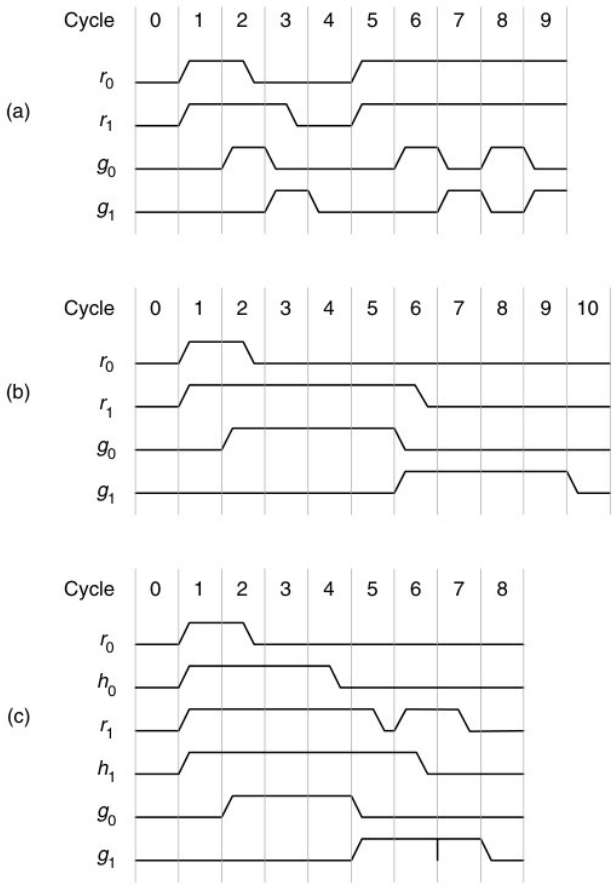


图 18.2 仲裁时序：单周期授权与多周期固定时长授权

图 18.2（b）显示了每次授权持续四个周期的情况；例如，这种情况会出现在一个路由器中，其中交换机需要四个周期来传输每个 flit，并且 flit 传输不能被中断。在这里，VC 0 在周期 1 赢得仲裁，并在周期 2 到 5 获得交换机使用权。VC 1 持续置位其请求，必须等到周期 6 才能访问交换机。

最后，图 18.2（c）显示了资源被授予直到释放的情况。在这种情况下，VC i 通过请求线 r_i 和保持线 h_i 与仲裁器接口，如图 18.1（b）所示。在赢得仲裁后，只要保持线保持为高，VC 就独占使用该资源。例如，这里 VC 0 持有交换机 3 个周期（周期 2 到 4），然后 VC 1 持有交换机 2 个周期（周期 5 和 6）。

一些仲裁器通过保持资源直到请求线被撤销来实现可变长度的授权。然而，这种方法要求同一代理在两次使用资源之间至少有一个空闲周期。使用单独的保持线，代理可

以在同一周期内释放资源并再次请求它。VC 1 在周期 6 正是这样做的，再次赢得交换机一个周期（周期 7）。
第 18 章仲裁

18.2 公平性

仲裁器的一个关键属性是它的公平性。直观地说，公平的仲裁器是对不同的请求者提供平等服务的仲裁器。然而，平等的确切含义可能因应用而异。公平性的三个有用定义是：
弱公平性：每个请求最终都会被服务。

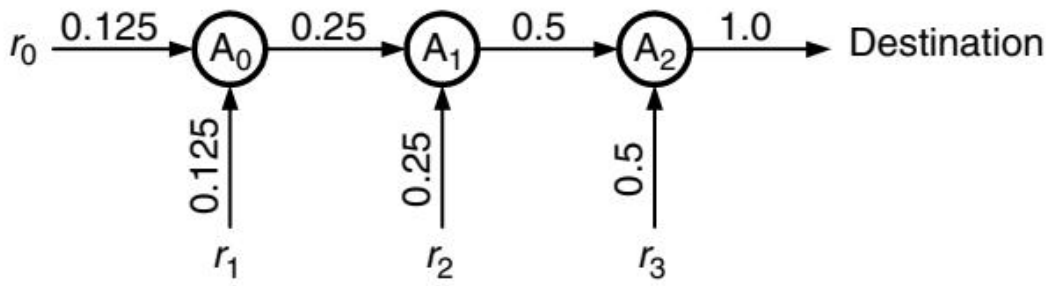


图 18.3 在具有多个仲裁器的系统中，每个仲裁器可以在本地是公平的，而在系统级别整体仲裁是不公平的。在这个例子中，请求者 r_3 获得的目的地带宽是 r_1 的四倍，尽管三个仲裁器中的每一个在强意义上都是公平的。

强公平性：请求者被服务的频率相等。这意味着，在足够多的仲裁次数 N 上平均，一个请求者被服务的次数与另一个请求者被服务的次数之差在 ϵ 以内。通常，我们将强公平性修改为加权公平性，使得请求者 i 被服务的次数与其权重 w_i 成正比。
FIFO 公平性：请求者按照它们发出请求的顺序被服务。这就像面包店的顾客，每个人取一个号，按照到达的顺序被服务。
即使仲裁器在本地是公平的，使用该仲裁器的系统也可能不公平，如图 18.3 所示。该图说明了四个源 $r_0, \ldots, r_3$ 向单个目的地发送数据包的情况。这些源通过三个强公平的 2：1 仲裁器合并。尽管每个仲裁器公平地将其输出带宽的一半分配给它的两个输入，但带宽并没有公平地分配给四个源。源 r_3 获得总带宽的一半，因为它只需要参与一次仲裁，而源 r_0 和 r_1 只获得带宽的 $\frac{1}{8}$——因为它们必须参与三轮仲裁。同样的情况以及解决它的方法在第 15.4.1 节和本章第 18.6 节中讨论。

18.3 固定优先级仲裁器

如果我们按线性顺序分配优先级，我们可以将仲裁器构造为迭代电路，如图 18.4 中的固定优先级仲裁器所示。我们将仲裁器构造为位单元的线性阵列。每个位单元 i，如图 18.4（a）所示，接受一个请求输入 r_i 和一个进位输入 c_i，并生成一个授权输出 g_i 和一个进位输出 c_{i+1}。进位输入 c_i 表示资源尚未被授予更高优先级的请求，因此可用

于当前位单元。如果当前请求为真且进位为真，则置位授权线并撤销进位输出，表示资源已被授予且不再可用。

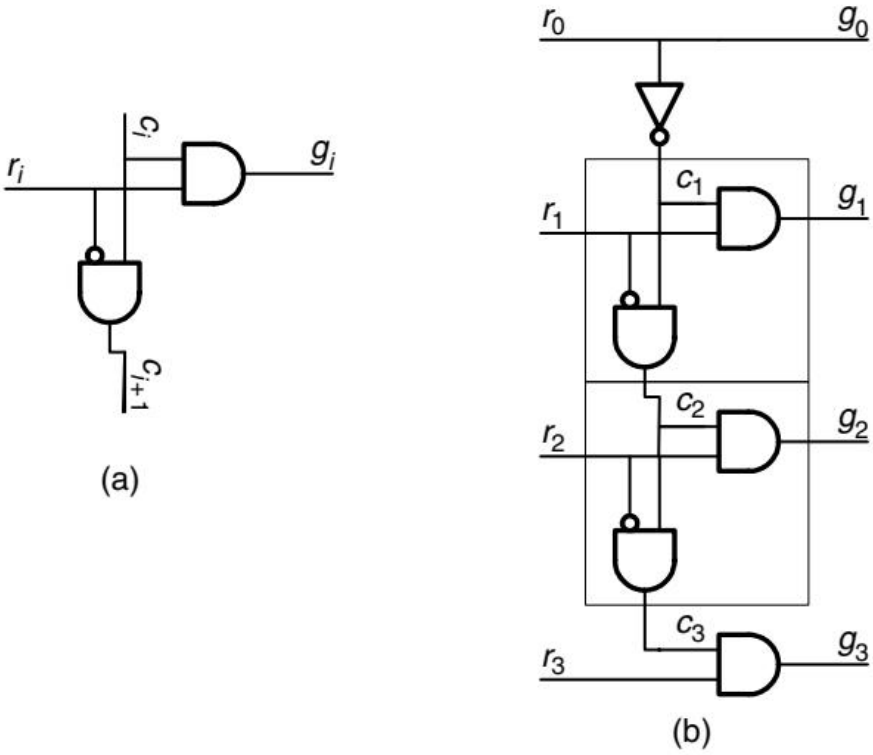


图 18.4 固定优先级仲裁器。（a）迭代仲裁器的位单元。（b）四位固定优先级仲裁器。

我们可以将此逻辑表示为方程形式：

$$g_i = r_i \wedge c_i$$
$$c_{i+1} = \neg r_i \wedge c_i$$

图 18.4（b）显示了以这种方式构造的四位固定优先级仲裁器。[1] 该仲裁器由四个图 18.4（a）的位单元组成。然而，第一个和最后一个位单元已被简化。第一个位单元利用了 c_0 始终为 1 的事实。而最后一个位单元利用了不需要生成 c_4 的事实。

乍一看，迭代仲裁器的延迟似乎必须与输入数量成线性关系。确实，如果按照图 18.4 所示构造，情况就是如此，因为在最坏情况下，进位必须从仲裁器的一端传播到另一端。然而，通过采用进位前瞻技术——类似于加法器中使用的技术——我们可以构建这些仲裁器，使其操作时间随输入数量对数增长。见练习 18.1。

尽管图 18.4 中的仲裁器有助于说明迭代构造，但在实践中并不实用，因为它完全不公平。它甚至不具备弱公平性。如果请求 r_0 持续有效，其他请求将永远得不到服务。

18.3 固定优先级仲裁器

18.4 可变优先级迭代仲裁器

我们可以通过逐周期改变优先级来构建公平的迭代仲裁器，如图 18.5 所示。独热优先级信号 p 用于选择最高优先级的请求。p 中有一位被置位。r 的对应位具有高优先级，并且优先级从该点开始沿环形进位链循环递减。2 该仲裁器的逻辑方程为：

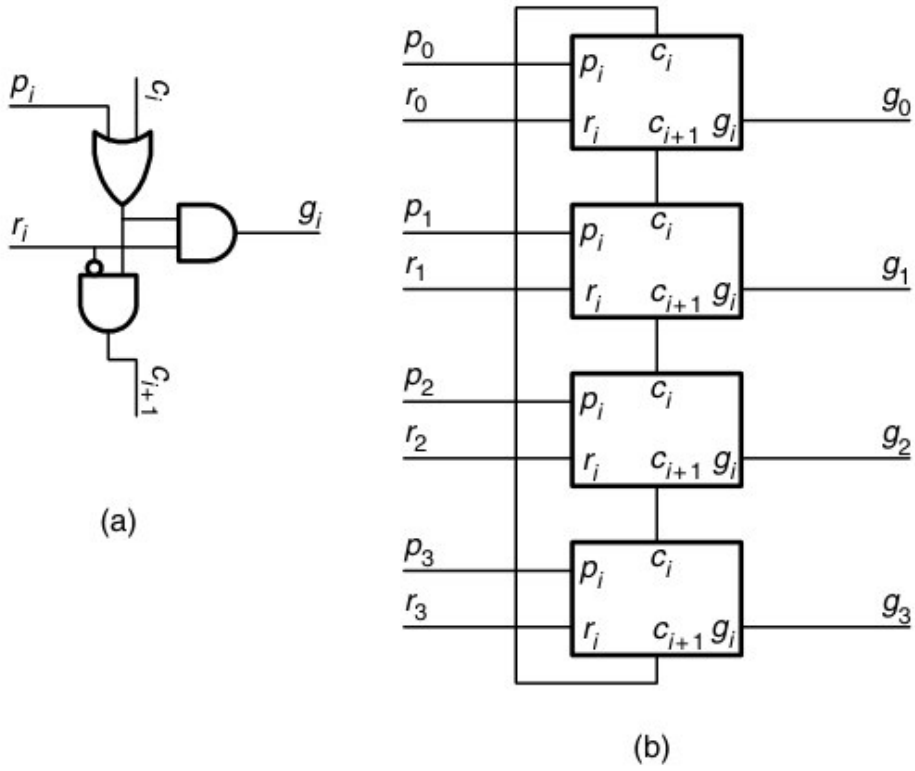


图 18.5 可变优先级迭代仲裁器及其四位实现

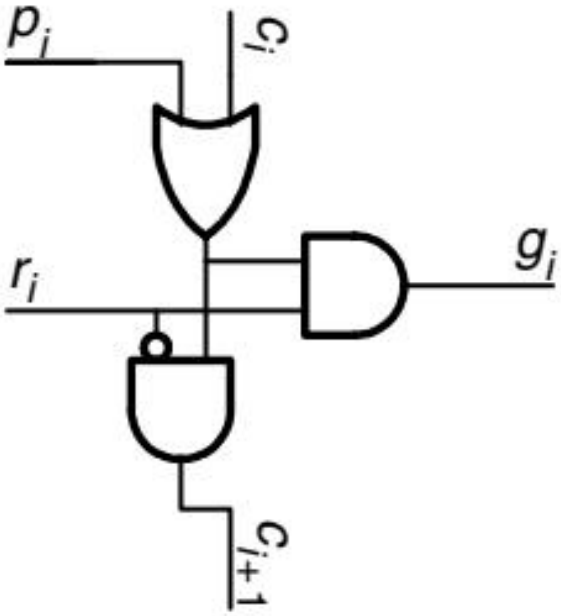
$$
\begin{aligned}
g_i &= r_i \wedge (c_i \vee p_i) \\
c_{i+1} &= \neg r_i \wedge (c_i \vee p_i) \\
c_0 &= c_n
\end{aligned}
$$

18.4.1 oblivious 仲裁器

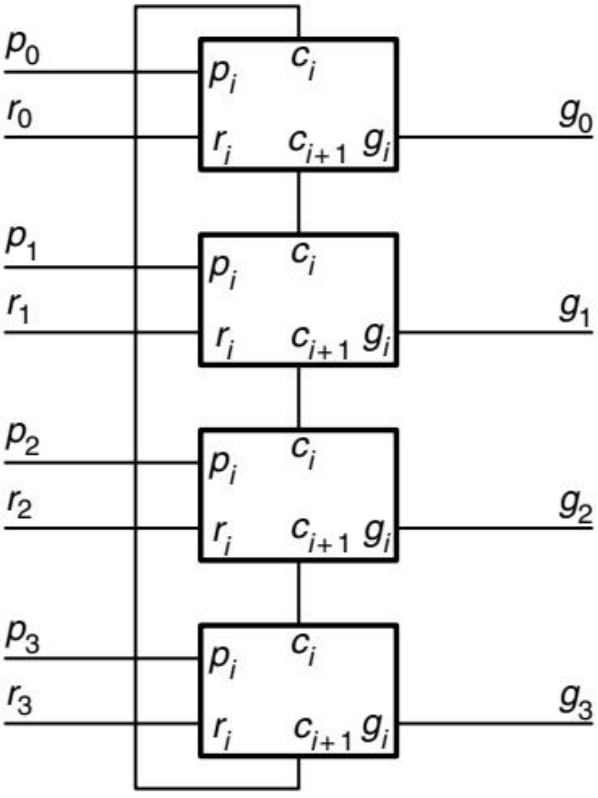
可以采用不同的方案生成图 18.5 电路的优先级输入，从而产生不同的仲裁行为。如果在生成 p 时不使用任何关于 r 或 g 的信息，则得到 oblivious 仲裁器。例如，用移位寄存器生成 p 会使优先级每个周期旋转一个位置，而通过解码随机数发生器的输出来生成 p 则得到每个周期具有不同随机优先级的仲裁器。

图 18.5 可变优先级迭代仲裁器。高优先级请求输入由独热 p 信号选择。（a）可变优先级仲裁器的一位切片。（b）由四个这样的位切片构成的四位仲裁器。

旋转仲裁器和随机仲裁器都具有弱公平性。最终，每个请求都会成为高优先级并获得服务。然而，这些仲裁器不提供强公平性。考虑两个相邻输入 r_i 和 r_{i+1} 反复向一个 n 输入仲裁器请求服务，而所有其他请求输入保持低电平的情况。只有当 p_{i+1} 为真时，



(a)



(b)

请求 r_{i+1} 才赢得仲裁；而在其他 n-1 种可能的优先级输入下，r_i 赢得仲裁。因此，r_i 赢得仲裁的频率是 r_{i+1} 的 n-1 倍。这种不公平性可以通过使用轮询仲裁器来克服。

18.4.2 轮询仲裁器

轮询仲裁器遵循这样的原则：刚刚得到服务的请求在下一轮仲裁中应具有最低优先级。这可以通过从当前授权向量 g 生成下一个优先级向量 p 来实现。在 Verilog 中，该逻辑表示为：

$$\text{assign next_p } = |g?\{g[n-2:0], g[n-1]\} : p;$$

图 18.6 以原理图形式展示了一个四位轮询仲裁器。如果当前周期发出了授权，则 g_i 线中的一条会变高，导致下一个周期 p_{i+1} 变高。这使得紧邻获得授权的请求在下一个周期具有最高优先级，而获得授权的请求具有最低优先级。如果当前周期没有断言任何授权，则 anyg 为低，优先级生成器保持其当前状态。

轮询仲裁器表现出强公平性。请求 r_i 得到服务后，它变为最低优先级。所有其他待处理的请求将在优先级再次轮转到 r_i 之前得到服务。

18.4.3 授权保持电路

到目前为止讨论的仲裁器一次分配资源一个周期。收到授权的客户端只能使用资源一个周期，然后必须再次仲裁以获取资源。这种逐周期仲裁对于许多应用来说是可以的。然而，如第 18.1 节所述，在某些应用中，客户端需要连续多个周期不间断地使用资源。例如，客户端可能需要连续三个周期不间断地访问交换机，以将数据包移入输出缓冲区。

可以通过使用授权保持电路来延长仲裁器中授权的持续时间，如图 18.7 所示。该电路的 Verilog 代码如下所示。如果某一位在上一个周期有授权（last 为真），并且该位的保持线（h）被断言，则该位将继续获得授权（g 保持高），并且仲裁器的输出（gc）被信号 anyhold 抑制。我们已经在图 2.6 中看到了这种授权保持电路的一个略微不同的形式的例子。

18.4 可变优先级迭代仲裁器

图 18.7 带有授权保持电路的仲裁器保持授权信号 g_i，并禁用进一步的仲裁，直到保持线 h_i 释放。

```
assign g = anyhold ? hold : gc ; // n-bit grant vector
assign hold = last & h ; // n-bit hold vector
assign anyhold = |hold ; // hold not zero
always @(posedge clk) last = g ; // last cycle's grant vector
```

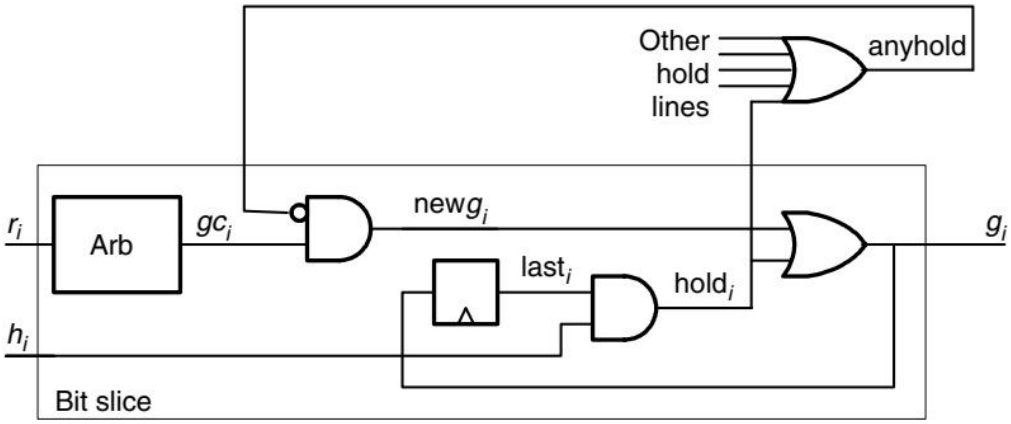
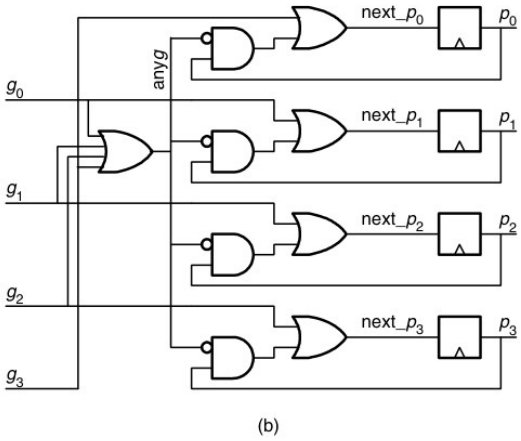


图 18.6 轮询仲裁器使最后获胜的请求在下一轮仲裁中具有最低优先级。当没有请求时，优先级保持不变。(a) 轮询优先级生成器的一位切片。(b) 由四个这样的切片构成的四位优先级生成器。



igure 18.6 A round-robin arbiter makes the last winning request lowest priority for the next round of arbitration. When there are no requests, the priority is unchanged. (a) A one-bit slice of a round-robin priority generator. (b) A four-bit priority generator build from four of these slices.

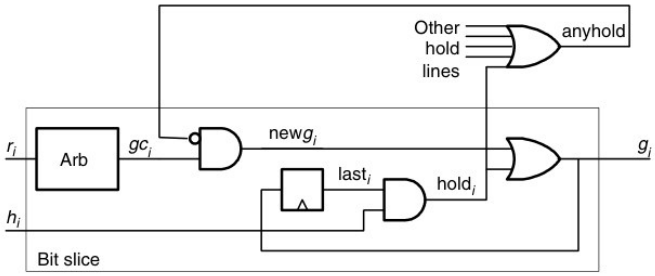
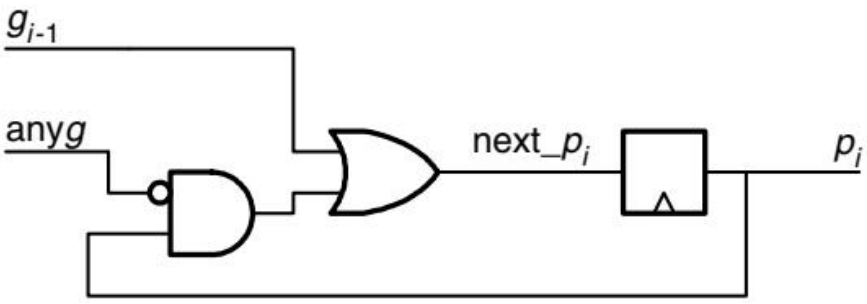
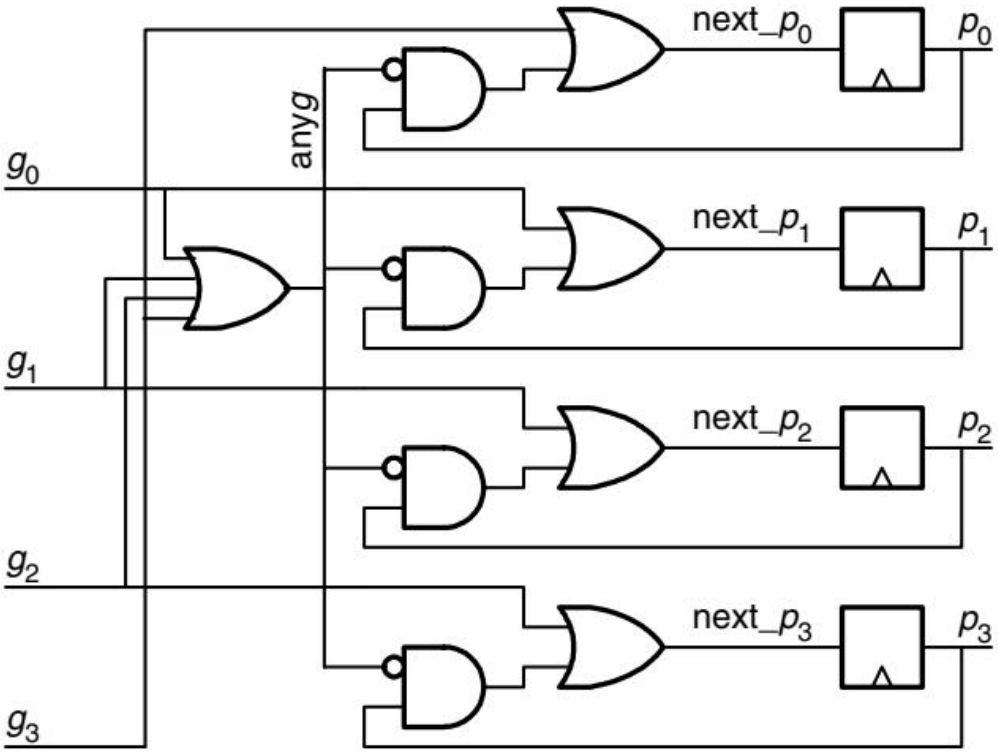


图 18.7 带 grant-hold 电路的仲裁器在保持信号释放前持续授权


g_{i-1}
any$g$
next_p_i
p_i

(a)


next_p_0
p_0
any$g$
g_0
next_p_1
p_1
g_1
next_p_2
p_2
g_2
next_p_3
p_3
g_3

(b)

18.4.4 加权轮询仲裁器

一些应用要求仲裁器在受控程度上不公平，以便一个请求者比另一个请求者获得更多的授权。加权轮询仲裁器表现出这种行为。每个请求者 i 被分配一个权重 w_i，表示请求者 i 应获得的授权的最大比例 f_i，根据 $f_i = \frac{w_i}{W}$，其中 $W = \sum_{j=0}^{n-1} w_j$。权重大的请求者将获得较大比例的授权，而权重小的请求者获得较小比例。例如，假设一个加权轮询仲裁器有四个输入，权重分别为 1、3、5 和 7。这些输入将分别获得 $\frac{1}{16}$、$\frac{3}{16}$、$\frac{5}{16}$ 和 $\frac{7}{16}$ 的授权。

如图 18.8 所示，加权轮询仲裁器可以通过在仲裁器前放置一个电路来实现，该电路禁用已用完配额的输入的请求。该图展示了一个加权轮询仲裁器的一位切片，它由一个权重寄存器、一个计数器、一个与门和一个传统仲裁器组成。当预置线被断言时，每个位切片中的计数器被加载该切片的权重。只要计数器非零，与门就使能，请求被传递到仲裁器。每次请求者获得授权时，计数器递减。当计数器达到零（请求者已获得其份额的授权）时，与门被禁用，来自该请求者的请求被禁用，直到下一次预置。

预置线每隔 W 个周期被激活一次。如果所有输入都请求其配额，则计数器在两次预置之间都会达到零。如果某些输入未使用其配额，则其他输入的计数器将在预置间隔结束前达到零，资源将保持空闲直到预置间隔结束。在某些情况下，使用多级仲裁器来允许已消耗其配额的请求者竞争这些原本空闲的周期。

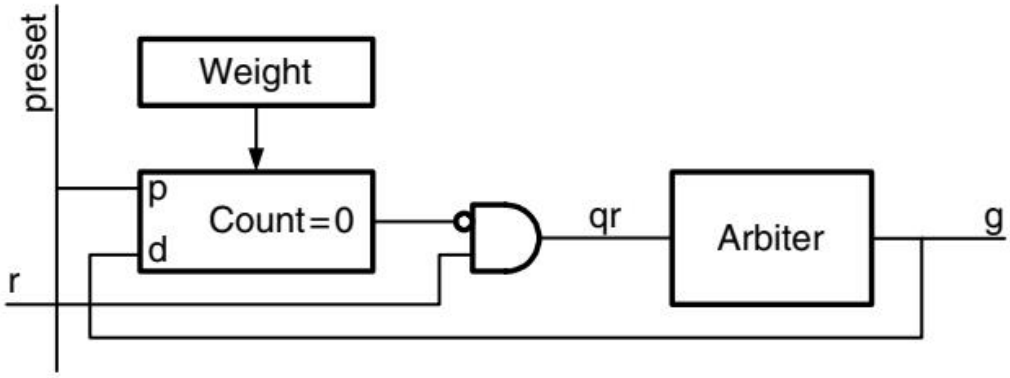


图 18.8 加权轮询仲裁器的一位切片。

选择权重位数需要在精度和突发性之间进行权衡。大量的权重位允许以非常高的精度指定权重。不幸的是，大权重也会导致突发性。请求者仅保证在预置间隔 W 个周期内获得其份额，而增加权重的大小会使该间隔更大。在该间隔内，使用是不均匀的，低权重请求者在间隔开始时获得超过其份额的资源，而高权重请求者在结束时弥补回来。（见练习 18.2。）

18.5 矩阵仲裁器

矩阵仲裁器通过为所有 i < j 维护一个状态位 w_{ij} 的三角阵列来实现最近最少服务优先方案。位于第 i 行和第 j 列的位 w_{ij} 表示请求 i 优先于请求 j。只需要维护矩阵的上三

角部分，因为对角线元素不需要，并且对于所有 $i \neq j$ 有 $w_{ji} = \neg w_{ij}$。每次请求 k 被授权时，它清除其行中的所有位 w_{k*}，并设置其列中的所有位 w_{*k}，以将自己设为最低优先级，因为它是最近被服务的。

图 18.9 显示了一个四输入矩阵仲裁器。状态保存在矩阵上三角部分中由实心框表示的六个触发器中。矩阵下三角部分中的每个阴影框代表对角对称实心框的互补输出。当请求被断言时，它与所在行的状态位进行与运算，以禁用任何较低优先级的请求。一列中所有与门的输出进行或运算，以生成相应请求的禁用信号。例如，如果位 w_{02} 被置位且请求 r0 被断言，则信号 dis2 将被断言以禁用较低优先级的请求 2。如果请求未被禁用，则它通过单个与门传播到相应的授权输出。当授权被断言时，它清除其行中的状态位并设置其列中的状态位，使其请求在后续仲裁中具有最低优先级。状态位的修改发生在时钟的下一个上升沿。见图 18.10。

为了正确操作，矩阵仲裁器必须初始化为合法状态。3 并非阵列的所有状态都有意义，因为对于 n 输入仲裁器，有 $2^{n(n-1)/2}$ 个状态，而输入只有 $n!$ 种排列。导致循环的状态尤其危险，因为它们可能导致死锁。例如，如果 $w_{01} = w_{12} = 1$ 且 $w_{02} = 0$，并且请求 0、1 和 2 都被断言，则这些请求将相互禁用，不会发出任何授权。

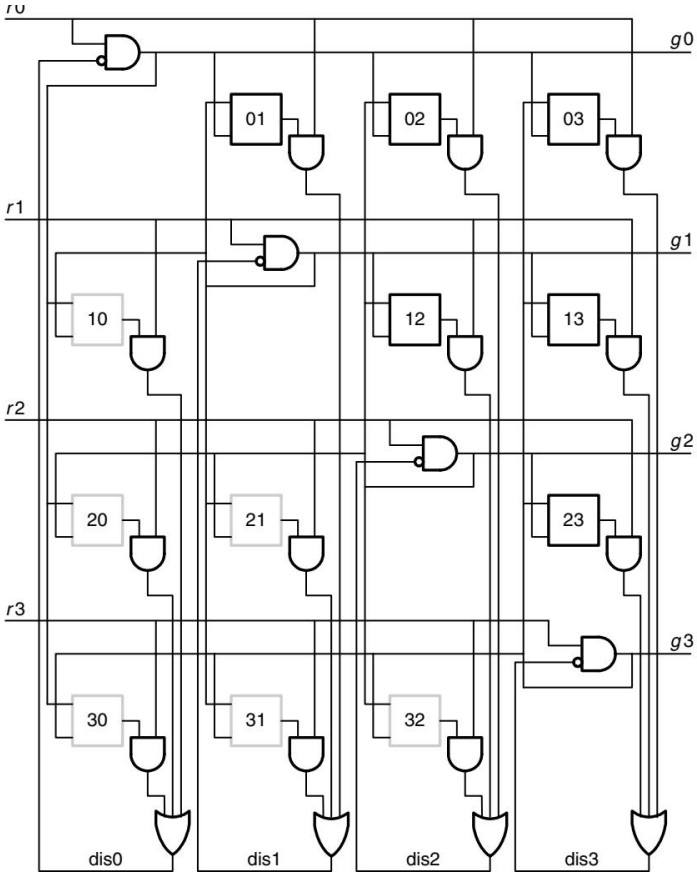


图 18.9 四输入矩阵仲裁器及其用于维护最近最少服务优先级的状态位

图 18.9 矩阵仲裁器维护一个状态位 w_{ij} 的三角阵列，实现最近最少服务的优先级顺序。如果 w_{ij} 为真，则请求 i 优先于请求 j。当请求 k 赢得仲裁时，它清除位 w_{k^*} 并设置位 w_{*k}，使自己成为最低优先级。

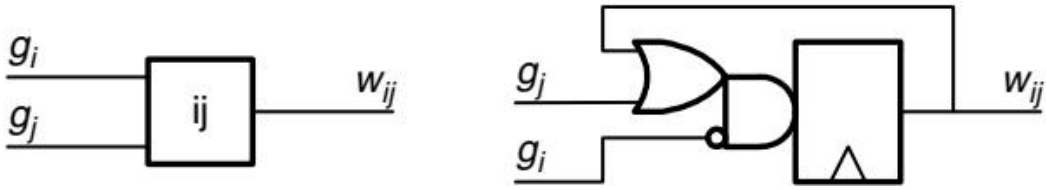


图 18.10 矩阵仲裁器的每个状态位 w_{ij} 由一个触发器组成，该触发器在列授权 g_j 时同步置位，在行授权 g_i 时复位。

18.5 矩阵仲裁器矩阵仲裁器是我们最喜欢的用于少量输入的仲裁器，因为它速度快、实现成本低，并且提供强公平性。

18.6 排队仲裁器

矩阵仲裁器提供最近最少服务优先级，而排队仲裁器提供 FIFO 优先级。如图 18.11 所示，它通过在请求被断言时为每个请求分配一个时间戳来实现这一点。在每个时间步，通过比较器树选择最早的时间戳。

图 18.11 中的定时器块是一个计数器，在每个到达间隔（通常每个时钟周期）递增一个时间戳。当请求到达时 4，当前时间戳被锁存在时间戳块中，并与请求的索引相关联；请求 r_0 的索引为 0，请求 r_1 的索引为 1，依此类推。在时间戳块的输出端，请求由索引-时间戳对 i, t_i 表示。没有请求的输入由一个唯一的时间值表示，该值被认为大于所有可能的时间戳，以及一个不译码为授权的未使用索引。然后比较器树选择获胜索引，即具有最低（最早）时间戳的索引 i_w。树的每个节点执行两个请求之间的成对比较：具有较低时间戳的请求（索引和时间戳）赢得比较并进入树的下一级。最后，输出索引被译码以给出径向授权信号。

仲裁器的成本很大程度上由用于表示时间戳的位数决定，因为这决定了时间戳块中寄存器的大小和比较器树中比较器的宽度。两个时间参数的比率决定了所需的时间戳宽度 w_t、时间戳范围 Δt 和到达间隔 t_a。

$$w_t = \log_2 \frac{\Delta t}{t_a}$$

这两个参数通常以时钟周期为单位表示。Δt 必须足够大，以覆盖请求到达时间的预期最大差异，并且还必须足够大以容纳定时器回绕：

$$\Delta t = 2nT_{\max} \tag{18.1}$$

其中 n 是仲裁器的输入数量，T_{max} 是最大服务时间。因子二在下面描述。

当需要较大的 Δt 时，可以通过增加 t_a 来减小 w_t，但代价是精度降低。仲裁器按照请求到达时间四舍五入到最近的 t_a 的顺序服务请求。t_a 越大，可能乱序服务的请求就越多。

当定时器从其最大值回绕到零时，需要注意防止新请求被分配比旧请求更小的时间

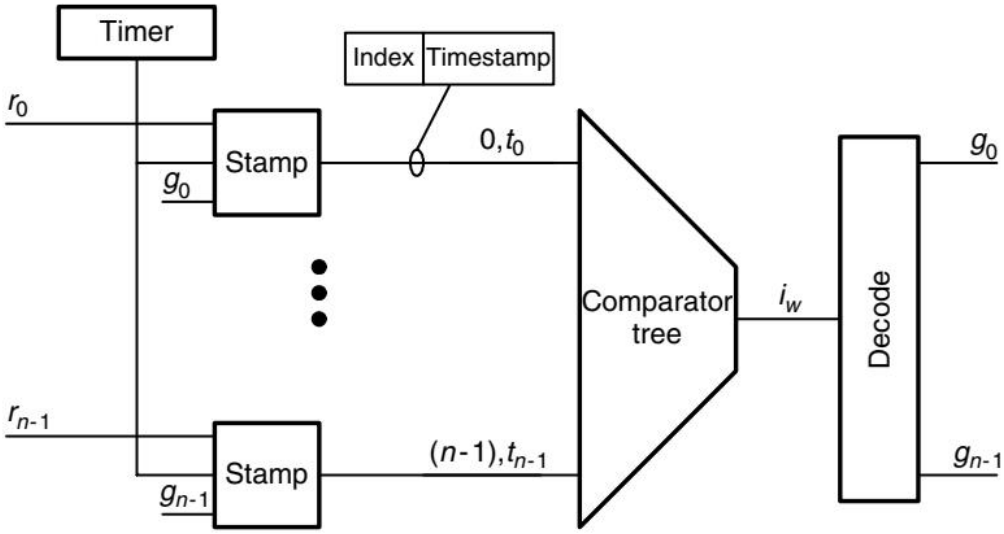


图 18.11 排队仲裁器通过为每个请求分配时间戳并使用比较器树选择最早的时间戳，按到达顺序服务请求。

戳，从而插队到队列头部。

这种不公平行为可以通过在比较之前将所有时间戳的最高有效位与定时器的最高有效位进行同或运算来防止。例如，考虑 $\Delta t = 16$ 的情况，因此我们有一个 $w_t = 4$ 位的定时器，且 $t_a = 1$。请求时间的最大差异为 $nT_{max} = 8$。当定时器从 15 回绕到 0 时，所有时间戳将在 [8，15] 范围内。因此，如果我们对每个时间戳的最高有效位取反，我们将旧时间戳转换到 [0，7] 范围，而新时间戳被赋值为 8。类似地，当计数器从 7 回绕到 8 时，最旧的时间戳值为 0。此时，我们取消时间戳最高有效位的取反，这样将被赋值为 8 的新请求将显得比这些现有请求更年轻。这种方法确实要求 Δt 是请求时间最大差异的两倍，因此公式 18.1 中有因子二。练习 18.7 探讨如何放宽这一要求，使得 Δt 只需略大于 nT_{max}。

在服务时间较小的应用中，范围需求非常适中。例如，考虑一个四输入仲裁器，其中所有服务时间都是一个周期。在这种情况下 $nT_{max} = 4$，并且使用最高有效位取反回绕方法，$\Delta t = 8$。因此，三位时间戳就足够了。如果我们愿意放宽精度使得 $t_a = 2$，我们可以用两位时间戳来应付。通过选择 $t_a = 4$，我们可以将时间戳减少到一位。然而，此时排队仲裁器实现的公平性并不比轮询仲裁器实现的公平性更好。

排队仲裁器在通过多级仲裁确保全局（或系统级）公平性方面特别有用。这个问题在第 18.2 节中讨论并在图 18.3 中说明。为了实现系统级公平性，每个请求（例如，每个数据包）在进入系统时、在排队等待任何仲裁之前被分配一个时间戳。这是通过一个时间戳计数器（在整个系统中同步）和每个系统入口点的一个标记模块来完成的。一旦在入口处被标记，每个请求保持这个初始时间戳并将其用于所有仲裁。它不会在每个仲裁器处被分配新的时间戳。此外，系统中的所有队列必须按时间戳排序，以便最旧的时间戳先被服务。以这种方式使用选择最低全局时间戳的仲裁器的系统将提供强大的系统级公平性。例如，在图 18.3 的情况下，所有请求者将获得总带宽的相同比例

(0.25)。
18.6 排队仲裁器

18.7 练习

18.1 先行进位仲裁器。设计一个 64 输入的固定优先级仲裁器，其运行时间与输入数量的对数成正比。提示：将输入划分为 4 位组，并为每个组计算一个传播信号，如果进入该组的进位将导致离开该组的进位，则该信号为真。也就是说，进位将通过该组传播。然后使用这些 4 位传播信号为 16 位组生成相应的传播信号。
18.2 加权轮询。假设你有一个 4 输入加权轮询仲裁器，权重分别为 5、5、5 和 49，并且所有输入始终在发出请求。描述在这种情况下仲裁器返回的授权时间序列，以及它与理想时间序列的不同之处。可选地，设计一个实现理想时间序列的加权轮询仲裁器。
18.3 矩阵仲裁器的操作。假设你有一个如图 18.9 所示的 4 输入矩阵仲裁器。通过清除所有 6 个状态位来初始化仲裁器。然后在 4 个周期内（每周期一个）断言以下请求向量：$r_3, \ldots, r_0 = 1111, 1111, 1010, 1001$。每个周期断言哪些授权信号，矩阵的最终状态是什么?
18.4 排队仲裁器的设计。你需要设计一个具有 8 个输入的排队仲裁器。任何输入的最大服务时间为 8 个周期。然而，任何 8 个服务时间的总和保证小于 32 个周期。你可以容忍 $t_a = 4$ 个周期的仲裁器定时精度。假设你使用最高有效位取反方案来处理定时器回绕，构造这个仲裁器需要多少位时间戳?
18.5 定时器回绕。描述如何在类似于图 18.11 的排队仲裁器中处理定时器回绕，其中 $\Delta t = nT_{max} + 1$。
18.6 矩阵仲裁器状态中的错误。矩阵仲裁器是否存在任何非法状态（i > j 时的 w_{ij} 值），其中一组请求可能导致多个授权？另外，提出一种简单的方法来检测和纠正仲裁器状态中的任何错误。

注释

- 1. 这与图 2.6 电路中所用的仲裁器类型相同。
- “0”
- 2. 一些时序验证器难以处理这种环形进位链。可以通过复制仲裁器来避免此问题：将第一个仲裁器的进位输入连接到“0”，将第一个仲裁器的进位输出连接到第二个仲裁器的进位输入，并将授权信号进行或运算。
- 3. 让仲裁器具备某种机制，以便在因位错误而进入非法状态时能够恢复，也是一个好主意。
- 4. 请求在请求线从 0 跳变到 1 的周期内到达，或者在收到授权后请求线保持高电平的周期内到达。

第十九章　分配

仲裁器将单个资源分配给一组请求者中的一个，而分配器则在一组资源和一组请求者之间执行匹配，每个请求者可以请求一个或多个资源。例如，考虑一组路由器输入单元，每个单元持有几个发往交换机不同输出端口的数据片。我们已经在第 17.2.2 节中介绍了分配器，在那里我们讨论了它们用于分配交叉开关的用途。在每个周期，交换机分配器必须在输入单元和输出端口之间执行匹配，以便每个输入端口最多选择一个数据片，并且每个输出端口最多选择一个发往它的数据片。

19.1　表示方法

一个 $n \times m$ 分配器是一个接受 n 个 m 位向量作为输入并生成 n 个 m 位向量作为输出的单元，如图 19.1 所示，其中 n = 4，m = 3。当请求输入 r_{ij} 被断言时，请求者 i 想要访问资源 j。每个请求者可以同时请求资源的任意子集。对于路由器设计中使用的分配器，请求者通常对应于交换机输入，资源对应于交换机输出。因此，我们通常将请求者和资源分别称为输入和输出。

分配器考虑请求并生成授权向量，遵循三条规则：

1. $g_{ij} \Rightarrow r_{ij}$，只有当相应的请求被断言时，才能断言授权。
2. $g_{ij} \Rightarrow \neg g_{ik} \forall k \neq j$，每个输入（请求者）最多只能断言一个授权。
3. $g_{ij} \Rightarrow \neg g_{kj} \forall k \neq i$，每个输出（资源）最多只能断言一个授权。

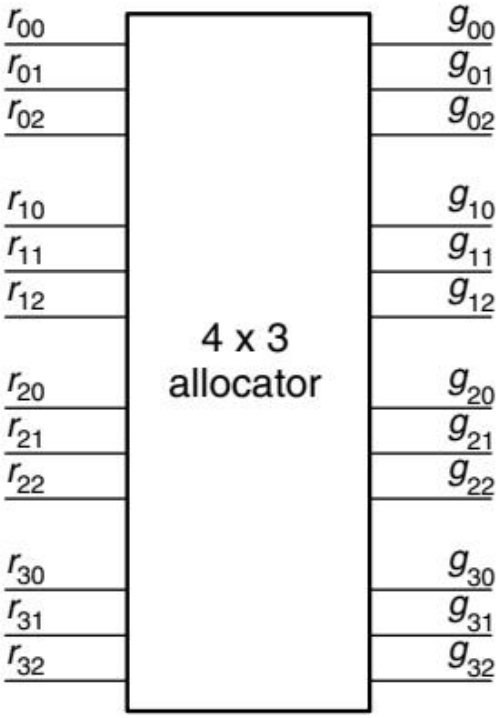
分配器可以被视为接受一个包含各个请求 r_{ij} 的 $n \times m$ 请求矩阵 R，并生成一个包含各个授权 g_{ij} 的授权矩阵 G。R 是一个任意二值矩阵。1 G 也是一个二值矩阵，仅在对应于 R 中非零条目的位置包含 1（规则 1），每行至多有一个 1（规则 2），每列至多有一个 1（规则 3）。

例如，考虑以下请求矩阵 R 和两个可能的授权矩阵 G_1 和 G_2：

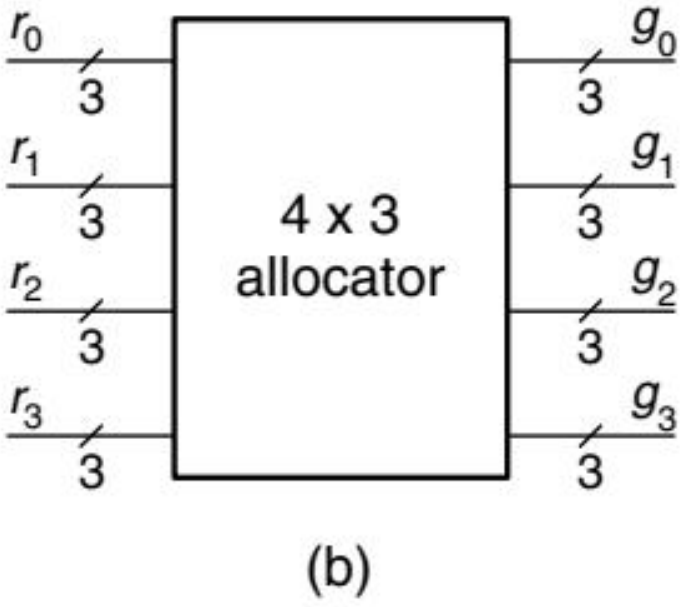
$$R = \begin{bmatrix} 111 \\ 110 \\ 100 \\ 010 \end{bmatrix} \quad G_1 = \begin{bmatrix} 100 \\ 010 \\ 000 \\ 000 \end{bmatrix} \quad G_2 = \begin{bmatrix} 001 \\ 010 \\ 100 \\ 000 \end{bmatrix}$$

对于请求矩阵 R，G_1 和 G_2 都是有效的授权矩阵，因为它们符合上面列出的三条规则。然而，G_2 通常是一个更理想的授权矩阵，因为它将所有三个资源分配给了输入。2 像 G_2 这样包含最大可能数量分配的分配问题解决方案称为最大匹配。像 G_1 这样，在不移除现有授权之一的情况下无法服务额外请求的矩阵称为极大匹配。

第 19 章分配

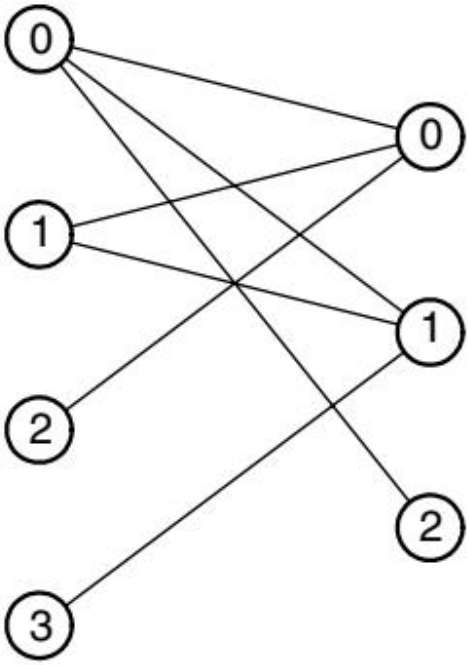


(a)

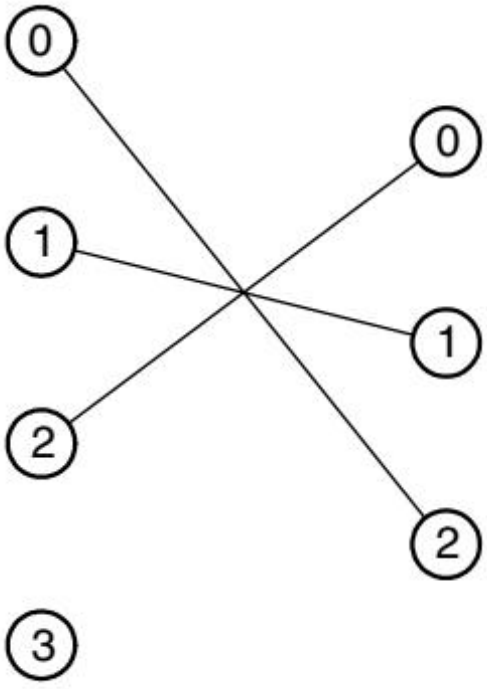


(b)

图 19.1 n×m 分配器的输入请求向量、输出授权向量与两种符号表示



(a)



(b)

图 19.2　分配问题的二分图表示及请求与授权矩阵

矩阵 G_1 说明了计算良好匹配的困难。一旦在矩阵的第一行设置了授权 g_{00}，就不再可能实现最大匹配。因为输入 2 只请求资源 0，将该资源分配给任何其他输入都会导致输入 2 空闲。将资源 1 分配给输入 0 也会阻止最大匹配，因为输入 1 和输入 2 不能同时分配给资源 0。对于困难的请求矩阵，做出一个糟糕的分配可能导致次优匹配。

分配问题也可以表示为二分图。3 例如，图 19.2（a）显示了请求矩阵 R 的二分图表示。以这种形式呈现，分配问题与二分图匹配问题相同：即找到一个最大子集，使得每个顶点在子集中至多与一条边相关联。图 19.2（b）显示了图 19.2（a）的一个最大匹配，对应于授权矩阵 G_2。

对于任何二分图，最大匹配可以在 $O(P^{2.5})$ 时间内计算，其中 P 是端口数 [83]。一般来说，计算最大尺寸匹配需要回溯——授权可以暂时添加，但在算法过程中稍后移除。不幸的是，交换机分配器或虚拟通道分配器无法承受这种精确解决方案的复杂性，并且回溯的必要性使得最大尺寸算法难以流水线化。由于通常只有一个周期，或至多几个周期可用于执行分配，因此最大尺寸匹配成为高速实现更现实的目标。然而，正如我们将看到的，极大匹配也可能相当昂贵，在实践中，硬件分配器通常引入简单的启发式方法，提供快速但近似的解决方案。

19.1 表示方法

19.2 精确算法

尽管在大多数路由器应用可用的时间预算内，精确最大匹配不可行，但有时我们可以在离线情况下计算这样的匹配。此外，在评估新的启发式方法时，我们希望计算最大匹配以与我们的启发式结果进行比较。在这些情况下，我们可以通过迭代增广次优匹配来计算最大匹配。

增广路径算法在精神上类似于用于调度 Clos 网络的循环算法（图 6.11）。首先，我们从二分图的一个次优匹配 M 开始。从这个匹配中，我们构建一个有向剩余图。剩余图具有与表示请求矩阵的二分图相同的边和节点，但边是有方向的：如果一条边在当前匹配 M 中，它从其输出指向其输入（在图 19.3 中从右到左）；否则边从其输入指向其输出。现在，找到一条增广路径，即通过剩余图从非匹配输入到非匹配输出的任何有向路径。然后使用该路径更新匹配：路径中从输入到输出的任何边被添加到匹配中，路径中从输出到相应输入的边从匹配中移除。这给出了一个包含更多一条边的匹配，重复此过程，直到剩余图不再允许增广路径。

为了理解该算法的工作原理，考虑以下请求矩阵：

$$R = \begin{bmatrix} 111100 \\ 010100 \\ 010000 \\ 010111 \\ 000010 \\ 000110 \end{bmatrix}. \tag{19.1}$$

图 19.3 展示了表示该请求矩阵的二部图以及在该图上构造最大匹配的过程。图 19.3（a）显示了带有初始匹配的图，匹配边用粗体表示。已匹配的输入和输出用灰色显示。这个初始匹配恰好是极大的。我们构造相应的残量图，如图 19.3（b）所示。该图的（c）面板展示了如何从一条未分配的输入（2）到一条未分配的输出（0）构造增广路径。该路径经过边（i2，o1）、边（o1，i0）和边（i0，o0）。找到路径后，路径上的边在匹配中改变其分配状态，如图的（d）面板所示：从左到右的边（i2，o1）和（i0，o0）被添加到匹配中，而从右到左的边（o1，i0）被移除。

然后重复此过程。（e）面板展示了如何在（d）面板的配置上找到从输入 5 到输出 5 的增广路径。沿着第二条增广路径更新边的分配，得到（f）面板的匹配。此时，算法结束，因为残量图中不再存在增广路径。根据 Ford 和 Fulkerson[67] 的理论，我们知道如果不存在增广路径，则匹配是最大的。

19.3 可分离分配器

虽然增广路径方法总能找到最大匹配，但它难以并行化或流水线化，并且对于延迟重要的应用来说太慢。延迟敏感的应用通常采用快速启发式方法，这些方法能找到良好的匹配，但不保证最大，有时甚至不是极大的。这种最优性的缺失可以通过在分配的资源中提供额外的加速（第 19.7 节）或通过执行多次迭代（第 19.3.1 节和第 19.3.2 节）来补偿，或两者兼有。

大多数启发式分配器基于基本的可分离分配器。在可分离分配器中，我们将分配作为两组仲裁来执行：一组跨输入，一组跨输出。这种仲裁可以按任一顺序执行。在输入优先的可分离分配器中，首先执行仲裁以在每个输入端口选择一个请求。然后，这些输入仲裁器的输出被输入到一组输出仲裁器，以为每个输出端口选择一个请求。结果是一个合法的匹配，因为每个输入和每个输出至多有一个授权被断言。然而，结果甚至可能不是极大的，更不用说最大了。一个输入请求有可能赢得输入仲裁，锁定对另一个输出的唯一请求，然后输掉输出仲裁。这导致一个输入和一个输出，本可以简单地连接，却都处于空闲状态。

图 19.4 显示了一个 4×3（4 输入 $\times$3 输出）输入优先的可分离分配器。每个输入端口对每个输出有单独的请求线。例如，对于输入 2 的一个 flit 请求输出 1，请求线 r_{21} 被断言。第一级四个三输入仲裁器为每个输入端口选择获胜请求。对于每个输入端口 i，信号 x_{ij} 中只有一个会被断言。这个输入仲裁的结果，即信号 x_{ij}，被转发到一级三个

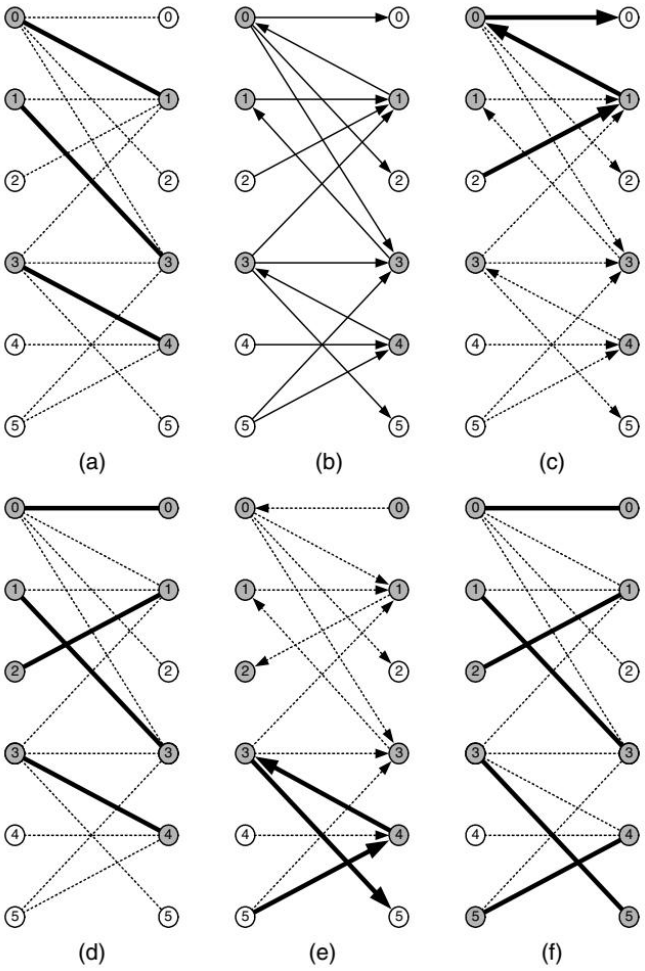


图 19.3 增广路径算法示例。

4 输入输出仲裁器，每个输出一个。输出仲裁器为每个输出端口选择获胜请求并断言授权信号 g_{ij}。输出仲裁器确保每个输出只有一个授权被断言，输入仲裁器确保每个输入只有一个授权被断言。因此，结果是一个合法的匹配。

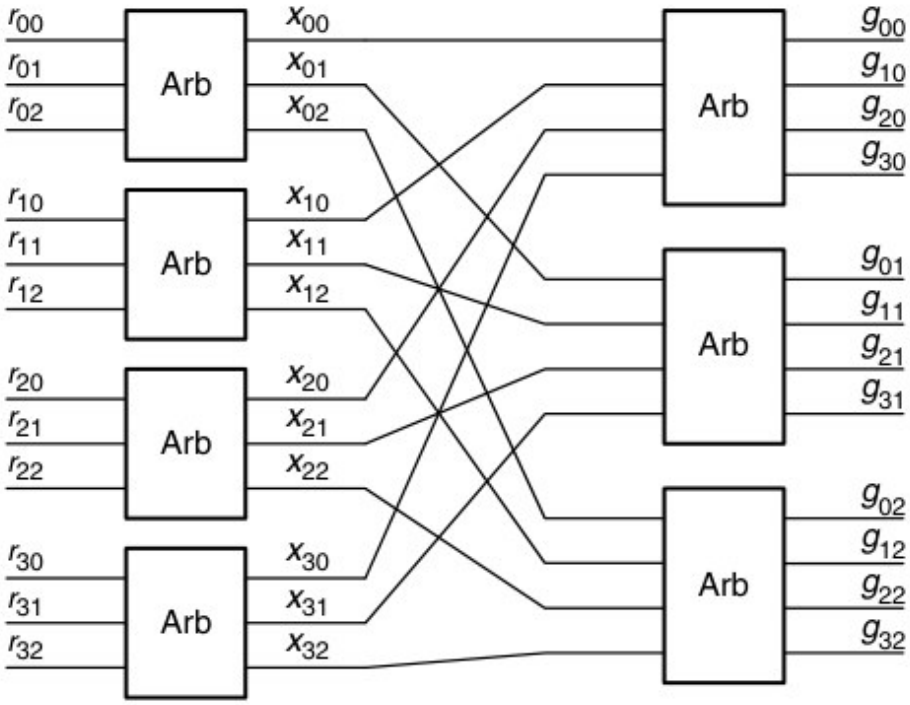


图 19.4 四输入三输出的输入优先可分离分配器

可分离分配器也可以通过先执行输出仲裁再执行输入仲裁来实现。图 19.5 显示了一个 4×3 输出优先的可分离分配器。在这种情况下，第一级三个 4 输入仲裁器为每个输出端口选择获胜请求。对于每个输出端口 j，结果信号 y_{ij} 中只有一个会被断言。然后四个 3 输入输入仲裁器以 y_{ij} 为输入，为每个输入选择获胜请求，并将此结果输出在授权信号 g_{ij} 上，确保对于每个输入 i，g_{ij} 中至多有一个被断言。

输入优先的可分离分配器接受一个请求矩阵，先跨行仲裁，然后跨列仲裁。相反，输出优先的可分离分配器先跨列仲裁，然后跨行仲裁。对于方阵请求矩阵，两者效果相同。对于矩形矩阵，先跨较短维度进行仲裁有优势（例如对我们的 4×3 仲裁器先进行输入仲裁），因为这倾向于将更多请求传播到输出级。因此，对于输入加速比输出加速更大的交换机，通常更高效地执行输入优先分配。

例如，考虑请求矩阵

$$R = \begin{bmatrix} 111 \\ 110 \\ 010 \\ 011 \end{bmatrix}. \tag{19.2}$$

该矩阵的一种可能的输入优先可分离分配如图 19.6 所示。在这个例子中，每个仲裁器选择第一个被断言的输入。因此，输入仲裁器之后的中间请求矩阵 X 为

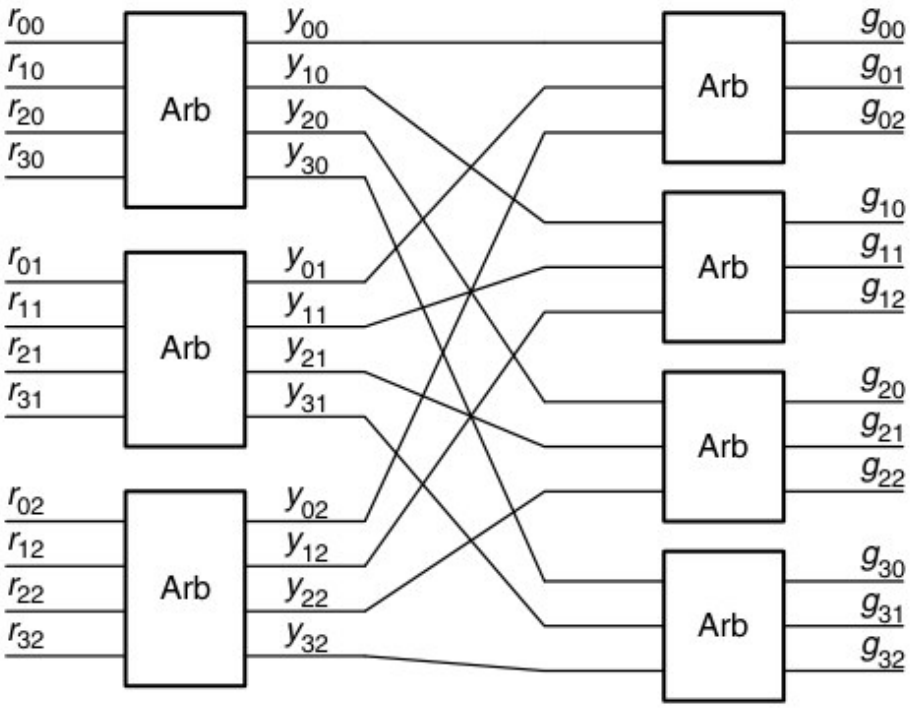


图 19.5 四输入三输出的输出优先可分离分配器

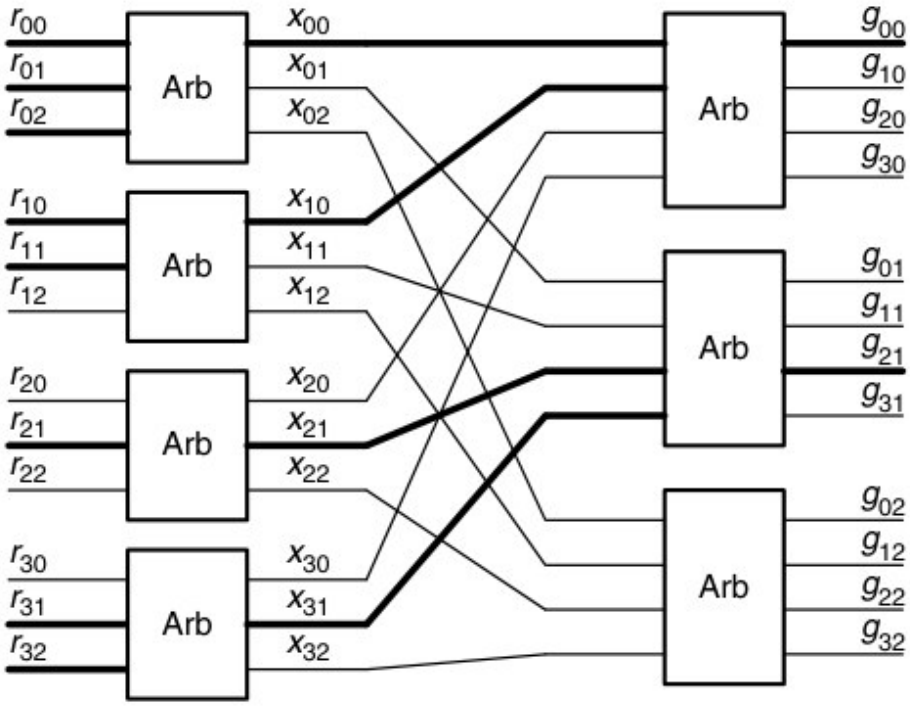


图 19.6 可分离分配产生非最大匹配的实例

$$X = \begin{bmatrix} 100 \\ 100 \\ 010 \\ 010 \end{bmatrix}.$$

注意 X 消除了输入冲突，因此每行至多有一个非零项。然后输出仲裁器消除输出冲突，给出最终授权矩阵 G，每列也至多有一个非零项：

$$G = \begin{bmatrix} 100 \\ 000 \\ 010 \\ 000 \end{bmatrix}.$$

如果第一个或最后一个输入仲裁器选择其最后一个请求而不是第一个，我们本可以分配到所有三个输出。

将相同的请求矩阵（公式 19.2）应用于输出优先分配器，其中每个仲裁器也初始化为选择第一个被断言的输入，得到中间矩阵和授权矩阵，如下所示。

$$Y = \begin{bmatrix} 111 \\ 000 \\ 000 \\ 000 \end{bmatrix}, \quad G = \begin{bmatrix} 100 \\ 000 \\ 000 \\ 000 \end{bmatrix}.$$

这里，只进行了一个授权，因为输出仲裁器的初始化导致所有三个都选择了相同的输入。在实践中，仲裁器如此完全一致的情况很少发生。输入优先与输出优先可分离分配器的影响在练习 19.6 中进一步探讨。

通常采用两种技术来改善可分离分配器所产生的匹配质量。首先，可以错开给定级中不同仲裁器的高优先级输入，以降低多个输入仲裁器都选择发往同一输出的请求的可能性。例如，波前分配器（第 19.4 节）虽然不是可分离分配器，但会将每个输入仲裁器的高优先级输入设置为相对于前一个输入偏移一个位置，并在每个周期轮转该优先级。PIM（第 19.3.1 节）、iSLIP（第 19.3.2 节）和 LOA（第 19.3.3 节）分配器也采用不同方法来错开输入优先级。

其次，如果时间允许，可以通过执行多次迭代来改善可分离分配器的匹配。在每次迭代中，与之前所有迭代中已授予的请求在行或列上冲突的请求将被消除，并对剩余请求执行可分离分配。迭代期间产生的任何授予将与之前迭代的授予累积，生成累积授予矩阵。例如，对上述示例应用第二次迭代将导致除 r_{32} 之外的所有请求被抑制，r_{32} 无干扰地生成 g_{32}。第二次迭代的第二个请求矩阵 R_2、第二个中间矩阵 X_2、第二个授予矩阵 G_2 以及累积授予矩阵分别为

$$R_2 = X_2 = G_2 = \begin{bmatrix} 000 \\ 000 \\ 000 \\ 001 \end{bmatrix}, \quad G = \begin{bmatrix} 100 \\ 000 \\ 010 \\ 001 \end{bmatrix}.$$

19.3.1 并行迭代匹配

并行迭代匹配（PIM）执行随机可分离分配的多次迭代。每次迭代中的每次仲裁都以随机优先级进行。随机化仲裁以概率方式错开输入仲裁器，使得它们不太可能都选择同一个输入。随机化仲裁还消除了模式敏感饥饿的可能性，即某个输入由于确定性优先级调整而被反复锁定。

第 17.2.2 节描述了随机可分离分配的单次迭代，公式 17.1 推导了这种分配器的吞吐量作为交换机加速比的函数。PIM 的收敛性在练习 19.5 中讨论。

19.3.2 iSLIP

iSLIP 是一种可分离分配方法，使用轮询仲裁器，并且仅当仲裁器产生获胜授予时才更新其优先级。iSLIP 既可用于单次通过，也可用作迭代匹配算法。通过轮转获胜仲裁器，iSLIP 错开输入仲裁器的优先级，从而减少输出级的冲突。优先级的更新仅在仲裁产生授予时发生，并且与轮询仲裁器一样，优先级更新使得获胜请求在下一轮具有最低优先级。

作为 iSLIP 操作的示例，假设我们的请求矩阵如公式 19.2 所示，并且输入和输出仲裁器以输入 0 为高优先级开始。即输入优先级向量初始为 $I_0 = \{0, 0, 0, 0\}$，输出优先级向量初始为 $O_0 = \{0, 0, 0\}$。此外，假设分配器使用单次迭代，并且每个周期呈现相同的请求矩阵。

在第一个周期，分配与图 19.6 所示相同。由于只有输入仲裁器 0 和 2 生成了随后产生获胜授予的中间请求，因此只有这两个仲裁器的优先级被推进，得到 $I_1 = \{1, 0, 2, 0\}$。类似地，输出仲裁器 0 和 1 的优先级被推进，得到 $O_1 = \{1, 3, 0\}$。注意，尽管输入仲裁器 1 有获胜者 r_{10} 获胜生成 x_{10}，但该输入仲裁器的优先级未被推进，因为 x_{10} 在输出级失败。输入仲裁器的优先级仅当其获胜者在输出仲裁中获胜时才会被推进。

使用这些新的优先级向量，第二个周期的仲裁结果为：

$$X_1 = \begin{bmatrix} 010 \\ 100 \\ 010 \\ 010 \end{bmatrix}, \quad G_1 = \begin{bmatrix} 000 \\ 100 \\ 000 \\ 010 \end{bmatrix}.$$

此周期后，输入仲裁器 1 和 2 推进，得到 $I_2 = \{1, 1, 2, 2\}$。输出仲裁器 1 和 2 再次推进，得到 $O_2 = \{2, 0, 0\}$。过程继续如下：

$$X_2 = \begin{bmatrix} 010 \\ 010 \\ 010 \\ 001 \end{bmatrix}, \quad G_2 = \begin{bmatrix} 010 \\ 000 \\ 000 \\ 001 \end{bmatrix}, \quad I_3 = \{2,1,2,0\} O_3 = \{2,1,0\}$$

注意，虽然我们的 iSLIP 示例是在输入优先可分离分配器的背景下展示的，但 iSLIP 也可以使用输出优先可分离分配器来实现。

19.3.3 孤独输出分配器

单次通过可分离分配器通常产生较差的匹配，因为许多输入仲裁器都选择同一个热门输出，而少数对孤独输出的请求则不太可能通过输入级。如图 19.7 所示的孤独输出分配器通过在输入仲裁器之前增加一个阶段来克服这个问题，该阶段统计每个输出的请求数量。然后输入仲裁器优先处理请求计数低的输出——即孤独输出——的请求。这减少了输出级的冲突数量，从而产生更好的匹配。

例如，孤独输出分配器在我们的示例请求矩阵上的操作如下所示，以 R、C、X 和 G 矩阵的内容表示。计数（C）矩阵为每个请求标记竞争所请求输出的请求数量。在此示例中，输出 0 和 2 各有两个请求，输出 1 有四个请求。因此，C 矩阵与 R 矩阵相同，但列 0 和 2 中的所有非零元素替换为 2，列 1 中的所有非零元素替换为 4。输入仲裁器每行选择一个请求，优先选择编号较小的请求 4，生成矩阵 X。优先选择低计数请求使得中间请求矩阵 X 在更多列中有条目，从而在输出级产生更少的冲突。例如，这导致输入 3 的仲裁器断言 x_{32}，给输出 2 其唯一的请求。输出级在每个输出中选择一个请求以生成授予矩阵 G。

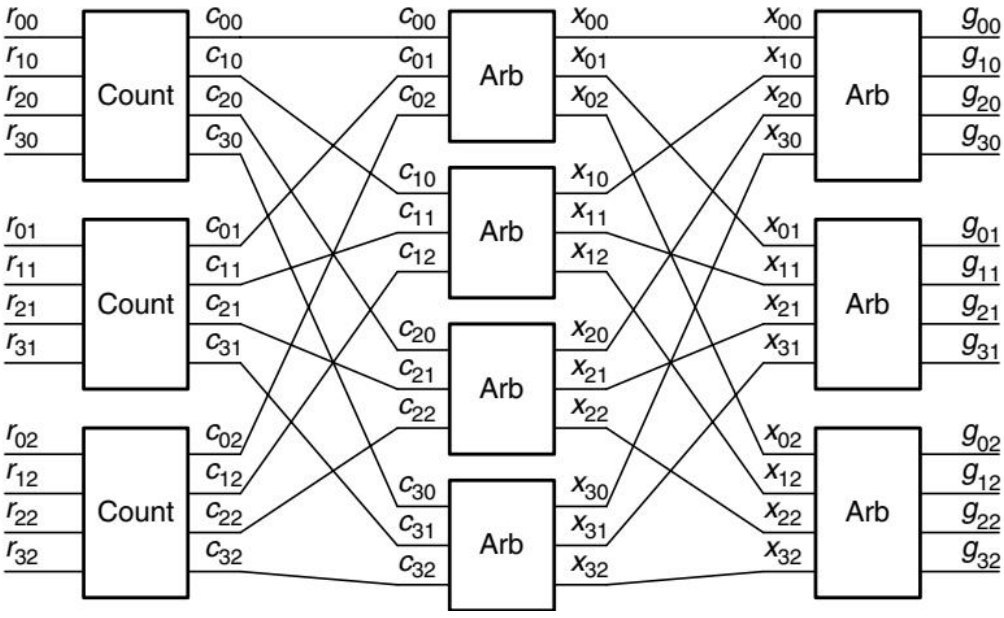


图 19.7 孤独输出分配器。第一阶段统计每个输出的请求数量。然后第二阶段的输入分配器优先处理请求计数最低的输出（孤独输出）的请求。这减少了最后阶段输出仲裁器中的冲突。

$$R = \begin{bmatrix} 111 \\ 110 \\ 010 \\ 011 \end{bmatrix}, \quad C = \begin{bmatrix} 242 \\ 240 \\ 040 \\ 042 \end{bmatrix}, \quad X = \begin{bmatrix} 100 \\ 100 \\ 010 \\ 001 \end{bmatrix}, \quad G = \begin{bmatrix} 100 \\ 000 \\ 010 \\ 001 \end{bmatrix}$$

19.4 波前分配器

与上述可分离分配器不同，波前分配器同时仲裁对输入和输出的请求。波前分配器的结构如图 19.8 所示，每个分配器单元的邏辑如图 19.9 所示。波前分配器通过向一组对角单元授予行令牌和列令牌来工作从而实际上给予该组优先权。一个拥有行（列）令牌但无法使用该令牌的单元将令牌向右（下）传递，在数组末端回绕。这些令牌从优先对角组以波前方式传播，分配器因此得名。如果一个有请求的单元同时收到行令牌和列令牌——无论是由于它属于初始优先组，还是因为令牌在数组中传递——它都会授权该请求并停止令牌的传播。为了提高公平性，接收令牌的对角组每个周期轮换一次。然而，这只能确保弱公平性。（见练习 19.2。）

在一个 $n \times n$ 仲裁器中，对角组 k 包含满足 $(i+j) \mod n = k$ 的单元 x_{ij}。因此，例如，在图 19.8 的 3×3 分配器中，由信号 p_0 选择的优先组 0 由单元 x_{00}、x_{21} 和 x_{12} 组成。因为每个对角组必须恰好包含每一行和每一列中的一个单元，所以所有波前分配器都必须是方形的。然而，可以通过添加虚拟行或列使数组成为方形来实现非方形波前分配器。例如，我们上面示例中的 4×3 分配器可以使用 4×4 数组来实现。分配器中的优先组不必是对角线，只要它们每个都包含每一行和每一列的一个元素即可。（见练习 19.2。）

分配器单元的细节如图 19.9 所示。当单元是优先组的成员时，信号 pri 被置位，通过一组或门生成行令牌 xpri 和列令牌 ypri。如果单元不是优先组的成员，则通过信号 xin 接收行令牌，通过 yin 接收列令牌。如果一个单元有行令牌 xpri、列令牌 ypri 和请求 req_{ij}，它通过一个三输入与门生成授权 grant_{ij}。如果授权被置位，它通过一对与门禁止行令牌和列令牌的进一步传播。否则，如果没有授权，行（列）令牌通过 xout（yout）传递到右边（下边）的下一个单元。

该单元是图 18.5 中迭代仲裁器单元的二维推广。在二维中，令牌或进位的抑制必须由授权（如图 19.9）而不是请求（如图 18.5）来执行，以避免在只接收到两个令牌中的一个时阻塞令牌。与图 18.5 的迭代仲裁器一样，波前仲裁器似乎包含具有组合环路的邏辑。然而，由于其中一个优先输入始终被置位，所有环路都被一个具有置位输入的或门打破。因此，邏辑是无环的。然而，循环结构给许多现代时序分析 CAD 工具带来了问题。

图 19.10 展示了波前分配器在我们示例 4×3 请求矩阵上的操作。使用 4×4 波前分配器来处理这个第 3 列没有请求的非方形问题。在图中，发出请求的单元用浅色阴影表示，获得授权的单元用深色阴影表示。置位的优先权和令牌信号用粗线突出显示。

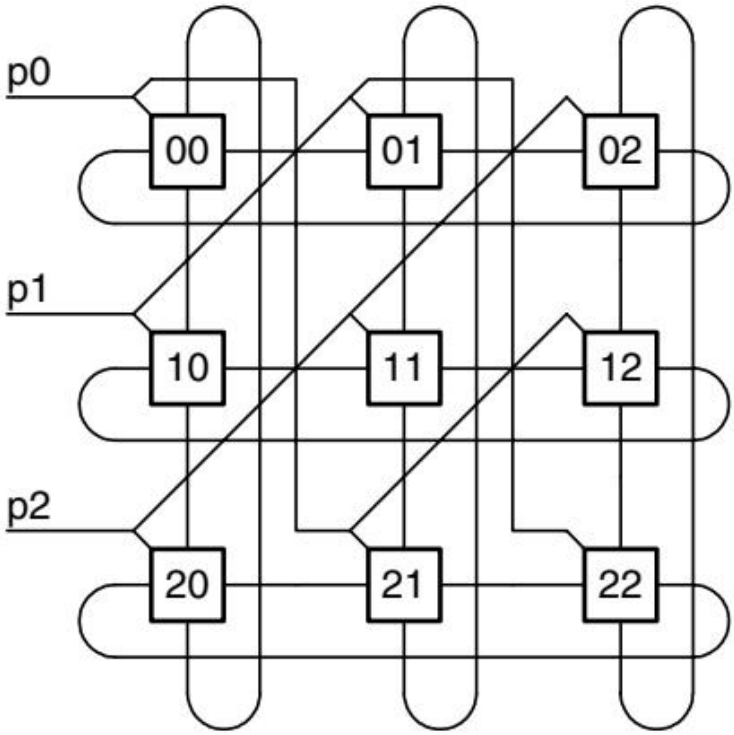


图 19.8 波前分配器通过在分配器单元的二维数组中传递行令牌和列令牌来同时执行输入和输出仲裁。每个周期，行令牌和列令牌被授予其中一个对角单元组（p_0、p_1 或 p_2）。如果一个有请求的单元同时拥有行令牌和列令牌，它就会授权该请求。拥有令牌但无法进行授权的单元将令牌沿其行或列传递。

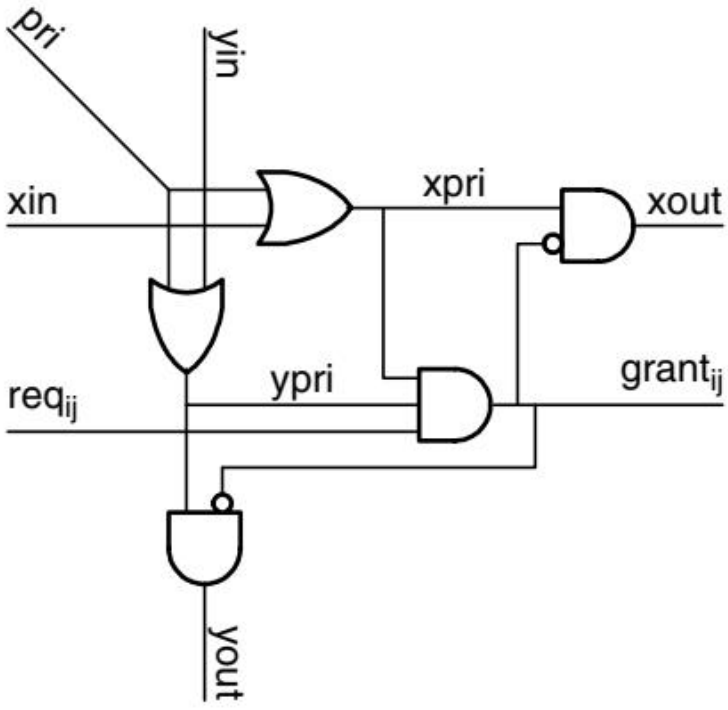


图 19.9 波前分配器单元的逻辑电路与令牌传递机制

在示例中，优先信号 p_3 被置位，向单元 x_{30}、x_{21}、x_{12} 和 x_{03} 授予令牌。在这些单元中，x_{21} 有一个请求，因此使用令牌进行授权。其他三个单元 x_{30}、x_{12} 和 x_{03} 将它们的令牌向右和向下传播。例如，x_{30} 将其行令牌向右传播到 x_{31}。尽管 x_{31} 有一个请求，但它不能使用这个行令牌，因为 x_{21} 已经消耗了第 1 列的列令牌，所以它将令牌传递到 x_{32}。单元 x_{32} 从 x_{30} 接收行令牌，从 x_{12} 接收列令牌，因此能够进行授权。以类似的方式，x_{00} 在从 x_{30} 接收列令牌并从 x_{03} 接收行令牌后进行授权。注意，第 1 行的行令牌和第 3 列的列令牌从未被使用，因此它们一直绕循环传播。

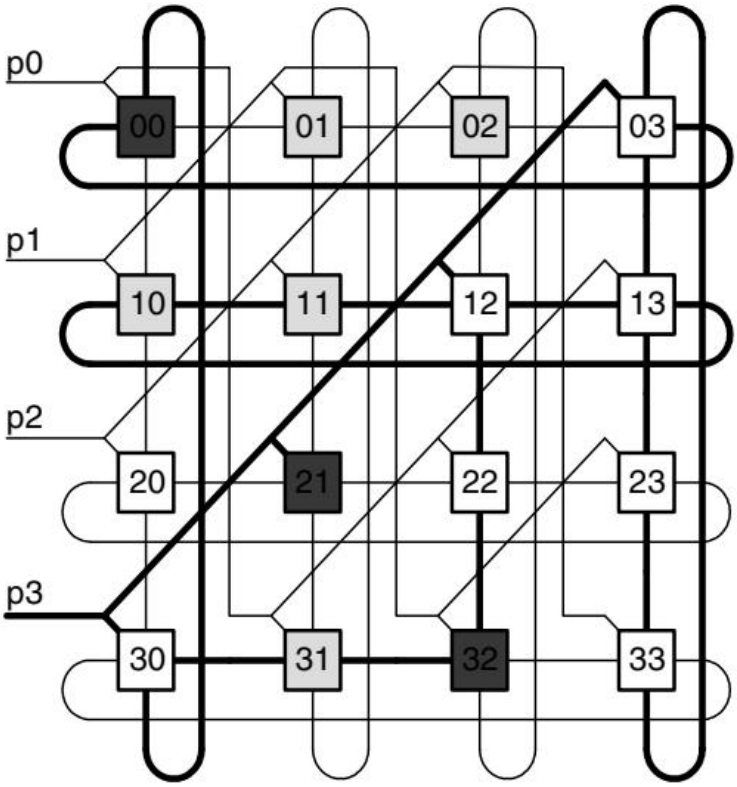


图 19.10 波前分配示例。阴影单元表示请求，深色阴影单元表示被授权的请求。粗线显示置位的优先权和令牌信号。

19.5 增量分配与批量分配

在许多路由器中，每 i 个周期只需进行一次分配，其中 i 为某个整数。例如，如果一个路由器中每个 flit 的宽度是内部数据通路宽度的 4 倍，因此需要 4 个周期才能通过交换开关，那么只需每 i = 4 个周期执行一次分配。这种降低的分配频率可以通过运行 i = 4 次 PIM 或 iSLIP 迭代来利用，从而提高分配器生成的匹配基数。或者，不执行这种批量分配，而是可以增量地进行分配，每个周期完成分配的一部分。

采用增量分配时，使用一个简单的单周期分配器在每个周期生成尽可能最佳的分配。在给定周期获得授权的 flit 立即开始通过交换开关，并且请求矩阵中相应的行和列被移除，直到 flit 完成传输，就像在迭代分配器中一样。与迭代分配器的区别在于，每个

flit 在收到授权后立即开始传输，而不是等待批处理周期结束。由于每个 flit 可能在不同的周期开始传输，它们也可能在不同的周期完成传输。这意味着不同的端口在不同的周期变为可用，而不是像批量分配那样所有端口同时变为可用。这使得提供公平分配变得更加困难。

图 19.11 展示了增量分配和批量分配在公式 19.1 的请求矩阵 R 上的应用。在这个例子中，授权 g_{21} 和 g_{44} 在第 1 个周期被断言，授权 g_{13} 在第 2 个周期被断言，授权 g_{00} 和 g_{35} 在第 3 个周期被断言。使用增量分配器（图 19.11[a]）时，每个 flit 在授权断言后的下一个周期开始通过交换开关。另一方面，使用批量分配（图 19.11[b]）时，所有 flit 的传输都推迟到第 5 个周期，即分配器的所有 4 次迭代完成之后。

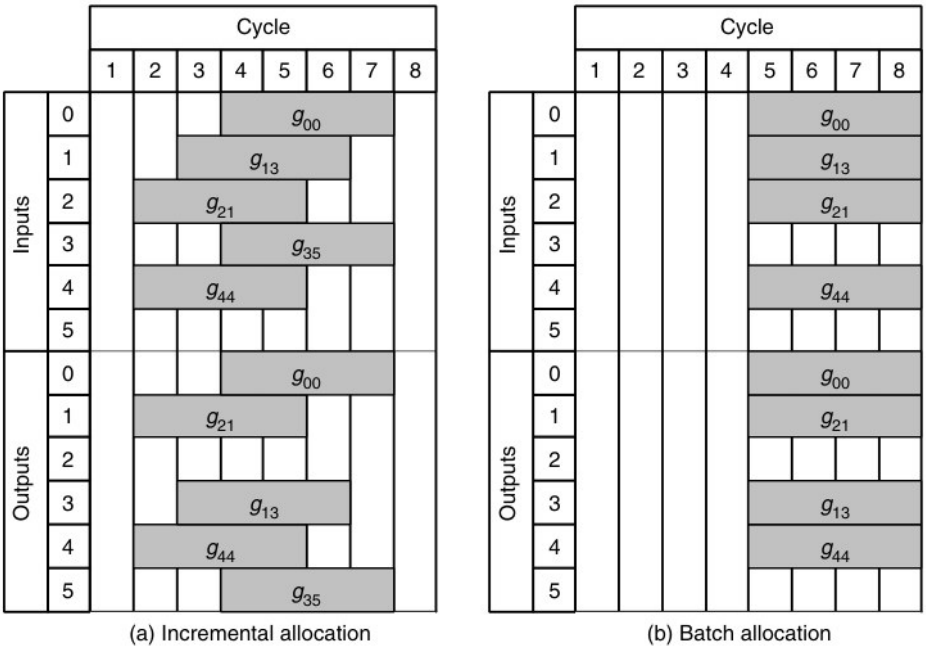


图 19.11 增量分配与批量分配的对比

增量分配降低了延迟，并使分配能够处理未分割成 flit 的可变长度数据包。这些优势是以略微降低的带宽和降低的公平性为代价的。延迟降低是因为每个 flit 可以在收到授权后立即通过交换开关，而不必等待整个批处理时间。在轻负载网络中，大多数 flit 在单个分配周期后离开。带宽略有降低，因为切换 flit 的开始时间会导致一个或多个空闲周期，而批量调度器会填充这些周期。

19.5 增量分配与批量分配

在任意时间开始和结束交换开关遍历的能力也使分配器能够处理可变长度数据包。每个数据包在其整个长度期间持有交换开关资源（输入和输出），然后释放它们。使用批量分配器要做到这一点，需要将每个数据包长度向上舍入到下一个整批大小。虽然直接交换数据包而不是交换 flit 消除了分段开销，但除非限制数据包长度，否则它也会增加竞争延迟，因为竞争数据包可能必须等待最大长度的数据包。一种折衷方案是将大于某个大小 X（例如两个 flit）的数据包分段，并直接交换小于此大小的数据包而不分段。这将分段开销限制在小于 $X/(X+1)$（例如，$\frac{2}{3}$），同时将竞争延迟限制在两

个 flit 时间。

使用增量调度器保证任何类型的公平性都需要特别小心。考虑图 19.11 中的情况，其中 r_{21} 和 r_{00} 持续被断言，并且单个数据包断言了 r_{20}。为了服务 r_{20}，它必须同时获得输入 2 和输出 0。然而，当输入 2 在第 2 个周期变为可用时，输出 0 仍在忙于处理 g_{00}。因此，r_{20} 无法获得输入 2 的授权，而贪婪调度器会反复将输入 2 分配给 r_{21}。类似地，贪婪分配器在第 4 个周期总是将输出 0 分配给 r_{00}。只要使用贪婪分配器，只要 r_{00} 和 r_{21} 被断言并在不同的周期被服务，r_{20} 就永远不会被服务。

在增量分配器中，可以通过允许请求在等待给定时间后独立获取输入端口和输出端口来保证公平性。在上面的例子中，在 r_{20} 等待一段时间（例如 16 个周期）后，它提高优先级以允许独立的端口获取。然后它在第 2 个周期获取输入 2，并保持此输入空闲，直到它在第 4 个周期获取输出 0。

19.6 多级分配

一些应用需要多级分配。例如，在路由器中，我们可能希望首先授权高优先级数据包的请求，然后将任何剩余资源分配给低优先级数据包。在另一个应用中，我们可能希望首先授权多播流量的请求（见练习 19.3），然后将剩余端口分配给单播流量。

这种优先多级分配可以如图 19.12 所示执行。公式 19.3 中展示了一个例子。首先执行高优先级分配——从请求矩阵 Ra 生成授权矩阵 Ga。然后从授权矩阵 Ga 构造掩码矩阵 Ma，掩码所有包含授权的行和列。如果 Ga 的第 i 行或第 j 列中有任何位被置位，则掩码矩阵的位 Ma_{ij} 被设置为零，从而屏蔽下游请求。即，

$$Ma_{ij} = \left(\neg \bigvee_{k=0}^{n-1} Ga_{kj}\right) \bigwedge \left(\neg \bigvee_{k=0}^{m-1} Ga_{jk}\right).$$

然后，该掩码与请求矩阵 Rb 进行按位与运算，以消除与 Ga 中的授权相冲突的请求。剩余的请求 $Rb' = Rb \wedge Ma$ 被输入到第二个分配器，生成授权矩阵 Gb。该过程可以应用于多于两个阶段，每个阶段的请求都被所有上游授权所掩蔽。

19.6 多级分配

$$Ra = \begin{bmatrix} 1010 \\ 0000 \\ 1000 \\ 0000 \end{bmatrix}, \quad Ga = \begin{bmatrix} 0010 \\ 0000 \\ 1000 \\ 0000 \end{bmatrix}, \quad Ma = \begin{bmatrix} 0000 \\ 0101 \\ 0000 \\ 0101 \end{bmatrix},$$

$$Rb = \begin{bmatrix} 1111 \\ 0111 \\ 0011 \\ 0001 \end{bmatrix}, \quad Rb' = \begin{bmatrix} 0000 \\ 0101 \\ 0000 \\ 0001 \end{bmatrix}, \quad Gb = \begin{bmatrix} 0000 \\ 0100 \\ 0000 \\ 0001 \end{bmatrix} \tag{19.3}$$

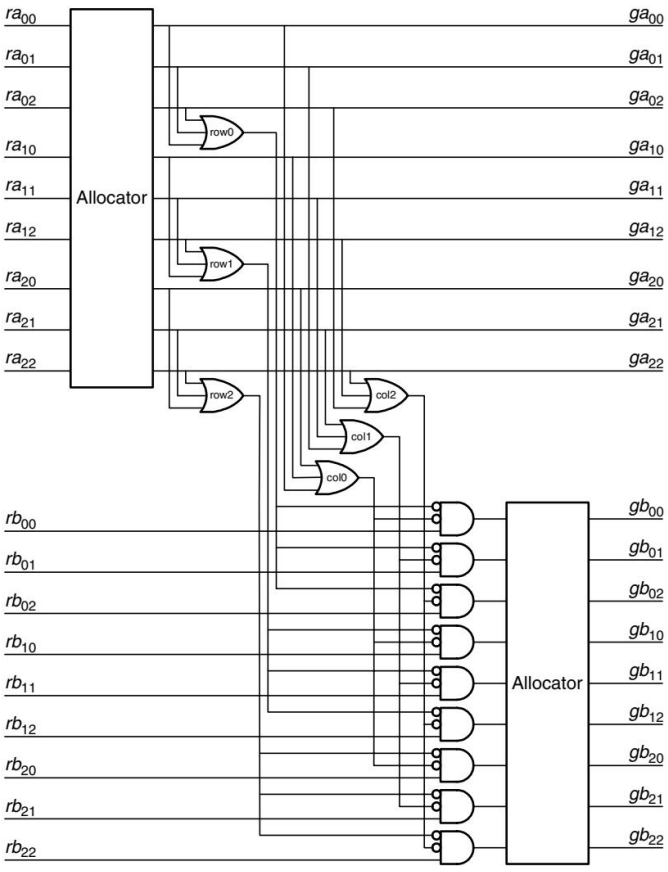


图 19.12　两级分配。对请求矩阵 Ra 执行分配，生成授权矩阵 Ga。此授权矩阵也用于创建掩码——从第二请求矩阵 Rb 中消除 Ga 使用的行和列。然后使用 Rb 的掩码版本执行第二次分配，生成授权矩阵 Gb。由于掩码，Gb 保证不会干扰 Ga。

19.7 分配器的性能

为了衡量分配器的性能，可以使用分配器来调度通过交叉开关的 flit 传输。为了将分配器与其他因素（如有限缓冲区大小）隔离开来，对交换机做出了一些理想假设。首先，交换机的每个输入都馈入一个无限大的缓冲区池。此外，每个输入缓冲区池被划分为虚拟输出队列。虚拟输出队列本质上是一组虚拟通道缓冲区，每个输出对应一个虚拟通道。当一个到达的 flit 到达输入时，它根据其期望的输出端口立即被存储到相应的虚拟输出队列中。这可以防止队头阻塞，即发往交换机一个端口的数据被等待发往另一个端口的数据所阻塞。最后，所有延迟，如路由和交换机遍历，都被忽略。一个 flit 的延迟仅仅是它在输入缓冲区中花费的时间——如果它到达并立即被调度，那么这个时间为零。在以下实验中，假设使用 8×8 交换机和均匀流量。

图 19.13 显示了本章描述的每种分配器的平均延迟与提供流量的关系。PIM 首先饱和，大约在 66% 的提供流量处，LOA 略有改善，饱和点约为 69%。然而，LOA 在其饱和点之前确实提供了良好的平均延迟。波前分配器和 iSLIP 分配器最终都接近 100% 的吞吐量，这超出了图表的延迟刻度范围。此外，如图所示，与更简单的 iSLIP 技术相比，波前分配器还提供了显著更低的延迟。

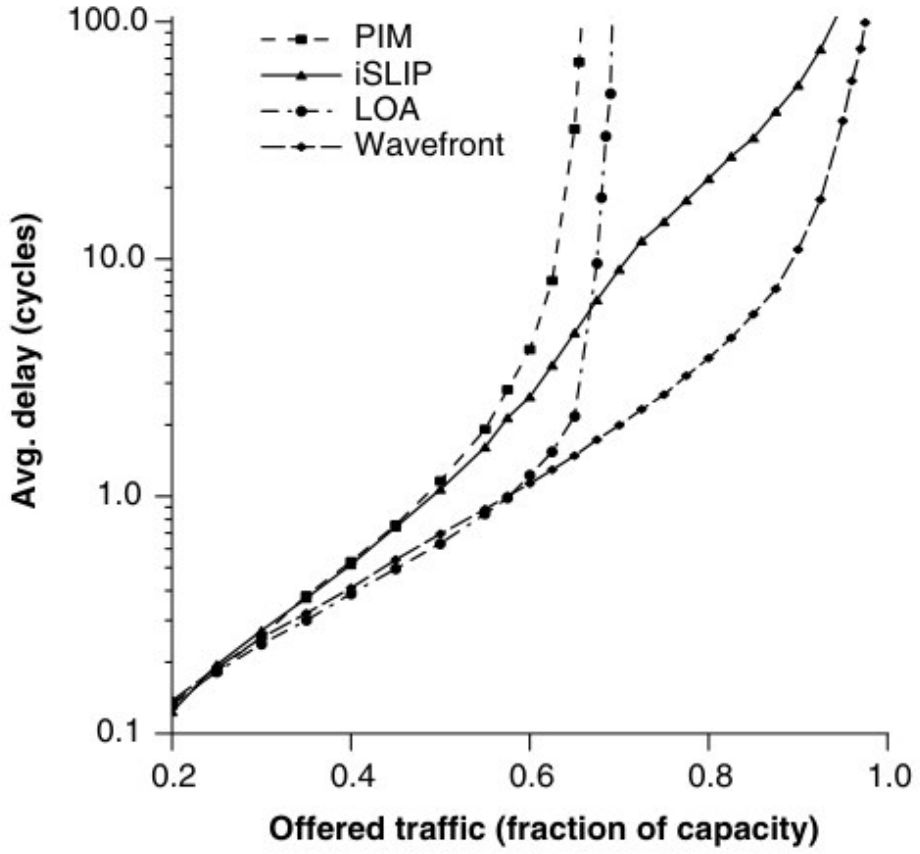


图 19.13 单轮分配器在 8×8 交叉开关上的平均延迟与提供负载关系

虽然这些单迭代、可分离的技术在低复杂度下具有合理的性能，但额外的迭代可以显著提高这种性能。图 19.14 显示了 PIM 在多次迭代下的性能。单次迭代的 PIM1 在

66% 处饱和，而两次迭代将饱和点扩展到接近 90%，三次迭代后吞吐量接近 100%。对于此配置，超过三次迭代后性能提升很小。未显示的是 iSLIP 额外迭代的好处。尽管 iSLIP 在均匀流量下通过单次迭代实现了 100% 的吞吐量，但额外的一次迭代大大降低了延迟，使其与波前分配器具有竞争力。在这种情况下，将 iSLIP 的迭代次数增加到两次以上只能获得很小的改进。

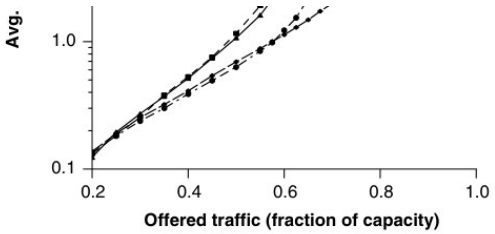


Figure 19.13 Performance of single iteration allocators. Each curve shows the average delay versus offered traffic for an *8* × *8* crossbar under uniform traffic.

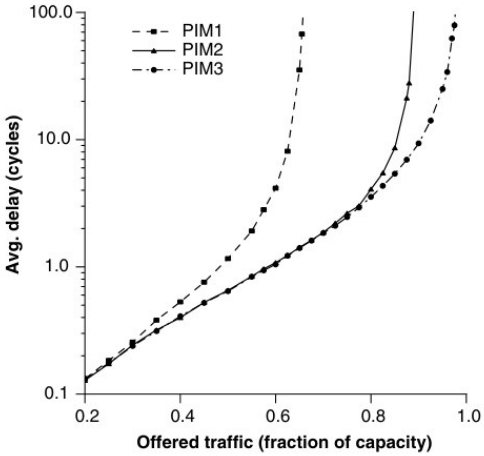


图 19.14 PIM 分配器迭代次数对 8×8 交叉开关性能的影响

19.7 分配器的性能

如第 17.2.2 节所述，提高分配器性能的另一种方法是为相应的交换机提供输入加速、输出加速或两者兼有。图 19.15 显示了具有不同输入和输出加速的 LOA。与无加速（IS=1，OS=1）的情况（饱和点约为 69%）相比，输入加速为 2（IS=2，OS=1）改善了 LOA 的饱和点（接近 95%）和延迟。在输入加速的基础上增加输出加速（IS=2，OS=2）将交换机吞吐量提高到 100%。

我们还可以相对于通道速率加速交叉开关和分配器，以提高简单分配器的性能：对于以通道速率到达的每个数据包，S 个数据包可以穿过交叉开关，其中 S 是加速比。在图 19.16 中，显示了具有加速的 LOA 的性能。相对于通道速率 25% 的加速（S=1.25）使饱和吞吐量增加约 25%，达到约 85%。这一趋势在 50% 的加速（S=1.5）下继续，吞吐量接近 98%。超过这一点（S=1.75），额外的加速在吞吐量或延迟方面几乎不提供任何增益。

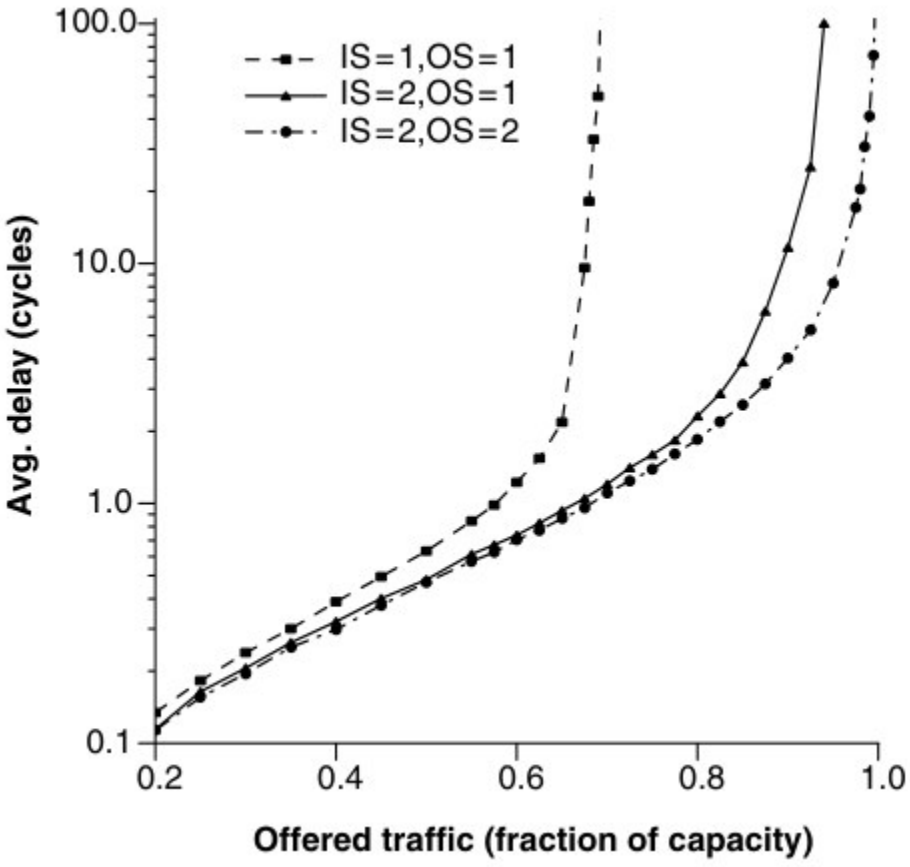


图 19.15　均匀流量下 LOA 分配器的性能

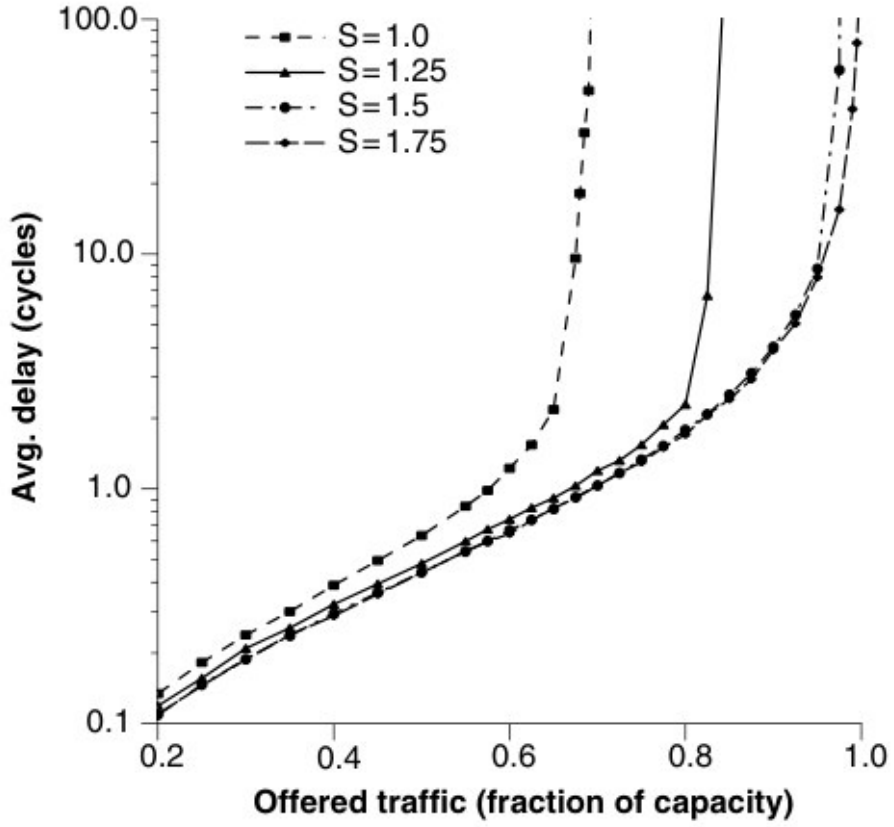


图 19.16 输出孤立分配器（LOA）在不同加速比下的性能

19.8 案例研究：Tiny Tera 分配器

如第 7.4 节所述，Tiny Tera 是一种快速分组交换机，最初在斯坦福大学设计，后来由 Abrizio 商业化 [125]。Tiny Tera 的核心围绕一个 32 端口的交叉开关组织，必须每 51 纳秒重新配置一次。设计者选择了具有输出优先仲裁和三次迭代的 iSLIP 分配算法（第 19.3.2 节），这需要流水线和激进的仲裁器设计来满足他们的时序目标 [79]。

当执行 iSLIP 算法的多次迭代时，似乎可分离分配器中的两个仲裁阶段都必须在开始下一次迭代之前完成。然后，迭代的结果可以用于掩蔽（停用）下一次迭代中与已匹配的输入或输出相对应的任何请求。使用这种方法，每次迭代将需要两个仲裁步骤的时间。然而，一个巧妙的优化可以将迭代时间几乎减半。Tiny Tera 中使用的优化基于以下观察：如果分配器第一级中的任何输出仲裁器选择了特定的输入，则该输入将在第二级仲裁器之后被匹配。因此，可以通过对第一级仲裁的结果进行适当的或运算来计算输入掩码。流水线架构以及用于计算输入掩码 im_i 的逻辑

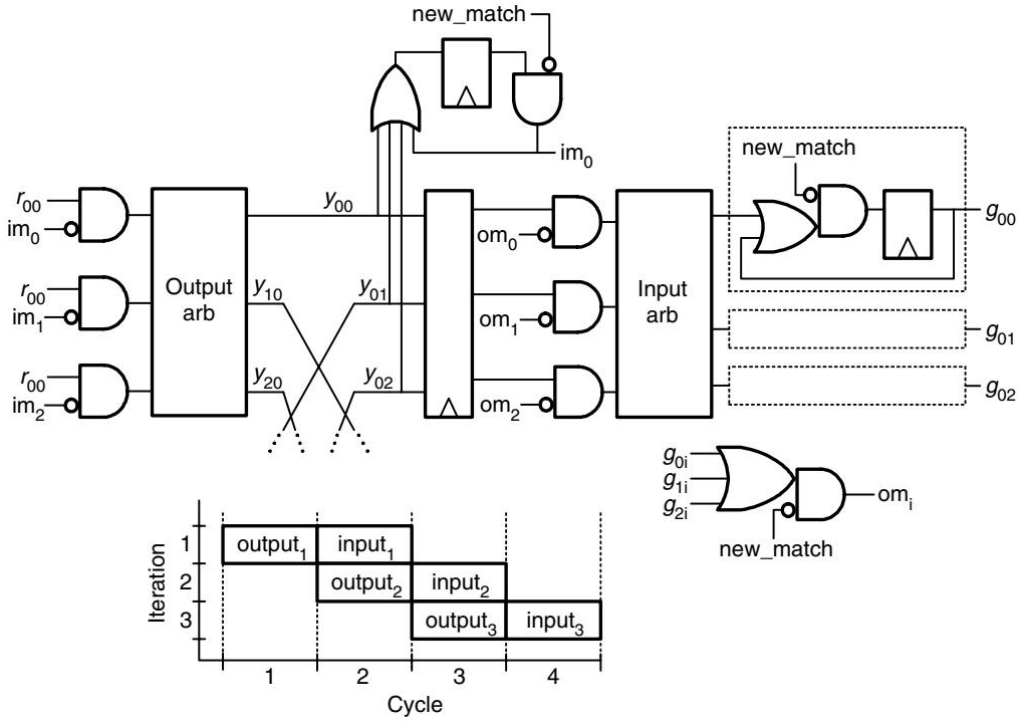


图 19.17 用于 Tiny Tera 分组交换机中多次迭代 iSLIP 分配器的流水线实现的逻辑。为了清晰起见，使用了一个 3×3 分配器，仅显示了其中一部分。`new_match` 信号在新分配的第一次周期中断言。还显示了分配器操作的流水线图——输出的仲裁与来自前一次迭代的输入的仲裁并行进行。

图 19.17 展示了 3×3 iSLIP 分配器的流水线实现逻辑。根据第一级的结果生成输入掩码，可将迭代时间缩短为单次仲裁加上 OR 操作所需的时间，后者通常远小于仲裁时间，对于 Tiny Tera 中使用的大型仲裁器尤其如此。分配器的第二流水线级包含用于输入的仲裁器，这些仲裁器产生授权信号。这些授权信号同样经过 OR 运算以生成输出掩码 om_i。然而，输出掩码并非作用于原始请求，而是对第二级仲裁器的输入进行掩码。这并不能阻止来自已匹配输出的虚假请求通过第一级分配器传播。但由于输出

掩码在输入仲裁器之前施加，这些虚假请求永远不会影响最终的授权结果。
为了说明分配器流水线中输入掩码和输出掩码的应用，考虑图 19.18 所示的具有 2 次迭代的 2×2 分配。如图所示，所有可能的请求均被置位。在第一次迭代中，两个输出仲裁器都选择了输入 0。该信息用于在后续周期中将输入 0 标记（灰色）为已匹配。在第二个周期中，第一次迭代的输入仲裁将来自输入 0 的请求授权给输出 0。同时，第二次迭代的两个输出仲裁都选择了输入 1。到第三个周期开始时，输出掩码正确地反映了输出 0 已匹配的事实。这使得在计算第二次迭代的授权之前，来自输入 1 到输出 0 的虚假请求能够被掩蔽。

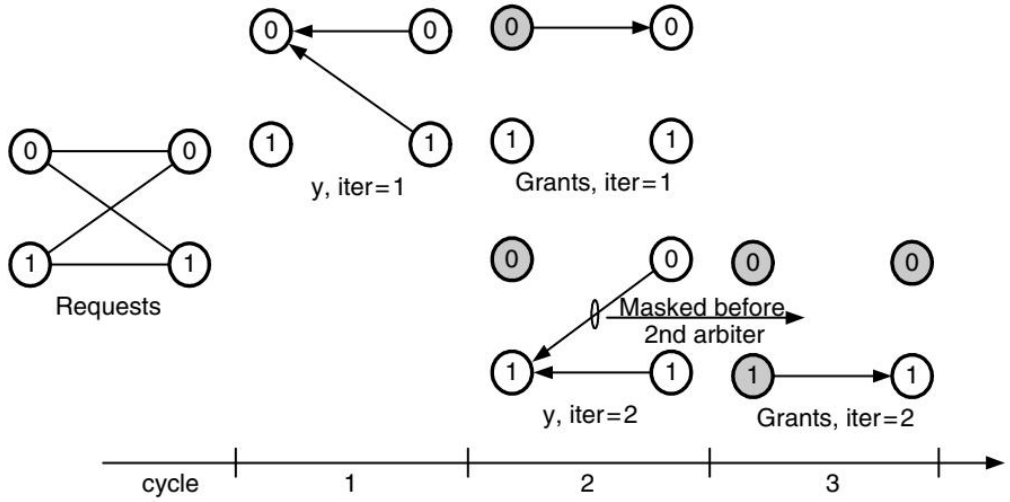


图 19.18 具有 2 次迭代的 2×2 分配的流水线 iSLIP 算法示例。在分配的每个步骤中，被掩码的输入（输出）由左侧（右侧）灰色节点表示。

19.9 文献注释

最早的计算最大匹配的算法由 Hall 以及 Ford 和 Fulkerson 提出。增广路径算法的复杂度为 $O(|V||E|)$，而 Hopcroft 和 Karp 则开发了一种 $O(|V|^{1/2}|E|)$ 时间的算法 [83]。正如 McKeown 等人所示，最大规模匹配并不能保证在非均匀流量下达到 100% 的吞吐率，而更复杂的最大权重匹配则可以 [126]。此后引入了若干技术来近似最大权重匹配，同时仍能实现 100% 的吞吐率 [183，71]。并行迭代匹配最早由 Anderson 等人开发 [11]。iSLIP 最早由 McKeown 研究和描述 [123]，并在 Tiny Tera 分组交换机中实现 [125，79]。Tamir 开发了波前分配器 [180]。增量分配在 Alpha 21364 的路由器中得到了有效利用 [130，131]。

19.10 习题

19.1 可分离分配器的性能。对于以下请求矩阵，找出单趟可分离分配器可能生成的最佳和最差授权矩阵：

$$R = \begin{bmatrix} 111111 \\ 111110 \\ 111100 \\ 100000 \\ 010000 \\ 001000 \end{bmatrix}$$

19.2 波前分配器中的随机化与历史。考虑一个 4×4 波前分配器，其优先级组按图 19.10 连接，请求矩阵为

$$R = \begin{bmatrix} 1001 \\ 0011 \\ 0110 \\ 1100 \end{bmatrix}.$$

（a）如果每个周期递增优先级组 $(p_0, \ldots, p_3, p_0, \ldots)$ 且请求矩阵 R 保持不变，那么授予 R 中对应条目的授权比例是多少？这如何影响使用该分配器的交换机的吞吐率?

（b）与简单递增优先级组的原始策略相比，每个周期随机选择优先级组是否能改善该请求矩阵的性能?

（c）一般来说，波前分配器单元可以按照任何恰好包含每行和每列各一个单元的模式（被优先化的单元必须是一个排列）来设定优先级。如果每个周期使用随机排列来设置单元优先级，那么与前述两种方法相比，分配器在 R 上的性能如何?

19.3 多播分配。解释如何扩展简单的可分离分配器，使其除了单播分配外还能处理多播分配。多播分配器为每个输入端口接受两个额外输入：多播请求 r_m，当端口希望同时分配多个输出时置位；以及多播位向量 b_m，每个输出对应一位，用于指示哪些输出将被分配给该多播。

19.4 增量多播分配。在路由器中，多播请求可以原子方式处理，即坚持一次性分配整个多播，也可以增量方式处理，即将多播集划分为更小的组。在极端情况下，一个多播集可以划分为若干单播——从输入到每个多播输出各一个。解释如何构建一个执行贪婪增量多播分配的路由器：每个周期分配尽可能多的可用输出，然后将剩余输出推迟到后续周期。

19.5 PIM 的收敛性。在均匀流量下，PIM 收敛到极大匹配所需的平均迭代次数是多少?在所有可能的请求模式下，收敛所需的最坏情况迭代次数是多少?

19.6 带输入加速的输入优先与输出优先分配。考虑一个 N 输入 N 输出交换机（其中 S 为输入加速比，N 为端口数）的分配。

（a）如果使用单趟 PIM 分配器且请求均匀（所有输入以等概率请求输出），输入优先分配的吞吐率是多少？如果使用输出优先分配呢?

（b）现在假设端口数 N 很大。S 取什么整数值时，输入优先和输出优先分配器之间的吞吐率差异最大？该差异是多少?

19.7 仿真：比较加速比为 1 的交换机上四次迭代 PIM 分配器与加速比为 2 的交换机上单次迭代 PIM 分配器的性能。
19.10 习题

注释

- 1. 在某些应用中，R 可以是整数值——即可以对同一输出进行多次请求。但是，我们在此不考虑这些应用。
- 2. 实际上，对于非均匀流量，最大尺寸匹配并不总是最佳 [124]。
- 3. 回想一下，在二分图中，顶点可以划分为两个集合 AA 和 BB，所有边连接 AA 中的一个顶点和 BB 中的一个顶点。
- 4. 在本例中，两个具有相同计数的请求（例如，c00c_{00} 和 c02c_{02} 之间）通过优先给予编号较低的请求来打破平局。通常，孤独输出仲裁器使用轮转优先级来打破平局，当仲裁结果产生授权时优先级轮转——如同 iSLIP。

第二十章　网络接口

互连网络与网络客户端之间的接口往往是影响互连网络性能的主要因素。精心设计的网络接口应当不引人注目——使客户端能够以网络本身提供的最低延迟充分利用全部带宽。遗憾的是，许多接口并非如此透明。设计拙劣的接口可能成为吞吐量瓶颈，并大幅增加网络延迟。

本章简要探讨三类网络接口所涉及的问题。处理器-网络接口的设计应能提供从处理器到网络的高带宽通路，同时避免将消息复制到内存的开销，也避免穿越 I/O 接口的瓶颈。成功的接口应能最小化处理器开销并保证安全，防止出错进程禁用网络。

共享内存接口用于通过互连网络将处理器连接到内存控制器。这类接口可以实现简单的远程内存访问，也可以实现复杂的缓存一致性协议。延迟在共享内存接口中至关重要，因为此类接口处于远程内存访问的关键路径上。

线卡接口将外部网络通道与用作交换结构的互连网络连接起来。线卡接口的主要功能是提供排队和分组调度。排队发生在输入线路与交换结构之间，以及交换结构与输出线路之间。通常为每个输出子端口、分组类对提供输入队列，这样发往一个子端口的分组不会阻塞发往另一子端口的分组，低优先级类的分组也不会阻塞高优先级类的分组。与处理器-网络接口一样，此类排队系统的设计也涉及性能和安全两方面的问题。在输出侧，队列将交换结构的速率（通常具有加速比）与输出线路的速率相匹配，并可能对不同流量类别执行速率整形。

20.1　处理器-网络接口

互连网络的许多应用其核心都涉及在连接到网络的处理器之间传递消息。消息传递并行计算机中的网络显然属于这一类，但不那么明显的是，大多数 I/O 网络和许多分组交换结构也涉及在与 I/O 设备或线路接口相关联的处理器之间传递消息。

图 20.1 显示了若干处理器节点 $P_1, \ldots, P_N$，它们通过互连网络交换消息进行通信。在大多数应用中，消息长度呈双峰分布。短消息长度约为 32 字节，用于发送请求和控制信息（读取磁盘扇区 x）。长消息为 1 KB 或更长，用于传输数据块（磁盘扇区 x 的内容）。我们关注短消息和长消息所达到的延迟和吞吐量。在实践中，在短消息上实现良好吞吐量通常是最困难的问题。

良好的消息传递接口必须做到两件事：它必须提供通往网络的低开销通路，并且必须防止行为不当的进程利用网络干扰其他进程。访问路径应具有低延迟，以便短消息可以在几个周期内发送出去。该路径还应避免穿越系统中的低带宽瓶颈点——如内存接口。一般而言，网络接口应避免复制消息，尤其是与内存之间的复制，因为这会增加延迟，并且常常导致带宽瓶颈。

网络接口设计的一个关键方面是它连接到处理器节点的位置。图 20.2 显示了一个典型的处理器节点。处理器芯片包含寄存器堆和片上缓存，通过桥接芯片连接到 DRAM 存储器和 I/O 设备。1 网络接口可以连接到这些点中的任何一个。

最高效的网络接口直接连接到处理器的内部寄存器。这允许直接由处理器寄存器或缓存组成小消息，而无需经历穿越片外内存接口的延迟。遗憾的是，大多数网络接口连接到 I/O 总线，因为这样对现有组件的修改最少。这些接口在互连网络和内存之间传输消息。这会产生相当大的延迟，因为在现代处理器上引发外部总线周期可能需要 30 多个周期。它还导致消息的每个字在通信两端都要两次穿越内存接口。这给内存带宽带来沉重负担，在某些情况下可能成为带宽瓶颈。这种基于 I/O 的网络接口在设计上类似于标准外设接口，此处不再进一步讨论。

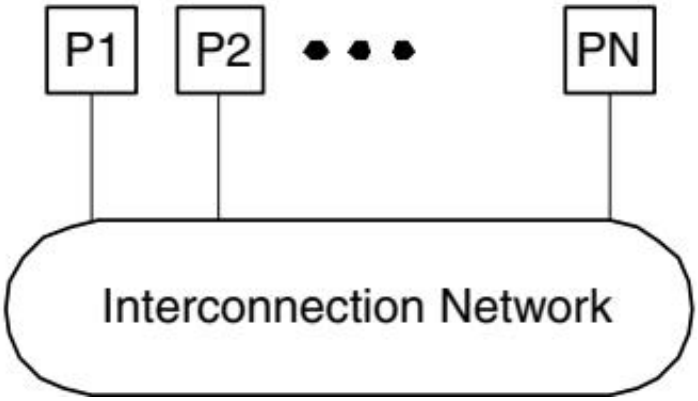


图 20.1 若干处理器 P_1 至 P_N 通过互连网络相互传递消息。

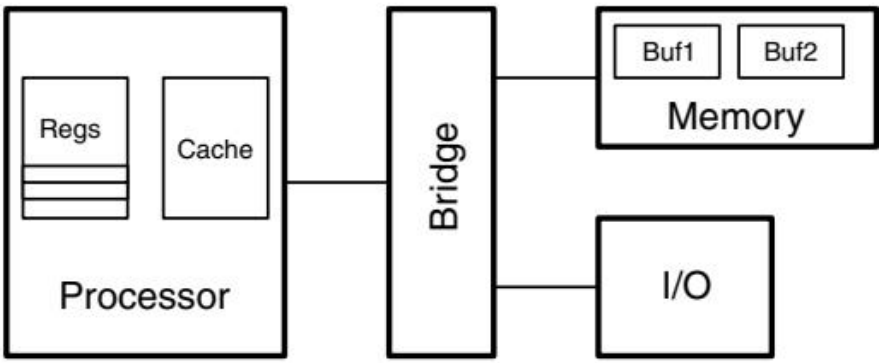


图 20.2 典型的处理器节点由带有内部寄存器和缓存的处理器组成，通过桥接芯片连接到 DRAM 存储器和 I/O 设备。网络接口可以连接到 I/O 总线、与处理器集成，或与桥接芯片集成。

20.1.1 双寄存器接口

图 20.3 展示了一种简单的互连网络双寄存器接口。2 消息通过单个网络输出寄存器发送，传出消息的每个字都写入该寄存器；消息通过单个网络输入寄存器接收，传入消息的每个字都从该寄存器读取。要发送消息，处理器只需将消息的每个字移动到网络输出寄存器。使用特殊的移动指令传输消息的最后一个字并终止消息。要接收消息，

处理器从网络输入寄存器读取传入消息的字。每次读取都会使消息的下一个字出队，因此从同一寄存器进行的下一次读取将返回该寄存器的下一个字。为了与消息到达同步，处理器可以在读取输入寄存器之前测试消息是否存在，或者可以在输入寄存器上阻塞直到消息到达。

双寄存器接口提供了一种简单的接口，能够高效处理短消息，而不会产生任何内存开销。短消息可以直接从处理器寄存器中的数据发送，也可以直接接收到处理器寄存器中。然而，这种接口有两个局限性。首先，对于长消息，尤其是那些必须传输内存驻留数据块的消息，处理器被占用充当 DMA 引擎，在内存和网络接口寄存器之间传输数据。在某些应用中，这种处理器开销可能过高。

20.1 处理器-网络接口

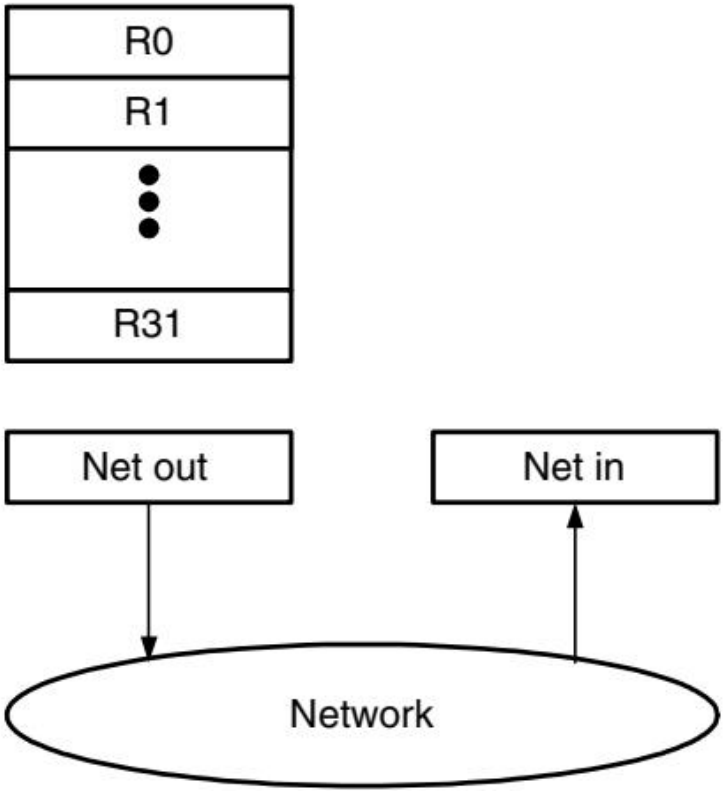


图 20.3 双寄存器网络接口。通过一次一个字地移动到网络输出寄存器来发送消息。通过从网络输入寄存器读取来接收消息。

双寄存器接口的第二个问题是它不能保护网络免受处理器上运行的软件的影响。行为不当的处理器可以发送消息的第一部分，然后无限期延迟发送消息的结尾。部分消息可以无限期占用网络资源，如缓冲区和虚拟通道，干扰其他进程和节点对网络的使用。进程还可以通过不读取输入寄存器中的消息来占用网络，干扰其他进程。

为了使网络接口安全，它必须保证进程不能无限期持有共享网络资源。任何共享资源，例如用于发送消息的缓冲区或虚拟通道，必须在有限时间内释放，无论发送或接收进程的行为如何。可能被无限期持有的资源不应共享。例如，如果进程不能保证在有限时间内释放，则每个物理通道上的虚拟通道可以专用于每个进程。

20.1.2 寄存器映射接口

解决双寄存器接口安全问题的一种方法是从处理器通用寄存器的子集中原子地发送消息，如图 20.4 所示。3 处理器在处理器寄存器中组成消息，然后通过一条 $send$ 指令原子地将消息发送到网络接口，该指令指定包含消息第一个和最后一个字的寄存器。这种消息传输机制是安全的，因为处理器无法在网络中留下部分消息。然而，这种接口非常有限，因为它阻止处理器发送长消息，这迫使长消息被分段，通过消耗通用寄存器造成寄存器压力，4 并且仍然迫使处理器充当 DMA 引擎。

20.1.3 基于描述符的接口

如图 20.5 所示，基于描述符的消息发送机制克服了图 20.4 中寄存器发送机制的局限性。使用这种方法，处理器在一组专用的消息描述符寄存器中组成消息。这个寄存器组足够大，可以容纳一个工作集的消息描述符。每个描述符可以包含要插入消息的立即值、对处理器寄存器的引用或对内存块的引用。图中显示的示例消息包含每种描述符类型各一个。基于描述符的消息发送是安全的，并消除了与寄存器接口相关的处理器开销。实际上，它将处理器开销卸载到一个协处理器，该协处理器逐步执行描述符并组成消息。

20.1.4 消息接收

以安全的方式执行消息接收且没有处理器开销，最容易的方法是将协处理器或多线程处理器的单独线程专用于接收消息。5 消息线程自己处理简单消息，并将更复杂的消息排队供用户线程处理。一些常见的消息类型，例如共享内存引用，甚至可以由专用硬件处理以提高效率。这种接口是安全的，因为接收线程可以被验证为总是在有限时间内从网络中移除消息。

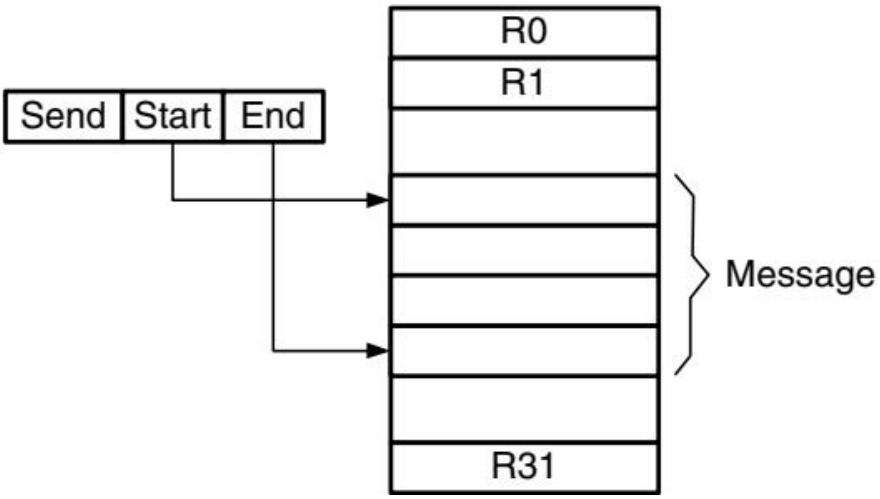


图 20.4 短消息可以直接从处理器的通用寄存器发送。

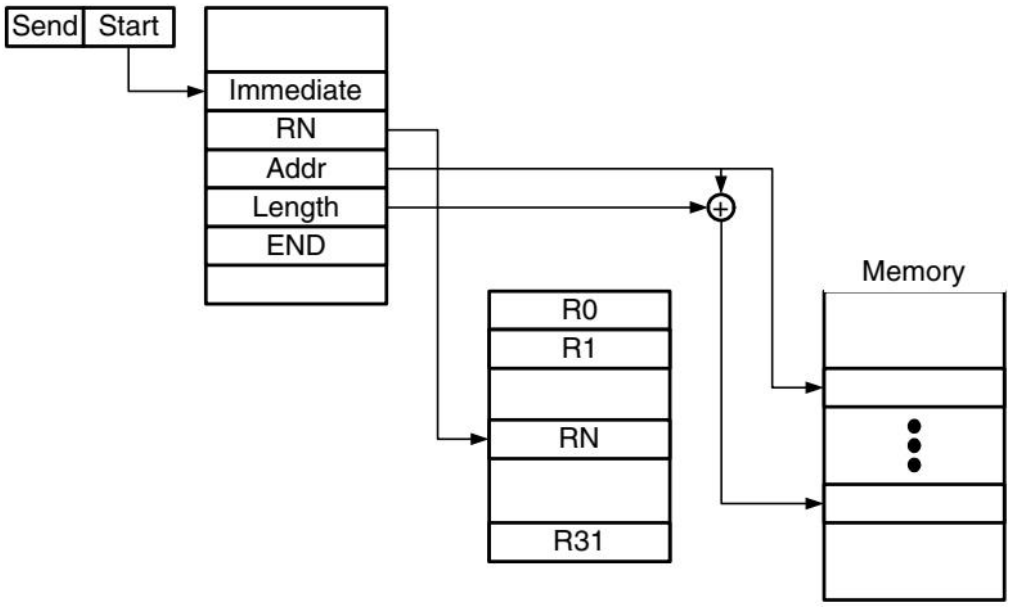


图 20.5 基于描述符的寄存器网络接口。消息在一组包含描述符的专用发送寄存器中组成。每个描述符可以包含立即值、对处理器寄存器的引用或对内存块的引用。

20.2 共享内存接口

在共享内存多处理器（第 1.2.1 节）中，互连网络用于将消息从处理器传送到内存。在不允许远程缓存的系统中，例如 Cray T3E，消息是简单的读写请求和回复。在支持远程数据一致性缓存的系统中，例如 SGI Origin 2000，使用更大的消息词汇表来实现缓存一致性协议。在任一情况下，都使用两个网络接口。处理器-网络接口响应处理器加载和存储操作（这些操作未命中缓存）以及缓存行逐出来格式化消息。在内存控制器侧，内存-网络接口从网络接收请求，执行请求的操作，并发送回复消息。在典型的共享内存处理节点中，这两个接口位于同一位置，并且可以共享网络注入和提取端口。然而，它们的功能在逻辑上是分开的。由于延迟在共享内存多处理器的互连网络中至关重要，这些接口经过优化，可以在短短几个时钟周期内响应处理器或内存事件注入请求消息。

20.2.1 处理器-网络接口

处理器-网络接口的简化框图如图 20.6 所示。每当处理器执行加载或存储操作时，它都会在内存请求寄存器（Req Reg）中放置一个请求。6 请求记录指定了请求的类型（读或写、可缓存或不可缓存等）、要访问的物理地址 7，对于写请求，还指定要写入的数据。每个请求都被标记，以便处理器在从内存系统返回时能够识别相应的回复。在许多系统中，标记编码了如何处理回复（例如，将回复数据存储到寄存器 R62 中）。请求首先被呈现给缓存。如果访问的地址驻留在缓存中，则读或写操作在缓存中执行，缓存将回复（对于读操作，包括请求的数据）放入内存回复寄存器中。

如果请求在缓存中未命中，则将其发布到未命中状态保持寄存器（MSHR）[105]，并

初始化 MSHR 的状态。响应 MSHR 所采取的动作取决于操作类型以及机器是否支持缓存一致性协议。首先，让我们考虑一个简单机器上的读操作，该机器不允许缓存远程数据。在这种情况下，MSHR 状态被初始化为挂起读。看到此状态后，消息传输块格式化一个读请求消息，将其寻址到包含所请求地址的节点，并将其注入网络。消息注入后，请求的状态更新为已请求读。有时需要进行转换步骤，将地址转换为节点号——即将内存地址转换为网络地址。读请求消息包含目标节点、消息类型（读）以及要读取的地址。

网络最终将返回一个读回复消息以响应请求。读回复消息的地址字段用于识别等待所请求数据的 MSHR。所有匹配的 MSHR 都用数据更新，其状态变为读完成。完成的 MSHR 依次被转发到处理器回复寄存器，处理器使用标记字段将数据引导到正确的位置。当每个完成的操作被转发到处理器时，其状态变为空闲，释放 MSHR 以处理另一个请求。

20.2 共享内存接口

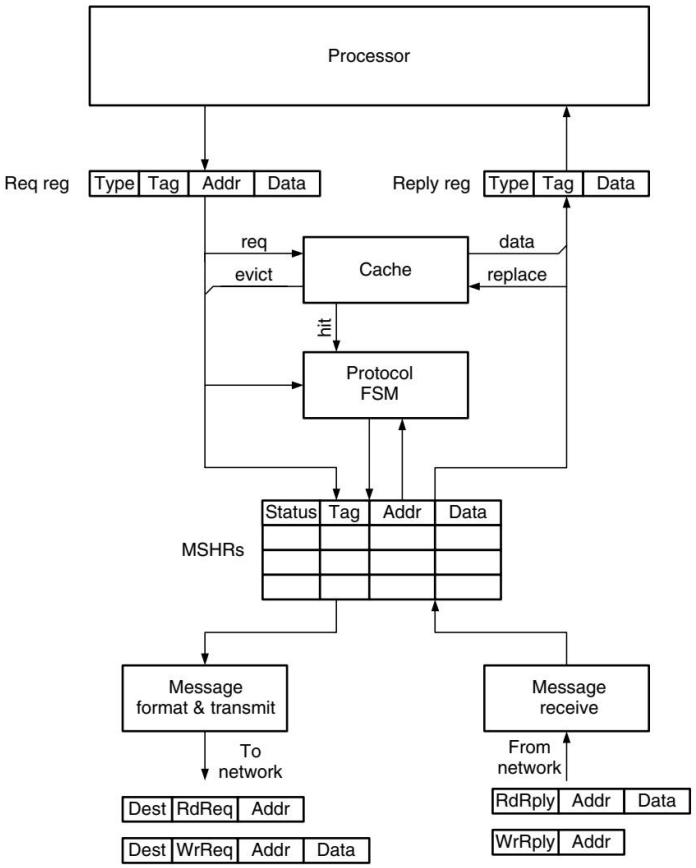


图 20.6 共享内存多处理器的处理器-网络接口。未命中缓存的内存请求被发布到未命中状态保持寄存器（MSHR），并发送请求消息以执行所请求的操作。从网络接收的回复消息与 MSHR 中挂起的操作匹配，完成的操作被转发到处理器。

不可缓存的写操作的处理方式与读操作相同，不同之处在于数据包含在请求消息中，而不是回复消息中。此外，对具有挂起请求的地址进行写操作需要发送第二个写请求消息，并且需要一种机制来保证对同一地址的两个写操作按顺序接收。

MSHR 充当所有未完成请求的记分板。当请求在缓存中未命中时，在 MSHR 中创建一个条目并初始化状态。处理请求的代理（协议 FSM 和消息传输块）监视 MSHR 条目的状态，并在检测到需要动作的条目时，启动适当的动作。这种监视通常通过使用状态字段的独热编码并在反映特定状态的位的逻辑或上触发代理来完成。

MSHR 还用于合并对同一位置的请求。如果对已有挂起读的位置请求第二次读，当第二个请求发布到 MSHR 时将检测到地址匹配，并且不会发送冗余的读请求消息。当收到第一个请求的回复时，它将满足该地址的所有挂起请求。

MSHR 的数量决定了在任何给定时间点可以挂起的内存引用的数量。当所有 MSHR 都满时，下一个未命中缓存的内存引用必须在请求寄存器中停顿，直到 MSHR 可用。典型设计有 4 到 32 个 MSHR。通过消除 MSHR 并将每个请求的整个状态随请求消息一起转发，可以构建处理更多未完成引用的共享内存网络接口。然而，在本节中，我们将讨论限制在基于 MSHR 的接口。

20.2.2 缓存一致性

在支持通过一致性协议缓存远程数据的机器中，操作与上述不可缓存读和写的描述类似，但有三个主要区别。首先，所有操作都以缓存行为单位执行。例如，读请求读取整个缓存行，然后将其存储在本地缓存中。其次，协议需要更大的消息词汇表。例如，使用单独的消息来请求只读状态和读写状态的缓存行。还使用额外的消息来转发和无效化缓存行。最后，一致性协议要求处理器-网络接口不仅响应处理器动作，还要响应接收到的消息而发送消息。

对缓存一致性协议的完整讨论超出了本书的范围。8 不过，一个简单的一致性协议要求处理器发送：读请求消息（在读缺失时发送）、读独占消息（在写缺失时发送以获取一行）、写回消息（发送以将脏行逐出到内存）、转发消息（发送以将脏缓存行转发给新的所有者）以及无效确认（发送以确认干净行已被无效化）。处理器需要处理接收读回复消息（带有只读缓存行）、转发消息（带有读写缓存行）、无效请求（要求将只读行无效化）以及转发请求（要求转发独占缓存行）。接收到这些消息中的每一个都会导致更新现有的 MSHR 条目，或者（对于无效请求或转发请求）创建新的 MSHR 条目。MSHR 条目的状态字段会触发协议 FSM 和消息发送单元执行响应消息所需的任何操作。例如，无效请求要求协议状态机将指定行无效化，并发送回复消息以表明无效化已完成。类似地，转发请求消息要求协议 FSM 将一行无效化，并在转发消息中将其内容转发到指定节点。

携带数据的一致性消息会携带整个缓存行。现代机器上的缓存行大小从 8 字节（Cray X-1 上的一个字）到 512 字节（IBM Power4 上的 L2 行大小）不等，而 128 字节的行大小是典型的。128 字节行的传输通常一次进行一个或两个字（8 到 16 字节），因此需要 8 到 16 个周期。为了最小化延迟，消息注入与从缓存传输缓存行是流水线化的。消息的头部一旦根据 MSHR 条目中的数据格式化完成就被注入，而不是等待整个缓存行被读取。然后，缓存行的每个字在被读取时被注入。此外，为了减少延迟，许多

协议首先读取（并发送）所请求的字，然后按环绕顺序发送该行的其余部分。
为缓存一致性共享内存多处理器设计处理器-网络接口的一个关键问题是占用率，即关键资源因每次内存访问而忙碌（被占用）的时间量。在设计良好的接口中，资源（缓存、MSHR 以及消息发送和接收单元）在每次内存访问时仅被占用一个周期（或每个字一个周期）。在某些接口中，例如那些使用软件实现一致性协议的接口，一个资源可能被占用数十个周期（或更长）。在这种情况下，这个忙碌的资源很快成为限制吞吐量的瓶颈。

20.2.3 内存-网络接口

图 20.7 展示了内存—网络接口。它接收处理器—内存接口发来的内存请求消息，并发送应答。和处理器—内存接口一样，这个接口的设计重点也是低延迟。
从网络收到的消息用于初始化事务状态保持寄存器（transaction status holding register，TSHR）。当所有 TSHR 都忙碌时，一个小型请求队列可以暂存少量请求，以推迟请求向网络上游反压的时刻。每个 TSHR 与处理器侧的 MSHR 类似，用于跟踪一个未完成内存事务的状态。内存存储体控制器和消息发送单元都监视 TSHR；其中状态字段的变化会触发相应动作。N 个内存存储体各自的控制器根据未完成事务执行读写，在 TSHR 的数据字段与内存存储体之间传输数据。消息发送单元则在事务完成时组织并发送相应的应答消息。
例如，一个不可缓存读请求会把 TSHR 初始化为“读等待”状态，同时写入地址和源节点字段。当与地址中的存储体选择位对应的内存存储体空闲时，控制器开始内存访问，并把状态改为“存储体已激活”。在该存储体返回第一个数据字之前两个周期，状态改为“读取完成”；这一状态促使消息发送单元组织面向源节点（Src）、包含所请求地址的读应答消息头。当消息头已经注入网络时，所选存储体的第一个数据字也已就绪，后续数据字可直接从存储体注入网络，完成应答。最后，TSHR 条目被标记为空闲。
如果请求只是简单读写，而且保证按顺序完成，就可以用一个普通队列或每个内存存储体一个队列来替代 TSHR，从而简化设计。每个请求到达请求队首后，等待所需存储体可用（后面的请求也会因此停顿）；存储体空闲时发起该请求，并将其放入待完成请求队列，以替代 TSHR。内存操作完成后，与位于待完成队首的对应请求配对，消息发送单元据此生成应答消息。队列比 TSHR 简单、成本较低，但无法实现复杂协议所需的各种动作。
对于缓存一致性请求，TSHR 用于保存协议消息之间的事务状态。9 以独占读取请求为例。该请求会创建一个 TSHR 条目，并将状态设置为“独占读取目录待处理”。这会激活目录单元，由它确定所请求缓存行的当前状态。如果该行处于共享状态，目录单元会将 TSHR 状态设置为“读取待处理、无效化待处理”，并用共享该行的节点列表以及这些节点的计数（在未显示的字段中）更新 TSHR。由状态中的“读取待处理”部分触发所选（按地址）内存库开始读取所请求的行。同时，状态中的“无效化待处理”

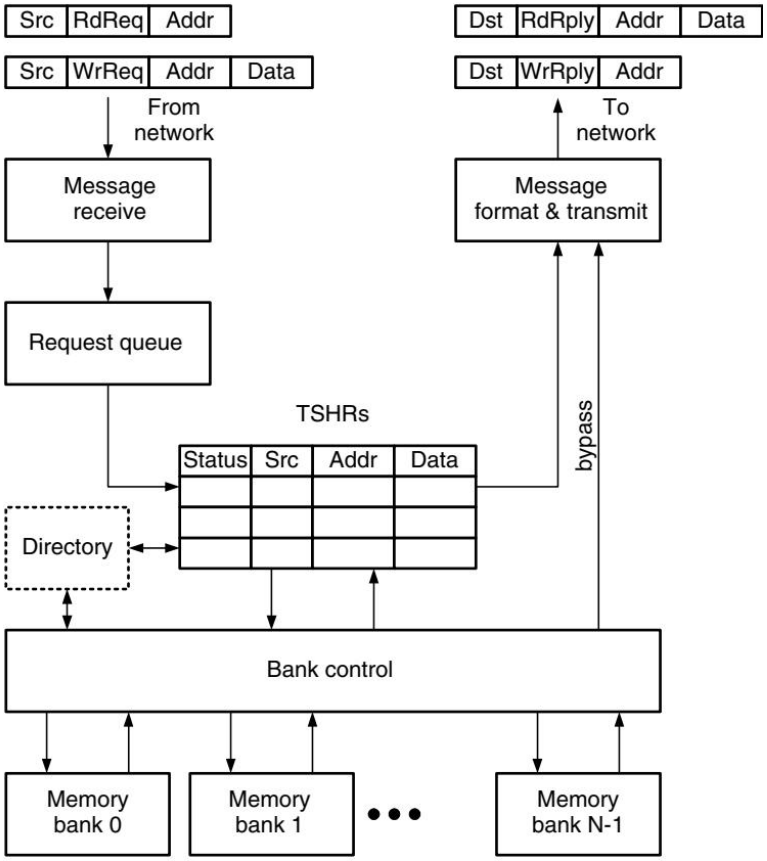


图 20.7 内存—网络接口：从网络接收内存操作请求并将其排队。挂起的事务保存在 TSHR 中，等待目录和／或存储体访问；事务完成后，组织应答消息并将其重新注入网络。

部分会触发消息发送单元，一次发送一条无效化消息，并在每次发送时更新 TSHR 中的计数。当所有无效化请求都已发送后，TSHR 状态被设置为“等待无效化回复”。随着每个无效化回复的接收，计数会更新；当所有回复都收到后，状态变为“无效化完成”。如果读取也已完成，则会触发消息发送单元发送回复消息。10

20.3 线缆-交换结构接口

在采用互连网络作为交换结构的包交换机或路由器中，网络接口必须在互连网络之前和之后都提供队列，如图 20.8 所示。输入队列用于防止发往不同输出的数据包之间相互干扰。如果某个输出 A 暂时拥塞，那么在数据包等待发往 A 时阻塞所有进入网络的数据包是不可接受的。在网络交换机中，被阻塞的数据包最终会被丢弃，因为没有机制提供背压来阻止数据包从输入线路到达。

为了避免包交换机输入端的这种队头阻塞，交换机在每个输入端为发往每个输出的数据包提供单独的输出虚拟队列。发往被阻塞输出的数据包在交换结构之前排队，从而允许发往其他输出的数据包继续前进。在实践中，通常不仅为网络的每个输出端口提供队列，还为每个输出端口的每个子端口按流量类别提供队列。这可以防止高优先级流量被低优先级流量阻塞，并防止输出端口的一个子端口（例如，10 接口线卡上的一个 1 Gbit 以太网接口）阻塞发往其他子端口的流量。

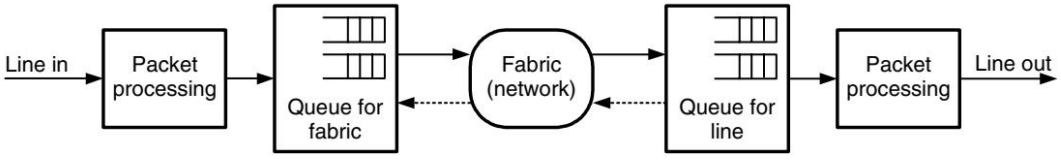


图 20.8 分组路由器或交换机需要在互连网络的输入端和输出端设置队列。输入队列保存等待通过网络传送的数据包；输出队列保存等待通过外部线路发送的数据包。

在互连网络的输出端，另一组队列暂存数据包，等待调度到输出线路上发送。之所以需要它们，是因为交换结构通常具有内部加速比（speedup）：从互连网络流向线卡的带宽高于线卡对外线路的带宽。线卡出口侧所需缓冲区数量较少，只需为每个“子端口 × 服务类别”设置一个队列。但这些缓冲区的容量通常较大，因为在许多应用中，它们必须能够缓存持续 10 ms 或更长时间的输出节点瞬时过载流量。

例如，考虑一台配有 256 张线卡的路由器，每张线卡容量为 20 Gbit/s，分成 8 个各为 2.5 Gbit/s 的子端口。互连网络作为连接线卡的交换结构，为每张线卡提供一个输入端口和一个输出端口。线卡支持 4 个服务类别，并要求各类别之间遵循严格优先级。在这种情况下，每张线卡需要总共 8K 个输入队列（256 个端口 × 每端口 8 个子端口 × 4 个类别），而输出侧只需 32 个队列（8 个子端口 × 4 个类别）。

典型线卡还在输入与输出路径上包含数据包处理逻辑，用于改写数据包并更新统计计数器。这些处理与交换结构接口相对独立，因此不再展开讨论。

有些应用会从输出队列管理器到输入队列管理器建立一套独立于互连网络内部流控

的端到端流控机制。图中的虚线表示这一机制。它通常通过互连网络发送专用控制数据包，实现对特定输入队列数据包流的启动或暂停。

为避免低优先级数据包阻塞高优先级数据包，以及避免发往拥塞输出子端口的流量阻塞同一类别中发往其他子端口的流量，互连网络必须对不同服务类别、不同目的输出子端口的流量提供互不干扰的服务。一种直接但代价较高的实现方式，是为每个“子端口 × 服务类别”配置独立的虚拟网络，即在每条物理通道上提供专门的一组虚拟通道。

实际运行时，一个数据包从输入线路到达后，首先由数据包处理器分类并确定输出端口。输入队列管理器随后将其加入相应输入队列。如果该队列原先为空，就向交换结构调度器发出请求；该调度器也属于图 20.8 中的队列模块。调度器持续跟踪输入队列以及互连网络输入端口的状态，在等待的数据包中反复选择具有最高优先级、且目的输出未被阻塞的数据包，将其送入网络。

由于互连网络的带宽高于输入线路，输入队列几乎总是空的；少数非空队列通常也很短。只有某个输出被阻塞时，输入队列才会显著增长。因此，输入队列管理器通常可把大部分排队数据包保存在片上存储器中，避免将它们写入片外存储器、随后再读回所造成的功耗。

队列管理器和调度器的框图如图 20.9 所示。队列管理器维护一个状态向量 S、片上的头指针和尾指针 h 和 t，以及片外的头指针和尾指针 H 和 T。状态向量指示队列是完全驻留在片上（地址由 h 和 t 指示），还是队列的尾部在片外（地址由 H 和 T 指示）而队列的头部在片上（如 h 和 t 所示）。

到达的包带有队列号标记。到达时，使用该编号查找队列状态。如果状态表明队列的片外部分为空，并且有空间将包追加到队列的片上部分，则将包插入片上队列。否则，将其插入片外。所有对片外存储器的队列访问都跨多个 DRAM bank 进行条带化，以平衡 bank 上的负载。与每个 bank 关联的读队列和写队列会缓冲请求，直到该 bank 变为可用。

当队列管理器将包插入队列时，调度器将它们移除。当调度器选择一个非空队列来提供下一个包时，它从片上队列中取出该包。所有队列的头部始终在片上，因此调度器从不从片外存储器读取包。如果在取出一个包后，具有非空片外尾部的片上队列的大小低于低水位标记，则发起一个请求，从片外队列传输足够的数据以填充片上队列。将队列头部存储在片上、队列尾部存储在片外，几乎不会产生片外内存流量，因为大多数队列永远不会长到需要片外队列尾部。

由于内存带宽在功耗和引脚数方面代价高昂，因此重要的是，数据包在进入线卡时最多排队一次，在离开线卡时最多排队一次。这类似于在处理器-内存接口中避免内存拷贝。遗憾的是，许多路由器不遵循这一原则，而是将数据包多次排队：一次在数据包处理器中，一次在执行流量整形的流量管理器中，一次在交换结构调度器中。通过精心设计，可以在数据包仅被写入和读出内存一次的情况下实现相同的功能。

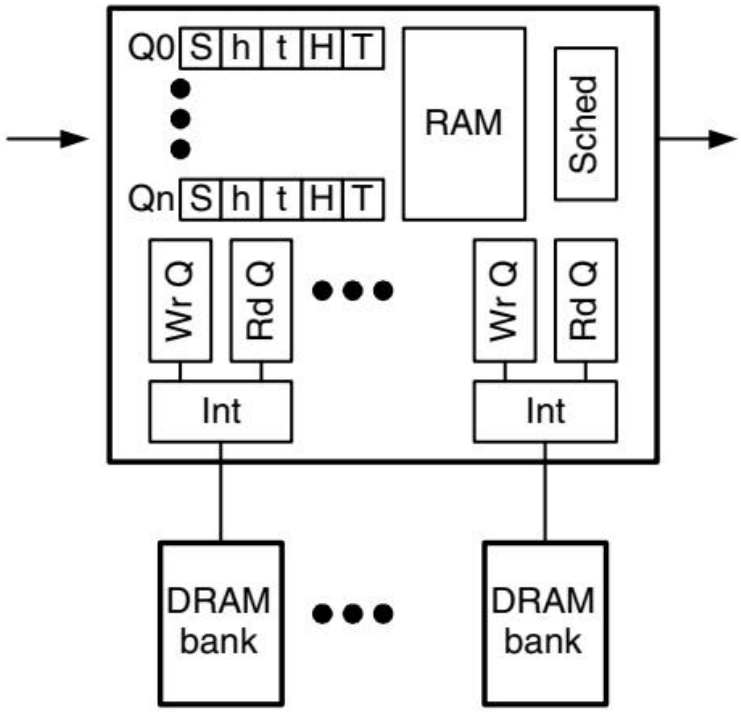


图 20.9 队列管理器将短队列保存在片上存储器中，并将长队列溢出到片外的分 bank DRAM 存储器。

20.4 案例研究：MIT M-Machine 网络接口

M-Machine 是麻省理工学院和斯坦福大学联合构建的实验性多计算机，旨在演示多个片上多线程处理器之间细粒度通信的机制 [93]。M-Machine 包含一个二维环面互连网络，并配有寄存器映射的网络接口 [112]。该接口同时支持消息传递和共享内存两种计算模型。它提供了低开销的通信，同时不牺牲进程的安全性和隔离性。

M-Machine 由通过二维环面网络相连的多个处理节点组成。每个处理节点采用一片多 ALU 处理器（Multi-ALU Processor，MAP）芯片（图 20.10）。每片 MAP 包含三个 64 位多线程处理器、一个内存子系统、一个二维环面路由器和一个网络接口。在同一芯片上不同处理器的相同线程槽中运行的线程，可以通过寄存器高效地通信与同步。M-Machine 体系结构还有许多值得研究的特点，本节主要讨论它的网络接口。

在 M-Machine 中，消息直接在处理器寄存器中组合，并通过消息 `SEND` 指令原子地发送，相关方法见第 20.1.2 节及图 20.4。MAP 芯片上三个处理器的每个线程具有 14 个 64 位整数寄存器和 15 个 64 位浮点寄存器。线程从寄存器 I4 或 F4（整数寄存器 4 或浮点寄存器 4）开始，在连续的一组寄存器中组织消息，然后执行发送指令。

图 20.11 展示了 M-Machine 的 `SEND` 指令格式。该指令包括四个字段：`length`、`dest`、`handler` 和 `CCR`。`length` 指定从 I4 开始读取多少个寄存器，以构成消息体；`dest` 指定包含目的虚拟地址的寄存器，该地址经转换后确定目标处理单元（PE），同时也包含在消息中；`handler` 指定收到消息后所执行处理代码的虚拟地址，从而支持消息驱动的计算模型 [53]。最后，`CCR` 指定一个条件码寄存器，初始值为假；消息一旦进入网

络，该寄存器就设为真，此时原寄存器可自由覆盖。利用 CCR 通知发送完成，可以使 SEND 指令的完成过程与后续指令的执行重叠。

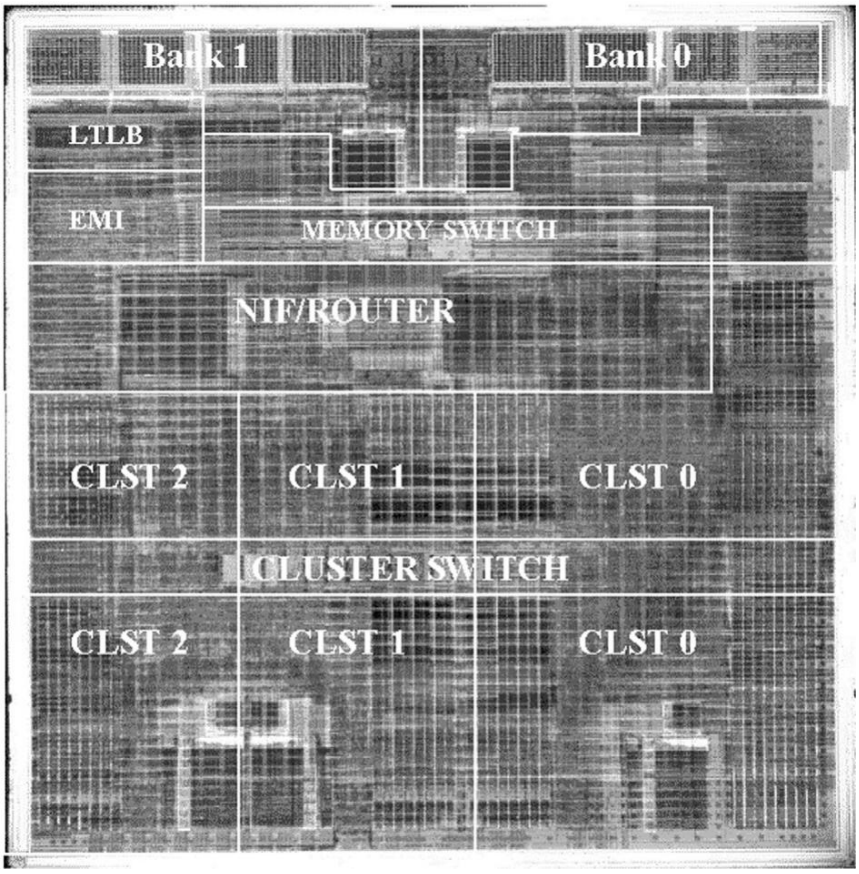


图 20.10 M-Machine 所采用的 MAP 多 ALU 处理器芯片

寄存器映射的 M-Machine SEND 机制旨在保留前代 J-Machine 双寄存器 SEND 指令的效率，同时通过将整个消息发送变为原子操作来实现安全性。在发送提交之前（以 CCR 被置位为信号），不使用任何网络资源，所有状态都保存在处理器寄存器中，这些寄存器在进程切换时会被换出。一旦发送提交，消息就完全转移到网络输入队列中，由于网络的无死锁和无活锁特性，它最终将被递送到目标节点。由于发送是原子的，因此不存在这样的危险：一个线程开始发送消息，然后发生故障或被换出，导致网络接口被半发送的消息阻塞，从而无法被其他线程使用——这种情况可能在 J-Machine 上发生。

M-Machine SEND 机制的主要限制源于 M-Machine 寄存器文件的小尺寸——14 个整数寄存器和 15 个浮点寄存器。在如此小的寄存器组下，消息大小被限制在最多 10 或 11 个寄存器，即使组装中等大小的消息也会造成相当大的寄存器压力，导致寄存器溢出到栈上。如果寄存器文件更大，该机制将更加有效。

为了确保系统级（而非网络级）的无死锁，M-Machine 采用了返回发送方的约定。每个处理节点维护一个缓冲区来处理返回的消息。空闲空间计数器 FS 反映该缓冲区中的可用空间。在发送消息之前，节点会检查 FS > L（其中 L 是消息长度），然后将 FS 减去 L，保证缓冲区中有足够的空间容纳正在发送的消息。当消息成功接收后，会发

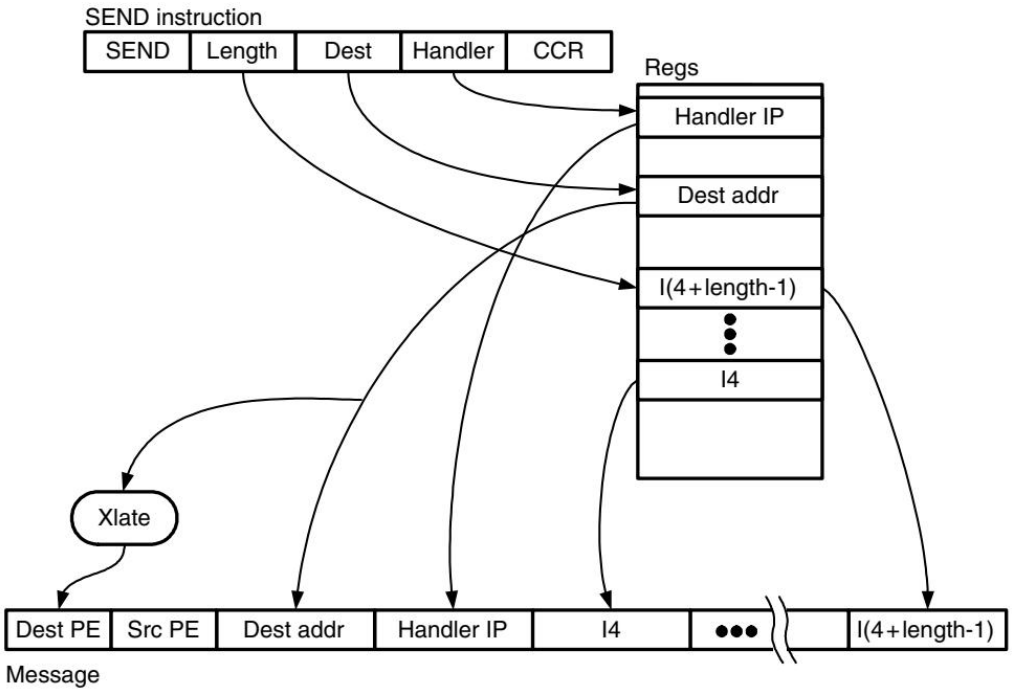


图 20.11 M-Machine 的 SEND 指令发送从寄存器 I4（对于 FSEND 则是 F4）开始、在寄存器文件中组装的消息。该指令指定消息长度（要读取的寄存器数量）、目标虚拟地址、收到消息后应运行的消息处理程序，以及当消息被网络接口接受时要设置的条件码寄存器。

送一个确认，使 FS 增加 L——将保留的空间返还到池中。如果接收节点无法接受消息，则将其返回给发送节点。返回操作在单独的虚拟通道和注入/提取缓冲区上进行，以避免请求-应答死锁。发送节点会将返回的消息缓冲在为其保留的空间中，并重试发送。M-Machine 的网络接收通过一对寄存器进行，如图 20.12 所示。到达的消息被排入两个接收队列之一——每个逻辑网络一个。图中仅显示了一个。对于每个队列，一个接收线程在多线程处理节点上的专用线程槽中运行。接收线程可以通过读取寄存器 115（MsgBody 寄存器）来读取队列中的下一个字（队列头）。要跳过当前消息的剩余字并前进到下一个消息的头部，接收线程读取寄存器 116（MsgHead 寄存器）。如果请求的字（消息体或头部）尚未到达队列，接收线程将阻塞直到它到达。在读取 115 或 116 之后，接收指针前进，将队列头移动到刚读取的字之后。

由于接收线程必须始终可用以从网络中取出消息，因此它只允许在响应到达消息时执行短的、有界的计算。这通过要求每条消息的处理程序指令指针（IP）字是一个不可伪造的指向消息处理程序的指针来规范，该处理程序已被验证具有所需的行为。系统消息处理程序读取处理程序 IP 并跳转到它。对于可以快速处理且没有故障或延迟可能的消息（如确认、物理内存写、物理内存读等），消息处理程序直接执行工作。对于其他消息类型，消息处理程序将消息排入适当的系统队列并返回处理下一条消息。M-Machine 为在上述基于寄存器的消息传递系统之上实现共享内存提供了特殊支持。内存请求查询片上缓存和本地转换后备缓冲区（LTLB）。如果请求的位置在缓存中或映射到本地内存，则访问在硬件中完成。如果访问在 LTLB 中未命中，或者 LTLB 指示需要远程访问，则所需信息——事件原因、相关地址、要写入的数据（如果有）以

及返回数据的延续（线程 ID 和寄存器号）——被排入节点的事件队列。在专用事件处理程序线程槽中运行的线程处理此队列中的事件，其方式与消息处理程序线程处理来自两个消息队列的消息完全相同。

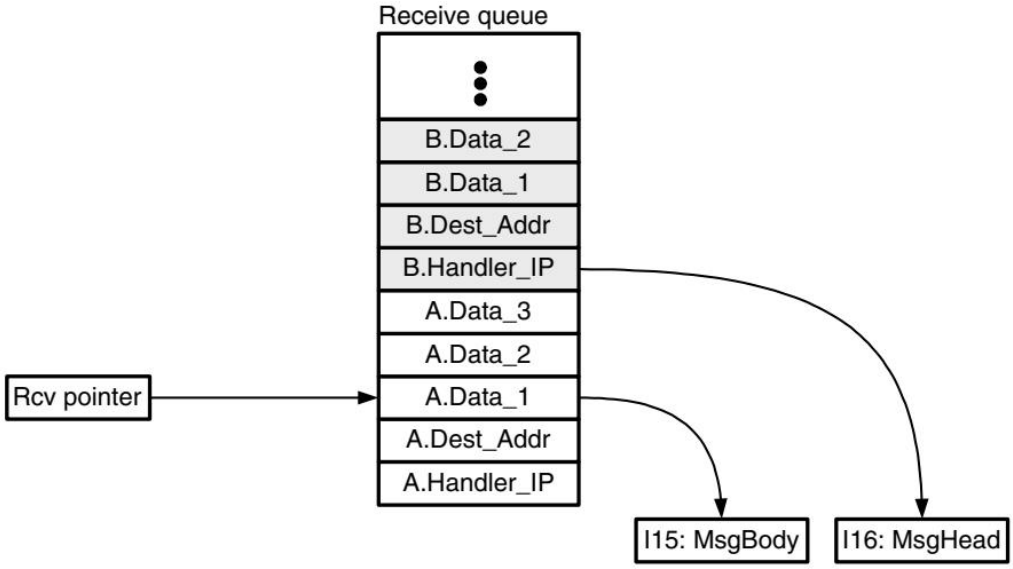


图 20.12 M-Machine 消息接收。到达的消息存储在两个接收队列之一中。从 I15（MsgBody 寄存器）读取会读取消息的下一个字。从 I16（MsgHead 寄存器）读取会读取下一条消息的头部。无论哪种情况，接收指针都会前进以指向刚读取的字之后的字。

例如，事件处理程序线程将通过向相关地址发送一条消息来处理远程读取，该消息带有远程读取处理程序的处理程序 IP，并包含延续信息作为有效载荷。在远程节点上，消息处理程序线程将跳转到远程读取处理程序 IP，执行读取，并发送回复消息。由此产生的远程内存访问时间仅比使用专用硬件执行远程内存访问的机器稍长 [32]。然而，在软件中实现这些处理程序使得能够实验不同的缓存一致性协议和内存策略。

20.5 文献注释

早期的大多数处理器网络接口都连接在 I/O 总线上，采用程序传输或 DMA 传输消息。Joerg 和 Henry 研究了不同网络接口架构和放置位置 [82]。MARS 加速器使用双寄存器接口收发消息 [4]。J-Machine 使用发送指令从寄存器文件组合消息，但把收到的消息送入本地内存 [53]。AP1000 通过把接口与缓存存储器相关联，加快了基于内存的网络接口 [80]。M-Machine 实现了可把连续寄存器组作为消息发送的指令 [112]。SHRIMP 多计算机 [22, 23] 使用连接标准工作站节点的 I/O 附接式处理器—内存接口，通过在不同节点的地址空间之间映射内存窗口来降低通信开销。Myrinet [24] 同样使用 I/O 附接式接口，并包含一个本地处理器来处理协议。Berkeley NOW 项目 [10] 就是使用这类接口构建的多计算机实例。这些接口常与消息传递库配合，例如 Berkeley 的 Active Messages [189] 或 Illinois Fast Messages [139]。Kroft [105] 首先提出使用 MSHR 来重叠多个缓存未命中。DASH 是早期基于网络的共享内存多处理器之一，详见 Lenoski

等人的研究 [115]；SGI Origin 2000 则是这类机器中较早的商业产品 [108]。有关共享内存多处理器，可参见 Lenoski 和 Weber [116]。片上/片外拆分排队的描述见美国专利 6,078,565 [16] 以及 Iyer 等人的论文 [87]。

20.6 习题

20.1 **比较消息接口开销**。考虑使用以下三种接口发送短消息（128 位）和长消息（32 Kb）：（a）双寄存器接口；（b）寄存器映射接口；（c）基于描述符的接口。假设短消息初始存放在处理器寄存器中，长消息初始存放在内存中。比较每种接口发送两种长度消息时的处理器开销，包括发送时间和处理器占用程度。

20.2 **缓存一致性协议**。某缓存一致性协议允许每条缓存行在各处理器上处于以下三态之一：无效（处理器没有该行副本）、共享（处理器拥有只读副本）、独占（处理器拥有可能已被修改的独占副本）。描述对这三种状态下的缓存行执行读或写操作时，需要发送哪些消息、其先后顺序如何。

20.3 **保护双寄存器接口**。解释恶意线程如何利用双寄存器接口无限期占用网络。写出一小段体现问题的代码，并提出一种防止该问题的方法。

20.4 **通过寄存器映射接口发送长消息**。设某处理器具有 64 个通用寄存器，并提供第 20.1.2 节所述的寄存器映射接口。现在需要从内存缓冲区发送一条包含 1024 个字的消息。编写执行该发送操作的代码片段，并提出降低长消息发送开销的方法。

20.5 **组织基于描述符的消息**。写出一条描述符消息所需的寄存器内容，使其能向另一个节点发送一块包含 1024 个字的内存缓冲区。消息应包括目的地址、标识消息类型及长度的消息头，以及数据本身。

20.6 **单侧存储的线路—网络接口**。考虑一种线路—网络接口：仅在交换结构的输出侧把数据包缓存到内存中，而输入侧只有一个可容纳 100 个数据包的小型片上队列。假设路由器需支持 128 张线卡，每张线卡处理 4 类流量；并假设输入流量均匀分布在所有输出节点上（这个假设并不现实）。如何保证小型输入队列不丢弃任何数据包？请画出方案示意图。

注释

- 1. 在现代 PC 中，这被称为北桥。
- 2. 这种类型的寄存器网络接口在 MARS 加速器 [4] 中用于发送和接收，在 J-Machine[53] 中用于发送。
- 3. 这种类型的消息发送机制在 M-Machine [112] 中实现（第 20.4 节）。
- 4. 寄存器压力问题可以通过在特殊寄存器组而不是通用寄存器中形成消息来解决。
- 5. M-Machine [112] 使用两个独立的接收线程来处理两类到达的消息。

- 6. 在高性能处理器中，内存系统设计为每个周期接受多个（通常为两到四个）内存请求。为简单起见，我们仅考虑每个周期一个请求。
- 7. 在大多数系统中，处理器提供虚拟地址，该地址在缓存访问的同时被转换为物理地址。
- 8. 感兴趣的读者可参阅 [115，116]。
- 9. 这种瞬态本可以保存在目录中，但将其分离到 TSHR 中可以节省目录空间和访问。
- 10. 许多计算机采用以下优化方法：将无效化确认直接转发给接收节点，从而使独占读事务的关键路径由四跳缩短为三跳。
- 12. 在某些应用中，调度器可能采用更复杂的调度策略。例如，它可以通过以固定间隔从某些队列中计量包来对流量进行整形。
- 13. 当这种情况发生时，一些路由器开始对阻塞的输入队列应用随机早期丢弃策略，而不是等到这些包到达输出节点后再应用此策略。

第二十一章　错误控制

许多互连网络应用都要求很高的可靠性和可用性。大型并行计算机要求其互连网络能够连续运行数万小时而不丢失数据包。互联网路由器可以容忍少量丢包，但路由器本身必须达到 99.999% 的可用性，即每年允许的停机时间仅约五分钟。输入/输出系统也有类似的可用性要求。

互连网络通常由数百乃至数千个组件组成，包括路由器、通道和连接器。所有组件的故障率累积起来，往往高于应用所能接受的水平。因此，网络必须采用错误控制机制，使得即使部分组件发生暂态或永久故障，系统仍能够不间断运行，并尽可能避免丢包。

21.1　认识故障：失效模式与故障模型

处理错误的第一步是理解组件故障的性质，然后建立简化模型，以分析故障及其应对方法。我们将系统中可能发生的故障按“失效模式”（failure mode）分类。失效模式是导致错误的物理原因。例如，通道上的高斯噪声、连接器腐蚀、冷焊点失效、电源输出驱动器开路、α 粒子撞击、芯片导体电迁移、器件阈值电压漂移、操作人员拔错模块以及软件故障，都属于不同的失效模式。

由于失效模式通常非常复杂，我们采用简单的故障模型（fault model）描述其与系统行为有关的方面，同时忽略不必要的细节。表 21.1 列出了互连网络中常见的若干失效模式、对应故障模型以及典型故障率。有些故障（如高斯噪声或 α 粒子撞击）只会引起一个或多个位的暂态错误，不会永久破坏机器的运行；另一些故障（如连接器损坏或线路电迁移）则会导致某个模块永久失效。

表 21.1　互连网络的典型故障模式与故障模型

失效模式	故障模型	典型值	单位
通道上的高斯噪声	暂态位错误	10^{-20}	BER（错误/位）
α 粒子撞击存储器（每芯片）	软错误	10^{-9}	SER（s^{-1}）
α 粒子撞击逻辑电路（每芯片）	暂态位错误	10^{-10}	BER（s^{-1}）
导体电迁移	固定值故障	1	MTBF（FIT）
器件阈值漂移	固定值故障	1	MTBF（FIT）
连接器腐蚀导致开路	固定值故障	10	MTBF（FIT）
冷焊点故障	固定值故障	10	MTBF（FIT）
电源故障	故障即停	10^{4}	MTBF（FIT）
操作人员移除正常模块	故障即停	10^{5}	MTBF（FIT）
软件故障	故障即停或拜占庭故障	10^{4}	MTBF（FIT）

暂态故障通常用误码率（BER）或软错误率（SER）建模。这些错误发生率的量纲为

s^{-1}；其倒数表示平均错误间隔时间。[1] 对永久性故障，一种模型是固定值故障（stuck-at fault），即假定某个逻辑节点永久停留在逻辑 0 或逻辑 1。另一种模型是故障即停（fail-stop），即假定链路或路由器等组件停止工作，并通知相邻模块自己已退出服务。这类故障通常用平均故障间隔时间（MTBF）描述，常以小时计；也可以用每 10^9 小时的故障次数（FIT）表示。实际设计中，经常通过自检将固定值故障乃至过于频繁的暂态错误转化为故障即停：例如，通道监测自身运行情况，一旦检测到错误就关闭自身。表 21.1 中列出的故障率乍看很小，但在大型系统里累积后可能相当可观。例如，一个包含 1,024 个节点、链路带宽为 $10\,\mathrm{Gbit/s}$ 的三维环面网络，若每条链路 BER 为 10^{-15}，则单条链路的错误率是 10^{-5} 次／秒。全系统共有 6,144 条通道，合计约 6×10^{-2} 次／秒，即平均每 16 秒出现一次错误。即便每条链路的 BER 达到 10^{-20}，系统整体也仍有约 6×10^{-7} 次／秒的错误率，约每月两次。幸运的是，通过控制错误的影响，我们仍能用这些链路构建高度可靠的系统。

有些故障属于拜占庭故障（Byzantine failure）：系统并不停止运行，而是继续以违反协议的方式行动，可能导致相邻模块失效。拜占庭故障很难处理，所幸实际发生率很低。为避免这类故障，系统应具有充分的自检能力，在故障模块造成更大破坏之前将其关闭。软件故障有时也可能表现出拜占庭行为，但同样可以监测其执行过程，一旦违反规定的不变量便停止执行。

电源、冷却风扇和时钟发生器等组件的故障，通常通过冗余设计配合现场更换处理。例如，配置冗余电源后，任一电源失效均可被其他电源掩蔽；监控硬件能检测这一故障，并可在第二个电源故障之前更换失效模块。冷却风扇可采用类似的 $N+1$ 冗余。对于需要全局时钟的系统，可分发多路时钟；一旦检测到时钟故障，各模块便切换到另一时钟，并用本地锁相环（PLL）在切换期间维持可靠时钟。本章以下假定电源、冷却和时钟等关键基础设施已遵循良好工程实践，讨论集中于链路和路由器故障。

表 21.1 的故障率针对组件寿命中的主要工作阶段。许多组件在寿命初期和末期的故障率要高得多，如图 21.1 所示。有缺陷的组件常在运行数小时后就失效，称为早期失效（infant mortality），从而使早期故障率较高。经历几百小时运行后，质量边缘的组件逐步被淘汰，故障率趋于稳定，直到组件开始老化。老化开始后，故障率再次上升。为减少早期失效，通常会在正式安装前使组件在高应力条件下运行一段时间，即进行老炼（burn-in）。对存在寿命上限的组件，则在其进入老化高故障率阶段之前按计划更换。例如，可在较高温度、电压和频率下对路由器芯片老炼 100 小时；对于预计寿命为 10^5 小时（约 10 年）的磁盘驱动器，可以在运行 5×10^4 小时（约 5 年）后更换，以避免老化失效。

在本节其余部分，我们假定系统及组件已被设计成把各种失效模式统一归约为暂态错误，或路由器、通道、网络接口的故障即停；还假定通过老炼和预防性更换使错误率基本保持常数。暂态错误可以表现为通道上传输位出错，也可以表现为状态位自发翻转。下面将介绍用来把其他故障归约到这些故障模型的检测和监控方法。

[1] 若链路的 BER 以错误数／位为单位，乘以链路带宽（位／秒）即可换算成错误数／秒。

21.2　错误控制过程：检测、隔离与恢复

所有错误控制都包含三个基本步骤：检测、隔离和恢复。首先必须知道问题已经发生。在互连网络中，错误检测可以通过检查 flit（流控单元）的奇偶校验位或校验字段，发现通道中的位错误。检测到错误或故障后，必须将其隔离，防止错误继续传播。例如，如果 flit 中的虚拟通道标识符发生错误，就要阻止它错误地修改其他虚拟通道的状态。最后是从错误中恢复并重新正常运行。在这一例子中，可以要求重传出错的 flit，重传可以在链路级进行，也可以由原始源节点发起。

21.3　链路级错误控制

大多数互连网络采用分层错误控制：从物理链路级开始，再到路由器级、网络级，最后是系统级或端到端级。在链路级，链路逻辑努力掩蔽错误（可能付出额外延迟），如果不能掩蔽，则关闭链路，将故障转化为故障即停模型。链路两端的逻辑模块共同完成位错误的检测、隔离与恢复。发生永久性故障时，链路逻辑可围绕故障重新配置链路，或者关闭链路。

21.3.1　链路监测

链路级错误检测通过在传输信息中加入冗余编码来实现，即使用错误控制码（ECC）。[2] 简单的奇偶校验足以检测任何单个位错误。不过，大多数链路采用长度足够的循环冗余校验（CRC），使多位错误漏检的概率极低。一个 n 位 CRC 字段能够检测除大约 2^n 个多位错误模式中的一个以外的错误，以及所有错误位数少于 n 的情况。

错误检查可以在不同粒度上进行：flit、数据包或包含多个数据包的帧。对较大单元（如帧）进行检查虽然更有效率，但必须等到整个单元接收完成才能发现错误，从而增加了隔离难度。当帧级 CRC 发现错误时，出错的 flit 或数据包可能已经越过多条后续通道，甚至破坏了多个路由器的内部状态。为了避免错误传播，许多路由器对每个 flit 都进行检查，并单独保护关键首部信息，使其在整条 flit 尚未接收并验证完毕时也能安全使用。

图 21.2 给出了链路监控的一个例子，其中 flit 格式包含两个 CRC 字段。CRC1 校验在第一个周期到达的首部字段（虚拟通道 ID、flit 类型和信用量）；较长的 CRC2 则对整个 flit（包括 CRC1）计算校验。为首周期到达的首部字段单独提供 CRC，可以立即使用这些字段，而不用等待整个 flit 的校验完成。把首部和其 CRC 都纳入较长 CRC 的计算，则能用更强的检测能力再次保护首部；不过，如果短 CRC 漏检、长 CRC 才检测到错误，错误可能已经传播。

链路监测应当持续进行。若空闲链路不受检查，某个故障可能潜伏很久，直到下一条 flit 到来才被发现。因此，在链路空闲时，应传送内容随机的专用空闲 flit，持续激励和测试链路。也可以在基于 flit 的监控之外，增加连续的帧级监控机制。

[2] ECC 的详细讨论超出了本书范围，相关介绍可参见文献 [21]。

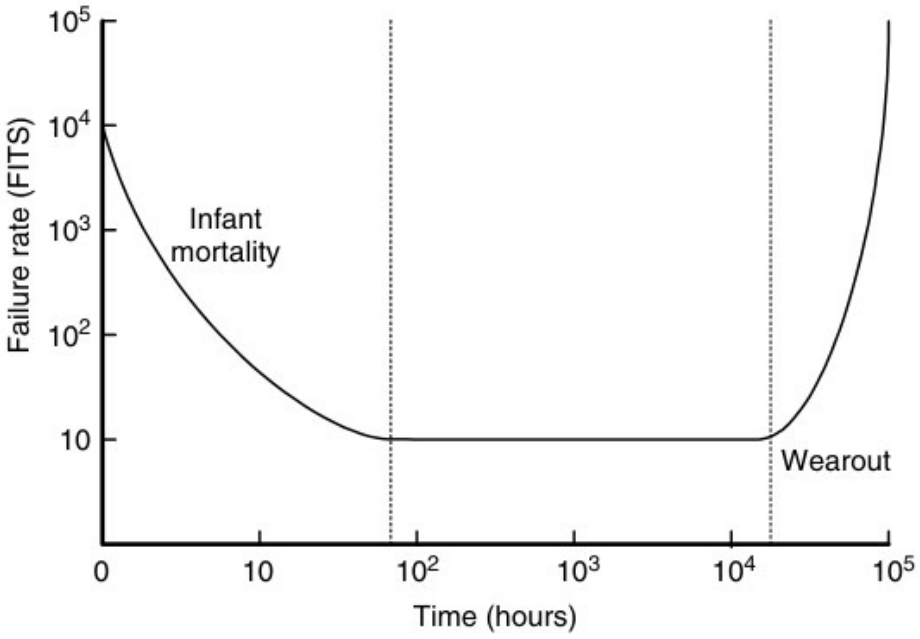


图 21.1 浴盆曲线：组件在生命周期不同阶段的故障率变化

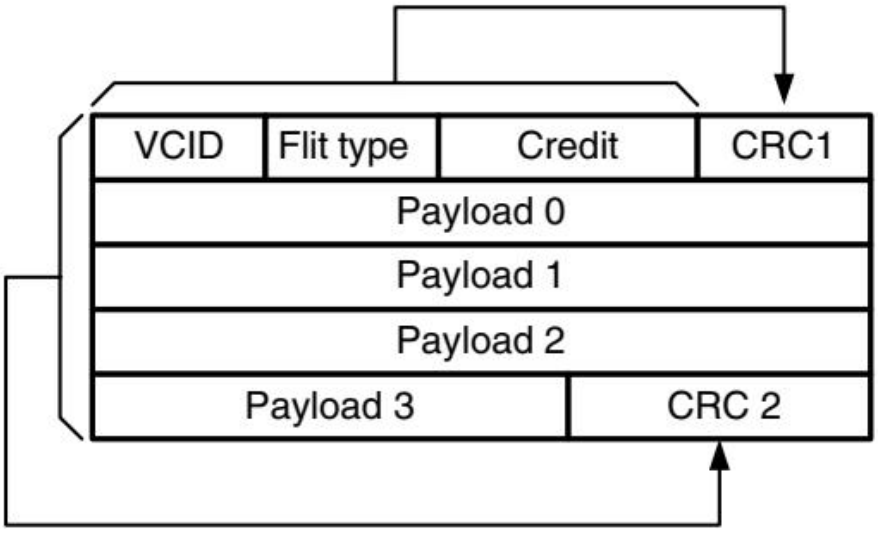


图 21.2 带有错误检查字段的典型 flit 格式：CRC1 检查首周期到达的首部字段；CRC2 检查整个 flit（含 CRC1）。

21.3.2　链路级重传

检测到错误以后，链路逻辑必须进入隔离阶段。大多数链路错误通过掩蔽处理，使错误对路由器逻辑不可见。掩蔽既隔离了错误，也在同一步骤中完成恢复。最简单的掩蔽方法是重传出错的 flit；也可使用前向纠错码（FEC），在保护单元（通常为一个 flit）中附带足够的冗余信息，既能检测错误又能纠正错误。但在往返延迟较短的互连网络中，通常更倾向于重传，因为它更简单、开销较低，并且能够纠正更多类型的错误。图 21.3 给出了简单的重传系统，图 21.4 则说明其时序。发送端每发出一个 flit，就在发送 flit 缓冲区中保留副本，直到收到正确接收的确认。[3] 每条发送的 flit 都附有标记（通常为其在发送缓冲区中的地址），便于确认消息确定是哪条 flit。接收端检查每条到达的 flit：若正确，就向发送端返回包含 flit 标记和接收状态的确认（正确、错误或忽略）。[4] 收到确认后，发送端丢弃该 flit 的缓存副本。若 flit 接收出错，发送端切换多路复用器，重传相应的 flit。多数情况下错误是暂态的，第二次发送即可成功。

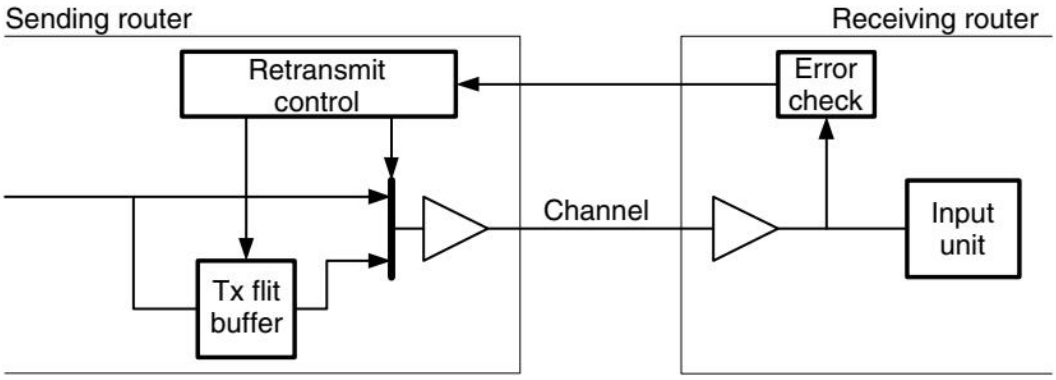


图 21.3　使用链路级 flit 重传掩蔽错误的示意图：发送端保存已发 flit 副本，发现错误后重传。

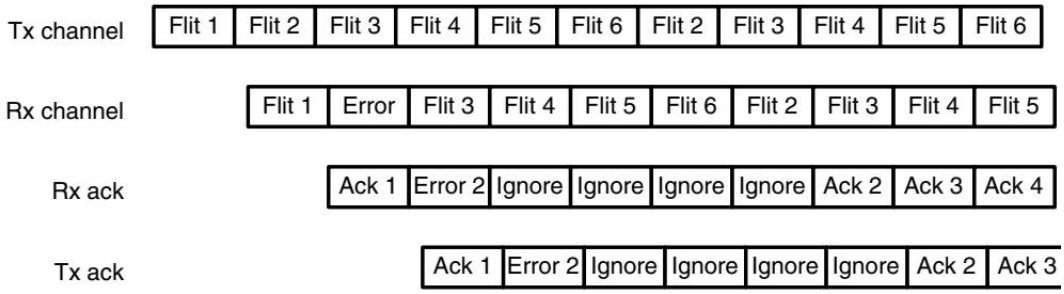


图 21.4　链路级重传时序：flit 2 出错后，接收端请求重传并暂时忽略后续 flit；发送端从出错位置重新发送。

虽然原则上只需重传单条出错的 flit，但这样会打乱通道上 flit 的顺序，显著增加路由器输入逻辑的复杂性。例如，一个数据包的主体 flit 可能在首部 flit 之前到达。为了避免这种情况，更简单的办法是把发送过程回退到出错的 flit，并从该处开始重传发送

[3] 采用输出缓冲区进行交换输出加速的实现，通常会把发送 flit 缓冲区和输出缓冲区合并。

[4] 确认消息本身也必须进行错误检查，并在出错时重传；通常将其搭载于反方向传输的 flit 中实现。

缓冲区中所有后续 flit。重传期间新到达的 flit 加入缓冲区尾部。当发送指针追上缓冲区尾端后，多路复用器再切换回直接发送交换结构输出的 flit。

图 21.4 的时序图展示了重传过程。发送端每发送一条 flit，接收端就返回确认或错误指示。flit 2 在传输过程中被破坏；接收端检查其 CRC、检测到错误后，向发送端发出错误指示。在这一错误通知返回之前，发送端又发送了 flit 3～6。这些 flit 被接收端忽略，但仍留在发送缓冲区中。发送端收到错误通知后，退回到 flit 2，从这一位置起重新发送所有 flit。接收端在收到重传的 flit 2 后停止忽略后续 flit，恢复正常运行。从接收端来看，错误已被掩蔽；与无错误运行相比，只多出五个 flit 时间的延迟，即错误通知往返所需的时间。

发送缓冲区通过三个指针管理，如图 21.5 所示。在无错误时，发送指针和尾指针分别起到先进先出发送队列的头、尾指针作用。新 flit 在尾指针处加入缓冲区，从发送指针处开始发送；若路由器直接从交换结构发送，发送指针和尾指针指向同一位置。但与普通 FIFO 不同，已发送 flit 所占的位置不能立即复用，必须保留到收到确认为止。确认指针标识尚未得到确认的最旧 flit。每收到一个确认，确认指针前进，释放一个 flit 缓冲区位置；如果收到错误指示，则把发送指针复位到确认指针。

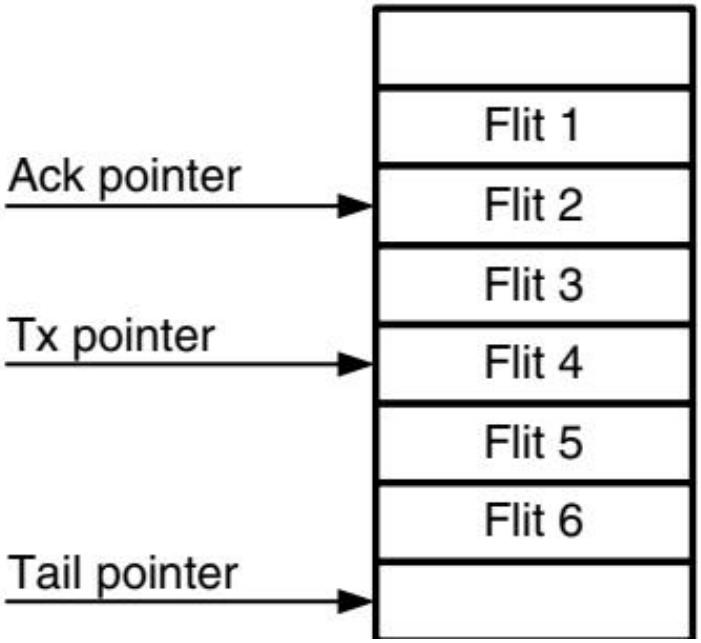


图 21.5 发送 flit 缓冲区的三个指针：确认指针、发送指针与尾指针分别跟踪待确认、待发送和缓冲区末端。

21.3.3 通道重配置、降级与关闭

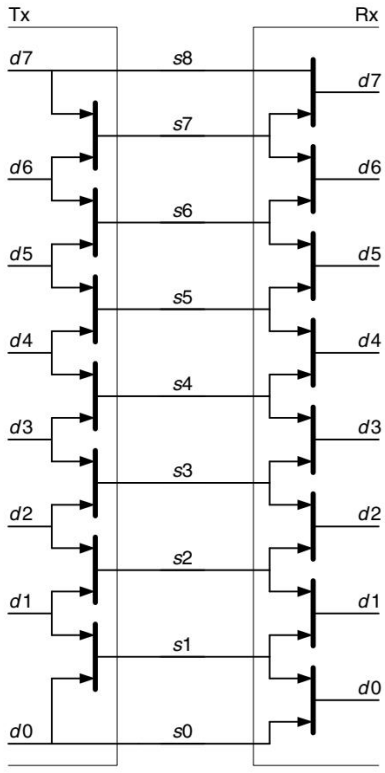
错误隔离过程的一个重要方面，是防止已知的故障组件继续干扰业务。如果某条通道反复发生错误，或其 BER 明显高于预期，就可能意味着通道某部分发生了永久故障。为检测此类持续性错误，每条通道都设有错误计数器，记录其错误率；当错误率超过阈值时，通道即被判定为故障。例如，若某条通道预期错误率为每秒 10^{-5} 次（约每天

一次），而它在 10^4 秒（约三小时）内记录超过 10 次错误，则应宣布该通道故障并将其停用。这种 BER 监测及故障通道停用通常由监控软件持续轮询网络组件状态来执行。一些多位通道提供一个或多个备用信号位，因此在单条信号线发生故障时，可以围绕故障位重新配置通道。检测到永久性错误后，系统先运行诊断过程，找出失效的信号位；如果条件允许，便重新配置通道以绕开这些坏位。

图 21.6 给出了带有一个备用位的 8 位通道。发送端和接收端各有一组八个二选一多路复用器，可把通道的八个数据位 $d_0, \ldots, d_7$ 映射到九条信号线 $s_0, \ldots, s_8$ 中任意八条正常线路。例如，当 s_3（可能因为连接器故障）失效时，发送端把 d_3 至 d_7 顺移，使其经 s_4 至 s_8 传输；接收端再将这些位还原到原来的位置。

另一些通道没有备用位，但能够只用正常工作的信号线，以较低速率重新配置后继续传输。例如，8 位通道坏掉一条线后，仍可通过余下七条线以原速率的 $7/8$ 工作。这是渐进退化（graceful degradation）的一个例子：链路不是在第一个位失效时完全关闭，而是逐步损失带宽。即使链路在第一个永久位错误后必须故障即停，也可以在网络级实施渐进退化。

如果无法重新配置通道以掩蔽永久性错误，则应将该通道保持关闭（把通道错误归约为故障即停），并在错误控制体系的上一级，即网络级，通过绕开失效通道的路由进行处理。关闭通道时，发送 flit 缓冲区中尚存的 flit 可被送回输入控制器重新路由，也可以直接丢弃。



(a)

当故障通道将一个数据包截为两部分时，情况会更加复杂。通道失效时，首部 flit 可能已在下游数跳之外，而尾部 flit 仍在上游。故障链路上游失去首部的剩余部分必须

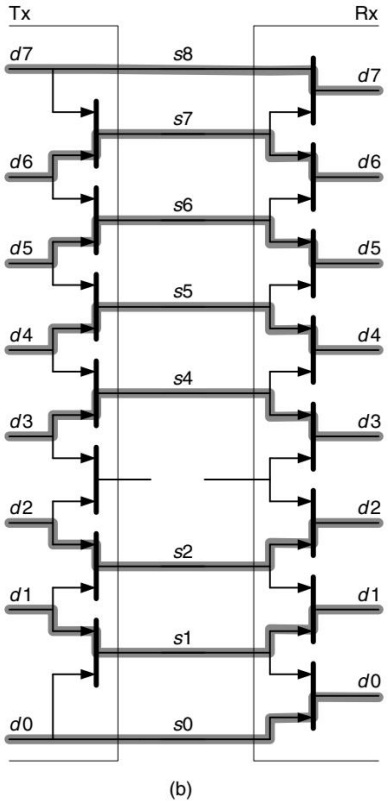


图 21.6 带一个备用位的八位通道及第 3 位故障时的重配置

丢弃，因为它不再具有首部携带的路由信息，不知道应去往哪里。更棘手的是没有尾部的包首：首部 flit 会继续朝目的地传播，并沿途分配虚拟通道；若这些资源不能释放，需要使用它们的其他数据包便无法继续前进。为了释放资源，输入控制器应在所连接通道发生故障时，对每条正在接收数据包的虚拟通道合成一个尾部 flit。由于通道故障而被拆分并丢弃的数据包，可以借助端到端错误控制（第 21.6 节）恢复。

21.4 路由器错误控制

有些失效模式导致链路错误，另一些则导致路由器暂态或永久性故障。路由器存储器和逻辑中的软错误发生频率虽低于通道位错误，但在大型高可靠系统中也必须加以控制。与链路错误一样，检测、隔离、恢复三项原则同样适用于路由器错误。

检测路由器错误的一种直接方法是复制路由器逻辑，并逐周期比较其代表性信号的异或结果。一份逻辑作为主模块，产生所有实际输出；另一份影子模块接收相同输入，但除比较外，其输出不被系统使用。检查电路比较两份逻辑的输出和内部信号。仅比较模块输出通常无法及时发现故障：一些错误可能破坏路由器内部状态，直到数千个周期后才反映在输出端。为简化比较，主模块和影子模块一般会将成千上万个内部信号及输出信号通过异或压缩成少量状态线（例如 32 条），然后比较对应的压缩状态线。只要其中一个内部信号出现差异，压缩状态往往就会不同，从而检测到错误。

路由器存储器和内部总线中的错误可通过 ECC 检测。存储器与总线上常用单错纠正、

双错检测（SECDED）码来掩蔽位错误。对于 n 位存储字或总线字，可为其添加大约 $\log_2(n)+1$ 个校验位。为了掩蔽存储器错误，还应定期扫描存储器：读取每个地址并纠正其中的单个位错误。通过扫描尽早发现并纠正首个错误，可以降低首个错误尚未纠正时又发生第二个、导致无法纠错的概率。

路由器还可通过一致性检查检测错误。协议一致性检查例如确保两个首部 flit 之间至少存在一个尾部 flit。信用量一致性检查定期确认通道接收端的输入控制器与发送端的输出控制器，对于输入控制器中的空闲缓冲区数量有一致的认识。状态一致性检查则确保收到的输入符合输入控制器或虚拟通道的当前状态。一旦违反其中任一一致性条件，就表示检测到了错误，随后必须将其隔离。

路由器错误一经检测，就需要隔离并恢复。对于无法掩蔽的错误，最简单的隔离方式是停止出错的整个路由器，或者仅停止其中发生错误的部分（如输入控制器），使其退出服务。这相当于把错误归约为故障即停，防止故障破坏相邻路由器状态。为了实现隔离，路由器可划分为多个故障隔离区域。通常每个输入控制器是一个可独立退出服务的区域，而整个路由器构成更大的外围隔离区域；如果分配器或其他共享逻辑出错，则必须让整个路由器退出服务。

如果错误只是暂态的，在把全部状态复位到初始值后，路由器或输入控制器可以重新启动。经过故障组件的数据包会被丢弃，与链路故障时一样。故障组件重新启动时，还必须与相邻模块同步状态。例如，路由器重启时，应向每个相邻路由器发送控制 flit，重新建立两者之间的通道并初始化信用量计数。

若发生永久性故障，组件无法重启，必须更换。可靠性设计的一部分，是使故障模块可以在系统运行期间更换，即热插拔（hot swapping）。路由器等有源组件通常可按这种方式更换，但实际可更换的最小单位（现场可更换单元，FRU）可能大于一台路由器。例如，若四个路由器封装在同一块印制电路板上，那么更换其中一个坏路由器时，必须拆下整块板，使四个路由器都暂时退出服务。

除现场更换外，还可以预先配置足够多的冗余单元（通道和路由器），使系统在整个预定寿命内仍能以足够高的概率维持所需性能。如果系统部署在现场维修困难的地点（例如轨道空间），这种方案尤其有吸引力。

21.5　网络级错误控制

网络级是从链路级开始的分层错误控制体系的下一个层次。在网络级，我们把链路和路由器的故障均建模为故障即停，网络必须绕开这些失效组件路由数据包。最简便的实现方式是自适应路由：将已停用链路从自适应路由函数的可选集合中删除，数据包通过剩余可用链路之一转发。

网络级错误控制也可以采用基于表的无关路由实现。故障刚发生时，数据包仍按照原路由表转发；选到失效链路的数据包要么被丢弃，要么在局部改道。局部改道可以用一系列跳转代替原本穿越失效链路的一跳，例如把“向东一跳”改成“向北、向东、向南”，最后抵达同一中间节点。此后系统重新计算路由表，彻底避开失效链路。

21.6 端到端错误控制

如果在链路或路由器故障隔离过程中丢弃了数据包，可以使用端到端的数据包重传予以恢复。它的工作原理类似链路级的 flit 重传，但重传对象从 flit 变成整个数据包，重传范围也从单条链路扩展到源节点和目的节点之间的整个路由路径。

端到端恢复的第一步，是由发送节点保存每个已发出数据包的副本，直到收到相应的确认。如果在超时之前没有收到确认，或收到否定确认（某些错误情况下会发生），则重新发送该数据包。必要时重复这一过程，直到收到确认，再丢弃保存的副本并释放发送数据包缓冲区的空间。

端到端重传可能导致目的节点收到同一数据包的两个副本，例如在延迟到达的确认刚被收到之前，源节点恰好重传了数据包。处理重复包的方法是：为每个数据包分配序列号，每个接收节点保存最近 T 个 flit 时间间隔内接收的普通包和重传包的序列号表。每个数据包到达时，如果首部标识它是重传包，就与已收到的普通包列表比较；如果是普通包，就与重传包列表比较。只要发现重复，就丢弃这个数据包。与普通包列表比较的代价较高，但发生频率低；与重传包列表比较则通常很便宜，因为平时没有重传包，即便有也往往只是一两个。

在网络级、路由器级和链路级错误控制之上加入端到端重传，可以使互连网络达到任意高的可靠性。不过，互连网络的客户端——如处理节点、网络线路卡或 I/O 设备——本身仍可能失效。所幸同样的可靠性设计原则也适用于这些设备，例如使用连接到不同网络终端的冗余客户端，从而构建完整的高可靠系统。

21.7 文献注释

Siewiorek [167] 对可靠系统设计作了很好的综述，涵盖故障机制、故障模型和错误控制技术。错误控制码见文献 [21]，更严格的数学分析见 [19]。MIT Reliable Router [52] 采用了本章介绍的多项技术，包括链路监测、链路级重传、链路关闭以及网络级故障掩蔽。自适应路由算法 [36,118] 有助于在故障链路关闭后绕开故障组件。文献 [157] 讨论端到端错误控制，并论证了它既是必要的，也是充分的。

21.8 习题

21.1 系统 MTBF。考虑一个包含 1,024 个节点的三维环面网络。每个节点有 2 颗逻辑芯片、8 颗存储芯片，并通过带宽均为 16 Gbit/s 的链路与 6 个邻居相连。利用表 21.1 的数值计算该系统的 MTBF。

21.2 具有重传机制的 MTBF。对习题 21.1 的机器，假定链路级重传能够掩蔽全部链路错误、路由器均有自检能力，且采用端到端数据包重传，重新计算 MTBF。

21.3 链路级重传。考虑第 21.3.2 节描述的链路级重传系统。画出一幅时序示意图，说明 flit 已正确接收，但反向发送的确认消息在传输中出错时会发生什么。

21.4 路由器重启。假设路由器 X 检测到内部故障后关闭并重启。描述应按何种顺序恢复其输入与输出链路，才能在尽可能少地干扰上游路由器 W 和下游路由器 Y 的情况下使 X 重新上线。尤其要讨论：(a) 数据包的主体与尾部 flit 在 W，首部 flit 在 X 时；(b) 主体与尾部 flit 在 X，首部 flit 在 Y 时；(c) X 重启时，来自 W 的新数据包紧随首部 flit 已位于 X 的那个数据包之后到达时，分别会发生什么。

21.5 带宽与错误率的关系。假设路由器使用一种 I/O 链路技术，其错误率为 $\exp(-(t_{\text{bit}} - 200\,\text{ps}))$，其中位时间 $t_{\text{bit}} = 1/f$。链路要求 flit 错误率低于 10^{-15}。考虑 ECC 冗余开销，分别求 (a) 不重传，(b) 使用可检测任意单位错误的简单奇偶校验进行重传，以及 (c) 使用可检测任意双位错误的 ECC 进行重传时，链路可达到的有效频率。设 flit 载荷为 64 位，奇偶校验开销为 1 位，双错检测码开销为 6 位。

21.6 链路级与端到端重传。 Saltzer 等人 [157] 认为，只要系统具备端到端错误控制，就不再需要其他错误控制。本题将考察这一论点。

(a) 一个互连网络发送 1,024 位数据包，每包分成 64 位 flit。每个数据包平均经过 10 跳，每条链路 BER 为 10^{-5}。链路带宽为 $1\,\text{Gbit/s}$，路由器每跳延迟 20 ns。目的节点对每个数据包进行检查，若包出错则请求重传。为简化分析，假定错误控制码能检出一切错误，确认或否定确认消息的传送不会出错。求数据包接收出错的概率，以及最终的数据包吞吐量和平均延迟。

(b) 现在考虑同一系统，但除了端到端重传外还采用链路级重传。每条链路接收每个数据包的每个 flit 时都会检查错误，若 flit 出错则请求重传。链路级重传会如何影响吞吐量和平均延迟?

原书注释

1. 表 21.1 中的数值只表示典型 2003 年系统相应错误率的数量级。不同技术和系统的错误率可能相差很大；进行错误分析时，应取得目标系统所用技术的实际数据。

2. 将以错误数／位表示的链路 BER 乘以链路带宽（位／秒），即可得到错误数／秒。

3. ECC 的深入讨论不在本书范围内，相关介绍参见 [21]。

4. 若使用输出缓冲区来支持交换结构输出加速，发送 flit 缓冲区通常与输出缓冲区合并。

5. 确认消息也必须检查错误，并在出错时重传。通常把确认搭载到反向传送的 flit 中；若该 flit 任意部分（包括确认）出错，就重传整个 flit。

6. 若某通道上的每条 flit 均传输失败，它会在最初 10 次故障后被停用，不必等待完整的 10^4 秒观测间隔。

7. Reliable Router [52] 通过在含有数据包 flit 的每个路由器中保留首部 flit 副本，使被切断的无首部包尾有可能改道。

8. Reliable Router 使用独特的令牌协议，避免接收端重复包检查。

第二十二章　总线

总线是最简单、应用也最广泛的一种互连网络。它用一条共享通道连接多个模块，这条通道充当广播媒介。总线通常由连接到所有模块的一组信号线（或一根信号线）实现。一个模块通过总线发送消息，所有其他模块都能接收到该消息。很多时候，消息实际上只发往其中一个模块，其余模块会忽略它。接收者也常通过总线向原发送者返回应答。例如，处理器读取存储器时，会向某个存储器模块发送包含待读取字地址的消息；存储器模块随后将所请求的数据返回处理器，完成一次事务。总线协议规定任意时刻哪个模块有权发送，并定义模块之间的消息格式与事务行为。

总线有两个重要特性，常被其上层通信协议利用：广播和串行化。在总线上发送多播或广播消息的代价与发送点对点消息相同，因为每一条消息在物理上都会被广播给所有模块。因此，通过总线分发全局信息十分方便。由于任意时刻只能有一个模块在总线上发送消息，各条消息按照固定且无歧义的顺序出现，即被串行化。监听式缓存一致性协议同时利用这两个特性：当某条缓存行将被写入时，把其地址广播给所有模块，使它们能够使本地副本无效（或更新副本）；写入被串行化后，也能明确某一地址最后写入的值。在一般互连网络上，广播不再免费，串行化也需要显式同步，因此此类协议要复杂得多。

总线简单且成本较低，因而广泛用于数据通路内部传输、处理器–存储器互连、路由器或交换机线卡之间的连接，以及处理器与 I/O 设备之间的连接。不过，总线性能受到两项因素限制：首先，多点连接的总线在电气上难以实现高速运行 [55]；其次，总线本质上是串行的，同一时刻只能传送一条消息。对性能要求超出总线能力的应用，通常采用点对点互连网络。

22.1　总线基础

图 22.1 给出连接模块 A～D 的典型总线数据通路。每个模块经双向接口接入总线：当发送使能信号 E_T 有效时，模块可将内部发送信号 T 驱动到总线上；接收使能 E_R 有效时，可从总线采样到内部信号 R。例如，当模块 A 向模块 C 发送消息时，A 置位 E_{TA}，将 T_A 驱动到总线上；同一周期内，C 置位 E_{RC}，把总线消息采样到 R_C。

在物理上，总线可以只是一根导线（串行总线），也可以是一组可并行传输完整消息的导线（并行总线）。介于两者之间的实现则用较少的并行导线分若干周期传送一条消息。

从电气角度看，由于每个模块连接处的支线和阻抗不连续，总线很难在高频下可靠工作 [55]；这些接口的具体电气问题不在本书讨论范围内。从逻辑角度看，发送接口必须允许某模块在其发送使能有效时驱动总线。发送接口可以是三态驱动器（图 22.2a）、

开漏驱动器（图 22.2b），也可以是点接发射极驱动器（图 22.2c）。后两者的优点是多个模块发送使能短暂重叠时不会造成电源与地之间的短路。接收接口包括一个适用于总线信号电平的接收器，以及在接收使能有效时锁存总线上消息的寄存器。对串行总线或多周期总线，该寄存器可能需要经过多个总线周期才能组装完成一条消息。

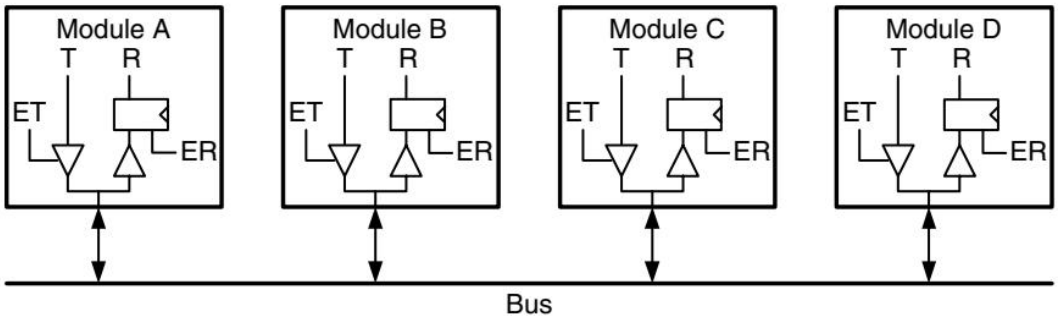


图 22.1 典型总线的数据通路：模块 A～D 共用一条广播通道，通过发送使能 ET 和接收使能 ER 控制数据传送。

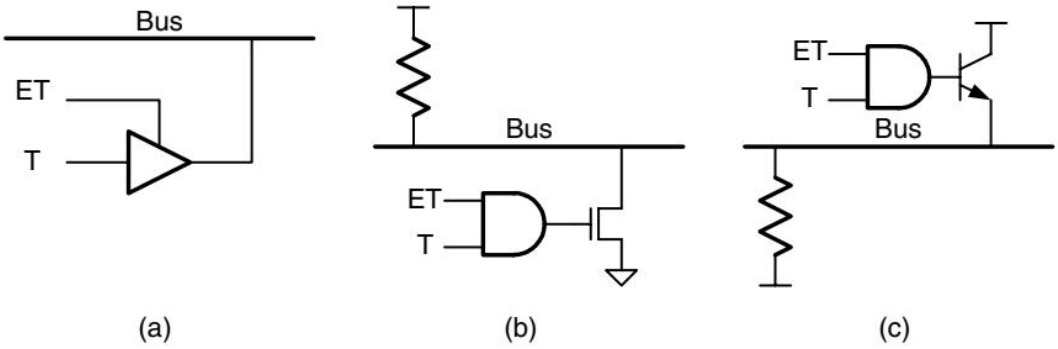


图 22.2 典型总线发送接口：（a）三态驱动器；（b）开漏驱动器；（c）点接发射极驱动器。

总线的操作单位包括周期、消息和事务。与一般互连网络一样，消息是从发送者传递给一个或多个接收者的逻辑信息单元。例如，向存储器发送一条读取请求是一条消息，而返回的数据是另一条消息。事务则是一组相关消息，共同完成某个完整操作。一个存储器读取事务通常包含请求消息和响应消息。总线周期表示总线执行一次基本传输所需的时间；消息可能跨越多个周期，而一次事务也可能包含多条消息。

总线可采用外部时序控制或内部时序控制。外部时序控制总线的全部发送、接收使能信号由中央外部时序器控制；内部时序控制总线则由参与通信的模块通过总线上的仲裁、地址和握手信号协调操作。对于内部时序控制总线，在传送实际数据之前，必须首先确定发送者和接收者，必要时还要通过仲裁取得总线使用权。

在异步总线上，每个总线周期由请求和确认信号决定时序。执行一个总线周期时，发送端先把数据驱动到总线上，再置位请求信号 Req。接收端看到 Req 后，从总线上采样数据，并置位确认信号 Ack，表示传输完成。发送端收到 Ack 后关闭驱动器，释放总线供下一次传输使用。

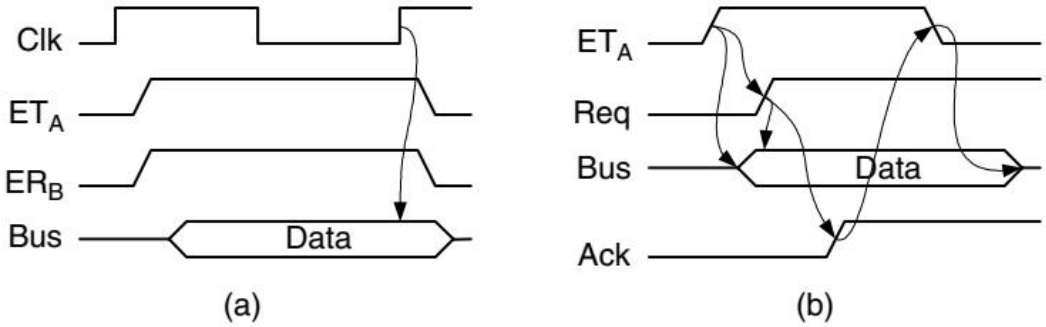


图 22.3　使用总线连接处理器 P 与 16 个存储器模块的示例：读事务先发送地址与命令，再由存储器返回数据。

通过内部时序控制总线发送消息，需要依次完成仲裁、寻址、传输和确认。仲裁决定哪个模块有权发起事务。仲裁胜出者通常称为总线主设备。在最简单的总线中，某一个模块（例如处理器）始终是主设备，因此无须仲裁；在较复杂的系统中，则需要通过简单协议选出主设备。

寻址阶段选定要接收消息的一个或多个模块。有些总线将寻址作为独立步骤，在每条定向消息之前先发送广播地址；另一些总线则把地址放在消息内部，在传输阶段一起发送。图 22.4 给出了内部寻址消息的示例。此时，总线上的所有模块都收到消息，再检查控制和地址字段，判断消息是否与自己有关。在串行总线上，控制与寻址信息会先发送，以便无关模块尽早停止接收后续 phit。注意，这里的地址字段是总线上模块的地址，而不是存储器地址等上层协议地址。总线协议把读请求中包含的存储器地址视为消息数据。

图 22.4：内部寻址的总线消息。消息由指定消息类型（如存储器读）的控制字段、总线地址和要传输的数据（如存储器地址）组成。

Control	Address	Data

传输阶段真正将数据从发送模块移动到接收模块。在简单总线中，这可能是唯一的阶段。传输完成后可以接着执行确认阶段，由接收模块确认数据已无误接收。如果未收到确认，或接收端报告传输错误，发送端可以尝试恢复，例如重传消息。

图 22.5 展示了一条简单的并行总线，将处理器 P 连接到 16 个存储器模块 M_0～M_{15}。总线包含并行信号线，分别传输 4 位控制、4 位模块地址和 32 位数据。控制字段表示总线周期的类型；当没有模块驱动总线时即为空闲周期。地址字段为读写事务选择 16 个模块中的一个。数据字段承载所有消息的有效载荷，包括存储器地址与数据。由于只有处理器可以发起事务，所以无须仲裁。寻址与传输并行进行，四条地址线在每次事务中选择目标存储器模块。

图中的时序示意了总线读写事务。两种事务都包含两条消息：第一条由处理器发给特定存储器模块，读取目标为 M_4、写入目标为 M_3；第二条由选定的存储器模块返回给处理器。对于读事务，这两条消息各占一个周期。对于写事务，写请求需要两个周期，

分别在数据线上发送地址与待写入的数据。

segment signals. To perform a bus cycle, the transmitter drives the data onto the bus and then asserts the request signal *Req*. Upon seeing the request signal, the receiver samples the data off the bus and asserts an acknowledge signal *Ack* to indicate that the data transfer is complete. Upon receiving the acknowledge signal, the transmitter turns its driver off, freeing the bus for the next transfer.

Sending a message on an internally sequenced bus involves the steps of arbitration, addressing, transfer, and acknowledgment. Arbitration determines which module gets to initiate a transaction. The winner of the arbitration is often called the *bus master*. In the simplest buses, one module (e.g., the processor) is always the bus master and no arbitration is needed. In more complex systems, a simple protocol is used to elect a master.

The addressing phase selects the module (or modules) to receive a message. Some buses perform addressing as a separate step, sending a broadcast address message before each directed message, while others send the address as a part of each message during the transfer. An example of such an internally addressed message is shown in Figure 22.4. In this case, all of the modules on the bus receive the message and then examine the control and address fields to determine if the message applies to them. In serial buses, control and addressing information is sent first so a module can examine these phits of the message and stop receiving as soon as it determines that it is not a target of the message. Note that the address field here is the address of a module on the bus and *not* an address at a higher level of protocol (such as a memory address). A memory address, for example in a memory read request message, is considered data by the bus protocol.

图 22.4 集中式总线仲裁：模块先向中央仲裁器提出请求，获得授权后才能发送。

读事务发生在周期 1～3。周期 1 中，处理器在控制线上驱动 Rd 发起读请求，地址线选定目标模块（本例是 M_4），数据线发送要读取的模块内部地址（本例为十六进制 1000_{16}）。周期 2 总线保持空闲，存储器模块在内部访问请求的数据。周期 3，存储器模块驱动总线发出 Reply 消息，控制线标记 Reply，数据线承载读出的数据。虽然周期 3 由 M_4 驱动总线，却无须仲裁，因为它只是响应处理器的请求而不是发起新事务，此时也不会有其他模块同时响应。

写事务紧随读事务，发生在周期 4～6。处理器向模块 M_3 发送占用两个周期的写请求。周期 4 是写地址周期，控制线上标记 WrA，数据线上传送写入目标地址 2000_{16}；周期 5 是写数据周期，控制线上标记 WrD，数据线上传送待写入数据 1234_{16}。

无论是读事务还是写事务，如果目标存储器模块在规定时限内没有回应，就会发生超时。超时后，总线控制权返回处理器，处理器可以发起新事务。超时机制可防止对未实现地址或无响应模块的访问无限期占用总线。

22.2 总线仲裁

当多个模块都可以发起事务时，各模块在开始事务前必须仲裁总线使用权。第 18 章介绍的任一种仲裁器都可用于实现这一机制。

图 22.6 显示了辐射式仲裁（radial arbitration）。其名称来自请求线和授权线像车轮辐

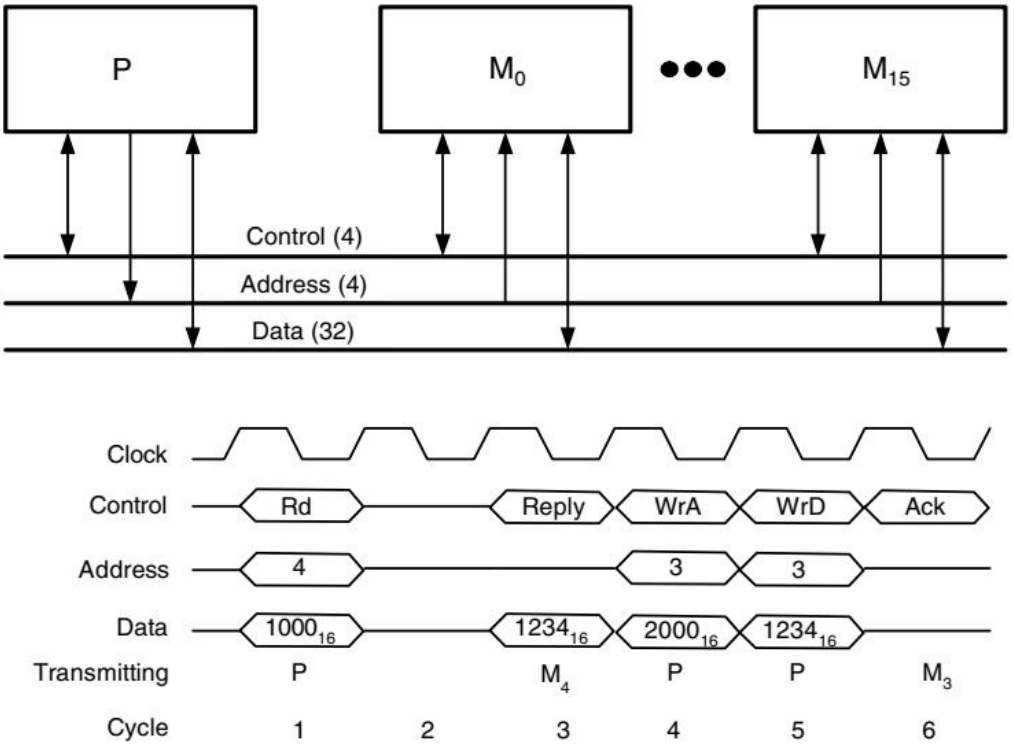


图 22.5　简单共享总线的读写事务：处理器 P 与 16 个存储器模块 M_0～M_{15} 通过控制、模块地址和数据线相连。一次读事务由地址请求周期 Rd 和应答周期 Reply 组成；一次写事务用 WrA、WrD 两个周期发送地址与数据，并由单周期 Ack 应答。

条一样从各个模块分别连接到中央仲裁器。这些信号虽然可能与总线信号使用同一连接器，却不作为共享总线信号相连，而是各自独立接到仲裁器。模块要发起事务时先置位请求线；收到授权后即成为总线主设备，并可发起事务。公平性规则可能要求模块在事务最后一个周期撤销请求，以便让其他模块使用总线。另一种方法是仲裁器监视事务，在其最后一周期撤销授权，并同时把总线授权给其他模块。

图 22.7 展示了同步总线的仲裁时序。模块 1 在周期 1 请求总线，在周期 2 得到授权，在周期 3 开始事务。与模块 1 始终拥有总线或此前已获得授权的情形相比，即使不存在竞争，仲裁也会给事务增加两个周期的延迟。模块 2 在模块 1 获得授权后提出请求，因此必须等模块 1 释放总线才能开始事务。仲裁造成第一笔事务结束与第二笔事务开始之间有两个空闲周期。通过把下一笔事务的仲裁与当前事务的执行重叠起来，即流水化仲裁，可以消除这一空闲间隔。

为了省去中央仲裁器，早期一些总线使用菊花链仲裁，如图 22.8 所示。固定优先级仲裁器（第 18.3 节）被分布到各模块中。模块 1 的输入进位 carry0 固定为高电平，因此只要提出请求就总能获得授权；如果它不请求，就把进位信号传给模块 2，依此类推。菊花链仲裁器主要具有历史意义：固定优先级以及进位信号跨多个模块传输所造成的高延迟，使其很少用于现代系统。

顺序式分布仲裁器利用线或（wired-OR）总线的信号线执行实际仲裁。如果总线由开漏或射极跟随器发送器实现（图 22.2b、c），各总线线上的值就等于所有已使能发送

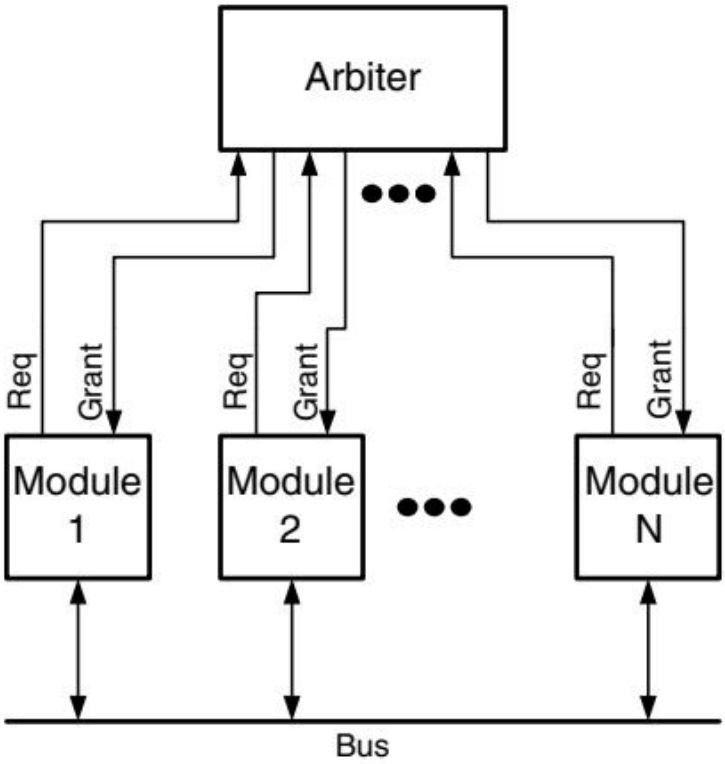


图 22.6 采用辐射式仲裁的总线：请求发起事务的模块向中央仲裁器发出请求信号；收到授权后，该模块成为总线主设备并可发起事务。

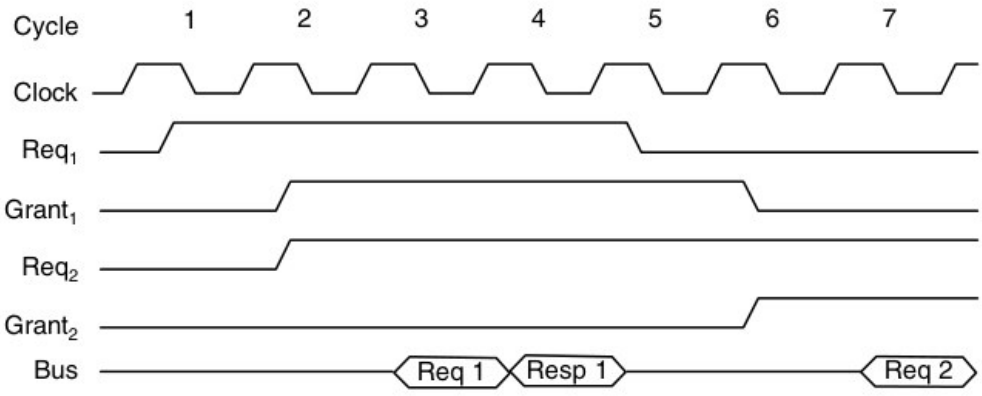


图 22.7 同步总线仲裁的请求、授权、事务与释放时序

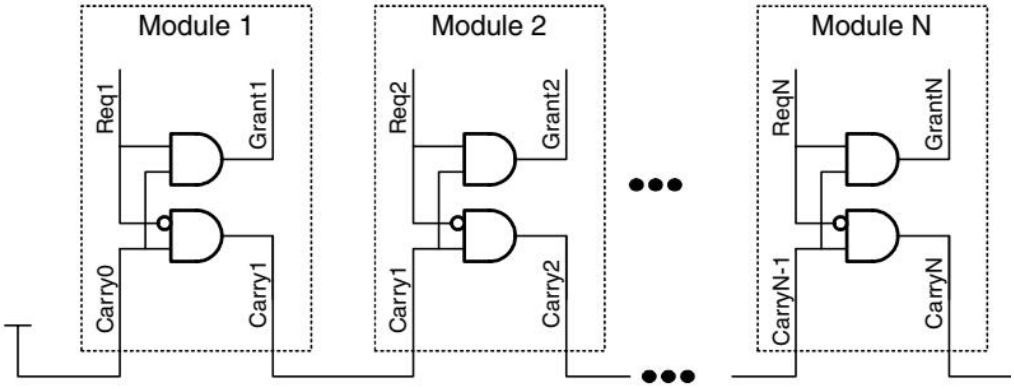


图 22.8　菊花链仲裁：固定优先级的仲裁逻辑分布于各模块；某模块不需要总线时，便将授权信号传递给下一个模块。

器所驱动信号的逻辑或（也可能采用反逻辑）。可以利用这一特性，按模块优先级地址从高到低逐位仲裁，如图 22.9 的算法所示。每一轮中，所有仍在竞争的模块都将自己的优先级值驱动到总线上，总线对这些值执行线或。模块随后比较当前位上的总线值与自己的优先级位；如果总线值更高，说明其他模块具有更高优先级，该模块退出竞争。遍历全部位后，只有优先级最高的模块留下，成为总线主设备。以下伪代码保持原书的信号命名：

表 22.1 给出了分布式仲裁示例：优先级分别为 9、10 和 7（二进制 1001、1010、0111）的三个模块竞争总线使用权。首轮各模块线或得到 1111，比较最高位后，优先级为 7 的模块退出。其余两个模块的线或结果为 1011；比较位 2 时没有模块退出。比较位 1 时，优先级为 9 的模块退出，最终优先级为 10 的模块胜出。

分布式仲裁使用的优先级可以任意分配，但必须互不相同。若每个模块以自身地址作为优先级，就构成固定优先级仲裁。若要实现“最近最少获得服务”方案，可以令当前总线主设备的优先级降为 0，而其他模块在下一轮仲裁前均提高一级。

分布式仲裁的优点是不需要中央仲裁逻辑，能够直接利用普通总线信号线；缺点是速度慢，且仲裁期间会占用总线，无法与事务并行执行仲裁。速度慢的原因是算法每一轮都必须让线或信号沿整条总线传播并稳定。相比之下，在高速辐射式仲裁中，请求信号汇集到中央仲裁器后，每轮只需几个逻辑门延迟，即数百皮秒而非数纳秒。

另一种曾在一些共享内存多处理器中使用的方法是复制仲裁器：每个总线模块上各放置一份仲裁器，把所有请求广播给全部仲裁器副本。系统复位时，各副本初始化为相同状态。每轮仲裁中，各副本都从同一状态出发，接收相同的请求集合并计算相同授权结果。获得授权的模块在本地使用这一结果，其他模块忽略它。

复制仲裁器消除了中央仲裁器，却带来若干明显缺点。首先，必须将全部请求分发到所有模块，增加总线信号线数量及请求线负载。更重要的是，它复制了状态，这是可靠设计的大忌：对 N 个模块有 N 份仲裁器状态。如果其中任意副本由于软错误或同步错误而状态分歧，就可能在同一周期给两个模块授予总线使用权。为防止这种情况，需要额外逻辑持续比较各副本状态并维持同步，明显增加复杂度。

表 22.1 分布式总线仲裁：优先级分别为 9、10、7 的三个模块的竞争过程

比较位	总线线或值	模块 9（1001）	模块 10（1010）	模块 7（0111）
3	1111	参与	参与	退出
2	1011	参与	参与	已退出
1	1011	退出	参与	已退出
0	1010	已退出	获胜	已退出

22.3 高性能总线协议

22.3.1 总线流水线化

图 22.7 中，总线经常因等待仲裁而空闲。对共享内存多处理器等几乎每笔事务都需要仲裁的应用，这会显著降低性能。解决办法是把总线事务的各个阶段流水化，提高总线的有效使用率。

图 22.10 给出存储器写事务（a）及具有一个周期访问延迟的读事务（b）的流水线时序与资源预约表。每张图上方列出完成事务所需的总线周期，左边列出涉及的资源。读事务首先经历三个仲裁周期：AR 发送仲裁请求，ARB 由仲裁器做出决策，AG 将授权结果返回请求者。

	1	2	3	4	5
AR	ARB	AG	RQ	ACK	
Arb req					
Arbiter					
Arb grant					
Bus					

图 22.10：存储器写事务（a）与读事务（b）的流水线阶段及资源预约表。顶部是总线周期，侧边是相关资源。深色方格表示该事务在相应周期独占某资源，浅色方格表示资源可共享。

	1	2	3	4	5	6
AR	ARB	AG	RQ	P	RPLY	
Arb req						
Arbiter						
Arb grant						
Bus						

仲裁结束后，读事务再经历请求周期 RQ（将地址发送给存储器模块）、处理周期 P（完成存储器访问）及应答周期 RPLY（将数据返回请求者）。

资源预约表显示了每个周期内哪些资源被占用。深色方格代表被事务独占的资源：某一周期内只能有一笔事务获得授权，也只能有一笔事务使用总线。浅色方格代表可共享资源：任意多笔事务都可在同一周期提出仲裁请求并参与仲裁。

资源预约表提供了发起事务的简单规则：对于固定延迟的事务，只要其预约表所要求的独占资源没有被先前事务预留，就可在该周期启动。应用此规则可知，一笔读事务可以在另一笔读事务的下一个周期启动，但第三笔读事务必须在前两笔之后再等待两个周期；而写事务必须在读事务启动后三个周期才能发起。

图 22.11 展示六笔事务在流水化总线上的时序：两笔写事务各需五个周期，四笔读事务各需六个周期。如果每笔事务完成后才开始下一笔的仲裁，即不采用流水化，总共需要 34 个周期。流水化总线则通过阶段重叠完全隐藏了仲裁延迟，有时还可将互不冲突的事务阶段叠加。例如周期 12 中，Read 5 的 RQ 请求与 Read 4 的 P 处理阶段同时进行。

不过，若读事务后接写事务，二者流水线阶段不匹配，总线在读事务的 P 周期仍会空闲。例如，Write 2 必须等到周期 7 才能发出请求，以免其 ACK 与 Read 1 的 RPLY 冲突。对延迟可变的事务，情况更严重：若某次读访问的处理阶段可能持续 0～20 个周期，则每笔读事务完成前都难以启动其他事务，造成处理阶段期间总线空闲。

图 22.11：四笔读事务和两笔写事务在流水化总线上的执行时序。各事务遵循图 22.10 的时序；流水化执行共 17 周期，非流水化执行需 34 周期。

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Read 1	AR	ARB	AG	RQ	P	RPLY											
Write 2		AR	ARB	AG	Stall	Stall	RQ	ACK									
Write 3			AR	ARB	Stall	Stall	AG	Stall	RQ	ACK							
Read 4				AR	Stall	Stall	ARB	Stall	AG	Stall	RQ	P	RPLY				
Read 5							AR	Stall	ARB	Stall	AG	RQ	P	RPLY			
Read 6									AR	Stall	ARB	AG	Stall	Stall	RQ	P	RPLY
Bus busy																	

22.3 High Performance Bus Protocol

22.3.2　拆分事务总线

把一次事务拆成两个事务，并允许总线在两条消息之间重新仲裁，就能消除等待响应期间的总线空闲。分离事务总线（split-transaction bus）通常正是为了在请求与应答之间存在较长、可变等待时间时释放总线。应答消息也被视为必须先仲裁并获得总线使用权的独立事务，因此等待期间总线可以服务其他事务。

图 22.12 将图 22.11 的同一事务序列放到分离事务总线上执行。原来的每笔事务都被拆成两笔：第一笔仲裁总线并发送请求 RQ，第二笔重新仲裁总线并发送应答 RPLY 或确认 ACK。例如，Read 1 在周期 4 发出 RQ 后即结束其请求事务；其应答由 Rply 1 事务在周期 8 发出。

由于这里假设仲裁需要三个周期，请求与应答或确认之间的最短延迟是四个周期，这

显著增加了单次操作的延迟。换来的好处是总线利用率得到最大化：只要还有等待发起的事务，每个周期总线都可用于发送 RQ、RPLY 或 ACK。

图 22.13 表明，在请求和响应之间存在较长且可变的延迟时，分离事务能显著提高吞吐量。图中有三笔事务，每笔的完成时间可从一个周期（无等待）到六个周期不等。不使用分离事务的流水化总线必须将它们串行执行，如图 22.13a 所示。

```
For module i
    bit = n-1 ;                          bit of priority currently being tested
    participating = 1                    this module still a contender
    priority = pri(i)                    priority of module i
    while (bit>=0 && participating)      for each bit
        drive priority onto bus          one bus cycle here
        if (bus[bit] > priority[bit])    someone has higher priority
            participating = 0            drop out of competition
        bit = bit-1                      next bit
    master = participating               only winning module still participating
```

图 22.9 使用线或（wired-OR）总线执行分布式仲裁的算法

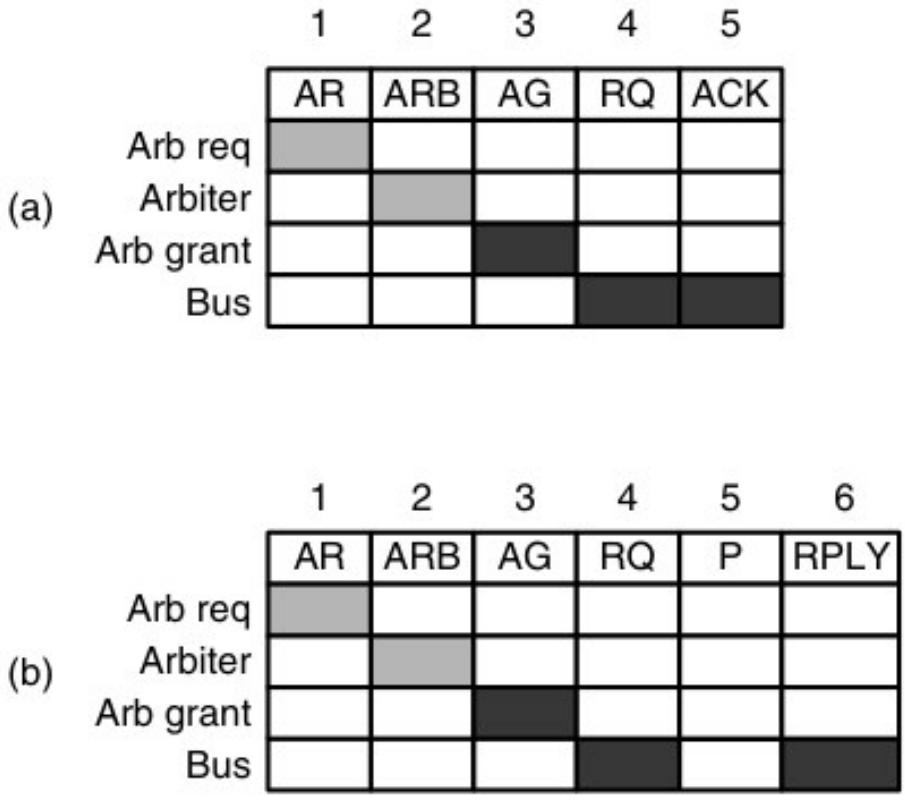


图 22.10 存储器读写事务的总线流水线时序和预约表

因为每笔事务必须从请求之后的周期起为自己的应答预留总线。流水化虽然能隐藏仲裁延迟（图中未画出），却不能使这些事务彼此重叠。

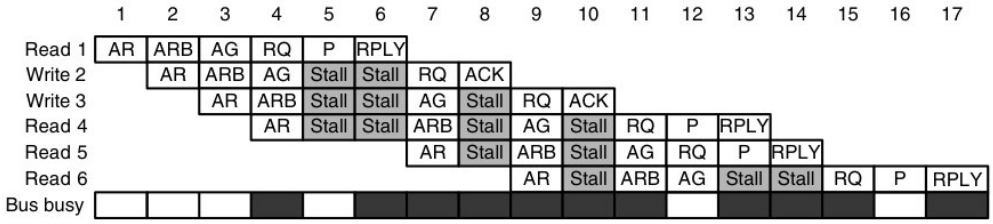


图 22.11　流水线化总线上四次读与两次写操作的执行时序

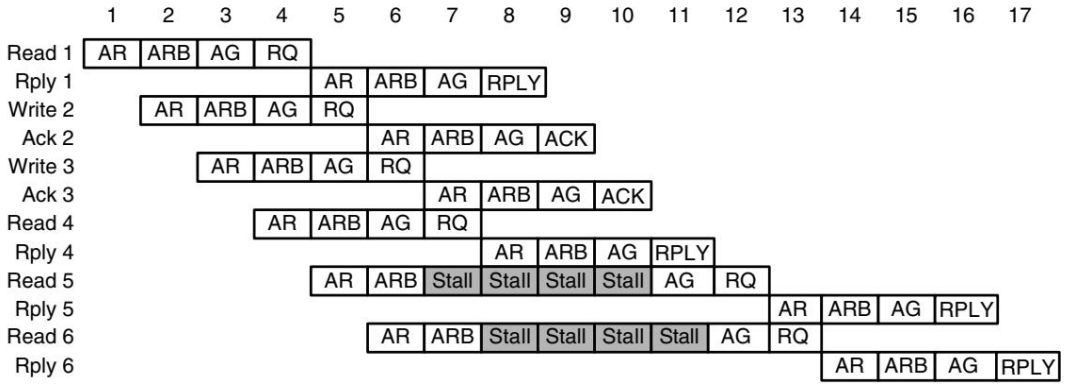


图 22.12　分离事务总线上的读写执行示例：操作延迟可能增大，但可减少总线空闲周期。

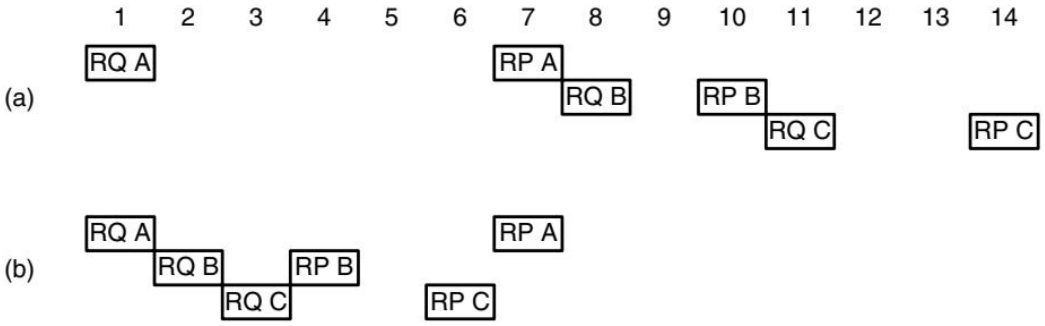


图 22.13　可变延迟事务的执行：（a）流水化总线；（b）分离事务总线。

使用分离事务总线时，三笔请求可以完全重叠，如图 22.13b 所示。由于 RQ A 无须为其应答保留总线，RQ B、RQ C 及其各自的响应，都可以在第一笔请求等待期间发送。如果 RP C 恰好延迟一个周期，就会与 RP A 在同一周期准备好发送；仲裁器只会授权其中一个，另一个应答需要等待一个周期。

与一般互连网络一样，分离事务总线必须能识别每条应答对应的请求。在非分离事务总线上，回复出现的时间就能确定其所属请求；在分离事务总线上，仲裁可能改变应答顺序，例如图 22.13 的 RP C 和 RP A 完全可能以相反顺序出现。

为识别应答，每条请求都携带一个标签，使其与所有尚未收到应答的请求唯一地区分开来。请求者在请求消息中发送标签，响应者保存该标签，并将它放入回复消息中。请求者通过匹配标签确定回复属于哪一笔请求。

在某些总线上，标签作为请求消息的独立字段显式传输，例如早期 Sequent Symmetry 多处理器总线。该系统在请求者赢得仲裁时，从中央标签池给它分配标签。其他系统（如 SCSI 外设总线）则隐式地利用请求中的源地址、目的地址和序列号共同构成标签。

22.3.3 突发消息

总线事务的开销相当可观。每次事务都必须进行仲裁、寻址和某种形式的确认。如果每笔事务仅传一个字，那么协议开销很容易超过有效载荷本身，即开销可能高于 100%。

增大每条消息传输的数据量，可以降低以百分比表示的事务开销。例如，在加载或写回缓存行、向外设传送数据块时，每条消息发送多个字，远比每条只发送一个字高效。选择每条消息的数据传输量，本质上相当于选择总线的 flit 大小。

图 22.14 说明突发（burst）消息如何降低开销。图 22.14a 中每条消息需要两个额外周期，却只传一个字：三个周期中只有一个用于有效数据，效率为 $1/3$。图 22.14b 中一条消息传四个字，效率提升到 $2/3$，即六周期传四字。若一条消息传八个字，效率将升至 $4/5$。一般而言，若开销为 x 周期、传输 n 个数据字，则效率为 $n/(n+x)$。

直觉上，为了在大块数据传输中使效率尽量接近 1，似乎应选择尽可能大的突发长度。然而，突发越长，高优先级请求者在得到总线使用权之前可能等待得越久。若请求在图 22.14b 的消息首周期提出，就要等六周期才能获授权；若突发长度是 256 个字，就可能等 258 周期。很多应用无法接受如此长的仲裁延迟。

为同时获得长突发消息的效率和高优先级请求的低延迟，一些总线允许中断突发传输，随后恢复或从头重启。图 22.15 显示消息 A（八个数据字）被单字消息 B 中断，然后继续传输。恢复消息的第一个忙周期发送一个标识标签，指明正在恢复哪条消息；如果中断消息本身还会被中断，就可能有多条消息等待恢复，因此必须有这样的标识。另一种方案是放弃被中断消息，并从头重新传输。

图 22.14：突发传输消息。（a）三个周期仅传一个字，效率 $1/3$；（b）两个周期开销由四个数据字分摊，效率提高到 $2/3$。

Arb	Cmd	Addr	Data	Cmd	Addr	Data	Cmd	Addr	Data	Cmd	Addr	Data
Request												
Arb	Cmd	Addr	Data	Data	Data	Data						
Burst request												

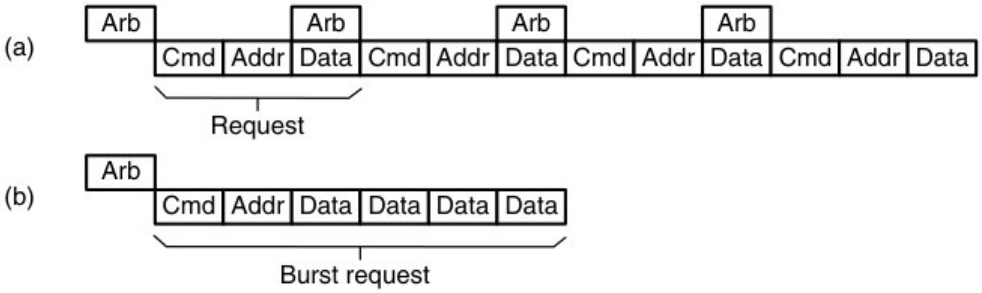


图 22.14　普通数据传输与突发传输的时序和效率比较

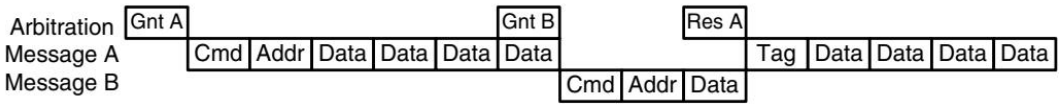


图 22.15　突发传输的中断与恢复：消息 A 被消息 B 中断，随后继续传送。

但是，“中断并从头重启”方案要求重复传送一部分数据，因此可能严重损害性能。中断长消息，类似把长数据包分成多个 flit，使高优先级数据包不用等到整个低优先级数据包传送完毕，便能竞争通道带宽。主要区别是：数据包中的每个 flit 都要承担 flit 级开销；总线消息恢复的额外开销则只在消息真正被中断时才产生。

22.4　从总线到网络

许多过去由总线承担的通信任务，如今已改由网络完成。迁移过程中，设计者常试图在点对点网络上复制总线的特性。某些特性（如所有模块共享统一接口）很容易模拟；另一些特性（如全部事务的全局串行化和向所有模块广播）则更难实现。

总线的主要优点之一，是为所有通信模块提供统一接口，促进模块化和互操作。例如，原书出版时期的许多小型计算机使用 PCI（外围组件互连）总线连接外设。众多计算机与外设按同一总线规范设计，因此 PCI 外设可以与任意符合该规范的计算机互连。多数网络也能提供类似的统一接口。网络可为所有连接模块提供公用接口（见第 20 章）。虽然此类接口在原书所述时期尚未完全标准化，但 PCI Express、InfiniBand 和光纤通道交换结构等标准正在改变这一状况。统一总线接口带来的模块化优势，正逐渐由互连网络继承。

一些系统希望保留总线的串行化和广播特性，却无法使用共享物理媒介，因为电气约束不允许。例如，片上互连的寄生电阻和电容，要求每隔数毫米就插入中继器才能维

持信号速度；因此，不可能把一条大型片上总线实现为可由众多模块共同驱动和采样的单一电气节点。

图 22.16 展示了一种在逻辑上等价于总线、却完全由短距离点对点链路构成的通信系统。每个模块在发送使能时将信号驱动到或网络，在未使能时向或网络驱动零。因此，或网络的输出总是当前获得发送权的模块所驱动的值，再通过中继器链分发给所有模块。若将线性的或链和中继器链改为树形网络，就能在一定程度上降低传播延迟（见习题 22.5）。

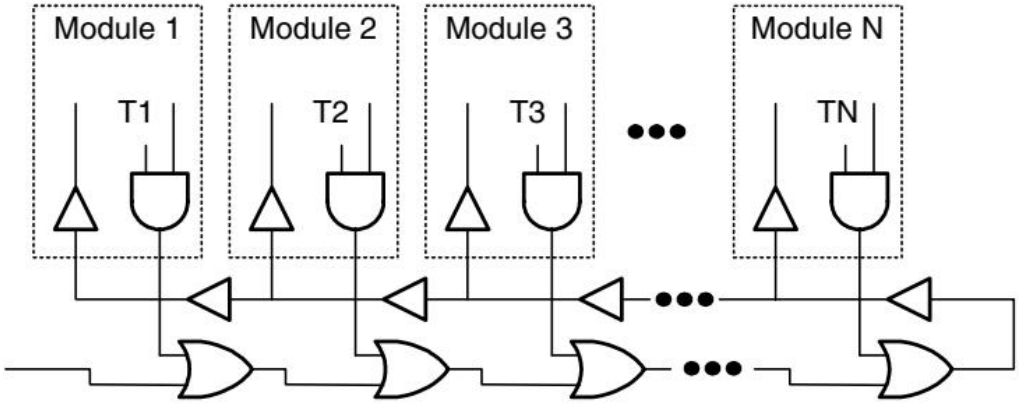


图 22.16 通过对模块发送输出进行逻辑或运算，实现无需共享物理媒介的逻辑总线。

一些系统希望维持串行广播总线的语义，例如支持监听式缓存一致性协议，但又无法接受任一时刻只传一条消息的性能限制。典型的第一步是加宽总线，例如使用 256 位宽度以便并行传输 32 字节缓存行。然而，这虽然加快了大块数据传输，却没有解决多个地址周期需要同时进行的问题。

进一步提高性能的办法，是使用多条并行总线，或至少使用多条并行地址总线。例如，四条并行地址总线可以同时执行四个地址周期。通过给并行总线分配确定顺序，仍然可以维持串行化语义：同一周期 t 中，总线 i 上到达的地址先于编号更大的总线 j 上到达的地址。不过接口成本会随总线数量超线性增长，因为需要逻辑和连接检查每条地址与所有更早总线上的地址是否冲突。这种提高带宽的方法代价高昂，但也曾成功使用多年，例如 Sun Ultra Enterprise 10000 服务器采用了四路复制总线系统 [35]。

性能扩展的最终一步，是放弃总线语义，直接采用网络。这在性能、成本和可扩展性方面均有优势，并能避免为大多数不需要广播的消息承担全网络广播开销。确实需要广播时，可采用硬件多播，或通过分发树发送多条消息。需要串行化时，可将相关消息发送到共同节点，按到达顺序排列；它只串行化相关消息，而不像总线那样无区别地串行化所有消息。

部分传统总线协议还要求消息保持顺序。例如，在共享内存机器中，处理器 P 发往存储器 M 的两条写消息 A、B，必须按照发送顺序交付，否则存储器状态可能错误。总线一次只能完成一项通信，因此天然保证消息顺序；网络中后发的 B 却可能在某处超过 A。网络可以让需要排序的消息始终使用相同路径（及相同虚拟通道），也可以为消息标序列号并在目的节点重新排序。

当然，在网络上强行运行总线协议，有点像给喷气发动机装螺旋桨。如果系统本来就围绕互连网络设计，最好针对网络特性优化通信协议，而不是勉强套用为串行广播媒介设计的协议。例如，原书所述的大型共享内存系统已采用一般互连网络，并运行不依赖广播的目录式一致性协议。

22.5　案例研究：PCI 总线

在原书出版时期，PCI 总线 [143] 是使用最广泛的总线之一，用来连接从嵌入式系统到大型服务器等不同计算机的外设，也综合体现了本章讨论的许多总线特性。

PCI 是采用地址／数据复用线和流水化仲裁的同步总线，提供 32 位及 64 位版本，常见频率为 33、66 和 133 MHz。在 64 位、133 MHz 的最高配置下，可维持约 1 GB/s 的突发传输速率。

表 22.2　PCI 总线主要信号

信号	功能说明
CLK	总线时钟，所有总线信号均在其上升沿采样。
AD[31:0]	32 位地址/数据复用总线。地址周期由发起方驱动地址，数据周期由写入方或读取目标驱动数据。
C/BE[3:0]	命令/字节使能。地址周期表示总线命令，数据周期指示有效字节。
FRAME#	发起方用以开始事务；撤销表示最后一个数据周期开始。
IRDY#	发起方就绪信号；只有它与 TRDY# 同时有效才传输数据。
TRDY#	目标设备就绪信号。
REQ#	模块向集中式仲裁器请求总线使用权。
GNT#	集中式仲裁器发出的总线授权信号。

一笔 PCI 事务首先经历一个地址周期，之后是一个或多个数据周期。根据发起者与目标设备的响应速度，地址周期和数据周期之间还可以插入等待周期。PCI 同时支持单字与突发事务。图 22.17 给出了读取两个数据字的 PCI 事务：周期 1，主设备（发起者）置位 FRAME#，并在 AD 线上驱动地址，在 C/BE 线上驱动总线命令（存储器读）。周期 2，主设备置位 IRDY#，表示准备接收第一个数据字；但由于目标设备尚未置位 TRDY#，该周期没有数据传输。这个空闲周期对于读操作是必要的，以便 AD 总线从主设备驱动切换到目标设备驱动，避免两个驱动器同时工作。周期 3，目标设备提供数据并置位 TRDY#，由于 IRDY# 同时有效，第一个数据字成功传输。周期 4，目标设备提供第二个字并保持 TRDY# 有效，但此时 IRDY# 无效，因此传输必须等待。周期 5，主设备再次置位 IRDY#，第二个字传输完成，同时撤销 FRAME#，表明这是本事务的最后一次数据传输。周期 6，FRAME# 和 IRDY# 均无效，表示事务结束；下一总线主设备随后可以在周期 7 开始新事务。

写事务与图 22.17 的读事务基本相同，但在数据阶段由发起者而非目标设备驱动 AD

信号线。由于地址和数据都由同一模块（发起者）驱动，写事务在地址周期之后无须用于切换总线驱动方向的空闲周期。

图 22.18 展示 PCI 仲裁如何流水化。周期 1 中，模块 1 和模块 2 都置位各自的 REQ# 信号，请求成为主设备。全局仲裁器决定将总线授予模块 1，并在周期 2 置位 GNT1#；模块 1 收到授权后，于周期 3 置位 FRAME# 发起事务。模块 1 一开始执行事务，仲裁器就在周期 4 撤销 GNT1# 并置位 GNT2#，通知模块 2 当前事务完成后可获得总线。模块 1 继续执行，在周期 5 同时撤销 FRAME# 和 IRDY#，表示事务结束。模块 2 检测到这一状态，于周期 6 置位 FRAME# 开始自己的事务。

PCI 事务不是分离事务。一次读事务同时包含发起者发往目标设备的请求和目标设备返回的应答。不过，PCI 可以中止并重试事务，防止长延迟操作持续占用总线。目标设备可先锁存请求信息，再置位 STOP# 而非 TRDY#，中止当前事务。主设备随后择机重试，期望目标设备此时已完成长延迟操作并能提供所需数据。PCI 规范称之为“延迟事务”。它与分离事务一样，允许其他事务在长延迟操作期间继续运行；但其效率明显低于专门的分离事务机制，因此只适合处理例外情况，而不是像分离事务那样处理大多数总线请求。

PCI 规范还包含一组模块初始化协议。除地址周期中通过 AD 总线寻址外，还可通过辐射式选择信号 IDSEL，按插槽编号选定模块。系统利用 IDSEL 寻址执行配置读写周期，读写模块的配置寄存器。典型初始化过程先读取若干配置寄存器以识别模块类型，再由控制器写入配置寄存器，设定模块地址及各类选项。

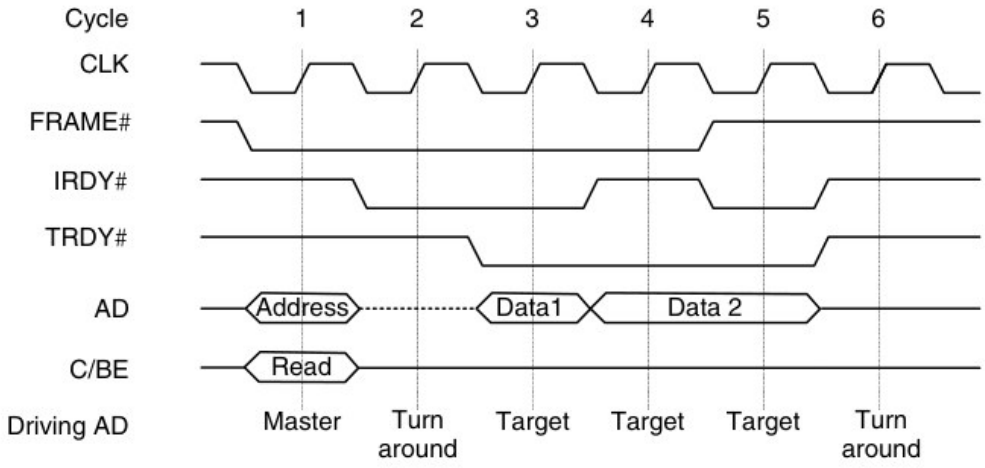


图 22.17 PCI 总线读事务的时序

22.6 文献注释

总线技术历史悠久，很难确定其确切起源；20 世纪 40～50 年代的早期计算机中已经使用总线。Digital 的 Unibus 推动了总线作为标准外设接口的普及。PCI [143] 和 SCSI [173] 是原书出版时期的典型外设总线。20 世纪 60～80 年代，多数小型机和微机用总线连接 CPU 与存储器，之后现代 PC 逐渐改为经北桥芯片实现点对点连接。Rambus

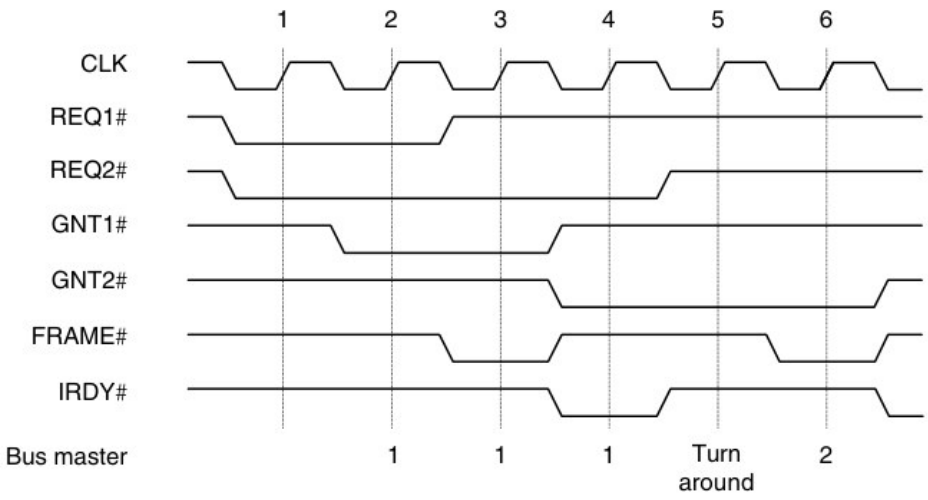


图 22.18　PCI 总线仲裁时序

DRDRAM 总线 [41] 是连接高带宽 DRAM 的一个高性能实例。Sonics 片上网络使用或树实现基于点对点链路的逻辑总线 [193]；Sun Ultra Enterprise 10000 [35] 同样用点对点互连实现总线，并通过多条地址总线提高吞吐量。上述例子也从侧面说明了为何网络常优于共享总线。更全面的总线技术综述见《Digital Bus Handbook》[72]。

22.7　习题

22.1 复用总线与非复用总线。假设所有总线事务都需要主设备向从设备发送包含 32 位地址和 4 位控制字段的地址消息。读事务占全部事务的 70%，由从设备向主设备返回 32 位数据；写事务占 30%，由主设备向从设备发送 32 位数据。如果资源允许，写数据可以与地址同时传送。设计总线时一共有 40 条信号线可用。请比较地址／数据复用与非复用两种总线，在充分流水化的假设下，设计吞吐量最大的方案。

22.2 提前决胜的分布式仲裁。图 22.9 的分布式仲裁在模块地址有 b 位时，需要 b 个周期。说明在哪些情况下可以更早结束仲裁。令 $b = 8$，模拟 i 个模块同时随机请求总线 $(0 < i \leq b)$，绘制平均仲裁时间随 i 变化的曲线。

22.3 独立回复通道的流水化总线。对图 22.11 的事务序列，画出使用独立回复总线后的时序图，假设回复总线独立仲裁。比较该方案与原方案完成整个序列所需的时间。

22.4 总线上的虚拟通道。假设总线传送的消息被分为多个 flit，每个 flit 都携带虚拟通道编号，以便较短的高优先级消息中断较长的消息。给出协议开销和最大等待时间随 flit 大小变化的曲线。设最大消息长度为 64 KB，支持 8 条虚拟通道。

22.5 树形总线。说明如何将图 22.16 中使用点对点链路实现的逻辑总线改为树形的或网络及中继器网络，以降低延迟。设网络位于一块 12 mm 见方芯片上，每个模块 2 mm 见方；为了维持信号速度，线路上每 2 mm 需要设置一个中继器或逻辑门。假设信号通过 2 mm 导线和一个逻辑门各用一个时间单位。比较 36 模块系统在线性结构与树形结构中的最大延迟。

22.6 仿真。分离事务总线除了用标签关联请求与应答外，也可强制要求应答与请求以相同顺序出现，使每条应答能按其在应答流中的位置识别。设事务延迟在 1 至 $T_{\max}$ 周期之间均匀分布。通过仿真比较采用标签和采用有序应答的总线平均延迟。

总线章节原著注释

1. 为容忍不同模块之间的时钟偏斜，NuBus 一类协议在时钟周期约四分之三处采样信号，使信号有足够时间传播并稳定。

2. PCI 信号名中的井号 “#” 表示该信号为低电平有效。

第二十三章　性能分析

从本书前面的讨论可知，互连网络设计以性能需求为起点，同时受到封装与成本约束。这些条件共同决定特定网络应采用什么样的拓扑、路由算法和流控机制。在网络设计的初期，我们用零负载延迟和吞吐量等简单指标来指导上述选择。这些指标当然依赖若干假设；例如，在多数拓扑和路由分析中，我们假设流控完全理想。早期使用简化模型是合理的，但要准确描述网络的真实性能，就必须采用更精细的模型。

本章首先定义网络性能的基本度量，并说明如何估计与解释它们。接着介绍用于高层次网络行为建模的基本解析工具，包括排队模型与概率方法。第 24 章讨论详细网络模拟器的使用，第 25 章则展示多种网络配置的仿真结果。

图 23.1 给出了互连网络性能测量的标准实验配置。要测量互连网络性能，我们在网络的每个终端或端口连接测试仪器。每个源端都有一个数据包发生器和源队列，目的端则设有统计接收数据包的仪器。数据包发生器按指定的流量模式与注入过程产生数据包，源队列在网络暂时无法接收数据包时保存这些数据包。这样，输入业务量就可以独立于网络本身进行控制。

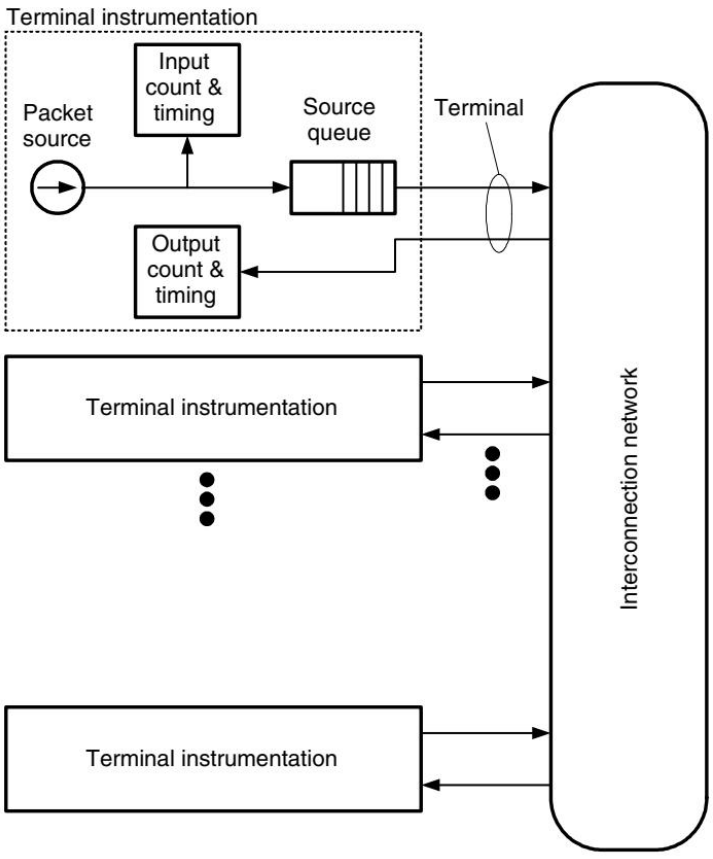


图 23.1　互连网络性能测量的标准实验配置。

这种开环测量配置使流量参数能够独立于网络进行控制。如果没有源队列，数据包源就只能在网络准备好接受新包时才生成数据包，从而导致注入流量受到网络负载影响。在拥塞条件下，这样的反馈会把网络的性能缺陷隐藏起来。用开环方法测量时，测量者能够指定网络所承受的业务需求，并单独观察其已接纳流量与延迟。

闭环测量系统中，网络本身会反过来影响流量，因此适合测量整个系统的性能。例如，可以通过运行实际应用来估计多计算机系统的性能；应用发出新通信请求的时间，可能取决于先前请求是否已完成。在这类闭环实验中，网络延迟会影响应用行为，因此不容易将网络性能与应用的其他因素分离。

我们通常最关注网络的稳态性能，即在稳定统计特性的流量源作用下，网络达到平衡之后的表现。当网络中数据包数量的统计分布不再随时间变化时，可认为网络已经达到平衡。不同初始状态可能产生一段暂态，因此不能把网络启动后的全部观测值都直接用于稳态统计。

测量稳态性能时，实验（或仿真）通常分为预热、测量和排空三个阶段。首先运行 N_1 个预热周期，使网络接近平衡；随后运行 N_2 个测量周期，只在这一阶段统计目标性能指标；最后继续运行一段排空周期，以便已进入网络的测量数据包能够送达目的地。在预热和排空阶段，流量源仍会产生数据包，但这些数据包本身不计入测量指标。设计实验时应充分选择预热长度，使网络达到稳态，并注意准确记录跨阶段边界的包。在预热和排空阶段，数据包源仍继续产生数据包，但这些包不参与统计；它们仍会影响网络状态，因而对测量期的数据包延迟可能产生影响。

23.1 互连网络性能指标

虽然针对某个网络可以采用多种性能测量与呈现方式，本章着重讨论三个基本量：吞吐量、延迟和容错性。它们的名称看似通用，但精确定义高度依赖具体测量方法，尤其是测量系统的设置。

23.1.1 吞吐量

吞吐量是网络在给定流量模式下成功交付数据包的速率。测量方法是在固定时间窗口内统计各目的节点收到的数据包数，并换算为单位时间的包数、字节数或位数。计算吞吐量时，应只把真正送达目的端的数据包计入。

为了分析吞吐量与业务需求的关系，通常绘制“已接纳流量”（吞吐量）随“提供流量”（网络输入需求）变化的曲线，如图 23.2 所示。负载低于饱和点时，已接纳吞吐量随提供负载近似线性增长；接近网络容量时，由于通道和路由器资源争用，增长开始减缓。超过饱和点后，提供负载继续提高却不会带来相应吞吐量增长，某些不稳定网络的实际吞吐量甚至会下降。

通常将吞吐量表示为网络容量的比例。这既方便直观理解网络性能，也可比较不同规模和拓扑的网络。例如，图 23.2 中八元二维网格的总容量为 $\Theta_{U,M} = 4b/k = b/2$，其中 b 为通道带宽。所有源节点对提供流量贡献相同，因此若提供流量为容量的 λ 倍，

每个源以 $b\lambda/2$ 的速率生成数据包。该网络在容量的 43% 附近达到饱和。

对于某些不稳定网络，超过饱和点后成功交付的数据包总数可能仍然不变，但这些数据包的分布已不再符合指定流量模式。要正确测量这类网络的吞吐量，必须针对指定的流量模式进行衡量：在所有网络输入端施加该流量，并分别测量每条业务流的已接纳流量。符合指定流量模式的吞吐量应取所有业务流中“已接纳／提供流量”比例的最小值。

从数学上看，设流量矩阵 Λ 的每一行、每一列之和均为 1。当提供流量为 $\alpha\Lambda$ 时，源节点 i 以速率 $\alpha\lambda_{ij}$ 生成发往目的节点 j 的数据包。目的节点记录各来源到达的流量，并计算每条业务流的速率 λ'_{ij}。吞吐量定义为各业务流的 λ' 与 λ 之比中的最小值：

$$\Theta = \min_{ij} \left(\frac{\lambda'_{ij}}{\lambda_{ij}} \right). \tag{23.1}$$

当提供需求超过饱和点时，可将目的节点实际收到的流量矩阵表示为：

$$\Lambda' = \Theta\Lambda + X \tag{23.2}$$

也就是说，交付的流量包含 Θ 个单位的目标流量模式，加上额外流量 X（严格正矩阵）。测量吞吐量时，额外流量不予计入，因为它并不符合所指定的流量模式。

网络不稳定往往源于公平性问题。过去的研究在超过饱和点后常只报告所有业务流的平均已接纳流量，事实上把额外流量 X 也算作了性能。若改为报告各业务流中最小的已接纳比例，就能发现随着负载上升是否有业务流发生饥饿。饥饿通常由不公平的流控造成。图 23.2 中饱和点之后已接纳流量大致保持不变，表明即使超过饱和负载，该网络仍能公平分配带宽；而图 23.3 中饱和后吞吐量急剧下降，提示流控可能不公平。即使网络在业务流之间公平分配带宽，提供流量提高时，已接纳流量也可能下降。这种现象可能由非最短路径路由造成。由于路由决策基于不完全信息，饱和点之后竞争加剧、误路由可能增多，使每个数据包的平均跳数增加，从而进一步增加通道负载。如果路由算法没有专门防范这种反馈，饱和后的吞吐量就可能迅速降低。

采用无死锁逃逸路径的自适应路由算法也可能出现类似情况。提供流量增加后，更多数据包被迫转入逃逸路径，最终已接纳吞吐量会向逃逸路由算法的吞吐量退化，而不是维持自适应算法的性能。第 25.2 节将进一步讨论。

“已接纳流量—提供流量”曲线能细致展示网络在某一给定流量模式下的吞吐量表现。若要快速比较大量不同流量模式，可绘制这些模式的饱和吞吐量直方图。典型实验会生成大量随机置换流量模式，例如 10^3～10^6 个。第 25.1 节给出了相关示例。

23.1.2　延迟

延迟指数据包从源节点穿越网络到达目的节点所需的时间。此前我们对延迟的讨论主要集中于网络零负载延迟，未考虑多个数据包共享资源时产生的竞争延迟。将竞争延迟纳入模型或仿真后，延迟就成为提供流量的函数，适合画成曲线。图 23.4 给出一个“延迟—提供流量”示例。

测量延迟时，仍采用图 23.1 的实验配置。通常让提供流量 $\alpha\Lambda$ 从 $\alpha = 0$ 扫描至饱和吞吐量 $\alpha = \Theta$。当 $\alpha > \Theta$ 时，排队延迟理论上趋于无穷，无法进行稳态测量。对单个数据包，从源节点产生消息第一个位的时刻计时，到消息最后一个位离开网络的时刻为止。总体延迟取所有数据包的平均值。有时也可报告延迟直方图、最坏情况延迟以及个别业务流的延迟统计。与吞吐量一样，提供流量用网络容量的比例表示。

“延迟—提供流量”曲线通常具有典型形状：从零负载延迟的水平渐近线附近起步，随负载增大向上弯曲，并在饱和吞吐量处趋近竖直渐近线。低负载时，延迟接近零负载延迟。

当流量增加时，数据包必须等待缓冲区和通道资源，竞争延迟随之增大。提供流量接近网络的饱和吞吐量时，延迟曲线逐渐趋于竖直。基础排队论（第 23.2.1 节）能在很大程度上解释曲线形状。

平均数据包延迟通常很有用，但研究所有数据包或某类数据包的延迟分布也能揭示更多问题。例如，使用虚拟通道流控时，可保留部分通道专供高优先级业务使用；把高优先级和普通业务的延迟分布分别绘图，就可以判断优先级机制是否有效。还可只考察某一源—目的节点对之间的所有数据包，这对非最短路径路由算法特别有帮助，能够反映该节点对之间路由长度的一般情况。第 25.2 节包含多个延迟分布图示。

23.1.3 容错性

容错性指网络在存在一个或多个故障时仍能完成通信任务的能力。最重要的问题是：发生故障后网络是否仍能运行。如果网络在任意单个节点或通道失效时，仍能在所有正常终端之间交付数据包，就称为单点容错网络。对许多系统，具备单点容错能力就足够，因为单个故障概率已很低，同时发生多个故障的概率更低。此外，高可用性系统通常会主动维护，在下一个故障发生之前更换坏组件。

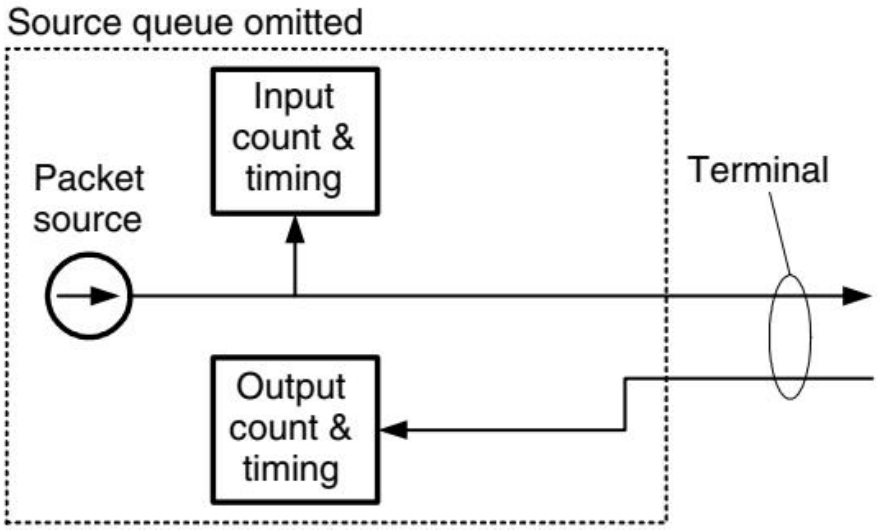
容错要求会影响许多网络设计决策。例如，需要单点容错的网络不能只使用确定性路由，否则一条通道故障就可能断开所有经过它通信的节点对。若网络性能随故障数量增加而逐步下降，而不是突然失效，就称为渐进退化。第 25.3 节将研究容错网络的性能。

23.1.4 常见测量陷阱

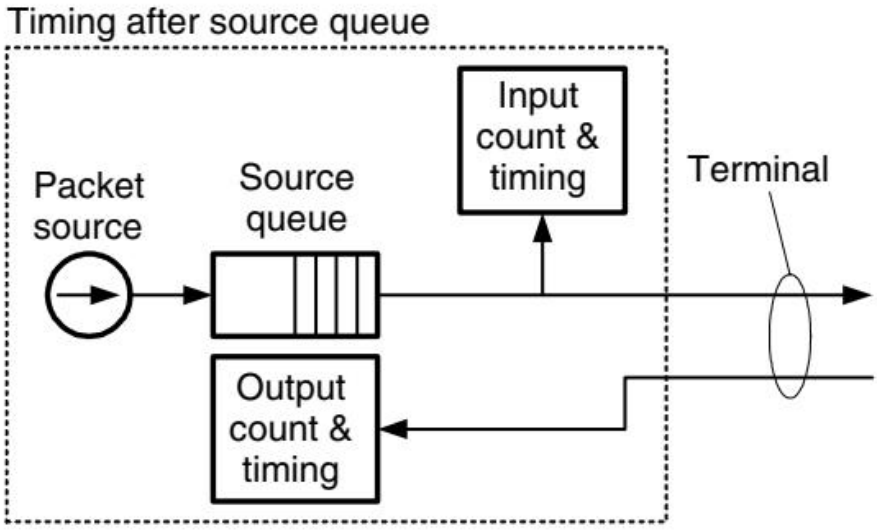
在互连网络性能测量中，有几类常见错误反复出现。作者坦言自己也曾犯过所有这些错误，希望读者能从中吸取经验，避免重蹈覆辙。

遗漏源队列。测量网络性能时，经常没有正确使用源队列。图 23.5 展示两种错误做法。图 23.5a 完全没有源队列：若源端生成数据包时输入端口正忙，只能丢包，或等输入端口可用时再注入。如果选择等待，则后续数据包的生成也必须推迟，否则仍然需要一个队列。无论哪种方式，网络中的竞争都会改变输入流量模式。图 23.5b 虽有源队列，却从源队列输出端而非输入端开始计数和计时。

在不设无限源队列的条件下尝试测量网络性能，会把系统变成闭环，导致输入流量模


Source queue omitted
Input count & timing
Packet source
Terminal
Output count & timing

(a)


Timing after source queue
Input count & timing
Packet source
Source queue
Terminal
Output count & timing

(b)

式偏离目标模式，同时漏掉进入网络之前的等待延迟。(a) 输入通道繁忙时丢包或延后注入；(b) 从源队列输出端而非输入端开始统计和计时。

省略源队列的测量有两个原因导致结果无效。第一，测量点处施加的流量模式与预期模式不同。考虑图 23.6 的两节点网络：源端本应从节点 1 到 0、从 0 到 1 各注入一个单位的流量，但如果在源队列之后统计，就只会记录到从 0 到 1 实际被接纳的 0.1 个单位流量，以及反方向的一个单位流量。

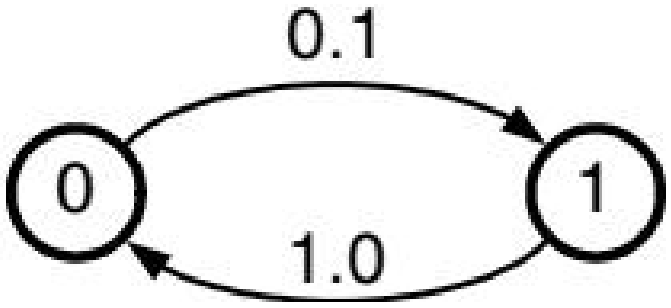


图 23.6：两节点网络，其中一条慢链路。计划施加的模式是双向各一单位流量；若在源队列输出端测量，实际得到的模式却变为 0→1 仅 0.1 单位、1→0 为 1 单位。

这显然不是我们原本希望测量的流量模式。

第二，省略源队列还会低估数据包延迟。在队列较浅且使用阻塞式流控的网络中，高负载时相当大一部分竞争延迟发生在源队列内；数据包一旦真正进入网络，反而可能很快到达。若忽略等待输入通道的时间，就会大幅低估从数据包生成到交付的真实延迟。

用平均交付流量代替最小交付流量。如果说省略源队列会扭曲网络输入端的测量，那么对所有业务流采用平均交付流量而非最小交付比例，就会扭曲输出端的测量。根本问题仍是实际测量的流量模式并非预先指定的模式。

考虑图 23.6 的网络。如果对两个目的节点的已接纳流量取平均，就会得出网络在 0.55 个单位流量处饱和的结论，即 1 和 0.1 的平均值。然而，这一结论不正确：网络没有以预定比例交付目标业务。

$$\Lambda = 0.55 \begin{bmatrix} 01 \\ 10 \end{bmatrix}, \tag{23.3}$$

but rather is delivering

$$\Lambda = 0.1 \begin{bmatrix} 01 \\ 10 \end{bmatrix} + \begin{bmatrix} 00 \\ 0.90 \end{bmatrix}. \tag{23.4}$$

换言之，网络交付的是 0.1 单位的指定流量模式，外加一部分其他流量。对各业务流取最小交付比例，才能得到真实情况。

在高负载下部分业务流饥饿的网络中，报告平均交付流量可能完全掩盖问题。例如，混沌路由 [26] 对已进入网络的数据包赋予高于新注入包的绝对优先级。对某些流量

模式，某些输入节点的业务流在高负载下可能彻底饥饿，因为它们无法并入忙碌通道。仅报告平均交付流量会让网络看起来很稳定，在超过饱和点后仍维持峰值吞吐量；但实际上部分业务流在这些负载下完全得不到服务，因此网络并不稳定。

把延迟和已接纳流量合画在一幅图中。有些研究者把“延迟—提供流量”曲线和“已接纳—提供流量”曲线合成一个图，如图 23.7 所示。这种图实际上以已接纳流量而不是提供流量作为延迟的横轴，曲线上不同点代表不同的提供负载。对不稳定网络，超过饱和点后已接纳流量下降，曲线就会向左折回。

这种画法最大的缺陷是，它暴露了测量没有包含源队列的事实。如果把源队列计入，那么提供负载一旦超过饱和点，稳态延迟就趋于无穷；曲线不会向左折回，而应在峰值已接纳流量位置接近渐近线。

此类回折图通常也不反映稳态性能。在多数不稳定网络中，提供流量一旦略微超过饱和点，稳态已接纳流量就会骤降到饱和后的某个值，之后再增加提供流量，交付速率仍保持在较低水平。原因是到达饱和点时，源队列会开始无限增长；在稳态中队列长度无界，增加提供流量只会加快它增长的速度，并不改变其稳态性质。当然，对饱和点之后的网络测量稳态，需要很长的仿真时间。大多数逐步向左折回的曲线意味着仿真运行得还不够久，未达到稳态。

没有测量测试期间生成的全部数据包。一种常见错误是根据测量窗口内送达目的节点的数据包计算平均延迟，而不是根据在该窗口内生成的数据包计算。前者使用了有偏样本：它遗漏了那些在测量窗口结束很久之后才到达的高延迟数据包，截断了延迟分布，因而低估平均延迟。预热结束后生成的数据包可以形成无偏样本，但根据到达时刻选择数据包则不具有无偏性。

只报告随机流量结果。均匀随机流量能自然地平衡不同通道的负载，因此是一种相当温和的业务模式。它虽不如最近邻流量理想，但仍接近最有利的工作负载之一，容易掩盖网络中的许多设计缺陷，尤其是糟糕的路由算法。多数情况下，应当用多个对抗性流量模式（第 3.2 节）和大量随机生成的置换模式，补充随机流量测试。

23.2　分析

互连网络性能可通过多种工具测量：解析分析、仿真和实际实验各有作用。通常先用数学模型进行解析分析，以较小代价估计性能，并理解不同因素的影响。通过建立描述一整类网络的参数化方程，还可以同时评价不同配置的整个网络族。不过，解析分析通常必须作出一些近似假设，可能影响结果精度。

获得解析估计后，通常通过仿真验证分析，并获得更精确的性能估计。如果模型可靠，仿真可以给出准确结果，但代价更高：每次运行可能相当耗时，而且通常只对应一个网络配置、流量模式和负载点。仿真也往往只输出汇总性能数据，不会自动解释形成结果的原因。仿真精度取决于模型精度；若模拟器精确建模路由器时序、仲裁和通道延迟，结果可以非常准确；若忽略这些因素，结果则可能很不可靠。

网络建成后，还需要实际实验测量性能，验证仿真模型。不过到了这一步，如果发现

性能问题，修改硬件往往既困难又耗时、成本高昂。

本节讨论网络性能评估的第一步——解析分析，介绍两项基本工具：排队论和概率论。排队论适合研究数据包大部分时间在队列中等待的网络；如果争用时间主要源于阻塞而不是排队，则概率分析往往更有用。

23.2.1 排队论

互连网络中的数据包经常在各类队列中等待：源队列是测量配置的一部分，路径上每一跳的输入缓冲区也都是队列。利用排队论预测数据包在各队列中的平均等待时间，就能估算数据包延迟的若干组成部分。

排队论的全面介绍不在本书范围之内。本节只讲解其基础，以及如何把它用于分析简单互连网络；更多细节可参见排队论著作 [99]。

图 23.8 给出了基本排队系统。数据包源以每秒 λ 个包的速率产生数据包，数据包进入队列等待服务。服务器依次从队列中取出并处理数据包，平均服务时间为 T 秒，因此平均服务率为每秒 $\mu = 1/T$ 个包。

在互连网络中，数据包源可以是向网络注入数据包的终端，也可以是从另一节点向本节点交付数据包的通道。因此到达率 λ 可以是输入终端的业务速率，也可以是某一网络通道上叠加后的业务速率。相应地，服务器可以是从网络接收数据包的终端，或将数据包转发到下个节点的通道。在这两种情况下，服务时间 T 都是数据包经过该通道或终端所需的时间，因此与包长成正比：$T = L/b$。

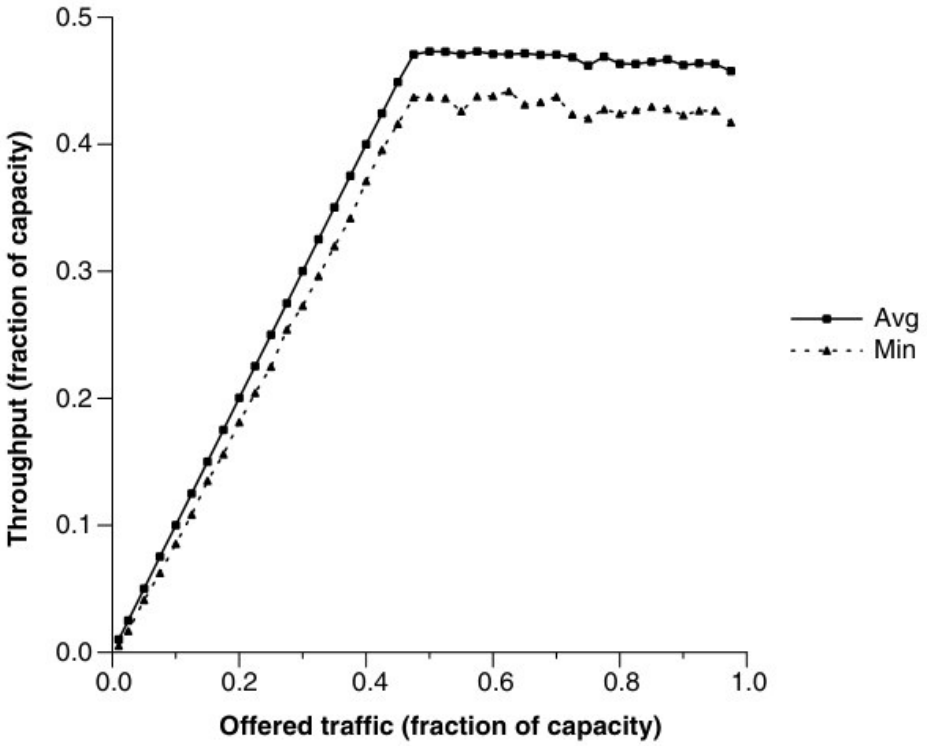


图 23.2 补码流量下二维 8 元网格的吞吐量与提供负载关系

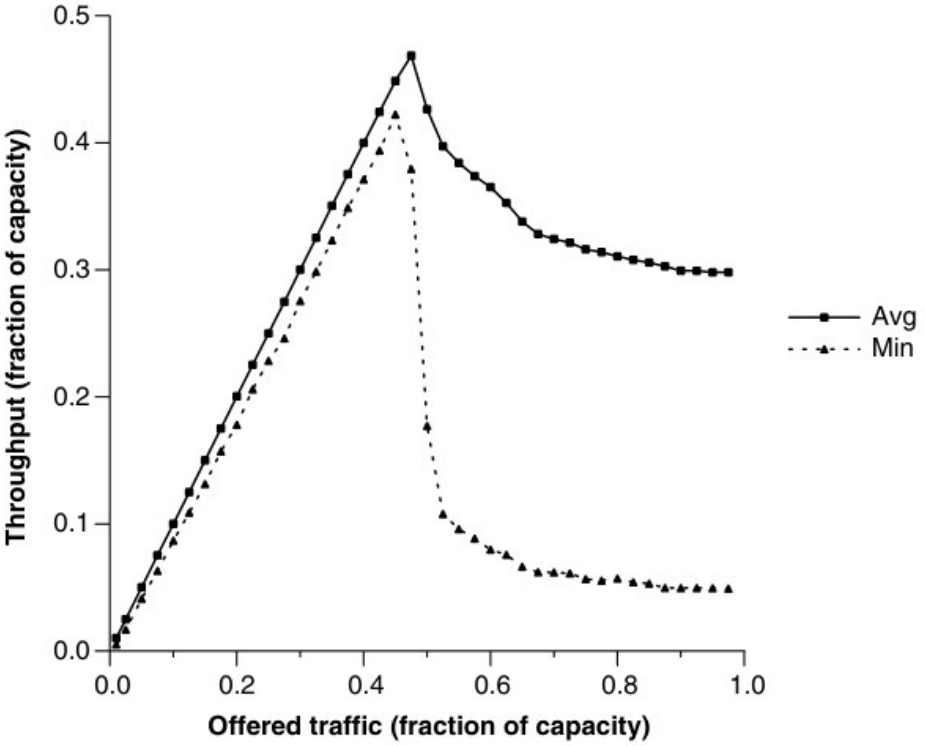


图 23.3　饱和后吞吐量骤降所暴露的网络不公平性

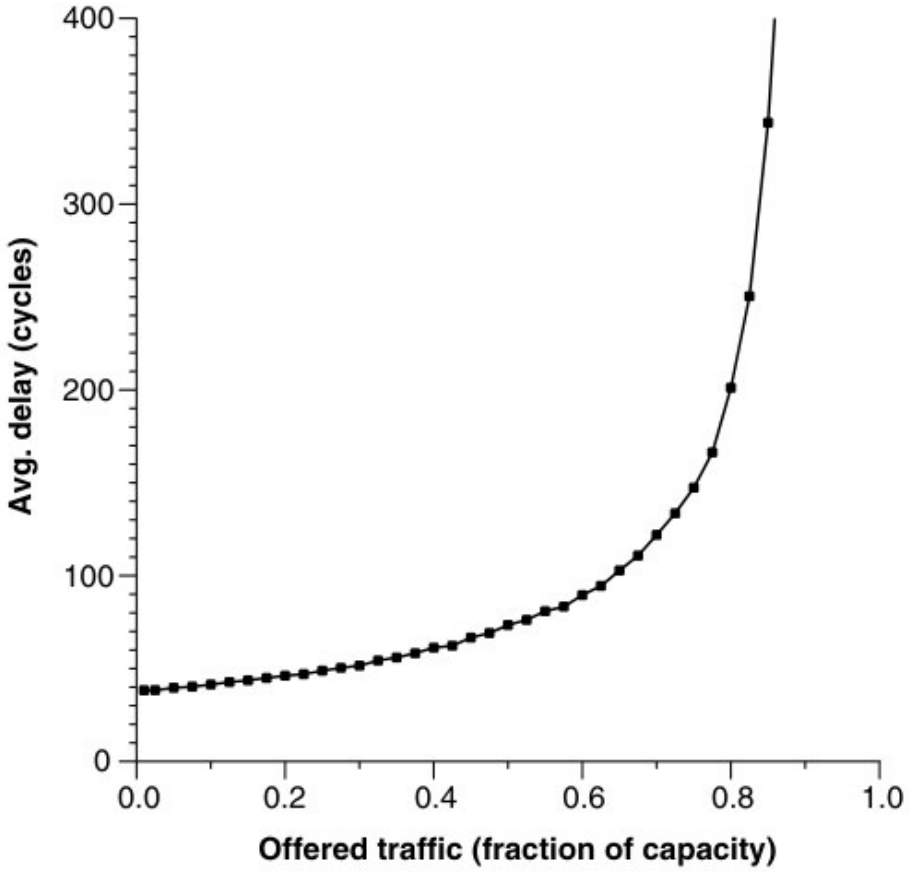


图 23.4　均匀流量、维序路由下二维 8 元网格的平均延迟与提供负载关系

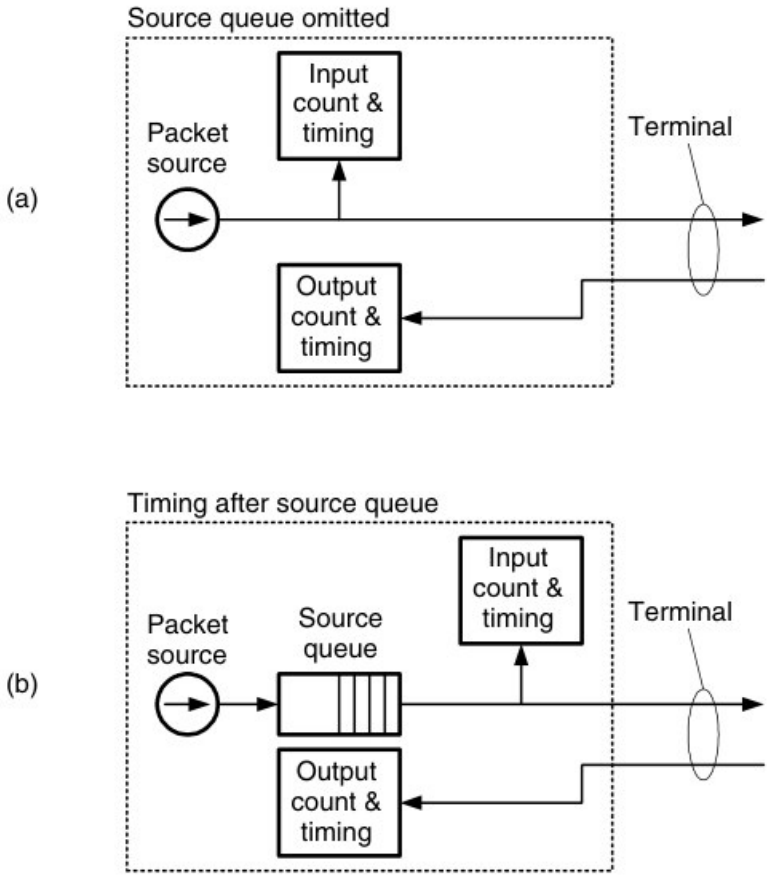


图 23.5 不设置无限源端队列时测量网络性能造成的闭环效应

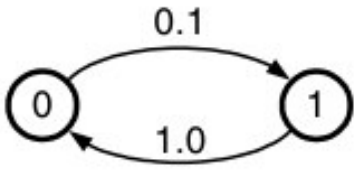


图 23.6 含一条低速链路的双节点网络及错误测量位置的影响

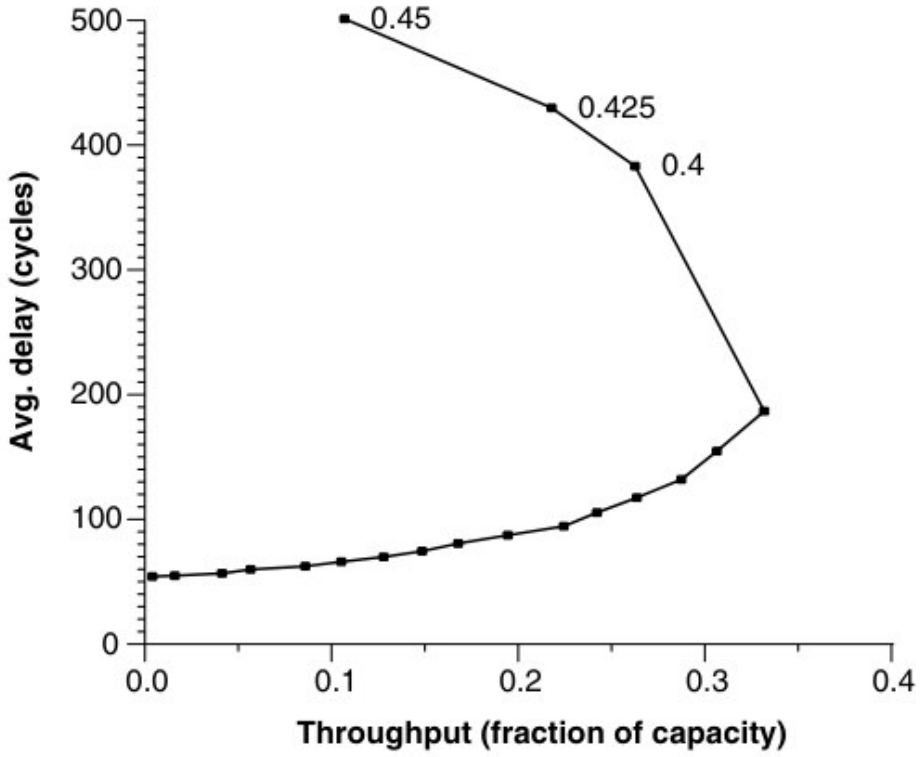


图 23.7　错误绘制延迟与已接受流量曲线的方法

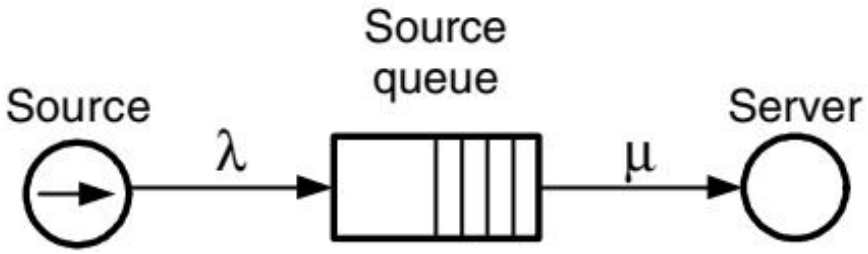


图 23.8　基本排队系统：数据包源以速率 λ 注入队列，服务单元以速率 $\mu = 1/T$ 取出数据包。

为了简化分析，通常需要一些并不完全准确的假设。首先假设队列容量无限大，输入数据包的到达间隔和输出服务时间都服从指数分布。指数过程也称为无记忆过程，因为下一事件（例如数据包到达）发生的概率与过去事件无关。这是对到达过程的简化，更是对服务时间的近似；但这些假设使我们可以用图 23.9 的马尔可夫链对简单排队系统建模。

图 23.9 的马尔可夫链描述队列状态转移。图中每个圆圈代表一种状态，其标号就是该状态下队列中数据包的数量。新包以速率 λ 到达，使状态向右移动；数据包以速率 μ 离开，使状态向左移动。在稳态中，流入每一状态的速率必须等于流出速率。对状态 p_0，这一平衡条件为：

$$\lambda p_0 = \mu p_1.$$

这里，λp_0 是从状态 p_0 离开的转移速率，而 μp_1 是进入状态 p_0 的转移速率。因此可写成：

$$p_1 = \frac{\lambda}{\mu} p_0.$$

若将系统利用率定义为 $\rho = \lambda/\mu$，则上述关系也可以写成：

$$p_1 = \rho p_0.$$

对状态 p_1 写出平衡方程，可推导 $p_2 = \rho p_1$；对每个状态重复同样的过程，可得到：

$$p_i = \rho p_{i-1} = \rho^i p_0.$$

然后，利用归一化条件 $\sum_i p_i = 1$，可得：

$$p_0 = 1 - \rho,$$

$$p_i = \rho p_{i-1} = \rho^i p_0.$$

Then, using the constraint that $\sum_i p_i = 1$, we see that

$$p_0 = 1 - \rho,$$

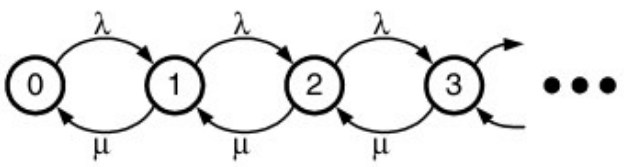


图 23.9 队列的马尔可夫链模型：各队列长度状态与到达、服务迁移

$$p_i = (1-\rho)\rho^i.$$

现在已知处于每种状态的概率，可以计算队列中的平均数据包数：

$$E(N_Q) = \sum_i i p_i = \sum_i i(1-\rho)\rho^i = \frac{\rho}{1-\rho}. \tag{23.5}$$

由于新到达的数据包必须等待队列中已有的每个数据包完成服务，所以平均等待时间为：

$$E(T_W) = E(T)E(N_Q) = \frac{T\rho}{(1-\rho)} = \frac{\rho}{\mu(1-\rho)}. \tag{23.6}$$

等待时间 T_W、服务时间 T 和队列中数据包数量 N_Q 之间的这种关系十分重要，通常称为利特尔定律（Little’s law）。

类似地，可以计算队列中数据包数量的方差：

$$E(\sigma^2_{N_Q}) = \sum_i i^2 p_i = \sum_i i^2(1-\rho)\rho^i = \frac{\rho}{(1-\rho)^2}. \tag{23.7}$$

至此，我们得到了著名的 M/M/1 排队系统的性质。所谓 M/M/1，是指数据包到达间隔服从指数分布、服务时间也服从指数分布且只有一个服务器的系统。为了说明服务时间方差对排队行为的影响，这里不作推导而直接给出：对于服务时间确定的 M/D/1 队列，平均占用量仅为 M/M/1 队列的一半：

$$E(N_Q) = \frac{\rho}{2(1-\rho)}(\mathrm{M/D/1}). \tag{23.8}$$

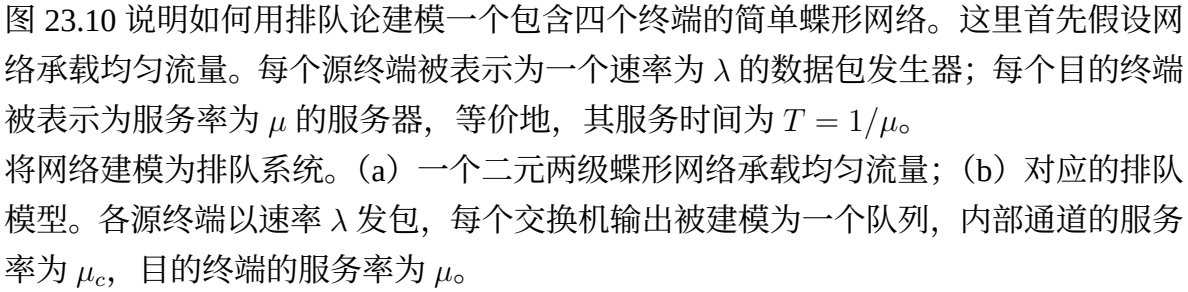
更一般地，对于服务时间服从任意分布的 M/G/1 队列，其平均占用量为：

$$E(N_Q) = \frac{\lambda^2\overline{X^2}}{2(1-\rho)}(\mathrm{M/G/1}) \tag{23.9}$$

其中，$\overline{X^2}$ 表示服务时间的二阶矩 [20]。在许多网络中，数据包长度以及相应的服务时间是恒定的，或者至少比指数分布更接近常量。因此通常应采用式（23.8）或（23.9），而不是式（23.5）。

还需要强调，若 $\lambda \geq \mu$，亦即 $\rho \geq 1$，则式（23.5）至（23.9）都不再成立。在这一不适用区域内，公式可能给出负值，但实际延迟是无限大或未定义。把这些表达式组合到更复杂的模型中后，又在不适用区域求值，是一种很常见的错误。

图 23.10 说明如何用排队论建模一个包含四个终端的简单蝶形网络。这里首先假设网络承载均匀流量。每个源终端被表示为一个速率为 λ 的数据包发生器；每个目的终端被表示为服务率为 μ 的服务器，等价地，其服务时间为 $T = 1/\mu$。



将网络建模为排队系统。（a）一个二元两级蝶形网络承载均匀流量；（b）对应的排队模型。各源终端以速率 λ 发包，每个交换机输出被建模为一个队列，内部通道的服务率为 μ_c，目的终端的服务率为 μ。

每个 2×2 交换机都被表示为每个输入上的分流器、每个输出上的汇合器，以及两个输出队列。分流器把速率为 λ 的输入流拆成两条速率为 $\lambda/2$ 的流，分别发往交换机的两个输出。若流量不均匀，各输出的分流比例也可以不同。在每个输出端，汇合器把来自两个输入的速率为 $\lambda/2$ 的流合并成速率为 λ 的流，并送入输出队列。汇合后的流量决定队列的到达率。第一级队列由服务率为 μ_c 的内部通道服务，第二级队列则由服务率为 μ 的目的终端服务。

交换机模型中的分流器和汇合器利用了泊松到达过程的特性：把到达间隔服从指数分布的一条数据流拆分，会得到多条仍具有指数到达间隔的流，而且各子流速率之和等于原流速率。同样，将多条指数间隔的数据流合并，结果仍具有指数到达间隔，其速率等于各输入流速率之和。

无论实际交换机是否在输出端排队，这里都将其建模为输出排队交换机。原因是难以用简单排队模型直接刻画输入排队或虚拟输出队列交换机中的交换争用。如果争用导致输出通道有一部分时间空闲，则可以通过降低队列服务率来近似计入这段空闲时间。

如果进一步假设所有数据包等长，则内部通道和目的终端的服务时间都为确定值。因此可使用式（23.8）的 M/D/1 模型计算各队列，得到网络的争用延迟：

$$T_c = \frac{\lambda}{2\mu_c(\mu_c - \lambda)} + \frac{\lambda}{2\mu(\mu - \lambda)}. \tag{23.10}$$

第一项表示在第一级交换机队列中等待的时间，第二项表示在第二级队列中等待的时间。一般而言，排队模型的延迟表达式会包含源与目的之间每个队列的贡献。若网络承载非均匀流量，则平均延迟应表示为网络中各条不同路径延迟的加权和（习题 23.1）。

图 23.11 给出了内部通道带宽为终端带宽 1.5 倍（$\mu_c = 1.5\mu$）时，争用延迟 T_C 随提供流量 λ 变化的曲线。图中令 $\mu = 1$ 进行归一化，共有三条曲线：两个队列各自的平均等待时间及两者之和（总争用延迟）。单个队列的延迟曲线具有典型的 $\rho/(1-\rho)$ 形状。总延迟由负载更重的瓶颈队列主导，本例中即第二级队列。通常，不论多个队列像本例这样串联，还是在负载不平衡时并联（见习题 23.1），瓶颈队列都会主导延迟—吞吐量曲线。

23.2.2 概率分析

作为排队论的补充，我们使用概率分析来研究无缓冲网络和网络部件。第 2.6 节已经介绍过用概率分析估计采用丢弃式流控的网络性能的例子；本节将讨论如何估计采用阻塞式流控的网络性能。

例如，考虑图 23.12 所示的一个没有排队缓冲、采用阻塞式流控的 2×2 交换机。假设各输出端的服务时间相同且确定，记为 T_o；两个输入的流量速率相同，$\lambda_1 = \lambda_2 = \lambda$；各输入发往两个输出的流量均匀分布，即对所有 i, j 有 $\lambda_{ij} = \lambda/2$。为了计算输入通道（例如 i_1）保持忙碌的时间 T_i，首先计算该输入上的一个数据包必须等待的概率 P_w。

$$P_w = \frac{\lambda T_o}{2} = \frac{\rho_o}{2}. \tag{23.11}$$

这正是目标输出端正在为另一输入服务的时间比例。

如果到达 i_1 的数据包必须等待，则其条件平均等待时间为：

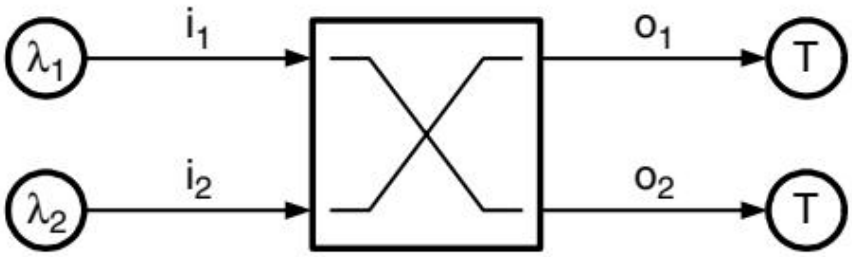
$$T_{wc} = \frac{T_o}{2}. \tag{23.12}$$

综合式（23.11）与（23.12），可得到对所有数据包取平均后的等待时间：

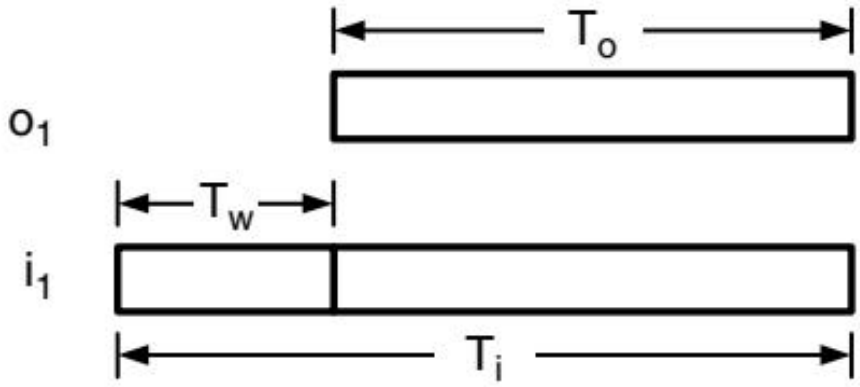
$$T_w = P_w T_{wc} = \frac{\lambda T_o^2}{4} = \frac{\rho_o T_o}{4}.$$

因此，输入通道保持忙碌的时间为：

$$T_i = T_o + T_w = T_o\left(1 + \frac{\lambda T_o}{4}\right) = T_o\left(1 + \frac{\rho_o}{4}\right). \tag{23.13}$$



(a)



(b)

图 23.12：无缓冲交换机的延迟与吞吐量计算。(a) 一个无缓冲 2×2 交换机连接两个数据包源和两个服务器；(b) 输入通道的平均忙碌时间 T_i，等于输出服务时间 T_o 与等待输出解除阻塞的平均时间 T_w 之和。

由式（23.13）可知，交换机输入端的忙碌时间增加量与输出链路的占空比 $\rho_o = \lambda T_o$ 成正比。

在发生碰撞的条件下，平均等待时间 T_{wc} 对服务时间的方差十分敏感。当服务时间为确定值时，如式（23.12），一个平均到达的数据包在另一输入的数据包完成服务约一半时遇到忙碌的交换机，因此等待 $T_o/2$。如果输出服务时间服从指数分布，则发生碰撞后的期望等待时间会加倍：

$$T_{wc} = T_o, (\text{ExponentiallydistributedT})$$

$$T_w = \frac{\lambda T_o^2}{2} = \frac{\rho_o T_o}{2},$$

$$T_i = T_o \left(1 + \frac{\lambda T_o}{2}\right) = T_o \left(1 + \frac{\rho_o}{2}\right).$$

服务时间方差增大时等待时间也会增长，因为数据包遇到长服务时间的数据包并等待它们的概率，高于遇到短服务时间数据包的概率；这是由于长服务时间的数据包占据了总忙碌时间的更大比例。在分析多级网络时，各级服务时间的方差可能不同。例如，即使输出服务时间固定，图 23.12 的交换机在下一阶段观察到的服务时间也具有非零方差。

23.3 验证

排队论、概率分析或仿真能否准确预测互连网络性能，取决于所使用模型的质量。如果模型能够捕捉网络中所有相关行为，分析结果可以很精确，也能帮助我们快速评估大范围设计空间、识别影响性能的关键因素。但如果漏掉网络的某个关键方面、对其建模不准确，或作出不恰当的近似，分析结果就可能与实际性能显著不同。仿真模型也是如此：模型不准确，仿真结果也不会准确。

验证（validation）是指将解析模型或仿真的结果与已知可靠数据进行比较，从而检查其准确性的过程。例如，若已从实际网络获取“延迟—提供流量”的实验数据，就可以在解析模型和仿真中重现实验条件，并将结果与实际数据比较。如果数值吻合，我们就有理由相信，模型已正确考虑该网络在所测运行条件下的重要影响因素，并可能同样适用于相近的网络和运行条件。

如果我们对某个仿真模型有较高信心，有时也会使用仿真结果验证解析模型。在无法获得实验测量数据时，这种方法尤其有用。图 23.11 就是一个此类验证的实例。

验证模型时，需要在具有代表性的一系列运行点和网络配置下，将预测值与可靠数据进行比较。模型在某个运行点得到验证，并不意味着它在显著不同的运行点仍然准确，因为影响性能的因素可能发生变化。例如，模型对零负载延迟的预测与实际一致，并不能保证其在接近饱和时仍能准确预测延迟。

23.4　案例研究：BBN Monarch 网络的效率与损耗

BBN Advanced Computer Systems 将 BBN Monarch [156] 设计为 Butterfly TC-2000（见第 4.5 节）的后继系统。Monarch 包含多项引人注目的体系结构特性，但遗憾的是，这台机器最终没有建成。Monarch 是一台具有统一内存访问特性的共享内存计算机，最多可有 64K 个处理器，通过一个类似蝶形的网络访问相同数量的内存模块。网络采用丢弃式流控，对内存地址进行散列以随机化目的内存模块，并使用访问合并来应对热点流量。为了简化合并，内存访问按同步时帧（frame）组织：每个处理器可在每帧开始时发起一次内存访问；同一帧内对同一地址的访问会在网络内部节点合并，随后只发送一个合并后的访问包到内存模块。

如图 23.13 所示，Monarch 网络由交换机级和集中器级交替组成。每个交换机最多有 12 个输入和 32 个输出，可判定最多 4 位地址。其输出端口被划分为若干组，使多个数据包可以同时沿同一方向前进。例如，一个交换机可以配置成 16 组、每组 2 个输出端口，也可以配置成 8 组、每组 4 个输出端口。到达任意输入的数据包都会竞争某个输出组：只要获得所需组中的任一通道，就可进入网络下一层；否则该数据包将被丢弃。

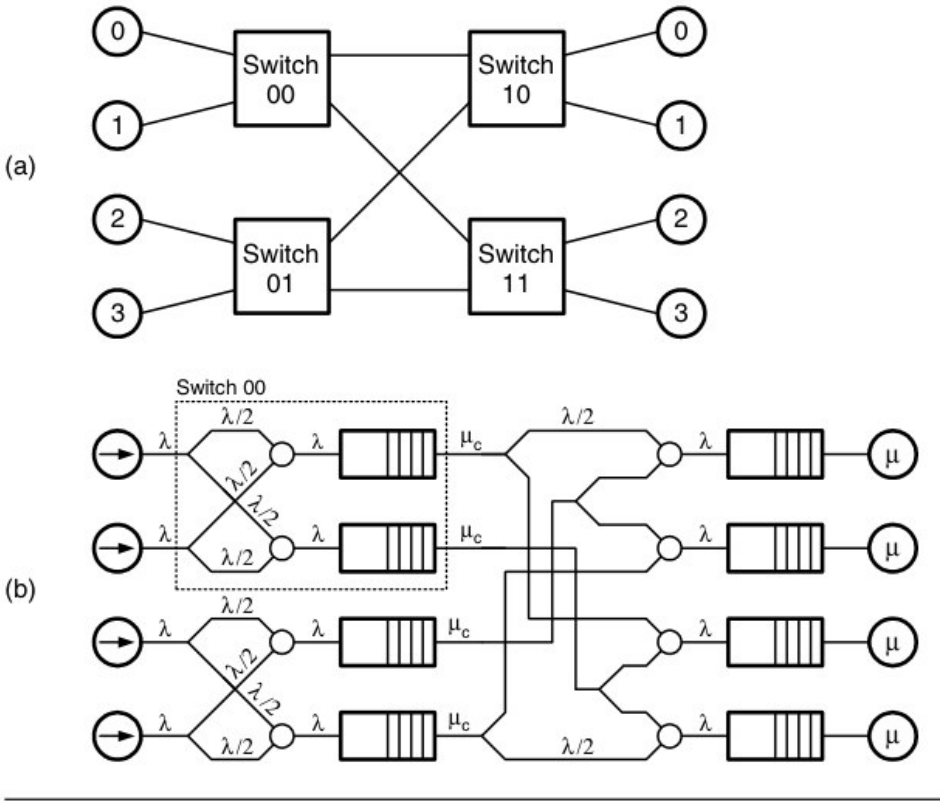


图 23.10　网络的排队模型：（a）二元两级网络承载均匀流量；（b）其排队系统模型。源终端的到达率为 λ，交换机每个输出对应一个队列；内部通道与目的终端的服务率分别为 μ_c 和 μ。

集中器（concentrator）用于将大量低负载通道复用到较少的高负载通道中。例如，在必须跨越多个封装层次、成本很高的通道之前，可以先进行这种集中。每个集中器芯

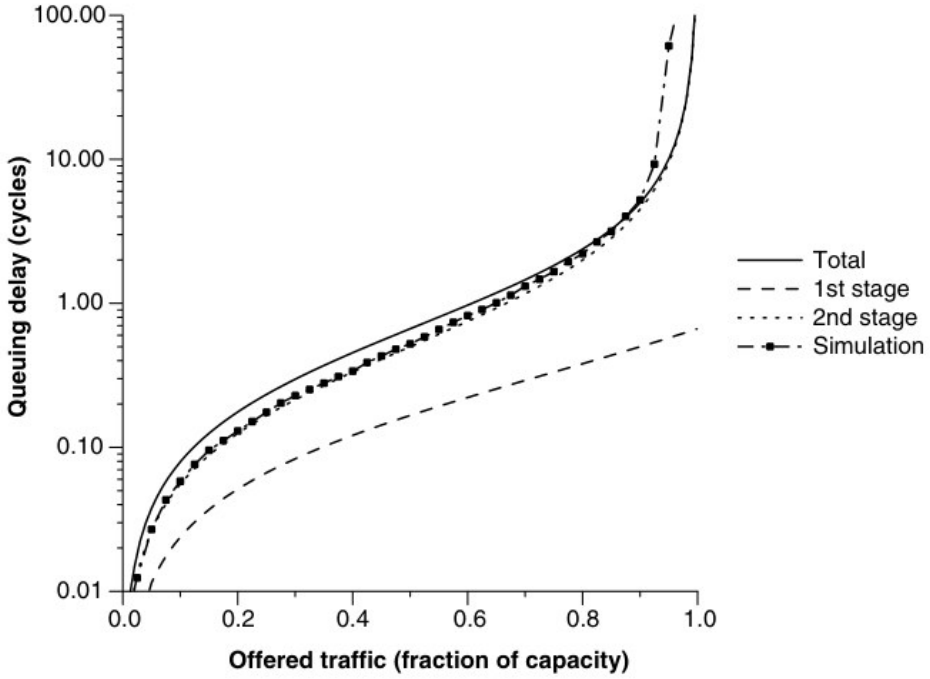


图 23.11 排队模型中争用延迟与提供负载的关系

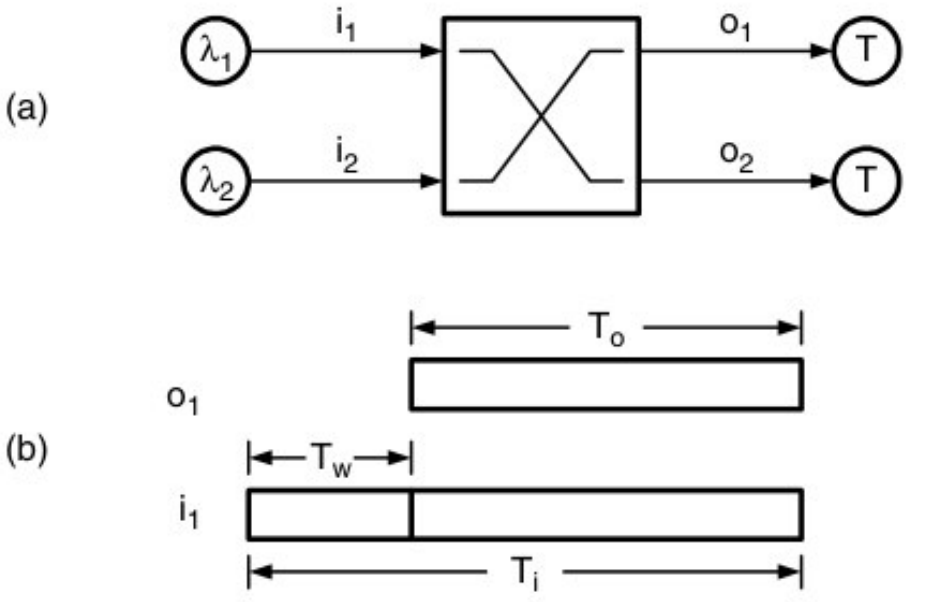


图 23.12 BBN Monarch 网络（1990）：交换级与集中级交替组成，以兼顾效率和成本。

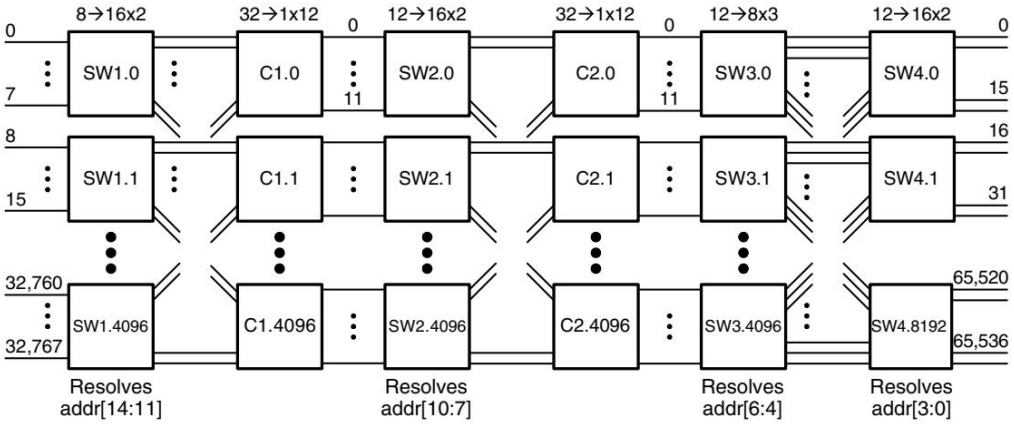


图 23.13　BBN Monarch 网络（1990 年）由交换机级与集中器级交替组成。各级的集中比经过调整，以提高效率并减少丢弃式流控造成的丢包。每条通道以串行方式运行，带宽为 350 Mbit/s。

片最多有 32 个输入和 12 个输出，所有输入和输出相互等价。每帧中，任意 12 个输入上到达的至多 12 个数据包都可路由到 12 个输出；同一帧内其余输入上到达的额外数据包则被丢弃。

我们可以使用概率分析计算 Monarch 网络各级的丢包概率。假设所有输入通道负载均匀，而且业务在输出之间均匀分布。地址散列和访问合并机制有助于使这一假设接近实际。考虑一个拥有 I 个输入、G 个输出组、每组 C 条通道的交换机。令某输入忙碌的概率（输入占空比）为 P_i，那么输入 i 上有一个数据包请求输出组 j 的概率为 $P_{ij} = P_i/G$。因此，所有输入中恰有 k 个数据包请求输出组 j 的概率由二项分布给出：

$$P_k = \binom{I}{k} \left(\frac{P_i}{G}\right)^k \left(1 - \frac{P_i}{G}\right)^{I-k}.$$

输出通道处于忙碌状态的概率 P_o，可以通过对输出组占用量为 k 的各种情况求和得到。当 $k \le C$ 时，组 j 中一条输出通道为空闲的比例为 $(C-k)/C$；当 $k \ge C$ 时，这一比例为零。因此得到如下求和表达式：

$$\begin{aligned} P_o &= 1 - \textstyle\sum_{k=0}^{C-1} \left(\frac{C-k}{C}\right) P_k \\ &= 1 - \textstyle\sum_{k=0}^{C-1} \left(\frac{C-k}{C}\right) \binom{I}{k} \left(\frac{P_i}{G}\right)^k \left(1 - \frac{P_i}{G}\right)^{I-k}. \end{aligned} \tag{23.14}$$

已知输出占空比之后，可以把本级效率定义为到达输入的数据包中成功获得输出的比例，即总输出流量与总输入流量之比：

$$P(\text{transmit}) = \frac{GCP_o}{IP_i}, \tag{23.15}$$

于是，丢包率为：

$$P(\text{ drop }) = 1 - P(\text{ transmit }) = 1 - \frac{GCP_o}{IP_i}. \tag{23.16}$$

表 23.1　65,536 端口 Monarch 网络配置与丢包分析

级	I	G	C	P_i	效率	丢包率
交换机 1	8	16	2	1.00	0.977	0.023
集中器 1	32	1	12	0.24	0.993	0.007
交换机 2	12	16	2	0.65	0.975	0.025
集中器 2	32	1	12	0.24	0.995	0.005
交换机 3	12	8	3	0.63	0.986	0.014
交换机 4	12	16	2	0.62	0.977	0.023
合计					**0.907**	**0.093**

说明：六级网络包含四级交换机和两级集中器。I 为输入数，G 为可独立寻址的输出组数，C 为每组通道数。P_i 为每级输入负载，效率和丢包率分别为该级数据包转发和丢弃的概率（原书引自文献 [156]）。

表 23.1 显示，对于表中的 64K 端口 Monarch 网络，即使输入占空比达到 100%，总丢包率仍低于 10%。这是以显著增加链路数量为代价实现的：除第一级外，各交换机输入的忙碌比例低于 65%，而各集中器输入的忙碌比例低于 25%。习题 23.7 将进一步研究链路占空比与丢包率之间的关系。

给网络增加缓冲区可以减少丢包：当输出忙碌时暂时保存数据包，而不是立即将其丢弃。在有缓冲但没有反压的网络中，只有全部输出都忙碌且全部缓冲区都已满时，数据包才会被丢弃。习题 23.9 将研究有缓冲的、类似 Monarch 的网络分析。

23.5　文献注释

排队论的一般性参考资料可见 Kleinrock [99]；Bertekas [20] 介绍了更针对网络的建模技术。Dally [46] 和 Agarwal [2] 给出了网络概率分析的实例。Dally [47] 还分析了部分占满的有限容量队列。Pinkston 和 Warnakulasuriya 为 k 元 n 维立方体网络中的死锁建立了模型 [152]。

23.6　习题

23.1 **非均匀流量的排队模型**。对图 23.10 的网络，设源 0、1 将业务均匀发往目的 2、3，反之源 2、3 将业务均匀发往目的 0、1。计算该流量模式下排队模型的“延迟—提供流量”曲线。第一级和第二级交换机队列的平均占用量分别是多少？

23.2 **集中器中的排队延迟**。一个 4:1 集中器由四个终端供给，每个终端以 1 Gbit/s 的线速发送 16 字节数据包，平均每微秒发送一个包（平均带宽 128 Mbit/s）。集中器的网络通道带宽为 2 Gbit/s。假设包到达间隔服从指数分布，计算：（a）2 Gbit/s 网络通道超额订用的概率（同时发送数据包的输入超过两个）；（b）数据包在集中器输入端经历的平均等待时间（排队延迟）。

23.3 **丢弃式流控的排队模型**。推导习题 2.9 中提出的排队模型。假设数据包平均在网络中途被丢弃，并立即重新注入一个输入队列。
23.4 **电路交换蝶形网络的吞吐量**。使用概率分析计算采用电路交换的二元三级蝶形网络（2-ary 3-fly）的期望吞吐量。假设均匀流量，服务时间 $T_0 = 4$ 个周期。
23.5 k **输入交换机的概率模型**。将式（23.13）扩展到具有均匀流量的 k 输入交换机。
23.6 **任意流量模式的概率建模**。扩展习题 23.5 推导的公式，以适用于任意流量矩阵，其中 λ_{ij} 表示从输入 i 到输出 j 的业务速率。
23.7 **Monarch 网络的占空比与丢包率权衡**。对类似表 23.4 所分析的 Monarch 网络，分别以（a）交换机输入占空比和（b）集中器输入占空比为自变量，绘制网络丢包率的变化曲线。需要调整各级的参数 C 来改变占空比。
23.8 **Monarch 网络的成本与丢包率权衡**。假设 Monarch 网络的成本与交换机、集中器的引脚总数成正比。通过改变 G 和 C，绘制网络丢包率随网络成本变化的曲线。对曲线上每个给定丢包率，通过实验确定实现这一丢包率所需成本（引脚数）最小的网络配置。
23.9 **Monarch 网络中的缓冲**。在采用丢弃式流控的网络中，加入少量缓冲，仅当全部缓冲区占满时才丢包，可以大幅降低式（23.16）给出的丢包概率。假设每个输入可缓存 F 个包，重新分析表 23.4 的网络。每帧中，新到达的数据包与已缓冲的数据包一起竞争输出；对请求输出组 j 的数据包，最先获得许可的 C 个包占用输出通道；未获得通道的包中，各输入的前 F 个包进入缓冲，其余丢弃。对于 $F = 8$，表 23.4 所示配置的丢包概率是多少?
23.10 **仿真**。对习题 23.1 的模型与流量模式，绘制饱和吞吐量与内部通道带宽 μ_c 的关系，并将模型结果与网络仿真结果比较。
23.11 **仿真**。将习题 23.4 计算的吞吐量与网络仿真结果比较，并讨论差异。
23.12 **仿真**。对习题 23.9 的有缓冲 Monarch 网络，比较模型结果与仿真结果，并讨论差异。

原著注释

- 如果网络不公平，最后几个测量数据包可能需要很长时间才能送达；若网络存在饥饿，排空阶段甚至可能永远无法结束。
- 一类性能图有时称为 Burton Normal Form（简称 BNF），名称来自 Burton Smith，勿与 Backus–Naur Form 混淆。
- 随机变量 X 服从指数分布时，$P(X < y) = 1 - \exp(-y/\mu)$。
- 习题 23.1 将讨论一个更具有挑战性的流量模式。

第二十四章　仿真

第 23 章讨论的排队论和概率分析技术能够描述网络性能的很多方面，但仍有一些情形过于复杂，无法在不作大量简化假设的前提下建立解析模型。仿真通过构建网络运行过程的计算机模型，能够研究复杂的拓扑、路由、流控和路由器微架构之间的相互作用。当然，仿真得到的结果只有在模型和实验方法正确时才可靠。

本章介绍仿真的输入、测量和设计基础。并不是每个网络模拟器都需要刻画路由器微架构的所有细节，因此首先讨论不同的仿真精细程度，以及准确性与运行速度之间的权衡；然后介绍应用驱动与合成的网络工作负载、性能测量方法、统计置信度以及模拟器的实现方式。

24.1　仿真详细程度

在开始网络仿真之前，设计者必须选择合适的模型精细程度。使用极其精确的模拟器虽然可以降低建模误差风险，但可能花费不必要的计算资源，尤其是在设计空间探索的早期阶段。相反，过度简化的模拟器运行很快，却可能忽略决定某些应用性能的重要资源争用和时序细节。选择时应综合考虑研究问题、所需准确度与可接受的运行时间。

图 24.1 展示了网络仿真常见的精细程度层级。最粗略的接口级模型只提供网络接口功能，并利用简单的路径延迟近似来交付数据包。随着模型更详细，先加入网络资源的容量约束，再逐步引入 flit 级路由器行为和硬件电路细节。一般而言，向更精细的层级移动会提高准确性，也会增加仿真开销。

容量级模型处于纯行为模型与详细网络模型之间。它为网络能力加入一些基本约束，例如通道带宽以及网络中可同时在途的数据包数量上限。在这个层级，资源争用已经可以影响数据包延迟，但仍不需要模拟每条 flit 在每一级路由器中的具体活动。

最后，在 flit 级模型中引入网络微架构细节，对每一条 flit 建模，并考虑缓冲区、交换结构和分配器等组成部件。进一步细化到硬件级时，还可能模拟信号线、电路和时序的更多具体属性。

接口级模型　■ 对网络接口建模，提供数据包交付功能。
■ 仅根据节点距离给出简单的数据包延迟近似。

容量级模型　■ 引入资源容量的简单约束，例如通道带宽或网络中在途数据包总数的上限。
■ 资源争用会影响数据包延迟。

Flit 级模型　■ 以 flit 为单位追踪资源使用情况，需要详细建模大多数路由器部件，包括缓冲区、交换结构和仲裁器。
■ 数据包延迟可以准确到 flit 时间或路由器周期。

硬件级模型　■ 加入硬件实现细节，以估算面积和时序。
■ 可用绝对时间（例如秒）表示延迟。
在 flit 级，需要准确模拟缓冲、交换和仲裁等资源。如果这些结构建模正确，该层级的性能信息可以很准确，但通常仍以 flit 时间（周期）表示。进入硬件级后，可用物理设计的具体时序替代 flit 时间，并估算路由器微架构的实现成本和面积。

24.2　网络工作负载

网络工作负载，是指在一段时间内施加到各个网络终端上的流量模式（例如数据包序列）。理解和建模负载，有助于设计与评价网络拓扑和路由函数。最逼真的流量来自真实使用网络的客户端，即应用驱动的工作负载。例如，在共享内存互连中，网络流量由各处理节点交换的一致性消息构成。如果我们不仅模拟网络，还模拟处理器上运行的应用，所得工作负载就与该应用的实际需求一致。虽然应用驱动的工作负载最准确，但仅靠这种方法往往难以覆盖全部预期流量。仍以上述共享内存系统为例，工作负载与具体应用直接绑定；扩展测试工作负载就需要开发新的应用，成本通常很高。另一种方法是精心设计合成工作负载，既捕获网络预期承受的主要流量特征，又保持灵活性。

24.2.1　应用驱动的工作负载

理想情况下，应在应用驱动的工作负载下测量互连网络性能，即由目标应用直接产生送入网络的消息序列。一种方式是除网络外，同时模拟网络的客户端。例如，对处理器—内存互连，必须同时建立处理器和内存模块的模型，再在处理单元上运行相关应用来评估网络性能。应用产生的内存消息和处理器间消息构成网络流量。这种“全系统”仿真常称为执行驱动（execution-driven）工作负载。执行驱动方式固然准确，但有一个缺点：网络反馈会影响工作负载。因此，设计者难以单独隔离互连网络的瓶颈——修改网络设计不仅会改变网络本身，也可能改变应用产生的流量。
另一种方法是不同时模拟网络与客户端，而是先捕获某个应用发送的消息序列，再在网络仿真中“重放”该序列。这种轨迹驱动（trace-driven）工作负载可以从实际运行的系统采集，也可以从前述执行驱动仿真中采集。例如，对 IP 路由器，可以记录一段时间内到达的数据包，并存下每个包的到达时间、长度和目的地址；随后在网络仿真中重新生成相同的序列。采集轨迹时，可以使用精细程度较低的执行驱动网络模型，以快速完成采集；随后仅对网络进行详细仿真，并重复使用该轨迹。由于轨迹已预先固定，网络反馈不会再改变工作负载。这可能降低准确性，但对某些应用影响不大，例如 IP 路由器中的反馈通常只在较长时间尺度上影响流量。

24.2.2 合成工作负载

如前所述，应用驱动工作负载可能难以开发和控制，因此有必要使用合成工作负载，捕获应用流量的关键特征，同时便于设计和修改。我们从三个相互独立的维度讨论合成工作负载：注入过程、流量模式和数据包长度。在某些情况下，这三个维度并非真正独立；习题 24.3 将讨论这一问题。

任何注入过程首先关心的参数都是每周期平均注入的数据包数量，即注入速率。要以速率 r 注入数据包，最简单的方法可能是周期性注入。图 24.2（a）展示速率为 r 的周期性注入过程：二值变量 A 等于 1 表示注入一个包，否则等于 0。相邻注入事件的周期始终为 $T = 1/r$，所以平均注入速率显然为 r。

周期性注入显得过于简单，因为它不能体现真实流量中常见的波动。这就引出了网络仿真中最常用的注入模型之一——伯努利过程。对速率为 r 的伯努利过程，注入变量 A 是随机变量，其注入概率为 $P(A = 1) = r$。这等价于每个周期投掷一枚正面概率为 r 的偏置硬币；如果出现正面，就注入一个数据包。如图 24.2（b）所示，各注入事件的间隔服从几何分布。

伯努利过程虽然引入了随机性，却没有内部状态，仍然过于简单，不能刻画随时间变化或具有相关性的流量过程。实际应用中的许多流量源都具有时间变化特征。例如，互联网流量有昼夜变化，因为白天在线用户比夜晚多。对缓慢变化的流量过程，伯努利过程在较短时间尺度上可能仍是合理近似，但有些注入过程在短时间尺度上也会快速变化。

快速变化的注入过程通常称为突发性（bursty）过程。针对语音连接、视频流和 HTTP 流量等不同业务的突发特征，研究人员提出了许多模型。其中一种流行的模型是马尔可夫调制过程（Markov-modulated process，MMP）：它使用马尔可夫链的当前状态来调节伯努利注入过程的速率。

图 24.3 给出两状态 MMP 的例子。“开启”（on）与“关闭”（off）两种状态的注入速率分别为 r_1 和 0。每个周期中，从关闭态转到开启态的概率为 α，从开启态转到关闭态的概率为 β。该模型描述了突发注入过程：在突发期间以速率 r_1 注入，突发之间则没有注入。突发平均持续时间为 $1/\beta$，相邻突发之间的平均间隔为 $1/\alpha$。为了计算总体注入速率，需要得到开启态和关闭态的稳态分布。令 x_0 表示处于关闭态的概率，x_1 表示处于开启态的概率，则稳态条件为：

$$\alpha x_0 = \beta x_1.$$

由于 $x_0 + x_1 = 1$，处于开启态的稳态概率为：

$$x_1 = \frac{\alpha}{\alpha + \beta}.$$

图 24.3：两状态马尔可夫调制过程（MMP）。关闭态不注入数据包；开启态以速率 r_1 进行伯努利注入。

因此，这个 MMP 的平均注入速率为：

changed based on preliminary results.

Finally, the microarchitectural details of the network are incorporated at the *flit level*. Individual flits are modeled in this level, requiring the introduction of structures

Interface level

- Models network interfaces and provides packet delivery.
- Generates simple approximations for packet latency based purely on distance.

Capacity level

- Adds simple constraints on resource capacities, such as channel bandwidths or bounds on the total number of packets in flight.
- Resource contention can affect packet latency.

Flit level

- Resource usage is tracked on a flit-by-flit basis, requiring a detailed modeling of most router components, including buffers, switches, and allocators.
- Packet latency is accurately modeled in terms of flit times or router cycles.

Hardware level

- Adds implementation details of the hardware, yielding area and timing information.
- Latencies can be expressed in terms of absolute time (that is, seconds).

图 24.1　网络仿真细节层级：仿真速度与精度的权衡

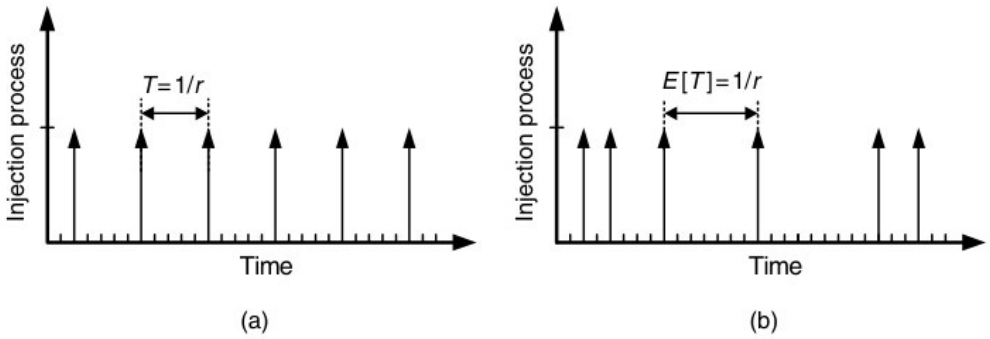


图 24.2　周期性注入过程与伯努利注入过程的时间序列

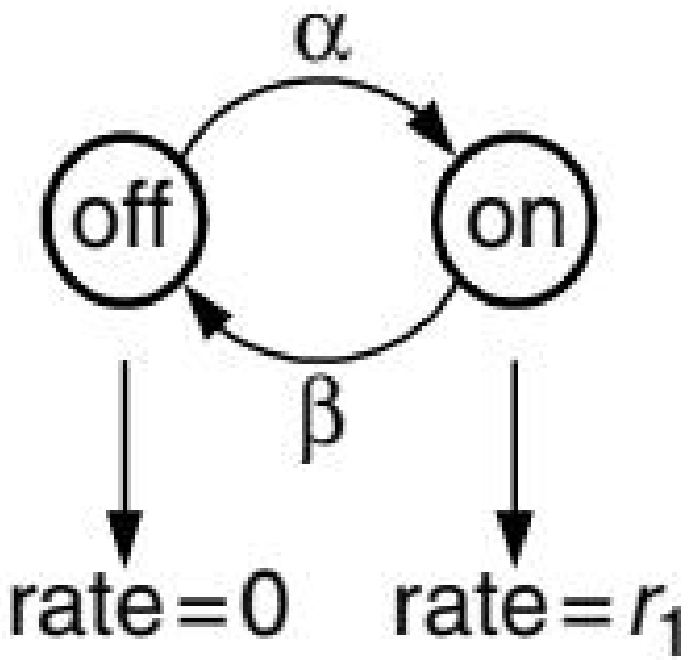


图 24.3 两状态马尔可夫调制注入过程（MMP）：关断状态不注入数据包，开启状态以速率 r_1 进行伯努利注入。

$$r = r_1 x_1 = \frac{\alpha r_1}{\alpha + \beta}.$$

习题 24.1 将分析更复杂的 MMP。

流量模式的基础知识已在第 3.2 节介绍，下面只需讨论如何确定数据包长度。一个直接的方法是利用第 24.2.1 节所述应用驱动工作负载收集的包长分布。图 24.4 展示了从某 IP 路由器获得的此类轨迹。随后，每次注入数据包时，按该包长在采集分布中出现的比例，随机选择其长度。

24.3 仿真测量

估计网络性能时，主要存在两类误差：系统误差与抽样误差 [66]。系统误差来自测量或仿真自身的偏差；在网络仿真中，它通常由模拟器的初始化引起。采用适当的预热时间（见第 24.3.1 节）能够降低系统误差的影响。

网络预热完成后，通常假定其已进入稳态：网络统计特性不再随时间变化。此时，重点转向对网络进行抽样，以准确估计目标性能参数。第 24.3.2 节介绍两种常见的抽样方法——批均值法和独立重复法。将它们与第 24.3.3 节的置信区间结合起来，就能以严谨的统计方法测量网络参数并评价估计的精度。

24.3.1　模拟器预热

为简化实现，多数模拟器在注入任何数据包之前，都把缓冲区设为空、所有资源设为空闲。这虽然容易实现，却会给网络测量引入系统误差。例如，仿真初期注入的数据包经过的网络较空闲，争用较少，传输速度更快；而随着缓冲区逐渐填满，后续数据包面临更严重的争用，延迟也随之增加。经过一段时间后，初始化状态的影响逐渐可以忽略，这时称模拟器已预热。忽略预热结束前的所有事件，能够减少系统误差对测量的影响。遗憾的是，没有普遍适用的预热长度计算方法，但多数方法大致遵循以下步骤：

1. 根据启发式方法，给出初始预热周期 T_{wu}。
2. 收集统计数据，但忽略估计预热期内的样本。
3. 检验剩余样本是否平稳。如果已平稳，就采用 T_{wu} 作为预热长度；否则增加 T_{wu}，重复步骤 2 和 3。虽然针对这一过程的细节已有许多复杂方法，这里介绍一种实践中通常有效的简单方法。首先把初始预热长度设为可快速模拟的少量事件（约 100～1000 个）；如果仿真很快进入稳态，这个较小的初值可限制估计预热期所需的额外开销。

在一次仿真运行中收集统计数据，忽略预热期内发生的事件。本例需要估计平均数据包延迟，因此对事件采取分批平均：每个样本点是多个数据包延迟的均值。每批包含的样本数量称为批大小（batch size），应在统计上具有意义，至少包含 30～50 个样本。本例批大小为 100，所以第一个样本代表前 100 个数据包的平均延迟，第二个代表接下来 100 个数据包，以此类推。

利用预热期之后的样本，对数据作线性拟合。如果拟合直线在预先规定的精度内近似水平，就认为网络已进入稳态；否则延长预热期并重复该过程。

图 24.5 展示网格网络中数据包延迟随仿真时间变化的例子。即使每批使用 100 个样本，单次仿真的曲线在前 6000 个包到达之后仍显得不稳定。这是因为我们在抽样一个随机过程，抽样误差是不可避免的。若对若干次独立运行取集合平均，便可大幅减小抽样误差。图 24.5 表明，将 10 次独立仿真运行组合成一个集合后，系统看起来在约 4000 个样本点后趋于稳定。

虽然可以仅用一次仿真运行来估计预热期，但综合多次独立运行会大幅降低低估预热时间的风险，代价是需要更多仿真时间。

24.3.2　稳态抽样

在经过适当预热、仿真达到稳态之后，下一步是测量所研究的随机过程。本节介绍网络模拟器中常用的两种基本测量方法：批均值法（batch means）和独立重复法（replication）。批均值法在一次较长的仿真运行中进行测量。与估计预热期时通过分批平滑样本的做法类似，先把样本划分为许多批，并分别积累统计数据。不过，这里每批的长度根据整个仿真的运行长度选取，通常使总批数为 20～30。给定一次仿真的观测序列 $\{X_0, X_1, \ldots, X_{n-1}\}$，根据批数 k 和批大小 s 构造各批 B_i。为简化符号，设 $n = sk$，则批均值为：

$$\bar{B}_i = \frac{1}{s}\sum_{j=0}^{s-1} X_{si+j}, \quad 0 \le i < k.$$

总体样本均值即各批均值的平均值：

$$\bar{B} = \frac{1}{k}\sum_{i=0}^{k-1} \bar{B}_i.$$

此时分批看起来似乎没有意义，因为求出的总体均值 $\bar{B}$ 与直接对原始样本求均值相同。不过，下一节将说明，为了评估均值估计的可靠程度，需要估计所研究过程的标准差。对批均值法，样本标准差 σ_S 可估计为：

$$\sigma_S^2 = \frac{1}{k-1}\sum_{i=0}^{k-1} (\bar{B} - \bar{B}_i)^2.$$

批均值法中的每个样本都是大量原始样本的平均值，因此批均值之间的方差会大幅降低，从而减小估计的标准差，提高对平均值估计结果的置信度。

批均值法的主要缺点源于统计独立性。理想情况下，各批应互相独立，否则批之间的相关性可能使总体均值 $\bar{B}$ 等统计量出现偏差。实际上，来自一次连续仿真的各批并不独立：前一批结束时仍滞留在网络队列中的数据包，在后一批开始时仍然存在。增加批大小可以减弱这种相关性，这也解释了为什么通常只使用 20～30 批。

另一种抽样方法——独立重复法——通过多次独立运行模拟器，避免批之间的相关性。它不像批均值法那样运行一个很长的仿真，而是收集多次较短运行的结果。通常，重复运行次数超过 30 次后，继续增加次数对精度的改善不大；此时更值得增加单次运行长度。此外，每次运行至少应包含数百个样本，以便相关统计量接近正态。对第 k 次运行的观测序列 $\{X_0^{(k)}, X_1^{(k)}, \ldots, X_{n-1}^{(k)}\}$，先计算该次运行的均值：

$$\bar{R}_k = \frac{1}{n}\sum_{i=0}^{n-1} X_i^{(k)}, \qquad 0 \le k < r.$$

于是，总样本均值 $\bar{R}$ 是各次独立运行均值的平均：

$$\bar{R} = \frac{1}{r}\sum_{k=0}^{r-1} \bar{R}_k.$$

校订注：原英文书此处两条均值公式排印时遗漏了归一化分母，且求和起始下标不统一；这里按“各次运行求均值、再对均值求平均”的文字定义补上 $1/n$ 和 $1/r$。

标准差则可以估计为：

$$\sigma_S^2 = \frac{1}{r-1}\sum_{k=0}^{r-1} (\bar{R} - \bar{R}_k)^2.$$

独立重复法虽然消除了不同运行的均值样本 R_i 之间的相关性，却更容易受到初始化带来的系统误差影响。批均值法只有一次初始化，而独立重复法会执行多次初始化。

为了降低其影响，建议每次运行中收集的样本数量至少为该次预热期丢弃样本数的 5～10 倍。

24.3.3　置信区间

通过批均值法或独立重复法收集仿真样本后，一个自然的问题是：这些数据在多大程度上代表了真实的随机过程？置信区间是一种能够定量回答这一问题的统计工具。对给定样本组，置信区间提供一个取值范围，使随机过程的真实均值以指定的置信水平落入这个范围。

计算置信区间时，通常假设所测过程是平稳的，而且相关统计量近似服从正态分布。对网络仿真，前述预热程序用于确保过程达到平稳。原始随机过程本身未必正态分布，但中心极限定理指出，足够多独立或弱相关随机观测的样本均值分布会趋近正态。由于多数仿真包含数百至数千个样本，这一条件通常可以得到较好的近似。

对平稳过程的 n 个观测 $\{X_0, X_1, \ldots, X_{n-1}\}$，样本均值 $\bar{X}$ 为：

$$\bar{X} = \frac{1}{n}\sum_{i=0}^{n-1} X_i$$

它是对过程真实均值 $\widetilde{X}$ 的最佳估计。置信水平为 $100(1-\delta)\%$ 时，真实均值的可能范围可用样本均值表达为：

$$\bar{X} - \frac{\sigma_S t_{n-1,\delta/2}}{\sqrt{n}} \le \tilde{X} \le \bar{X} + \frac{\sigma_S t_{n-1,\delta/2}}{\sqrt{n}}.$$

该式意味着，在 $100(1-\delta)\%$ 的置信水平下，待测参数的真实均值 $\widetilde{X}$ 位于样本均值加减某个误差项所定义的区间内。可以看到，这一误差项按 $n^{-1/2}$ 缩小，且与样本标准差 σ_S 以及参数 $t_{n-1,\delta/2}$ 成正比。样本标准差可直接由观测数据估算为：

$$\sigma_S^2 = \frac{1}{n-1}\sum_{i=0}^{n-1}(\bar{X} - X_i)^2$$

也可以采用第 24.3.2 节两种抽样方法对应的标准差估计。

误差项中的最后一个参数 $t_{n-1,\delta/2}$ 来自 Student 的 t 分布，用于考虑用有限样本估计标准差的不确定性。参数 $n-1$ 是计算 σ_S 使用的样本数减一，称为自由度；$\delta/2$ 是双侧置信区间两端对应的尾部概率。表 24.1 给出了 95% 和 99% 置信区间的若干 t 值；更多数值可参见标准数学用表，例如文献 [201]。

表 24.1　不同置信度下的 Student t 分布

n-1	95%	99%
1	6.3137	31.821
5	2.0150	3.3649
10	1.8125	2.7638

50	1.6759	2.4033
100	1.6602	2.3642
∞	1.6449	2.3263

图 24.6 给出了批均值法估计平均延迟时，置信区间随已测数据包到达数量变化的情况。计算每个中间置信区间时，都把截至该点的样本划分成 30 个等大的批。虽然区间收敛看起来很慢，但其大小遵循预期的 $n^{-1/2}$ 规律。收敛缓慢的另一个原因是被测网络接近饱和。若以平均延迟为基准，大约 300,000 次包到达后，相对置信区间收窄到 5%；要收窄到约 2.5%，则需要接近 1,000,000 次包到达。还应注意，前几个置信区间甚至没有覆盖最终的平均延迟，这反映出不同批之间的相关性会对区间估计的准确性产生负面影响。

24.4 模拟器设计

离散事件模拟器的一般设计超出了本书范围，但网络模拟器的设计者和使用者应该了解几个基本问题。首先介绍两种基本实现方式：按周期推进的模拟器和事件驱动模拟器。然后讨论网络仿真特有的问题——如何建模将注入过程与网络解耦的无限源队列。几乎所有模拟器还都需要随机数以生成注入过程，因此我们也会介绍随机数生成的关键问题。最后给出模拟器出现意外行为时的一些实用建议。

24.4.1 仿真方法

网络模拟器常用两种设计方法：周期驱动仿真和事件驱动仿真。为了说明它们的差异，我们分别使用这两种方式建模一个简单的输出排队路由器节点。该节点把到达的数据包立即转发至与其所需输出端口对应的输出队列。各输出队列的写入带宽足以同时接收来自全部输入端口的数据包。为简化模型，假设节点总延迟为一个仿真周期，并采用无限缓冲区。

在周期驱动仿真中，时间一般按两个阶段推进：一个阶段主要负责读取全局状态，另一个阶段负责写入全局状态。把读写分开，就可以先执行所有需要读取全局状态的过程，然后再执行可能修改状态的过程。

图 24.7 使用周期驱动方法模拟上述共享存储式交换机。一个仿真周期开始时，数据包到达各节点，随后执行第一个阶段。所有读取全局状态的函数（本例中为 `ReadInputs`）都要向模拟器注册，并在该阶段被调用。对每个节点，`ReadInputs` 只调用一次，负责读取该节点各输入端到达的包，并将它们存入相应的队列。

在周期的第二个阶段，模拟器调用负责写入全局状态的函数——本例为 `WriteOutputs`。它为每个输出选择一个合适的数据包，并把该包写入下一节点的输入端，以供下一个周期中的 `ReadInputs` 读取。这一简单的两阶段过程贯穿整个仿真。各阶段内部的具体功能可以不同，但必须满足一个关键不变式：同一阶段的函数无论以什么顺序执行，都不应改变仿真结果。习题 24.6 将进一步讨论这一性质。

周期驱动仿真的替代方案是事件驱动仿真。与周期驱动方法不同，事件驱动方法不依附全局时钟，可以提供更灵活的建模方式，尤其适合底层设计本身是异步的情况。事件驱动仿真基于一个极其简单的事件框架：每个事件是包含三个字段的数据结构，分别为调用时间、动作（函数调用）和数据（函数参数）。模拟器把所有待处理事件放入事件队列，并按发生时间排序；每次取出时间最早的事件，执行对应动作。事件处理函数既可更新状态，也可根据处理结果安排未来事件。

```
void ReadInputs(int node) {
    Packet *p;
    int input, output;
    // 访问该节点的每个输入端口，读取到达的数据包
    for (input = 0; input < NumInputs(); input++) {
        p = ReadArrival(node, input);
        if (Valid(p)) {
            output = OutputPort(node, p);
            AddToQueue(node, output, p);
        }
    }
}

void WriteOutputs(int node) {
    Packet *p;
    int output;
    // 对每个输出队列，选择转发到下一节点的数据包
    for (output = 0; output < NumOutputs(); output++) {
        if (!OutputQueueEmpty(node, output)) {
            p = SelectFromQueue(node, output);
            WriteOutgoing(node, output, p);
        }
    }
}
```

代码校订：原印本循环条件后的变量自增写为 `inputs++` 与 `outputs++`，与声明的循环变量不一致；按其算法意图分别校为 `input++` 与 `output++`。

为了更好地理解事件驱动仿真，图 24.8 给出了前述输出排队交换机在事件驱动模拟器中的实现。仿真从创建某个数据包到达节点的 `Arrival` 事件开始。与周期驱动方法一样，收到数据包后会将其放入对应输出端口的队列；但事件驱动模拟器还必须安排一个负责调度输出的事件。这里把这项任务交给第一个到达空队列的数据包，以确保只要队列非空就一定存在输出调度事件，同时避免创建重复事件。`AddEvent` 将新事件加入事件队列；第一个参数表示相对于当前时刻的执行延迟，本例中为一个周期。后面依次传入动作名称 `ScheduleOutput` 及其参数。

```
void Arrival(int node, Packet *p) {
    int output;
    // 将到达的数据包送入对应输出队列
    output = OutputPort(node, p);
```

```
    // 如果输出队列原为空，安排一次输出调度事件
    if (OutputQueueEmpty(node, output)) {
        AddEvent(1, ScheduleOutput, node, output);
    }
    AddToQueue(node, output, p);
}

void ScheduleOutput(int node, int output) {
    Packet *p;
    int next;
    // 从输出队列选取数据包，转发到下一节点
    p = SelectFromQueue(node, output);
    next = DownStreamNode(node, output);
    AddEvent(1, Arrival, next, p);
    // 队列仍非空时，再安排下一次输出调度
    if (!OutputQueueEmpty(node, output)) {
        AddEvent(2, ScheduleOutput, node, output);
    }
}
```

`ScheduleOutput` 动作从输出队列选取数据包，并将其转发到下一节点。转发通过在下一节点为该包创建新的 `Arrival` 事件实现。若队列仍非空，就在两个时间单位之后再次安排输出调度事件。此处以两个时间单位为间隔，是因为数据包到达事件与输出调度事件使用交错的时间步，类似周期驱动模拟器中的读写两阶段。同一时间步内的事件应允许按任意顺序执行，而不改变仿真结果。

与周期驱动模拟器简单的两阶段循环不同，事件驱动仿真必须维护一份按时间排序的待执行事件列表，以确定下一事件；并且向该列表插入新事件应尽可能高效。因此，自然会采用堆结构。普通二叉堆支持以 $O(\log n)$ 的时间插入事件和取出下一事件；而专门为事件驱动模拟器设计的数据结构还可以进一步提升效率。例如，日历队列（calendar queue）[31] 可在平均 $O(1)$ 时间内完成事件的插入和取出。

24.4.2 源端队列建模

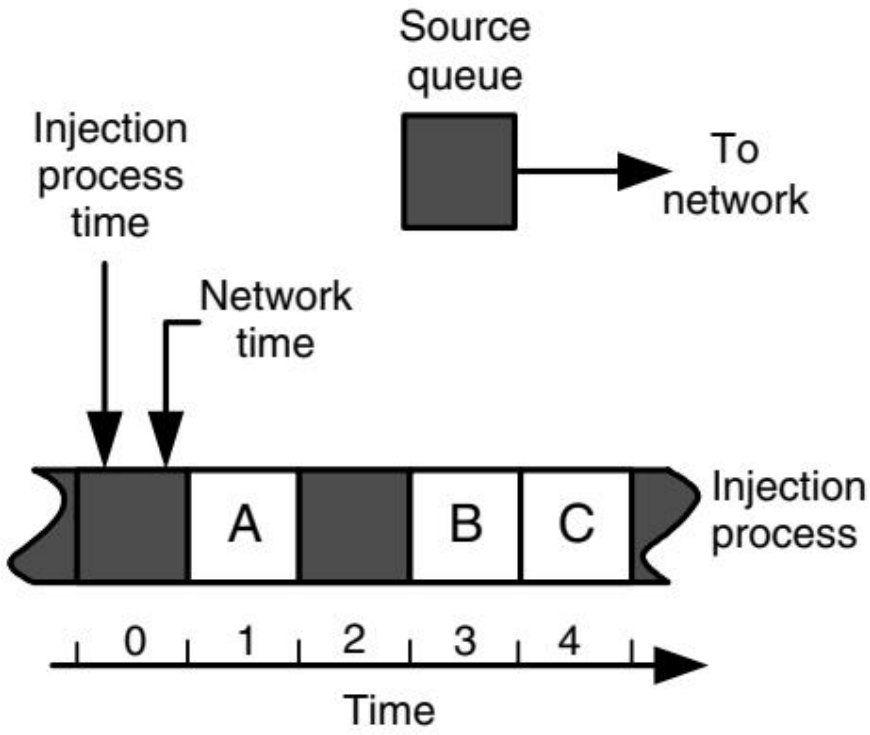
如第 23.1 节所述，互连网络的标准开环测量配置使用无限源队列，将各注入过程与网络解耦。虽然这允许独立控制注入过程，但当仿真网络的负载超过饱和点时，会带来实际实现问题：源队列无界增长，保存的数据包数近似与仿真时长成正比。如果模拟器为队列中每个包都分配内存，仿真内存占用也会无界增长。更麻烦的是，若某种流控技术依据数据包年龄分配优先级，便必须精确记录队列中各包的年龄，这又使压缩表示输入队列变得困难。

一个直接的实现方式是让注入过程与网络仿真同步运行：每个仿真周期中，注入过程都有机会向自己的源队列加入一个新包。但在任一周期，网络只能看到各源队列的队首包；其余排队数据包并不立即影响仿真。根据这一观察，可以用一种简单高效的方法建模源队列，而不必显式存储所有数据包：让注入过程“落后”于网络，运行在过

去的时间线上。只有当对应的源队列变空时，才推进该注入过程的时间。

图 24.9 展示了这种源注入队列的运行过程。注入过程的历史以一段带状时间轴表示，每个槽对应一个仿真周期；灰色空槽表示该周期没有注入包，其余槽则包含待注入的数据包。相应地，源队列为空时用灰色表示，有包时标注其名称。因为注入过程允许落后于网络，源队列实际上只需要容纳一个数据包。

本例开始时，网络时间和注入过程时间都位于周期 0，源队列为空（图 24.9（a））。网络时间向前推进时，检查源队列；若为空，就同步推进注入过程，直到生成数据包或赶上网络时间。这使数据包 A 被注入，注入过程时间推进一个周期（图 24.9（b））。在周期 2 和 3 重复该检查，但由于源队列仍被 A 占用，网络时间推进两个周期，而注入过程时间保持在周期 1（图 24.9（c））。A 在周期 3 离开源队列，周期 4 开始时队列为空。于是先把注入过程时间推进至周期 2，但该周期没有注入，继续推进到周期 3 后生成数据包 B（图 24.9（d））。



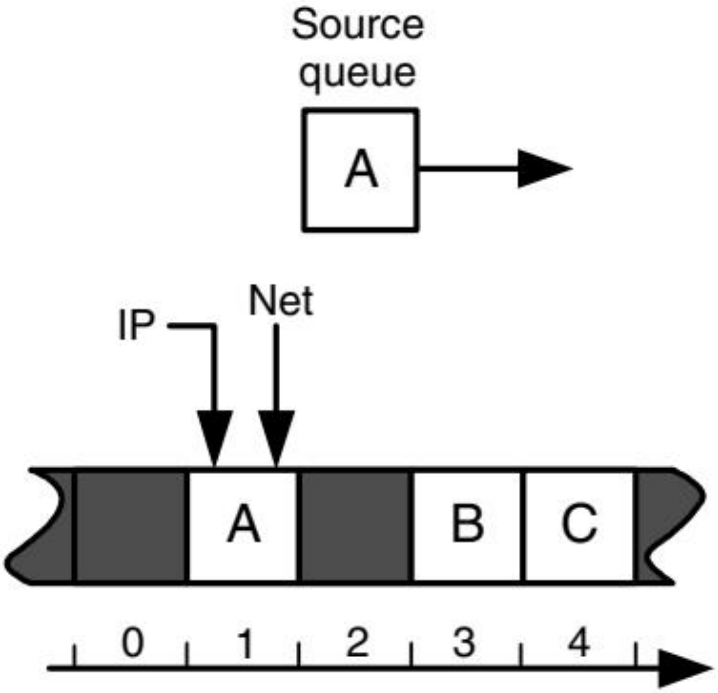
(a)

图 24.9：允许注入过程时间落后于网络时间的源队列工作示例。

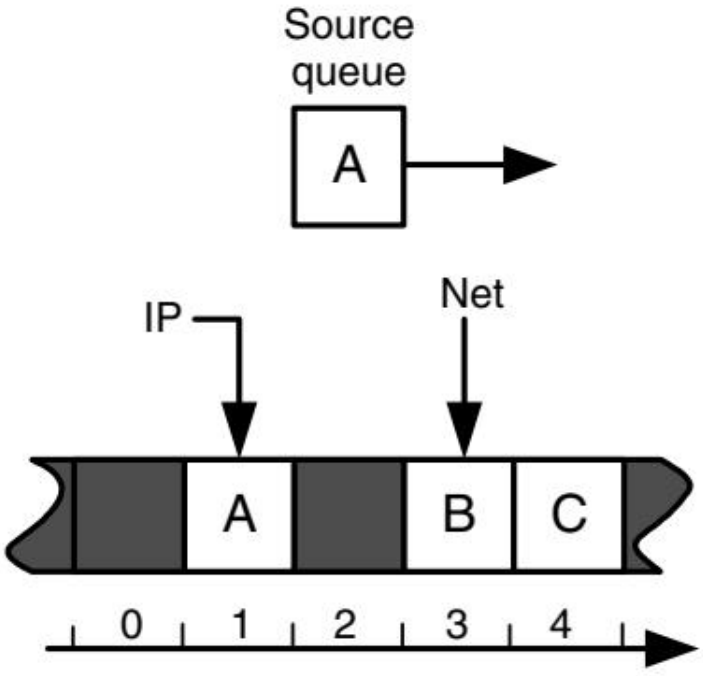
24.4.3　随机数生成

由于网络仿真使用的许多流量过程都是随机的，模拟器需要生成随机数才能正确实现这些流量源。例如，下面的类 C 伪代码模拟了一个注入速率为 r 的典型伯努利过程：

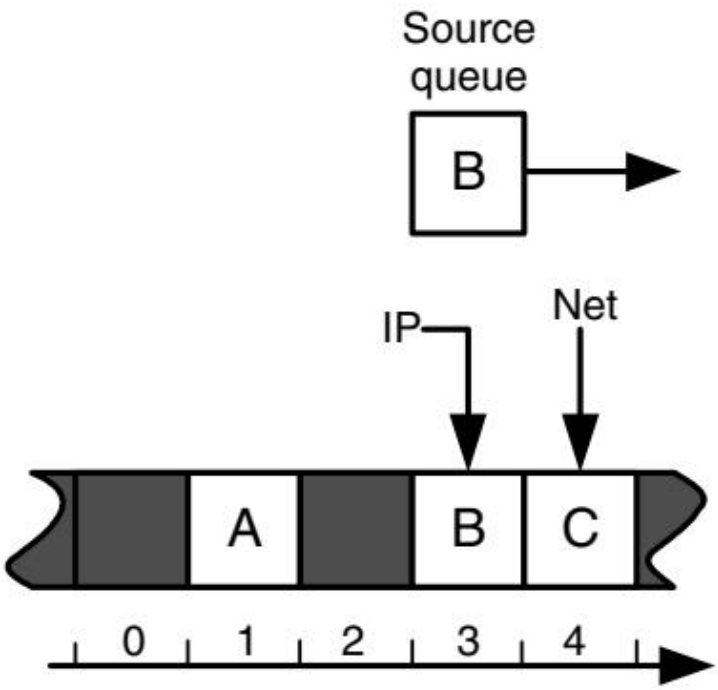
```
if (random() < r) {
    inject_packet();
}
```


Source
queue
A
IP
Net
A
B
C
0
1
2
3
4

(b)


Source
queue
A
IP
Net
A
B
C
0
1
2
3
4

(c)



(d)

`random` 函数返回一个在 0 与 1 之间均匀分布的浮点值。这段代码看起来很简单，但它隐藏了一个棘手的问题：如何真正生成所需的随机数。

生成真随机数的方法有很多，尤其广泛应用于安全与加密系统。它们通常采样某种自然噪声源，例如电阻中的热噪声（Johnson 噪声）或反向偏置二极管中的噪声。但基于实际实现成本考虑，大多数软件采用伪随机数发生器（pseudorandom number generator, PRNG）。

多数软件 PRNG 的基本结构相同：使用一个种子值（seed）保存发生器状态。每当用户请求一个新随机数时，算法计算“随机”输出及下一状态，将输出返回用户，并用新状态更新种子。这个过程实际上是确定性的，并不真正随机；但只要选用合适的 PRNG 算法，输出就能通过许多随机性统计检验。

遗憾的是，程序员常用的一些随机数发生器并不理想，例如许多 C 语言库提供的 `rand` 和 `drand48`。`rand` 生成的低位通常周期很短，随机性较差；`drand48` 略有改进，但仍有明显问题。因此，设计者应使用既有严格理论依据、又经过实证检验的发生器。可参考 Knuth [101]、Matsumoto 和 Kurita [121]、Park 和 Miller [141] 所介绍的方法。

最后，优良的 PRNG 相比真随机源还有一个重要的实践优势：确定性。只要初始种子固定，随机数序列及程序行为就保持确定。通过改变种子可以使不同运行具有随机差异，而固定种子则能复现特定结果，便于实验比较和调试。

24.4.4 故障排查

与其他模拟器一样，使用网络模拟器的最佳做法，是先用直觉、数量级估算或简化模型推测预期结果，再用仿真验证。大多数时候，预测与仿真会比较接近，但也会出现差异。下面列出定位网络模拟器异常结果的一些通用方法：

■ 确认网络仿真的细节层级足以回答所研究的问题。

■ 检查资源之间是否存在不公平或负载不平衡。这种问题很常见，却往往被简单模型忽略。

■ 通过识别瓶颈资源来缩小问题范围。例如，增加虚拟通道数量、缓冲区容量等流控资源后，如果吞吐量仍低于估计值，问题很可能不在流控；可继续检查路由函数等其他部分。

■ 收集统计数据。所有流量是否都正确到达预定目的？是否有极少数数据包滞留网络的时间远长于其他包？统计异常常常是有价值的线索。

■ 将模拟器配置成易于解析的简单情形，例如均匀流量、伯努利到达、低提供负载和大缓冲区，以便与解析解对照。

■ 模拟器自身也可能有程序错误。可与另一模拟器比较，或深入检查现有模拟器的代码。

24.5 文献注释

关于仿真基础，包括模拟器设计、输入过程、测量和 PRNG，文献 [29] 与 [109] 都是很好的参考资料。

应用驱动工作负载有许多来源。例如 SPLASH [169, 194] 等并行计算基准或数据库基准 [77]，都可用来测试处理器互连网络；互联网数据包轨迹也可以从多个在线渠道获取。

Heffes 和 Lucantoni [81] 讨论了使用 MMP 建模语音与数据流量；Jain 和 Routhier [90] 则提出用数据包列车（packet trains）表示突发性。但要注意，即使某些模型捕获了突发特征，它们仍可能远比真实流量的统计结构简单。例如，Leland 等人发现以太网流量可能具有自相似性，而很多简单模型根本无法表示这一性质 [202]。Kim 和 Chien [96] 研究了包长对互连网络性能的影响，指出长短消息之间的交互作用可能十分显著。

Knuth [101] 是早期严格分析 PRNG 的先驱之一。Park 和 Miller [141] 的综述提出了“最小标准”PRNG，至今仍是经过充分检验、足以胜任不少应用的方法。文献 [111] 综述了更新的 PRNG 进展。真随机数发生器也有多种实现，例如文献 [149] 利用片上大电阻作为噪声源。

24.6 习题

24.1 **四状态 MMP**。计算图 24.10 所示四状态马尔可夫调制过程的平均注入速率 r，并说明该过程与图 24.3 的简单开关（on/off）过程有何联系。

24.2 (σ, ρ) **调节器的性能**。将图 24.3 的开关型 MMP 作为一个 (σ, ρ) 调节器（见第 15.2.1 节）的未调节输入。令 $\sigma = 1$、$\rho = 1/2$，即每隔一个时间槽产生一个令牌。注入过程在开启态的速率 r_1 必须满足什么条件，才能保证系统稳定（即调节器中数据包队列的期望长度有限）？在 $\alpha = 1/4$、$\beta = 1/8$ 时，构建该系统的马尔可夫链模型，求调节器队列的平均长度。针对不同 r_1 对模型进行仿真，记录平均队列长度，并使用利特尔定律（式 23.6）求调节器的期望延迟。

24.3 **相关工作负载**。本章把网络工作负载的三个方面作为独立维度讨论，但在很多应用中，这些维度可能存在强相关性。例如，特定源—目的对之间传输的长数据包可能比短数据包更少见。请描述一个可能出现这种现象的应用，并提出一种方法，将这种相关性纳入注入过程。

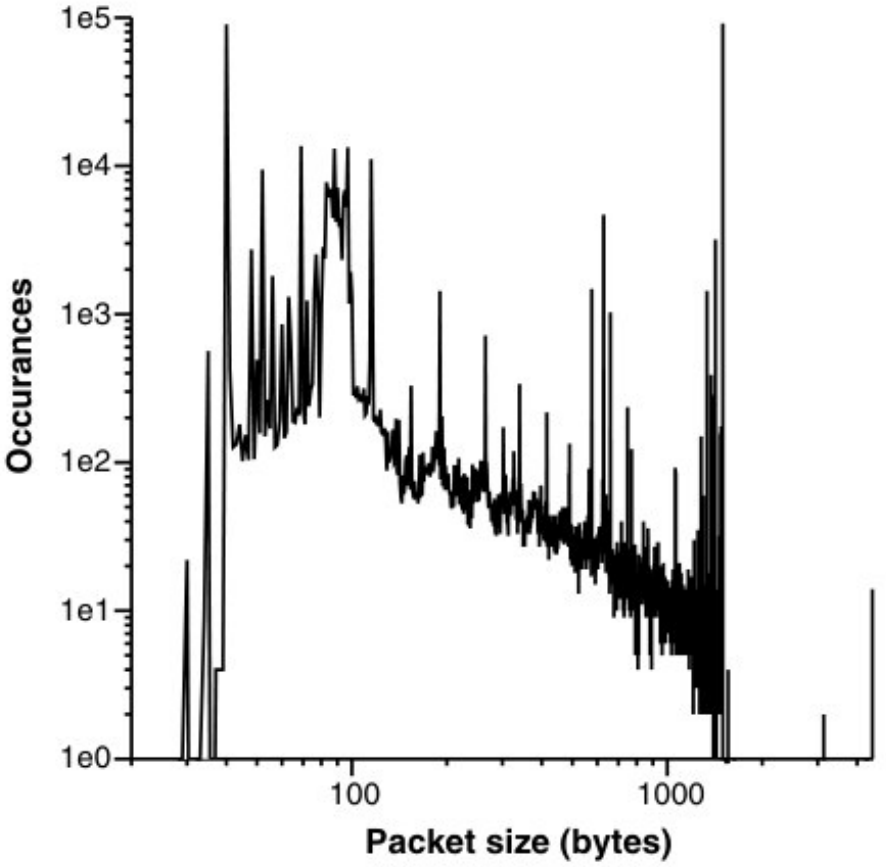


图 24.4　2002 年 10 月 4 日采集的互联网 IP 数据包长度分布

24.4 **硬币的公平性**。给定一枚硬币，要求判断它是否公平，即正反面是否各以 50% 的概率出现。前 11 次投掷的结果如下：

$\{H, T, T, T, H, T, H, H, T, H, T\}$

其中有 5 次正面（H）和 6 次反面（T）。

（a）你能以 95% 的置信度断言这枚硬币“近似公平”吗（正面概率在 49%～51% 之间）？如果不能，是否能在相同置信水平下对硬币的某个性质得出结论？本例的自由度为 $11 - 1 = 10$，表 24.1 中的 Student t 分布可供参考。可将正面赋值为 1、反面赋值

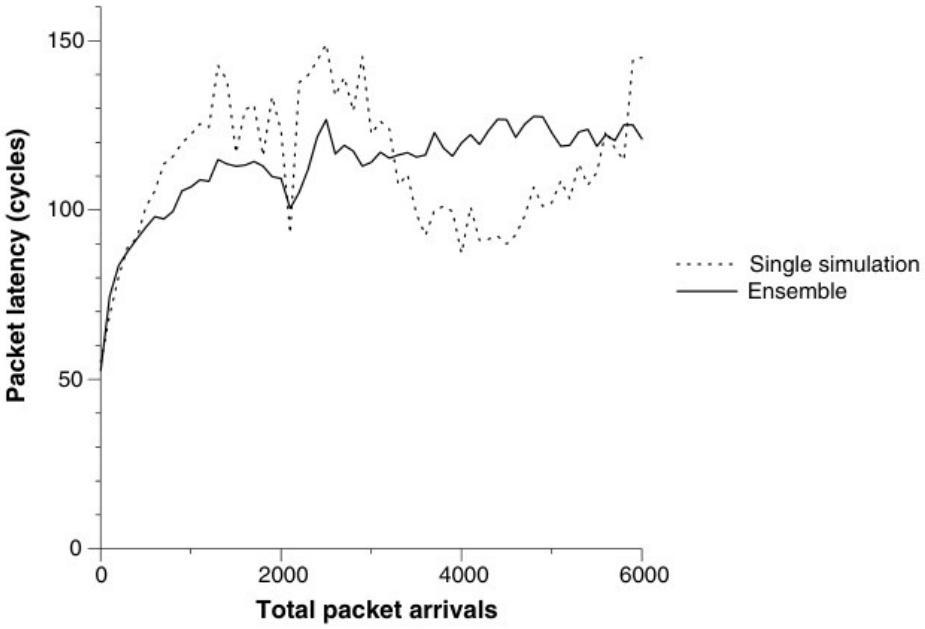


图 24.5 容量 70% 负载下二维 8 元网格的平均数据包延迟采样

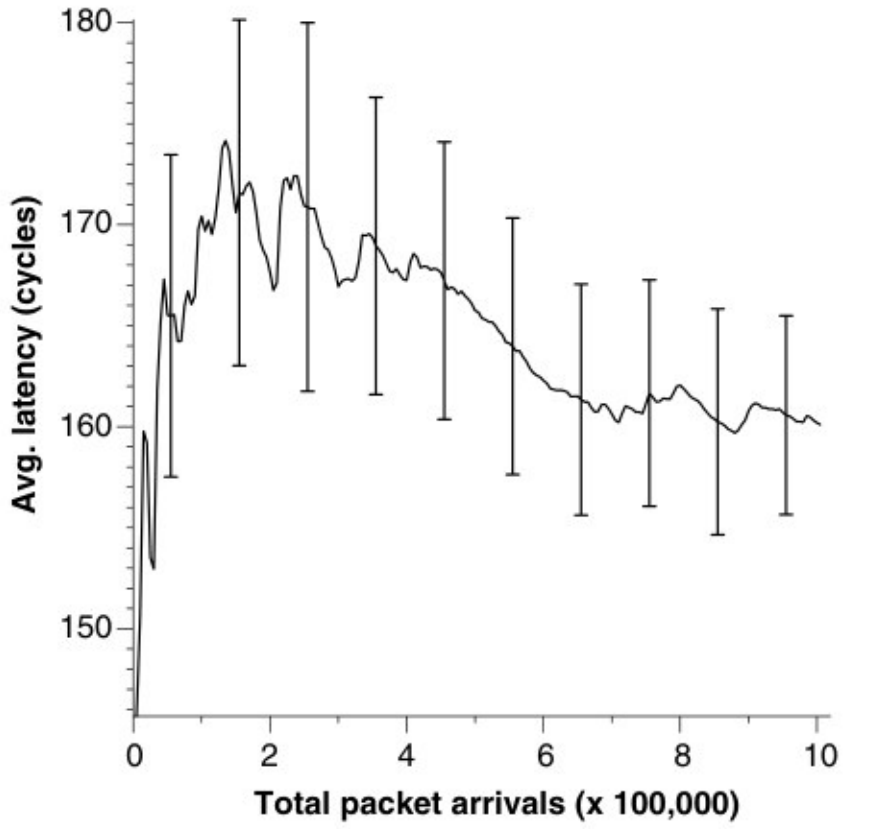


图 24.6 接近饱和时批均值法估计的平均延迟及 95% 置信区间

```
void WriteOutputs( int node ) {
  Packet *p;
  int output;

  // Visit each output queue at the node and select
  // a packet to be forwarded to the next node

  for ( output = 0; output < NumOutputs( ); outputs++ ) {
    if ( ! OutputQueueEmpty( node, output ) ) {
      p = SelectFromQueue( node, output );
      WriteOutgoing( node, output, p );
    }
  }
}
```

图 24.7　基于周期的输出排队交换网络仿真 C 风格代码示例

```
    if ( OutputQueueEmpty( node, output ) ) {
      AddEvent( 1, ScheduleOutput, node, output );
    }

    AddToQueue( node, output, p );
  }

  void ScheduleOutput( int node, int output ) {
    Packet *p;
    int    next;

    // Select a packet from the output queue
    // and forward it to the next node

    p    = SelectFromQueue( node, output );
    next = DownStreamNode( node, output );

    AddEvent( 1, Arrival, next, p );

    // If the output queue still contains
    // packets, another ScheduleOutput event
    // should occur in the next cycle

    if ( ! OutputQueueEmpty( node, output ) ) {
      AddEvent( 2, ScheduleOutput, node, output );
    }
  }
```

图 24.8　事件驱动型网络模拟器的简化实现示例

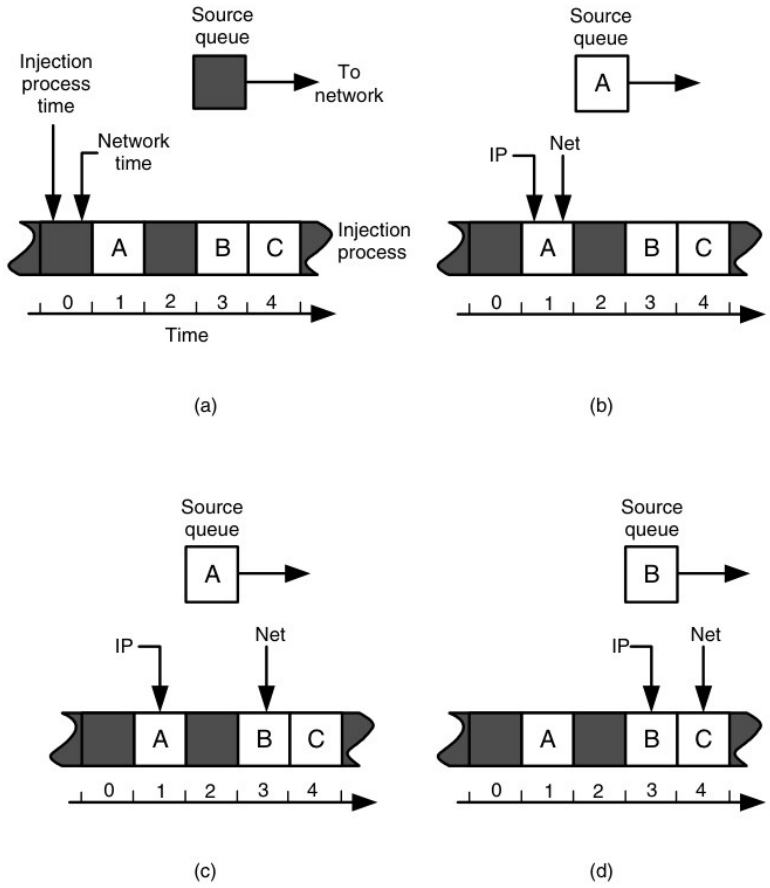


图 24.9 滞后注入过程（lagging injection process）的操作示例

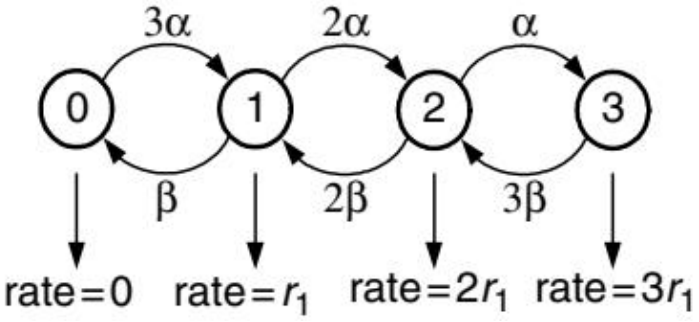


图 24.10 四状态马尔可夫调制注入过程（MMP）

为 0。
(b) 如果再投掷 90 次（总样本数 101），又出现 36 次正面（累计 41 次），能否对硬币的公平性给出更有力的判断?
24.5 **骰子的公平性**。按照习题 24.4 的方法进行类似实验，但改为掷一枚公平的六面骰子。持续掷骰，直到你能以 95% 的置信度断言其均值在 $(1+2+3+4+5+6)/6 = 3.5$ 的 ± 0.05 范围内。需要掷多少次?（使用 PRNG 模拟骰子会更快。）计算置信区间时，使用自由度趋于无穷时的 Student t 分布。此时你能否确信骰子公平，即六个面出现的概率相同？如果不能，应如何验证?
24.6 **单阶段仿真**。考虑为图 24.7 的输出排队交换机采用“单阶段”仿真，将 `ReadInputs` 与 `WriteOutputs` 合并。请解释在确定节点求值顺序时可能出现什么问题。在哪些条件下单阶段方法可能正常工作？提示：考虑被仿真网络的拓扑结构。
24.7 **滞后注入过程**。为第 24.4.2 节所述滞后注入过程编写伪代码，该过程每个网络周期调用一次。使用以下函数访问网络状态：`sourceq_empty()`——源队列无数据包时返回真，否则为假；`get_net_time()`——返回当前网络时间；`inject_packet()`——让注入过程运行一个周期，若该周期注入数据包则返回真，否则为假。假设滞后源队列的时间保存在变量 `q_time` 中。
24.8 **PRNG 的质量**。下列代码实现了一种常见的伪随机数发生器：乘法线性同余发生器。

```
int seed;
int random() {
    seed = (65539L * seed) % 0x80000000L;
    return seed;
}
```

首次调用 `random` 之前需要设置种子值。注意，`%` 是 C 语言中的取模运算符，十六进制数 `0x80000000` 等于 2^{31}。原英文印本将该十六进制常量误排为 `0x8000000`（少一个零），与书中 2^{31} 的说明不符；这里依照原式意图修正。
在网络模拟器中，一个典型用途是在 64 节点网络的均匀流量下选择目的节点：

```
dest = random() % 64;
```

实现这个随机数发生器及其生成随机目的节点的调用，并评论生成目的地址的“随机性”。提示：这不是一个好的随机数发生器。

本章注释　仿真细节层次越高，模型越精确，但计算时间和开销也越大。文中部分仿真方法依据文献 [142]；在统计估计中，一般无须进行超过 20～30 次相互独立的重复运行。关于伪随机数发生器的历史实例，可参见文献 [141]。

第二十五章　仿真实例

前面我们已经讨论了网络设计的各个方面以及网络仿真工具。本章给出若干仿真实例。这些实例并不打算全面研究某一种网络或路由器设计，而是旨在介绍一些适用于典型互连网络的实验方法，并突出若干有趣、甚至有违直觉的结果。

本章所有仿真均使用附录 C 所述的详细 flit（流控单元）级模拟器完成。除非另有说明，路由器采用输入排队结构，输入加速比为 2，并使用虚拟通道流控。每个输入端口有 8 个虚拟通道，每条虚拟通道有 8 个 flit 缓冲位置，因此每个输入端口共可缓冲 64 个 flit。所有数据包长度均为 20 个 flit。虚拟通道分配和交换分配均采用 iSLIP 算法。仿真考虑实际流水线延迟，每跳路由器延迟为 3 个周期。

25.1　路由

正如前面各章所述，路由设计必须在低业务量时的低延迟与业务量增大时的高饱和吞吐量之间取得精细的平衡。我们首先考察路由算法的延迟，分析零负载延迟、理想吞吐量等简单指标与网络实际性能之间的关系。值得注意的是，不同算法能够实现的理想性能比例并不相同。除常见的总体平均延迟外，我们还考察不同路由算法造成的数据包延迟分布。第二组实验则专门研究吞吐量，在随机抽样的多种流量模式上比较两种路由算法。

25.1.1　延迟

本组实验在八元二维网格（8-ary 2-mesh）上从延迟角度研究路由的影响。首先考虑互连网络研究中最常见的曲线之一：均匀流量下延迟随提供负载变化的关系，见图 25.1。图中比较四种算法：维序路由（DOR）、第 9.2.2 节及文献 [135] 所述的随机最小路径算法（ROMM）、Valiant 随机路由（VAL）和最小自适应路由（MAD）。这里的 MAD 采用 Duato 算法构建，以维序路由作为保证无死锁的子函数。

当流量较低时，零负载延迟能够准确估计仿真中的延迟。以一个 flit 通过一条通道所需的时间作为一个周期，路由器模型的延迟为 $t_r = 3$ 个周期；由于数据包长度为 20 个 flit，序列化延迟为 20 个周期。因此，对于最小路径算法，零负载延迟为

$$T_0 = t_r H_{\text{avg}} + T_s = 3\left(\frac{16}{3}\right) + 20 = 36 \text{ 个周期.}$$

图中结果与此一致。类似地，VAL 的零负载延迟为 52 个周期。随着业务量增大，争用延迟逐渐占据主导地位，延迟曲线的垂直渐近线由不同路由算法的饱和吞吐量决定。由于仿真准确建模了流控，路由算法实际达到的饱和吞吐量只是其理想吞吐量的一部

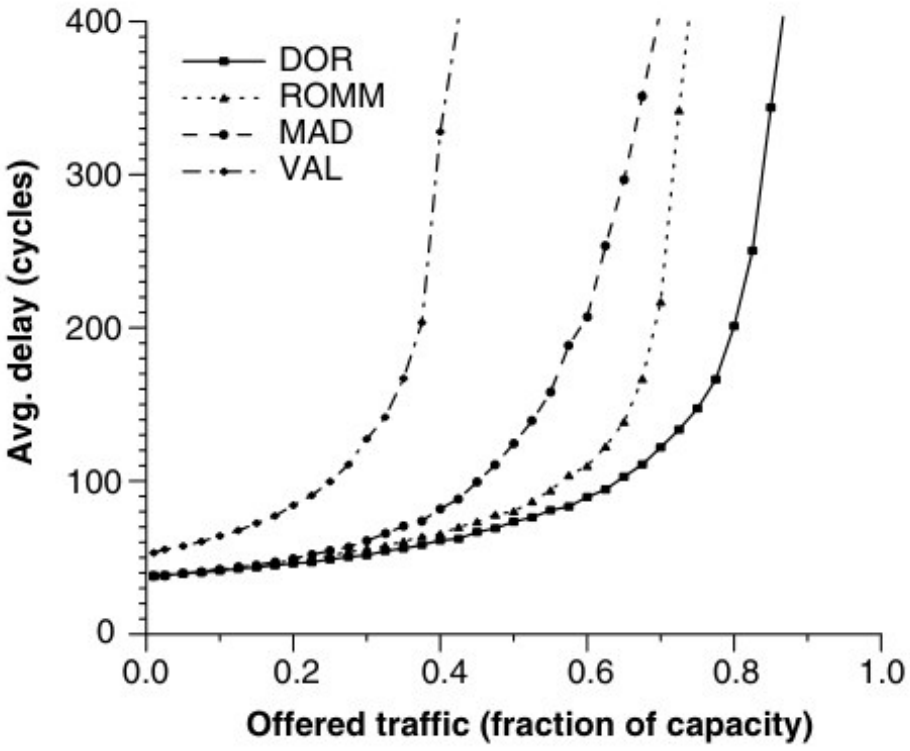


图 25.1　均匀流量下四种路由算法的延迟与负载关系

分。最小路径算法（DOR、ROMM、MAD）的理想值均为网络容量的 100%，而 VAL 的理想值仅为容量的 50%。出人意料的是，不同算法达到其理想值的比例也有所不同：DOR 接近其理想值的 90%，而 ROMM 和 MAD 只能达到大约 75%。原因在于，为避免死锁，我们对 ROMM 和 MAD 的虚拟通道进行了分区。维序路由在网格中天然无死锁，因此任何路由都可以自由使用任意虚拟通道。虚拟通道分区同大多数资源分区一样，可能造成负载不均衡，从而降低 ROMM 和 MAD 的实际吞吐量。VAL 虽然也需要划分资源来避免死锁，但由于天然具有负载均衡能力，仍能达到其理想值的约 85%。图 25.2 使用相同拓扑和相同四种路由算法，但将流量模式改为转置流量（transpose）。这种非对称模式更难实现负载均衡。DOR 的表现较差，在容量的约 35% 处饱和；ROMM 更好，吞吐量提高到约 62%；MAD 的表现最佳，吞吐量超过 DOR 的两倍，在容量的 75% 以上才饱和；VAL 也优于 DOR，达到约 43%。转置等困难流量模式下的性能很重要，但容易处理的局部流量（例如相邻节点流量）同样重要，如图 25.3 所示。对于相邻节点流量，尽管所有最小路径算法具有相同的理想吞吐量，DOR 的简洁性和天然无死锁特征却使它优于更复杂的 ROMM 和 MAD。与预期一致，VAL 在这三种流量模式下的性能相同。

除了上述三个实验使用的总体平均延迟外，单个数据包的延迟还可以揭示实际观测值的范围与分布。图 25.4 给出此前实验中八元二维网格在均匀流量、维序路由下，两个源—目的节点对的数据包延迟分布。提供负载保持为网络容量的 20%。在这种低负载下，数据包很少发生争用，多数都以最短周期数到达，因此分布左端形成明显尖峰。从 $(0,0)$ 到 $(4,4)$ 的数据包延迟较高，仅仅是因为它们需要经过更多跳。

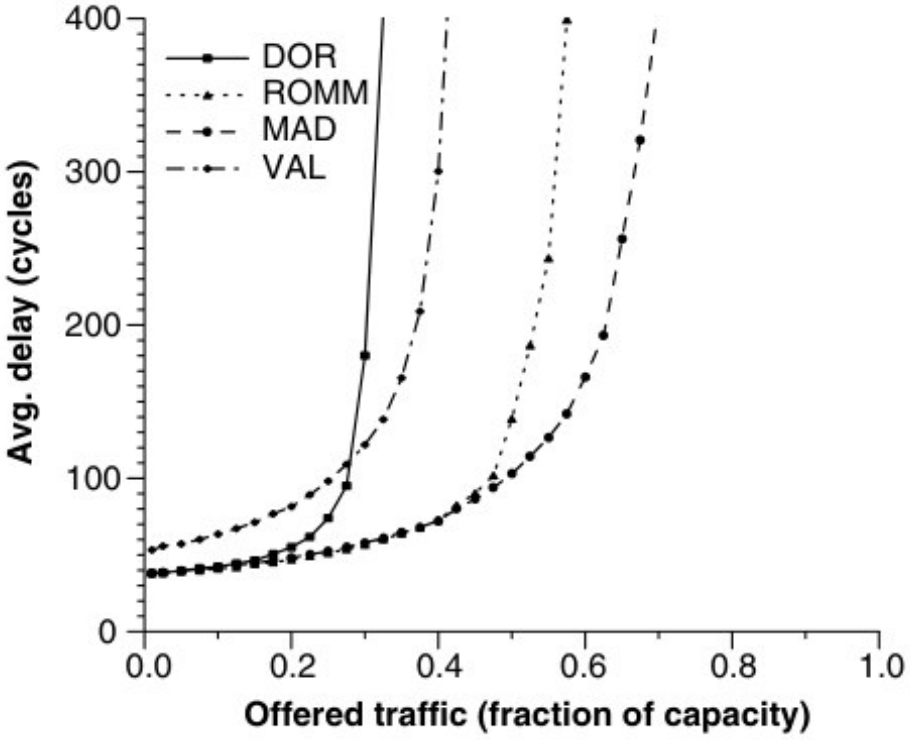


图 25.2 转置流量下四种路由算法的延迟与负载关系

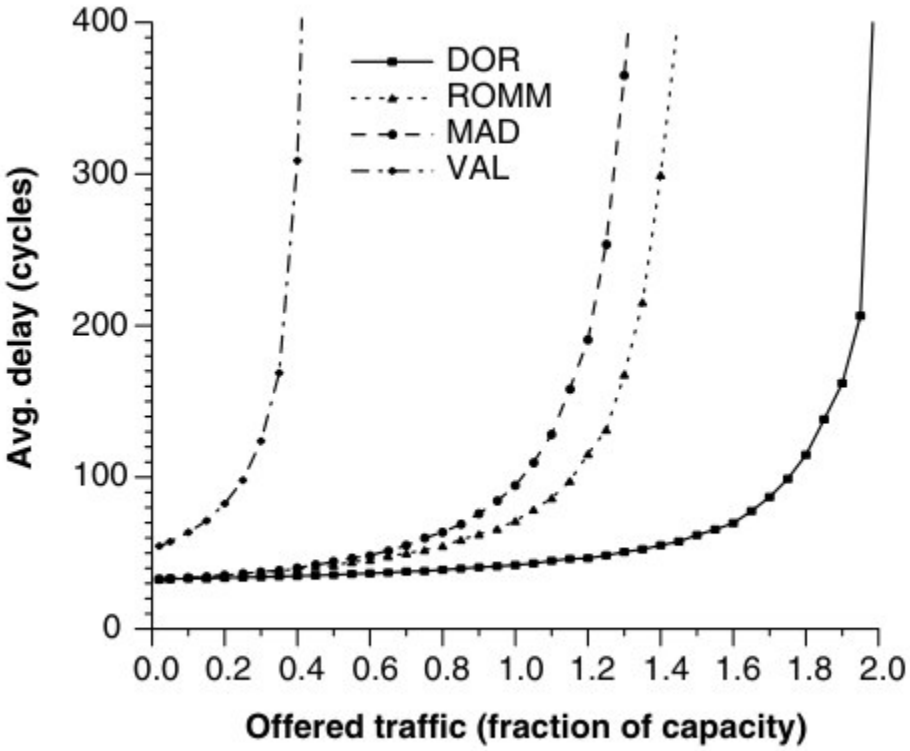


图 25.3 相邻节点流量下四种路由算法的性能比较

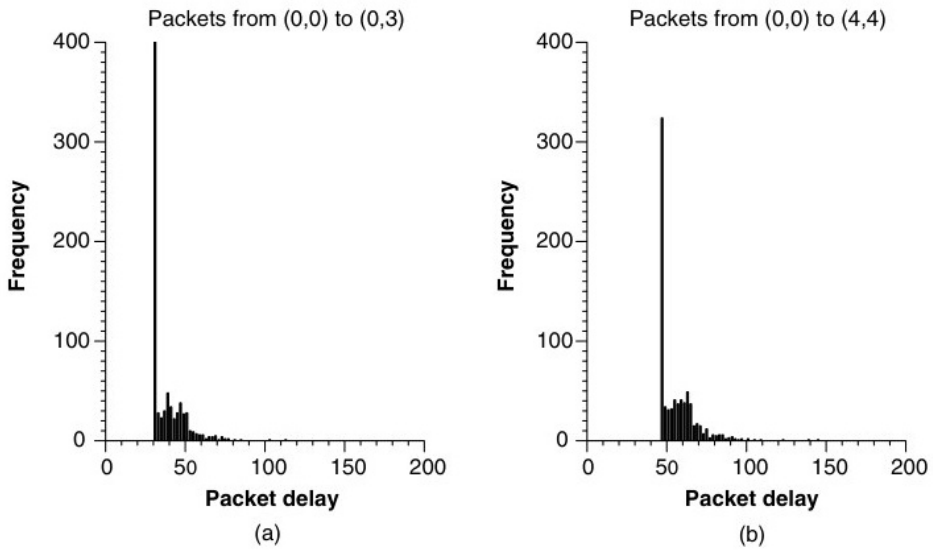


图 25.4　八元二维网格在均匀流量下使用维序路由时的数据包延迟分布：（a）$(0,0)$ 至 $(0,3)$；（b）$(0,0)$ 至 $(4,4)$。提供负载为网络容量的 20%。

同样的仿真改用 Valiant 路由后，会得到更加有趣的分布，如图 25.5 所示。由于数据包首先前往一个随机中间节点，路径长度会出现很大差异。每一种具体路径长度都会产生类似维序路由下的延迟分布，而 VAL 的总分布就是许多此类分布的加权叠加。从 $(0,0)$ 路由到 $(0,3)$ 时，多数数据包走的是非最小路径，所以整体呈钟形；而从 $(0,0)$ 到 $(4,4)$ 时，源与目的节点之间距离更远，随机中间节点落入最小路径象限的机会更大。一旦落在该象限，总路径就是最小路径，这解释了后一种情况下分布向左移动的现象。

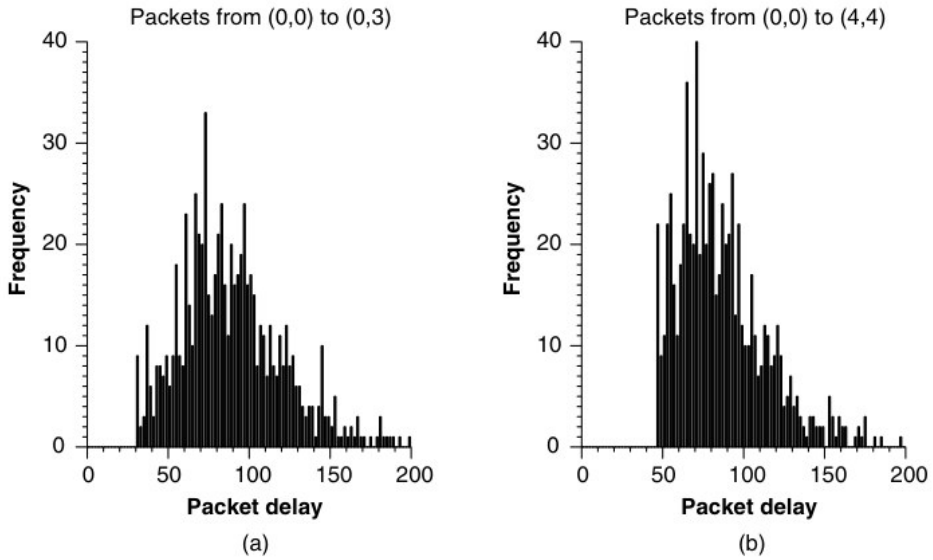


图 25.5　八元二维网格在均匀流量下使用 Valiant 随机路由时的数据包延迟分布：（a）$(0,0)$ 至 $(0,3)$；（b）$(0,0)$ 至 $(4,4)$。提供负载为网络容量的 20%。

25.1.2 吞吐量分布

接下来只考察路由算法的吞吐量表现。用于测试互连网络的标准流量模式能够显示网络在若干极端情况下的性能，却未必能很好地反映网络的平均行为。为了克服只使用少数几种流量模式的局限，可以对大量随机排列流量模式进行抽样，并测量网络的吞吐量分布。

本实验对八元二维立方体网络（8-ary 2-cube）的 500 个随机排列样本，分别测试维序路由与最小自适应路由的吞吐量。图 25.6 给出两种算法的饱和吞吐量分布。DOR 的分布有两个明显峰值，分别位于容量的约 27% 和 31%，500 个样本的平均吞吐量约为容量的 29.4%。最小自适应路由的分布更均匀，只有一个位于容量约 33% 附近的宽峰，平均吞吐量约为 33.3%。这一数值比维序路由高约 13.3%，体现了自适应路由的潜在优势。

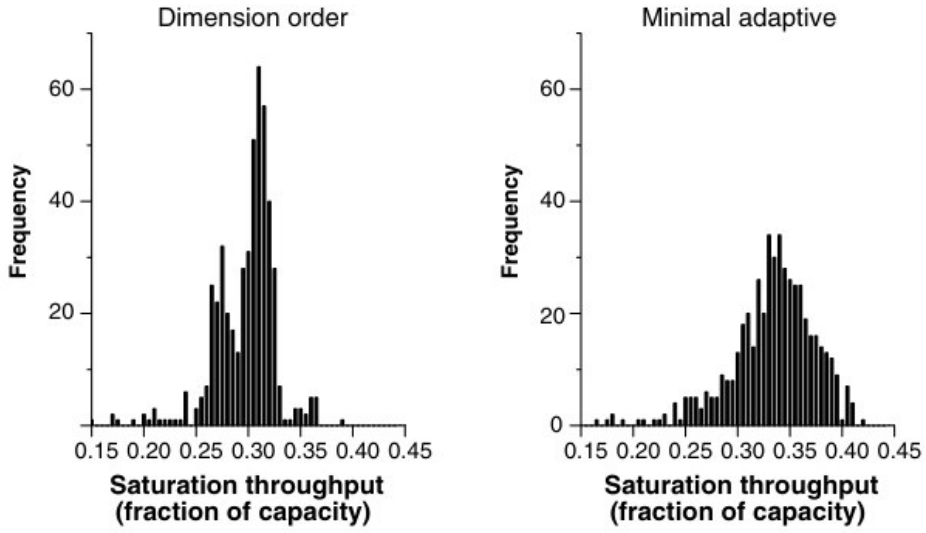


图 25.6 八元二维环面网络中 500 种随机排列流量模式的饱和吞吐量分布：(a) 维序路由；(b) 最小自适应路由。

25.2 流控性能

网络所用的路由算法确定了可实现吞吐量的上界（理想吞吐量）与数据包延迟的下界（零负载延迟）。网络实际运行性能能在多大程度上接近这两个界限，则取决于流控机制。本节的实验关注网络及其工作负载的不同方面如何显著影响流控机制的性能。

25.2.1 虚拟通道

典型的虚拟通道路由器设计会预留固定数量的硬件资源作为虚拟通道缓冲区。设计者必须决定如何划分这些资源以最大化网络性能。例如：较少但缓冲区较深的虚拟通道，是否比数量更多但缓冲区较浅的虚拟通道更好？

图 25.7 比较八元二维网格网络上不同虚拟通道分区方案的性能。各配置的缓冲总量

保持不变，即虚拟通道数量与每个虚拟通道深度的乘积相同。实验显示几个趋势。第一，虚拟通道数量增加时，网络吞吐量一般也会提高。虽然图中没有表现出来，8 条虚拟通道配置的饱和吞吐量略高于 4 条虚拟通道配置。第二个趋势掩盖了这一事实：增加虚拟通道数量往往会使饱和前的延迟变大，这是因为更多虚拟通道导致数据包之间更频繁的交错，使数据包在网络中的占用时间被拉长。可以在交换分配器中给予未阻塞、且上一轮已经获得分配的数据包优先权，从而减轻这种交错效应。但设计者必须谨慎，防止出现饥饿，以及可变长数据包引起的公平性问题。

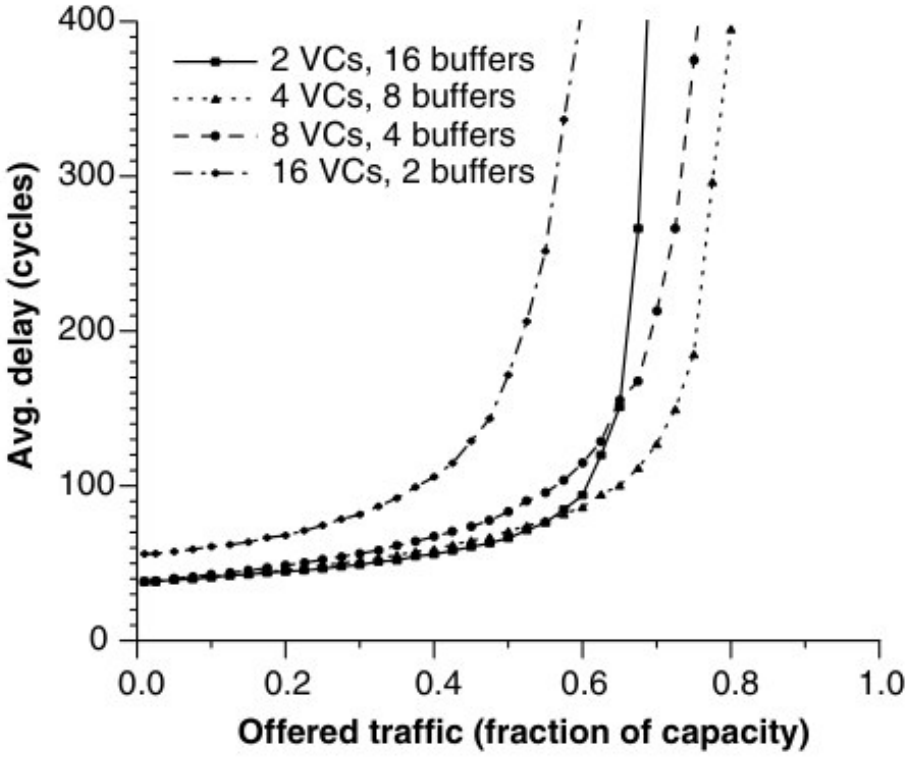


图 25.7　虚拟通道数量和缓冲深度对网络延迟及吞吐量的影响

16 条虚拟通道配置是吞吐量趋势中的一个例外。它的零负载延迟不同，正是底层问题的关键迹象。路由器模型包含流水线延迟，因此缓冲区的信用返回环路（credit loop）延迟大于一个周期。当虚拟通道缓冲深度小到不足以覆盖这一延迟时，虚拟通道便无法持续达到 100% 利用率，必须停下来等待信用返回。这与第 16.3 节描述的效应相同。由此产生的信用停顿同时影响零负载延迟和饱和吞吐量。

虽然通常可以用缓冲总容量近似虚拟通道的硬件成本，但增加虚拟通道数量并不总是在延迟方面没有代价。一般来说，虚拟通道越多，虚拟通道分配时间越长，可能影响路由器流水线；这反过来可能提高零负载延迟，并因重新分配虚拟通道所需时间增加而略微降低饱和吞吐量。流水线限制和上述覆盖信用环路所需的最小缓冲深度，往往限制了 “虚拟通道数量接近最大、每条缓冲却很浅” 的配置。然而，非干扰性等其他重要因素，仍可能使这种极端分区成为有吸引力的设计点。

25.2.2 网络规模

网络规模可能显著影响它能实现多少比例的理想吞吐量，如图 25.8 所示。该图给出四种网格网络在均匀流量下的延迟—提供负载曲线。需要注意，四种网络的通道宽度和路由器都完全相同；因此，它们的容量取决于网络的基数。例如，四元三维网格（4-ary 3-mesh）和四元四维网格（4-ary 4-mesh）的容量都是 $4b/k = b$；八元二维网格（8-ary 2-mesh）的容量为 $4b/k = b/2$，即四元网络的一半。

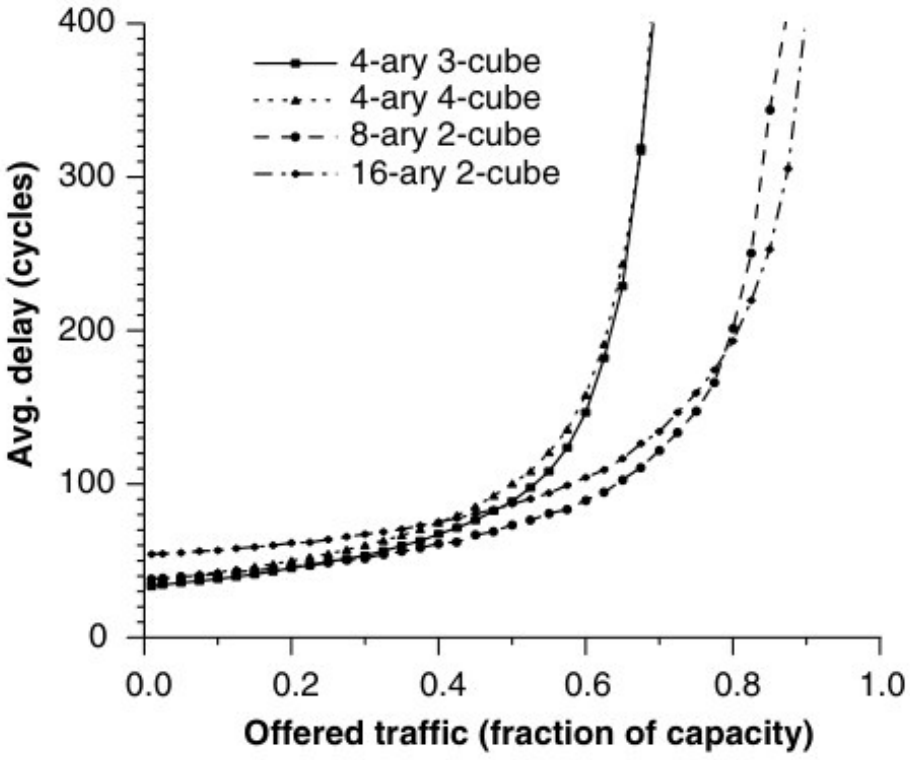


图 25.8 不同网格网络规模与基数下的延迟及吞吐量

人们可能自然地认为，用相同路由器构建的不同网络应能达到大致相同的容量利用比例，实际结果却并非如此：该比例似乎取决于网络基数。两种四元网络都在约 65% 时开始饱和，八元和十六元网络则分别在约 80% 和 83% 时饱和。进一步仿真也证实，实现的容量比例由网络基数决定这一趋势。

这种结果可以从不同网络规模与流控机制的相互作用来解释。使用容量百分比，可以比较容量不同的网络，却掩盖了各节点注入的绝对吞吐量。由上面的容量关系可知，在达到饱和之前，四元网络中的每个节点理想情况下可以注入八元二维网格中节点两倍的业务量、十六元二维网格中节点四倍的业务量。虽然各节点仿真使用的注入过程相同，总体业务量却显著不同：四元网络由较少节点注入较多流量，较大基数网络则由更多节点注入较少流量。这种差别会显著影响流控；少数高强度源造成的瞬时负载（突发性）比大量低强度源更大。流控机制要缓解这种突发性，效果主要取决于各节点可用的缓冲容量。由于所有网络使用相同路由器，较小基数网络缓解突发性的能力相对较差，因此实现的容量比例更低。

25.2.3　注入过程

上一节关于网络规模的实验表明，底层流量的突发性会影响流控效率。本节通过改变网络仿真的注入过程，进一步考察这一现象。

网络中最简单的一种突发性来源就是数据包大小。即使某个源的平均注入率非常低，最小注入单位仍是一个数据包，而一个数据包可以包含许多 flit。它可以视为一股目的节点相同的 flit 突发流。图 25.9 给出了数据包大小对网络性能的影响。

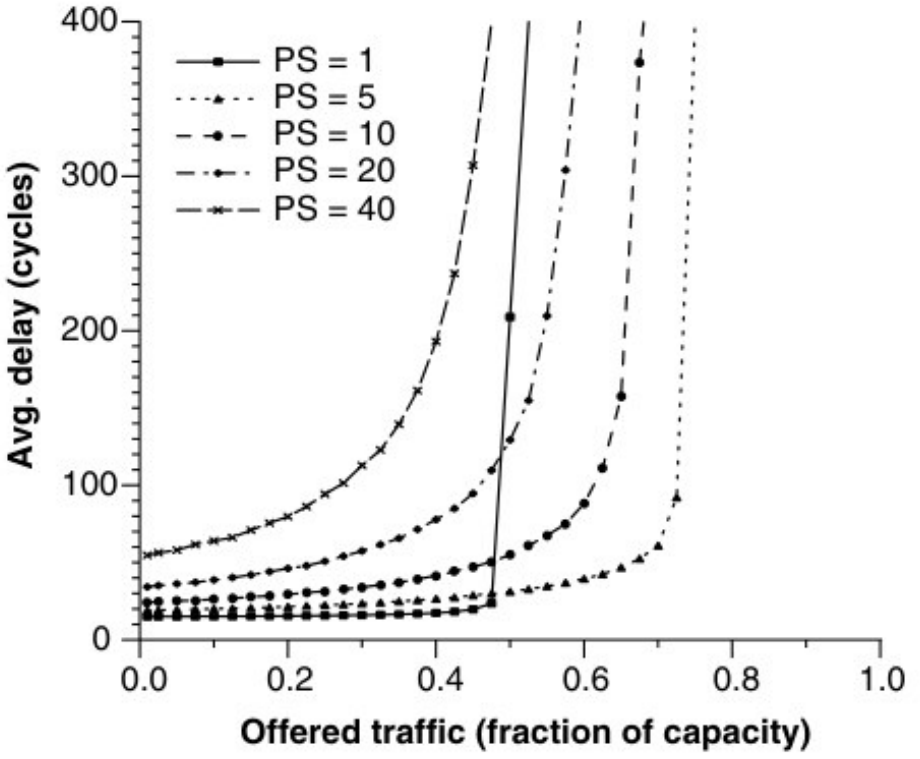


图 25.9　数据包长度对网络延迟与饱和吞吐量的影响

数据中的主要趋势是：数据包越长，延迟越高、吞吐量越低。较长数据包由于序列化开销较大，其零负载延迟本来就处于劣势。此外，流控机制并不完美，数据包越长，越难有效利用资源。例如，一个数据包包含 40 个 flit，而每个路由器的缓冲深度只有 8 个 flit，则它至少跨越 5 个路由器。一旦被暂时阻塞，至少 5 个路由器的资源也会被占住，导致饱和吞吐量下降。相比之下，较短的数据包占用的资源更少。

总体趋势中的例外是数据包大小为一个 flit（$PS = 1$）的情况。路由器模型对虚拟通道重新分配采取第 16.4 节图 16.7(a) 所示的保守方法，因此虚拟通道能够再次使用之前，需要经过若干周期。数据包越短，虚拟通道使用时间中花在重新分配阶段的比例就越大。当数据包只有一个 flit 时，路由器的有效虚拟通道数量大幅下降。进一步仿真确认：只要缩短重新分配时间，或增加虚拟通道数，这一异常就会消失。

为进一步考察注入过程的影响，我们使用第 24.2.2 节讨论的双状态马尔可夫调制过程（MMP），模拟网格网络。数据包长度仍固定为 20 个 flit。图 25.10 展示 MMP 不同参数下的网络性能。每种 MMP 由 α 和 β 两个参数控制，决定突发之间的间隔与突发持

续时间。$1/\alpha$ 是“开启”突发之间的平均间隔；图中三条曲线对应的平均间隔分别为 1、200 和 400 个周期。类似地，β 可解释为平均突发持续时间的倒数；第一条曲线的突发持续时间为无穷大，第二和第三条曲线的平均持续时间分别为 100 和 50 个周期。

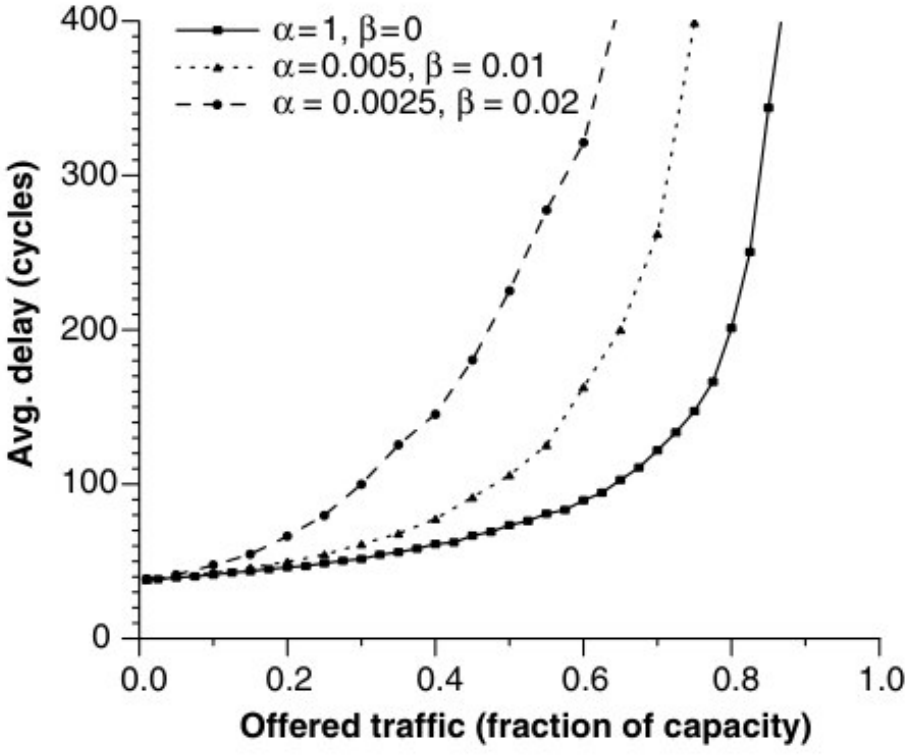


图 25.10 不同突发注入参数下的网络延迟与吞吐量

第一种 MMP 的突发长度无穷大，意味着它始终处于开启状态，因而退化为伯努利注入过程；此时 α 的取值不影响稳态。根据第 24.2.2 节的分析，突发期间的注入速率是平均注入速率的 $1+\beta/\alpha$ 倍，β/α 越大，突发期注入越强烈。第二种 MMP 中，$\beta/\alpha = 0.01/0.005 = 2$；当提供负载为容量的 40% 时，注入过程在没有数据包的阶段和以容量的 $40(1+2) = 120\%$ 进行伯努利注入的阶段之间交替。如预期所示，这种突发行为会增大平均延迟、降低饱和吞吐量（图 25.10）。第三种 MMP 的突发更短且更剧烈，$\beta/\alpha = 0.02/0.0025 = 8$，因此平均延迟和饱和吞吐量相对于第二条曲线进一步恶化。

25.2.4 优先级划分

大多数网络延迟指标关注消息的总体延迟或平均延迟，但不同流控方法能够显著改变单条消息延迟的分布。对于关心最坏情况延迟、抖动、公平性，或网络中存在不同优先级业务的应用，控制这些分布尤其重要。

考虑具有两类消息的网络。一类可以是要求低延迟、低抖动的实时视频流量；另一类可以是容忍延迟的数据传输。为了确保高优先级视频流量满足低延迟要求，可以赋予它相对于数据传输流量的绝对优先权。

图 25.11 给出了在二元六级蝶形网络（2-ary 6-fly）中建立两种优先级后的延迟分布。实验把 10% 的流量作为高优先级，其余 90% 为低优先级，在接近饱和状态采集数据。为了在路由器模型中体现优先级，使用第 19.3 节所述、由优先级仲裁器构成的可分离分配器。优先级仲裁器总是先选择优先级最高的请求者，当优先级相同时再通过轮转策略裁决。

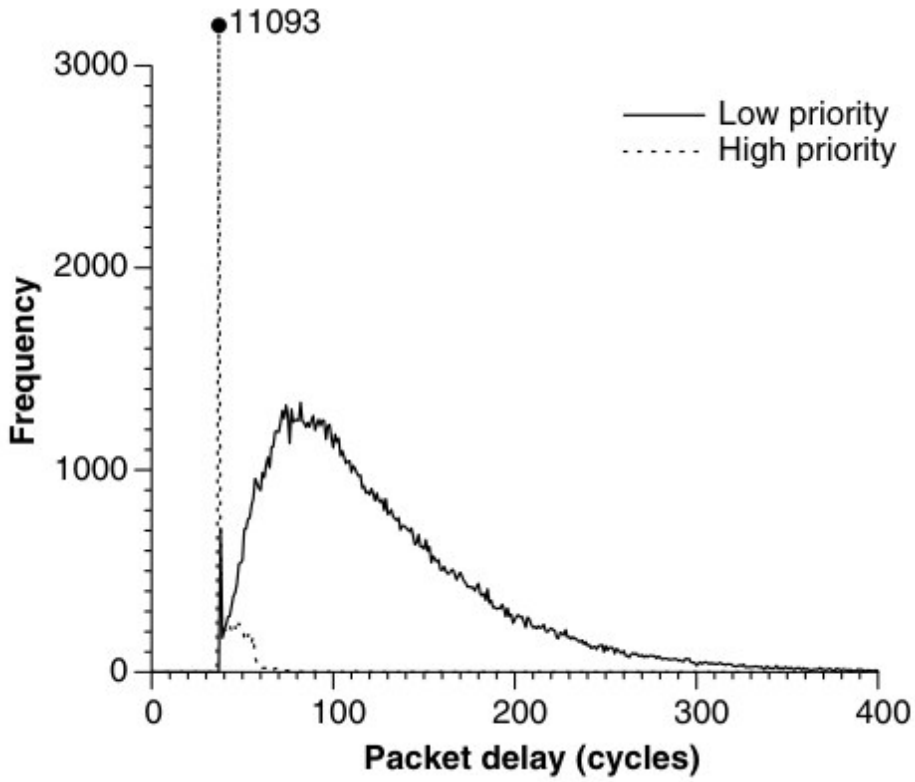


图 25.11　高优先级与低优先级消息的延迟分布

图中分布给出各种延迟值出现的次数。尽管网络接近饱和，抽取的 15,612 条高优先级消息中仍有 11,093 条（约 71%）在网络最小延迟 37 个周期内送达，近 99% 的高优先级消息在 70 个周期内送达。低优先级消息的分布则反映出接近饱和时较高的平均延迟。虽然超过 98% 的低优先级流量在 300 个周期内送达，但分布尾部一直延伸至 700 多个周期。

能够在两类业务之间形成这种差异，是因为高优先级流量只占网络总流量的一小部分。优先级是绝对的，所以高优先级流量看到的网络负载很轻，不到容量的 10%，争用延迟很小。不过，随着高优先级流量占比增大，两类流量之间的差异会逐渐减小；当几乎所有流量都属于高优先级时，设置优先级就几乎没有收益。

另一种重要的消息优先级机制近似于第 15.4.1 节介绍的基于年龄的公平性（age-based fairness）。它与上述双优先级案例一样，通过路由器中的优先级分配器实现；区别在于，请求者发生争用时，优先为最早注入网络的数据包分配资源。数据包的 “年龄” 是自其首次注入网络以来经过的网络周期数。

图 25.12 比较二元六级蝶形网络采用和不采用基于年龄的仲裁时的数据包延迟分布。

测量时网络同样接近饱和。不使用年龄仲裁时，有更多数据包能在大约 50 个周期内快速送达，但延迟分布尾部很长，仍有相当多数据包需要超过 600 个周期才能送达。使用年龄仲裁时，能够很快送达的数据包略少，但分布尾部受到抑制，大多数数据包在 400 个周期内到达。

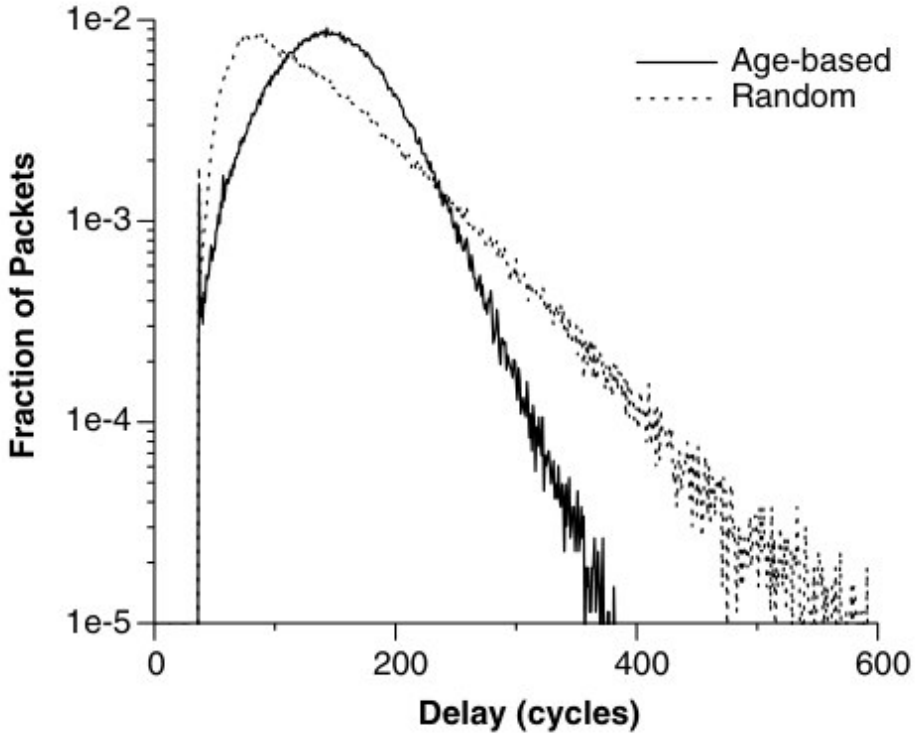


图 25.12 基于年龄的仲裁与随机仲裁的数据包延迟分布比较

25.2.5 稳定性

当网络接近并超过饱和吞吐量时，设计者的关注点通常从延迟转向流控技术的公平性。如果多个业务流未能公平分享饱和通道，网络可能变得不稳定：某些流量发生饥饿，当负载超过饱和点时，其吞吐量可能急剧下降。图 25.13 对比不稳定网络和两种通过公平性机制确保稳定的流控方案。为了避免某些贪婪流掩盖饥饿流，该图采用第 23.1.1 节所述的最小接纳吞吐量指标。

三种流控技术在饱和之前的性能相近，都在网络容量的约 43% 处达到饱和。超过饱和点后，三者开始分化。完全不使用公平性机制时，随着提供负载继续增加，吞吐量骤降至容量的 5% 以下。这种不公平性类似第 15.4.1 节停车场问题：所需跳数较少、因而资源仲裁次数较少的数据包，会获得不成比例的带宽。相比之下，加入基于年龄的仲裁后，超过饱和点时吞吐量仍然稳定。采用 “每个目的节点一条虚拟通道” 的非干扰网络，在饱和后同样保持稳定，但吞吐量会有一定下降，最后稳定在容量的约 35%。

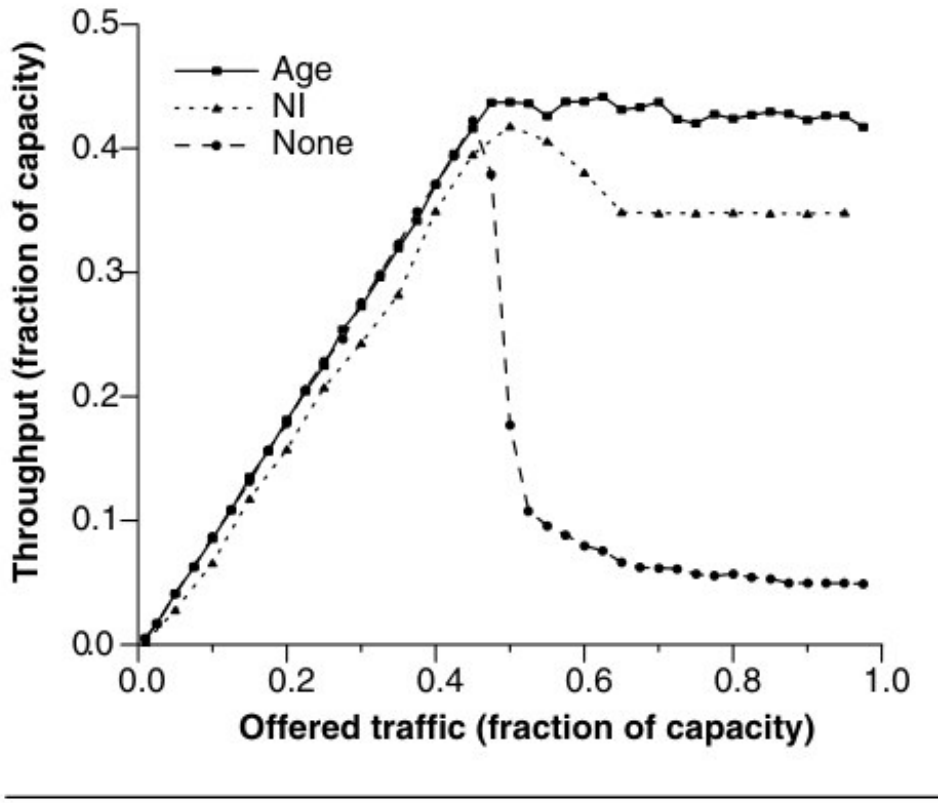


图 25.13　公平性机制对网络稳定性及最小接纳吞吐量的影响

25.3　容错性

对许多互连网络而言，存在一个或多个故障时仍可运行，是一项重要特性。此外，人们还希望网络性能随故障增加而平稳退化。图 25.14 给出了这种渐进退化的示例。

实验采用八元二维网格网络，改变失效链路数量（横轴），使用习题 14.8 所述的容错平面自适应路由变体进行仿真。[1]对每个故障数量，在均匀流量下测量网络的饱和吞吐量（纵轴）。由于不同的故障链路分布可能对吞吐量造成不同程度的影响，每个吞吐量数据点取 30 种失效链路配置的平均值；同时通过竖直误差棒给出样本标准差。误差棒上下端点分别为平均吞吐量加、减一个标准差。

无故障网络的吞吐量略高于容量的 60%。少量故障只引起较小的吞吐量下降，表明网络具有一定的渐进退化能力。即使故障数增至 12，吞吐量下降速度也只略有增加。同时，标准差随着故障数缓慢上升，说明多个故障的组合影响可能比单个故障更严重。例如，当故障集中在网络中的一小片区域时，通往该区域节点的通道数量可能减少，剩余通道的负载便会显著增加。故障越多，故障聚集导致某个节点近乎隔离的概率越大；在极端情况下，网络可能被分割，以致某些节点之间不存在可用路径。

[1]按照原书说明，实验将故障区域设置为凸形，从而确保平面自适应路由仍能连通所有网络节点；相关细节见原著参考文献 [36]。

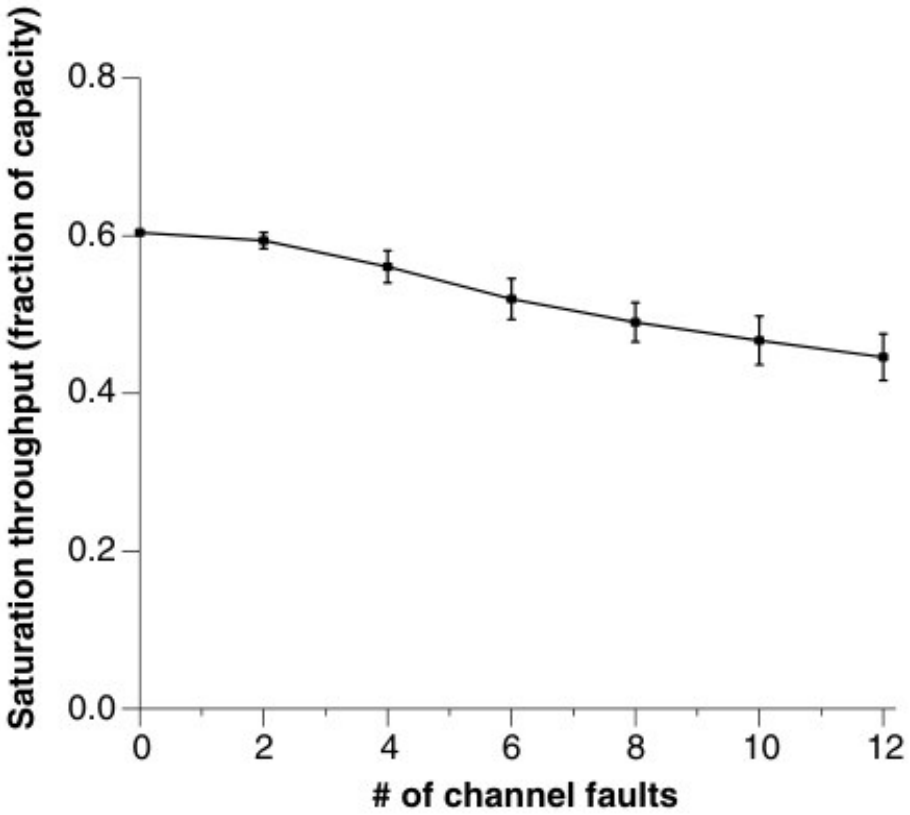


图 25.14 失效链路数量对网络饱和吞吐量的影响（误差棒为标准差）

附录 A　符号约定

符号	定义	计量单位
b	通道带宽，即信号频率与通道宽度的乘积，$b = fw$。	bits/s
B_B	二分带宽：穿越网络最小二分割的全部通道带宽之和。	bits/s
B_C	通道二分割宽度：穿越系统最小二分割的通道数量。	channels
B_n	节点带宽：进入或离开节点的通道带宽总和；它等于节点引脚容量与信号频率的乘积，$B_n = fW_n$。	bits/s
B_s	系统带宽：可穿越系统中部的最大导线数量所提供的带宽；$B_s = fW_s$。	bits/s
C	网络中全部通道的集合。	
d	占空比。	
D	距离：源节点到目的节点路径上经过的通道物理长度。	m
δ	节点度数：连接到某节点的通道数量。	channels
f	频率：每根通道信号线提供的带宽。	1/s
F	每条虚拟通道的微片缓冲区数量。	
γ	通道负载：通道上的带宽与网络输入带宽的比值；等价地，单位输入负载施加到某通道上的负载。	
H	跳数：源节点至目的节点路径所经过的通道数量。	channels
l	通道长度，即一条通道的物理长度。	m
λ_{xy}	路由概率：从节点 x 路由到节点 y 的流量占总流量的比例。	
L	数据包长度。	bits
N	网络中节点的集合。	
N	网络的终端节点数。	
P	路由器的端口数量。	
R_{xy}	从 x 到 y 的全部最短路由集合。	
R'_{xy}	从 x 到 y 的全部路由集合，包括最短和非最短路由。	
t	通道传播延迟：信号穿过一条通道所需的时间。	s
t_r	路由器延迟：信号通过一台路由器的延迟。	s

t_c	信用延迟：信用在路由器两端流经路由器时经历的周期数。	cycles
t_{ck}	时钟周期。	s
t_{crt}	信用往返延迟：微片离开路由器的 SA 阶段，到信用返回并允许下一微片复用同一缓冲区的往返时间。	cycles
t_f	微片延迟：微片通过一台路由器经历的时钟周期数。	cycles
T	端到端延迟：数据包的头微片进入网络，到尾微片离开网络的时间。	s
T_h	头部延迟：数据包头微片到达目的地所需的时间。	s
T_r	路由器延迟分量：T_h 中由路径上的路由器产生的部分，$T_r = Ht_r$。	s
T_s	串行化延迟：在一条链路上发送整个数据包所需时间。原书符号表记作 $T_s = b/L$，与量纲关系不一致，校订说明另有记录。	s
T_w	飞行时间：T_h 中由通道导线传播造成的延迟部分。	s
Θ	吞吐量：网络每个输入端能够接受的最大业务流量。	bits/s
v	信号传播速度：信号沿通道传播的速度，$t = l/v$。	m/s
w	通道宽度。	bits
W_n	节点导线容量：能够进入或离开节点的信号线数量；当拓扑受节点限制时，W_n 会限制网络带宽。	signals
W_s	系统导线容量：能够穿过系统中部的信号线数量；与信号频率 f 一起限定二分带宽 B_B。	signals

附录 B　术语释义

英汉专业术语表（保留英文词条，便于与原书互查）

accepted traffic（已接纳流量）——见“吞吐量”（throughput）。

adaptive routing（自适应路由）——数据包的路径根据网络当前状态动态决定。例如，路由算法可以为避免拥塞或故障链路而选择不同路径。

ASIC（专用集成电路）——application-specific integrated circuit。

availability（可用性）——系统正常运行的时间比例，常用若干个“9”表示。例如，五个 9 的可用性相当于 99.999% 的时间正常运行。

backpressure（反压）——反映下游资源使用程度的信息。流控利用反压防止缓冲区溢出；在一些算法中，反压也可用于动态选择路由。

blocking（阻塞）——如果一个网络不能同时满足所有输入与输出之间一一配对的电路连接请求，则称为阻塞网络。

CAM（内容可寻址存储器）——content-addressable memory。

capacity（容量）——网络在均匀随机流量下的吞吐能力。用容量对测得吞吐量归一化，可使不同规模网络之间的性能比较更有意义。

dateline（日期线）——在环形通道（或环面网络的某一维）上设置的一条概念分界线。数据包跨越该线时从较低的资源类别切换到较高类别，以打破通道依赖环，避免死锁。

deadlock（死锁）——一组持有资源的主体又等待其他资源，且等待关系形成环路，使其无法继续推进。许多网络死锁可通过限制路由依赖或采用虚拟通道资源类别来避免。

deterministic routing（确定性路由）——数据包所走路径完全由其源地址和目的地址决定。在给定源—目的对之间，所有数据包采用相同路径，与当前网络状态无关。

downstream（下游）——对于某通道，数据包沿该通道的前进方向是下游方向；通道目的端的路由器称为下游路由器。反压等控制信息则沿相反方向传播。

ECL（发射极耦合逻辑）——emitter-coupled logic。

escape channels（逃逸通道）——为每个数据包提供一组无死锁的可用路径。其他通道可以提供更灵活的路由选择，但出现循环依赖时，数据包仍可通过逃逸通道继续前进。

fault tolerance（容错性）——网络检测、隔离故障资源并从故障中恢复的能力。

FPGA（现场可编程门阵列）——field-programmable gate array。

flit（流控单元）——flow control digit，路由器分配资源时采用的最小单位。可变长度的数据包通常划分为一个或多个固定长度 flit，以简化流控。

flow（业务流）——在同一个源—目的对之间传输的数据包序列，通常也是提供服务质量保证的单位。一个源或目的可同时参与多条业务流。

flow control（流量控制）——对网络资源的调度和分配，包括通道带宽、缓冲空间及

控制状态等。

folding（折叠）——利用拓扑的镜像对称性合并节点。例如，把 Clos 网络的第一级与第三级交换部件折叠，可形成共享部件的网络结构。

hot-spot（热点）——某个资源的需求显著超过其他同类资源。例如，大量输入同时访问某个目的终端时，该终端成为流量热点。

jitter（抖动）——同一业务流内两个数据包经历的延迟之间的最大差值。视频流等实时业务常要求较小抖动，以保持均匀的交付速率。

latency（延迟）——一个数据单位（通常是数据包或消息）通过网络所需的时间，常定义为首位在源端注入到末位在目的端交付之间的经过时间。

livelock（活锁）——数据包始终无法到达目的地，但不同于死锁，数据包可能持续在网络中移动，却没有真正向目的地取得进展。

load balance（负载均衡）——网络资源使用程度的均匀性。资源在相近负载下达到饱和，通常意味着网络负载均衡较好。

loss（丢包率）——网络丢弃的消息比例。共享内存互连等应用通常不允许丢包，因为丢失的消息可能破坏系统正确性；某些通信系统则能借助重传等机制处理丢包。

LSB（最低有效位）——least significant bit。

message（消息）——网络接口提供的逻辑数据传输单位。消息长度未必有上界，因此一般需要划分为一个或多个有界长度的数据包，由网络逐包处理。

MIMD（多指令流多数据流）——multiple-instruction, multiple-data 并行计算机。

minimal（最短路由）——源与目的之间的路由如果经过可能的最少跳数，便是最短路由。在网格与环面网络中，最短路由的下一跳通常限定在能减少剩余距离的方向上。

MSB（最高有效位）——most significant bit。

multicast（组播）——一个数据包发送至多个目的终端。广播（broadcast）是发送至所有目的终端的组播。

non-blocking（非阻塞）——对于输入和输出之间任意一一配对的连接请求，都能够同时建立连接的网络。

non-interfering（互不干扰）——与非阻塞网络类似，它也能对各种输入—输出请求模式提供完整带宽，但进一步强调不同业务流在资源分配中不会互相削弱性能。

oblivious routing（无关路由）——数据包可选的路径集合只由源和目的决定，可以用随机方式在这组路径中选择，但路径选择不直接依赖网络当前的拥塞状态。

offered traffic（提供流量）——源终端产生并试图送入网络的总流量（bit/s）。网络低于饱和负载时，提供流量通常全部被接纳；超过饱和点后，已接纳流量可能低于提供流量。

packet（数据包）——互连网络内进行路由的单位。消息一般被分成一个或多个长度可变但有上界的数据包，以供网络处理和转发。

path diversity（路径多样性）——网络中每个源—目的对之间可用的不同路径数量。路径多样性越高，通常越有利于容错和流量负载均衡。

PCI（外围部件互连）——peripheral component interconnect。

permutation（置换流量）——每个输入仅向一个输出发送业务，且每个输出只从一个输入接收业务的流量模式；对应流量矩阵的非零项构成一个置换。

phit（物理传输单元）——physical digit，路由器物理接口一次处理的最小数据单位。一个 flit 可由一个或多个 phit 构成。

quality of service（QoS，服务质量）——特定业务流或业务类别获得的带宽、延迟和／或抖动保证。QoS 策略区别对待不同流量，并为其提供指定服务。

RAM（随机存取存储器）——random-access memory。

reliability（可靠性）——网络在某一时刻正常工作的概率。

routing（路由）——为网络中的数据包选择传输路径的过程。理想路由应在保持较短路径的同时均衡各通道负载。

saturation（饱和）——当施加到资源上的需求超过其服务能力时，资源进入饱和。例如，提供到通道的流量超过通道带宽时，该通道达到饱和。

self-throttling（自限流）——当网络接近饱和时，其提供负载会自然下降的特性。例如，共享内存多处理器中的通信延迟可能使处理器减慢后续请求的产生速率。

serialization（串行化）——数据包或 flit 等较大的数据单元通过狭窄通道传输时，需要在多个连续时隙中分批传送，由此产生串行化延迟。

SIMD（单指令流多数据流）——single-instruction, multiple-data 并行计算机。

SONET（同步光网络）——synchronous optical network。

speedup（加速比／内部超额配置）——为某资源配置超过理想情况下所需的服务能力，以弥补网络其他设计折中或限制。例如，使交换结构内部带宽高于对外线路带宽。

stiff backpressure（强反压）——类似机械系统中弹簧的刚度，下游拥塞如果能迅速传播到上游路由器，就称反压较强。这可避免过多数据包流入拥塞区域。

stable（稳定）——当提供流量继续超过饱和吞吐量时，网络吞吐量仍保持不下降，则称该网络稳定。

STS（同步传输信号）——synchronous transport signal；STS-N 表示第 N 级同步传输信号，速率为 $N \times 51.84$ Mbit/s。

throughput（吞吐量）——网络每秒实际交付目的终端的流量，通常以 bit/s 表示。

topology（拓扑）——网络中路由节点、通道和终端的静态连接组织方式。

traffic（流量）——提供给网络的各个数据包的注入时刻和目的地址序列。通常可通过静态流量模式和随时间变化的注入过程共同建模。

TTL（晶体管—晶体管逻辑）——transistor-transistor logic。

unicast（单播）——数据包只有一个目的终端的通信形式，与组播相对。

upstream（上游）——对某条通道，信用（credit）和其他流控信息沿反向控制链路返回时所朝的方向为上游；通道源端的路由器称为上游路由器。

附录 C　网络模拟器

互连网络设计者应对网络性能形成扎实的直觉，但准确的模拟器仍然是验证这些判断、分析具体设计取舍的重要工具。为帮助完成这类分析，作者编写的本书配套模拟器可从原书提供的网站（http://cva.stanford.edu/）免费获取。本书第 19 章和第 25 章的全部结果均使用这一模拟器生成。模拟器在微片（flit）级建模网络，支持多种网络拓扑和路由算法；路由器的缓冲方式、加速比和流水线时序均可配置，同时还提供多种简单的流量模型与流量模式。在实现上，模拟器采用第 24.4.1 节介绍的双阶段周期驱动模拟方法。

参考文献

Bibliography

[1] Bülent Abali and Cevdet Aykanat. “Routing algorithms for IBM SP1.” In Proc. of the First International Parallel Computer Routing and Communication Workshop (PCRCW), pages 161–175, Seattle, 1994.

[2] Anant Agarwal. “Limits on interconnection network performance.” IEEE Transactions on Parallel and Distributed Systems, 2(4):398–412, 1991.

[3] P. Agrawal, W.J. Dally, W.C. Fischer, H.V. Jagadish, A.S. Krishnakumar, and R. Tutundjain. “MARS: A multiprocessor-based programmable accelerator.” IEEE Design and Test of Computers, 4(5):28–37, Feb. 1987.

[4] Prathima Agrawal and William J. Dally. ”A hardware logic simulation system.” IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 9(1):19–29, Jan. 1990.

[5] Hamid Ahmadi and Wolfgang E. Denzel. ”Survey of modern high performance switching.” IEEE Journal on Selected Areas in Communications, 7(7):1091-1103, Sept. 1989.

[6] Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin. Network flows: Theory, algorithms, and applications. Upper Saddle River, New Jersey Prentice-Hall, Inc. 1993.

[7] Sheldon B. Akers and Balakrishnan Krishnamurthy. “A group-theoritic model for symmetric interconnection networks.” IEEE Transactions on Computers, 38(4):555–566, April 1989.

[8] James D. Allen, Patrick T. Gaughan, David E. Schimmel, and Sudhakar Yalamanchili. ”Ariadne —an adaptive router for fault-tolerant multicomputers.” In Proc. of the International Symposium on Computer Architecture (ISCA), pages 278–288, Chicago, 1994.

[9] Robert Alverson, David Callahan, Daniel Cummings, Brian Koblenz, Allan Porterfield, and Burton Smith. ”The Tera computer system.” In Proc. of the International Conference on Supercomputing, pages 1–6, 1990.

[10] T. Anderson, D. Culler, and D. Patterson. “A case for NOW (networks of workstations)” IEEE Micro, 15(1): 54–64, Feb. 1995.

[11] Thomas E. Anderson, Susan S. Owicki, James B. Saxe, and Charles P. Thacker. ”High speed switch scheduling for local area networks.” ACM Transactions on Computer Systems, 11(4):319–352, Nov. 1993.

[12] P. Baran. “On distributed communication networks.” IEEE Transactions on Communications Systems, 12(1):1–9, March 1964.

[13] George H. Barnes, Richard M. Brown, Maso Kato, David J. Kuck, Daniel L. Slotnick,

and Richard A. Stokes. ”The ILLIAC IV computer.” IEEE Transactions on Computers, 17(8):746–757, Aug. 1968.
[14] K. E. Batcher. ”Sorting networks and their applications.” In AFIPS Conference Proceedings 32, page 307. Montvale, N.J.: AFIPS Press. 1968.
[15] K.E. Batcher. “The flip network in STARAN.” In Proc. of the International Conference on Parallel Processing, pages 65–71, 1976.
[16] Simoni Ben-Michael, Michael Ben-Nun, and Yifat Ben-Shahar. “Method and apparatus to expand an on chip fifo into local memory.” United States Patent 6,078,565. June 2000.
[17] V.E. Beneš. Rearrangeable three stage connecting networks. Bell System Technical Journal, 41:1481–1492, 1962.
[18] V.E. Beneš. Mathematical theory of connecting networks and telephone traffic. New York: Academic Press. 1965.
[19] Elwyn R. Berlekamp. Algebraic Coding Theory, Revised Edition. Berkeley, Aegean Press, June 1984.
[20] D. Bertsekas and R. Gallager. Data Networks. Upper Saddle River, N.J.: Prentice-Hall, Inc. 2nd edition, 1992.
[21] Richard E. Blahut. Algebraic Codes for Data Transmission. Cambridge University Press, 2002.
[22] Matthias A. Blumrich, Richard D. Alpert, Yuqun Chen, Douglas W. Clark, Stefanos N. Damianakis, Cezary Dubnicki, Edward W. Felten, Liviu Iftode, Kai Li, Margaret Martonosi, and Robert A. Shillner. “Design choices in the SHRIMP system: An empirical study.” In Proc. of the International Symposium on Computer Architecture (ISCA), pages 330–341, 1998.

[23] Matthias A. Blumrich, Kai Li, Richard Alpert, Cezary Dubnicki, Edward W. Felten, and Jonathan Sandberg. “Virtual memory mapped network interface for the SHRIMP multicomputer.” In Proc. of the International Symposium on Computer Architecture (ISCA), pages 142–153, April 1994.
[24] Nannette J. Boden, Danny Cohen, Robert E. Felderman, Alan E. Kulawik, Charles E. Seitz, Jakov N. Seizovic, and Wen-King Su. ”Myrinet: a gigabit-per-second local area network.” IEEE Micro, pages 29–36, Feb. 1995.
[25] Kevin Bolding. “Non-uniformities introduced by virtual channel deadlock prevention.” Technical Report UW-CSE-92-07-07, University of Washington, July 1992.
[26] Kevin Bolding, Melanie Fulgham, and Lawrence Snyder. “The case for chaotic adaptive routing.” IEEE Transactions on Computers, 12(46):1281–1292, Dec. 1997.
[27] Rajendra V. Boppana and Suresh Chalasani. ”A comparison of adaptive wormhole routing algorithms.” In Proc. of the International Symposium on Computer Architecture (ISCA), pages 351-360, 1993.

[28] Allan Borodin and John E. Hopcroft. “Routing, merging, and sorting on parallel models of computation.” Journal of Computer and System Sciences, 30:130–145, 1985.

[29] Paul Bratley, Bennett L. Fox, and Linus E. Schrage. A guide to simulation. New York: Springer-Verlag, 2nd edition, 1986.

[30] Mark S. Brirrittella, Richard E. Kessler, Steven M. Oberlin, Randal S. Passint, and Greg Thorson. “System for allocating messages between virtual channels to avoid deadlock and to optimize the amount of message traffic on each type of virtual channel.” United States Patent 5,583,990. Dec. 1996.

[31] Randy Brown. “Calendar queues: A fast $O(1)$ priority queue implementation of the simulation event set problem.” Communications of the ACM, 31(10):1220–1227, Oct. 1988.

[32] Nicholas P. Carter. Processor Mechanisms for Software Shared Memory. Ph.D. thesis, Massachusetts Institute of Technology, Feb. 1999.

[33] Philip P. Carvey, William J. Dally, and Larry R. Dennison. “Apparatus and methods for connecting modules using remote switching.” United States Patent 6,205,532. March 2001.

[34] Philip P. Carvey, William J. Dally, and Larry R. Dennison. “Composite trunking.” United States Patent 6,359,879. March 2002.

[35] Alan Charlesworth. “Starfire —extending the SMP envelope.”IEEE Micro, 18(1):39–49, Jan./Feb. 1998.

[36] Andrew A. Chien and Jae H. Kim. ”Planar-adaptive routing: Low-cost adaptive networks for multiprocessors.” In Proc. of the International Symposium on Computer Architecture (ISCA), pages 268-277, 1992.

[37] Charles Clos. “A study of non-blocking switching networks.” Bell System Technical Journal, 32:406–424, 1953.

[38] P. Close. ”The iPSC/2 node architecture.” In Proc. of the Conference on Hypercube Concurrent Computers and Applications, pages 43-55, Jan. 1988.

[39] R. Cole and J. Hopcroft. “On edge coloring bipartite graphs.” SIAM Journal on Computing, 11:540–546, 1982.

[40] Richard Cole, Kirstin Ost, and Stefan Schirra. “Edge-coloring bipartite multi-graphs in $O(E \log D)$ time.” Combinatorica, 21(1):5–12, 2001.

[41] Richard Crisp. ”Direct Rambus technology: The new main memory standard.” IEEE Micro, 17(6):18-28, Nov./Dec. 1997.

[42] Rene L. Cruz. “A calculus for network delay, part I: Network elements in isolation.” IEEE Transactions on Information Theory, 37(1):114–131, Jan. 1991.

[43] Rene L. Cruz. ”A calculus for network delay, part II: Network analysis.” IEEE Transactions on Information Theory, 37(1):132-141, Jan. 1991.

[44] William J. Dally. ”Virtual-channel flow control.” In Proc. of the International Symposium on Computer Architecture (ISCA), pages 60–68, May 1990.

[45] William J. Dally. ”Express cube: Improving the performance of k-ary n-cube intercon-

nection networks." IEEE Transactions on Computers, 40(9):1016–1023, Sept. 1991.
[46] William J. Dally. "Performance analysis of k-ary n-cube interconnection networks." IEEE Transactions on Computers, 39(6):775–785, June 1991.
[47] William J. Dally. "Virtual-channel flow control." IEEE Transactions on Parallel and Distributed Systems, 3(2):194–205, March 1992.
[48] William J. Dally and Hiromichi Aoki. "Deadlock-free adaptive routing in multicomputer networks using virtual channels." IEEE Transactions on Parallel and Distributed Systems, 4(4):466–475, April 1993.
[49] William J. Dally, P. P. Carvey, and L. R. Dennison. "The Avici terabit switch/router." In Proc. of the Symposium on Hot Interconnects, pages 41–50, Aug. 1998.
[50] William J. Dally, Philip P. Carvey, Larry R. Dennison, and Allen P. King. "Internet switch router." United States Patent 6,370,145. April 2002.
[51] William J. Dally, Andrew Chang, Andrew Chien, Stuart Fiske, Waldemar Horwat, John Keen, Richard Lethin, Michael Noakes, Peter Nuth, Ellen Spertus, Deborah Wallach, and Scott D. Wills. "The J-machine." In Retrospective in 25 Years of the International Symposia on Computer Architecture, pages 54–58, 1998.
[52] William J. Dally, Larry R. Dennison, David Harris, Kinhong Kan, and Thucydides Xanthopoulos. "The Reliable Router: A reliable and high-performance communication substrate for parallel computers." In Proc. of the First International Parallel Computer Routing and Communication Workshop (PCRCW), Seattle, May 1994.
[53] William J. Dally, J. A. Stuart Fiske, John S. Keen, Richard A. Lethin, Michael D. Noakes, Peter R. Nuth, Roy E. Davison, and Gregory A. Fyler. "The message-driven processor —a multicomputer processing node with efficient mechanisms." IEEE Micro, 12(2):23–39, April 1992.
[54] William J. Dally and John Poulton. "Transmitter equalization for 4-Gbps signaling." IEEE Micro, 17(1):48–56, Jan./Feb. 1997.
[55] William J. Dally and John W. Poulton. Digital Systems Engineering. Cambridge University Press, 1998.
[56] William J. Dally and Charles L. Seitz. "The torus routing chip." Journal of Parallel and Distributed Computing, 1(3):187–196, 1986.
[57] William J. Dally and Charles L. Seitz. "Deadlock free message routing in multiprocessor interconnection networks." IEEE Transactions on Computers, 36(5):547–553, May 1987.
[58] Alan Demers, Srinivasan Keshav, and Scott Shenker. "Analysis and simulation of a fair queueing algorithm." Proc. of ACM SIGCOMM, 19(4):1–12, Sept. 1989.
[59] Willibald Doeringer, Günter Karjoth, and Mehdi Nassehi. "Routing on longest-matching prefixes." IEEE/ACM Transactions on Networking, 4(1):86–97, Feb. 1996.
[60] José Duato. "A new theory of deadlock-free adaptive routing in wormhole networks." IEEE Transactions on Parallel and Distributed Systems, 4(12):1320–1331, Dec. 1993.

[61] José Duato. “A necessary and sufficient condition for deadlock-free adaptive routing in wormhole networks.” IEEE Transactions on Parallel and Distributed Systems, 6(10):1055–1067, Oct. 1995.
[62] José Duato. “A necessary and sufficient condition for deadlock-free routing in cut-through and store-and-forward networks.” IEEE Transactions on Parallel and Distributed Systems, 7(6):841–854, Aug. 1996.
[63] Jose Duato, Sudhakar Yalamanchili, Blanca Caminero, Damon S. Love, and Francisco J. Quiles. “MMR: A high-performance multimedia router —architecture and design trade-offs.” In Proc. of the International Symposium on High-Performance Computer Architecture (HPCA), pages 300–309, 1999.
[64] A.M. Duguid. “Structural properties of switching networks.” Technical Report BTL-7, Brown University, 1959.
[65] T-Y. Feng. “Data manipulating functions in parallel processors and their implementations.” IEEE Transactions on Computers, 23(3):309–318, March 1974.
[66] George S. Fishman. Discrete-Event Simulation: Modeling, Programming, and Analysis. New York: Springer-Verlag, 2001.
[67] L.R. Ford and D.R. Fulkerson. “Maximal flow through a network.” Canadian Journal of Mathematics, pages 399–404, 1956.
[68] Edward Fredkin. “Trie memory.” Communications of the ACM, 3(9):490–499, August 1960.
[69] Mike Galles. “Scalable pipelined interconnect for distributed endpoint routing: The SGI SPIDER chip.” In Proc. of the Symposium on Hot Interconnects, pages 141–146, Aug. 1996.
[70] D. Gelernter. ”A DAG-based algorithm for prevention of store-and-forward deadlock in packet networks.” IEEE Transactions on Computers, 30(10):709-715, Oct. 1981.
[71] Paolo Giaccone, Balaji Prabhakar, and Devavrat Shah. ”Towards simple, high-performance schedulers for high-aggregate bandwidth switches.” In Proc. of IEEE INFOCOM, pages 1160-1169, New York, June 2002.
[72] Joseph Di Giacomo. Digital Bus Handbook. McGraw-Hill Professional, Jan. 1990.
[73] Christopher J. Glass and Lionel M. Ni. ”The turn model for adaptive routing.” In Proc. of the International Symposium on Computer Architecture (ISCA), pages 278-287, 1992.
[74] S. Jamaloddin Golestani. “A stop-and-go queueing framework for congestion management.” Proc. of ACM SIGCOMM, 20(4):8–18, Aug. 1990.
[75] Allan Gottlieb, Ralph Grishman, Clyde P. Kruskal, Kevin P. McAuliffe, Larry Rudolph, and Marc Snir. “The NYU Ultracomputer —Designing a MIMD shared memory parallel computer.” IEEE Transactions on Computers, 32(2):175–189, Feb. 1983.
[76] Luis Gravano, Gustavo D. Pifarré, Pablo E. Berman, and Jorge L.C. Sanz. ”Adaptive deadlock- and livelock-free routing with all minimal paths in torus networks.” IEEE Transac-

tions on Parallel and Distributed Systems, 5(12):1233–1251, Dec. 1994.
[77] Jim Gray, editor. The Benchmark Handbook. San Mateo, CA: Morgan Kaufmann, 2nd edition, 1993.
[78] K.D. Gunther. "Prevention of deadlocks in packet-switched data transport systems." IEEE Transactions on Communications, 29(4), 1981.
[79] Pankaj Gupta and Nick McKeown. "Designing and implementing a fast cross-bar scheduler." IEEE Micro, 19(1):20–28, Jan./Feb. 1999.
[80] Kenichi Hayashi, Tunehisa Doi, Takeshi Horie, Yoichi Koyanagi, Osamu Shiraki, Nobutaka Imamura, Toshiyuki Shimizu, Hiroaki Ishihata, and Tatsuya Shindo. "AP1000+: Architectural support of PUT/GET interface for parallelizing compiler." In Proc of. Architectural Support for Programming Languages and Operating Systems (ASPLOS), pages 196–207, San Jose, CA, 1994.
[81] H. Heffes and D.M. Lucantoni. "A Markov modulated characterization of packetized voice and data traffic and related statistical multiplexer performance." IEEE Journal on Selected Areas in Communications, 4(6):856–868, Sept. 1986.
[82] Dana S. Henry and Christopher F. Joerg. "A tightly-coupled processor-network interface." In Proc of. Architectural Support for Programming Languages and Operating Systems (ASPLOS), pages 111–121, 1992.
[83] J. E. Hopcroft and R. M. Karp. "An $n^{5/2}$ algorithm for maximum matching in bipartite graphs." SIAM Journal on Computing, 2:225–231, 1973.
[84] Robert W. Horst. "TNet: a reliable system area network." IEEE Micro, 15(1):37–45, Feb. 1995.
[85] BBN Advanced Computers Incorporated. "Butterfly parallel processor overview." BBN Report No. 6148, March 1986.
[86] INMOS. The T9000 Transputer Products Overview Manual, 1991.
[87] Sundar Iyer, Ramana Rao Kompella, and Nick McKeown. "Analysis of a memory architecture for fast packet buffers." In Proc. of the IEEE Workshop on High Performance Switching and Routing (HPSR), pages 368-373, Dallas, TX, May 2001.
[88] Jeffrey M. Jaffe. "Bottleneck flow control." IEEE Transactions on Communications, 29(7):954–962, July 1981.
[89] A. Jain, W. Anderson, T. Benninghoff, D. Berucci, M. Braganza, J. Burnetie, T. Chang, J. Eble, R. Faber, O. Gowda, J. Grodstein, G. Hess, J. Kowaleski, A. Kumar, B. Miller, R. Mueller, P. Paul, J. Pickholtz, S. Russell, M. Shen, T. Truex, A. Vardharajan, D. Xanthopoulus, and T. Zou. "A 1.2GHz Alpha microprocessor with 44.8Gb/s chip pin bandwidth." In Proc. of the IEEE International Solid-State Circuits Conference (ISSCC), pages 240–241, San Francisco, Feb. 2001.
[90] Raj Jain and Shawn A. Routhier. "Packet trains – measurements and a new model for computer network traffic." IEEE Journal on Selected Areas in Communications, 4(6):986–

995, Sept. 1986.
[91] Christos Kaklamanis, Danny Krizanc, and Thanasis Tsantilas. “Tight bounds for oblivious routing in the hypercube.” In Proc. of the Symposium on Parallel Algorithms and Architectures (SPAA), pages 31–36, 1990.
[92] Manolis Katevenis, Panagiota Vatsolaki, and Aristides Efthymiou. “Pipelined memory shared buffer for VLSI switches.” Proc. of ACM SIGCOMM, 25(4):39–48, Oct. 1995.
[93] Stephen W. Keckler, William J. Dally, Daniel Maskit, Nicholas P. Carter, Andrew Chang, and Whay Sing Lee. “Exploiting fine-grain thread level parallelism on the MIT multi-ALU processor.”In Proc. of the International Symposium on Computer Architecture (ISCA), pages 306–317, Barcelona, Spain, July 1998.
[94] P. Kermani and L. Kleinrock. ”Virtual-cut through: a new computer communications switching technique.” Computer Networks, 3(4):267-286, 1979.
[95] R.E. Kessler and J.L. Schwarzmeier. “Cray T3D: a new dimension for Cray Research.” In Proc. of the IEEE Computer Society International Conference (COMPCON), pages 176–182, Feb. 1993.
[96] Jae H. Kim and Andrew A. Chien. “Network performance under bimodal traffic loads.” Journal of Parallel and Distributed Computing, 28(1):43–64, Jul. 1995.
[97] Jae H. Kim and Andrew A. Chien. “Rotating combined queueing (RCQ): bandwidth and latency guarantees in low-cost, high-performance networks.”In Proc. of the International Symposium on Computer Architecture (ISCA), pages 226–236, May 1996.
[98] Jae H. Kim, Ziqiang Liu, and Andrew A. Chien. “Compressionless routing: A framework for adaptive and fault-tolerant routing.” IEEE Transactions on Parallel and Distributed Systems, 8(3):229–244, March 1997.
[99] Leonard Kleinrock. Queuing Systems, Volume 1. New York: John Wiley & Sons, Inc., 1975.
[100] Leonard Kleinrock and Farouk Kamoun. “Hierarchical routing for large networks: Performance evaluation and optimization.” Computer Networks, 1:154–174, 1977.
[101] Donald E. Knuth. Seminumerical Algorithms. Reading, Mass.: Addison-Wesley, 3rd edition, 1997.
[102] Donald E. Knuth. Sorting and Searching. Reading, Mass.: Addison-Wesley, 2nd edition, 1998.
[103] D.König. “Graphok és alkalmazásuk a determinánsok és a halmazok elméletére [Hungarian].” Mathematikai és Természettudományi Értesito, 34:104–119, 1916.
[104] S. Konstantinidou and L. Snyder. “The Chaos Router: A practical application of randomization in network routing.” Proc. of the Symposium on Parallel Algorithms and Architectures (SPAA), pages 21–30, 1990.
[105] David Kroft. “Lockup-free instruction fetch/prefetch cache organization.” In Proc. of the International Symposium on Computer Architecture (ISCA), pages 81–88, 1981.

[106] Clyde P. Kruskal and Marc Snir. "A unified theory of interconnection network structure." Theoretical Computer Science, 48(3):75–94, 1986.
[107] Vijay P. Kumar, T. V. Lashman, and Dimitrios Stiliadis. "Beyond best effort: Router architectures for the differentiated services of tomorrow's internet." IEEE Communications Magazine, pages 152–164, May 1998.
[108] James Laudon and Daniel Lenoski. "The SGI Origin: a ccNUMA highly scalable server." In Proc. of the International Symposium on Computer Architecture (ISCA), pages 241–251, June 1997.
[109] Averill M. Law and David W. Kelton. Simulation Modeling and Analysis. New York: McGraw Hill, 3rd edition, 2000.
[110] D. H. Lawrie. "Access and alignment of data in an array processor." IEEE Transactions on Computers, 24:1145–1155, Dec. 1975.
[111] Pierre L'Ecuyer. "Random numbers for simulation." Communications of the ACM, 33(10):85-97, Oct. 1990.
[112] Whay Sing Lee, William J. Dally, Stephen W. Keckler, Nicholas P. Carter, and Andrew Chang. "Efficient, protected message interface in the MIT M-Machine." IEEE Computer, 31(11):69–75, Nov. 1998.
[113] Charles E. Leiserson. "Fat-trees: Universal networks for hardware efficient supercomputing." IEEE Transactions on Computers, 34(10):892–901, October 1985.
[114] Charles E. Leiserson, Zahi S. Abuhamdeh, David C. Douglas, Carl R. Feynman, Mahesh N. Ganmukhi, Jeffrey V. Hill, W. Daniel Hillis, Bradley C. Kuszmaul, Margaret A. St Pierre, David S. Wells, Monica C. Wong-Chan, Shaw-Wen Yang, and Robert Zak. "The network architecture of the Connection Machine CM-5." Journal of Parallel and Distributed Computing, 33(2):145–158, 1996.
[115] Daniel Lenoski, James Laudon, Kourosh Gharachorloo, Wolf-Dietrich Weber, Anoop Gupta, John Henessy, Mark Horowitz, and Monica S. Lam. "The Stanford Dash multiprocessor." IEEE Computer, 25(3):63–79, March 1992.
[116] Daniel E. Lenoski and Wolf-Dietrich Weber. Scalable Shared-Memory Multiprocessing. Morgan Kaufmann, 1995.
[117] Sigurd L. Lillevik. "The Touchstone 30 gigaflop DELTA prototype." In DMCC, pages 671–677, April 1991.
[118] Daniel H. Linder and Jim C. Harden. "An adaptive and fault tolerant wormhole routing strategy for k-ary n-cubes." IEEE Transactions on Computers, 40(1):2–12, Jan. 1991.
[119] Alain. J. Martin. "The TORUS: An exercise in constructing a processing surface." In Proc. of the 2 nd Caltech Conference on VLSI, pages 527–537, Jan. 1981.
[120] G. M. Masson and B. W. Jordan Jr. "Generalized multi-stage connection networks." Networks, 2:191–209, 1972.
[121] Makoto Matsumoto and Yoshiharu Kurita. "Twisted GFSR generators." ACM Trans-

actions on Modeling and Computer Simulation, 2(3):179–194, July 1992.
[122] Anthony J. McAuley and Paul Francis. ”Fast routing table lookup using CAMs.” In Proc. of IEEE INFOCOM, pages 1382-1391, San Francisco, March 1993.
[123] Nick McKeown. ”The iSLIP scheduling algorithm for input-queued switches.” IEEE/ACM Transactions on Networking, 7(2):188–201, April 1999.
[124] Nick McKeown, Venkat Anatharam, and Jean Warland. “Achieving 100% throughput in an input-queued switch.” In Proc. of IEEE INFOCOM, pages 296–302, San Francisco, 1996.
[125] Nick McKeown, Martin Izzard, Adisak Mekkittikul, Bill Ellersick, and Mark Horowitz. “The Tiny Tera: A packet switch core.” IEEE Micro, 17(1):26–33, Jan./Feb. 1997.
[126] Nick McKeown, Adisak Mekkittikul, Venkat Anantharam, and Jean Walrand. ”Achieving 100% throughput in an input-queued switch.” In Proc. of IEEE INFOCOM, pages 296–302, San Francisco, March 1996.
[127] J. McQuillan. “Adaptive routing algorithms for distributed computer networks.” Technical Report BBN Tech. Rep. 2831, Cambridge, Mass.: Bolt Beranek and Newman Inc., May 1974.
[128] Dikran S. Meliksetian and C.Y. Roger Chen. “Optimal routing algorithm and the diameter of the cube-connected cycles.” IEEE Transactions on Parallel and Distributed Systems, 4(10):1172–1178, Oct. 1993.
[129] P.M. Merlin and P.J. Schweitzer. “Deadlock avoidance in store-and-forward networks—I: Store and forward deadlock.” IEEE Transactions on Communications, 28(3):345–352, March 1980.
[130] Shubhendu S. Mukherjee, Federico Silla, Peter Bannon, Joel Emer, Steve Lang, and David Webb. “A comparative study of arbitration algorithms for the Alpha 21364 pipelined router.” In Proc of. Architectural Support for Programming Languages and Operating Systems (ASPLOS), pages 223–234, San Jose, CA, Oct. 2002.
[131] Shuhendu S. Mukherjee, Peter Bannno, Steven Lang, Aaron Spink, and David Webb. ”The Alpha 21364 network architecture.” In Proc. of the Symposium on Hot Interconnects, pages 113–117, Aug. 2001.
[132] J. Nagle. “On packet switches with infinite storage.” IEEE Transactions on Communications, 35(4):435–438, April 1987.
[133] T. Nakata, Y. Kanoh, K. Tatsukawa, S. Yanagida, N. Nishi, and H. Takayama. ”Architecture and the software environment of parallel computer Cenju-4.” NEC Research and Development Journal, 39:385–390, Oct. 1998.
[134] nCUBE Corporation. nCUBE Processor Manual, 1990.
[135] Ted Nesson and S. Lennart Johnsson. “ROMM routing on mesh and torus networks.” In Proc. of the Symposium on Parallel Algorithms and Architectures (SPAA), pages 275–287, Santa Barbara, CA, 1995.

[136] Michael D. Noakes, Deborah A. Wallach, and William J. Dally. ”The J-machine multicomputer: An architectural evaluation.” In Proc. of the International Symposium on Computer Architecture (ISCA), pages 224-235, May 1993.
[137] Satoshi Nojima, Eiichi Tsutsui, Haruki Fukuda, and Masamichi Hashimoto. ”Integrated services packet network using bus matrix switch.” IEEE Journal on Selected Areas in Communications, 5(8):1284–1292, Oct. 1987.
[138] Peter R. Nuth and William J. Dally. “The J-machine network.” In Proc. of the International Conference on Computer Design, pages 420–423, Oct. 1992.
[139] Scott Pakin, Vijay Karamcheti, and Andrew A. Chien. ”Fast Messages: Efficient, portable communication for workstation clusters and MPPs.” IEEE Concurrency, 5(2):60–73, April/June 1997.
[140] J.F. Palmer. “The NCUBE family of parallel supercomputers.” In Proc. of the International Conference on Computer Design, 1986.
[141] Stephen K. Park and Keith W. Miller. “Random number generators: good ones are hard to find.” Communications of the ACM, 31(10):1192–1201, Oct. 1988.
[142] Krzysztof Pawlikowski. “Steady-state simulation of queueing processes: A survey of problems and solutions.” ACM Computing Surveys, 22(2):123–170, June 1990.
[143] PCI Special Interests Group, Portland, OR. PCI Local Bus Specification, 2001. Revision 2.3.
[144] M.C. Pease. “The indirect binary n-cube microprocessor array.” IEEE Transactions on Computers, 26(5):458–473, May 1977.
[145] Li-Shiuan Peh and William J. Dally. “Flit-reservation flow control.” In Proc. of the International Symposium on High-Performance Computer Architecture (HPCA), pages 73–84, Toulouse, France, Jan. 1999.
[146] Li-Shiuan Peh and William J. Dally. “A delay model and speculative architecture for pipelined routers.” In Proc. of the International Symposium on High-Performance Computer Architecture (HPCA), pages 255–266, Monterrey, Mexico, Jan. 2001.
[147] Tong-Bi Pei and Charles Zukowski. “VLSI implementation of routing tables: tries and CAMs.” In Proc. of IEEE INFOCOM, pages 512–524, April 1991.
[148] Larry R. Peterson and Bruce S. Davie. Computer Networks: a systems approach. San Francisco: Morgan Kaufmann, 2nd edition, 2000.
[149] Craig S. Petrie and J. Alvin Connelly. “A noise-based IC random number generator for applications in cryptography.” IEEE Transactions on Circuits and Systems I: Fundamental Theory and Applications, 47(5):615–621, May 2000.
[150] Greg Pfister. High Performance Mass Storage and Parallel I/O, ”An Introduction to the InfiniBand Architecture,” Chapter 42, pages 617-632. IEEE Press and Wiley Press, 2001.
[151] Gregory F. Pfister and V. Alan Norton. “Hot spot contention and combining in multistage interconnection networks.” IEEE Transactions on Computers, 34(10):943–948, Oct.

1985.
[152] Timothy Mark Pinkston and Sugath Warnakulasuriya. “Characterization of deadlocks in k-ary n-cube networks.” IEEE Transactions on Parallel and Distributed Systems, 10(9):904–921, Sept. 1999.
[153] Franco P. Preparata and Jean Vuillemin. “The cube-connected cycles: a versatile network for parallel computation.” Communications of the ACM, 24(5):300–309, May 1981.
[154] Martin De Prycker. Asynchronous Transfer Mode: Solution for Broadband ISDN. Prentice Hall, 3rd edition, 1995.
[155] J. Rattner. “Concurrent processing: a new direction in scientific computing.” In AFIPS Conference Proceedings, National Computer Conference, volume 54, pages 157–166, 1985.
[156] Randall D. Rettberg, William R. Crowther, Philip P. Carvey, and Raymond S. Tomlinson. “The monarch parallel processor hardware design.” IEEE Computer, 23(4):18–28, April 1990.
[157] Jerome H. Saltzer, David P. Reed, and David D. Clark. “End-to-end arguments in system design.” ACM Transactions on Computer Systems, 2(4):277–288, Nov. 1984.
[158] Nicola Santoro and Ramez Khatib. “Labeling and implicit routing in networks.” The Computer Journal, 28(1):5–8, 1985.
[159] Michael D. Schroeder, Andrew D. Birrell, Michael Burrows, Hal Murray, Roger M. Needham, Thomas L. Rodeheffer, Edwin H. Satterthwaite, and Charles P. Thacker. ”Autonet: a high-speed, self-configuring local area network using point-to-point links.” IEEE Journal on Selected Areas in Communications, 9(8):1318–1335, Oct. 1991.
[160] Loren Schwiebert. “Deadlock-free oblivious wormhole routing with cyclic dependencies.” IEEE Transactions on Computers, 50(9):865–876, Sept. 2001.
[161] Steven L. Scott and Greg Thorson. “Optimized routing in the Cray T3D”. In Proc. of the First International Parallel Computer Routing and Communication Workshop (PCRCW), pages 281–294, Seattle, May 1994.
[162] Steven L. Scott and Gregory M. Thorson. “The Cray T3E network: Adaptive routing in a high performance 3D torus.” In Proc. of the Symposium on Hot Interconnects, pages 147–156, Aug. 1996.
[163] Charles L. Seitz. ”The Cosmic Cube.” Communications of the ACM, 28(1):22-33, Jan. 1985.
[164] Charles L. Seitz, W. C. Athas, C. M. Flaig, A. J. Martin, J. Seizovic, C. S. Steele, and W.-K. Su. “The architecture and programming of the Ametek series 2010 multicomputer.” In Proc. of the Conference on Hypercube Concurrent Computers and Applications, pages 33–36, 1988.
[165] Carlo H. Séquin. ”Doubly twisted torus networks for VLSI processing arrays.” In Proc. of the International Symposium on Computer Architecture (ISCA), pages 471-480, 1981.
[166] H. J. Siegel. “Interconnection networks for SIMD machines.” IEEE Transactions on

Computers, 12(6):57–65, June 1979.
[167] Daniel Siewiorek and Robert Swarz. Theory and Practice of Reliable System Design. Digital Press, Dec. 1983.
[168] Arjun Singh, William J. Dally, Brian Towles, and Amit K. Gupta. "Locality-preserving randomized oblivious routing on torus networks." In Proc. of the Symposium on Parallel Algorithms and Architectures (SPAA), pages 9–19, Winnipeg, Manitoba, Canada, Aug. 2002.
[169] Jaswinder Pal Singh, Wolf-Dietrich Weber, and Anoop Gupta. "SPLASH: Stanford parallel applications for shared-memory." ACM SIGARCH Computer Architecture News, 1(20):5–44, March 1992.
[170] Rajeev Sivaram, Craig B. Stunkel, and Dhabaleswar K. Panda. "HIPIQS: A high-performance switch architecture using input queuing." IEEE Transactions on Parallel and Distributed Systems, 13(3):275–289, May 1998.
[171] David Slepian. "Two theorems on a particular crossbar switching network." 1952.
[172] Daniel L. Slotnick, W. Carl Borck, and Robert C. McReynolds. "The Soloman computer." In Proc. of the AFIPS Spring Joint Computer Conference, volume 22, pages 97–107, New York: Spartan Books, 1967.
[173] Small computer systems interface—2 (SCSI-2). ANSI X3.131-1994 (R1999), 1999.
[174] Burton J. Smith. "Architecture and applications of the HEP multiprocessor computer system." In Proc. of SPIE: Real-Time Signal Processing IV, volume 298, pages 241–248, 1981.
[175] Harold S. Stone. "Parallel processing with the perfect shuffle." IEEE Transactions on Computers, 20(2):153–161, Feb. 1971.
[176] C. B. Stunkel and P. H. Hochschild. "SP2 high-performance switch architecture." In Proc. of the Symposium on Hot Interconnects, pages 115–121, Stanford, Ca., Aug. 1994.
[177] Craig B. Stunkel, Jay Herring, Bulent Abali, and Rajeev Sivaram. "A new switch chip for IBM RS/6000 SP systems." In Proc. of the ACM/IEEE Conference on Supercomputing, Portland, Or., Nov. 1999.
[178] Craig B. Stunkel, Dennis G. Shea, Don G. Grice, Peter H. Hochschild, and Michael Tsao. "The SP1 high-performance switch." In Proc. of the Scalable High Performance Computing Conference, pages 150–157, May 1994.
[179] H. Sullivan and T. R. Bashkow. "A large scale, homogeneous, fully distributed parallel machine." In Proc. of the International Symposium on Computer Architecture (ISCA), pages 105–124, March 1977.
[180] Yuval Tamir and Hsin-Chou Chi. "Symmetric crossbar arbiters for VLSI communication switches." IEEE Transactions on Parallel and Distributed Systems, 4(1):13–27, Jan. 1993.
[181] Yuval Tamir and Gregory L. Frazier. "High performance multi-queue buffers for VLSI communication switches." In Proc. of the International Symposium on Computer Architec-

ture (ISCA), pages 343–354, June 1988.
[182] Yuval Tamir and Gregory L. Frazier. “Dynamically-allocated multi-queue buffers for VLSI communication switches.” IEEE Transactions on Computers, 41(6):725–737, June 1992.
[183] Leandros Tassiulas. “Linear complexity algorithms for maximum throughput in radio networks and input queued switches.” In Proc. of IEEE INFOCOM, pages 533–539, New York, April 1998.
[184] Fouad A. Tobagi. “Fast packet switch architectures for broadband integrated services digital networks.” Proc. of the IEEE, 78(1):137–167, Jan. 1990.
[185] Brian Towles and William J. Dally. “Worst-case traffic for oblivious routing functions.” In Proc. of the Symposium on Parallel Algorithms and Architectures (SPAA), pages 1–8, Winnipeg, Manitoba, Canada, Aug. 2002.
[186] Anjan K. V. and Timothy Mark Pinkston. “An efficient, fully adaptive deadlock recovery scheme: DISHA.” In Proc. of the International Symposium on Computer Architecture (ISCA), pages 201–210, 1997.
[187] L. G. Valiant and G. J. Brebner. “Universal schemes for parallel communication.” In Proc. of the ACM Symposium of the Theory of Computing, pages 263–277, Milwaukee, Minn., 1981.
[188] J. van Leeuwen and R. B. Tan. “Interval routing.” The Computer Journal, 30(4): 298–307, 1987.
[189] Thorsten von Eicken, David E. Culler, Seth Copen Goldstein, and Klaus Erik Schauser. “Active messages: A mechanism for integrated communication and computation.” In Proc. of the International Symposium on Computer Architecture (ISCA), pages 256–266, Gold Coast, Australia, 1992.
[190] Abraham Waksman. “A permutation network.” Journal of the ACM, 15(1): 159–163, Jan. 1968.
[191] Duncan J. Watts and Steven H. Strogatz. “Collective dynamics of ’ small-world’ networks.” Nature, 393:440–442, June 1998.
[192] C. Whitby-Strevens. ”The transputer.” In Proc. of the International Symposium on Computer Architecture (ISCA), pages 292-300, June 1985.
[193] Drew Wingard. “MicroNetwork-based integration for SOCs.” In Proc. of the Design Automation Conference, pages 673–677, Las Vegas, June 2001.
[194] Steven Cameron Woo, Moriyoshi Ohara, Evan Torrie, Jaswinder Pal Singh, and Anoop Gupta. ”The SPLASH-2 programs: Characterization and methodological considerations.” In Proc. of the International Symposium on Computer Architecture (ISCA), pages 24–36, June 1995.
[195] C.W. Wu and T. Feng. “On a class of multistage interconnection networks.” IEEE Transactions on Computers, 29(8):694–702, August 1980.

[196] Yuanyuan Yang and Gerald M. Masson. “Nonblocking broadcast switching networks.” IEEE Transactions on Computers, 40(9):1005–1015, Sept. 1991.
[197] Kenneth Yeager. The MIPS R10000 superscalar microprocessor. IEEE Micro, 16(2):28–40, April 1996.
[198] Ki Hwan Yum, Eun Jung Kim, Chinta R. Das, and Aniruddha S. Vaidya. “MediaWorm: a QoS capable router architecture for clusters.” IEEE Transactions on Parallel and Distributed Systems, 13(12):1261–1274, Dec. 2002.
[199] Robert C. Zak, Charles E. Leiserson, Bradley C. Kuzmaul, Shaw-Wen Yang, W. Daniel Hillis, David C. Douglas, and David Potter. “Parallel computer system including arrangement for transferring messages from a source processor to selected ones of a plurality of destination processors and combining responses.” United States Patent 5,265,207. Nov. 1993.
[200] L. Zhang. ”VirtualClock: A new traffic control algorithm for packet-switched networks.” ACM Transactions on Computer Systems, 9(2):101-124, May 1991.
[201] Daniel Zwillinger, editor. CRC Standard Mathematical Tables and Formulae. Chemical Rubber Company Press, 30th edition, 1995.
[202] Will E. Leland, Murad S. Taqque, Walter Willinger, and Daniel V. Wilson. ”On the self-similar nature of Ethernet traffic.” IEEE/ACM Transactions on Networking, 2(1): 1-15, Feb. 1994.

A

ABR (available bit rate) service class, 300 Accepted traffic, 21, 38–40, 452–454, 458–459, 515
Accepted vs. offered traffic curve, 452–454
Ack/nack flow control, 249–250
Active state, 237
Adaptive routing, 162, 189, 208, 272–277, 422, 515 basics, 189–192 chaotic, 195, 458 deadlock avoidance, 272–277
Duato’s protocol for deadlock-free adaptive algorithms, 276–277 fully, 193–195 load-balanced, 195–196 minimal, 192–193 routing subfunctions and extended dependences, 272–276 stiff flow control, 191
Thinking Machines CM-5, 196–200
Age-based arbitration, 295–296
Age-based priority, 279
Agents, 257 circuit switching, 258 deadlock, 258–259 flit-buffer flow control, 258–259 packet-buffer flow control, 258, 260–261 wait-for and holds relationships, 259
Aggregate resource allocation, 291–292
Algorithmic routing, 211–212
All-at-once routing, 163
Allocation, 363 batch, 376–378 exact algorithms, 366–367 grant matrix, 364 incremental,

376–378 maximal matching, 365 maximum matching, 364 max-min fairness, 296–297 request matrix, 364
Allocators, 363
greedy, 378
incremental, 376–378
iSLIP, 371–372
lonely output (LOA), 372–373 multistage, 378–380
parallel iterative matching (PIM), 371 performance, 380–383 separable, 367–373 sliding limit, 334 speedup, 382 variable length packets, 378
Verilog example, 35–36
wavefront, 373–376
Alpha 21364 router, 22, 256, 281, 321–324, 385
Alpha-particle strike, 411–412
Ametek S14, 22
Analysis of performance. See Performance
Application-driven workloads, 475–476
Arbiters, 349 distributed, 434 fixed-priority, 352–353, 432 grant-hold circuit, 355–357 iterative, 352–353 matrix, 358–360 oblivious, 354–355 queueing, 360–362 radial, 432–433, 435 replicated, 435–436 round-robin, 355 timing, 349–350 variable length grants, 350 variable priority iterative, 354 weighted round-robin, 357–358
Arbitration, 349
age-based, 295–296
buses, 430–434
fairness, 351–352
FIFO fairness, 351
locally fair, 295, 351
strong fairness, 351
timing, 349–350
weak fairness, 351
Index
ASIC (application-specific integrated circuit), 515 Asynchronous bus, 429–430 ATM (Asynchronous Transfer Mode) service classes, 299–300 switches and Batcher sorting network, 137 virtual circuits, 293 Augmenting path algorithm, 366–367 Availability, 286, 411, 515 Avici TSR (Terabit Switch Router), 157, 183–186, 216, 217, 299, 300–302 routing, 183–186 virtual networks, 183, 300–302
B Backpressure, 245–248, 515 Backtracking, 160 Bandwidth average, 4 bisection, 48 channels, 47 cuts, 48 peak, 4 peak-to-average, 9–10 Batch allocation, 376–378 Batch means method, 481 Batcher bitonic sorting networks, 135–137, 155 Batcher-banyan network, 137

BBN Butterfly, 84–86, 230 BBN Monarch, 230, 468–470 Behavioral simulations, 474 Beneš networks, 21, 134–135, 176–178 BER (bit-error rate), 412 Bernoulli process, 477 Best efforts classes, 286 Best-effort services, 294–297 Binary 2-fly networks, 153 Binary n-cube networks, 22, 65 Bipartite graphs allocators, 365 Clos networks, 122 Bipartite multigraphs, 126 Bisections, 21, 48 bandwidth, 48 butterfly networks, 79–80 channel, 48 channel load bound, 52 mesh networks, 92–93 packaging model, 60–61 torus networks, 92–93 Bit permutations, 50–51 Bit slicing, 149–151 Avici TSR, 183 Tiny Tera, 155–157 Bit-complement traffic pattern, 50–51 Bitonic sorting networks, 135 Bit-reversal traffic patterns, 41, 50–51, 58 Blocked packets, 221 Blocking, 111, 515 BNF (Burton normal form), 459–460 Boards, cabling example, 28–31 packaging 60–62 SGI Origin 2000, 64–69 Body flit, 224 BR (bit-rotation) traffic, 50–51, 57 Broadcast, 425 Buffered flow control, 233 Bufferless flow control, 225–230 Buffers acyclic dependences, 265 allocating to flits or packets, 233–234 classes, 264–267 circular, 328–330 credits, 312–313 data structures, 328–333 decoupling channel allocation, 234 empty stall, 310–312 exclusive class, 267 linked list, 330–333 management, 245–248 non-exclusive class, 267 organizing, 244, 325 partitioning, 326–328 sharing across virtual channels, 326 turnaround time, 251 Burn-in of components, 413 Burst messages, 439–441 Burstiness, 287–290 flows, 287 Markov modulated process, 477–478 performance affects, 501–503 synthetic workloads, 477 Burton normal form (BNF), 459–460 Bus master, 430 Bus switches, 335–338 Buses, 3, 427 acknowledge phase, 431 addressing phase, 430 arbitration, 430, 432–436 asynchronous, 429–431 basics, 428–432 broadcast, 427 burst messages, 439–441 cycles, 429 daisy-chain arbitration, 433–434 electrical constraints, 428, 441 externally sequenced, 429 idle cycle, 431 internally sequenced, 429 interrupting burst messages, 440–441 messages, 427, 429 parallel, 431, 442 performance limitations, 428 pipelining, 436–438 read transaction, 431–432 receive interface, 429 reservation table, 436–438 SECDED (single error correcting, double error detecting) codes, 421 serialization, 427 split-transaction, 438–439 synchronous, 429–431 transactions, 429 transfer phase, 431 transmit interface, 428–429 write transaction 431–432 Butterfly networks, 22, 27–31, 75 2-D view, 81 BBN Butterfly, 84–86 BBN Monarch, 468–470 Beneš network, 84 bisection, 79 channel bisection, 79 channel load, 79 channel width, 79–80 destination-tag routing, 31–32, 165–166, 174 dropping flow control example, 32–33 extra stages, 81–84 hop count, 79 isomorphic, 77–78 latency, 79–80 load imbalance, 81–84 long wires, 75 packaging, 78–80 path diversity, 75, 79, 81–84 performance analysis, 36–42, 463–465, 468–470 reverse traffic pattern, 79 router design example, 33–36 serialization latency, 80 structure, 75–77 switch nodes, 78 throughput, 79 Valiant’s algorithm, 175–176 wire latency, 80 yzantine failures, 413 Cable, 26
Cache coherence, 263, 397–398 message ordering, 208 CAM (content addressable memory), 210, 515
Complement subrelation, 275 CAM-based node routing tables, 210–211
Component failures, 411

C
Channels, 46–47 bandwidth, 47 constraints on width, 60–64 critical length, 62 decoupling with buffers, 222 demand, 53 destination node, 47 error counter, 419 forward, 222 latency, 47 length of wired, 60 load, 17, 51–55 maximum load, 51–55 optical signaling, 63 packaging, 60–64 reconfiguration, 419–420 Capacity, 27, 55, 515 simulation level, 472 Cayley graphs, 63–64, 69
Components, 413, 419, 422 CBR (constant bit rate) service class, 300
Compressionless routing, 278 reverse, 222 saturated, 52 Channel load, 51–55 bounds on, 52–55 butterfly networks, 79 mesh networks, 93 torus networks, 93 shared, 13 shutdown, 419–420
Connected, 48
source node, 47 Channel slicing, 145–146, 152–155
Concentration, 9–10, 145–148, 176 factor, 146 state, 242–244 virtual, 239–244
Concentrators, 145–148
Conservative deadlock detection, 277
Confidence intervals, 482–484
Connection Machine. See Thinking Machines CM-5
Conservative speculation, 317 width, 14–15, 47, 60–64
Connector corrosion open, 411–412 Chaotic routing, 195, 458 Chunk, 345 Circuit boards. See Boards
Consistency checks, 421 Circuit switching, 111, 221–222, 228–230
Containment of errors, 414–415 allocating a circuit, 228 blocking request flit, 228–229
Constraints bisection, 21, 48 cable, 26, 62–63 board, 28–31 packaging, 60–64 signaling, 62–63 resource dependence graph example, 260 resources, 258, 260
Contention latency, 56
Control body flit, 252 serialization latency, 229
Control head flit, 252 Circular buffers, 328–332
Control plane, 305–306 Classes, allocating resources based on, 285–286 Clos networks, 21–22, 116–134, 176
Control state, 222–223
Corner-turn operations, 50–51 channel slicing, 155 crossbar switches, 116
Corrosion open, 411–412 more than three stages, 133–134 multicast routing, 128–132 path diversity, 118 properties, 116–118
Cray T3D, 22, 157, 168–170, 256, 279–280, 324, 347 routingcircuits,118 SGI Origin 2000, 66
Cosmic Cube, 22, 106 unicast routing, 118–126 strictly non-blocking, 118–121 symmetric, 116 Velio VC2002, 137–142 structure, 116–118
Cray T3E, 6, 22, 201, 256, 279–282, 324, 347, 394 Closed-loop measurement systems, 451

CM-5. See Thinking Machines CM-5
CRC (cyclic redundancy check), 150, 415–416 Cold solder joint failure, 411–412
Credit consistency checks, 421
Credit loop, 312–313
Credit-based flow control, 245–247
Credits, 245–247, 312–313, 319–321
Cross connect switch, 137–138
Cross dependences, 275 Crossbar switches, 19, 22, 112–116, 338–342 allocation, 339–340, 380 speedup, 338–342 structure, 112–116 Cube-connected cycles, 69 Cuts, 48 Cut-through flow control, 222, 234, 235–236 Alpha 21364, 322–323 extended dependences, 276 Cycle-based simulation, 485–488
D Daisy-chain arbitration, 431–432 Dance-hall architecture, 5–7, 22 Datapath, 305–306, 325 input buffer organization, 325–334 switches, 334–343 Dateline classes, 265–267, 270–271, 515 Deadlock, 18, 257, 516 agents, 258–259 avoiding, 263–272 circuit switching, 258 conservative detection, 277 dependence graph example, 260–261 dependencies external to network, 262–263 detection, 277 fully adaptive routing algorithm, 195 high-level (protocol), 262–263 holds relation, 259 progressive recovery, 278–279 recovery, 277–279 regressive recovery, 278 resource dependences, 260 resources, 257–259 wait-for relation, 259 Deadlock avoidance, 257 acyclic buffer dependence graph, 264 adaptive routing, 275 Cray T3E, 279–281 Duato’s protocol, 276–278 escape channels, 272–274, 514 hybrid deadlock avoidance, 270–272 resource classes, 263–267 restricted physical routes, 267–270 uphill-only resource allocation rule, 264 Deallocating channels, 222 Decoders, 33 Decoupling channels with buffers, 222 resource dependencies, 18 Deflection routing, 82, 230 Delayed transactions, 446 Delays, calculating deterministic, 288–290 probabilistic analysis, 465–467 queuing theory, 461 Dependences, 260 cross, 275 direct, 275 extended, 272–276 indirect, 273–275 Descriptor-based interfaces, 393 Destination-tag routing, 31–32, 165–166, 174 Deterministic livelock avoidance, 279 Deterministic delay bounds, 288–289 Deterministic routing, 17, 162, 164–168, 203–204, 516 Cray T3D, 168–170 destination-tag routing, 165–166 dimension-order routing, 166–168, 268–270 direction-order routing, 280 Digit permutations, 50–51 Dimension slicing, 145, 151–152 Cray T3D, 168–170 Dimension-order routing, 16–17, 166–168, 268–270 Cray T3D, 168–170 MIT J-Machine, 104 performance, 496–500 relative address, 166–167 shortest or preferred direction, 166–168 valid directions, 168 Valiant’s algorithm, 174–175 Direct dependence, 275 Direct networks, 13, 47, 90 Direction-order routing, 280 DISHA architecture, 278–279 Disk storage, 9–11 Distance classes, 263–266 Distributed arbitration, 434–435 Distributors, 145, 148–149 DOR. See Dimension-order routing Dotted-emitter driver interface, 426 Double speculation, 317 Downstream, 516 Dropping flow control, 32–33, 225–228 BBN Monarch, 468–470 performance, 36–42, 468–470 Duato’s protocol, 276–278
E ECC (error control code), 415, 421 ECL (emitter coupled logic), 516 E-cube routing. See

Dimension-order routing Edge-disjoint paths, 59 Edge-symmetric networks, 50, 53, 90 Electromigration, 411–412 End-to-end error control, 423 End-to-end flow control, 401 Ensemble average, 480 Error control, 411 bit-slicing, 150 containment, 414 detection, 414 end-to-end, 423 failure modes, 411–414 fault models, 411–414 link level, 415–422 network-level, 422 recovery, 414 router, 421–422 Errors masking, 416 physical causes of, 411 Escape channels, 272–274, 514 Event queue, 485–486 Event-driven simulation, 485–488 Exclusive buffer class, 267 Execution-driven workloads, 475 Exponential process, 462 Express cube networks, 100–102 Extended dependences, 272–276 Externally sequenced buses, 429
F Fabric scheduler, 402 Failed components, 422 Fail-stop faults, 412 Failure modes, 411–414 Fairness, 286, 378, 351–352 channel arbitration, 242 FIFO, 352 latency, 294–296 strong, 352 Fat tree networks, 22, 69 Thinking Machines CM-5, 196–200 Fault models, 411–414 Fault tolerance, 411, 456, 516 distributors, 148 simulation examples, 506–507 Fault-containment regions, 421 FEC (forward error-correcting) code, 416 FFT (Fast Fourier transform), 51 Fibre Channel Switched Fabric, 441 FIFO transmit queue, 418 FIFO fairness, 352 First-come-first-served priority, 360 Five-stage Clos network, 133–134, 141–142 Fixed-priority arbiters, 34, 352–353 Fixed-priority arbitration, 433–434 Flit level simulation, 472 Flit stalls, 310–312 Flit-buffer flow control, 233, 237–244 deadlock avoidance, 265–266,dependence graph example, 261–262 indirect dependence, 274 resources, 258–259 virtual-channel flow control, 239–244 wormhole flow control, 237–239 Flit-reservation flow control, 251–255 Flits (flow control digits), 18, 224–225, 516 checking for errors, 320 encoding, 319–321 error detection, 415–416 separating from credits, 319 sizes, 224–225 stalls, 310–312 VCID (virtual-channel identifier) field, 224 Flow control, 17–18, 32–33, 221 516 ack/nack, 249–250 allocation of resources, 222–225 avoiding deadlock, 18 buffered, 233 bufferless, 225–228 circuit switching, 228–230 control state, 222 credit-based, 245–246 cut-through, 234–236 dropping, 227 end-to-end, 401 flit-buffer, 237–244 flit-reservation, 251–255 on/off, 247–249 packet-buffer, 234–236 resources, 222–225 stiff, 191 store-and-forward, 234–235 time-space diagrams, 18 unfair, 452 units, 222–225 virtual-channel, 239–244 wormhole, 222, 237–239 Flow control performance dropping, 36–42, 468–470 injection processes, 501–503 network size, 500–501 prioritization, 503–505 stability, 505–506 virtual channels, 498–500 Flow identifier, 185 Flows, 208, 516 (σ,ρ) regulated, 287–288 average rate of, 287–288 burstiness, 287–290 controlling parameters, 288 fairness between, 286 packets, 208 regulator, 288 Folding, 516 Beneš networks, 176–178 Clos networks, 196–200, 216, Thinking Machines CM-5, 196–200 torus networks, 98 Forward channels, 222 FPGA (field-programmable gate array), 515 FRU (field replaceable unit), 422 Fully adaptive routing, 193–195
G Graceful degradation, 419, 456, 508 Grant matrix, 364 Grant-hold circuit, 355–357 Clos multicast, 132 Grooming switches, 137–138 Guaranteed services, 290–294
H Hard failure, 422 Hardware level simulation, 472–473 Header latency, 40, 55–56, trading with serialization, 153–154 Head flit, 224, 319–320 Head phit, 32, 224, 319–320 Head-of-

line blocking, 380 Hierarchical node-routing, 210 High performance bus protocol, 436–441 High-level (protocol) deadlock, 262–263 High-priority allocation, 378 Holds relation, 259 Hop count, 15–16, 48–49 Hop latency, 20, 56 Hot swapping, 422 Hot-spots, 297–298, 515 Hybrid deadlock avoidance, 270–272 Hypercube networks, 22, 69, 89
|IBM Colony router, 344–347 IBM SP series, 212–217, 256, 324, 344–347 IBM Vulcan network, 212–217 Idle state, 237 Illiac, 22, 106 Incremental allocation, 376–378 Incremental routing, 163–164 Indirect dependence, 273–275 Indirect networks, 47, 75 Valiant's randomized routing algorithm, 175–176 Infiniband, 439 Injection processes, 476–478, 503–505 Injection rate, 476 Input buffers, 305–307, 325–334 allocation, 333–334 data structures, 328–333 dynamic memory allocation, 333–334 organization, 325–334 partitioning, 326–328 sliding limit allocator, 334 Input queues, 402 Input reservation table, 253, 255 Input scheduler, 255 Input unit, 306 Input-first separable allocators, 368–370 Input-partitioned multiple-bus switches, 338 Integrated-node configuration, 5–6 Intel DELTA, 22 Intel iPSC series, 22, 106 Interface level simulation, 474 Interfaces. See Network interfaces Internally addressed messages, 430 Internally sequenced buses, 429 Inter-processor interconnection networks, 5–8, 21–22 I/O interconnection network, 8–11 I/O interfaces, 390–391 Irregular network topology, 46 ISLIP allocators, 371–372 Isomorphic butterfly networks, 77–78 Iterative arbiters, 352–354
J
J-Machine. See MIT J-Machine
K k-ary n-cubes. See Torus networks k-ary n-flies. See Butterfly networks k-ary n-meshes. See Mesh networks
L serialization, 40, 55–56 vs. offered traffic curve, 19–21, 455–456 Least recently served, 358–360 Line cards, 11, 400–401 Avici TSR, 183–185 Line-fabric interfaces, 400–403 Link errors, masking, 416 Link level error control, 415–420 Link level retransmission, 416–418 Link monitoring, 415–416 Linked lists, 330–333 defensive coding, 332–333 error control methods, 332–333 IBM Colony router, 344–347 Livelock, 279, 515 deterministic avoidance, 279 fully adaptive routing, 194–195 node-table routing, 209 probabilistic avoidance, 279 LOA (lonely output allocators), 372–373 Load balance, 51–52, 517 Load imbalance, 266–267 Load-balanced adaptive routing, 195–196 Load-balanced oblivious routing, 180 Locally fair arbitration, 295 Logical dimensions, 99–100 Lonely output allocators (LOA),Lookahead, 318–319 SGI SPIDER, 324 Looping algorithm, 122–125, 140–141 Loss, 285–286, 517 LSB (least significant bit), 517
M MAD (minimal-adaptive routing), 494–497 MAP (Multi-ALU Processor) chip, 403 Markov chain, 462–463 Markov modulated process, 477–478 Masking errors, 416 Matrix arbiters, 358–360 Matrix decomposition, 126–127 Matrix transpose operations, 50–51 Maximal matching, 364–365 Maximum channel load, 51–55 Maximum matching, 364 Max-min fairness, 296–297 MDP chip, 102–105 Measurement, 449–460 common pitfalls 456–

460 Measurement packets, 451–452 Memory request register, 395 Memory server, 262–263 Memory switches, 338 Memoryless process, 462 Memory-network interfaces, 398–400 Mesh networks, 89 bisection, 92–93 channel bisection, 92–93 channel load, 93 channel width, 94 dimension-order routing, 166–168 hop count, 96 latency, 95–96 packaging, 98–100 path diversity, 96–98 MIT J-Machine, 22, 102–106 Message handler, 406 Message-passing interfaces, 390 Messages, 2, 7, 223–224, 517 buses, 429 internally addressed, 430 interface, 390–394 size, 4–5 spatial distribution of, 50–51 MIMD (multiple-instruction-multiple data), 517 Minimal, 163, 517 Minimal quadrant, 178 MIT J-Machine, 22, 102–106, 157, 168, 347, 391, 407 MIT M-Machine, 393–394, 403–408 MIT Reliable Router, 281, 324, 420, 423 Modeling source queues, 488–490 MPP, 22 MSB (most significant bit), 517 MSHR (miss-status holding register), 395, 397 MTBF (mean-time between failures), 412 Multicast, 112, 517 Clos networks, 128–132 crossbar switch, 114 Multicommodity flow problem, 55 Multi-drop bus, 3, 428 Multiple-bus switches, 337–338 Multistage allocation, 378–380 Multistage networks butterfly networks, 75 folding Clos networks, 134–135 slicing, 153–155 Multistage switches, 334

N Nacks (negative acknowledgments), 225–228 ack/nack flow control, 249–250 nCUBE computers, 22, 106 Nearest neighbor traffic, 50–51 Negative-first routing, 269 Network input register, 391 Network interfaces, 389, 441 descriptor-based, 393 I/O, 389 line-fabric, 400–403 memory-network, 398–400 message-passing, 390 processor-network, 390–397 register-mapped, 392–393, 403 safe, 392 shared-memory, 394–395 two-register, 391–392 Network output register, 391 Network switches, 342–343 Network-level error control, 422 Node-arc incidence matrix, 54 Node-based routing algorithms, 163 Node-disjoint paths, 59 Nodes, 46–47 address in dimension, 98 bit slicing, 149–151 channel slicing, 152 combined, 47–48 dimension slicing, 151–152 destination, 47 packaging, 60–64 pin limit, 60 switch, 46–47 terminal, 46–47 Node-table routing, 208–211 Non-blocking networks, 111, 516 Beneš networks, 134–135 Clos networks, 116–134 crossbar networks, 112–116 rearrangeably non-blocking, 111–112 sorting networks, 135–137 strictly non-blocking, 111 vs. non-interfering networks, 112 Non-bursty flows, 287 Non-exclusive buffer class, 267 Non-interfering networks, 12, 112, 299, 401, 518 Avici TSR, 300–302 Non-minimal routing, 174–176, 180, 193–196 livelock, 194–195, 279

0 Oblivious arbiters, 354–355 Oblivious routing, 162, 173, 518 analysis, 180–183 Avici TSR, 183–186 IBM Vulcan network, 212–217 load-balanced, 180 minimal, 176–180 performance, 495–500 ROMM, 178–180, 186, 496–500 source routing, 204 table-based, 422 Valiant’s algorithm, 174–176 worst-case traffic pattern, 180–183 Offered traffic, 19–21, 38–41, 452–454, 518 On-chip networks, 2 On/off flow control, 247–249 Open-drain driver interface, 428–429 Open-loop measurement configuration, 450–451 Optical channel cost, 63 Origin 2000. See SGI Origin 2000 OR-network, 441–442 Output buffer, 306–307, 340–341, 343–344 Output reservation table, 253 Output scheduler, 253–255 Output-first separable allocators, 369–371

Output-partitioned multiple-bus switches, 338 Overlapping resource classes, 266–267
P Packaging, 60–64 bisections, 60–61 boards, 60–62 butterfly networks, 78–80 channels, 60–64 constraints, 60–64 cost, 60–64 crossbar networks, 115–116 examples, 27–31, 63–64, 80, 94–96 length of wired, 60 mesh networks, 98–100 MIT J-Machine, 104 nodes, 60–64 SGI Origin 2000, 64–69 torus networks, 98–100 two-level hierarchy, 61–62 Packet stalls, 310–311 Packet switching fabric, 11–12, 400–403 Packet-buffer flow control, 234–236 agents, 258 cut-through flow control, 234–236 deadlock avoidance, 263–265, 276–277 dependence graph example, 260–261 resources, 258 store-and-forward flow control, 234–235 Packets, 7–8, 223–234, 518 control state allocation, 223 deadlock, 258–259 flow identifier, 185 header, 223–224 latency, 55–56 packet-buffer flow control, 234–236 preserving order of, 208 routing information, 223–224 sequence number, 223–224 serialization latency, 55 sizes, 224–225 split across failed channel, 420–421 zero-load latency, 55–56 Parallel buses, 431, 442 Parallel iterative matching (PIM) allocators, 371 Partial destination-tag routing, 177–178 Partitioning input buffers, 326–328 Path diversity, 57–60, 518 butterfly networks, 81–84 Clos networks, 118 torus networks, 96–98 Paths, 48–49 PCI (Peripheral Component Interconnect) bus, 441, 443–446, 518 PCI-Express, 439 Peak bandwidth, 4 Pending read, 395 Performance, 19–21, 449 accepted vs. offered traffic curve, 452–454 analysis, 460 butterfly networks, 78–80 closed-loop measurement, 451 fault tolerance, 456 latency, 55–57, 455–456 latency vs. offered traffic curve, 19–21, 455–456 maximum channel load, 51–55 measurement, 449–452 measurement pitfalls, 456–460 mesh networks, 92–98 probabilistic analysis, 465–467 queuing theory, 461–465 simulation, 458–459 source queue, 450 steady-state measures, 451 terminal instrumentation, 450 throughput, 51–55, 452–454 torus networks, 92–98 transient measures, 451 Periodic process, 476–477 Permanent failures, 412 Permutation traffic, 50–51, 181–182, 518 Phits (physical digits), 32–33, 224, 518 encoding, 319–321 sizes, 225 Physical dimensions, 99–100 Piggybacking credits, 319–321 PIM (parallel iterative matching) allocators, 371 Pipelined buses, 436–438 Pipelines, 308–310 credit, 312–313 lookahead, 318–319 speculation, 316–318 stalls, 310–312 Planar-adaptive routing, 271–272 Poisson arrival process, 464 Port slicing. See Dimension slicing Ports. See Terminals Premium traffic, 302 Priority age-based, 279 classes, 286 performance, 503–505 Priority inversion, 296 PRNGs (pseudo-random number generators), 490–491 Probabilistic analysis, 465–467 Probabilistic livelock avoidance, 279 Processor-memory interconnect, 5–8 Processor-network interfaces, 390–397 Progressive deadlock recovery, 278–279 Protection events, 138 Protocol consistency checks, 421 Protocol deadlock. See High-level deadlock Protocols cache coherence, 208, 397–398 ordering requirements, 208, 441 requested word first, 398 Power supply failure, 412–413 Pseudo-random number generators (PRNGs), 490–491
Q QoS (quality of service), 4–5, 285, 518 ATM service classes, 299–300 Avici TSR, 300–302 best-effort services, 286, 294–297 burstiness, 287–290 delay, 287–290 fairness, 286, 294–297 flows, 286 guaranteed services, 286, 290–294 separation of resources, 297–299 ser-

vice classes, 285–286 service contracts, 285–286 Queue manager, 401–403 Queuing arbiters, 360–362 Queuing theory, 459–463 Queues. See Buffers
R Radial arbitration, 432–433 Random access memory (RAM), 518 Random arbiters, 355 Random number generation, 490–491 Random separable allocator, 339, 371 Random traffic, 50–51 Randomized routing. See Oblivious routing Read operation, 395 pending, 395 request, 395 complete, 395 Reallocating virtual channels, 313–316 Rearrangeable, 111–112, unicast routing on Clos networks, 122–126 Receive interface, 429 Recovery, 414 Register-mapped network interfaces, 392–393, 403 Regressive recovery, 278 Regulated flows, 287–288 Reliable Router. See MIT Reliable Router Reliability, 5, 411, 519 Repeaters, 62–63 Replicated arbiter, 435–436 Replication method, 482 Request matrix, 364 Reply packets, 8 Request packets, 8 Requesters, 363 Reservation table, 436–438 Residual graph, 366–367 Resources, 222–225 aggregate allocation, 291–292 allocators, 363 arbiters, 349 buffer capacity, 222 channel bandwidth, 222 control state, 222 classes for deadlock avoidance, 263–267 deadlock, 258–259 dependencies, 260 ordering, 263–267 separation of, 297–299 reserving, 292–294 units, 223 Return-to-sender convention, 406 Reverse channels, 222 Reverse traffic pattern, 79 Ring topology, 15–16, 90–91 ROMM, 178–180, 186, 496–500 Rotating arbiters, 355 Round-robin arbiters, 355, 371 Router error control, 421–422 Routem, 33, 36, 395, 375 Routers, 33–36, 305, 325 See also virtual-channel routers allocators, 363 Alpha 21364, 321–324 Avici TSR, 183–186 arbiters, 349 architecture, 305–310 bit-sliced, 150–151 control plane, 306–307 Cray T3D, 168–170 credit loop, 312–316 datapath, 305–306, 325 dimension-sliced, 151–152 errors control, 421–422 flit-reservation, 252–253 IBM Colony, 344–347 IBM Vulcan, 212–217 input buffers, 325–334 output organization, 343–344 pipeline, 308–310 stalls, 310–312 switches, 334–343 Thinking Machines CM-5, 196–200 Tiny Tera, 155–157 Verilog example, 35–36 Routing, 16–17, 31–32, 159, 173, 189, 203, 519 See also Adaptive routing, deterministic routing, and oblivious routing algorithmic, 211–212 all-at-once, 163–164 backtracking, 160 deadlock-free, 263, 272 greedy, 160–162 incremental, 163–164 mechanics, 203 minimal, 162–163 node-table, 208–211 non-minimal, 162–163 relations, 163–164 search-based, 196 performance, 495–500 source, 204–208 subfunctions, 272–276 table-based, 203–211 worst-case traffic, 182–183 Routing tables. See Table-based routing s Safe interfaces, 392 Sampling errors, 476 SANs (system-area networks), 10 Saturation, 21, 40, 52, 450, 519 Search-based routing, 196 SECDED (single error correcting, double error detecting) codes, 421 Sectors, 9 Self-throttling, 8, 12, 519 Separable allocators, 367–373 Alpha 21364, 321–324 input-first, 368–370 iSLIP, 371–372 lonely output allocator (LOA), 372–373 output-first, 369–371 parallel iterative matching (PIM), 371 multiple iterations, 371 Tiny Tera, 383–385 Separation of resources, 297–299 Sequence number, 223–224 Sequential distributed arbiters, 434 SER (soft-error rate), 412 Serialization latency, 15–16, 20, 55–56, 519 butterfly networks, 80 distributors, 149 torus networks, 95 tradeoff with head latency, 153–155 Service classes, 285–286 Service contracts, 285–286 SGI Origin 2000, 64–69, 394, 408 Shadow copy of

router logic, 421 Shared-memory interfaces, 394–400 Shuffle permutation, 50–51, 77 Shuffle-exchange network, 77 SIMD (single-instruction-multiple-data), 519 Simulation examples, 495 allocators, 380–384 fault tolerance, 508–509 flow control performance, 500–508 injection processes, 503–505 latency, 496–499 network size, 502–503 prioritization, 505–507 routing, 495–500 stability, 507–508 throughput distributions, 499–500 virtual channels, 500–502 Simulation measurements, 478–484 batch means method, 481 confidence intervals, 482–484 replication method, 482 simulator warm-up, 479–480 steady-state sampling, 481–482 Simulations, 473 application-driven workloads, 475–476 levels of detail, 473–475 network workloads, 475–478 synthetic workloads, 476–478 Simulator design, 484 cycle-based, 485–488 event-driven, 485–488 modeling source queues, 488–490 random number generation, 490–491 troubleshooting, 491 Single-point fault tolerant networks, 456 Slicing, 145 bit, 149–151 channel, 152 dimension, 151–152 multistage networks, 153–155 port, 151–152 Sliding limit allocators, 334 Software failures, 411–413 Software switching, 47 Solomon machine, 22, 106 SONET (synchronous optical network), 137–140, 183–185, 519 Sorting networks, 135–137 Source queue, 450, 486–487 Source routing, 204–208 Special purpose networks, 45–46 Speculation, 316–318 Speedup, 27–28, 130, 519 allocators, 382–383 buffer partitions, 326–327 crossbar switches, 338–342 speculation, 317 Split-transaction buses, 438–439 Stability, 228, 453–454, 507–508, 519 Starvation, 454 State consistency checks, 421 Stationary processes, 451 Steady state, 451, 481–484 Stiff backpressure, 191, 519 Strong fairness, 352 Store-and-forward flow control, 18, 234–235 Strictly non-blocking, 111 multicast routing on Clos networks, 128–132 unicast routing on Clos networks, 118–121 STS (synchronous transport signal), 137–141, 519 Stuck-at fault model, 412 Student’s t distribution, 483 Switch allocation stall, 310–311 Switch allocator, 306–307 Switch fabric, 11–12 Switch nodes, 47–48 Switch traversal speculation, 317–318 Switch-based crossbar networks, 113 speedup, 334 Symmetric bandwidth, 4 Symmetry, 49–50 Synchronous buses, 429–433 Synthetic workloads, 476–478 Systematic errors, 476 System-level networks, 2

T T3D. See Cray T3D T3E. See Cray T3E Table-based routing, 203–211 Avici TSR, 183–186 CAM-based, 210–211 node-table, 208–211 storing arbitrary-length strings, 206–207 source routing, 204–208 Tail flit, 224 TDM (time-division multiplexing), 137–142, 293–294 Terminal instrumentation, 450 Terminal nodes, 46–49 Thinking Machines CM-5, 69, 196–200, 69, 196–200, 216–217 Three-stage Clos networks, 116–118 Throughput, 21, 36–40, 51–55, 452–454, 520 butterfly networks, 79–80 capacity, 55, 515 distributions, 499–500 fairness, 296–297 ideal, 55 lower bound, 53 torus networks, 92–95 upper bound, 52–53 worst-case, 180–183 Time of flight, 56 Time-division multiplexing, 137–142, 293–294 Time-space diagrams, 18, 221, 233 Topologies, 13–16, 27–31, 45, 520 See also butterfly networks, Clos networks, concentration, distributors, fat tree networks, mesh networks, slicing, and torus networks bisections, 48 Cayley graphs, 63–64, 69 channels, 46–47 cube-connected cycles, 69 direct, 47–48 indirect, 47–48 latency, 55–57 Tornado pattern, 50–51, 161–162, 174 Torus net-

works, 89 bisection, 92–93 channel bisection, 92–93 channel load, 93 channel width, 94 Cray T3D, 168–170 Cray T3E, 279–281 dimension-order routing, 166–168 load-balanced adaptive routing, 195–196 load-balanced oblivious routing, 180 packaging, 98–100 path diversity, 96–98 minimal adaptive routing, 192–193 minimal oblivious routing, 178–180 mixed-radix, 92 serialization latency, 95 structure, 90–92 throughput, 92–95 unidirectional, 90 wire length, 100 Valiant's algorithm, 174–175 Trace-driven workloads, 473–474 Traffic, 50–51, 452–454, 520 accepted, 452 ATM classes, 299–300 classes, 285–286 offered, 452 patterns, 50–51 worst-case, 180–183 Transactions, 429 initiating, 430 overhead, 437–439 PCI bus, 443–446 split, 438–439 timing, 431–432 variable delay, 438 Transient failures, 412 Transpose traffic pattern, 50–51 Tree saturation, 297–299 Tri-state driver interface, 428–429 TSHR (transaction status holding register), 398–400 TSIs (time-slot interchangers), 139–142 TST (time-space-time) TDM switches, 139 TTL (transistor-transistor logic), 520 Turnaround time, 251 Turn-model, 268–270 Two-register interfaces, 391–392

U UBR (unspecified bit rate) service class, 300 Unicast, 112, 520 rearrangeable Clos networks, 122–126 strictly non-blocking Clos networks, 118–121 Uniform random traffic, 50–51 Unstable networks, 453 Uphill-only resource allocation rule, 263–266 Upstream, 520 v Valiant's randomized routing algorithm, 173–176, 496–500 Validation, 467–468 Variable priority iterative arbiters, 354 VBR (variable bit rate) service class, 300 VBR-rt (variable bit rate, real-time) service class, 300 VCID (virtual-channel identifier), 224 Velio VC2002, 137–142 Vertex-symmetric networks, 49–50 Virtual-channel flow control, 222, 237–244 active state, 237 idle state, 237 performance, 500–502 reallocating a channel, 313–316 waiting state, 237 Virtual circuits, 293 Virtual cut-through flow control. See Cut-through flow control Virtual networks, 300–302 Virtual output queues, 380, 400 Virtual-channel allocator, 306–307 Virtual-channel routers, 316–317 credit loop, 312–313 control plane, 306–307 datapath, 305–306 flit rate, 307 flit stalls, 310–312 input unit, 306 packet rate, 307 packet stalls, 310–311 pipeline, 308–310 route computation, 306–309 stalls, 310–312 state fields, 307 switch allocation, 307–309 switch traversal, 309–310 virtual-channel allocation, 307–309

W Wait for relation, 259, 260–262 Waiting state, 237 Warm-up period, 479–480 Wavefront allocators 373–376 Weak fairness, 351 Weighted random routing, 161–162 Weighted round-robin arbiters, 357–358 Wide-area networks, 2 Winner-take-all allocation, 242 Wire mat, 169 Wire-or bus, 432 Wires, 60–64 critical length, 62–63 Work-conserving multiplexer, 288–290 Workloads, 475–478 application-driven, 475–476 execution-driven, 475 synthetic, 476–478 trace-driven, 475–476 Wormhole flow control, 222, 237–239 Worst-case traffic patterns, 180–183 Write operation, 396 z

Zero-load latency, 20–21, 56, 455–456, 494

550 Index

原书索引

本索引采用英文原著的条目与页码（原书第 539–554 页）。由于译文重新分页，索引数字指向**英文原著页码**，并非本中文译本页码。为防止原索引条目遗漏或误配，暂完整保留英文原书的原始索引页，后续中文索引需按本译本分页重新编制。

Index

A
ABR (available bit rate) service class, 300
Accepted traffic, 21, 38–40, **452–454**, 458–459, 515
Accepted vs. offered traffic curve, 452–454
Ack/nack flow control, 249–250
Active state, 237
Adaptive routing, 162, **189**, 208, 272–277, 422, 515
  basics, 189–192
  chaotic, 195, 458
  deadlock avoidance, 272–277
  Duato's protocol for deadlock-free adaptive algorithms, 276–277
  fully, 193–195
  load-balanced, 195–196
  minimal, 192–193
  routing subfunctions and extended dependences, 272–276
  stiff flow control, 191
  Thinking Machines CM-5, 196–200
Age-based arbitration, 295–296
Age-based priority, 279
Agents, 257
  circuit switching, 258
  deadlock, 258–259
  flit-buffer flow control, 258–259
  packet-buffer flow control, 258, 260–261
  wait-for and holds relationships, 259
Aggregate resource allocation, 291–292
Algorithmic routing, 211–212
All-at-once routing, 163
Allocation, 363
  batch, 376–378
  exact algorithms, 366–367
  grant matrix, 364
  incremental, 376–378
  maximal matching, 365
  maximum matching, 364
  max-min fairness, 296–297
  request matrix, 364
Allocators, 363
  greedy, 378
  incremental, 376–378
  iSLIP, 371–372
  lonely output (LOA), 372–373
  multistage, 378–380
  parallel iterative matching (PIM), 371
  performance, 380–383
  separable, 367–373
  sliding limit, 334
  speedup, 382
  variable length packets, 378
  Verilog example, 35–36
  wavefront, 373–376
Alpha 21364 router, 22, 256, 281, **321–324**, 385
Alpha-particle strike, 411–412
Ametek S14, 22
Analysis of performance. *See* Performance
Application-driven workloads, 475–476
Arbiters, 349
  distributed, 434
  fixed-priority, 352–353, 432
  grant-hold circuit, 355–357
  iterative, 352–353
  matrix, 358–360
  oblivious, 354–355
  queueing, 360–362
  radial, 432–433, 435
  replicated, 435–436
  round-robin, 355
  timing, 349–350
  variable length grants, 350
  variable priority iterative, 354
  weighted round-robin, 357–358
Arbitration, 349
  age-based, 295–296
  buses, 430–434
  fairness, 351–352
  FIFO fairness, 351
  locally fair, 295, 351
  strong fairness, 351
  timing, 349–350
  weak fairness, 351

ASIC (application-specific integrated circuit), 515
Asynchronous bus, 429–430
ATM (Asynchronous Transfer Mode)
  service classes, 299–300
  switches and Batcher sorting network, 137
  virtual circuits, 293
Augmenting path algorithm, 366–367
Availability, 286, 411, 515
Avici TSR (Terabit Switch Router), 157, **183–186**, 216, 217, 299, **300–302**
  routing, 183–186
  virtual networks, 183, 300–302

B

Backpressure, 245–248, 515
Backtracking, 160
Bandwidth
  average, 4
  bisection, 48
  channels, 47
  cuts, 48
  peak, 4
  peak-to-average, 9–10
Batch allocation, 376–378
Batch means method, 481
Batcher bitonic sorting networks, 135–137, 155
Batcher-banyan network, 137
BBN Butterfly, **84–86**, 230
BBN Monarch, 230, **468–470**
Behavioral simulations, 474
Beneš networks, 21, 134–135, 176–178
BER (bit-error rate), 412
Bernoulli process, 477
Best efforts classes, 286
Best-effort services, 294–297
Binary 2-fly networks, 153
Binary n-cube networks, 22, 65
Bipartite graphs
  allocators, 365
  Clos networks, 122
Bipartite multigraphs, 126
Bisections, 21, **48**
  bandwidth, 48
  butterfly networks, 79–80
  channel, 48
  channel load bound, 52
  mesh networks, 92–93
  packaging model, 60–61
  torus networks, 92–93
Bit permutations, 50–51
Bit slicing, 149–151
  Avici TSR, 183
Tiny Tera, 155–157
Bit-complement traffic pattern, 50–51
Bitonic sorting networks, 135
Bit-reversal traffic patterns, 41, 50–51, 58
Blocked packets, 221
Blocking, 111, 515
BNF (Burton normal form), 459–460
Boards,
  cabling example, 28–31
  packaging 60–62
  SGI Origin 2000, 64–69
Body flit, 224
BR (bit-rotation) traffic, 50–51, 57
Broadcast, 425
Buffered flow control, 233
Bufferless flow control, 225–230
Buffers
  acyclic dependences, 265
  allocating to flits or packets, 233–234
  classes, 264–267
  circular, 328–330
  credits, 312–313
  data structures, 328–333
  decoupling channel allocation, 234
  empty stall, 310–312
  exclusive class, 267
  linked list, 330–333
  management, 245–248
  non-exclusive class, 267
  organizing, 244, 325
  partitioning, 326–328
  sharing across virtual channels, 326
  turnaround time, 251
Burn-in of components, 413
Burst messages, 439–441
Burstiness, 287–290
  flows, 287
  Markov modulated process, 477–478
  performance affects, 501–503
  synthetic workloads, 477
Burton normal form (BNF), 459–460
Bus master, 430
Bus switches, 335–338
Buses, 3, 427
  acknowledge phase, 431
  addressing phase, 430
  arbitration, 430, 432–436
  asynchronous, 429–431
  basics, 428–432
  broadcast, 427
  burst messages, 439–441
  cycles, 429
  daisy-chain arbitration, 433–434
  electrical constraints, 428, 441
  externally sequenced, 429
  idle cycle, 431
  internally sequenced, 429
  interrupting burst messages, 440–441
  messages, 427, 429
  parallel, 431, 442
  performance limitations, 428
  pipelining, 436–438
  read transaction, 431–432
  receive interface, 429
  reservation table, 436–438
  SECDED (single error correcting, double error detecting) codes, 421
  serialization, 427
  split-transaction, 438–439
  synchronous, 429–431
  transactions, 429
  transfer phase, 431
  transmit interface, 428–429
  write transaction 431–432
Butterfly networks, 22, 27–31, 75
  2-D view, 81
  BBN Butterfly, 84–86
  BBN Monarch, 468–470
  Beneš network, 84
  bisection, 79
  channel bisection, 79
  channel load, 79
  channel width, 79–80
  destination-tag routing, 31–32, **165–166**, 174
  dropping flow control example, 32–33
  extra stages, 81–84
  hop count, 79
  isomorphic, 77–78

latency, 79–80
load imbalance, 81–84
long wires, 75
packaging, 78–80
path diversity, 75, 79, **81–84**
performance analysis, 36–42, 463–465, 468–470
reverse traffic pattern, 79
router design example, 33–36
serialization latency, 80
structure, 75–77
switch nodes, 78
throughput, 79
Valiant's algorithm, 175–176
wire latency, 80
Byzantine failures, 413

C
Cable, 26
Cache coherence, 263, **397–398**
message ordering, 208
CAM (content addressable memory), 210, 515
CAM-based node routing tables, 210–211
Capacity, 27, **55**, 515
simulation level, 472
Cayley graphs, 63–64, 69
CBR (constant bit rate) service class, 300
Channel load, 51–55
bounds on, 52–55
butterfly networks, 79
mesh networks, 93
torus networks, 93
Channel slicing, 145–146, **152–155**
Channels, 46–47
bandwidth, 47
constraints on width, 60–64
critical length, 62
decoupling with buffers, 222
demand, 53
destination node, 47
error counter, 419
forward, 222
latency, 47
length of wired, 60
load, 17, **51–55**
maximum load, 51–55
optical signaling, 63
packaging, 60–64
reconfiguration, 419–420
reverse, 222
saturated, 52
shared, 13
shutdown, 419–420
source node, 47
state, 242–244
virtual, 239–244
width, 14–15, **47**, **60–64**
Chaotic routing, 195, 458
Chunk, 345
Circuit boards. *See* Boards
Circuit switching, 111, 221–222, 228–230
agents, 258
allocating a circuit, 228
blocking request flit, 228–229
contention time, 229
deadlock example, 258
header latency, 229
latency, 229
resource dependence graph example, 260
resources, 258, 260
serialization latency, 229
throughput, 230
zero-load latency, 229
Circular buffers, 328–332
Classes, allocating resources based on, 285–286
Clos networks, 21–22, **116–134**, 176
channel slicing, 155
crossbar switches, 116
folded, 176
more than three stages, 133–134
multicast routing, 128–132
path diversity, 118
properties, 116–118
rearrangeable, 122–126
routing circuits, 118
routing using matrix decomposition, 126–127
SGI Origin 2000, 66
strictly non-blocking, 118–121
structure, 116–118
symmetric, 116
unicast routing, 118–126
Velio VC2002, 137–142
Closed-loop measurement systems, 451
CM-5. *See* Thinking Machines CM-5
Cold solder joint failure, 411–412
Complement subrelation, 275
Component failures, 411
Components, 413, 419, 422
Compressionless routing, 278
Concentration, 9–10, **145–148**, 176
factor, 146
Concentrators, 145–148
Confidence intervals, 482–484
Connected, 48
Connection Machine. *See* Thinking Machines CM-5
Connector corrosion open, 411–412
Conservative deadlock detection, 277
Conservative speculation, 317
Consistency checks, 421
Constraints
bisection, 21, **48**
cable, 26, 62–63
board, 28–31
packaging, 60–64
signaling, 62–63
Containment of errors, 414–415
Contention latency, 56
Control body flit, 252
Control head flit, 252
Control plane, 305–306
Control state, 222–223
Corner-turn operations, 50–51
Corrosion open, 411–412
Cosmic Cube, 22, 106
Cray T3D, 22, 157, **168–170**, 256, 279–280, 324, 347
Cray T3E, 6, 22, 201, 256, **279–282**, 324, 347, 394
CRC (cyclic redundancy check), 150, 415–416
Credit consistency checks, 421
Credit loop, 312–313
Credit-based flow control, 245–247
Credits, 245–247, 312–313, 319–321
Cross connect switch, 137–138
Cross dependences, 275
Crossbar switches, 19, 22, **112–116**, **338–342**
allocation, 339–340, 380
performance, 341–342, 380–383
speedup, 338–342
structure, 112–116

Cube-connected cycles, 69
Cuts, 48
Cut-through flow control, 222, 234, **235–236**
Alpha 21364, 322–323
extended dependences, 276
Cycle-based simulation, 485–488

D
Daisy-chain arbitration, 431–432
Dance-hall architecture, 5–7, 22
Datapath, 305–306, **325**
input buffer organization, 325–334
switches, 334–343
Dateline classes, 265–267, 270–271, 515
Deadlock, 18, **257**, 516
agents, 258–259
avoiding, 263–272
circuit switching, 258
conservative detection, 277
dependence graph example, 260–261
dependences external to network, 262–263
detection, 277
fully adaptive routing algorithm, 195
high-level (protocol), 262–263
holds relation, 259
progressive recovery, 278–279
recovery, 277–279
regressive recovery, 278
resource dependences, 260
resources, 257–259
wait-for relation, 259
Deadlock avoidance, 257
acyclic buffer dependence graph, 264
adaptive routing, 275
Cray T3E, 279–281
Duato's protocol, 276–278
escape channels, 272–274, 514
hybrid deadlock avoidance, 270–272
resource classes, 263–267
restricted physical routes, 267–270
uphill-only resource allocation rule, 264
Deallocating channels, 222
Decoders, 33
Decoupling
channels with buffers, 222
resource dependencies, 18
Deflection routing, 82, 230
Delayed transactions, 446
Delays, calculating
deterministic, 288–290
probabilistic analysis, 465–467
queuing theory, 461
Dependences, 260
cross, 275
direct, 275
extended, 272–276
indirect, 273–275
Descriptor-based interfaces, 393
Destination-tag routing, 31–32, **165–166**, 174
Deterministic livelock avoidance, 279
Deterministic delay bounds, 288–289
Deterministic routing, 17, 162, **164–168**, 203–204, 516
Cray T3D, 168–170
destination-tag routing, 165–166
dimension-order routing, 166–168, 268–270
direction-order routing, 280
Digit permutations, 50–51
Dimension slicing, 145, 151–152
Cray T3D, 168–170
Dimension-order routing, 16–17, **166–168**, 268–270
Cray T3D, 168–170
MIT J-Machine, 104
performance, 496–500
relative address, 166–167
shortest or preferred direction, 166–168
valid directions, 168
Valiant's algorithm, 174–175
Direct dependence, 275
Direct networks, 13, **47**, 90
Direction-order routing, 280
DISHA architecture, 278–279
Disk storage, 9–11
Distance classes, 263–266
Distributed arbitration, 434–435
Distributors, 145, 148–149
DOR. *See* Dimension-order routing
Dotted-emitter driver interface, 426
Double speculation, 317
Downstream, 516
Dropping flow control, 32–33, 225–228
BBN Monarch, 468–470
performance, 36–42, 468–470
Duato's protocol, 276–278

E
ECC (error control code), 415, 421
ECL (emitter coupled logic), 516
E-cube routing. *See* Dimension-order routing
Edge-disjoint paths, 59
Edge-symmetric networks, 50, 53, 90
Electromigration, 411–412
End-to-end error control, 423
End-to-end flow control, 401
Ensemble average, 480
Error control, 411
bit-slicing, 150
containment, 414
detection, 414
end-to-end, 423
failure modes, 411–414
fault models, 411–414
link level, 415–422
network-level, 422
recovery, 414
router, 421–422
Errors
masking, 416
physical causes of, 411
Escape channels, 272–274, 514
Event queue, 485–486
Event-driven simulation, 485–488
Exclusive buffer class, 267
Execution-driven workloads, 475
Exponential process, 462
Express cube networks, 100–102
Extended dependences, 272–276
Externally sequenced buses, 429

F
Fabric scheduler, 402
Failed components, 422
Fail-stop faults, 412
Failure modes, 411–414
Fairness, 286, 378, 351–352

channel arbitration, 242
FIFO, 352
latency, 294–296
strong, 352
throughput, 296–297
weak, 351
Fat tree networks, 22, 69
Thinking Machines CM-5, 196–200
Fault models, 411–414
Fault tolerance, 411, 456, 516
distributors, 148
simulation examples, 506–507
Fault-containment regions, 421
FEC (forward error-correcting) code, 416
FFT (Fast Fourier transform), 51
Fibre Channel Switched Fabric, 441
FIFO transmit queue, 418
FIFO fairness, 352
First-come-first-served priority, 360
Five-stage Clos network, 133–134, 141–142
Fixed-priority arbiters, 34, 352–353
Fixed-priority arbitration, 433–434
Flit level simulation, 472
Flit stalls, 310–312
Flit-buffer flow control, 233, 237–244
agents, 258–259
deadlock avoidance, 265–266,
dependence graph example, 261–262
indirect dependence, 274
resources, 258–259
virtual-channel flow control, 239–244
wormhole flow control, 237–239
Flit-reservation flow control, 251–255
Flits (flow control digits), 18, **224–225**, 516
checking for errors, 320
encoding, 319–321
error detection, 415–416
separating from credits, 319
sizes, 224–225
stalls, 310–312
VCID (virtual-channel identifier) field, 224
Flow control, 17–18, 32–33, **221**, 516
ack/nack, 249–250
allocation of resources, 222–225
avoiding deadlock, 18
buffered, 233
bufferless, 225–228
circuit switching, 228–230
control state, 222
credit-based, 245–246
cut-through, 234–236
dropping, 227
end-to-end, 401
flit-buffer, 237–244
flit-reservation, 251–255
on/off, 247–249
packet-buffer, 234–236
resources, 222–225
stiff, 191
store-and-forward, 234–235
time-space diagrams, 18
unfair, 452
units, 222–225
virtual-channel, 239–244
wormhole, 222, 237–239
Flow control performance
dropping, 36–42, 468–470
injection processes, 501–503
network size, 500–501
prioritization, 503–505
stability, 505–506
virtual channels, 498–500
Flow identifier, 185
Flows, 208, 516
(σ,ρ) regulated, 287–288
average rate of, 287–288
burstiness, 287–290
controlling parameters, 288
fairness between, 286
packets, 208
regulator, 288
Folding, 516
Beneš networks, 176–178
Clos networks, 196–200, 216,
Thinking Machines CM-5, 196–200
torus networks, 98
Forward channels, 222
FPGA (field-programmable gate array), 515
FRU (field replaceable unit), 422
Fully adaptive routing, 193–195

G

Graceful degradation, 419, 456, 508
Grant matrix, 364
Grant-hold circuit, 355–357
Greedy routing
Clos multicast, 132
ring, 160–162
Velio VC2002, 140
Grooming switches, 137–138
Guaranteed services, 290–294

H

Hard failure, 422
Hardware level simulation, 472–473
Header latency, 40, **5**5–56,
trading with serialization, 153–154
Head flit, 224, 319–320
Head phit, 32, 224, 319–320
Head-of-line blocking, 380
Hierarchical node-routing, 210
High performance bus protocol, 436–441
High-level (protocol) deadlock, 262–263
High-priority allocation, 378
Holds relation, 259
Hop count, 15–16, **48–49**
Hop latency, 20, 56
Hot swapping, 422
Hot-spots, 297–298, 515
Hybrid deadlock avoidance, 270–272
Hypercube networks, 22, 69, **89**

I

IBM Colony router, 344–347
IBM SP series, 212–217, 256, 324, 344–347
IBM Vulcan network, 212–217
Idle state, 237
Illiac, 22, 106
Incremental allocation, 376–378
Incremental routing, 163–164

Indirect dependence, 273–275
Indirect networks, **47**, 75
  Valiant's randomized routing algorithm, 175–176
Infiniband, 439
Injection processes, 476–478, 503–505
Injection rate, 476
Input buffers, 305–307, 325–334
  allocation, 333–334
  data structures, 328–333
  dynamic memory allocation, 333–334
  organization, 325–334
  partitioning, 326–328
  sliding limit allocator, 334
Input queues, 402
Input reservation table, 253, 255
Input scheduler, 255
Input unit, 306
Input-first separable allocators, 368–370
Input-partitioned multiple-bus switches, 338
Integrated-node configuration, 5–6
Intel DELTA, 22
Intel iPSC series, 22, 106
Interface level simulation, 474
Interfaces. *See* Network interfaces
Internally addressed messages, 430
Internally sequenced buses, 429
Inter-processor interconnection networks, 5–8, 21–22
I/O interconnection network, 8–11
I/O interfaces, 390–391
Irregular network topology, 46
ISLIP allocators, 371–372
Isomorphic butterfly networks, 77–78
Iterative arbiters, 352–354

J

Jitter, 41, 285–287, 517
  VBR-rt service class, 300
J-Machine. *See* MIT J-Machine

K

k-ary *n*-cubes. *See* Torus networks
k-ary *n*-flies. *See* Butterfly networks
k-ary *n*-meshes. *See* Mesh networks

L

Latency, **55–57**, 517
  analysis, 36–42,
  average, 56
  bounds, 19–21
  butterfly networks, 79–80
  distribution, 41, 505–507
  fairness, 294–296
  header, 55
  insensitivity to, 10–11
  measuring, 450–452, 455–460
  queuing, 40
  serialization, 40, 55–56
  simulation examples, 496–507
  time of flight, 56
  time-evolution of, 478
  torus networks, 95–96
  vs. offered traffic curve, 19–21, **455–456**
  zero-load, 20, **56**
Least recently served, 358–360
Line cards, 11, **400–401**
  Avici TSR, 183–185
Line-fabric interfaces, 400–403
Link errors, masking, 416
Link level error control, 415–420
Link level retransmission, 416–418
Link monitoring, 415–416
Linked lists, 330–333
  defensive coding, 332–333
  error control methods, 332–333
  IBM Colony router, 344–347
Livelock, 279, 515
  deterministic avoidance, 279
  fully adaptive routing, 194–195
  node-table routing, 209
  probabilistic avoidance, 279
LOA (lonely output allocators), 372–373
Load balance, 51–52, 517
Load imbalance, 266–267
Load-balanced adaptive routing, 195–196
Load-balanced oblivious routing, 180
Locality, 5, 51
  tradeoff with load balance 173
Locally fair arbitration, 295
Logical dimensions, 99–100
Lonely output allocators (LOA),
Lookahead, 318–319
  SGI SPIDER, 324
Looping algorithm, 122–125, 140–141
Loss, 285–286, 517
LSB (least significant bit), 517

M

MAD (minimal-adaptive routing), 494–497
MAP (Multi-ALU Processor) chip, 403
Markov chain, 462–463
Markov modulated process, 477–478
Masking errors, 416
Matrix arbiters, 358–360
Matrix decomposition, 126–127
Matrix transpose operations, 50–51
Maximal matching, 364–365
Maximum channel load, 51–55
Maximum matching, 364
Max-min fairness, 296–297
MDP chip, 102–105
Measurement, 449–460
  common pitfalls 456–460
Measurement packets, 451–452
Memory request register, 395
Memory server, 262–263
Memory switches, 338
Memoryless process, 462
Memory-network interfaces, 398–400
Mesh networks, 89
  bisection, 92–93
  channel bisection, 92–93
  channel load, 93
  channel width, 94
  dimension-order routing, 166–168
  hop count, 96
  latency, 95–96
  packaging, 98–100
  path diversity, 96–98
  MIT J-Machine, 22, 102–106

mixed-radix, 92
serialization latency, 95
structure, 90–92
throughput, 92–95
unidirectional, 90
wire length, 100
Message handler, 406
Message-passing interfaces, 390
Messages, 2, 7, **2**23–224, 517
buses, 429
internally addressed, 430
interface, 390–394
size, 4–5
spatial distribution of, 50–51
MIMD (multiple-instruction-multiple data), 517
Minimal, 163, 517
Minimal adaptive routing, 192–193
deadlock-free, 276–277
performance, 495–500
Thinking Machines CM-5, 196–200
Minimal oblivious routing, 176–180
Minimal quadrant, 178
Minimum bisection, 48
butterfly networks,
mesh networks, 92–93
packaging, 60–64
torus networks, 92–93
Misrouting packets, **1**93–195, **2**25–228, 230
livelock, 193–195, 279
stability, 454
MIT J-Machine, 22, **1**02–106, 157, 168, 347, 391, 407
MIT M-Machine, 393–394, **4**03–408
MIT Reliable Router, 281, 324, 420, 423
Modeling source queues, 488–490
MPP, 22
MSB (most significant bit), 517
MSHR (miss-status holding register), 395, 397
MTBF (mean-time between failures), 412
Multicast, 112, 517
Clos networks, 128–132
crossbar switch, 114
Multicommodity flow problem, 55
Multi-drop bus, 3, 428
Multiple-bus switches, 337–338
Multistage allocation, 378–380
Multistage networks
butterfly networks, 75
folding Clos networks, 134–135
slicing, 153–155
Multistage switches, 334

N

Nacks (negative acknowledgments), 225–228
ack/nack flow control, 249–250
nCUBE computers, 22, 106
Nearest neighbor traffic, 50–51
Negative-first routing, 269
Network input register, 391
Network interfaces, **3**89, 441
descriptor-based, 393
I/O, 389
line-fabric, 400–403
memory-network, 398–400
message-passing, 390
processor-network, 390–397
register-mapped, 392–393, 403
safe, 392
shared-memory, 394–395
two-register, 391–392
Network output register, 391
Network switches, 342–343
Network-level error control, 422
Node-arc incidence matrix, 54
Node-based routing algorithms, 163
Node-disjoint paths, 59
Nodes, 46–47
address in dimension, 98
bit slicing, 149–151
channel slicing, 152
combined, 47–48
dimension slicing, 151–152
destination, 47
packaging, 60–64
pin limit, 60
switch, 46–47
terminal, 46–47
Node-table routing, 208–211
Non-blocking networks, **1**11, 516
Beneš networks, 134–135
Clos networks, 116–134
crossbar networks, 112–116
rearrangeably non-blocking, 111–112
sorting networks, 135–137
strictly non-blocking, 111
vs. non-interfering networks, 112
Non-bursty flows, 287
Non-exclusive buffer class, 267
Non-interfering networks, 12, **1**12, **2**99, 401, 518
Avici TSR, 300–302
Non-minimal routing, 174–176, 180, 193–196
livelock, 194–195, 279

O

Oblivious arbiters, 354–355
Oblivious routing, 162, **1**73, 518
analysis, 180–183
Avici TSR, 183–186
IBM Vulcan network, 212–217
load-balanced, 180
minimal, 176–180
performance, 495–500
ROMM, 178–180, 186, 496–500
source routing, 204
table-based, 422
Valiant's algorithm, 174–176
worst-case traffic pattern, 180–183
Offered traffic, 19–21, 38–41, **4**52–454, 518
On-chip networks, 2
On/off flow control, 247–249
Open-drain driver interface, 428–429
Open-loop measurement configuration, 450–451
Optical channel cost, 63
Origin 2000. *See* SGI Origin 2000
OR-network, 441–442
Output buffer, 306–307, 340–341, 343–344
Output reservation table, 253
Output scheduler, 253–255
Output-first separable allocators, 369–371

Output-partitioned multiple-bus switches, 338
Overlapping resource classes, 266–267

P
Packaging, 60–64
  bisections, 60–61
  boards, 60–62
  butterfly networks, 78–80
  channels, 60–64
  constraints, 60–64
  cost, 60–64
  crossbar networks, 115–116
  examples, 27–31, 63–64, 80, 94–96
  length of wired, 60
  mesh networks, 98–100
  MIT J-Machine, 104
  nodes, 60–64
  SGI Origin 2000, 64–69
  torus networks, 98–100
  two-level hierarchy, 61–62
Packet stalls, 310–311
Packet switching fabric, 11–12, 400–403
Packet-buffer flow control, 234–236
  agents, 258
  cut-through flow control, 234–236
  deadlock avoidance, 263–265, 276–277
  dependence graph example, 260–261
  resources, 258
  store-and-forward flow control, 234–235
Packets, 7–8, **2**23–234, 518
  control state allocation, 223
  deadlock, 258–259
  flow identifier, 185
  header, 223–224
  latency, 55–56
  packet-buffer flow control, 234–236
  preserving order of, 208
  routing information, 223–224
  sequence number, 223–224
  serialization latency, 55
  sizes, 224–225
  split across failed channel, 420–421
  zero-load latency, 55–56
Parallel buses, 431, 442
Parallel iterative matching (PIM) allocators, 371
Partial destination-tag routing, 177–178
Partitioning input buffers, 326–328
Path diversity, 57–60, 518
  butterfly networks, 81–84
  Clos networks, 118
  torus networks, 96–98
Paths, 48–49
PCI (Peripheral Component Interconnect) bus, 441, 443–446, 518
PCI-Express, 439
Peak bandwidth, 4
Pending read, 395
Performance, 19–21, **449**
  accepted vs. offered traffic curve, 452–454
  analysis, 460
  butterfly networks, 78–80
  closed-loop measurement, 451
  fault tolerance, 456
  latency, 55–57, 455–456
  latency vs. offered traffic curve, 19–21, 455–456
  maximum channel load, 51–55
  measurement, 449–452
  measurement pitfalls, 456–460
  mesh networks, 92–98
  probabilistic analysis, 465–467
  queuing theory, 461–465
  simulation, 458–459
  source queue, 450
  steady-state measures, 451
  terminal instrumentation, 450
  throughput, 51–55, 452–454
  torus networks, 92–98
  transient measures, 451
Periodic process, 476–477
Permanent failures, 412
Permutation traffic, 50–51, 181–182, 518
Phits (physical digits), 32–33, **224**, 518
  encoding, 319–321
  sizes, 225
Physical dimensions, 99–100
Piggybacking credits, 319–321
PIM (parallel iterative matching) allocators, 371
Pipelined buses, 436–438
Pipelines, 308–310
  credit, 312–313
  lookahead, 318–319
  speculation, 316–318
  stalls, 310–312
Planar-adaptive routing, 271–272
Poisson arrival process, 464
Port slicing. *See* Dimension slicing
Ports. *See* Terminals
Premium traffic, 302
Priority
  age-based, 279
  classes, 286
  performance, 503–505
Priority inversion, 296
PRNGs (pseudo-random number generators), 490–491
Probabilistic analysis, 465–467
Probabilistic livelock avoidance, 279
Processor-memory interconnect, 5–8
Processor-network interfaces, 390–397
Progressive deadlock recovery, 278–279
Protection events, 138
Protocol consistency checks, 421
Protocol deadlock. *See* High-level deadlock
Protocols
  cache coherence, 208, **397–398**
  ordering requirements, 208, 441
  requested word first, 398
Power supply failure, 412–413
Pseudo-random number generators (PRNGs), 490–491

Q
QoS (quality of service), 4–5, **285**, 518
  ATM service classes, 299–300
  Avici TSR, 300–302
  best-effort services, 286, 294–297
  burstiness, 287–290
  delay, 287–290

fairness, 286, 294–297
flows, 286
guaranteed services, 286, 290–294
separation of resources, 297–299
service classes, 285–286
service contracts, 285–286
Queue manager, 401–403
Queuing arbiters, 360–362
Queuing theory, 459–463
Queues. *See* Buffers

R
Radial arbitration, 432–433
Random access memory (RAM), 518
Random arbiters, 355
Random number generation, 490–491
Random separable allocator, 339, 371
Random traffic, 50–51
Randomized routing. *See* Oblivious routing
Read operation, 395
pending, 395
request, 395
complete, 395
Reallocating virtual channels, 313–316
Rearrangeable, 111–112,
unicast routing on Clos networks, 122–126
Receive interface, 429
Recovery, 414
Register-mapped network interfaces, 392–393, 403
Regressive recovery, 278
Regulated flows, 287–288
Reliable Router. *See* MIT Reliable Router
Reliability, 5, 411, 519
Repeaters, 62–63
Replicated arbiter, 435–436
Replication method, 482
Request matrix, 364
Reply packets, 8
Request packets, 8
Requesters, 363
Requests, 397
Reservation table, 436–438
Residual graph, 366–367
Resources, 222–225
aggregate allocation, 291–292
allocators, 363
arbiters, 349
buffer capacity, 222
channel bandwidth, 222
control state, 222
classes for deadlock avoidance, 263–267
deadlock, 258–259
dependences, 260
ordering, 263–267
separation of, 297–299
reserving, 292–294
units, 223
Return-to-sender convention, 406
Reverse channels, 222
Reverse traffic pattern, 79
Ring topology, 15–16, 90–91
ROMM, 178–180, 186, 496–500
Rotating arbiters, 355
Round-robin arbiters, 355, 371
Router error control, 421–422
Routers, 33–36, **305**, **325**
See also virtual-channel routers
allocators, 363
Alpha 21364, 321–324
Avici TSR, 183–186
arbiters, 349
architecture, 305–310
bit-sliced, 150–151
control plane, 306–307
Cray T3D, 168–170
credit loop, 312–316
datapath, 305–306, 325
dimension-sliced, 151–152
errors control, 421–422
flit-reservation, 252–253
IBM Colony, 344–347
IBM Vulcan, 212–217
input buffers, 325–334
output organization, 343–344
pipeline, 308–310
stalls, 310–312
switches, 334–343
Thinking Machines CM-5, 196–200
Tiny Tera, 155–157
Verilog example, 35–36
Routing, 16–17, 31–32, **1**59, **1**73, **1**89, **2**03, 519 *See also* Adaptive routing, deterministic routing, and oblivious routing
algorithmic, 211–212
all-at-once, 163–164
backtracking, 160
deadlock-free, 263, 272
greedy, 160–162
incremental, 163–164
mechanics, 203
minimal, 162–163
node-table, 208–211
non-minimal, 162–163
relations, 163–164
search-based, 196
performance, 495–500
source, 204–208
subfunctions, 272–276
table-based, 203–211
worst-case traffic, 182–183
Routing tables. *See* Table-based routing

S
Safe interfaces, 392
Sampling errors, 476
SANs (system-area networks), 10
Saturation, 21, 40, 52, 450, 519
Search-based routing, 196
SECDED (single error correcting, double error detecting) codes, 421
Sectors, 9
Self-throttling, 8, 12, 519
Separable allocators, 367–373
Alpha 21364, 321–324
input-first, 368–370
iSLIP, 371–372
lonely output allocator (LOA), 372–373
output-first, 369–371
parallel iterative matching (PIM), 371
multiple iterations, 371
Tiny Tera, 383–385
Separation of resources, 297–299
Sequence number, 223–224
Sequential distributed arbiters, 434
SER (soft-error rate), 412
Serialization latency, 15–16, 20, 55–56, 519
butterfly networks, 80
distributors, 149
torus networks, 95

tradeoff with head latency, 153–155
Service classes, 285–286
Service contracts, 285–286
SGI Origin 2000, **6**4–69, 394, 408
Shadow copy of router logic, 421
Shared-memory interfaces, 394–400
Shuffle permutation, 50–51, 77
Shuffle-exchange network, 77
SIMD (single-instruction-multiple-data), 519
Simulation examples, 495
allocators, 380–384
fault tolerance, 508–509
flow control performance, 500–508
injection processes, 503–505
latency, 496–499
network size, 502–503
prioritization, 505–507
routing, 495–500
stability, 507–508
throughput distributions, 499–500
virtual channels, 500–502
Simulation measurements, 478–484
batch means method, 481
confidence intervals, 482–484
replication method, 482
simulator warm-up, 479–480
steady-state sampling, 481–482
Simulations, 473
application-driven workloads, 475–476
levels of detail, 473–475
network workloads, 475–478
synthetic workloads, 476–478
Simulator design, 484
cycle-based, 485–488
event-driven, 485–488
modeling source queues, 488–490
random number generation, 490–491
troubleshooting, 491
Single-point fault tolerant networks, 456
Slicing, 145
bit, 149–151
channel, 152
dimension, 151–152
multistage networks, 153–155
port, 151–152
Sliding limit allocators, 334
Software failures, 411–413
Software switching, 47
Solomon machine, 22, 106
SONET (synchronous optical network), 137–140, 183–185, 519
Sorting networks, 135–137
Source queue, 450, 486–487
Source routing, 204–208
IBM Vulcan, 214–215
Special purpose networks, 45–46
Speculation, 316–318
Speedup, 27–28, 130, 519
allocators, 382–383
buffer partitions, 326–327
crossbar switches, 338–342
speculation, 317
Split-transaction buses, 438–439
Stability, 228, **453–454**, 507–508, 519
Starvation, 454
State consistency checks, 421
Stationary processes, 451
Steady state, 451, 481–484
Stiff backpressure, 191, 519
Strong fairness, 352
Store-and-forward flow control, 18, 234–235
Strictly non-blocking, 111
multicast routing on Clos networks, 128–132
unicast routing on Clos networks, 118–121
STS (synchronous transport signal), 137–141, 519
Stuck-at fault model, 412
Student's t distribution, 483
Switch allocation stall, 310–311
Switch allocator, 306–307
Switch fabric, 11–12
Switch nodes, 47–48
Switch traversal speculation, 317–318
Switch-based crossbar networks, 113
Switches, 334–343
bus, 335–338
crossbar, 338–342
memory, 338
multiple-bus, 337
network, 342–343
speedup, 334
Symmetric bandwidth, 4
Symmetry, 49–50
Synchronous buses, 429–433
Synthetic workloads, 476–478
Systematic errors, 476
System-level networks, 2

T

T3D. *See* Cray T3D
T3E. *See* Cray T3E
Table-based routing, 203–211
Avici TSR, 183–186
CAM-based, 210–211
node-table, 208–211
storing arbitrary-length strings, 206–207
source routing, 204–208
Tail flit, 224
TDM (time-division multiplexing), 137–142, 293–294
Terminal instrumentation, 450
Terminal nodes, 46–49
Thinking Machines CM-5, 69, **1**96–200, 69, 196–200, 216–217
Three-stage Clos networks, 116–118
Throughput, 21, 36–40, **5**1–55, **4**52–454, 520
butterfly networks, 79–80
capacity, 55, 515
distributions, 499–500
fairness, 296–297
ideal, 55
lower bound, 53
torus networks, 92–95
upper bound, 52–53
worst-case, 180–183
Time of flight, 56
Time-division multiplexing, 137–142, 293–294
Time-space diagrams, 18, 221, 233
Tiny Tera
allocator, 383–385
bit slicing, 155–157
Topologies, 13–16, 27–31, **4**5, 520
See also butterfly networks, Clos networks, concentration, distributors, fat tree networks, mesh

networks, slicing, and torus networks
bisections, 48
Cayley graphs, 63–64, 69
channels, 46–47
cost, 60–64
cube-connected cycles, 69
cuts, 48
direct, 47–48
indirect, 47–48
latency, 55–57
maximum channel load, 51–55
nodes, 46–47
packaging, 60–64
path diversity, 57–60
paths, 48–49
performance, 51–60
SGI Origin 2000, 64–69
symmetry, 49–50
throughput, 51–55
traffic patterns, 50–51
Tornado pattern, **50–51**, 161–162, 174
Torus networks, 89
bisection, 92–93
channel bisection, 92–93
channel load, 93
channel width, 94
Cray T3D, 168–170
Cray T3E, 279–281
dimension-order routing, 166–168
direction-order routing, 280
fully adaptive routing, 193–195
hop count, 96
latency, 95–96
load-balanced adaptive routing, 195–196
load-balanced oblivious routing, 180
packaging, 98–100
path diversity, 96–98
minimal adaptive routing, 192–193
minimal oblivious routing, 178–180
mixed-radix, 92
serialization latency, 95
structure, 90–92
throughput, 92–95
unidirectional, 90
wire length, 100
Valiant's algorithm, 174–175
Trace-driven workloads, 473–474
Traffic, 50–51, 452–454, 520
accepted, 452
ATM classes, 299–300
classes, 285–286
offered, 452
patterns, 50–51
worst-case, 180–183
Transactions, 429
initiating, 430
overhead, 437–439
PCI bus, 443–446
split, 438–439
timing, 431–432
variable delay, 438
Transient failures, 412
Transpose traffic pattern, 50–51
Tree saturation, 297–299
Tri-state driver interface, 428–429
TSHR (transaction status holding register), 398–400
TSIs (time-slot interchangers), 139–142
TST (time-space-time) TDM switches, 139
TTL (transistor-transistor logic), 520
Turnaround time, 251
Turn-model, 268–270
Two-register interfaces, 391–392

U

UBR (unspecified bit rate) service class, 300
Unicast, 112, 520
rearrangeable Clos networks, 122–126
strictly non-blocking Clos networks, 118–121
Uniform random traffic, 50–51
Unstable networks, 453
Uphill-only resource allocation rule, 263–266
Upstream, 520

V

Valiant's randomized routing algorithm, **173–176**, 496–500
Validation, 467–468
Variable priority iterative arbiters, 354
VBR (variable bit rate) service class, 300
VBR-rt (variable bit rate, real-time) service class, 300
VCID (virtual-channel identifier), 224
Velio VC2002, 137–142
Vertex-symmetric networks, 49–50
Virtual-channel flow control, 222, **237–244**
active state, 237
idle state, 237
performance, 500–502
reallocating a channel, 313–316
waiting state, 237
Virtual circuits, 293
Virtual cut-through flow control. *See* Cut-through flow control
Virtual networks, 300–302
Virtual output queues, 380, 400
Virtual-channel allocator, 306–307
Virtual-channel routers, 316–317
credit loop, 312–313
control plane, 306–307
datapath, 305–306
flit rate, 307
flit stalls, 310–312
input unit, 306
packet rate, 307
packet stalls, 310–311
pipeline, 308–310
route computation, 306–309
stalls, 310–312
state fields, 307
switch allocation, 307–309
switch traversal, 309–310
virtual-channel allocation, 307–309

W

Wait for relation, 259, 260–262
Waiting state, 237
Warm-up period, 479–480
Wavefront allocators 373–376
Weak fairness, 351
Weighted random routing, 161–162
Weighted round-robin arbiters, 357–358
Wide-area networks, 2

Winner-take-all allocation, 242
Wire mat, 169
Wire-or bus, 432
Wires, 60–64
  critical length, 62–63
Work-conserving multiplexer, 288–290
Workloads, 475–478
  application-driven, 475–476
  execution-driven, 475
  synthetic, 476–478
  trace-driven, 475–476
Wormhole flow control, 222, **237–239**
Worst-case traffic patterns, 180–183
Write operation, 396

Z

Zero-load latency, 20–21, **56**, 455–456, 494

Topology

$$\Theta_{ideal} = \frac{b}{\gamma_{max}}$$

$$T_0 = \underbrace{H_{min} t_r}_{\text{router}} + \underbrace{\frac{L}{b}}_{\text{serialization}} + \underbrace{D_{min}}_{\text{wire}}$$

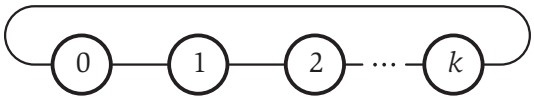
$$\gamma_{max} \geq \frac{NH_{min}}{C}$$

hop count bound

$$\gamma_{max} \geq \frac{N}{2B_C}$$

bisection bound

k-ary n-cube (Torus)



k-ary 1-cube

$$H_{min} = \begin{cases} \frac{nk}{4} & k \text{ even} \\ n\left(\frac{k}{4} - \frac{1}{4k}\right) & k \text{ odd} \end{cases}$$

$$\gamma_{max} = \begin{cases} \frac{k}{8} & k \text{ even} \\ \frac{k}{8} - \frac{1}{8k} & k \text{ odd} \end{cases}$$

uniform traffic

$$B_C = 4k^{n-1} = \frac{4N}{k} \qquad = 4n$$

Two-level packaging (k even)

$$w \leq \min\left(\frac{W_n}{4n}, \frac{kW_s}{4N}\right) \qquad b = wf$$

$$T_s = \frac{L}{b} \qquad \Theta_{ideal} = \frac{8b}{k}$$

k-ary n-mesh (Mesh)

k-ary 1-mesh

$$H_{min} = \begin{cases} \frac{nk}{4} & k \text{ even} \\ n\left(\frac{k}{3} - \frac{1}{3k}\right) & k \text{ odd} \end{cases}$$

$$\gamma_{max} = \begin{cases} \frac{k}{4} & k \text{ even} \\ \frac{k}{4} - \frac{1}{4k} & k \text{ odd} \end{cases}$$

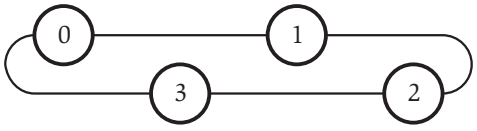
uniform traffic

$$B_C = 2k^{n-1} = \frac{2N}{k} \qquad = 4n$$

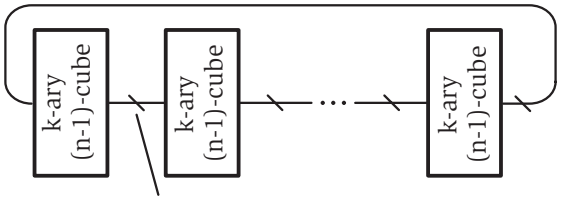
Two-level packaging (k even)

$$w \leq \min\left(\frac{W_n}{4n}, \frac{kW_s}{2N}\right) \qquad b = wf$$

$$T_s = \frac{L}{b} \qquad \Theta_{ideal} = \frac{4b}{k}$$



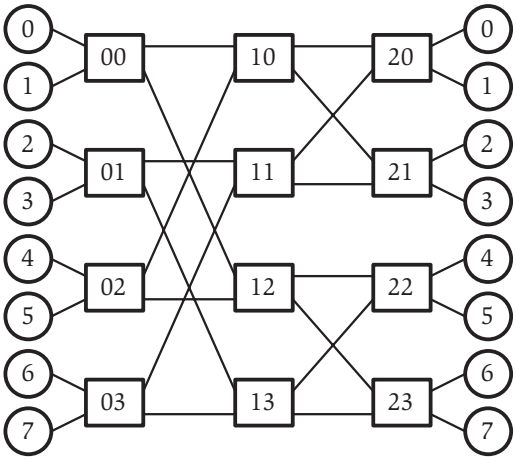
Folding to eliminate long channels from a torus layout.



k^{n-1} channels

Recursive construction of higher dimensional k-ary n-cubes.

k-ary n-fly (Butterfly)



2-ary 3-fly

$$\underbrace{\gamma_{\max} = 1}_{\text{uniform traffic}} \qquad \underbrace{\gamma_{\max,\text{wc}} = \sqrt{N}}_{\text{worst-case } (n \text{ even})}$$

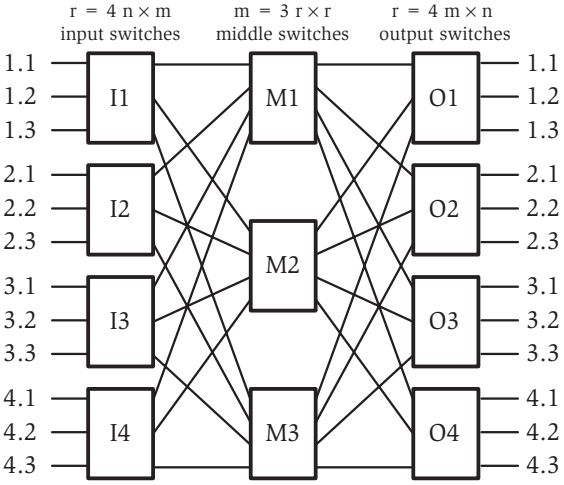
$$H_{\min} = n + 1 \qquad B_C = \frac{N}{2} \qquad = 2k$$

Two-level packaging:

$$w \le \min\left(\frac{W_n}{2k}, \frac{2W_s}{N}\right) \qquad b = wf$$

$$T_s = \frac{L}{b} \qquad \Theta_{\text{ideal}} = b$$

Clos



$(m = 3, n = 3, r = 4)$ Clos network

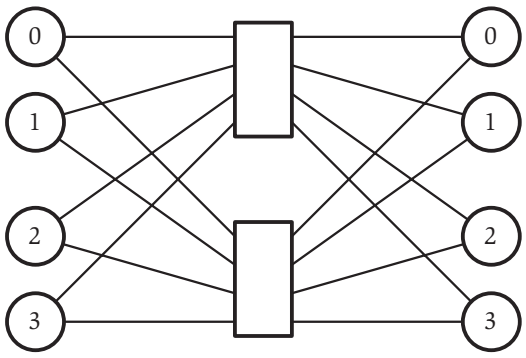
Strictly non-blocking if $m \quad 2n - 1$

Rearrangeably non-blocking if $\mathrm{m} \quad n$

Non-blocking for fanout f multicast if

$$f \le \frac{m(m-n)}{m(n-1)}$$

Channel Slicing



Channel sliced 4-ary 1-fly

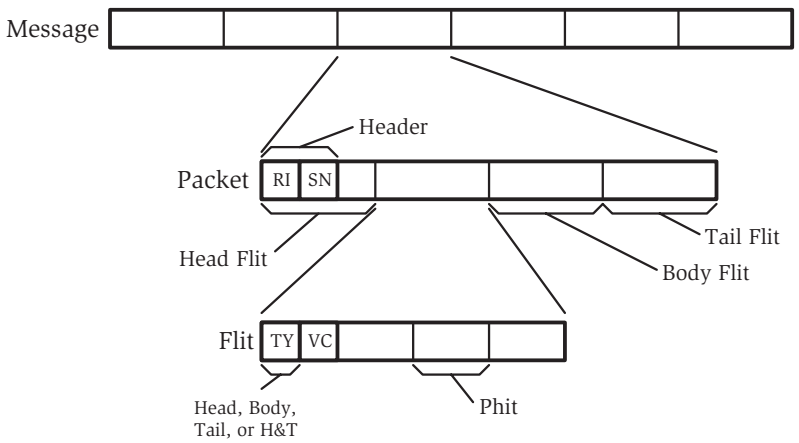
Channel slicing a butterfly with slicing factor x:

$$n = \frac{n}{1 + \log_k x}$$

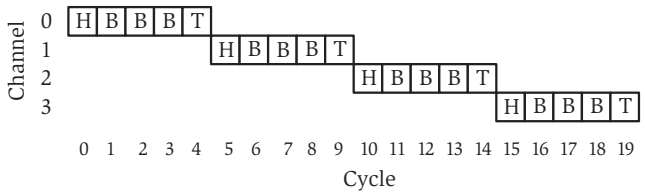
$$T_s = \frac{xL}{b}$$

$$T_h = t_r\left(\frac{n}{1 + \log_k x}\right)$$

Units of Resource Allocation

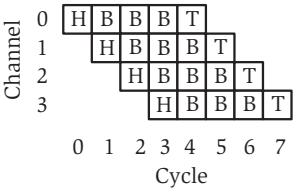


Store-and-forward Flow Control



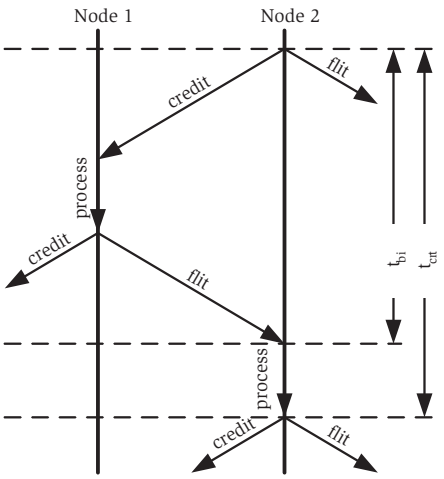
$$T_0 = H\left(t_r + \frac{L}{b}\right)$$

Cut-through / Wormhole FC



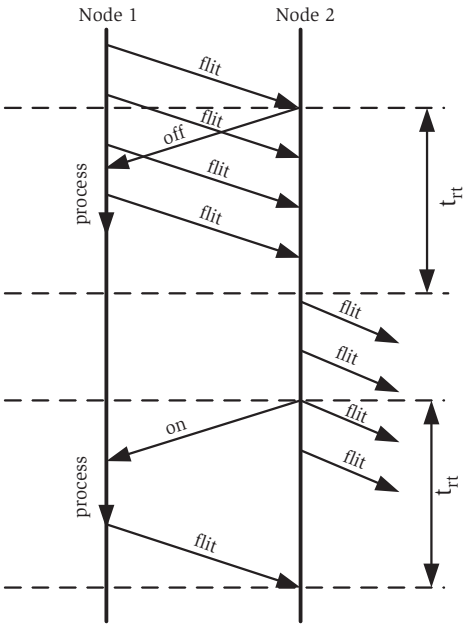
$$T_0 = Ht_r + \frac{L}{b}$$

Credit-based Flow Control



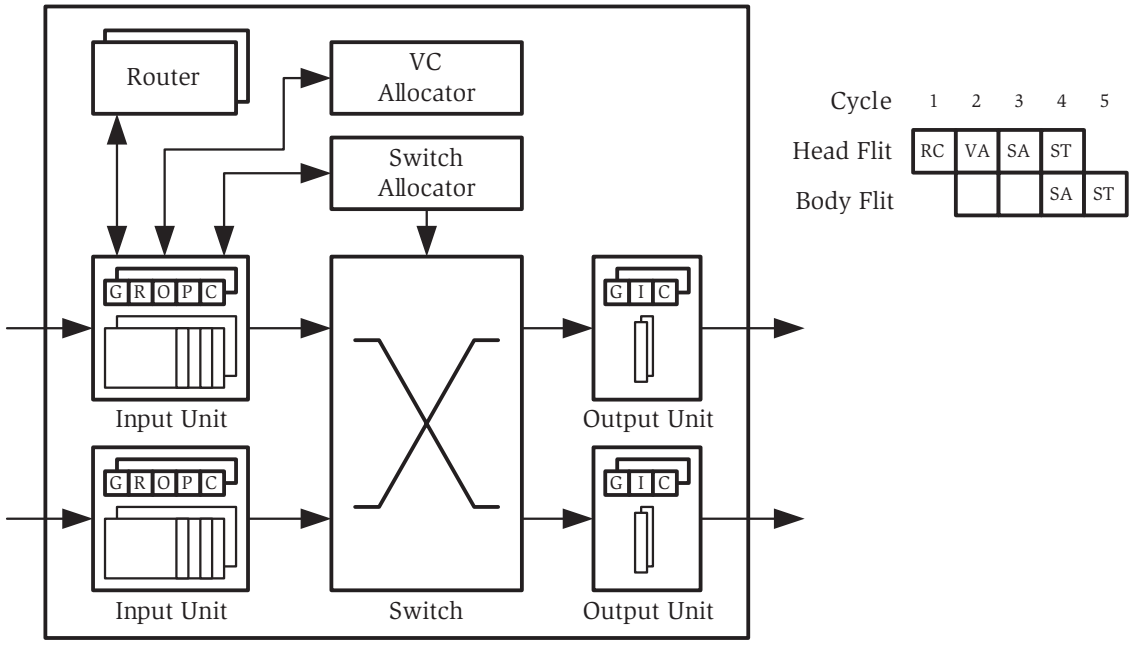
$$F \quad \frac{t_{\mathrm{crt}} b}{L_f}$$

On/Off Flow Control



$$F \quad \frac{2t_{\mathrm{rt}} b}{L_f}$$

An Input Queued, Virtual Channel Router



Latency Versus Offered Traffic